CMake для C++

Современный подход



Рафал Свидзиньски

Рафал Свидзиньски

CMake для C++

Rafał Świdziński

Modern CMake for C++

Second Edition

‹packt›
BIRMINGHAM—MUMBAI

Рафал Свидзиньски

CMake для C++

Современный подход

ДМК Пресс
издательство
Москва, 2025

УДК 004.438CMake+004.438C++
ББК 32.73.2я
С24

Свидзиньски Р.
С24 CMake для C++ / пер. с англ. А. А. Слинкина. – М.: ДМК Пресс, 2024. – 442 с.: ил.

ISBN 978-5-93700-326-3

Книга рассказывает о кросс-платформенной утилите CMake для автоматизации сборки программ из исходного кода. Вы научитесь использовать в своем проекте сторонние библиотеки, средства анализа программ и генераторы документации. Подробные пояснения и многочисленные практические примеры помогут вам писать более чистый и эффективный код на C++.

Издание предназначено разработчикам программного обеспечения, знакомым с программированием на C/C++ и желающим изучить CMake.

УДК 004.438CMake+004.438C++
ББК 32.73.2я

Copyright © Packt Publishing 2024. First published in the English language under the title Modern CMake for C++ – Second Edition – (9781805121800).

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN 978-1-80512-180-0 (англ.)
ISBN 978-5-93700-326-3 (рус.)

© 2024 Packt Publishing
© Перевод, оформление, издание, ДМК Пресс, 2024

Содержание

От издательства 15

Вступительное слово 16

Об авторе 17

О рецензентах 18

Предисловие 19

Глава 1. Первые шаги 23

Технические требования 24
Основные факты 24
- Что такое CMake? 25
- Как она работает? 26
  - Этап конфигурирования 27
  - Этап генерирования 27
  - Этап сборки 28

Установка CMake на различных платформах 30
- Docker 31
- Windows 32
- Linux 33
- macOS 34
- Сборка из исходного кода 34

Работа с командной строкой 35
- Командная строка CMake 35
  - Генерирование сборочной системы проекта 35
  - Сборка проекта 41
  - Установка проекта 44
  - Выполнение скрипта 45
  - Выполнение командной программы 45
  - Выполнение готовой конфигурации технологического процесса 46
  - Получение справки 46
- Командная строка CTest 47
- Командная строка CPack 47
- Графический интерфейс CMake 48

Командная строка CCMake 49
Навигация по файлам и каталогам проекта 49
Дерево исходного кода 50
Дерево сборки 50
Списковые файлы 51
Файл проекта 51
Файл кеша 52
Файл определения пакета 53
Сгенерированные файлы 53
Файлы в формате JSON и YAML 53
Файлы готовых конфигураций 54
Файловый API 54
Журнал конфигурирования 54
Игнорирование файлов в Git 55
Скрипты и модули 55
Скрипты 56
Служебные модули 56
Модули поиска 57
Резюме 57
Для дополнительного чтения 58

Глава 2. Язык CMake 59

Технические требования 60
Основы синтаксиса языка CMake 60
Комментарии 61
Вызов команд 62
Аргументы команд 63
Аргументы в квадратных скобках 64
Аргументы в кавычках 65
Аргументы без кавычек 66
Работа с переменными 67
Ссылки на переменные 68
Переменные окружения 69
Кешированные переменные 71
Как правильно использовать области видимости переменных в CMake 72
Списки 74
Управляющие конструкции в CMake 76
Условные блоки 76
Синтаксис условных команд 76
Циклы 80
Цикл while() 80
Цикл foreach() 80
Определения команд 82
Макросы 83

Функции 84
Процедурная парадигма в CMake 85
К вопросу о соглашениях об именовании 87
Наиболее употребительные команды 87
Команда message() 87
Команда include() 89
Команда include_guard() 90
Команда file() 90
Команда execute_process() 91
Резюме 91
Для дополнительного чтения 92

Глава 3. Использование CMake в популярных IDE 93

Знакомство с IDE 94
Выбор IDE 95
Выбирайте функционально насыщенную IDE 95
Выбирайте IDE, широко поддерживаемую в вашей организации 96
Не выбирайте IDE, исходя из целевой ОС и платформы 96
Выбирайте IDE, поддерживающую удаленную разработку 96
Установка комплекта инструментов 97
Использование примеров из этой книги совместно с IDE 98
Начало работы с CLion IDE 99
Почему она может вам понравиться 100
Первые шаги 101
Крутая штука: накачанный отладчик 103
Начало работы с Visual Studio Code 103
Почему она может вам понравиться 104
Первые шаги 105
Крутая штука: контейнеры разработчика 105
Начало работы с Visual Studio IDE 106
Почему она может вам понравиться 107
Первые шаги 107
Крутая штука: отладка в режиме горячей перезагрузки 108
Резюме 109
Для дополнительного чтения 110

Глава 4. Подготовка первого проекта на основе CMake 111

Технические требования 112
Основные директивы и команды 112
Задание минимальной версии CMake 113
Определение языков и метаданных 113
Разбиение проекта на части 115
Управление областью видимости с помощью каталогов 117

Когда использовать вложенные проекты ... 119
Внешние проекты должны оставаться внешними ... 119
Размышления о структуре проекта ... 120
Определение окружения ... 125
Определение операционной системы ... 125
Кросс-компиляция: что такое хост-система и целевая система? ... 126
Сокращенные переменные ... 126
Информация о хост-системе ... 127
Архитектура платформы 32- или 64-разрядная? ... 128
Каков порядок байтов в системе? ... 128
Конфигурирование комплекта инструментов ... 129
Задание стандарта C++ ... 129
Принудительная поддержка стандарта ... 130
Расширения производителя ... 130
Межпроцедурная оптимизация ... 131
Проверка возможностей, поддерживаемых компилятором ... 132
Компиляция тестового файла ... 132
Запрет внутренней сборки ... 134
Резюме ... 134
Для дополнительного чтения ... 135

Глава 5. Цели ... 136

Технические требования ... 137
Понятие цели ... 137
Определение исполняемых целей ... 138
Определение библиотечных целей ... 139
Специальные цели ... 139
Граф зависимостей ... 140
Визуализация зависимостей ... 142
Задание свойств целей ... 143
Что такое транзитивные требования использования? ... 144
Конфликты распространяемых свойств ... 147
Знакомство с псевдоцелями ... 148
Импортированные цели ... 149
Псевдонимы целей ... 149
Интерфейсные библиотеки ... 149
Объектные библиотеки ... 150
Цели сборки ... 151
Написание специальных команд ... 152
Использование специальной команды в качестве генератора ... 153
Использование специальной команды в качестве точки вклинивания ... 154
Резюме ... 155
Для дополнительного чтения ... 156

Глава 6. Генераторные выражения 157

Технические требования 158
Что такое генераторные выражения? 158
Основные синтаксические правила составления выражения общего вида ... 159
Вложенность 160
Условное расширение 161
Вычисление булева значения 162
Логические операторы 162
Сравнения 162
Запросы 163
Запросы и преобразования 163
Работа со строками, списками и путями 163
Параметризация конфигурирования и платформы сборки 165
Настройка комплекта инструментов 166
Запрос целеориентированной информации 168
Экранирование 170
Примеры 170
Конфигурации сборки 170
Системно-зависимые однострочные выражения 171
Интерфейсные библиотеки с зависящими от компилятора флагами 171
Вложенные генераторные выражения 172
Различие между условным выражением и вычислением оператора BOOL 173
Резюме 174
Для дополнительного чтения 175

Глава 7. Компиляция исходного кода на C++ с помощью CMake 176

Технические требования 177
Основы компиляции 177
Как устроена компиляция 178
Начальная конфигурация 180
Требование конкретных возможностей от компилятора 181
Управление исходными файлами для целей 182
Конфигурирование препроцессора 183
Задание путей к включаемым файлам 183
Определения препроцессора 184
Избегайте доступа к закрытым полям класса в автономных тестах 185
Использование фиксаций git для отслеживания откомпилированной версии 186
Конфигурирование заголовочных файлов 187
Конфигурирование оптимизатора 188
Общий уровень 190
Встраивание функций 191

Разворачивание циклов ... 192
Векторизация циклов ... 193
Управление процессом компиляции ... 194
Уменьшение времени компиляции ... 195
Предкомпиляция заголовочных файлов ... 195
Унифицированные сборки ... 197
Поиск ошибок ... 199
Конфигурирование ошибок и предупреждений ... 199
Отладка сборки ... 200
Предоставление информации отладчику ... 202
Резюме ... 204
Для дополнительного чтения ... 205

Глава 8. Компоновка исполняемых файлов и библиотек ... 206

Технические требования ... 207
Основы компоновки ... 207
Сборка библиотек разных типов ... 211
Статические библиотеки ... 212
Разделяемые библиотеки ... 212
Разделяемые модули ... 214
Позиционно-независимый код (PIC) ... 215
Решение проблем с помощью ODR ... 216
Разбираемся с динамически скомпонованными символами-дубликатами ... 218
Используйте пространства имен – не полагайтесь на компоновщик ... 220
Порядок компоновки и неразрешенные символы ... 220
Символы, на которые нет ссылок ... 223
Вынесение логики main() для тестирования ... 224
Резюме ... 227
Для дополнительного чтения ... 228

Глава 9. Управление зависимостями в CMake ... 229

Технические требования ... 230
Использование уже установленных зависимостей ... 230
Нахождение пакетов с помощью команды CMake find_package() ... 231
Написание модулей своими силами ... 236
Нахождение унаследованных пакетов с помощью FindPkgConfig ... 242
Использование зависимостей, отсутствующих в системе ... 244
FetchContent ... 245
Простой пример программы чтения YAML ... 246
Скачивание зависимостей ... 248
Обновление и исправление ... 251
Использование установленной зависимости, если это возможно ... 251
ExternalProject ... 253

Резюме 255
Для дополнительного чтения 255

Глава 10. Использование модулей C++20 256

Технические требования 257
Что такое модули C++20? 257
Написание проектов с поддержкой модулей C++20 260
Активация экспериментальной поддержки в CMake 3.26 и 3.27 260
Включение поддержки для CMake 3.28 и старше 262
Задание требований к компилятору 262
Объявление модуля C++ 263
Конфигурирование комплекта инструментов 264
Резюме 265
Для дополнительного чтения 266

Глава 11. Каркасы тестирования 267

Технические требования 268
Почему нужно задуматься об автоматизированных тестах? 268
Применение CTest для стандартизации тестирования в CMake 269
Режим сборки и тестирования 270
Режим тестирования 271
Опрос тестов 272
Фильтрация тестов 272
Тасование тестов 273
Обработка ошибок 274
Повтор тестов 274
Управление выводом 275
Разное 276
Создание элементарного автономного теста для CTest 277
Структурирование проектов для тестирования 281
Каркасы автономного тестирования 285
Catch2 286
GoogleTest 288
Работа с GTest 288
GMock 291
Генерирование отчетов о покрытии тестами 296
Применение LCOV для генерирования отчетов о покрытии 298
Как избежать SEGFAULT 301
Резюме 302
Для дополнительного чтения 303

Глава 12. Инструменты анализа программ 304

Технические требования 305

Принудительное форматирование 305
Использование статических проверщиков 309
clang-tidy 312
Cpplint 312
Cppcheck 313
include-what-you-use 313
Link What You Use 313
Динамический анализ с помощью Valgrind 314
Memcheck 315
Memcheck-Cover 318
Резюме 320
Для дополнительного чтения 321

Глава 13. Генерирование документации 322

Технические требования 323
Добавление Doxygen в проект 323
Генерирование документации, выглядящей современно 328
Обогащение вывода специальным HTML-кодом 330
Резюме 333
Для дополнительного чтения 334

Глава 14. Установка и упаковка 335

Технические требования 336
Экспорт без установки 336
Установка проектов в систему 339
Установка логических целей 340
Использование конечного каталога по умолчанию для разных платформ 342
Публичные заголовки 343
Низкоуровневая установка 345
Установка командами install(FILES) и install(PROGRAMS) 345
Работа с целыми каталогами 348
Вызов скриптов в процессе установки 351
Установка зависимостей времени выполнения 353
Создание повторно используемых пакетов 354
Проблемы перемещаемости целей 354
Установка файлов экспорта целей 356
Написание простых конфигурационных файлов 357
Создание продвинутых конфигурационных файлов 360
Генерирование файлов версий пакетов 363
Определение компонентов 365
Как использовать компоненты в find_package() 365
Как используются компоненты в команде install() 365

Управление символическими ссылками для версионированных разделяемых библиотек....367
Упаковка с помощью CPack....368
Резюме....370
Для дополнительного чтения....371

Глава 15. Создание профессионального проекта....372

Технические требования....373
Планирование работы....373
Структура проекта....377
Разделяемые и статические библиотеки....378
Файлы проекта....379
Сборка и управление зависимостями....380
Сборка библиотеки Calc....382
Сборка исполняемого файла консольного калькулятора Calc....384
Тестирование и анализ программы....388
Подготовка модуля Coverage....390
Подготовка модуля Memcheck....391
Тестовые сценарии....392
Добавление инструментов статического анализа....394
Установка и упаковка....396
Установка библиотеки....397
Установка исполняемого файла....398
Упаковка с помощью CPack....399
Документирование....399
Генерирование технической документации....400
Написание нетехнических документов для профессионального проекта....402
Резюме....404
Для дополнительного чтения....405

Глава 16. Создание готовых конфигураций CMake....406

Технические требования....407
Использование готовых конфигураций, определенных в проекте....407
Написание файла готовой конфигурации....408
Определение готовых конфигураций, зависящих от этапа....409
Общие характеристики всех готовых конфигураций....410
Уникальные имена полей....410
Факультативные поля....410
Ассоциация с готовыми конфигурациями этапа конфигурирования....411
Определение готовых конфигураций этапа конфигурирования....411
Определение готовых конфигураций этапа сборки....413
Определение готовых конфигураций этапа тестирования....414

Определение готовых конфигураций этапа упаковки 417
Добавление готовой конфигурации установки 418
Определение готовых конфигураций технологического процесса 419
Добавление условий и макросов 421
Резюме 423
Для дополнительного чтения 423

Приложение 424

Команда string() 424
Поиск и замена 425
Модификация 425
Сравнение 426
Хеширование 426
Генерирование 427
JSON 427
Команда list() 428
Чтение 428
Поиск 428
Модификация 429
Упорядочение 429
Команда file() 430
Чтение 430
Запись 430
Файловая система 431
Преобразование путей 431
Передача 432
Блокировка 432
Архивация 432
Команда math() 433

Предметный указатель 434

От издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг, мы будем очень благодарны, если вы сообщите о ней главному редактору по адресу dmkpress@gmail.com. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу электронной почты dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Вступительное слово

В постоянно развивающемся и видоизменяющемся ландшафте C++ уверенное владение программой CMake является неоценимым преимуществом для любого разработчика, стремящегося писать эффективный, удобный для сопровождения и масштабируемый код. Книга «CMake для C++» Рафала Свидзиньски станет маяком, который проведет как начинающих, так и опытных программистов сквозь бушующие волны CMake.

Эта книга – не просто руководство, это путешествие. Начинается она с основ, так чтобы даже человек, только приступающий к изучению программы CMake, мог разобраться в ее понятиях. Постепенно читатель знакомится с более продвинутыми методами и в конце концов осваивает весь потенциал CMake.

Характерной особенностью этой книги является прагматический подход. В текст вплетены реальные примеры и рекомендованные практики, благодаря чему читатель не только усваивает концепции, но и учится эффективно применять их в своих проектах.

Прочитав всю книгу, читатель не только будет глубоко понимать CMake, но и обретет новую уверенность в своих способностях справляться с многочисленными сложностями разработки на C++. Он будет вооружен знаниями и навыками, которые позволят писать более чистый и эффективный код, встав тем самым на путь, ведущий к рождению профессионала.

«CMake для C++» – не просто книга, это инструмент, который поднимает навыки разработки на C++ на новую высоту. Равно начинающему и эксперту эта книга поможет раскрыть полный потенциал CMake и сделать код более надежным, пригодным для сопровождения и масштабируемым.

Александр Кушнир,
главный программист, компания Biosense Webster

Об авторе

Рафал Свидзиньски работает инженером в Google, имеет за плечами 12-летний опыт разработки полного цикла. Возглавлял проекты для таких промышленных гигантов, как Cisco Meraki, Amazon и Ericsson, всей душой предан инновациям. Лондонец по духу, он всегда на переднем крае технического прогресса и занимается кучей личных проектов. Его недавнее увлечение применением ИИ в здравоохранении отражает стремление к прогрессу, несущему пользу людям. Рафал высоко ценит безупречное качество и мастерство и делится знаниями на канале YouTube и в своих книгах.

Зое – без тебя эта книга не была бы написана.

О рецензентах

Эрик Нулар получил степень магистра технических наук в Высшей государственной школе электротехники, электроники, информатики, гидравлики и телекоммуникаций и степень PhD по информатике в университете Версаль-Сен-Кантен-ан-Ивелин, Франция. За плечами Эрика 25 лет написания и компилирования исходного кода на разных языках. Является пользователем CMake с 2006 года, принимал активное участие в ее развитии. В настоящее время работает в компании Antidot, специализирующейся на разработке программного обеспечения для семантического поиска, ИИ и обеспечения доступности контента. Эрик входит в состав исследовательской группы, которая отвечает за включение новых технологий, в т. ч. ИИ и обработку естественного языка, во флагманский продукт компании, Fluid Topics.

В активе **Джованни Романо** 28 лет работы в ИТ – от разработки программного обеспечения до проектирования приложений и компонентов. В настоящее время работает старшим программистом в компании Leica Geosystem AG, где занимается проектированием SDK, микросервисов и серверных компонентов с низкой задержкой. Будучи амбассадором Nokia/Blackberry Qt, он верит в программное обеспечение с открытым исходным кодом и участвует в развитии этого каркаса. В сферу его интересов входят облачные приложения, Kubernetes, Docker и GitOps. Обожает писать на языке C и играть в теннис.

Предисловие

Создание высококачественного программного обеспечения – не простое дело. Разработчики, изучающие этот вопрос по материалам в сети, часто не могут понять, какая рекомендация самая актуальная, а каким методам пришли на смену более новые и совершенные. К тому же объяснения обычно хаотические, им недостает базы, контекста и структуры.

В книге «CMake для C++» предлагается простой и исчерпывающий подход к построению решений на C++. Вы не только научитесь использовать CMake в своих проектах, но и узнаете, как сделать их пригодными для сопровождения, элегантными и чистыми. Рассматривается автоматизация решения сложных задач, в т. ч. сборки, тестирования и упаковки.

Вы узнаете об организации каталогов с исходным кодом, о целях сборки и о создании пакетов. Вы научитесь компилировать и компоновать исполняемые файлы и библиотеки, поймете все детали этих процессов и будете знать, как оптимизировать каждый шаг. Кроме того, будет показано, как включать в проект внешние зависимости, например сторонние библиотеки, каркасы тестирования, инструменты анализа кода и генераторы документации. Наконец, вы научитесь экспортировать и устанавливать свое решение для внутреннего и внешнего использования.

Прочитав эту книгу, вы сможете уверенно использовать CMake на профессиональном уровне.

Для кого предназначена эта книга

Изучив C++, вы довольно быстро поймете, что одного лишь владения языком недостаточно для организации поставки программного продукта в соответствии с высокими стандартами качества. Книга, которую вы держите в руках, восполняет этот пробел: она адресована всем, кто стремится стать лучшим программистом или даже профессиональным инженером по сборке!

Читайте, если хотите изучить современную CMake с азов или подтянуть отставшие от времени навыки. Книга поможет вам создавать проекты на C++ высочайшего качества и отказаться от других сред сборки.

Краткое содержание книги

В главе 1 «Первые шаги» рассматривается установка CMake, работа с ее командным интерфейсом и основные строительные блоки проекта на основе CMake.

Глава 2 «Язык CMake» посвящена основным понятиям языка CMake, включая вызов команд, аргументы, переменные, управляющие структуры и комментарии.

В главе 3 «Использование CMake в популярных IDE» подчеркивается важность **интегрированных сред разработки** (**IDE**), даются рекомендации по выбору IDE и приводятся инструкции для Clion, Visual Studio Code и Visual Studio IDE.

В главе 4 «Подготовка первого проекта на основе CMake» вы узнаете, как сконфигурировать базовый проект CMake в файле верхнего уровня, как организовать дерево файлов и как подготовить комплект инструментов разработки.

В главе 5 «Цели» рассматривается понятие логических целей сборки, их свойства и типы, а также определение своих команд для проектов CMake.

В главе 6 «Генераторные выражения» объясняется назначение и синтаксис генераторных выражений, в т. ч. их использование в условных расширениях, запросах и преобразованиях.

Глава 7 «Компиляция исходного кода на C++ с помощью CMake» посвящена процессу компиляции, конфигурированию препроцессора и оптимизатора, а также способам уменьшения времени сборки и упрощения отладки.

В главе 8 «Компоновка исполняемых файлов и библиотек» вы узнаете о механизме компоновки, о различных типах библиотек, о правиле одного определения, о порядке компоновки и о том, как подготовить проект к тестированию.

В главе 9 «Управление зависимостями в CMake» рассматривается работа со сторонними библиотеками, добавление поддержки CMake в те из них, где она не была изначально предусмотрена, а также скачивание внешних зависимостей из интернета.

В главе 10 «Использование модулей C++20» описываются модули из стандарта C++20, их поддержка в CMake и задание соответствующей конфигурации комплекта инструментов.

Глава 11 «Каркасы тестирования» поможет вам понять важность автоматизированного тестирования, освоить встроенную в CMake поддержку тестирования и приступить к автономному тестированию с применением популярных каркасов.

В главе 12 «Инструменты анализа программ» показано, как автоматически форматировать исходный код и обнаруживать программные ошибки на этапах сборки и выполнения.

Глава 13 «Генерирование документации» посвящена использованию Doxygen для автоматизации создания документации по исходному коду и ее стилизации.

В главе 14 «Установка и упаковка» вы научитесь подготавливать проект к выпуску с установкой и без нее, создавать пакеты, допускающие повторное использование, и определять отдельные компоненты, включаемые в пакет.

В главе 15 «Создание профессионального проекта» вы примените все полученные ранее знания к разработке полного проекта профессионального уровня.

В главе 16 «Создание готовых конфигураций CMake» рассказано об инкапсуляции высокоуровневых настроек проекта в технологические процессы с помощью файлов готовых конфигураций CMake, что повышает эффективность подготовки проекта и управления им.

Приложение «Разные команды» может служить справочником по командам CMake для работы со строками, списками, файлами и математическими операциями.

Как извлечь максимум из этой книги

В книге предполагается базовое знакомство с C++ и Unix-подобными системами. Хотя знание Unix не является обязательным требованием, оно будет полезно для понимания приведенных в книге примеров.

Эта книга ориентирована на версию CMake 3.26, но большая часть описанных методов применима ко всем версиям, начиная с CMake 3.15 (возможности, добавленные позже, специально оговариваются). Материал некоторых глав доведен до уровня CMake 3.28.

Подготовка среды для выполнения примеров рассматривается в главах 1–3, но мы настоятельно рекомендуем использовать прилагаемый к книге Docker-образ, если вы знакомы с этим инструментом.

Графические выделения

В этой книге для выделения семантически значимой информации применяются различные стили.

`Код в тексте`: фрагменты кода, имена таблиц базы данных, папок и файлов, URL-адреса, данные, вводимые пользователем, и адреса в Twitter, например: «Смонтируйте скачанный файл образа `WebStorm-10*.dmg` в качестве дополнительного диска».

Отдельно стоящие фрагменты кода набраны так:

```
cmake_minimum_required(VERSION 3.26)
project(Hello)
add_executable(Hello hello.cpp)
```

Желая привлечь внимание к какой-то части кода, мы выделяем соответствующие строки или слова полужирным шрифтом:

```
cmake_minimum_required(VERSION 3.26)
project(Hello)
add_executable(Hello hello.cpp)
add_subdirectory(api)
```

Ввод и вывод команд напечатан следующим образом:

```
cmake --build <dir> --parallel [<number-of-jobs>]
cmake --build <dir> -j [<number-of-jobs>]
```

Новые термины, важные слова и слова на экране набраны **полужирным шрифтом**, например: «Выберите пункт **System info** на панели **Administration**».



Предупреждения и важные примечания оформлены так.



Советы и рекомендации выглядят так.

1

Первые шаги

В создании программ есть какое-то волшебство. Мы не только создаем работающий механизм, который начинает свою жизнь, но и очень часто являемся авторами идеи, лежащей в его основе.

Для воплощения своих идей мы нередко выполняем в цикле следующие операции: проектирование, кодирование, тестирование. Мы придумываем изменения, формулируем их на языке, понятном компилятору, и проверяем, все ли работает, как задумано. Для создания высококачественной программы из исходного кода мы должны снова и снова скрупулезно выполнять операции, чреватые ошибками: вызов нужных команд, проверку синтаксиса, компоновку двоичных файлов, прогон тестов, создание отчетов об ошибках и т. д.

Чтобы запомнить все шаги, нужно приложить немалые усилия. А нам бы хотелось вместо этого сосредоточиться на самом кодировании, а все остальное отдать на откуп автоматизированным средствам. В идеале процесс должен запускаться одной кнопкой, нажимаемой после внесения изменений в код. Он должен быть гибким, быстрым, расширяемым и работать одинаково в разных ОС и средах. Он должен быть поддержан несколькими **интегрированными средами разработки** (**IDE**). Более того, мы могли бы включить этот процесс в конвейеры **непрерывной интеграции** (Continuous Integration – **CI**), которые собирают и тестируют нашу программу при каждой попытке записать изменение в разделяемый репозиторий.

CMake удовлетворяет многие из этих потребностей, но для ее правильного конфигурирования и использования нужно немного поработать. Сама CMake не является источником сложности, сложность проистекает из предмета, с которым мы работаем. Но не беспокойтесь, мы методично пройдем все этапы обучения. И совсем скоро вы станете гуру по сборке проектов.

Я понимаю, что вы уже рветесь писать свои собственные проекты на основе CMake, именно этим мы и будем заниматься на протяжении большей части книги. Но поскольку вы будете создавать проекты прежде всего для пользователей (включая себя самого), важно сначала поставить себя на их место.

Вот с этого и начнем: как стать опытным пользователем *CMake*. Мы рассмотрим основополагающие вещи: что представляет собой этот инструмент,

каковы принципы его работы и как его установить. Затем мы перейдем к командной строке и режимам работы. И завершим обзором назначения различных файлов проекта, а заодно объясним, как можно использовать CMake, вообще не создавая проекта.

В этой главе будут рассмотрены следующие вопросы:

- основные факты;
- установка CMake на различных платформах;
- работа с командной строкой;
- навигация по файлам проекта;
- скрипты и модули.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch01.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Из этой главы вы узнаете, что **build tree** – это путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Для сборки программ на C++ понадобится также компилятор для вашей платформы. Если вы знакомы с Docker, то можете использовать образ, содержащий все необходимые инструменты, процедура описана в разделе «Установка CMake на различных платформах». Если вы предпочитаете устанавливать CMake вручную, то в том же разделе найдете инструкции по установке.

Основные факты

Компиляция исходного кода на C++ представляется довольно простым процессом. Начнем с классического примера программы «Hello World».

Показанный ниже код находится в файле `ch01/01-hello/hello.cpp`:

```
#include <iostream>
int main() {
  std::cout << "Hello World!" << std::endl;
  return 0;
}
```

Для создания исполняемого файла нам, конечно, понадобится компилятор C++. В состав CMake компилятор не входит, так что вам придется выбрать и установить его самостоятельно. Вот некоторые популярные варианты:

- компилятор Microsoft Visual C++;
- собрание компиляторов GNU;
- Clang/LLVM.

Большинство читателей знакомы с тем, что такое *компилятор*, поскольку при изучении C++ без него никак не обойтись, поэтому мы не станем описывать процедуру выбора и установки. В этой книге используется GNU GCC, потому что это хорошо зарекомендовавший себя компилятор с открытым исходным кодом, доступный на многих платформах.

В предположении, что компилятор уже установлен, запускается он одинаково в большинстве систем от разных производителей. При выполнении нужно указать имя файла в качестве аргумента:

```
$ g++ hello.cpp -o hello
```

В нашем коде нет ошибок, поэтому компилятор молча создаст исполняемый двоичный файл, понятный процессору на нашей машине. Его можно выполнить, вызвав по имени:

```
$ ./hello
Hello World!
```

Выполнить одну команду для сборки программы просто, но по мере разрастания проекта вы быстро придете к выводу, что хранить весь код в одном файле попросту невозможно. Согласно рекомендациям по созданию чистого кода, исходные файлы следует делать небольшими, а их набор тщательно организовывать. Компилировать каждый файл вручную – утомительное занятие, чреватое ошибками. Есть способ гораздо лучше.

Что такое CMake?

Допустим, что мы хотим автоматизировать сборку, написав скрипт, который обходит дерево проекта и компилирует все подряд. Чтобы избежать излишних компиляций, скрипт будет определять, был ли изменен исходный файл с момента последнего запуска. Еще нам нужен удобный способ управлять аргументами, передаваемыми компилятору для каждого файла, – хорошо бы, чтобы этот способ допускал конфигурирование. Кроме того, наш скрипт должен знать, как скомпоновать все откомпилированные файлы в один двоичный, а еще лучше – собирать целые решения, которые можно использовать повторно и включать в более крупные проекты в виде модулей.

Сборка программы – многосторонний процесс, в котором можно выделить следующие аспекты:

- компиляция исполняемых файлов и библиотек;

- управление зависимостями;
- тестирование;
- установка;
- упаковка;
- порождение документации;
- дополнительное тестирование.

Потребовалось бы очень много времени, чтобы прийти к истинно модульной и мощной утилите сборки программ на C++, отвечающей всем пожеланиям. Но такой умелец нашелся. Билл Хоффман из компании Kitware написал первые версии CMake больше 20 лет назад. Как вы уже догадались, программа оказалась очень успешной. Сегодня она располагает обширнейшей функциональностью и пользуется поддержкой сообщества. CMake активно развивается и стала промышленным стандартом для программистов на C и C++.

Задача автоматизации сборки кода гораздо старше CMake, поэтому не стоит удивляться тому, что вариантов ее решения много: GNU Make, Autotools, SCons, Ninja, Premake и другие. Так почему же CMake одержала победу?

У CMake есть несколько характеристик, которые, на мой субъективный взгляд, важны:

- она ставит на первое место поддержку современных компиляторов и комплектов инструментов;
- CMake по-настоящему кросс-платформенная – она поддерживает Windows, Linux, macOS и Cygwin;
- она генерирует файлы проектов для популярных IDE: Microsoft Visual Studio, Xcode, Eclipse CDT. Кроме того, она задает модель проекта для других, например CLion;
- CMake работает на нужном уровне абстракции, позволяя группировать файлы в повторно используемые цели и проекты;
- существуют тысячи проектов, собранных с помощью CMake, и вы легко можете включить их в свой проект;
- CMake считает тестирование, упаковку и установку неотъемлемыми частями процесса сборки;
- старые, вышедшие из употребления возможности объявляются нерекомендуемыми и исключаются, чтобы не обременять CMake лишним кодом.

CMake предлагает унифицированный подход. Неважно, собираете вы программу в IDE или прямо из командной строки; важно – что CMake берет на себя также все этапы, следующие за сборкой.

В вашем конвейере непрерывной интеграции и доставки (CI/CD) конфигурирование CMake и проектов сборки может оставаться одним и тем же, даже если все предшествующие среды различаются.

Как она работает?

Возможно, у вас сложилось впечатление, что CMake читает исходный код на одной стороне и порождает двоичные файлы на другой. Это правда, но не вся.

CMake ничего не может собрать самостоятельно – она полагается на другие инструменты, которые производят компиляцию, компоновку и решают иные задачи. Можете считать, что она играет роль прораба: знает, какие шаги нужно выполнить, какова конечная цель, как найти подходящих исполнителей и материалы.

Процесс включает три этапа:

- конфигурирование;
- генерирование;
- сборку.

Рассмотрим их поочередно.

Этап конфигурирования

На этом этапе читаются сведения о проекте, хранящемся в каталоге, который называется **деревом исходного кода**, и подготавливается выходной каталог, или **дерево сборки** для этапа генерирования.

Первым делом CMake проверяет, был ли проект сконфигурирован ранее, и читает кешированные конфигурационные переменные из файла `CMakeCache.txt`. При первом выполнении это еще не так, поэтому создается пустое дерево сборки и собираются сведения о среде, в которой CMake работает, например: архитектура, доступные компиляторы, установленные компоновщики и архиваторы. Кроме того, проверяется, можно ли правильно откомпилировать простую тестовую программу.

Затем разбирается и выполняется конфигурационный файл проекта `CMakeLists.txt` (да, конфигурация проектов CMake пишется на специальном языке). Этот файл составляет необходимый минимум любого проекта CMake (исходные файлы можно добавить позже). Он сообщает CMake о структуре проекта, его целях и зависимостях (библиотеках и других пакетах CMake).

В ходе этого процесса CMake сохраняет в дереве сборки собранную информацию, как то: сведения о системе, конфигурационные данные проекта, журналы и временные файлы; все это используется на следующем шаге. Конкретно файл `CMakeCache.txt` создается для хранения наименее изменчивой информации (например, путей к компиляторам и другим инструментам), что экономит время при последующем выполнении всей последовательности сборки.

Этап генерирования

После чтения конфигурации проекта CMake генерирует **сборочную систему** для того окружения, в котором работает. Сборочные системы – это просто специальные конфигурационные файлы для других инструментов сборки (например, Makefile для GNU Make или Ninja либо файлы проектов IDE для Visual Studio). На этом этапе CMake еще может применить последние штрихи к конфигурации сборки путем вычисления **генераторных выражений**.



Этап генерирования автоматически выполняется после этапа конфигурирования. Поэтому в этой книге и в других источниках оба этих этапа часто объединяют и говорят просто о «конфигурировании» или «генерировании» сборочной системы. Если вы хотите выполнить только этап конфигурирования, то можете воспользоваться утилитой `cmake-gui`.

Этап сборки

Для порождения окончательных артефактов проекта (например, исполняемых файлов и библиотек) CMake должна запустить подходящий инструмент сборки. Это можно сделать прямо в IDE или выполнив нужную команду CMake. А уже эти инструменты – компиляторы, компоновщики, средства статического и динамического анализа, каркасы тестирования, генераторы отчетов и прочие – выполняют цели сборки.

Красота такого решения – в его способности порождать сборочные системы по запросу для любой платформы с помощью одной и той же конфигурации (т. е. одних и тех же файлов проекта).

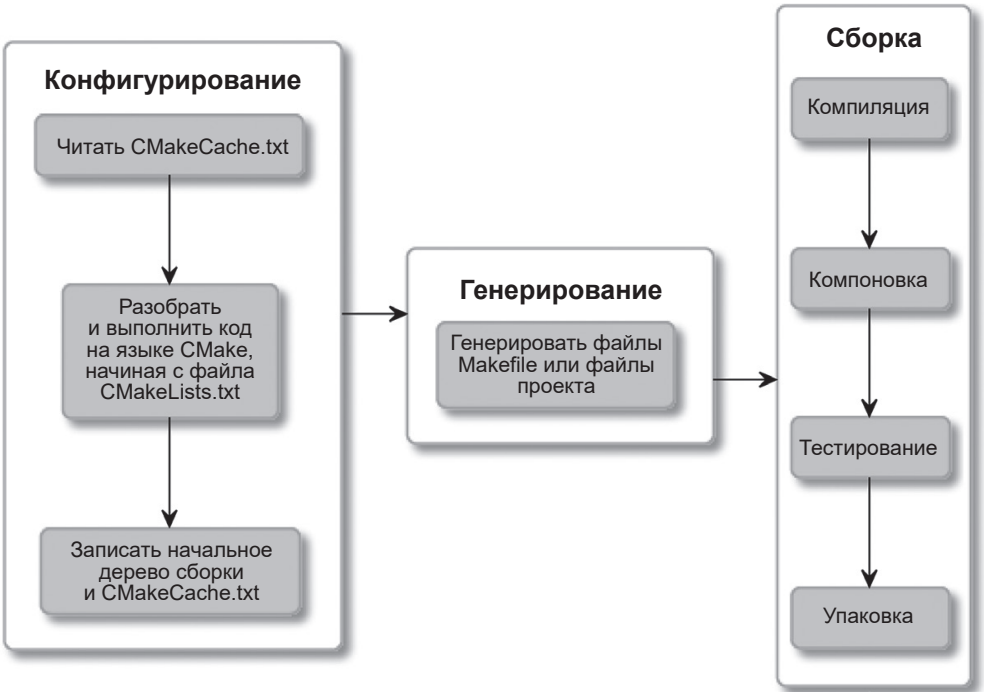


Рис. 1.1. Этапы CMake

Помните приложение `hello.cpp`? Собрать его с помощью CMake очень просто. Нужно лишь поместить такой файл `CMakeLists.txt` в один каталог с исходным файлом:

ch01/01-hello/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Hello)
add_executable(Hello hello.cpp)
```

После создания этого файла выполните следующие команды, оставаясь в том же каталоге:

```
cmake -B <build tree>
cmake --build <build tree>
```

Здесь `<build tree>` следует заменить путем к временному каталогу, в котором будут находиться сгенерированные файлы.

Ниже показан результат, полученный в системе Ubuntu, работающей в Docker-контейнере (Docker-контейнер – виртуальная машина, которая может работать внутри другой системы; мы обсудим этот вопрос в разделе «Установка CMake на различных платформах». Первая команда генерирует **сборочную систему**:

```
~/examples/ch01/01-hello# cmake -B ~/build_tree
-- The C compiler identification is GNU 11.3.0
-- The CXX compiler identification is GNU 11.3.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done (1.0s)
-- Generating done (0.1s)
-- Build files have been written to: /root/build_tree
```

Вторая команда **собирает** проект:

```
~/examples/ch01/01-hello# cmake --build ~/build_tree
Scanning dependencies of target Hello
[ 50%] Building CXX object CMakeFiles/Hello.dir/hello.cpp.o
[100%] Linking CXX executable Hello
[100%] Built target Hello
```

Осталось запустить откомпилированную программу:

```
~/examples/ch01/01-hello# ~/build_tree/Hello
Hello World!
```

Здесь мы сгенерировали сборочную систему, которая хранится в каталоге **дерева сборки**. После этого мы выполнили этап сборки и породили конечный двоичный файл, который смогли запустить.

Теперь, когда вы знаете, как выглядит результат, я уверен, что у вас на языке вертится куча вопросов. Каковы предварительные условия? Что означают эти команды? Зачем их две? Как написать свой файл проекта? Не торопитесь – на все вопросы будут даны ответы в следующих разделах.



В этой книге представлена наиболее важная информация, относящаяся к текущей версии CMake (на момент написания – 3.26). Чтобы не вводить вас в заблуждение, я сознательно избегал более не рекомендуемых возможностей, предполагая, что используется версия не младше 3.15, т. е. *современная CMake*. Если вам нужно больше информации, вы всегда можете найти ее последнюю, самую актуальную документацию по адресу https://cmake.org/cmake/help/.

Установка CMake на различных платформах

CMake – кросс-платформенная программа с открытым исходным кодом, написанная на C++. То есть вы можете откомпилировать ее самостоятельно, но вряд ли такое желание возникнет. Действительно, имеются уже готовые двоичные файлы, которые можно скачать на официальной веб-странице по адресу https://cmake.org/download/.

В Unix-подобных системах предлагаются пакеты, устанавливаемые прямо из командной строки.



Напомним, что в состав CMake компиляторы не входят. Если в вашей системе компилятор еще не установлен, то необходимо сделать это, прежде чем приступать к использованию CMake. Не забудьте прописать пути к его исполняемым файлам в переменной окружения PATH, чтобы CMake могла их найти.

Чтобы избежать проблем с поиском инструментов и зависимостей, я рекомендую для изучения применить первый метод установки: Docker. В реальной жизни вы, конечно, захотите использовать платформенные инструменты, если только не работаете в виртуализированной среде с самого начала.

Рассмотрим различные окружения, в которых может использоваться CMake.

Docker

Docker (https://www.docker.com/) – кросс-платформенное средство, предлагающее виртуализацию на уровне ОС, что позволяет поставлять приложения в виде точно определенных пакетов, называемых контейнерами. Это автономные единицы, содержащие некоторую программу со всеми необходимыми библиотеками, зависимостями и инструментами, необходимыми для ее работы. Docker исполняет свои контейнеры в облегченных окружениях, изолированных друг от друга.

Это очень удобно, когда нужно поставить целый комплект инструментов, необходимых в данном процессе, уже сконфигурированных и готовых к работе. Вы не поверите, как все упрощается, когда не нужно думать о мельчайших различиях между окружениями.

На платформе Docker имеется общедоступный репозиторий контейнеров, https://registry.hub.docker.com/, где выложены миллионы готовых к использованию образов.

Для вашего удобства я опубликовал два образа Docker:

- `swidzinski/cmake2:base`: образ на базе Ubuntu, содержащий сконфигурированные инструменты и зависимости, необходимые для сборки с помощью CMake;
- `swidzinski/cmake2:examples`: образ, основанный на том же комплекте инструментов и включающий все проекты и примеры из этой книги.

Первый вариант предназначен для читателей, которые просто хотят иметь чистую доску для сборки собственных проектов, а второй – для проработки примеров по ходу чтения книги.

Для установки Docker следуйте инструкциям в официальной документации (см. `docs.docker.com/get-docker`). Затем выполните в терминале следующие команды для скачивания образа и запуска контейнера:

```
$ docker pull swidzinski/cmake2:examples
$ docker run -it swidzinski/cmake2:examples
root@b55e271a85b2:root@b55e271a85b2:#
```

Заметим, что примеры находятся в каталогах с именами вида

```
devuser/examples/examples/ch<N>/<M>-<title>
```

Здесь `<N>` и `<M>` – дополненные нулями номера главы и примера соответственно (`01`, `08`, `12` и т. д.).

Windows

Установка в Windows не вызывает затруднений – просто скачайте 32- или 64-разрядную версию с официального сайта. Можете также выбрать переносимый ZIP-архив или пакет в формате MSI для Windows Installer, который добавит каталог CMake `bin` в переменную окружения `PATH` (рис. 1.2), так что вы сможете запускать CMake из любого каталога, не опасаясь такой ошибки:

```
cmake is not recognized as an internal or external command, operable program,
or batch file.
```

Выбрав ZIP-пакет, вы должны будете сделать это вручную. Установщик MSI предлагает удобный графический интерфейс пользователя:

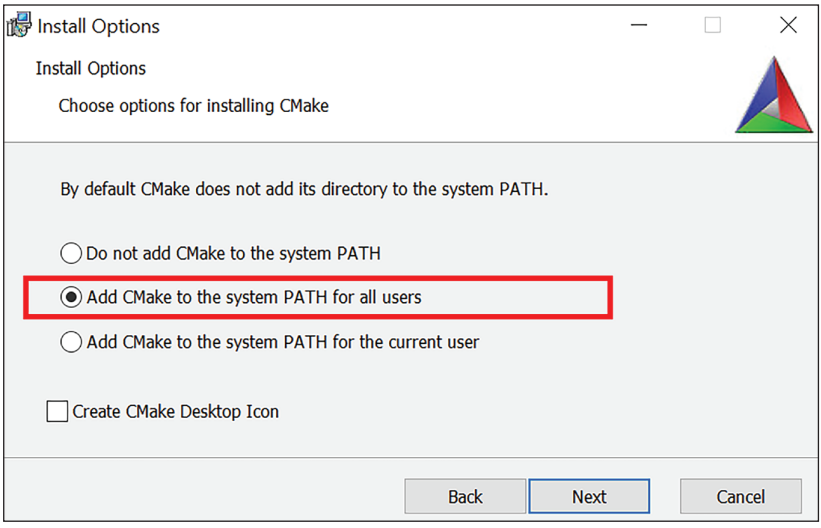


Рис. 1.2. Мастер установки может настроить переменную окружения PATH

Как уже было упомянуто, CMake – программа с открытым исходным кодом, поэтому вы можете собрать ее самостоятельно. Но в Windows вы должны будете сначала установить в систему двоичный экземпляр CMake. Такому сценарию следуют соразработчики CMake при создании новых версий.

Платформа Windows не отличается от других в том смысле, что для завершения процесса сборки, начатого CMake, необходим инструмент сборки. В данном случае наиболее популярна Visual Studio IDE, в состав которой входит компилятор C++. Некоммерческое издание доступно бесплатно с сайта Microsoft по адресу https://visualstudio.microsoft.com/downloads/.

Linux

В Linux CMake устанавливается как любой другой популярный пакет: вызовите менеджер пакетов из командной строки. В репозиториях пакетов обычно находится сравнительно недавняя версия CMake, но, как правило, не последняя. Если вас это устраивает и вы работаете с дистрибутивом Debian или Ubuntu, то проще всего установить нужный пакет:

```
$ sudo apt-get install cmake
```

Для дистрибутива Red Hat воспользуйтесь следующей командой:

```
$ yum install cmake
```



Заметим, что менеджер пакетов выбирает из репозитория последнюю доступную для вашей ОС версию. Часто в репозиториях хранится не самая актуальная версия, зато стабильная, проверенная временем. Сообразуйтесь со своими потребностями, но помните, что в старых версиях доступны не все возможности, описанные в книге.

Для получения самой свежей версии зайдите в раздел скачивания на официальном сайте CMake. Если вы знаете номер текущей версии, можете воспользоваться одной из следующих команд.

Для Linux x86_64:

```
$ VER=3.26.0 && wget https://github.com/Kitware/CMake/releases/download/
v$VER/cmake-$VER-linux-x86_64.sh && chmod +x cmake-$VER-linux-x86_64.sh &&
./cmake-$VER-linux-x86_64.sh
```

Для Linux AArch64:

```
$ VER=3.26.0 && wget https://github.com/Kitware/CMake/releases/download/
v$VER/cmake-$VER-Linux-aarch64.sh && chmod +x cmake-$VER-Linux-aarch64.sh
&& ./cmake-$VER-Linux-aarch64.sh
```

Или загляните в раздел «Сборка из исходного кода», где описано, как откомпилировать CMake самостоятельно.

macOS

Эта платформа также хорошо поддержана разработчиками CMake. Самый популярный вариант установки – MacPorts:

```
$ sudo port install cmake
```

На момент написания книги последней версией, доступной в MacPorts, была 3.24.4. Чтобы получить самую свежую версию, установите пакет `cmake-devel`:

```
$ sudo port install cmake-devel
```

Можете вместо этого воспользоваться менеджером пакетов Homebrew:

```
$ brew install cmake
```

Менеджеры пакетов для macOS выполняют все необходимые шаги, но помните, что можете получить не последнюю версию, если собираете не из исходного кода.

Сборка из исходного кода

Если вы пользуетесь другой платформой или просто хотите поработать с последними сборками, которые еще не достигли стадии выпускных (или не успели попасть в ваш любимый репозиторий пакетов), то скачайте исходный код с официального сайта и откомпилируйте сами:

```
$ wget https://github.com/Kitware/CMake/releases/
download/v3.26.0/cmake-3.26.0.tar.gz
$ tar xzf cmake-3.26.0.tar.gz
$ cd cmake-3.26.0
$ ./bootstrap
$ make
$ make install
```

Сборка из исходного кода сравнительно медленная и требует больше шагов. Но нет другого способа выбрать именно ту версию CMake, которая вам нужна. Особенно это полезно, когда пакеты в репозиториях вашей операционной системы публикуются с большой задержкой: чем старше версия ОС, тем меньше обновлений она получает.

Итак, вы установили CMake, давайте с ней поработаем!

Работа с командной строкой

Большая часть этой книги посвящена подготовке проектов CMake для пользователей. Чтобы учесть их потребности, мы должны ясно понимать, как пользователи взаимодействуют с CMake при различных сценариях. Это позволит тестировать файлы проектов и убеждаться в их правильности.

CMake представляет собой семейство инструментов и состоит из пяти исполняемых файлов:

- `cmake`: главная программа, которая конфигурирует, генерирует и собирает проекты;
- `ctest`: программа, управляющая прогоном тестов и формированием отчетов о результатах;
- `cpack`: упаковщик, используемый для генерирования установщиков и пакетов исходного кода;
- `cmake-gui`: графический интерфейс к `cmake`;
- `ccmake`: консольный оконный интерфейс к `cmake`.

Кроме того, компания Kitware, стоящая за CMake, предлагает отдельный инструмент CDash для более пристального надзора за правильностью сборки проектов.

Командная строка CMake

Программа `cmake` – главный двоичный исполняемый файл в комплекте CMake, она предоставляет несколько режимов работы (называемых также действиями):

- генерирование сборочной системы проекта;
- сборка проекта;
- установка проекта;
- выполнение скрипта;
- выполнение командной программы;
- выполнение готовой конфигурации технологического процесса;
- получение справки.

Рассмотрим работу в каждом режиме.

Генерирование сборочной системы проекта

Первый шаг сборки проекта – генерирование сборочной системы. Есть три варианта этой команды:

```
cmake [<options>] -S <source tree> -B <build tree>
cmake [<options>] <source tree>
cmake [<options>] <build tree>
```

Параметры <options> мы обсудим в последующих разделах. А прямо сейчас поговорим о выборе подходящего варианта команды. Важной особенностью CMake является поддержка *вынесенных сборок* (out-of-source build), т. е. хранение *артефактов сборки* в каталоге, отличном от дерева исходного кода. Это предпочтительный подход, поскольку он позволяет не засорять каталог с исходным кодом файлами, относящимися к сборке, и не загрязнять **систему управления версиями** (**СУВ**) случайными файлами и лишними директивами игнорирования.

Поэтому первая форма команды наиболее практична. Она позволяет задать пути к дереву исходного кода и порожденной сборочной системе с помощью флагов -S и -B соответственно:

```
cmake -S ./project -B ./build
```

CMake прочтет файлы проекта в каталоге ./project и сгенерирует сборочную систему в каталоге ./build (предварительно создав его при необходимости).

Мы можем пропустить один из аргументов, тогда cmake «догадается», что мы собирались использовать для соответствующей цели текущий каталог. Заметим, что если опущены оба аргумента, то сборка будет *внутренней* (in-source build), т. е. *артефакты сборки* будут храниться вместе с исходными файлами, чего мы как раз не хотим.



ПРИ РАБОТЕ С CMAKE ЗАДАВАЙТЕ КАТАЛОГИ ЯВНО

Не пользуйтесь вторым и третьим вариантами команды cmake <directory>, потому что при этом может получиться неразбериха. В главе 4 мы узнаем, как не дать пользователям совершить такую ошибку.

Как следует из приведенного выше фрагмента, одна и та же команда ведет себя по-разному, если в каталоге <directory> уже имеется сборка; она будет использовать кешированный путь к исходным файлам и повторит сборку именно там. Поскольку мы часто повторяем команды, пользуясь историей в программе терминала, это может привести к недоразумениям; прежде чем воспользоваться этим вариантом, обязательно проверяйте, является ли текущим нужный вам каталог.

Примеры

Сгенерировать дерево сборки в текущем каталоге, когда исходный код находится на один уровень выше:

```
cmake -S ..
```

Сгенерировать дерево сборки в каталоге ./build, когда исходный код находится в текущем каталоге:

```
cmake -B build
```

Выбор генератора

Как уже было сказано, на этапе генерирования можно задать несколько параметров. От выбора и конфигурации генератора зависит, какой инструмент сборки из имеющихся в системе будет применяться на следующем этапе сборки проекта, как будут выглядеть файлы сборки и какой будет структура дерева сборки.

Так ли это важно? По счастью, нет. CMake действительно поддерживает несколько сборочных систем на многих платформах, но если на вашей машине не установлено сразу несколько генераторов, то CMake сделает все правильно. Ее поведение можно переопределить, задав переменную окружения `CMAKE_GENERATOR` или указав генератор прямо в командной строке:

```
cmake -G <generator name> -S <source tree> -B <build tree>
```

Некоторые генераторы (например, Visual Studio) поддерживают более детальную спецификацию набора инструментов (компилятора) и платформы (компилятора или SDK). Дополнительно CMake просматривает переменные окружения, переопределяющие умолчания: `CMAKE_GENERATOR_TOOLSET` и `CMAKE_GENERATOR_PLATFORM`. Или же эти параметры можно задать прямо в командной строке:

```
cmake -G <generator name>
      -T <toolset spec>
      -A <platform name>
      -S <source tree> -B <build tree>
```

Пользователи Windows обычно хотят генерировать сборочную систему для своей любимой IDE. В Linux и macOS очень часто используют генераторы **Unix Makefiles** или **Ninja**.

Чтобы проверить, какие генераторы имеются в вашей системе, воспользуйтесь следующей командой:

```
cmake --help
```

В конце справки будет выведен полный список генераторов. Например, на машине с Windows 10 было напечатано (для удобства список сокращен):

```
Visual Studio 17 2022
Visual Studio 16 2019
Visual Studio 15 2017 [arch]
Visual Studio 14 2015 [arch]
Visual Studio 12 2013 [arch]
Visual Studio 11 2012 [arch]
Visual Studio 9 2008 [arch]
Borland Makefiles
NMake Makefiles
NMake Makefiles JOM
```

```
MSYS Makefiles
MinGW Makefiles
Green Hills MULTI
Unix Makefiles
Ninja
Ninja Multi-Config
Watcom WMake
CodeBlocks - MinGW Makefiles
CodeBlocks - NMake Makefiles
CodeBlocks - NMake Makefiles JOM
CodeBlocks - Ninja
CodeBlocks - Unix Makefiles
CodeLite - MinGW Makefiles
CodeLite - NMake Makefiles
CodeLite - Ninja
CodeLite - Unix Makefiles
Eclipse CDT4 - NMake Makefiles
Eclipse CDT4 - MinGW Makefiles
Eclipse CDT4 - Ninja
Eclipse CDT4 - Unix Makefiles
Kate - MinGW Makefiles
Kate - NMake Makefiles
Kate - Ninja
Kate - Unix Makefiles
Sublime Text 2 - MinGW Makefiles
Sublime Text 2 - NMake Makefiles
Sublime Text 2 - Ninja
Sublime Text 2 - Unix Makefiles
```

Как видите, CMake поддерживает кучу разных генераторов и IDE.

Управление кешем проекта

CMake запрашивает у системы различную информацию на этапе конфигурирования. Поскольку все это занимает заметное время, собранная информация кешируется в файле `CMakeCache.txt` в каталоге дерева сборки. Есть несколько параметров для управления поведением кеша.

Первый позволяет *предварительно заполнить кеш информацией*:

```
cmake -C <initial cache script> -S <source tree> -B <build tree>
```

Мы можем задать путь к файлу, содержащему только список команд `set()`, описывающих переменные, которые будут использованы для инициализации пустого дерева сборки. Как писать такие списковые файлы, мы обсудим в следующей главе.

Инициализировать и модифицировать уже имеющиеся в кеше переменные можно и по-другому (например, когда не хочется создавать файл ради задания всего лишь парочки переменных). Их можно задать прямо в командной строке:

```
cmake -D <var>[:<type>]=<value> -S <source tree> -B <build tree>
```

Часть `:<type>` необязательна (она используется графическими интерфейсами) и принимает следующие типы: `BOOL`, `FILEPATH`, `PATH`, `STRING` или `INTERNAL`. Если тип опущен, то CMake проверяет, существует ли переменная в файле `CMakeCache.txt`, и использует ее тип; в противном случае будет установлен тип `UNINITIALIZED`.

Особенно важна и часто задается в командной строке переменная `CMAKE_BUILD_TYPE`, определяющая **тип сборки**. В большинстве проектов CMake она используется в разных местах для определения таких вещей, как степень детальности диагностических сообщений, включение отладочной информации и уровня оптимизации для создаваемых артефактов.

Для генераторов с одной конфигурацией (например, GNU Make и Ninja) необходимо задавать **тип сборки** на этапе конфигурирования и генерировать отдельное дерево сборки для каждого типа конфигурации. Возможные значения: `Debug`, `Release`, `MinSizeRel` и `RelWithDebInfo`. Если тип сборки не задан, то поведение проектов, которые зависят от него, может оказаться неопределенным. Например:

```
cmake -S . -B ../build -D CMAKE_BUILD_TYPE=Release
```

Заметим, что генераторы, поддерживающие несколько конфигураций, конфигурируются на этапе сборки.

Для целей диагностики мы также можем создать список кешированных переменных, указав флаг `-L`:

```
cmake -L -S <source tree> -B <build tree>
```

Иногда авторы проектов могут задавать полезные справочные сообщения к переменным; чтобы напечатать их, добавьте модификатор `H`:

```
cmake -LH -S <source tree> -B <build tree>
cmake -LAH -S <source tree> -B <build tree>
```

Как ни странно, пользовательские переменные, добавленные вручную с помощью флага `-D`, не видны в этой распечатке, если только не указан один из поддерживаемых типов.

Чтобы *удалить* одну или несколько переменных, поступите так:

```
cmake -U <globbing_expr> -S <source tree> -B <build tree>
```

Здесь маска `<globbing_expr>` может содержать метасимволы `*` (любая строка) и `?` (любой символ). Но будьте внимательны, потому что так легко стереть больше переменных, чем вы хотели.

Флаги `-U` и `-D` могут повторяться несколько раз.

Отладка и трассировка

У команды cmake есть много параметров, позволяющих заглянуть под капот. Для получения общей информации о переменных, командах, макросах и других объектах выполните команду

```
cmake --system-information [file]
```

Необязательный аргумент file позволяет сохранить результат в файле. Если запустить эту команду в каталоге **дерева сборки**, то она напечатает дополнительную информацию о кешированных переменных и сообщениях из файлов журналов.

В наших проектах мы будем использовать команды message() для получения подробных отчетов о процессе сборки. CMake фильтрует сообщения, записанные в журнал, на основе текущего уровня протоколирования (по умолчанию STATUS). В следующей команде задан интересующий нас уровень протоколирования:

```
cmake --log-level=<level>
```

Здесь level может принимать значения ERROR, WARNING, NOTICE, STATUS, VERBOSE, DEBUG или TRACE. Этот параметр можно задать на постоянной основе в кешированной переменной CMAKE_MESSAGE_LOG_LEVEL.

Еще один интересный параметр позволяет *отображать контекст журнала* при каждом обращении к message(). Для отладки очень сложных проектов переменную CMAKE_MESSAGE_CONTEXT можно использовать как стек. Всякий раз, как код входит в интересующий вас контекст, вы можете присвоить ему понятное имя. Тогда сообщения будут дополнены текущим значением переменной CMAKE_MESSAGE_CONTEXT, например:

```
[some.context.example] Debug message.
```

За такой способ вывода в журнал отвечает параметр

```
cmake --log-context <source tree>
```

Мы подробнее обсудим команды именования контекстов и протоколирования в главе 2.

Если больше ничего не помогает и возникает необходимость в использовании тяжелых орудий, то имеется *режим трассировки*, в которой печатается каждая выполненная команда, а вместе с ней имя файла и номер строки, из которой она вызвана, а также список переданных аргументов. Активируется этот режим командой

```
cmake --trace
```

Понятно, что каждый раз так делать не стоит, потому что вывод будет очень длинным.

Создание готовых конфигураций

Существует очень много параметров, которые пользователи могут задавать для генерирования **дерева сборки** по вашему проекту. Поэтому легко что-то перепутать или пропустить. Разработчик проекта может упростить жизнь пользователям, предоставив файл `CMakePresets.json`, в котором заданы значения некоторых параметров по умолчанию.

Чтобы вывести список всех готовых технологических процессов, выполните команду

```
cmake --list-presets
```

Воспользоваться одной из имеющихся готовых конфигураций позволяет команда

```
cmake --preset=<preset> -S <source> -B <build tree>
```

За дополнительными сведениями обращайтесь к разделу «Навигация по файлам проекта» этой главы и к главе 16.

Очистка дерева сборки

Время от времени возникает необходимость стереть сгенерированные файлы, например из-за внесенных между сборками изменений в окружение или просто для того, чтобы начать работу с чистого листа. Каталог дерева сборки можно удалить вручную или включив в командную строку параметр `--fresh`:

```
cmake --fresh -S <source tree> -B <build tree>
```

Тогда CMake сотрет файл `CMakeCache.txt` и каталог `CMakeFiles/` способом, учитывающим особенности системы, и сгенерирует сборочную систему с нуля.

Сборка проекта

После того как дерево сборки сгенерировано, мы готовы к *сборке проекта*. CMake не только знает, как генерировать входные файлы для различных сборщиков, но и может запустить их с нужными аргументами.



НЕ ВЫЗЫВАЙТЕ MAKE НАПРЯМУЮ

Во многих онлайновых источниках рекомендуется запускать GNU Make напрямую после этапа генерирования. Поскольку GNU Make – генератор по умолчанию для Linux и macOS, эта рекомендация может сработать. Однако лучше пользоваться методом, описанным в этом разделе, потому что он не зависит от генератора и официально поддерживается на всех платформах. В результате вам не нужно думать обо всех возможных окружениях пользователей вашего приложения.

Порядок вызова команды режима сборки следующий:

```
cmake --build <build tree> [<options>] [-- <build-tool-options>]
```

В большинстве случаев для успешной сборки достаточно ограничиться минимумом параметров:

```
cmake --build <build tree>
```

Единственный обязательный аргумент – путь к сгенерированному дереву сборки. Это тот самый путь, который был передан в аргументе `-B` на этапе генерирования.

CMake позволяет задать основные параметры сборки, которые понимает любой сборщик. Если нужно задать специальные аргументы, понятные только выбранному вами сборщику, то укажите их в конце команды после двух символов `--`:

```
cmake --build <build tree> -- <build tool options>
```

Посмотрим, какие еще имеются параметры.

Запуск параллельной сборки

По умолчанию многие инструменты сборки используют несколько конкурентных процессов, чтобы задействовать возможности современных процессоров и компилировать несколько исходных файлов параллельно. Сборщики знают о структуре зависимостей в проекте, поэтому могут одновременно обрабатывать шаги, не зависящие друг от друга, чтобы сэкономить пользователю время.

Вы можете задать этот параметр, если хотите произвести сборку быстрее на многоядерной машине или принудительно установить режим однопоточной сборки для отладки.

Просто укажите количество задач с помощью любого из следующих параметров:

```
cmake --build <build tree> --parallel [<number of jobs>]
cmake --build <build tree> -j [<number of jobs>]
```

Альтернатива – задать то же самое в переменной окружения `CMAKE_BUILD_PARALLEL_LEVEL`. Если параметр задан в обоих местах, то командная строка имеет преимущество.

Выбор целей для сборки и очистки

Каждый проект состоит из одной или нескольких частей, называемых **целями** (мы будем обсуждать их во второй части книги). Обычно мы хотим собрать все имеющиеся цели, но иногда желательно некоторые пропустить или явно собрать цель, сознательно исключенную из обычных сборок. Это можно сделать следующим образом:

```
cmake --build <build tree> --target <target1> --target <target2> …
```

Чтобы задать несколько целей, перечислите их все, предпосылая каждой аргумент `-target`. Есть также сокращенная форма: `-t <target>`.

Очистка дерева сборки

Специальная цель `clean` обычно не собирается. Ее сборка означает удаление всех артефактов из каталога сборки, чтобы впоследствии можно было начать с чистого листа. Этот процесс запускается командой

```
cmake --build <build tree> -t clean
```

Кроме того, CMake предлагает удобный псевдоним на случай, если вы захотите сначала очистить, а затем произвести нормальную сборку:

```
cmake --build <build tree> --clean-first
```

Это действие отличается от очистки, упомянутой в разделе «Очистка дерева сборки», поскольку распространяется только на артефакты целей и ни на что более (в частности, кеш остается на месте).

Конфигурирование типа сборки для генераторов, поддерживающих несколько конфигураций

Итак, мы уже немного знаем о генераторах: они бывают разные. Некоторые умеют производить сборки типа `Debug` и `Release` в одном дереве. К генераторам, поддерживающим такую возможность, относятся Ninja Multi-Config, Xcode и Visual Studio. Все остальные генераторы поддерживают только одну конфигурацию и для каждой конфигурации нуждаются в отдельном дереве сборки.

Выберите тип сборки `Debug`, `Release`, `MinSizeRel` или `RelWithDebInfo` и задайте его следующим образом:

```
cmake --build <build tree> --config <cfg>
```

В противном случае CMake будет использовать по умолчанию тип `Debug`.

Отладка процесса сборки

Когда что-то пошло не так, нужно первым делом посмотреть на выходные сообщения. Но ветераны знают, что выводить все сообщения каждый раз – только запутывать себя, поэтому по умолчанию часто скрывают сообщения. А если все-таки нужно заглянуть под капот, то можно запросить у CMake более подробный журнал:

```
cmake --build <build tree> --verbose
cmake --build <build tree> -v
```

Того же эффекта можно достичь, задав кешированную переменную `CMAKE_VERBOSE_MAKEFILE`.

Установка проекта

После того как артефакты собраны, пользователь может установить их в систему. Обычно это означает, что файлы нужно скопировать в правильные каталоги, установить библиотеки или выполнить какую-то специальную логику из скрипта CMake.

Команда режима установки вызывается следующим образом:

```
cmake --install <build tree> [<options>]
```

Как и для прочих режимов работы, необходимо задать путь к сгенерированному дереву сборки:

```
cmake --install <build tree>
```

В режиме установки также можно задавать много дополнительных параметров. Посмотрим, на что они способны.

Выбор каталога установки

Мы можем добавить в начало пути установки произвольный префикс (например, если доступ к некоторым каталогам ограничен). Если к `/usr/local` добавить префикс `/home/user`, получится `/home/user/usr/local`.

Порядок вызова этой команды такой:

```
cmake --install <build tree> --install-prefix <prefix>
```

При работе с версией CMake 3.21 или более ранней придется использовать не столь очевидное имя параметра:

```
cmake --install <build tree> --prefix <prefix>
```

Заметим, что в Windows это не работает, потому что пути обычно начинаются с буквы диска.

Установка для генераторов с несколькими конфигурациями

Как и на этапе сборки, мы можем задать тип сборки для установки (за деталями обратитесь к разделу «Сборка проекта»). Допустимы значения `Debug`, `Release`, `MinSizeRel` и `RelWithDebInfo`. Порядок вызова следующий:

```
cmake --install <build tree> --config <cfg>
```

Выбор устанавливаемых компонентов

Разработчик может разбить свой проект на компоненты, устанавливаемые независимо. Мы будем подробно обсуждать концепцию компонентов в главе 14, а пока давайте считать, что это просто набор артефактов, которые необязательно использовать в каждом случае. Это может быть что-то

типа `application`, `docs` и `extra-tools`. Команда установки одного компонента имеет вид:

```
cmake --install <build tree> --component <component>
```

Задание прав доступа к файлам

Если установка производится на Unix-подобной платформе, то можно задать права доступа по умолчанию для устанавливаемых каталогов с помощью параметра вида `u=rwx,g=rx,o=rx`:

```
cmake --install <build tree>
--default-directory-permissions <permissions>
```

Отладка процесса установки

Как и на этапе сборки, мы можем заказать подробный вывод на этапе установки. Для этого годится любая из следующих команд:

```
cmake --install <build tree> --verbose
cmake --install <build tree> -v
```

Того же эффекта можно достичь, задав переменную окружения `VERBOSE`.

Выполнение скрипта

Проекты CMake конфигурируются на специализированном языке CMake. Он кросс-платформенный и весьма мощный. Раз уж он есть, так почему не воспользоваться им и для других задач? Понятное дело, CMake может выполнять автономные скрипты (подробнее об этом написано в разделе «Скрипты и модули») следующим образом:

```
cmake [{-D <var>=<value>}...] -P <cmake script file>
      [-- <unparsed options>...]
```

При выполнении такого скрипта нет этапов *конфигурирования* или *генерирования*, и кеш не затрагивается.

Передать скрипту значения можно двумя способами:

- в переменных с помощью параметра `-D`;
- в аргументах после пары символов `--`.

Во втором случае CMake создает переменные `CMAKE_ARGV<n>` для каждого переданного аргумента (включая `--`).

Выполнение командной программы

В редких случаях бывает необходимо запустить одну команду платформенно-независимым способом – например, скопировать файл или вычислить конт-

рольную сумму. Платформы не одинаковы, поэтому команды, имеющиеся в одной системе, могут отсутствовать в другой (или называться по-другому).

CMake предлагает режим, в котором наиболее употребительные команды можно выполнить единообразно на всех платформах. Порядок вызова такой:

```
cmake -E <command> [<options>]
```

Поскольку этот режим применяется ограниченно, мы не будем рассматривать его подробно. Но интересующимся предлагаю набрать `cmake -E` и получить список всех имеющихся команд. Просто чтобы вы могли составить впечатление о том, что предлагается, скажу, что CMake 3.26 поддерживает следующие команды: `capabilities`, `cat`, `chdir`, `compare_files`, `copy`, `copy_directory`, `copy_directory_if_different`, `copy_if_different`, `echo`, `echo_append`, `env`, `environment`, `make_directory`, `md5sum`, `sha1sum`, `sha224sum`, `sha256sum`, `sha384sum`, `sha512sum`, `remove`, `remove_directory`, `rename`, `rm`, `sleep`, `tar`, `time`, `touch`, `touch_nocreate`, `create_symlink`, `create_hardlink`, `true` и `false`.

Если нужной вам команды нет в этом списке или вам требуется более сложное поведение, попробуйте обернуть его скриптом и выполнить в режиме `-P`.

Выполнение готовой конфигурации технологического процесса

Выше мы говорили, что сборка с применением CMake включает три этапа: конфигурирование, генерирование и сборку. Дополнительно мы можем прогонять автоматизированные тесты и даже создавать дистрибутивные пакеты. Обычно пользователи должны вручную выполнять каждый шаг по отдельности, вызывая соответствующее действие `cmake` из командной строки. Но в более сложных проектах можно определить **готовые конфигурации технологических процессов**, объединяющие несколько шагов в одно действие, выполняемое всего одной командой. Пока что упомянем только, что пользователь может получить список имеющихся готовых конфигураций технологических процессов, выполнив команду

```
cmake --workflow --list-presets
```

Выполнить же готовую конфигурацию технологического процесса позволяет команда

```
cmake --workflow --preset <name>
```

Подробнее мы рассмотрим эту тему в главе 16.

Получение справки

Неудивительно, что CMake предлагает обширную справку, доступную из командной строки. Соответствующая команда:

```
cmake --help
```

Она печатает список тем, которые можно раскрыть и узнать, какие параметры нужно добавить, чтобы получить дополнительную информацию.

Командная строка CTest

Автоматизированное тестирование очень важно для создания и сопровождения высококачественного кода. В комплект CMake входит специализированная командная утилита для этой цели – CTest. Она позволяет стандартизировать способ прогона тестов и получения отчетов о результатах. Пользователю CMake необязательно знать детали тестирования конкретного проекта: какой каркас используется и как он запускается. CTest предоставляет удобный интерфейс для перечисления, фильтрации, переупорядочения, повторного запуска и хронометража прогона тестов.

Чтобы прогнать тесты для собранного проекта, нам нужно только вызвать `ctest` в сгенерированном дереве сборки:

```
$ ctest
Test project /tmp/build
Guessing configuration Debug
    Start 1: SystemInformationNew
1/1 Test #1: SystemInformationNew ......... Passed 3.19 sec
100% tests passed, 0 tests failed out of 1
Total Test time (real) = 3.24 sec
```

Мы посвятили этой теме целую главу 11.

Командная строка CPack

Собрав и протестировав свою замечательную программу, мы готовы поделиться ей с миром. Лишь немногие опытные пользователи способны уверенно работать с исходным кодом. А большинство пользуется заранее откомпилированными двоичными файлами из соображений удобства и экономии времени.

CMake и тут не бросает вас – как говорится, батарейки в комплекте. Инструмент CPack создает дистрибутивные пакеты для различных платформ: сжатые архивы, исполняемые установщики, мастера, пакеты для NuGet, macOS, DMG, RPM и т. д.

Принцип работы CPack такой же, как у CMake: она конфигурируется на языке CMake и предлагает на выбор много *генераторов пакетов* (не путайте с генераторами сборочных систем в CMake). Эти вопросы мы будем изучать в главе 14, поскольку инструмент предназначен для достаточно зрелых проектов CMake.

Графический интерфейс CMake

В состав CMake для Windows входит графическая версия для конфигурирования процесса сборки уже подготовленных проектов. Для Unix-подобных платформ имеется версия, собранная с библиотеками Qt. Для Ubuntu она предлагается в виде пакета `cmake-qt-gui`.

Для доступа к графической версии CMake GUI выполните файл `cmake-gui`.

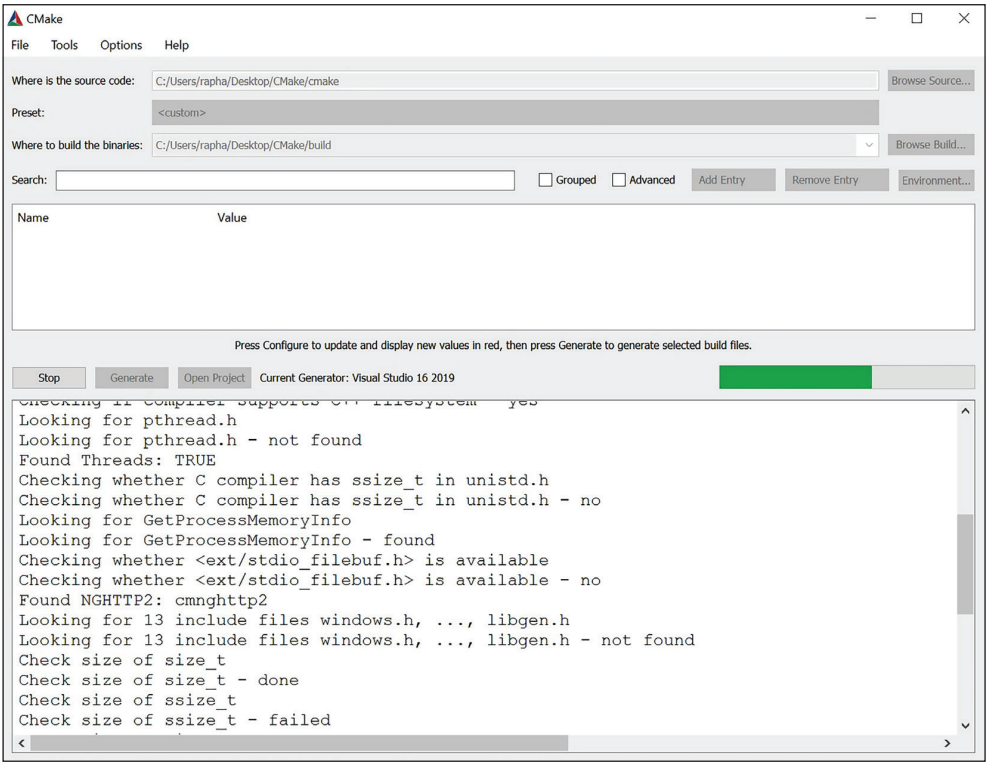


Рис. 1.3. Графический интерфейс CMake – этап конфигурирования сборочной системы с помощью генератора для Visual Studio 2019

Графический интерфейс удобен пользователям вашего приложения, он может быть полезен тем, кто плохо знаком с командной строкой.



ПОЛЬЗУЙТЕСЬ КОМАНДНЫМИ ИНСТРУМЕНТАМИ

Я, безусловно, рекомендую графический интерфейс конечным пользователям, но программистам – вроде вас – все же советую избегать ручных операций, которые требуют щелкать мышью в формах при каждой сборке программы. Особенно это важно в зрелых проектах, когда всю сборку можно выполнить без единого вмешательства пользователя.

Командная строка CCMake

Программа `ccmake` – это интерактивный текстовый интерфейс к CMake на Unix-подобных платформах (для Windows его можно собрать явно, но по умолчанию он недоступен). Я упоминаю об этом только для того, чтобы вы не растерялись, увидев его (см. рис. 1.4, но, как и в случае графического интерфейса, разработчикам удобнее редактировать файл `CMakeCache.txt` напрямую).

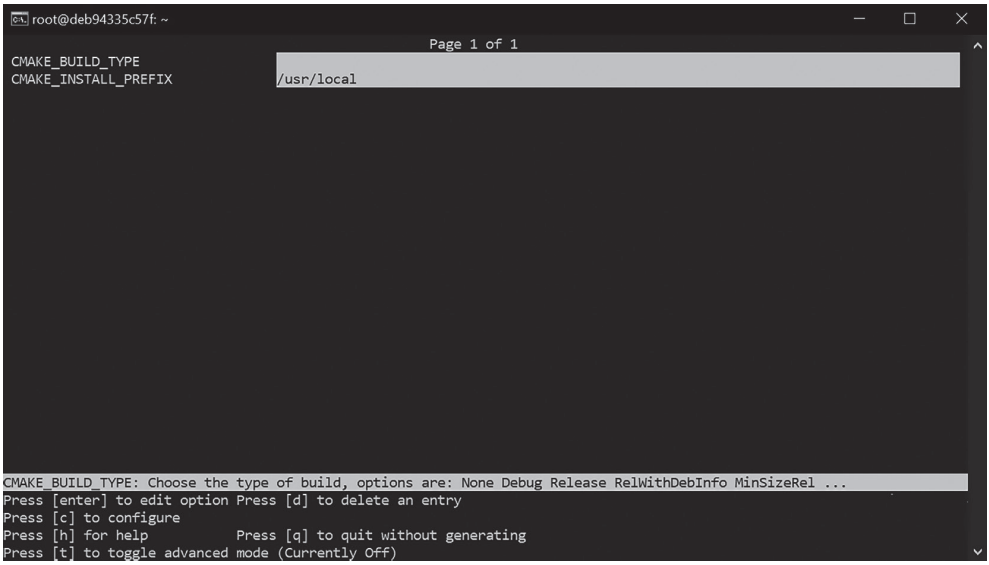


Рис. 1.4. Этап конфигурирования с применением ccmake

Оставив введение в командную строку позади, мы можем перейти к изучению структуры типичного проекта CMake.

Навигация по файлам и каталогам проекта

Проекты CMake включают множество файлов и каталогов. Давайте поговорим о том, для чего предназначен каждый из них. Имеется несколько общих категорий.

- Разумеется, есть файлы проекта, которые мы, разработчики, пишем и изменяем по мере развития проекта.
- Есть также файлы, которые генерирует CMake, и хотя они содержат команды на языке CMake, для редактирования они не предназначены. Все внесенные вручную изменения будут впоследствии затерты CMake.

- Некоторые файлы предназначены для опытных пользователей (но не разработчиков проекта), которые могут настроить процесс сборки проекта под свои потребности.
- Наконец, имеются временные файлы, которые предоставляют ценную информацию в конкретных контекстах.

В этом разделе мы также порекомендуем, какие файлы следует поместить в файл *ignore* **системы управления версиями** (**СУВ**).

Дерево исходного кода

Это каталог, в котором будет располагаться ваш проект (он также называется **корнем проекта**). Там находятся все исходные файлы на C++ и файлы проекта CMake.

Отметим наиболее важные факты, касающиеся этого каталога.

- В нем должен находиться конфигурационный файл `CMakeLists.txt`.
- Путь к этому каталогу задается пользователем с помощью аргумента `-S` команды `cmake` на этапе генерирования сборочной системы.
- Избегайте зашивания абсолютного пути к *дереву исходного кода* в код для CMake – пользователи вашей программы будут хранить проект в другом месте.

Здравая идея – инициализировать в этом каталоге репозиторий СУВ, например `Git`.

Дерево сборки

CMake создает этот каталог на пути, заданном пользователем. Там будет храниться сборочная система и все, что создается во время сборки: артефакты проекта, временная конфигурация, кеш, журналы сборки и результат платформенного инструмента сборки (например, GNU Make). Этот каталог также называется **корнем сборки** и **двоичным деревом**.

Отметим, что следует запомнить.

- Здесь создаются ваша конфигурация сборки (сборочная система) и артефакты сборки (исполняемые файлы и библиотеки, а также *объектные файлы* и архивы, необходимые для окончательной компоновки).
- Рекомендуется размещать этот каталог вне дерева исходного кода (так называемая **вынесенная сборка**). Таким образом мы сможем предотвратить загрязнение нашего проекта (**внутренняя сборка**).
- Этот каталог задается с помощью флага `-B` команды `cmake` на этапе *генерирования сборочной системы*.
- Этот каталог не предназначен для окончательного размещения сгенерированных файлов. Рекомендуется включать в проекты этап установки, который копирует конечные артефакты сборки туда, где они должны находиться, а временные файлы, созданные на этапе сборки, удаляет.

Не включайте этот каталог в СУВ – каждый пользователь выбирает его самостоятельно. Если у вас имеются основательные причины для внутренней сборки, добавьте этот каталог в файл игнорирования своей СУВ (например, `.gitignore`).

Списковые файлы

Файлы, содержащие код на языке CMake, называются списковыми файлами и могут включаться путем вызова команд `include()` и `find_package()` или косвенно командой `add_subdirectory()`. CMake не навязывает никаких правил именования этих файлов, но, по соглашению, они имеют расширение `.cmake`.

Файл проекта

Проекты CMake конфигурируются с помощью спискового файла `CMakeLists.txt` (по историческим причинам он имеет нетрадиционное расширение). Этот файл должен находиться в корне дерева исходного кода каждого проекта, он первым исполняется на этапе конфигурирования.

Верхнеуровневый файл `CMakeLists.txt` должен содержать как минимум две команды:

- `cmake_minimum_required(VERSION <x.xx>)`: задает ожидаемую версию CMake и сообщает CMake, как относится к унаследованным поведениям с помощью политик;
- `project(<name> <OPTIONS>)`: определяет имя проекта (оно будет храниться в переменной `PROJECT_NAME`) и его конфигурационные параметры (подробнее см. главу 2).

По мере усложнения программы может возникнуть желание разбить ее на меньшие части, которые можно конфигурировать и развивать по отдельности. CMake поддерживает эту идею с помощью подкаталогов со своими собственными файлами `CMakeLists.txt`. Структура проекта могла бы выглядеть, как показано ниже:

```
myProject/CMakeLists.txt
myProject/api/CMakeLists.txt
myProject/api/api.h
myProject/api/api.cpp
```

Тогда для объединения проекта в единое целое можно было написать очень простой верхнеуровневый файл `CMakeLists.txt`:

```
cmake_minimum_required(VERSION 3.26)
project(app)
message("Top level CMakeLists.txt")
add_subdirectory(api)
```

В верхнеуровневом файле описаны основные аспекты проекта: управление зависимостями, требования и определение окружения. Также мы видим команду `add_subdirectory(api)`, которая включает еще один файл `CMakeListst.txt` из подкаталога `api` для выполнения шагов, специфичных для API приложения.

Файл кеша

Кешированные переменные генерируются по списковым файлам и сохраняются в файле `CMakeCache.txt` при первом выполнении этапа конфигурирования. Этот файл находится в корне дерева сборки и имеет довольно простой формат (некоторые строки для краткости удалены):

```
# Это файл CMakeCache.
# Для сборки в каталоге: /root/build tree
# Сгенерирован CMake: /usr/local/bin/cmake
# Ниже описан синтаксис файла:
# KEY:TYPE=VALUE
# KEY - имя переменной в кеше.
# TYPE - сообщение графическому интерфейсу о типе VALUE,
# НЕ РЕДАКТИРОВАТЬ TYPE!.
# VALUE - текущее значение KEY.
########################
# EXTERNAL - внешние кешированные элементы
########################

# Флаги, используемые компилятором CXX для отладочных сборок:
CMAKE_CXX_FLAGS_DEBUG:STRING=/MDd /Zi /Ob0 /Od /RTC1

# ... дополнительные переменные ...
########################
# INTERNAL - внутренние кешированные элементы
########################

# Дополнительная версия cmake, использованная для создания
# текущего кеша
CMAKE_CACHE_MINOR_VERSION:INTERNAL=19
# ... дополнительные переменные ...
```

Как видим, формат описан в комментариях. Кешированные элементы в секции `EXTERNAL` могут быть модифицированы пользователями, а элементы в секции `INTERNAL` управляются CMake.

Отметим два важных момента.

- Этим файлом можно манипулировать вручную, а также путем вызова `cmake`, `ccmake` или `cmake-gui`.
- Проект можно сбросить в начальное состояние, удалив этот файл, он будет заново сгенерирован по списковым файлам.

Кешированные переменные можно читать из списковых файлов и записывать в них. Иногда вычисление ссылки на переменную вызывает сложности; мы рассмотрим этот вопрос подробнее в главе 2.

Файл определения пакета

Значительную часть экосистемы CMake составляют внешние пакеты, от которых могут зависеть проекты. Они предоставляют библиотеки и инструменты кросс-платформенным способом. Авторы пакетов, которые хотят поддержать CMake, поставляют их вместе с файлом конфигурации пакета.

Писать эти файлы мы научимся в главе 14. А пока запомните несколько интересных деталей.

- **Конфигурационные файлы** содержат информацию о том, как использовать двоичные библиотеки, заголовки и вспомогательные инструменты. Иногда они включают макросы и функции CMake, которые можно использовать в проекте.
- **Конфигурационные файлы** называются `<PackageName>-config.cmake` или `<PackageName>Config.cmake`.
- Для включения пакетов служит команда `find_package()`.

Если требуется конкретная версия пакета, то CMake ищет файл `<PackageName>-config-version.cmake` или `<PackageName>ConfigVersion.cmake`.

Если поставщик не предоставил конфигурационного файла для пакета, то иногда конфигурация включена в саму CMake или может быть добавлена в проект с помощью **модуля поиска**.

Сгенерированные файлы

На этапе генерирования `cmake` создает много файлов в дереве сборки. Редактировать их вручную не следует. CMake использует их в качестве конфигурации для установки, CTest и CPack.

Вам могут встретиться такие файлы:

- `cmake_install.cmake`;
- `CTestTestfile.cmake`;
- `CPackConfig.cmake`.

Если вы производите внутреннюю сборку, то, наверное, стоит добавить эти имена в файл игнорирования СУВ.

Файлы в формате JSON и YAML

В CMake также используются файлы в форматах JSON (JavaScript Object Notation) (JSON) и YAML (Yet Another Markup Language). Они нужны для взаимодействия с внешними инструментами (типа IDE) или в качестве легко разбираемого формата конфигурационных файлов.

Файлы готовых конфигураций

Задание сложной конфигурации проектов может оказаться нетривиальной задачей, поскольку приходится думать сразу о многих вещах: кешированных переменных, выборе генератора, пути к дереву сборки и т. д. – особенно если способов собрать проект несколько. Тут-то и приходят на помощь готовые конфигурации – вместо того чтобы вручную задавать все эти значения в командной строке, мы можем просто включить в состав проекта файл, содержащий все детали. Начиная с версии CMake 3.25 такие файлы позволяют также описывать **технологические процессы**, которые связывают все этапы (конфигурирование, сборка, тестирование и упаковка) в именованный список подлежащих выполнению шагов.

В разделе «Работа с командной строкой» выше мы упоминали, что пользователь может увидеть список готовых конфигураций с помощью графического интерфейса или команды `--list-presets`, а выбрать готовую конфигурацию для сборочной системы – с помощью параметра `--preset=<preset>`.

Готовые конфигурации хранятся в двух файлах:

- `CMakePresets.json`: в нем авторы проектов предоставляют официальные готовые конфигурации;
- `CMakeUserPresets.json`: этот файл предназначен для пользователей, которые хотят изменить конфигурацию проекта в соответствии со своими нуждами (можете добавить его в файл игнорирования СУВ).

Готовые конфигурации не являются обязательным компонентом проекта, они полезны только в сложных случаях. Дополнительные сведения см. в главе 16.

Файловый API

В версии CMake 3.14 появился API, позволяющий внешним инструментам запрашивать информацию о сборочной системе: пути к сгенерированным файлам, кешированные элементы, комплекты инструментов и т. п. Мы затрагиваем эту очень продвинутую тему только для того, чтобы вы не пугались, встретив фразу «файловый API» (file-based API) в документации. Само название содержит подсказку о принципе работы: JSON-файл, содержащий запрос, следует поместить в специальное место в дереве сборки. CMake прочитает этот файл во время генерирования сборочной системы и запишет ответ в другой файл, чтобы внешние приложения могли разобрать его.

Файловый API пришел на смену устаревшему механизму *серверного режима* (или `cmake-server`), который был окончательно удален в версии CMake 3.26.

Журнал конфигурирования

Начиная с версии 3.26 CMake предоставляет структурированный файл журнала для сложной отладки *этапа конфигурирования*:

```
<build tree>/CMakeFiles/CMakeConfigureLog.yaml
```

Это одна из тех возможностей, на которые обычно не обращаешь внимания, – пока не припрет.

Игнорирование файлов в Git

Систем управления версиями много, одна из самых популярных – Git. Приступая к новому проекту, нужно позаботиться о том, чтобы в репозиторий помещались только нужные файлы. Поддерживать гигиену проекта будет проще, если добавить имена не интересующих нас файлов в файл `.gitignore`. Например, можно исключить генерируемые файлы, файлы, зависящие от пользователя, и временные файлы. Git будет автоматически пропускать их при создании новых фиксаций. Вот какой файл я использую в собственных проектах:

ch01/01-hello/.gitignore

```
CMakeUserPresets.json
# Для внутренних сборок исключать их результаты:
build_debug/
build_release/

# Сгенерированные и пользовательские файлы
**/CMakeCache.txt
**/CMakeUserPresets.json
**/CTestTestfile.cmake
**/CPackConfig.cmake
**/cmake_install.cmake
**/install_manifest.txt
**/compile_commands.json
```

Теперь у вас есть карта для путешествий по стране проектных файлов. Некоторые файлы очень важны, и вы будете использовать их постоянно, другие – не так часто. Хотя их изучение может показаться пустой тратой времени, очень полезно знать, где *не нужно искать ответы*. Так или иначе, у нас остался нерассмотренным последний вопрос: какие еще замкнутые единицы можно создавать с помощью CMake?

Скрипты и модули

Основным предметом CMake являются проекты, собираемые с целью порождения артефактов, потребляемых другими системами (например, конвейерами CI/CD и платформами тестирования, или разворачиваемыми на других машинах, или хранящимися в репозиториях артефактов). Однако в CMake есть еще два объекта, в которых используется ее язык: скрипты и модули. Объясним, что это такое и чем они различаются.

Скрипты

CMake предлагает платформенно-независимый язык программирования, содержащий много полезных команд.

Написанные на нем скрипты можно включить в более крупный проект или использовать независимо. Считайте, что это способ единообразно выполнить какую-то кросс-платформенную работу. Обычно для этого приходится создавать Bash-скрипт для Linux, пакетные файлы или скрипты PowerShell для Windows и т. д. А CMake абстрагирует различия, так что один файл будет работать на всех платформах. Разумеется, можно было бы использовать внешние инструменты, такие как скрипты на Python, Perl или Ruby, но это дополнительная зависимость, которая усложняет проекты на C/C++. Так зачем еще один язык, если в большинстве случаев можно решить проблему гораздо проще? Используйте CMake!

Мы уже знаем, что для исполнения скрипта нужно задать параметр `-P`: `cmake -P script.cmake`.

Но предъявляются ли к скриптам какие-то особые требования? Не так много: скрипт может быть сколь угодно сложным, а может быть вообще пустым. Но все же рекомендуется выполнять команду `cmake_minimum_required()` в начале каждого скрипта. Она сообщает CMake, какие политики применять к последующим командам данного проекта (дополнительные сведения см. в главе 4).

Ниже приведен пример простого скрипта:

ch01/02-script/script.cmake

```
# Пример скрипта
cmake_minimum_required(VERSION 3.26.0)
message("Hello world")
file(WRITE Hello.txt "I am writing to a file")
```

При выполнении скриптов CMake не выполняет обычные этапы (например, конфигурирование или генерирование) и не использует кеш, поскольку в скрипте нет понятий **дерева исходного кода** или **дерева сборки**. Это означает, что в режиме скрипта команды CMake, относящиеся к конкретному проекту, недоступны. Вот и все! Хороших скриптов вам.

Служебные модули

В проектах CMake можно использовать внешние модули для расширения функциональности. Эти модули пишутся на языке CMake и содержат определения макросов, переменные и команды для выполнения произвольных функций. Это могут быть как сложные скрипты (например, предоставляемые `CPack` и `CTest`), так и довольно простые, например `AddFileDependencies` или `TestBigEndian`.

В дистрибутив CMake входит свыше 80 служебных модулей. Если этого мало, можете скачать дополнительные из интернета, поискав в администри-

руемых списках, например `https://github.com/onqtam/awesome-cmake`, или написать собственный модуль.

Для использования служебного модуля необходимо вызвать команду `include(<MODULE>)`. Ниже приведен простой пример:

ch01/03-module/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(ModuleExample)
include (TestBigEndian)
test_big_endian(IS_BIG_ENDIAN)
if(IS_BIG_ENDIAN)
message("BIG_ENDIAN")
else()
message("LITTLE_ENDIAN")
endif()
```

Об имеющихся модулях мы будем рассказывать по ходу дела. Полный список включенных в дистрибутив модулей имеется по адресу https://cmake.org/cmake/help/latest/manual/cmake-modules.7.html.

Модули поиска

В разделе «Файл определения пакета» я упомянул, что в CMake имеется механизм для поиска файлов, принадлежащих внешним зависимостям, которые не поддерживают CMake и не предоставляют конфигурационного файла пакета. Это делают модули поиска. CMake предлагает свыше 150 модулей поиска, умеющих находить такие пакеты среди установленных в системе. Как и в случае служебных модулей, множество модулей поиска имеется в сети. А если больше ничего не остается, можете написать свой собственный.

Для использования таких модулей следует вызвать команду `find_package()`, указав имя пакета в качестве аргумента. Модуль поиска проверяет все известные места, где может находиться указанный пакет. Если файлы найдены, то будут определены переменные, содержащие пути к ним (как описано в руководстве по модулю). После этого CMake может собрать зависимость.

Например, модуль `FindCURL` ищет популярную библиотеку *Client URL* и определяет следующие переменные: `CURL_FOUND`, `CURL_INCLUDE_DIRS`, `CURL_LIBRARIES` и `CURL_VERSION_STRING`.

Мы будем рассматривать модули поиска в главе 9.

Резюме

Теперь вы понимаете, что такое CMake и как она работает. Вы узнали об основных компонентах, входящих в состав семейства инструментов CMake, и о том, как эта программа устанавливается в различные системы. Теперь

вы, как опытный пользователь, в курсе всех способов запуска CMake из командной строки: генерирование сборочной системы, сборка проекта, установка, запуск скриптов, инструменты командной строки и печать справки. Вы имеете представление о CTest, CPack и приложениях с графическим интерфейсом. Это позволит вам создавать проекты, ориентированные на пользователей и других разработчиков. Кроме того, вы узнали, из чего состоит проект: каталоги, списковые файлы, конфигурационные файлы, готовые конфигурации и вспомогательные файлы. Вы понимаете, какие файлы следует игнорировать при включении в СУВ. Наконец, мы немного поговорили о файлах, не являющихся частью проекта: автономных скриптах и двух видах модулей: служебные и модули поиска.

В следующей главе вы узнаете о языке программирования CMake. Это позволит вам писать собственные списковые файлы и откроет путь к созданию своего первого скрипта, проекта и модуля.

Для дополнительного чтения

За дополнительными сведениями отсылаем вас к следующим источникам.

- Официальная страница CMake: https://cmake.org/.
- Генераторы с одной конфигурацией: https://cgold.readthedocs.io/en/latest/glossary/single-config.html.
- Разделение этапов в графическом интерфейсе к CMake: https://stackoverflow.com/questions/39401003/.

2

Язык CMake

Создание кода на **языке CMake** труднее, чем можно было бы ожидать. Когда вы впервые читаете списковый файл CMake, может сложиться впечатление, что язык настолько прост, что садись и пиши без всякой теории. Потом вы, возможно, попробуете внести изменения и поэкспериментировать с кодом, не вполне понимая, как он работает. Не стану порицать вас за это. Мы, программисты, обычно очень заняты, и вопросы сборки не кажутся настолько интересными, чтобы тратить время на их внимательное изучение. Стремясь поскорее закончить, мы склонны вносить кажущиеся правильными изменения в надежде, что они решат проблему. Такой подход к решению технических проблем называется *шаманским программированием*.

Язык CMake кажется тривиальным: но после внесения небольшого дополнения или исправления мы внезапно осознаем, что оно «не работает». Обычно время, потраченное на отладку, оказывается больше, чем время, за которое можно было бы изучить сам вопрос. По счастью, вам такая судьба не грозит, потому что из этой главы вы почерпнете большую часть важных знаний, необходимых для практического использования CMake.

В этой главе мы не только узнаем о структурных элементах языка CMake – **комментариях**, **командах**, **переменных** и **управляющих конструкциях**, – но и получим необходимые базовые знания и потренируемся на примерах.

CMake ставит вас в уникальное положение. С одной стороны, вы играете роль инженера по сборке и должны хорошо разбираться в компиляторах, платформах и смежных вопросах. С другой стороны, вы разработчик, который пишет код, генерирующий сборочную систему. Создание высококачественного кода – непростая задача, требующая многостороннего подхода. Код должен быть не только функциональным и удобочитаемым, его еще должно быть легко анализировать, расширять и сопровождать.

В заключение мы приведем список наиболее употребительных и практически полезных команд CMake. Команды, которые тоже используются, но не настолько часто, перечислены в приложении (справочные руководства по командам `string`, `list`, `file` и `math`).

В этой главе рассматриваются следующие вопросы:

- основы синтаксиса языка CMake;
- работа с переменными;

- списки;
- управляющие конструкции в CMake;
- наиболее употребительные команды.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch02.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Основы синтаксиса языка CMake

Код для CMake пишется во многом как код на любом другом императивном языке: строки выполняются сверху вниз и слева направо, иногда поток управления заходит во включаемый файл или в вызываемую функцию. Начальная точка выполнения определяется режимом (см. раздел «Работа с командной строкой» главы 1): это либо корневой файл дерева исходного кода (`CMakeLists.txt`), либо файл скрипта с расширением `.cmake`, переданный `cmake` в качестве аргумента.

Поскольку скрипты CMake используются в языке CMake всюду, кроме мест, связанных с проектом, мы будем использовать их в этой главе для освоения синтаксиса CMake. Научившись писать простые списковые файлы, мы сможем перейти к созданию настоящих файлов проекта, но это тема главы 4.

Напомним, что скрипты запускаются командой `cmake -P script.cmake`.



CMake поддерживает текстовые файлы в **7-разрядной кодировке ASCII**, чтобы обеспечить переносимость между платформами. Строки могут оканчиваться символами `\n` или `\r\n`. Версии CMake, начиная с 3.2, поддерживают кодировки **UTF-8** и **UTF-16** с факультативными **маркерами порядка байтов**.

Все, встречающееся в списковом файле CMake, – *комментарий* или *вызов команды*.

Комментарии

Как и в **C++**, имеется два вида комментариев: *однострочные* и *многострочные*. Но, в отличие от C++, многострочные комментарии могут быть вложенными. Однострочный комментарий начинается знаком решетки #:

```
# он может занимать всю строку
message("Hi"); # или находиться в конце строки после команды
```

Многострочные комментарии начинаются символом #, за которым следует открывающая квадратная скобка [, произвольное число знаков равенства = (в т. ч. ни одного) и еще одна квадратная скобка [. Чтобы закрыть многострочный комментарий, запишите столько же знаков равенства и закрывающих квадратных скобок]:

```
#[=[
многострочный комментарий
  #[[
    Вложенный многострочный комментарий
  #]]
#]=]
```

Можно ускоренно закрыть многострочный комментарий, добавив еще один знак # в его начальной строке:

```
##[=[ теперь это однострочный комментарий
больше не закомментирована
  #[[
    все еще вложенный комментарий
  #]]
#]=] теперь это однострочный комментарий
```

Знать, как используются комментарии, конечно, полезно, но возникает вопрос: когда это нужно делать? Поскольку написание списковых файлов – это по существу программирование, будет разумно следовать сложившейся практике кодирования.

Код, следующий таким практикам, часто называют *чистым* – этот термин долгие годы используется такими гуру разработки ПО, как Роберт К. Мартин, Мартин Фаулер и другие.

По вопросу о том, какие практики считать полезными, а какие – вредными, мнения разнятся. И комментарии не остались в стороне от этих споров. Все следует оценивать применительно к конкретной ситуации, но все согласны, что хороший комментарий должен обладать следующими свойствами:

- **информация:** комментарий помогает разобраться в сложностях применения таких вещей, как регулярные выражения или форматные строки;

- **намерение:** комментарий объясняет назначение кода, если оно не очевидно из реализации или интерфейса;
- **разъяснение:** комментарий объясняет концепции, которые трудно подвергнуть рефакторингу или изменить;
- **предупреждение о последствиях:** комментарий предупреждает о том, что некий код может привести к неработоспособности других частей программы;
- **привлечение внимания:** комментарий может подчеркивать важность идеи, которую трудно выразить в коде;
- **юридические оговорки:** комментарий может нести это необходимое зло, которое обычно выходит за рамки компетенции программиста.

Лучше избегать комментариев, заменяя их подходящим именованием, рефакторингом или исправлением кода. Опускайте комментарии в следующих случаях:

- **официальные:** они добавляются ради полноты, но на самом деле не важны;
- **избыточные:** повторяют то, что уже ясно выражено в коде;
- **сбивающие с толку:** те, что устарели или неправильны, поскольку не отражают внесенные в код изменения;
- **протокольные:** говорят о том, что и когда было изменено (для этого лучше использовать **систему управления версиями** (**СУВ**));
- **разделительные:** отмечают секции кода.

Создание элегантного кода – непростая задача, но это облегчает жизнь читателю. Поскольку мы тратим больше времени на чтение кода, чем на его написание, всегда нужно стремиться к тому, чтобы код было легко читать, а не просто закончить работу как можно быстрее. Я рекомендую ссылки, приведенные в конце этой главы: среди них есть хорошие материалы по написанию *чистого кода*. Если вас заинтересовали комментарии, то там приведена ссылка на мой ролик на YouTube «Which comments in your code ARE GOOD?», в котором эта тема рассматривается более глубоко.

Вызов команд

Пора уже что-то сделать! Вызов команд – это самая суть списковых файлов в CMake. Чтобы выполнить команду, необходимо указать ее имя, а затем в скобках – список **аргументов команды**, разделенных пробелами.

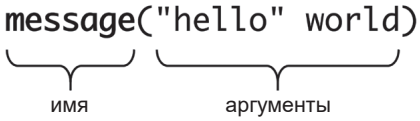


Рис. 2.1. Пример команды

Имена команд нечувствительны к регистру, но в сообществе CMake принято соглашение о «змеином стиле», когда слова записываются строчными

буквами и соединяются знаками подчеркивания. Вы можете определять собственные команды, этот вопрос рассматривается в разделе «Управляющие конструкции в CMake» ниже.

Существенное различие между CMake и C++ заключается в том, что в CMake вызов команды не является выражением. Это означает, что нельзя передать одну команду в качестве аргумента другой, потому что *все* внутри скобок рассматривается как аргумент конкретной команды.

Команды CMake также не завершаются точкой с запятой, поскольку в любой строке исходного кода может находиться только одна команда.

За командой может следовать комментарий:

```
command(argument1 "argument2" argument3) # комментарий
command2() #[[ многострочный комментарий
```

Но не наоборот:

```
#[[ скобки
]] command()
```

Как было сказано выше, все, встречающееся в списковом файле CMake, – либо *комментарий*, либо *вызов команды*. Да, действительно, синтаксис языка CMake очень прост, и в большинстве случаев это хорошо. Ограничения существуют (например, нельзя инкрементировать переменную-счетчик с помощью выражения), но, как правило, они приемлемы, потому что CMake и не претендует на роль языка общего назначения.

CMake предоставляет команды для манипулирования переменными, изменения потока выполнения, модификации файлов и многого другого. Чтобы упростить изложение, мы будем вводить новые команды по мере проработки примеров. Эти команды можно отнести к двум большим группам:

- **скриптовые команды:** они доступны всегда, изменяют состояние процессора команд, осуществляют доступ к переменным и влияют на другие команды и окружение;
- **проектные команды:** они доступны только в проектах, манипулируют состоянием проекта и целей сборки.

Практически все команды зависят от других элементов языка: переменных, условных предложений и, самое главное, от аргументов командной строки. Теперь посмотрим, как с ними работать.

Аргументы команд

Многие команды в CMake нуждаются в аргументах, определяющих их поведение. Как показано на рис. 2.1, кавычки вокруг аргументов могут показаться довольно странными. Одни аргументы требуют кавычек, другие – нет. В чем причина такого различия?

На внутреннем уровне CMake распознает только один тип данных – `string`. Поэтому любая команда ожидает получить нуль или более строк в качестве аргументов. CMake **вычисляет** каждый аргумент, приводя его к статической строке, и передает команде. Под *вычислением* понимается **строковая интерполяция**, т. е. подстановка значений вместо частей строки. Это может относиться к **управляющим последовательностям**, расширению **ссылок на переменные** (или *интерполяции переменных*) и распаковке **списков**.

В зависимости от контекста мы можем разрешать такое вычисление по мере необходимости. Поэтому CMake предлагает три типа аргументов:

- аргументы в квадратных скобках;
- аргументы в кавычках;
- аргументы без кавычек.

У каждого типа аргумента есть свои особенности и разные уровни вычисления.

Аргументы в квадратных скобках

Аргументы в скобках не вычисляются, потому что используются для передачи многострочных строк (multiline string) буквально в виде одного аргумента. Это значит, что такой аргумент может включать пробельные символы, в т. ч. знаки табуляции и новой строки.

Аргументы в скобках форматируются так же, как комментарии, – начинаются последовательностью `[=[` и заканчиваются последовательностью `]=]`, причем число знаков равенства должно совпадать (знаки равенства можно и опускать, но тогда в обеих последовательностях). Но, в отличие от комментариев, аргументы в скобках не могут быть вложенными.

Ниже приведен пример такого аргумента для команды `message()`, которая выводит все переданные аргументы на экран:

ch02/01-arguments/bracket.cmake

```
message([[multiline
  bracket
  argument
]])
message([==[
  because we used two equal-signs "=="
  this command receives only a single argument
  even if it includes two square brackets in a row
  { "petsArray" = [["mouse","cat"],["dog"]] }
]==])
```

В этом примере показаны различные формы аргументов в скобках. Обратите внимание, что размещение закрывающих скобок в отдельной строке в первом вызове порождает пустую строку на выходе:

```
$ cmake -P ch02/01-arguments/bracket.cmake
multiline
```

```
bracket
argument
  because we used two equal-signs "=="
  following is still a single argument:
  { "petsArray" = [["mouse","cat"],["dog"]] }
```

Вторая форма полезна, когда мы передаем текст, содержащий двойные квадратные скобки (]]), поскольку в этом случае они не будут интерпретироваться как признак конца аргумента.

Аргументы в квадратных скобках имеют ограниченное применение – как правило, они содержат длинные блоки текста, предназначенные для показа пользователю. В большинстве случаев нужно что-то более динамичное, например аргументы в кавычках.

Аргументы в кавычках

Аргументы в кавычках напоминают обычную строку в C++ – они позволяют сгруппировать несколько символов, включая пробельные, и в них расширяются *управляющие последовательности*. Как и строки в C++, они начинаются и завершаются символом двойной кавычки, а для включения этого символа внутрь строки ему нужно предпослать знак обратной косой черты, `\"`. Поддерживаются и другие хорошо известные управляющие последовательности: `\\` обозначает сам символ обратной косой черты, `\t` – символ табуляции, `\n` – символ новой строки, `\r` – символ возврата каретки.

Но на этом сходство со строками C++ заканчивается. Аргументы в кавычках могут занимать несколько строк, и в них интерполируются ссылки на переменные. Можете считать, что это встроенная функция `sprintf` из **C** или функция `std::format` из **C++20**. Чтобы вставить в аргумент ссылку на переменную, оберните ее имя: `${name}`. О ссылках на переменные мы будем говорить в разделе «Работа с переменными» ниже.

Сможете сказать, сколько строк будет в выводе следующего скрипта?

ch02/01-arguments/quoted.cmake

```
message("1. escape sequence: \" \n in a quoted argument")
message("2. multi...
  line")
message("3. and a variable reference: ${CMAKE_VERSION}")
```

Посмотрим:

```
$ cmake -P ch02/01-arguments/quoted.cmake
1. escape sequence: "
in a quoted argument
2. multi...
line
3. and a variable reference: 3.26.0
```

Да, мы имеем один экранированный символ двойной кавычки, один экранированный символ новой строки и один буквальный символ новой строки. Мы также обратились к встроенной переменной `CMAKE_VERSION`, которая была интерполирована в последнюю строку.

Аргументы без кавычек

Программисты привыкли к тому, что строки должны как-то обозначаться – одиночными кавычками, двойными кавычками или обратной косой чертой. CMake отходит от этой традиции и вводит *аргументы без кавычек*. Можно высказать мнение, что без ограничителей читать код проще. Так ли это? Решать вам.

В аргументах без кавычек вычисляются *управляющие последовательности* и *ссылки на переменные*. Но будьте осторожны с точками с запятой (`;`), поскольку в CMake они рассматриваются как разделители элементов списка. Если аргумент содержит точку с запятой, то CMake разбивает его на несколько аргументов. Если вам это не нужно, то экранируйте точки с запятой обратной косой чертой, `\;`. Мы еще будем говорить о точках с запятой в разделе «Списки» ниже.

Вам может показаться, что работать с такими аргументами неудобно, поэтому покажем наглядно, как они разбиваются на несколько.

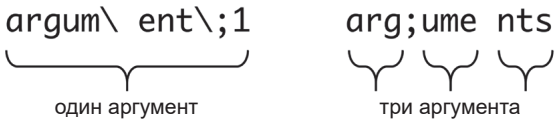


Рис. 2.2. Благодаря управляющим последовательностям отдельные лексемы интерпретируются как один аргумент

С аргументами без кавычек необходима сугубая осторожность. Некоторые команды CMake требуют вполне определенное число аргументов и лишние игнорируют. Если случайно окажется, что аргументы разделены, то будет иметь место трудно отлаживаемая ошибка.

В аргументах без кавычек не может быть неэкранированных кавычек (‘’), знаков решетки (#) и обратной косой черты (\). И как будто этого недостаточно, круглые скобки, (), разрешено использовать, только если они парные. То есть открывающей скобке должна соответствовать закрывающая – до закрытия списка аргументов команды.

Примеры ниже демонстрируют описанные правила:

ch02/01-arguments/unquoted.cmake

```
message(a\ single\ argument)
message(two arguments)
message(three;separated;arguments)
message(${CMAKE_VERSION}) # ссылка на переменную
message(()()())           # парные скобки
```

Что выводит этот код? Посмотрим:

```
$ cmake -P ch02/01-arguments/unquoted.cmake
a single argument
twoarguments
threeseparatedarguments
3.16.3
()()()
```

Даже простая команда типа `message()` весьма привередлива к отдельным аргументам без кавычек. Пробел в строке `single argument` напечатан правильно, потому что был явно экранирован. Однако слова `twoarguments` и `threeseparatearguments` *склеены*, потому что `message()` не добавляет пробелов по собственной инициативе.

С учетом всех этих сложностей, когда стоит использовать аргументы без кавычек? Некоторые команды CMake допускают факультативные аргументы, которым предшествует ключевое слово, означающее, что будет передан необязательный аргумент. В таких случаях задание ключевого слова в качестве аргумента без кавычек может сделать код понятнее. Например:

```
project(myProject VERSION 1.2.3)
```

В этой команде ключевое слово `VERSION` и следующий за ним аргумент `1.2.3` факультативны. Отметим, что ключевые слова чувствительны к регистру.

Разобравшись в сложностях и причудах аргументов в CMake, мы готовы перейти к следующей интересной теме – работе с переменными.

Работа с переменными

Переменные в CMake – на удивление сложный предмет. Мало того что есть три категории переменных – **нормальные**, **кешированные** и **окружения**, – так они еще располагаются в разных **областях видимости переменных**, и имеются специальные правила, диктующие, как одна *область видимости* влияет на другую. Очень часто непонимание этих правил становится причиной ошибок и страданий. Рекомендую внимательно прочитать этот раздел и убедиться, что вы все поняли; только потом можно двигаться дальше.

Начнем с основных фактов о переменных в CMake:

- имена переменных чувствительны к регистру и могут содержать почти любые символы;
- на внутреннем уровне значения всех переменных хранятся в виде строк, даже если некоторые команды интерпретируют их как значения других типов (в т. ч. *списков*!).

Основные команды манипулирования переменными – `set()` и `unset()`, но есть и другие, способные изменять значения переменных, например `string()` и `list()`.

Для объявления **нормальной переменной** мы просто вызываем `set()`, передавая имя и значение:

ch02/02-variables/set.cmake

```
set(MyString1 "Text1")
set([[My String2]] "Text2")
set("My String 3" "Text3")
message(${MyString1})
message(${My\ String2})
message(${My\ String\ 3})
```

Как видим, использование аргументов в квадратных скобках и в кавычках позволяет включать пробелы в имя переменной. Но, ссылаясь на нее впоследствии, мы должны будем экранировать пробелы символом \. Поэтому рекомендуется ограничиваться в именах переменных только буквами, цифрами, дефисами (-) и подчеркиваниями (_).

Кроме того, избегайте зарезервированных имен (в верхнем, нижнем или смешанном регистре), которые начинаются любой из следующих строк: `CMAKE_`, `_CMAKE_` или знаком подчеркивания _, за которыми следует имя любой команды CMake.

Чтобы стереть значение переменной, пользуйтесь командой `unset()`, например `unset(MyString1)`.



Команда `set()` принимает текстовое имя переменной в качестве первого аргумента, а команда `message()` требует ссылки на переменную, обернутую конструкцией `${}`.

Что будет, если передать команде `set()` обернутую переменную? Для ответа на этот вопрос нужно лучше разобраться с тем, что такое *ссылка на переменную*.

Ссылки на переменные

Я уже кратко упоминал ссылки в разделе «Аргументы команд», поскольку они вычисляются в аргументах в кавычках и без кавычек. Мы знаем, что для создания ссылки на ранее определенную переменную нужно использовать синтаксическую конструкцию `${}`, например `message(${MyString1})`.

В процессе вычисления CMake обходит *области видимости переменных* в направлении от самой внутренней к самой внешней и заменяет `${MyString1}` значением или пустой строкой, если переменная не найдена (CMake не выдает никаких сообщений об ошибке). Этот процесс называется *вычислением*, *расширением* или *интерполяцией переменной*.

Интерполяция производится изнутри наружу, начиная с самой глубоко вложенной пары фигурных скобок. Например, пусть встретилась ссылка `${MyOuter${MyInner}}`. Тогда:

1. CMake сначала попытается вычислить `MyInner`, а не искать переменную с именем `MyOuter${MyInner}`.
2. Если переменная `MyInner` успешно интерполирована, то CMake повторяет эту процедуру применительно к вновь образовавшейся ссылке – и так до тех пор, пока дальнейшее расширение окажется невозможным.

Во избежание неожиданностей мы рекомендуем не хранить лексемы расширения переменных в значениях переменных.

CMake доводит расширение переменной до конца и только после этого передает получившиеся значения команде в качестве аргументов. Потому-то при вызове `set(${MyInner} "Hi");` мы изменяем не переменную `MyInner`, а переменную, имя которой хранится в `MyInner`. Чаще всего это не то, чего мы хотели.

Ссылки на переменные ведут себя по-разному в зависимости от категории переменной, но в общем случае применимы следующие правила:

- конструкция `${}` используется для ссылки на *нормальные* или *кешированные* переменные;
- конструкция `$ENV{}` используется для ссылки на переменные *окружения*;
- конструкция `$CACHE{}` используется для ссылки на *кешированные* переменные.

Да, это так – конструкция `${}` позволяет получить значение переменной, принадлежащей одной из двух категорий: *нормальной* переменной, если она установлена в текущей области видимости, и *кешированной* переменной с таким же именем, если значение переменной не установлено или стерто. Если такой переменной нет, то результатом вычисления ссылки будет пустая строка.

В CMake предопределено много встроенных нормальных переменных, служащих разным целям. Например, можно передать в командной строке аргументы для скрипта после лексемы `--`, и они будут храниться в переменных `CMAKE_ARGV<n>` (а в переменной `CMAKE_ARGC` будет храниться их количество).

Познакомимся с другими категориями переменных, чтобы лучше понимать, что они собой представляют.

Переменные окружения

Это самый простой вид переменной. CMake создает копии переменных, которые присутствовали в окружении, из которого запущен процесс `cmake`, и делает их доступными в глобальной области видимости. Для ссылки на эти переменные используется конструкция `$ENV{<name>}`.

CMake может изменять эти переменные, но изменения вносятся только в локальную копию в процессе `cmake`, а не в само системное окружение. Более того, эти изменения не будут видны при последующих сборках или прогонах тестов, поэтому поступать так не рекомендуется.

Имейте в виду, что некоторые переменные окружения оказывают влияние на различные аспекты поведения CMake. Например, переменная `CXX` определяет, какой исполняемый файл будет вызываться для компиляции файлов,

написанных на C++. Мы будем рассматривать переменные окружения по ходу дела. А полный список доступен по адресу https://cmake.org/cmake/help/latest/manual/cmake-envvariables.7.html.



Важно понимать, что когда переменные окружения используются в качестве аргументов команд, их значения интерполируются на этапе генерирования сборочной системы. Это означает, что они будут навсегда встроены в дерево сборки, и изменение окружения на этапе сборки ни на что не повлияет.

Например, рассмотрим такой файл проекта:

ch02/03-environment/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.20.0)
project(Environment)
message("generated with " $ENV{myenv})
add_custom_target(EchoEnv ALL COMMAND echo "myenv in build is" $ENV{myenv})
```

Здесь мы видим два шага: переменная окружения `myenv` печатается на этапе конфигурирования, а затем с помощью команды `add_custom_target()` та же самая переменная печатается на этапе сборки. Что при этом происходит, показывает bash-скрипт, в котором одно значение используется на этапе конфигурирования, а другое – на этапе сборки:

ch02/03-environment/build.sh

```
#!/bin/bash
export myenv=first
echo myenv is now $myenv
cmake -B build .
cd build
export myenv=second
echo myenv is now $myenv
cmake --build .
```

Выполнение этого кода недвусмысленно показывает, что значение, заданное на этапе конфигурирования, остается в сгенерированной сборочной системе:

```
$ ./build.sh | grep -v "\-\-"
myenv is now first
generated with first
myenv is now second
Scanning dependencies of target EchoEnv
myenv in build is first
Built target EchoEnv
```

На этом мы временно завершаем обсуждение переменных окружения и переходим к последней категории: кешированным переменным.

Кешированные переменные

Впервые мы упомянули кешированные переменные при обсуждении параметров командной строки `cmake` в главе 1. По сути дела, это переменные, хранящиеся в файле `CMakeCache.txt` в дереве сборки. Они содержат информацию, собранную на *этапе конфигурирования* проекта. Их значения берутся из системы (пути к компиляторам, компоновщикам, инструментальным средствам и другим программам) и от пользователя, который предоставляет их в графическом интерфейсе или в командной строке с помощью параметра `-D`. Повторим, что кешированные переменные доступны только *проектам*, но не *скриптам*.

Кешированная переменная используется, если ссылка `${<name>}` не находит нормальную переменную в текущей области видимости, а кешированная переменная с таким же именем существует. Однако на них можно также сослаться явно с помощью синтаксической конструкции `$CACHE{<name>}` и определить с помощью специальной формы команды `set()`:

```
set(<variable> <value> CACHE <type> <docstring> [FORCE])
```

В отличие от команды `set()` для *нормальных переменных*, для кешированных необходимы дополнительные аргументы: `<type>` и `<docstring>`, поскольку они могут задаваться пользователем, и графическому интерфейсу эта информация необходима для правильного отображения.

Допустимы следующе *типы*:

- `BOOL`: булево значение вкл-выкл. В интерфейсе показывается флажком.
- `FILEPATH`: путь к файлу на диске. В интерфейсе открывается диалоговое окно выбора файла;
- `PATH`: путь к каталогу на диске. В интерфейсе открывается диалоговое окно выбора каталога;
- `STRING`: текстовая строка. В интерфейсе предлагается текстовое поле, но его можно заменить выпадающим списком, вызвав команду `set_property(CACHE <variable> STRINGS <values>)`;
- `INTERNAL`: текстовая строка. В интерфейсе переменные такого типа не показываются. Их можно использовать для хранения значений между прогонами. Этот тип неявно подразумевает наличие ключевого слова `FORCE`.

Значение `<doctring>` – просто метка, которая отображается в интерфейсе рядом с полем, чтобы предоставить дополнительные сведения пользователю. Она необходима даже для типа `INTERNAL`.

При задании кешированных переменных в коде действуют в основном те же правила, что для переменных окружения, – значения перезаписываются только для текущего выполнения CMake. Однако если переменной нет

в файле кеша или задан необязательный аргумент FORCE, то значение будет сохранено:

```
set(FOO "BAR" CACHE STRING "interesting value" FORCE)
```

Как и C++, CMake поддерживает области видимости переменных, хотя реализованы они специфическим образом.

Как правильно использовать области видимости переменных в CMake

Область видимости переменной – пожалуй, самая странная концепция в языке CMake. Быть может, это связано с тем, что мы привыкли к тому, как она реализована в языках общего назначения. Мы хотим объяснить это как можно раньше, потому что неправильное понимание областей действия часто является источником ошибок, которые трудно найти и исправить.

Поясним, что область видимости переменной – общая идея, призванная разделить уровни абстракции в программе. Области видимости вкладываются друг в друга, образуя дерево. Самая внешняя (корневая) область видимости называется **глобальной**. Любую область видимости можно назвать **локальной**, чтобы подчеркнуть, что речь идет о текущей или обсуждаемой в данный момент области. Области видимости создают границы между переменными, так что из *вложенной области видимости* можно обратиться к переменным, определенным в *объемлющей* области, но не наоборот.

В CMake есть два вида областей видимости:

- **файловая:** используется, когда блоки и пользовательские функции применяются внутри файла;
- **каталожная:** используется, когда команда add_subdirectory() вызывается для выполнения другого спискового файла CMakeLists.txt во вложенном каталоге.



Условные блоки, блоки циклов и макросы не порождают отдельных областей видимости.

Так что же такого необычного в областях видимости, реализованных в CMake? При создании *вложенной области видимости* CMake просто заполняет ее копиями всех переменных из *объемлющей области видимости*. Последующие команды воздействуют на эти копии. Но по завершении выполнения во *вложенной области* все копии удаляются, и восстанавливаются оригинальные переменные из *объемлющей области*.

Принцип работы областей видимости в CMake имеет интересные последствия, которые в других языках не наблюдаются. Если при выполнении во

вложенной области видимости вы сотрете (`unset()`) переменную, созданную в объемлющей области, то она исчезнет, но только в текущей вложенной области, потому что эта переменная является локальной копией. Если вы теперь сошлетесь на эту переменную, то CMake решит, что она не определена, проигнорирует объемлющие области видимости и продолжит поиск среди кешированных переменных (которые рассматриваются отдельно). Это необычно и может стать причиной ошибок.

Файловые области видимости открываются командами `block()` и `function()` (но не `macro()`) и закрываются командами `endblock()` и `endfunction()` соответственно. Мы будем рассматривать функции в разделе «Определения команд» ниже в этой главе. А пока посмотрим, как область видимости работает на практике в случае более простой команды `block()` (появившейся в версии CMake 3.25).

Рассмотрим следующий пример:

ch02/04-scope/scope.cmake

```
cmake_minimum_required(VERSION 3.26)

set(V 1)
message("> Global: ${V}")
block() # внешний блок
  message(" > Outer: ${V}")
  set(V 2)
  block() # внутренний блок
    message(" > Inner: ${V}")
    set(V 3)
    message(" < Inner: ${V}")
  endblock()
  message(" < Outer: ${V}")
endblock()
message("< Global: ${V}")
```

В начале мы присвоили переменной `V` значение `1` в *глобальной области видимости*. Сразу после входа во внешний и внутренний блок мы изменяем значение на `2` и `3` соответственно. Мы также печатаем переменную при входе и выходе из каждой области видимости:

```
> Global: 1
 > Outer: 1
  > Inner: 2
  < Inner: 3
 < Outer: 2
< Global: 1
```

Как было объяснено выше, при входе во вложенную область видимости значения переменных временно копируются из объемлющей области видимости, но при выходе восстанавливаются прежние значения. Это видно в последних двух строках вывода.

Команда `block()` может также распространять значения в объемлющие области видимости (в C++ это делается по умолчанию), но это должно быть указано явно с помощью ключевого слова `PROPAGATE`. Если бы мы включили распространение для внутреннего блока командой `block(PROPAGATE V)`, то вывод выглядел бы так:

```
> Global: 1
 > Outer: 1
  > Inner: 2
  < Inner: 3
 < Outer: 3
< Global: 1
```

Отметим, что мы воздействовали на область видимости внешнего блока, но не на глобальную область видимости.

Еще один метод модификации переменной в объемлющей области видимости – задать флаг `PARENT_SCOPE` в командах `set()` и `unset()`:

```
set(MyVariable "New Value" PARENT_SCOPE)
unset(MyVariable PARENT_SCOPE)
```

Но этот обходной путь имеет ограничения – он не позволяет получить доступ к переменным, отстоящим более чем один уровень выше. Также отметим, что использование флага `PARENT_SCOPE` не изменяет переменные к текущей области видимости.

Теперь, познакомившись с тем, как обрабатываются простые переменные, рассмотрим один частный случай: поскольку все переменные хранятся в строковом виде, CMake должна творчески подходить к работе с более сложными структурами, в частности со *списками*.

Списки

Для хранения **списка** CMake конкатенирует все элементы в строку, используя в качестве разделителя точку с запятой: `a;list;of;5;elements`. Точку с запятой, встречающуюся в элементе списка, можно экранировать обратной косой чертой: `a\;single\;element`.

Чтобы создать список, используется команда `set()`:

```
set(myList a list of five elements)
```

В силу способа хранения списков следующие команды делают в точности одно и то же:

```
set(myList "a;list;of;five;elements")
set(myList a list "of;five;elements")
```

CMake автоматически распаковывает списки в аргументах без кавычек. Передавая незакавыченную ссылку на `myList`, мы по сути дела увеличиваем число аргументов команды:

```
message("the list is:" ${myList})
```

Команда `message()` получит шесть аргументов: `"the list is:"`, `"a"`, `"list"`, `"of"`, `"five"` и `"elements"`. Это может иметь непредвиденные последствия, потому что вывод будет печататься без дополнительных пробелов между аргументами:

```
the list is:alistoffiveelements
```

Как видите, это очень простой механизм, и использовать его следует осторожно.

CMake предлагает команду `list()`, у которой имеется несколько подкоманд для чтения, поиска, изменения и упорядочения списков. Приведем их краткий перечень:

```
list(LENGTH <list> <out-var>)
list(GET <list> <element index> [<index> ...] <out-var>)
list(JOIN <list> <glue> <out-var>)
list(SUBLIST <list> <begin> <length> <out-var>)
list(FIND <list> <value> <out-var>)
list(APPEND <list> [<element>...])
list(FILTER <list> {INCLUDE | EXCLUDE} REGEX <regex>)
list(INSERT <list> <index> [<element>...])
list(POP_BACK <list> [<out-var>...])
list(POP_FRONT <list> [<out-var>...])
list(PREPEND <list> [<element>...])
list(REMOVE_ITEM <list> <value>...)
list(REMOVE_AT <list> <index>...)
list(REMOVE_DUPLICATES <list>)
list(TRANSFORM <list> <ACTION> [...])
list(REVERSE <list>)
list(SORT <list> [...])
```

Необходимость использовать в проектах списки возникает редко. Но если вы все же окажетесь в такой ситуации, то обратитесь к более полной справке по команде `list()` в приложении.

Теперь мы знаем, как работать со списками и разного рода переменными, и можем перейти к управлению потоком выполнения и управляющим конструкциям, имеющимся в CMake.

Управляющие конструкции в CMake

Язык CMake был бы неполон без **управляющих конструкций**! Как и все остальное, они предстают в форме команд, которые можно отнести к трем категориям: **условные блоки**, **циклы** и **определения команд**. Управляющие конструкции исполняются в скриптах и во время генерирования сборочной системы для проектов.

Условные блоки

Единственный условный блок, поддерживаемый CMake, – непритязательная команда `if()`. Все условные блоки должны заканчиваться командой `endif()` и могут иметь произвольное число команд `elseif()` и одну факультативную команду `else()`:

```
if(<condition>)
  <commands>
elseif(<condition>) # факультативный блок, может повторяться
  <commands>
else()              # факультативный блок
  <commands>
endif()
```

Как во многих других императивных языках, блок `if()-endif()` управляет тем, какие команды будут выполняться:

- если выражение `<condition>` в команде `if()` истинно, то выполняется первая ветвь;
- в противном случае CMake выполняет команды в ветви, принадлежащей первой команде `elseif()` в этом блоке, для которой условие истинно;
- если таких команд нет, то CMake проверяет, имеется ли команда `else()`, и если да, то выполняет команды в соответствующей ей части;
- если ни одно из вышеперечисленных условий не удовлетворяется, то выполнение продолжается с команды, следующей за `endif()`.

Заметим, что в условных блоках не создается локальной *области видимости*. Синтаксис выражения `<condition>` очень прост, рассмотрим его.

Синтаксис условных команд

Синтаксис команд `if()`, `elseif()` и `while()` одинаков.

Логические операторы

В условиях `if()` поддерживаются логические операторы `NOT`, `AND` и `OR`:

- `NOT <condition>`;
- `<condition> AND <condition>`;
- `<condition> OR <condition>`.

Условия могут быть вложенными и тогда должны заключаться в скобки. Как и все приличные языки, CMake учитывает порядок вычислений и начинает с самой внутренней пары скобок:

```
(<condition>) AND (<condition> OR (<condition>))
```

Вычисление строки и переменной

Исторически так сложилось (потому что синтаксис ссылки на переменную `${}` существовал не всегда), что CMake пытается вычислять *аргументы без кавычек*, как если бы это были *ссылки на переменные*. Иными словами, включение простого имени переменной (например, `QUX`) в условие эквивалентно конструкции `${QUX}`. Ниже приведен пример, содержащий подвох:

```
set(BAZ FALSE)
set(QUX "BAZ")
if(${QUX})
```

Здесь условие `if()` работает хитро – сначала вычисляется `${QUX}` и получается `BAZ`. Но поскольку это имя известной переменной, то она, в свою очередь, вычисляется, и получается строка, содержащая пять символов: `FALSE`. Строка считается *истинным* булевым значением, только если она совпадает с одной из следующих констант (без учета регистра): `ON`, `Y`, `YES`, `TRUE` или с *ненулевым числом*.

Таким образом, мы приходим к выводу, что результатом вычисления условия в примере выше является булево значение *false*.

Однако есть еще одна ловушка – что будет результатом вычисления, если команде `if()` передан аргумент без кавычек, интерпретируемый как имя переменной, содержащей, например, значение `BAR`? Рассмотрим пример:

```
set(FOO BAR)
if(FOO)
```

Согласно сказанному выше, результатом вроде должно было быть `false`, потому что строка `BAR` не совпадает ни с одной из тех, что вычисляются как `true`. Увы, это не так, поскольку CMake делает исключение, когда речь заходит о ссылках на переменные без кавычек. В отличие от аргументов в кавычках, результатом вычисления `FOO` будет не `BAR`, так что мы не получим команды `if("BAR")` (вычисление которой дало бы `false`). Вместо этого CMake вычисляет `if(FOO)` как `false`, только если `FOO` совпадает с одной из следующих констант (без учета регистра):

- `OFF`, `NO`, `FALSE`, `N`, `IGNORE` или `NOTFOUND`;
- строка, заканчивающаяся `-NOTFOUND`;
- пустая строка;
- нуль.

Поэтому если просто указать неопределенную переменную, то получим `false`:

```
if (CORGE)
```

Если же переменная была определена ранее, то результатом вычисления условия будет `true`:

```
set(CORGE "A VALUE")
if (CORGE)
```



Если вам кажется, что рекурсивное вычисление аргументов `if()` без кавычек слишком запутанное, то оберните ссылку на переменную аргументом в кавычках: `if("${CORGE}")`. В таком случае аргумент будет вычислен еще до передачи команде `if()`, и поведение станет таким же, как при вычислении строк.

Иными словами, CMake считает, что передать имя переменной команде `if()` – все равно, что спросить, верно ли, что переменная определена и имеет значение, отличное от булева *false*. Чтобы явно проверить, определена переменная или нет (не важно, какое у нее значение), можно использовать такие конструкции:

```
if(DEFINED <name>)
if(DEFINED CACHE{<name>})
if(DEFINED ENV{<name>})
```

Сравнение значений

Для сравнения значений предусмотрены следующие операторы:

`EQUAL`, `LESS`, `LESS_EQUAL`, `GREATER` и `GREATER_EQUAL`.

Обычные операторы сравнения, встречающие в других языках, – `==`, `>`, `<`, `!=` и т. д. – в CMake не работают.

Операторы можно использовать для сравнения числовых значений, например:

```
if (1 LESS 2)
```

Можно сравнивать версии программы в формате `major[.minor[.patch[.tweak]]]`, добавив префикс `VERSION_` к имени любого оператора:

```
if (1.3.4 VERSION_LESS_EQUAL 1.4)
```

Опущенные компоненты трактуются как нули, а компоненты номера версии, не являющиеся целыми числами, прерывают сравнение строк.

Для *лексикографического* сравнения строк добавьте префикс `STR` к имени оператора (обратите внимание, что знак подчеркивания отсутствует):

```
if ("A" STREQUAL "${B}")
```

Часто бывают нужны более сложные механизмы, чем обычное сравнение строк на равенство. К счастью, CMake поддерживает **регулярные выражения POSIX** (в документации по CMake упомянуты **расширенные регулярные выражения**, но ни о какой поддержке конкретных классов символов не говорится). Оператор `MATCHES` используется следующим образом:

```
<VARIABLE|STRING> MATCHES <regex>
```

Сопоставленные группы запоминаются в переменных `CMAKE_MATCH_<n>`.

Простые проверки

Мы уже упомянули одну простую проверку, `DEFINED`, но есть и другие, возвращающие `true`, если условие удовлетворяется. Можно проверить следующие условия:

- находится ли значение в списке: `<VARIABLE|STRING> IN_LIST <VARIABLE>`;
- доступна ли команда в данной версии CMake: `COMMAND <command-name>`;
- существует ли политика CMake: `POLICY <policy-id>` (этот вопрос рассматривается в главе 4);
- был ли добавлен тест в CTest командой `add_test()`: `TEST <test-name>`;
- определена ли цель сборки: `TARGET <target-name>`.

Цели сборки рассматриваются в главе 5, а пока скажем лишь, что цели – это логические единицы процесса сборки в проекте, создаваемые командами `add_executable()`, `add_library()` и `add_custom_target()`.

Исследование файловой системы

CMake предлагает много способов работы с файлами. Нам редко приходится манипулировать файлами напрямую, обычно мы предпочитаем более высокоуровневый подход. Краткий список относящихся к работе с файлами команд приведен в приложении. Но чаще всего возникает необходимость в следующих операторах (их поведение корректно определено только для абсолютных путей):

- `EXISTS <путь-к-файлу-или-каталогу>`: проверяет существование файла или каталога. Символические ссылки разрешаются (команда возвращает `true`, если существует цель символической ссылки);
- `<file1> IS_NEWER_THAN <file2>`: проверяет, какой файл был создан раньше. Возвращает `true`, если `file1` создан позже (или одновременно), чем `file2`, или если один из двух файлов не существует;
- `IS_DIRECTORY путь-к-каталогу`: проверяет, ведет ли путь к каталогу;
- `IS_SYMLINK имя-файла`: проверяет, является ли путь символической ссылкой;
- `IS_ABSOLUTE путь`: проверяет, является ли путь абсолютным.

Кроме того, начиная с версии 3.24 CMake поддерживает простое сравнение путей, которое сворачивает несколько разделителей в один, но никаких других нормализаций не выполняет:

```
if ("/a////b/c" PATH_EQUAL "/a/b/c") # возвращает true
```

О более сложных операциях с путями читайте документацию по команде `cmake_path()`.

На этом мы завершаем рассказ об условных командах и переходим к обсуждению следующей управляющей конструкции – циклов.

Циклы

С циклами в CMake все просто: имеются команды `while()` и `foreach()` для многократного выполнения одного и того же набора команд. Обе команды поддерживают механизмы управления циклом:

- команда `break()` выходит из цикла, прерывая его выполнение;
- команда `continue()` прерывает текущую итерацию и возвращается в начало цикла.

Отметим, что никакой цикл не создает локальной *области видимости*.

Цикл while()

Блок цикла открывается командой `while()` и закрывается командой `endwhile()`. Все промежуточные команды выполняются до тех пор, пока результатом вычисления выражения `<condition>` в команде `while()` является `true`. Синтаксис записи условия такой же, как в команде `if()`:

```
while(<condition>)
  <commands>
endwhile()
```

Наверное, вы понимаете, что – после введения дополнительных переменных – цикл `while` может заменить цикл `for`. Но вообще-то для этой цели лучше использовать цикл `foreach()`.

Цикл foreach()

Существует несколько вариантов блока `foreach()`, но в любом случае входящие в него команды выполняются для каждого значения из списка. Как и в любом другом блоке, имеются команды открытия и закрытия: `foreach()` и `endforeach()`.

Простейшая форма `foreach()` имитирует цикл `for` в C++:

```
foreach(<loop_var> RANGE <max>)
  <commands>
endforeach()
```

CMake перебирает значения от `0` до `<max>` (включительно). Если необходим более точный контроль, то можно использовать второй вариант: с заданием

<min>, <max> и факультативно <step>. Все аргументы должны быть неотрицательными целыми числами, и <min> должно быть меньше <max>:

```
foreach(<loop_var> RANGE <min> <max> [<step>])
```

Однако в полной мере foreach() раскрывается при работе со списками:

```
foreach(<loop_variable> IN [LISTS <lists>] [ITEMS <items>])
```

CMake извлекает элементы из одной или более списковых переменных <lists>, а также из списка значений <items>, определенного прямо внутри команды. Эти элементы по очереди помещаются в переменную <loop variable>. Затем для каждого элемента выполняются все команды, принадлежащие блоку. Вы можете задать только списки, только значения или то и другое вместе:

ch02/06-loops/foreach.cmake

```
set(MyList 1 2 3)
foreach(VAR IN LISTS MyList ITEMS e f)
  message(${VAR})
endforeach()
```

Этот код напечатает:

```
1
2
3
e
f
```

Или можно использовать сокращенный вариант (опустив ключевое слово IN) – будет напечатано то же самое:

```
foreach(VAR 1 2 3 e f)
```

Начиная с версии 3.17 foreach() умеет сшивать списки (ZIP_LISTS):

```
foreach(<loop_var>... IN ZIP_LISTS <lists>)
```

Это означает, что команда обходит сразу несколько списков и выполняет действия над соответственными элементами, имеющими одинаковые индексы. Рассмотрим пример:

ch02/06-loops/foreach.cmake

```
set(L1 "one;two;three;four")
set(L2 "1;2;3;4;5")
```

```
foreach(num IN ZIP_LISTS L1 L2)
  message("word=${num_0}, num=${num_1}")
endforeach()
```

CMake создает переменные num_<N> для каждого из указанных списков и помещает в них соответственные элементы.

Можно передать несколько имен переменных (по одной для каждого списка), тогда соответственные элементы будут помещаться в них:

```
foreach(word num IN ZIP_LISTS L1 L2)
  message("word=${word}, num=${num}")
```

В обоих случаях будет напечатан одинаковый результат:

```
word=one, num=1
word=two, num=2
word=three, num=3
word=four, num=4
```

Если длины списков различны, то переменные из более коротких списков будут пусты.

Отметим, что начиная с версии 3.21 переменные цикла foreach() ограничены локальной областью видимости цикла.

Определения команд

Определить собственную команду можно двумя способами: использовать команду macro() или function(). Объяснить различия между ними проще всего, сравнив макросы препроцессора в стиле C и настоящие функции C++.

Команда macro() больше похожа на инструкцию поиска и замены, чем на вызов подпрограммы, как в случае function(). В отличие от функций, макросы не создают записи в стеке вызовов. Это означает, что вызов return() в макросе вернет управление в точку на один уровень выше, чем если бы это была функция (и, возможно, остановит выполнение, если мы уже находимся на самом верхнем уровне).

Команда function() создает *локальную область видимости* для своих переменных, в отличие от macro(), которая работает в области видимости вызывающей стороны. Это может приводить к странным результатам, которые мы обсудим в следующем разделе.

Оба метода определения команд допускают передачу именованных аргументов, к которым можно обращаться в локальной области видимости определяемой команды. Кроме того, CMake предоставляет следующие переменные для доступа к значениям, относящимся к вызову:

- ${ARGC}: количество аргументов;
- ${ARGV}: список всех аргументов;

- `${ARGV<index>}`: значение аргумента с указанным индексом (нумерация начинается с 0) вне зависимости от того, ожидался такой аргумент или нет;
- `${ARGN}`: список анонимных аргументов, переданных вызывающей стороной после последнего ожидаемого аргумента.

Доступ к числовому аргументу с индексом, большим или равным `ARGC`, считается неопределенным поведением. Об обработке более сложных ситуаций (обычно когда число аргументов неизвестно) читайте в документации по команде `cmake_parse_arguments()`. Если вы решите определить команду с именованными аргументами, то при каждом вызове передавайте их все, иначе возникнет ошибка.

Макросы

Макрос определяется, как любой другой блок:

```
macro(<имя> [<аргумент>…])
  <команды>
endmacro()
```

После этого объявления мы можем выполнить макрос, вызвав его по имени (вызовы функций нечувствительны к регистру).

Как мы знаем, макросы не создают ни записи в стеке вызовов, ни *области видимости переменных*. В следующем примере иллюстрируются некоторые проблемы, возникающие в макросах из-за такого поведения.

ch02/08-definitions/macro.cmake

```
macro(MyMacro myVar)
  set(myVar "new value")
  message("argument: ${myVar}")
endmacro()
set(myVar "first value")
message("myVar is now: ${myVar}")
MyMacro("called value")
message("myVar is now: ${myVar}")
```

Вот что печатает этот скрипт:

```
$ cmake -P ch02/08-definitions/macro.cmake
myVar is now: first value
argument: called value
myVar is now: new value
```

Что случилось? Хотя мы явно присвоили `myVar` новое значение `new value`, это не повлияло на вывод `message("argument: ${myVar}")`! А дело в том, что аргументы, переданные макросу, рассматриваются не как настоящие переменные, а как константные инструкции поиска и замены.

С другой стороны, значение переменной myVar в глобальной области видимости было изменено с first value на new value. Такое поведение является *побочным эффектом* и считается дурной практикой, поскольку невозможно сказать, какие глобальные переменные будут изменены макросом, не прочитав его код. Рекомендуется всюду, где возможно, использовать функции, поскольку это поможет предотвратить много проблем.

Функции

Для объявления функции используется следующий синтаксис:

```
function(<имя> [<аргумент>...])
  <команды>
endfunction()
```

У функции должно быть имя и факультативный список имен ожидаемых аргументов. Как уже было сказано, функция создает свою *область видимости переменных*. Мы можем вызвать команду set(), передав ей один из именованных аргументов функции, но изменение будет иметь силу лишь внутри функции (если только не задан флаг PARENT_SCOPE, который обсуждался в разделе «Как правильно использовать области видимости переменных в CMake»).

К функциям применяются правила стека вызовов, что позволяет возвращать управление вызывающей стороне командой return(). Начиная с версии CMake 3.25 команда return() допускает факультативное ключевое слово PROPAGATE, за которым следует список имен переменных. Его цель такая же, как у команды block(), – передать значения указанных переменных из *локальной области видимости* в область видимости вызывающей стороны.

CMake устанавливает значения следующих переменных для каждой функции (начиная с версии 3.17):

- CMAKE_CURRENT_FUNCTION;
- CMAKE_CURRENT_FUNCTION_LIST_DIR;
- CMAKE_CURRENT_FUNCTION_LIST_FILE;
- CMAKE_CURRENT_FUNCTION_LIST_LINE.

Рассмотрим их применение на практике.

ch02/08-definitions/function.cmake

```
function(MyFunction FirstArg)
  message("Function: ${CMAKE_CURRENT_FUNCTION}")
  message("File: ${CMAKE_CURRENT_FUNCTION_LIST_FILE}")
  message("FirstArg: ${FirstArg}")
  set(FirstArg "new value")
  message("FirstArg again: ${FirstArg}")
  message("ARGV0: ${ARGV0} ARGV1: ${ARGV1} ARGC: ${ARGC}")
endfunction()
set(FirstArg "first value")
```

```
MyFunction("Value1" "Value2")
message("FirstArg in global scope: ${FirstArg}")
```

При выполнении этого скрипта командой `cmake -P function.cmake` будет напечатан следующий результат:

```
Function: MyFunction
File: /root/examples/ch02/08-definitions/function.cmake
FirstArg: Value1
FirstArg again: new value
ARGV0: Value1 ARGV1: Value2 ARGC: 2
FirstArg in global scope: first value
```

Как видим, общий синтаксис и сама идея функций очень похожи на макросы, но возможностей для тонких ошибок они предоставляют меньше.

Процедурная парадигма в CMake

Предположим, что мы хотим написать на CMake код по аналогии с тем, как пишем программу на C++. Мы создадим списковый файл `CMakeLists.txt`, который будет вызывать три определенные нами команды, которые сами могут вызывать команды.

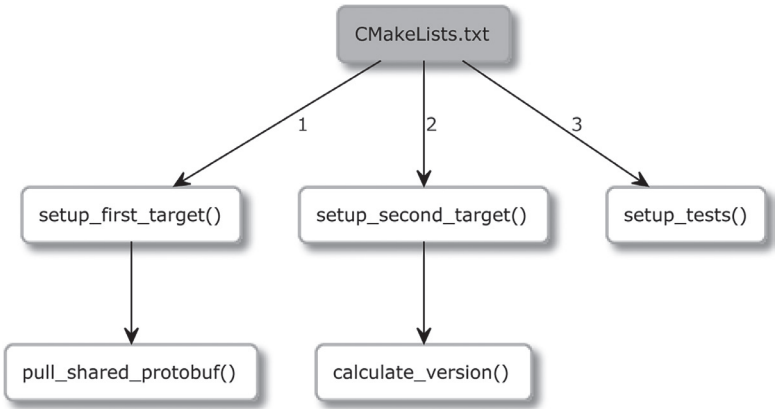


Рис. 2.3. Граф процедурных вызовов

В CMake кодирование в процедурном стиле может оказаться проблематичным, потому что определения команд должны предшествовать их вызову. Никак по-другому синтаксический анализатор CMake не работает. Ваш код будет выглядеть примерно так:

```
cmake_minimum_required(...)
project(Procedural)
```

```
# Определения
function(pull_shared_protobuf)
function(setup_first_target)
function(calculate_version)
function(setup_second_target)
function(setup_tests)

# Вызовы
setup_first_target()
setup_second_target()
setup_tests()
```

Кошмар! Все шиворот-навыворот! Будет очень трудно понять такой код, потому что самый нижний уровень абстракции находится в начале файла. В правильно структурированной программе самые общие шаги помещают в первую подпрограмму, потом идут чуть более детальные подпрограммы, а самые детализированные шаги оказываются в конце файла.

У этой проблемы есть решения, например переместить определения команд в другие файлы и разделить области видимости между каталогами (каталоги с областями видимости будут подробно рассмотрены в главе 4). Но есть и еще один простой и элегантный подход – объявить макрос точки входа в начале файла и вызвать его в самом конце файла:

```
macro(main)
  first_step()
  second_step()
  third_step()
endmacro()

function(first_step)
function(second_step)
function(third_step)

main()
```

При таком подходе код развивается от общего к частному, а поскольку мы не вызываем макрос `main()` до самого конца, CMake не будет жаловаться на выполнение еще не определенных команд.

Почему в этом случае используется макрос, а не функция? Хорошо иметь неограниченный доступ к глобальным переменным, а так как мы не передаем никаких аргументов `main()`, то и нет нужды беспокоиться об обычных подводных камнях.

Простой пример применения этой идеи вы найдете в списковом файле `ch02/09-procedural/CMakeLists.txt` в репозитории этой книги на GitHub.

К вопросу о соглашениях об именовании

Известно, что выбор имен – большая проблема в разработке программного обеспечения, но решение должно быть легко читать, понимать, а значит, и сопровождать. При написании скриптов и проектов на языке CMake мы должны придерживаться тех же правил создания *чистого кода*, что и при разработке любой другой программы:

- остановиться на каком-то одном стиле написания имен (в сообществе CMake общепринятым стандартом является `змеиный_стиль`);
- использовать короткие, но осмысленные имена (избегайте таких имен, как `func()` и `f()`);
- избегать игры слов и зауми при выборе имен;
- выбирать легко произносимые и распознаваемые имена, не требующие умственного напряжения.

Теперь, зная, как правильно вызывать команды, рассмотрим те, что пригодятся нам в самом начале.

Наиболее употребительные команды

CMake предлагает много команд, позволяющих работать с переменными и окружением. Некоторые из них подробно рассмотрены в приложении, например `list()`, `string()` и `file()`. Другие, скажем `find_file()`, `find_package()` и `find_path()`, лучше отложить до глав, где будут рассматриваться соответствующие темы. А в этом разделе мы дадим краткий обзор команд, полезных в большинстве ситуаций:

- `message()`;
- `include()`;
- `include_guard()`;
- `file()`;
- `execute_process()`.

Команда message()

Мы хорошо знаем и любим безотказную команду `message()`, которая печатает текст на стандартный вывод. Однако в ней скрыто куда больше, чем кажется на первый взгляд. С помощью аргумента `MODE` мы сможем настроить поведение команды: `message(<РЕЖИМ> "text to print")`.

Распознаются следующие режимы:

- `FATAL_ERROR`: обработка и генерирование прекращаются;
- `SEND_ERROR`: обработка продолжается, но без генерирования;
- `WARNING`: обработка продолжается;

- AUTHOR_WARNING: предупреждение CMake. Обработка продолжается;
- DEPRECATION: работает в соответствии с тем, какая из переменных CMAKE_ERROR_DEPRECATED и CMAKE_WARN_DEPRECATED установлена;
- NOTICE или режим опущен (по умолчанию): печатает сообщение на stderr, чтобы привлечь внимание пользователя;
- STATUS: обработка продолжается, этот режим рекомендуется использовать для основных информационных сообщений пользователю;
- VERBOSE: обработка продолжается, этот режим следует использовать для печати более подробной информации, которая обычно необязательна;
- DEBUG: обработка продолжается, сообщение должно содержать детали, которые могут быть полезны для отладки возникшей в проекте проблемы;
- TRACE: обработка продолжается, этот режим рекомендуется использовать для печати сообщений во время разработки проекта. Обычно такие сообщения удаляются перед публикацией проекта.

Выбор подходящего режима – дополнительная работа, но это может сэкономить время на отладке за счет цветового кодирования печатаемого текста в зависимости от уровня серьезности (начиная с версии 3.21) или даже остановить выполнение в случае неисправимой ошибки.

ch02/10-useful/message_error.cmake

```
message(FATAL_ERROR "Stop processing")
message("This won't be printed.")
```

Сообщения печатаются или не печатаются в зависимости от текущего уровня протоколирования (по умолчанию STATUS). Как его изменить, мы говорили в разделе «Параметры для отладки и трассировки» предыдущей главы.

В главе 1 я упоминал применение при отладке переменной CMAKE_MESSAGE_CONTEXT, а теперь пришло время рассмотреть этот вопрос глубже. Ведь сейчас мы уже знаем о трех важных понятиях, относящихся к этому предмету: списки, области видимости и функции.

Когда отладка сталкивается с трудностями, может быть чрезвычайно полезно понять, в каком контексте выдано сообщение. Рассмотрим следующий вывод, где сообщения, напечатанные из функции foo, имеют соответствующий префикс:

```
$ cmake -P message_context.cmake --log-context
[top] Before `foo`
[top.foo] foo message
[top] After `foo`
```

Вот как это работает:

ch02/10-useful/message_context.cmake

```
function(foo)
  list(APPEND CMAKE_MESSAGE_CONTEXT "foo")
  message("foo message")
```

```
endfunction()

list(APPEND CMAKE_MESSAGE_CONTEXT "top")
message("Before `foo`")
foo()
message("After `foo`")
```

Рассмотрим этот код по частям.

1. Сначала мы добавляем `top` в конец отслеживающей контекст переменной `CMAKE_MESSAGE_CONTEXT`, затем печатаем начальное сообщение `Before` `` `foo` ``, в которое включается соответствующий префикс `[top]`.
2. При входе в функцию `foo()` мы добавляем в список новый контекст `foo` и печатаем другое сообщение, в которое включается расширенный префикс `[top.foo]`.
3. Наконец, после выхода из функции мы печатаем сообщение `After` `` `foo` ``. Оно содержит только первоначальный префикс `[foo]`. Почему? Из-за правил *области видимости переменной*: измененная переменная `CMAKE_MESSAGE_CONTEXT` существует только до конца области видимости функции, а затем восстанавливается ее первоначальное значение.

Еще один полезный прием при работе с `message()` – добавить отступ в список `CMAKE_MESSAGE_INDENT` (точно так же, как в случае `CMAKE_MESSAGE_CONTEXT`):

```
list(APPEND CMAKE_MESSAGE_INDENT " ")
message("Before `foo`")
foo()
message("After `foo`")
```

Тогда вывод скрипта будет немного проще:

```
Before `foo`
  foo message
After `foo`
```

Поскольку в CMake нет ни настоящего отладчика с точками останова, ни других инструментов, умение помещать информативные сообщения в журнал становится очень полезным, когда что-то идет не по плану.

Команда include()

Разнесение кода по разным файлам как средство ограничения хаоса – очень полезная вещь. Затем мы можем сослаться на них из родительского спискового файла, вызвав функцию `include()`:

```
include(<file|module> [OPTIONAL] [RESULT_VARIABLE <var>])
```

Если задано имя файла (путь с расширением .cmake), то CMake попытается его открыть и выполнить.

Заметим, что при этом не создается отдельная вложенная *область видимости переменных*, т. е. любые изменения переменных, произведенные во включенном файле, видны вызывающей стороне.

CMake сочтет ошибкой отсутствие файла, если мы явно не укажем, что файл факультативный, добавив ключевое слово OPTIONAL. Если нужно узнать, успешно ли завершилась include(), то можно добавить ключевое слово RESULT_VARIABLE, сопровождаемое именем переменной. В эту переменную будет помещен полный путь к включенному файлу в случае успеха и значение NOTFOUND, если файл не найден.

При выполнении в режиме скрипта все относительные пути разрешаются относительно текущего рабочего каталога.

Чтобы искать относительно самого скрипта, задайте абсолютный путь:

```
include("${CMAKE_CURRENT_LIST_DIR}/<filename>.cmake")
```

Если мы не зададим путь, но зададим имя модуля (с расширением .cmake или без него), то CMake попытается найти и включить модуль. CMake ищет файл с именем <module>.cmake сначала на пути CMAKE_MODULE_PATH, а затем в каталоге модулей CMake.

По мере того как CMake обходит дерево исходного кода и включает различные списковые файлы, устанавливаются следующие переменные: CMAKE_CURRENT_LIST_DIR, CMAKE_CURRENT_LIST_FILE, CMAKE_PARENT_LIST_FILE и CMAKE_CURRENT_LIST_LINE.

Команда include_guard()

Иногда при включении файлов, содержащих побочные эффекты, требуется сделать так, чтобы они включались только один раз. Для этого предназначена функция include_guard([DIRECTORY|GLOBAL]).

Поместите include_guard() в начало включаемого файла. Когда CMake встречает эту команду впервые, она отмечает данный факт в текущей области видимости. Если файл будет включен снова (быть может, потому что вы не контролируете все файлы в составе проекта), обработка на этом остановится.

Если мы хотим защититься от включения в не связанных между собой областях видимости функций, в которых нет общих переменных, то должны передать аргумент DIRECTORY или GLOBAL. Ключевое слово DIRECTORY применяет защиту к текущему каталогу и его подкаталогам, а GLOBAL – ко всей сборке.

Команда file()

Чтобы вы лучше представляли себе, на что способны скрипты CMake, рассмотрим наиболее полезные варианты команды работы с файлами:

```
file(READ <filename> <out-var> [...])
file({WRITE | APPEND} <filename> <content>...)
file(DOWNLOAD <url> [<file>] [...])
```

В двух словах, команда `file()` позволяет читать, записывать и передавать файлы, а также работать с файловой системой, блокировками файлов, путями и архивами – и все это системно-независимым образом. Дополнительные сведения см. в приложении.

Команда execute_process()

Время от времени приходится прибегать к внешним средствам, имеющимся в системе (в конце концов, CMake – это прежде всего генератор сборочных систем). CMake предлагает команду для этой цели: `execute_process()` умеет выполнять другие процессы и собирать их вывод. Эта команда особенно хороша для скриптов, хотя может использоваться также в проектах – **но только на этапе конфигурирования**. Вот как она вызывается:

```
execute_process(COMMAND <cmd1> [<arguments>]... [OPTIONS])
```

CMake пользуется API операционной системы для создания дочернего процесса (поэтому операторы оболочки – `&&`, `||` и `>` – работать не будут). Однако можно сцеплять команды, подавая вывод одной на ввод другой, – нужно просто несколько раз задать аргументы `COMMAND <cmd> <arguments>`.

Факультативный аргумент `TIMEOUT <seconds>` позволяет завершить процесс, если он не уложился в отведенное время. При желании можете также задать рабочий каталог с помощью параметра `WORKING_DIRECTORY <directory>`.

Коды выхода всех задач можно собрать в список, задав аргумент `RESULTS_VARIABLE <variable>`. Если вас интересует только результат последней выполненной команды, то воспользуйтесь формой `RESULT_VARIABLE <variable>`.

Для запоминания вывода CMake предлагает два аргумента: `OUTPUT_VARIABLE` и `ERROR_VARIABLE` (используются одинаково). Если вы хотите объединить выводы на `stdout` и `stderr`, то укажите одну и ту же переменную в обоих аргументах.

Напомним, что, создавая проекты, которыми будут пользоваться другие люди, следует проверять, что команда, которую вы планируете включить, доступна на всех поддерживаемых вами платформах.

Резюме

Эта глава открыла путь к программированию на языке CMake – теперь вы знаете, как писать информативные комментарии и использовать встроенные команды, и понимаете, как задавать их аргументы. Этих знаний достаточ-

но, чтобы разобраться в необычном синтаксисе списковых файлов CMake, которые вы можете встретить в проектах, созданных другими людьми. Мы рассмотрели переменные разных видов в CMake (*нормальные*, *кешированные* и *окружения*) – как к ним обращаться, как присваивать и стирать их значения. Мы обсудили, как работают *области видимости переменных* на уровне файла и каталога, как их создать, какие проблемы при этом могут возникнуть и как их решать. Мы также поговорили о списках и управляющих конструкциях. Мы изучили синтаксис условий, логические операторы, вычисление аргументов без кавычек, а также строк и переменных в условиях. Мы узнали, как сравнивать значения, выполнять простые проверки и опрашивать состояние файлов в системе. Зная все это, мы можем писать условные блоки и циклы `while`; попутно мы также обсудили синтаксис циклов `foreach`.

Понимая, как определять свои команды с помощью макросов и функций, мы, без сомнения, сможем писать более чистый процедурный код. Мы также обсудили стратегии улучшения структуры кода и создания понятных и информативных имен.

Наконец, мы познакомились с командой `message()` и ее уровнями протоколирования. Мы также узнали, как разнести код по нескольким списковым файлам и включить их в главный файл, и рассмотрели ряд других полезных команд. Теперь мы готовы перейти к следующей главе, посвященной использованию CMake в популярных IDE.

Для дополнительного чтения

Дополнительные сведения по вопросам, рассмотренным в этой главе, можно найти в следующих источниках:

- «Clean Code: A Handbook of Agile Software Craftsmanship» (Robert C. Martin): https://amzn.to/3cm69DD;
- «Refactoring: Improving the Design of Existing Code» (Martin Fowler): https://amzn.to/3cmWk8o;
- «Which comments in your code ARE GOOD?» (Rafał Świdzinski): https://youtu.be/4t9bpo0THb8;
- «What's the CMake syntax to set and use variables?» (StackOverflow): https://stackoverflow.com/questions/31037882/whats-the-cmake-syntax-to-set-and-use-variables.

3

Использование CMake в популярных IDE

Программирование – одновременно искусство и глубоко технический процесс, и, как все мы хорошо знаем, оно представляет значительные трудности. Поэтому мы хотели бы оптимизировать этот процесс настолько, насколько возможно. Не так уж много примеров ситуаций, когда достаточно повернуть ручку, как все сразу станет лучше, но использование **интегрированных сред разработки** (**IDE**) – один из таких редких случаев.

Если вы не работали с настоящей IDE раньше (или думаете, что текстовый процессор типа Emacs или Vim – лучшее, на что можно рассчитывать), то эта глава для вас. Если же вы профессиональный программист и уже знакомы с этой темой, то можете пролистать ее, чтобы узнать о лучших представителях и, быть может, задуматься о переходе либо, наоборот, удостовериться, что инструмент, с которым вы сейчас работаете, и вправду лучший.

Поскольку мы хотели сделать материал доступным для только приступающих к изучению предмета, эта глава содержит мягкое введение в выбор IDE. Мы поговорим о том, зачем она может вам понадобиться и как выбрать ту, что лучше всего отвечает вашим потребностям. Разумеется, вариантов много, но, как обычно бывает, некоторые лучше остальных. К сожалению, нет универсального рецепта, который был бы оптимальным для всех. Многие факторы влияют на производительность труда, которой можно добиться при правильном выборе инструмента. Мы обсудим некоторые соображения, которые могут быть важны, если вы работаете в организации определенного размера, и постараемся передать нюансы, не обременяя вас сложностью. Затем последует краткое введение в комплекты инструментов, и по ходу дела мы обсудим имеющиеся варианты.

После этого мы опишем отличительные свойства некоторых популярных IDE: изощренной CLion, адаптивной Visual Studio Code и мощнейшей Visual Studio.

В этой главе рассматриваются следующие основные темы:

- знакомство с IDE;
- начало работы с CLion IDE;
- начало работы с Visual Studio Code;
- начало работы с Visual Studio IDE.

Знакомство с IDE

В этом разделе мы обсудим, как IDE значительно повышают скорость разработки и качество кода. Для начала объясним, что такое IDE, для тех, кто этого не знает.

Зачем и как выбирать IDE? IDE, или интегрированная среда разработки, – это всеобъемлющий инструмент, объединяющий различные специализированные инструменты, которые упрощают процесс разработки ПО. Профессиональное создание проекта включает много шагов: проектирование, кодирование, сборка, тестирование, упаковка, выпуск и сопровождение. Каждый шаг состоит из меньших задач, и общая сложность может оказаться ошеломляющей. IDE предлагает решение, предоставляя платформу и набор инструментов, сконфигурированный ее создателями. Такая интеграция позволяет органично использовать инструменты, не настраивая их по отдельности для каждого проекта.

Сердцевину IDE составляют редактор кода, компилятор и отладчик. Они спроектированы так, чтобы обеспечить достаточную интеграцию, что позволяет отредактировать код, сразу же откомпилировать его и запустить с присоединенным отладчиком. IDE может включать комплекты инструментов сборки или давать разработчикам возможность самостоятельно выбрать компилятор и отладчик. Редакторы обычно являются центральной частью программы, но зачастую допускают расширение с помощью плагинов, например для цветового выделения, форматирования и т. д.

Более развитые IDE предлагают весьма изощренные средства, например отладку в режиме горячей перезагрузки (доступна в Visual Studio 2022, далее объясняется подробнее). Эта возможность позволяет исполнять код в отладчике, редактировать его и продолжать выполнение без перезапуска программы. Имеются также средства рефакторинга, позволяющие переименовывать символы и выносить код в отдельную функцию, и средства статического анализа для выявления ошибок еще до компиляции. IDE также предоставляют возможность работать с Git и другими системами управления версиями, что очень важно для разрешения конфликтов и не только.

Я уверен, что теперь вы понимаете, насколько полезно начать использовать IDE на ранних стадиях и стандартизировать ее применение в организации. Теперь разберемся, почем так важно выбрать IDE, отвечающую именно вашим требованиям.

Выбор IDE

Существует много редакторов кода, которые сообщество считает очень близкими к полноценной IDE. Прежде чем делать выбор, всегда рекомендуется осмотреться, особенно принимая во внимание короткие циклы выпуска современного ПО и быстрые изменения ландшафта.

Мой опыт работы в крупных компаниях показывает, что очень редко бывает так, что IDE предлагает настолько привлекательную возможность, чтобы заставить кого-то изменить своим привычкам. Привычка – вторая натура разработчика, и забывать об этом не следует. Напомним, что раз уж вы освоили какую-то IDE, она, скорее всего, останется с вами в обозримом будущем. Именно поэтому все еще можно встретить разработчиков, использующих Vim (консольный текстовый редактор, выпущенный в 1991 году), дополненный кучей плагинов, делающих его таким же мощным, как более современные IDE с графическим интерфейсом. И незачем на них давить.

Есть разные причины, по которым программисты предпочитают одну IDE всем прочим; некоторые из них и вправду важны (скорость, надежность, функциональная насыщенность, полнота), тогда как другие … ну, скажем, не очень. Я хочу поделиться своим взглядом на этот выбор и надеюсь, что вы сочтете его полезным.

Выбирайте функционально насыщенную IDE

Если вы только начинаете работать над проектом, то, возможно, помышляете о простом текстовом редакторе и запуске немногих команд для сборки кода. Это, безусловно, работоспособный подход, особенно если ваша цель – разобраться в основах. Заодно вы поймете, что может испытывать начинающий без IDE.

С другой стороны, IDE были созданы не без причины. Они упрощают многочисленные задачи, решаемые разработчиками на протяжении жизненного цикла проекта, и в этом их великая ценность. Хотя поначалу голова может пойти кругом, все же выбирайте функционально насыщенную IDE, которая включает все необходимые средства. Хорошо бы, чтобы она была максимально полной, но не забывайте о цене, потому что IDE могут оказаться не по карману малым предприятиям или индивидуальным разработчикам. Необходимо соблюсти баланс между временем, потраченным на управление проектом вручную, и стоимостью полнофункциональной IDE.

Независимо от цены выбирайте IDE с сильным сообществом, которое сможет помочь при возникновении проблем. Изучите форумы сообщества и популярные сайты вопросов и ответов типа `StackOverflow.com` – важно понять, получают ли пользователи ответы на заданные вопросы. Кроме того, выбирайте IDE, которая активно разрабатывается уважаемой компанией. Вы же не хотите тратить время на программу, которая обновляется редко и может быть заброшена в будущем. Например, не так давно Atom, редактор, созданный GitHub, был предан забвению после 7 лет разработки.

Выбирайте IDE, широко поддерживаемую в вашей организации

Как ни странно, не всем разработчикам близка эта точка зрения. Возможно, вы хорошо освоили другой инструмент в университете, на предыдущей работе или при разработке своего личного проекта. Привычка, как было сказано выше, может побудить вас не обращать внимания на рекомендации компании и остаться верным своему выбору. Не поддавайтесь этому соблазну. Со временем такой выбор принесет только проблемы. Мой опыт работы в Ericsson, Amazon и Cisco показал, что только однажды идея сконфигурировать нестандартную IDE и работать с ней оказалась плодотворной. И то только потому, что мне удалось заручиться достаточной поддержкой со стороны организации, чтобы решать возникающие проблемы коллективно.

Вашей главной целью должно быть написание кода, а не борьба с неподдерживаемой IDE. Для изучения рекомендуемого ПО, возможно, придется приложить усилия, но это лучше, чем восстать против нормы (и да, Vim проиграл эту битву, пора двигаться дальше).

Не выбирайте IDE, исходя из целевой ОС и платформы

Возможно, вы думаете, что раз вы разрабатываете ПО для Linux, то нужно использовать и машину с Linux, и IDE, работающую в Linux. Но C++ – переносимый язык, т. е. он должен компилироваться и работать одинаково на любой платформе, коль скоро код написан правильно. Разумеется, могут возникнуть проблемы с библиотеками, поскольку не все они устанавливаются по умолчанию, а некоторые могут быть «заточены» под конкретную платформу.

Строго ориентироваться на целевую платформу не всегда необходимо, а иногда даже контрпродуктивно. Например, если вы ориентируетесь на старую или пользующуюся **долгосрочной поддержкой** (Long-Term Support – **LTS**) версию ОС, то, возможно, не сможете использовать последнюю версию комплекта инструментов. Если вы хотите вести разработку на платформе, отличной от целевой, то можете это сделать.

В таком случае подумайте о **кросс-компиляции** или **удаленной разработке**. Кросс-компиляция подразумевает использование специального комплекта инструментов, который позволяет компилировать на одной платформе (например, Windows), но получать готовые файлы для другой платформы (например, Linux). Этот подход широко применяется в индустрии и поддержан CMake. Альтернатива – удаленная разработка, когда вы копируете код на целевую машину и собираете его там с помощью локального комплекта инструментов. Этот метод поддерживается многими IDE и дает ряд преимуществ, которые мы обсудим в следующем разделе.

Выбирайте IDE, поддерживающую удаленную разработку

Хотя это и не должно быть главным критерием, при прочих равных условиях поддержка удаленной разработки является преимуществом. Рано или поздно даже профессиональным разработчикам встречаются проекты, ориентиро-

ванные на платформу, отличную от их привычной ОС, – из-за смены команд, проектов и даже компаний.

Если ваша любимая IDE поддерживает удаленную разработку, то можете продолжать работать с ней, воспользовавшись возможностью компилировать и отлаживать код на другой ОС и просматривать результаты в интерфейсе IDE. Основное преимущество удаленной разработки перед кросс-компиляцией – интегрированная поддержка отладчика, вследствие чего процесс оказывается чище и не нуждается в конфигурировании CMake на уровне проекта. Кроме того, компании часто предоставляют мощные удаленные компьютеры, что позволяет разработчикам использовать менее дорогие, облегченные локальные устройства.

Конечно, можно возразить, что кросс-компиляция обеспечивает лучший контроль над средой разработки, позволяя вносить временные изменения для тестирования. Не нужно тратить сетевые ресурсы на копирование кода, поддерживать низкоскоростные интернет-подключения и организовывать офлайновую работу. Однако с учетом того, что в большинстве программных разработок все равно нужен доступ к интернету для получения информации, это преимущество не выглядит убедительным. Виртуализированные среды, например Docker, позволяют запустить локальную производственную копию и настроить подключения для удаленной разработки, обеспечив безопасность, настраиваемость под нужды пользователя и возможность собирать и развертывать контейнеры.

Приведенные выше соображения в большей мере относятся к работе в крупных корпорациях, где все движется не быстро, а внедрять значительные изменения трудно. Эти соображения не отрицают возможности полноценного использования CMake, если вы решите отдать приоритет другим аспектам IDE, исходя из конкретной ситуации.

Установка комплекта инструментов

Как уже было сказано, IDE интегрирует все необходимые инструменты для упрощения и ускорения разработки ПО. Главная часть этого процесса – сборка двоичных файлов, иногда в фоновом режиме или динамически, чтобы предоставить разработчикам дополнительную информацию. Комплекты инструментов включают компиляторы, компоновщики, архиваторы, оптимизаторы, отладчики и реализации стандартной библиотеки C++. Они также могут включать другие полезные утилиты, в т. ч. `bash`, `make`, `gawk`, `grep` и т. д., используемые для сборки программ.

Некоторые IDE включают комплекты инструментов или загрузчики комплектов, другие не включают. Проще всего запустить установленную IDE и проверить, сможете ли вы откомпилировать простую тестовую программу. CMake обычно делает это по умолчанию на этапе конфигурирования, который большинство IDE выполняют как часть инициализации новых проектов. Если этот процесс завершается неудачно, то IDE или менеджер пакетов ОС могут предложить вам установить необходимые инструменты. Следуйте их указаниям, потому что этот путь обычно хорошо отработан.

Если предложение не поступило или вы хотите использовать конкретный комплект инструментов, то ниже перечислено несколько вариантов для разных платформ.

- **GNU GCC** (https://gcc.gnu.org/) для Linux, Windows (при посредстве MinGW или Cygwin), macOS и многих других платформ. GCC – один из самых популярных и широко используемых компиляторов C++, поддерживающий широкий круг платформ и архитектур.
- **Clang/LLVM** (https://clang.llvm.org/) для Linux, Windows, macOS и многих других платформ. Clang – это компилятор переднего плана для языков C, C++ и Objective-C, а на заднем плане используется LLVM.
- **Microsoft Visual Studio/MSVC** (https://visualstudio.microsoft.com/) преимущественно для Windows с кросс-платформенной поддержкой на уровне Visual Studio Code и CMake. MSVC – компилятор C++, предлагаемый Microsoft, обычно используется в составе Visual Studio IDE.
- **MinGW-w64** (http://mingw-w64.org/) для Windows. MinGW-w64 – плод развития оригинального проекта MinGW, нацеленный на предоставление улучшенной поддержки 64-разрядной Windows и новых API.
- **Apple Clang** (https://developer.apple.com/xcode/cpp/) для macOS, iOS, iPadOS, watchOS и tvOS. Версия Clang от компании Apple, оптимизированная для экосистемы оборудования и программного обеспечения Apple. Интегрирована в Xcode.
- **Cygwin** (https://www.cygwin.com/) для Windows. Cygwin обеспечивает совместимое с POSIX окружение в Windows, что позволяет использовать GCC и другие инструменты GNU.

Если вы хотите поскорее начать, не вдаваясь в детали каждого комплекта инструментов, то можете согласиться с моими предпочтениями: если IDE не предоставляет комплекта инструментов, использовать MinGW в Windows, Clang/LLVM в Linux и Apple Clang в macOS. Каждый их них хорошо приспособлен к своей основной платформе и, как правило, является лучшим из возможных.

Использование примеров из этой книги совместно с IDE

К этой книге прилагается обширная подборка примеров проектов CMake, выложенная в репозиторий на GitHub по адресу: https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E.

Естественно возникает вопрос: как воспользоваться этим репозиторием из представленных здесь IDE? Надо понимать, что учебник по созданию профессиональных проектов сам таковым не является. Это набор проектов разной степени полноты, упрощенных там, где это возможно. К сожалению

(а может быть, к счастью), IDE не проектируются для загрузки десятков проектов и удобного управления ими. Обычно все их средства сосредоточены на загрузке одного активно редактируемого проекта.

Это ставит нас в неловкое положение: осуществлять навигацию по набору примеров в IDE трудно. После загрузки набора примеров в IDE и открытия каталога примеров большинство IDE обнаружат несколько файлов `CMakeLists.txt` и попросят вас выбрать какой-то один. После этого начнется обычный процесс инициализации, будут созданы временные файлы и в конечном итоге выполнены этапы конфигурирования и генерирования с целью привести проект в состояние, когда можно произвести сборку. Как легко догадаться, все это делается только для примера, чей файл `CMakeLists.txt` был выбран. Большинство IDE предлагают способы переключаться между каталогами (проектами) в рабочем пространстве, но это может оказаться не так просто, как хотелось бы.

Если у вас возникают сложности, то есть два пути: не использовать IDE для сборки примеров (ограничившись консольными командами) или каждый раз загружать пример в новый проект. Я рекомендую первый способ, если вы в первую очередь хотите попрактиковаться в использовании команд, поскольку это может пригодиться в будущем и позволит лучше понять, что происходит за кулисами. Обычно это хороший путь для инженеров по сборке, т. к. эти знания будут использоваться часто. С другой стороны, если вы работаете с одним проектом, в основном как разработчик, сосредоточенный на бизнес-логике кода, то, пожалуй, раннее освоение IDE предпочтительнее. В любом случае выбор одного пути не мешает время от времени переходить на другой.

А теперь сделаем обзор самых популярных на сегодня IDE и посмотрим, какие вам больше подходят. Любая из них хорошо вам послужит, неважно, работаете вы в крупной корпорации или нет.

Начало работы с CLion IDE

CLion – платная кросс-платформенная IDE для Windows, macOS и Linux, разработанная компанией JetBrains. Пользование программой основано на модели подписки; за $99.00 (в начале 2024 года) вы получаете лицензию на один год для личного пользования. Крупные организации платят больше, стартапы – меньше. Студенты или авторы проектов с открытым исходным кодом могут получить лицензию бесплатно. Для тестирования ПО предлагается 30-дневный пробный период. Это единственная IDE в нашем списке, которая не предлагает бесплатно урезанную версию для сообщества. Тем не менее это солидная программа, разрабатываемая уважаемой компанией, и вполне возможно, что она стоит своих денег.

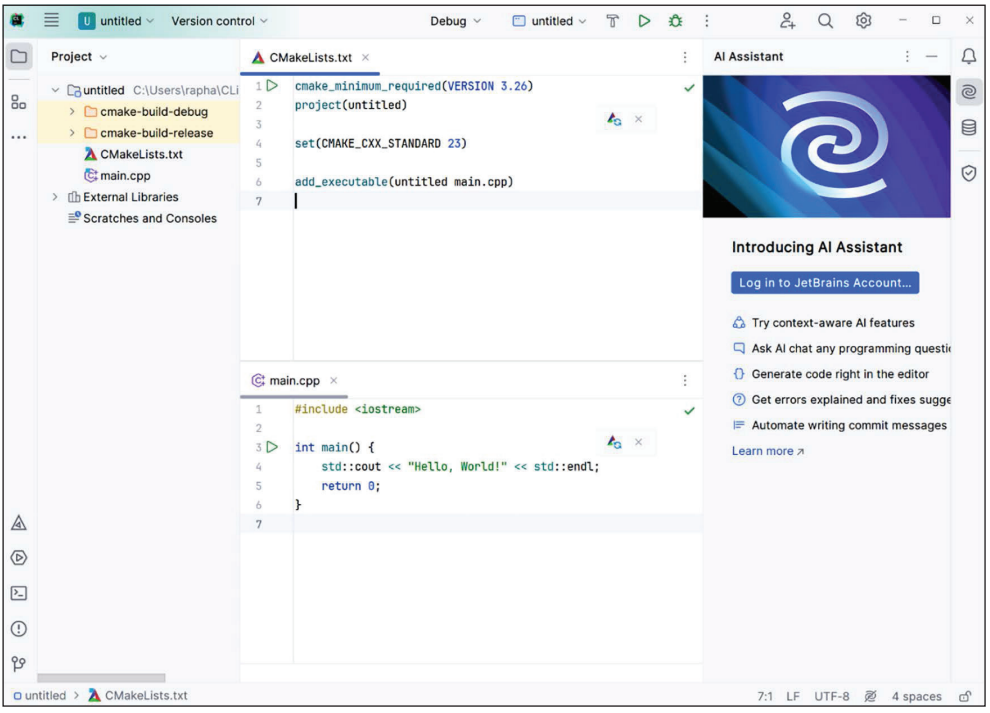


Рис. 3.1. Главное окно CLion IDE

Как видите, это полноценная IDE, готовая выполнить любую работу, которую вы ей поручите.

Почему она может вам понравиться

В отличие от альтернатив, C и C++ – единственные языки, поддерживаемые CLion. Многие возможности этой IDE специально спроектированы для поддержки именно такого окружения и отвечают менталитету программиста на C/C++. Это особенно хорошо видно при сравнении с функционалом других IDE: анализ кода, навигация по коду, интегрированный отладчик и средства рефакторинга можно найти у конкурентов, например в Visual Studio IDE. Однако они не так глубоко ориентированы на C/C++. Впрочем, измерить это объективно очень трудно.

Как бы то ни было, CMake изначально полностью интегрирована в CLion и является основным вариантом формата проекта. Альтернативы, Autotools и Makefile, также поддерживаются с самых первых версий, и в конечном итоге проекты, использующие их, могут мигрировать на CMake. Отметим, что

CLion поддерживает CTest в сочетании с многочисленными каркасами автономного тестирования и имеет специальные процессы для генерирования кода, прогона тестов, сбора и представления результатов. Можно использовать Google Test, Catch, Boost.Test и doctest.

Лично мне особенно нравится умение работать с Docker для разработки программ на C++ в контейнерах, но об этом позже. А пока посмотрим, как начать работу с CLion.

Первые шаги

Скачав CLion с официального сайта (https://www.jetbrains.com/clion), вы можете приступить к обычному процессу установки на свою платформу. В состав CLion входит типичный визуальный установщик для Windows (рис. 3.2) и macOS (рис. 3.3).

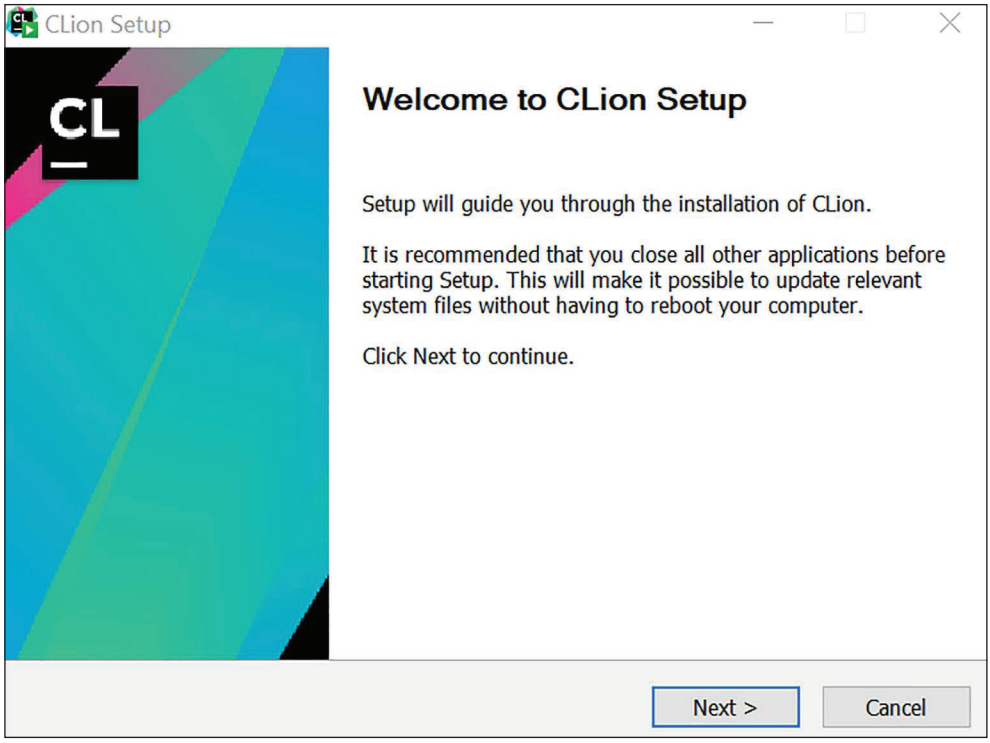


Рис. 3.2. Установка CLion для Windows

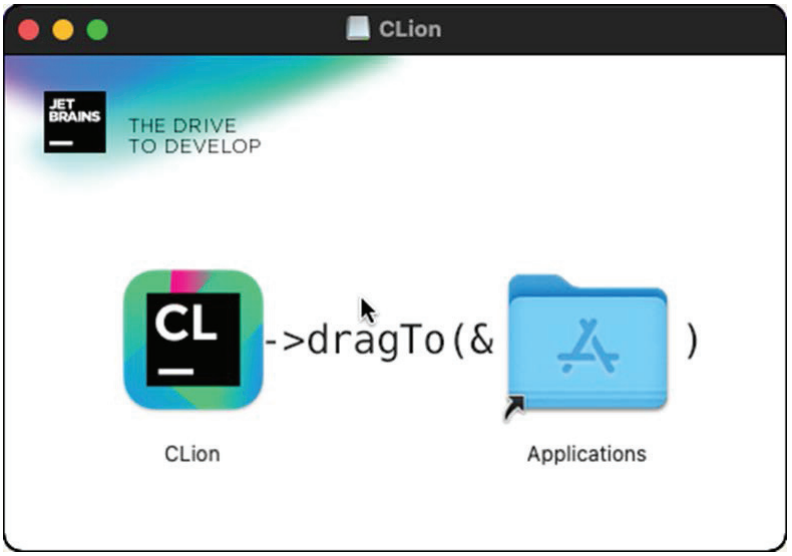


Рис. 3.3. Установка CLion для macOS

В Linux нужно распаковать скачанный архив и выполнить скрипт установки:

```
tar -xzf CLion-<version>.tar.gz
./CLion-<version>/bin/CLion.sh
```

Эти инструкции могут оказаться устаревшими, поэтому проверяйте на сайте CLion.

При первом выполнении будет предложено ввести лицензионный код или начать 30-дневный бесплатный пробный период. При выборе второго варианта вы сможете поэкспериментировать с IDE и решить, подходит ли она вам. Далее вы можете создать новый проект и выбрать конкретную версию C++. Сразу после этого CLion обнаружит доступные компиляторы и версии CMake и попробует собрать тестовый проект, чтобы удостовериться, что все настроено правильно. На некоторых платформах (например, macOS) может быть автоматически выдано предложение установить инструменты разработки. На других вам, возможно, придется настроить их самостоятельно и прописать пути в переменной окружения `PATH`.

Далее сконфигурируйте комплект инструментов под свои потребности. Инструменты конфигурируются отдельно для каждого проекта, поэтому создайте проект CMake по умолчанию. Затем перейдите в меню **Settings**/**Preferences** (*Ctrl*/*Command*+*Alt*+*S*) и выберите **Build, Execution, Deployment** > **CMake**. На этой вкладке вы можете сконфигурировать профиль сборки. Имеет смысл добавить профиль `Release` для сборки оптимизированных артефактов без отладочных символов. Для этого нажмите значок + над списком профилей. CLion создаст профиль `Release` по умолчанию. Для переключения между профилями служит выпадающий список в верхней части окна.

Теперь для компиляции и запуска программы под отладчиком нужно просто нажать клавишу *F9*. Читайте документацию по CLion, поскольку в ней есть множество полезных функций. Я же познакомлю вас с одной из моих любимых: отладчиком.

Крутая штука: накачанный отладчик

Возможности отладки в CLion смело можно назвать передовыми, они специально проектировались в расчете на C++. Я был рад обнаружить одно из самых последних добавлений, отладку CMake, которая включает много стандартных отладочных средств: пошаговое исполнение кода, точки останова, точки наблюдения, встроенное исследование значений по ходу выполнения и многое другое. Возможность исследовать переменные в различных областях видимости (кешированные, окружения и в текущей области) особенно удобна, когда что-то работает не так, как ожидалось.

Для отладки C++ в вашем распоряжении много стандартных средств, предоставляемых отладчиком **GNU Project Debugger** (**GDB**), в том числе ассемблерный код, точки останова, пошаговое исполнение, точки наблюдения и т. д., но есть также существенные дополнения. В CLion имеется параллельный просмотр стеков, позволяющий видеть все потоки на графовой диаграмме вместе с текущими стеками каждого потока. Имеется расширенное представление памяти, позволяющее видеть размещение работающей программы в памяти и модифицировать память на лету. CLion предлагает много других инструментов, помогающих понять происходящее: представление регистров, дизассемблирование кода, консоль отладчика, отладка дампа памяти, отладка произвольных исполняемых файлов и т. д.

И вишенка на торте – в CLion имеется прекрасное средство вычисления выражений (**Evaluate Expression**), которое творит чудеса и даже позволяет модифицировать объекты во время выполнения. Просто щелкните правой кнопкой мыши по строчке кода и выберите этот пункт из меню.

О CLion все; пора познакомиться с другой IDE.

Начало работы с Visual Studio Code

Visual Studio Code (**VS Code**) – бесплатная кросс-платформенная IDE, разработанная Microsoft и доступная для Windows, macOS и Linux. Не путайте ее с другим продуктом Microsoft, Visual Studio IDE (которая обычно называется с прибавлением года выпуска, например Visual Studio 2022).

VS Code ценят за ее обширную экосистему расширений и поддержку сотен языков программирования (по существующим оценкам, свыше 220!). Когда Microsoft приобрела GitHub, VS Code подавалась как своего рода замена Atom.

Общий дизайн IDE вполне современный, как видно по рис. 3.4.

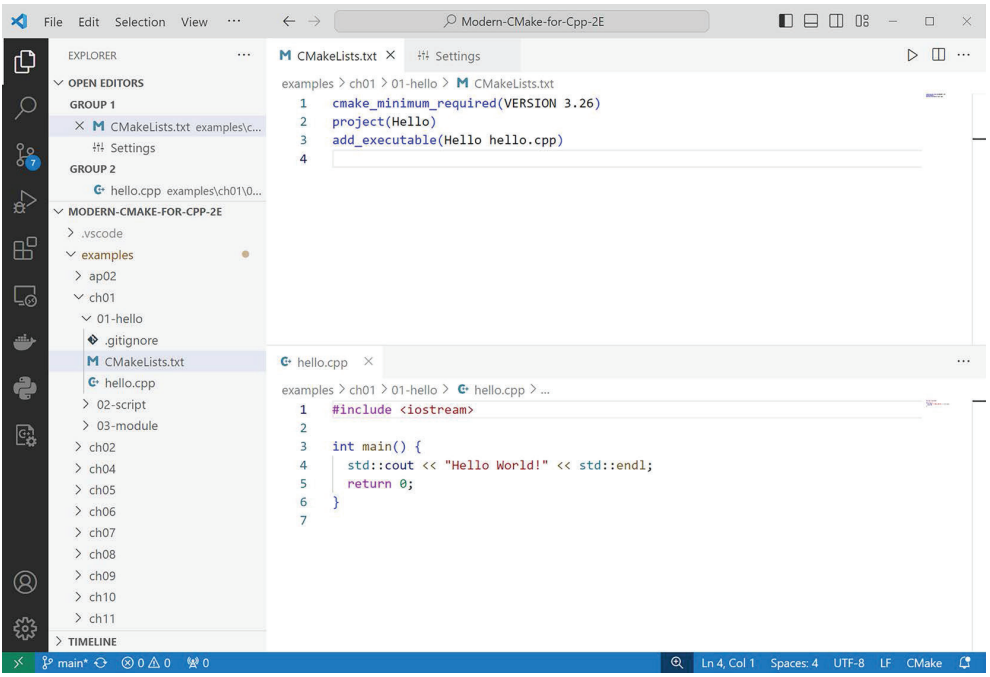


Рис. 3.4. Главное окно VS Code

Посмотрим, что же выделяет VS Code из ряда других IDE.

Почему она может вам понравиться

C++ не стоит на первом месте среди языков, поддерживаемых VSC, но близок к нему благодаря многочисленным изощренным языковым расширениям. Архитектура системы позволяет переключаться между разными языками, оставаясь в единой среде.

Изучать этот инструмент не особенно трудно, потому что большинство расширений остаются верными базовой функциональности пользовательского интерфейса, а не реализуют собственные продвинутые интерфейсы. Многие функции доступны из палитры команд (открывающейся при нажатии *F1*), которая требует ввести имя команды, а не нажимать значок или кнопку. Это разумный компромисс, позволяющий не жертвовать чистотой, скоростью и бесплатностью VSC. На самом деле эта IDE загружается так быстро, что я предпочитаю использовать ее в качестве текстового редактора общего назначения, даже когда не работаю ни над каким проектом.

Но при этом VS Code – на удивление мощная IDE благодаря огромной библиотеке по-настоящему хороших расширений, подавляющее большинство которых бесплатны. Имеются специальные расширения для C++ и CMake, и в следующем разделе мы посмотрим, как они конфигурируются.

Первые шаги

VSC можно скачать с официального сайта https://code.visualstudio.com/. Сайт предлагает весьма обширный список загрузок для Windows и macOS, а также для многих дистрибутивов Linux: Debian, Ubuntu, Red Hat, Fedora и SUSE. Установка IDE производится обычным для вашей платформы способом. Затем вы, наверное, захотите установить расширения, для чего перейдите в окно **Extensions Marketplace** (*Ctrl*/*Command*+*Shift*+*X*). Рекомендую начать со следующих:

- C/C++ от Microsoft;
- C/C++ Extension Pack от Microsoft;
- CMake от twxs;
- CMake Tools от Microsoft.

Все они поддерживают обычную подсветку кода и возможность компилировать, выполнять и отлаживать код прямо в IDE, но комплект инструментов вам, возможно, придется установить самостоятельно. Обычно VS Code предлагает установить расширения во всплывающем окне при попытке открыть соответствующий файл, так что необязательно целенаправленно искать их.

Я также рекомендую установить расширение **Remote – SSH by Microsoft**, если вы занимаетесь удаленной разработкой, поскольку оно сделает вашу работу более удобной; это расширение не только берет на себя заботу о синхронизации файлов, но и позволяет вести удаленную отладку, присоединив отладчик на удаленной машине.

Однако есть еще более интересное расширение, меняющее привычную схему работы над проектами. Рассмотрим его.

Крутая штука: контейнеры разработчика

Если вы развертываете приложение в производственной среде, все равно – путем поставки откомпилированного артефакта или выполнения процесса сборки, – то важно позаботиться о присутствии всех зависимостей. В противном случае неизбежны всяческие проблемы. Но даже если с зависимостями все в порядке, из-за различия версий или конфигураций решение может вести себя не так, как во время разработки или в промежуточной среде. Я неоднократно наблюдал такое. Пока виртуализация не вошла в наш быт, борьба с проблемами из-за различий среды была частью нашей жизни.

С появлением облегченных контейнеров типа Docker все заметно упростилось. У нас появилась возможность запускать минимальную операционную систему, содержащую ваш сервис в своем собственном изолированном пространстве. Это позволило упаковать все зависимости в контейнер, избавив разработчиков от большой головной боли.

До недавнего времени разработка внутри контейнера подразумевала ручную сборку, запуск и подключение контейнера к удаленному сеансу из IDE. Не то чтобы этот процесс был очень трудным, но он требовал ручных шагов, которые разные разработчики могли выполнять по-разному.

В последние годы Microsoft выпустила открытый стандарт под названием Dev Containers (https://containers.dev/), чтобы устранить это мелкое неудобство. Спецификация состоит в основном из файла `devcontainer.json`, который можно поместить в репозиторий проекта; он содержит инструкции IDE о том, как настраивать среду разработки в контейнере.

Чтобы воспользоваться этой возможностью, установите расширение **Dev Containers by Microsoft** и укажите в нем на репозиторий правильно подготовленного проекта. Если вас не пугают трудности переключения главного `CMakeLists.txt`, можете попробовать это на репозитории книги: git@github.com:PacktPublishing/Modern-CMake-for-Cpp-2E.git.

Могу подтвердить, что другие IDE, в частности CLion, принимают этот стандарт, так что имеет смысл действовать в соответствии с ним, если вы столкнулись с описанными выше неудобствами. А мы перейдем к следующему продукту от Microsoft.

Начало работы с Visual Studio IDE

Visual Studio (**VS**) – это IDE для Windows, разработанная Microsoft. VS была доступна и для macOS, но в августе 2024 разработка на этой платформе прекращается. Важно отличать ее от VS Code, *другой IDE* от Microsoft.

VS поставляется в трех версиях: Community, Professional и Enterprise. Версия Community бесплатна и рассчитана на компании с числом пользователей не более пяти. Более крупные компании должны уплачивать лицензионный сбор, начинающийся от 45 долларов в месяц за каждого пользователя. На рис. 3.5 показано, как выглядит версия VS Community.

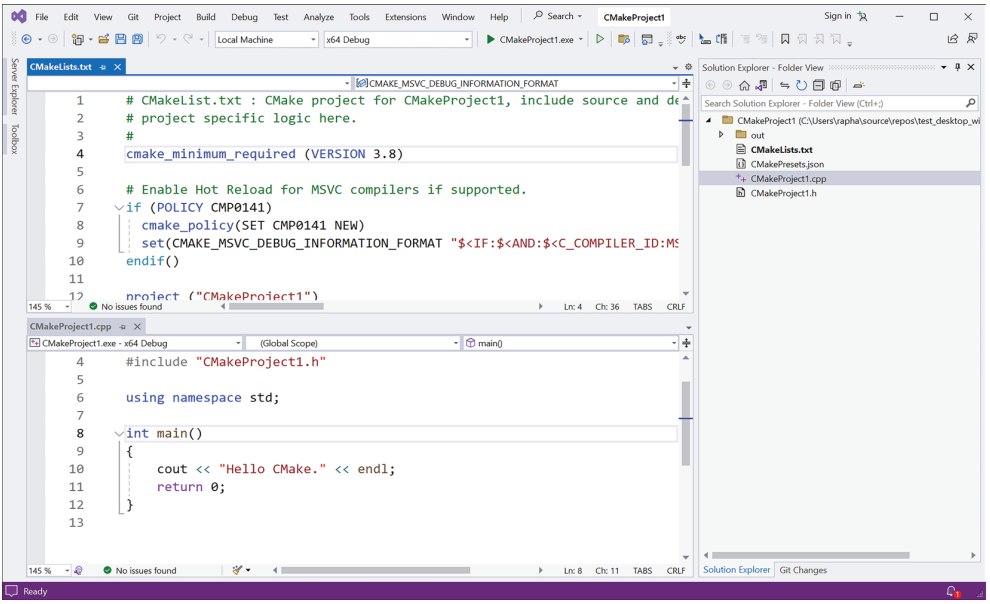


Рис. 3.5. Главное окно VS 2022

Как и в других IDE, рассмотренных в этой главе, можно установить темный фон. Рассмотрим некоторые возможности этой IDE.

Почему она может вам понравиться

Эта IDE имеет много общего с VS Code, способ работы похож, но детали куда лучше проработаны. Функциональность очень богата, часто используются мастера и различные визуальные элементы. Многие функции присутствуют изначально, а не в виде расширений (хотя имеется весьма обширный рынок пакетов, предлагающих дополнительную функциональность). Иными словами, это VSC, но гораздо более развитая.

В зависимости от версии предлагается более или менее широкий ассортимент средств тестирования: автономное, производительности, нагрузочное, ручное, Test Explorer, оценка покрытия тестами, IntelliTest и профилирование кода. В частности, профилировщик – очень полезный инструмент, включенный даже в версию Community.

Тем, кто проектирует персональные приложения для Windows, VS предлагает визуальные редакторы и большой набор компонентов. Имеется развитая поддержка платформы **Universal Windows Platform (UWP)** – стандарта UI для приложений Windows, появившаяся в версии Windows 10. Эта поддержка обеспечивает изящный современный дизайн, оптимизированный для адаптивных элементов управления, которые хорошо масштабируются на разных экранах.

Стоит также упомянуть, что хотя VS работает только в Windows, в ней можно разрабатывать проекты, ориентированные на Linux и мобильные платформы (Android и iOS). Имеется также поддержка для разработчиков игр, использующих платформенные библиотеки Windows и Unreal Engine.

Первые шаги

Эта IDE доступна только для Windows, процедура установки стандартная. Начните со скачивания установщика с сайта https://visualstudio.microsoft.com/free-developeroffers/. После запуска установщика будет предложено выбрать версию (Community, Professional или Enterprise) и интересующие вас рабочие нагрузки.

Рабочие нагрузки – это просто набор функций, описывающих конкретный язык, среду или формат программ. Примерами могут служить Python, Node.js или .NET. Нас, разумеется, интересуют рабочие нагрузки, относящиеся к C++ (рис. 3.6); существует развитая поддержка для различных вариантов использования:

- разработка персональных приложений на C++;
- разработка на платформе Universal Windows Platform;
- разработка игр на C++;
- разработка мобильных приложений на C++;
- разработка для Linux на C++.

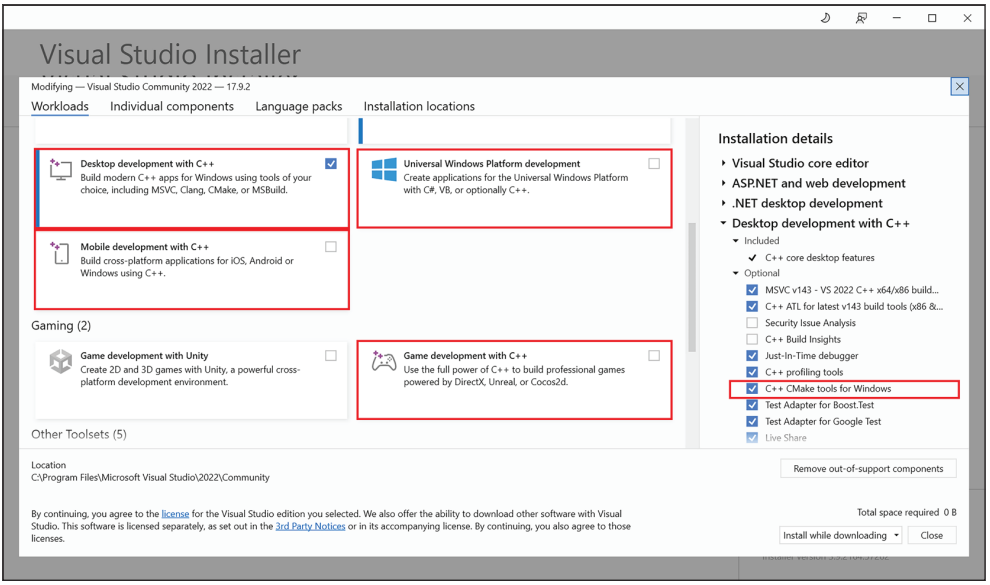


Рис. 3.6. Окно установщика VS

Выберите те, что отвечают разрабатываемому приложению, и нажмите **Install**. Не старайтесь установить все подряд на всякий случай, вы всегда сможете изменить решение позже, еще раз запустив установщик. Если вы хотите сконфигурировать компоненты рабочих нагрузок более точно, то не забудьте поднять флажок **C++ CMake tools for Windows** или **C++ CMake tools for Linux**, чтобы получить доступ к поддержке CMake.

По завершении установки можете запустить IDE и выбрать пункт **Create a new project** в начальном окне. Будет предложено несколько шаблонов в зависимости от установленных ранее рабочих нагрузок. Для работы с CMake выберите шаблон **CMake Project**. В других вариантах CMake может и не использоваться. Создав проект, можете запустить его, нажав зеленую кнопку в верхней части окна; код будет откомпилирован, и вы увидите, что выполнилась простая программа, которая выводит

```
Hello CMake.
```

Теперь вы готовы к работе с CMake в Visual Studio.

Крутая штука: отладка в режиме горячей перезагрузки

Хотя Visual Studio потребляет больше ресурсов и дольше загружается, предлагаемые ей возможности не имеют равных. Одна из них – горячая перезагрузка (Hot Reload). Работает она следующим образом: откройте проект C++, запустите его под отладчиком, внесите в код изменение, нажмите кнопку

Hot Reload (или *Alt+10*) – и ваши изменения немедленно отразятся на работающем приложении с сохранением его состояния.

Чтобы включить поддержку горячей перезагрузки, установите следующие два параметра в меню **Project > Properties > C/C++ > General**:

- **Debug Information Format** должен быть равен **Program Database for Edit and Continue /ZI**;
- **Enable Incremental Linking** должен быть равен **Yes /INCREMENTAL**.

Механизм работы горячей перезагрузки может показаться чародейством, но иметь такую возможность чрезвычайно полезно. Есть, правда, некоторые ограничения: нельзя изменять глобальные и статические данные, схему размещения объекта в памяти или «путешествовать во времени» (например, изменять конструктор уже сконструированного объекта).

Дополнительные сведения о горячей перезагрузке можете найти в официальной документации по адресу https://learn.microsoft.com/en-us/visualstudio/debugger/hot-reload.

На этом мы завершаем обзор трех основных IDE. Кривая обучения поначалу может показаться крутой, но обещаю, что усилия, потраченные на изучение любой из этих платформ, очень быстро окупятся, когда вы перейдете к более трудным задачам.

Резюме

В этой главе мы познакомились с использованием IDE для оптимизации процесса программирования, уделив особое внимание тем IDE, которые глубоко интегрированы с CMake. Мы описали преимущества IDE и подход к выбору той, что лучше всего отвечает потребностям – вашим или организации.

Мы начали с обсуждения важности IDE в плане повышения скорости разработки и качества кода, объяснили, что такое IDE и как она упрощает разработку ПО благодаря интеграции таких инструментов, как редакторы кода, компиляторы и отладчики. Затем мы напомнили о комплектах инструментов и отметили необходимость их установки в случае отсутствия в системе. Был представлен краткий список наиболее популярных вариантов.

Мы обсудили, как начать работу с CLion, упомянули о ее уникальных возможностях и кинули беглый взгляд на средства отладки. VS Code, бесплатная кросс-платформенная IDE от Microsoft, получила признание за обширную экосистему расширений и поддержку множества языков программирования. Мы кратко описали начальную установку и основные расширения, а более подробно поговорили о контейнерах разработчика. VS IDE, работающая только в Windows, предлагает функционально насыщенную среду, способную удовлетворить самые разные потребности пользователей. Мы описали процесс установки и основные возможности, уделив особое внимание отладке в режиме горячей перезагрузки.

В разделах, посвященных каждой IDE, были приведены соображения о том, почему вам может понравиться именно она, а также описаны первые шаги

и какая-то одна особенность, выделяющая эту IDE из ряда других. Попутно мы подчеркнули, как важна поддержка удаленной разработки.

Короче говоря, эта глава задумана как руководство для программистов, стремящихся понять основы IDE и выбрать подходящую для себя. Дан обзор важнейших функций, уникальных преимуществ и эффективного использования в сочетании с CMake. В следующей главе мы поговорим о настройке проекта с использованием CMake.

Для дополнительного чтения

Дополнительные сведения по вопросам, рассмотренным в этой главе, можно найти в следующих источниках.

- Qt Creator, еще одна IDE с поддержкой CMake: https://www.qt.io/product/development-tools.
- Eclipse – IDE для разработчиков на C/C++, также поддерживающая CMake: https://www.eclipse.org/downloads/packages/release/2023-12/r/eclipse-idecc-developers.
- Xcode для macOS тоже можно использовать совместно с CMake: https://medium.com/practical-coding/migrating-to-cmake-in-c-and-gettingit-working-with-xcode-50b7bb80ae3d.
- CodeLite – еще один вариант благодаря плагину CMake: https://docs.codelite.org/plugins/cmake/.

4

Подготовка первого проекта на основе CMake

Теперь у нас достаточно информации, чтобы начать разговор о главной функции CMake: *сборке проектов*. В CMake **проект** содержит все исходные файлы и конфигурационные данные, необходимые для управления процессом оживления наших решений. Конфигурирование начинается выполнением проверок: поддерживается ли целевая платформа, присутствуют ли все необходимые зависимости и инструменты, совместим ли компилятор с предъявленными требованиями.

По завершении предварительных проверок CMake переходит к генерированию сборочной системы, «заточенной» под выбранный инструмент сборки. Затем сборочная система выполняется, т. е. исходные файлы компилируются и компонуются с зависимостями для создания выходных артефактов.

Результирующие артефакты могут быть доставлены потребителям различными способами. Их можно распространять в виде двоичных пакетов, которые пользователи устанавливают с помощью менеджеров пакетов. Или же в виде установщика, состоящего всего из одного файла. Кроме того, у конечного пользователя есть возможность создать артефакты самостоятельно, скачав проекты из общего репозитория ПО с открытым исходным кодом. В последнем случае пользователь может использовать CMake для компиляции проекта на своей машине и его последующей установки.

Задействование всех возможностей проектов CMake может значительно упростить разработку и повысить качество сгенерированного кода. Благодаря CMake удается автоматизировать многие рутинные задачи, например прогон тестов после сборки и выполнение проверки покрытия тестами, форматеров, валидаторов, линтеров и других инструментов. Такая автоматизация не только экономит время, но и гарантирует единообразие и повышение качества кода на всех этапах процесса разработки.

Чтобы раскрыть всю мощь проектов CMake, мы сначала примем несколько ключевых решений: как правильно сконфигурировать проект в целом и как разбить его на части и организовать дерево исходного кода, так чтобы все файлы попали в нужные каталоги. Если с самого начала выбрать разумную структуру и организацию, то проектом CMake можно будет эффективно управлять, и он сможет развиваться и масштабироваться.

Далее мы рассмотрим среду сборки проекта. Мы поймем, с какой архитектурой имеем дело, какие инструменты есть в нашем распоряжении, какие возможности они поддерживают и какой стандарт языка мы будем использовать. Чтобы убедиться, что все работает как часы, мы откомпилируем тестовую программу на C++ и посмотрим, удовлетворяет ли выбранный компилятор нашим требованиям к проекту.

В этой главе будут рассмотрены следующие вопросы:

- основные директивы и команды;
- разбиение проекта на части;
- размышления о структуре проекта;
- определение среды;
- конфигурирование комплекта инструментов;
- запрет внутренней сборки.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch04.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Основные директивы и команды

В главе 1 мы уже видели определение простого проекта. Вернемся к нему. Это каталог, содержащий файл `CMakeLists.txt` с несколькими командами конфигурирования языкового процессора:

chapter01/01-hello/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Hello)
add_executable(Hello hello.cpp)
```

В той же главе мы узнали о нескольких простых командах. Сейчас поговорим о них подробнее.

Задание минимальной версии CMake

Важно помещать команду `cmake_minimum_required()` в начало каждого файла проекта и скрипта. Она не только проверяет, что в системе установлена нужная версия CMake, но и неявно вызывает другую команду `cmake_policy(VERSION)`, которая определяет, какие политики будут использованы в проекте. Политики определяют, как ведут себя команды CMake, они вводились по ходу разработки CMake для адаптации к изменениям и улучшениям в поддерживаемых языках и самой CMake.

Чтобы язык оставался простым и чистым, команда CMake ввела политики всюду, где вносится несовместимое с предыдущей версией изменение. Каждая политика активирует новое поведение, связанное с этим изменением. Политики гарантируют, что проекты смогут адаптироваться к развитию CMake, сохраняя совместимость со старой кодовой базой.

Вызывая `cmake_minimum_required()`, мы сообщаем CMake о необходимости применить политики по умолчанию, сконфигурированные для версии, указанной в аргументе. При переходе на новую версию CMake нам не нужно переживать, что проект перестанет работать, потому что новые политики, входящие в состав новой версии, не будут активированы.

Политики могут оказывать влияние на все аспекты CMake, включая такие важные команды, как `project()`. Поэтому так важно указывать в начале файла `CMakeLists.txt` версию, с которой вы работаете. В противном случае ждите предупреждений и ошибок.

В каждой версии CMake добавляются различные политики. Но вникать в эти детали вам необязательно до тех пор, пока не возникнут проблемы при переводе старых проектов на последнюю версию CMake. В таких случаях рекомендуется обратиться к документации по политикам, где вы найдете полную информацию и инструкции: https://cmake.org/cmake/help/latest/manual/cmake-policies.7.html.

Определение языков и метаданных

Рекомендуется помещать команду `project()` сразу после `cmake_minimum_required()`, хотя технически это и необязательно. Тем самым вы будете уверены, что при конфигурировании проекта используются правильные политики. Можно применять любую из двух форм:

```
project(<PROJECT-NAME> [<language-name>...])
```

или

```
project(<PROJECT-NAME>
        [VERSION <major>[.<minor>[.<patch>[.<tweak>]]]]
        [DESCRIPTION <project-description-string>]
        [HOMEPAGE_URL <url-string>]
        [LANGUAGES <language-name>...])
```

Аргумент `<PROJECT-NAME>` должен присутствовать, а остальные факультативны. При выполнении этой команды неявно устанавливаются следующие переменные:

```
PROJECT_NAME
CMAKE_PROJECT_NAME (только в CMakeLists.txt верхнего уровня)
PROJECT_IS_TOP_LEVEL, <PROJECT-NAME>_IS_TOP_LEVEL
PROJECT_SOURCE_DIR, <PROJECT-NAME>_SOURCE_DIR
PROJECT_BINARY_DIR, <PROJECT-NAME>_BINARY_DIR
```

Какие языки поддерживаются? Их немало. И можно одновременно использовать несколько! Ниже приведен перечень языков, допустимых при конфигурировании проекта:

- `ASM`, `ASM_NASM`, `ASM_MASM`, `ASMMARMASM`, `ASM-ATT`: диалекты ассемблера;
- `C`: C;
- `CXX`: C++;
- `CUDA`: Compute Unified Device Architecture от компании Nvidia;
- `OBJC`: Objective-C;
- `OBJCXX`: Objective-C++;
- `Fortran`: Fortran;
- `HIP`: Heterogeneous(-compute) Interface for Portability (для платформ Nvidia и AMD);
- `ISPC`: язык Implicit SPMD Program Compiler;
- `CSharp`: C#;
- `Java`: Java (требует дополнительных шагов, см. документацию).

По умолчанию CMake подключает к проекту C и C++, поэтому в своих проектах на C++, возможно, стоит указать только `CXX`. Почему? Потому что команда `project()` проверяет наличие компиляторов для выбранных вами языков, поэтому лучше ограничиться необходимыми, чтобы сэкономить время на этапе конфигурирования, пропустив ненужные проверки.

Присутствие ключевого слова `VERSION` автоматически устанавливает переменные, которые можно использовать для конфигурирования пакетов или включать в заголовочные файлы, которые будет видеть компилятор (мы рассмотрим этот вопрос в разделе «Конфигурирование заголовочных файлов» главы 7):

```
PROJECT_VERSION, <PROJECT-NAME>_VERSION
CMAKE_PROJECT_VERSION (только в CMakeLists.txt верхнего уровня)
PROJECT_VERSION_MAJOR, <PROJECT-NAME>_VERSION_MAJOR
PROJECT_VERSION_MINOR, <PROJECT-NAME>_VERSION_MINOR
PROJECT_VERSION_PATCH, <PROJECT-NAME>_VERSION_PATCH
PROJECT_VERSION_TWEAK, <PROJECT-NAME>_VERSION_TWEAK
```

Можно также задать `DESCRIPTION` и `HOMEPAGE_URL`, что приведет к установке следующих переменных для аналогичных целей:

```
PROJECT_DESCRIPTION, <PROJECT-NAME>_DESCRIPTION
PROJECT_HOMEPAGE_URL, <PROJECT-NAME>_HOMEPAGE_URL
```

Команды `cmake_minimum_required()` и `project()` позволяют создать базовый списковый файл и инициализировать пустой проект. Хотя структура, быть может, и не так уж важна для небольших проектов с единственным файлом, по мере расширения кодовой базы она становится делом первостепенной важности. Как подготовиться к этому?

Разбиение проекта на части

По мере того как наша программа растет в терминах количества строк и файлов, становится очевидным, что нужно как-то решать назревшую проблему: либо разбивать проект на части, либо смириться с риском быть погребенным под сложностью. К этой проблеме есть два подхода: разделить код CMake и переместить исходные файлы в подкаталоги. В обоих случаях мы следуем принципу проектирования, который называется **разделением обязанностей**. Проще говоря, мы разбиваем код на меньшие части, группируем части со взаимосвязанной функциональностью, а прочие разносим с целью установить четкие границы.

Мы уже немного говорили о разбиении кода CMake при обсуждении списковых файлов в главе 1. Тогда речь шла о команде `include()`, которая позволяет CMake исполнять код из внешнего файла.

Этот метод помогает разделить обязанности, но лишь отчасти – специализированный код выносится в отдельные файлы и даже может разделяться между несвязанными проектами, но по-прежнему загрязняет глобальную область видимости своей внутренней логикой, если автор недостаточно аккуратен.

Дело в том, что вызов `include()` не создает дополнительных областей видимости и никаких новых границ сверх тех, что уже определены в файле. Почему это может оказаться проблемой, мы обсудим на примере программы, которая обслуживает нужды небольшой компании по сдаче автомобилей в аренду. В этой программе много исходных файлов, определяющих различные аспекты ее работы: управление клиентами, автомобилями, парков-

ками, долгосрочными контрактами, историей технического обслуживания, сведениями о работниках и т. д. Если бы мы должны были хранить все файлы в одном каталоге, задача найти что-то превратилась бы в сущий кошмар. Поэтому мы создадим несколько подкаталогов главного каталога проекта и переместим в них связанные общей тематикой файлы. Наш файл `CMakeLists.txt` тогда мог бы иметь такой вид:

ch04/01-partition/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(Rental CXX)
add_executable(Rental
               main.cpp
               cars/car.cpp
# еще файлы в других каталогах
)
```

Это хорошо, но, как видите, в файле верхнего уровня мы по-прежнему имеем список исходных файлов из вложенного каталога! Чтобы разделить обязанности еще сильнее, мы могли бы вынести список исходных файлов в другой списковый файл и сохранить его в переменной `sources`:

ch04/02-include/cars/cars.cmake

```
set(sources
    cars/car.cpp
# еще файлы в других каталогах
)
```

Теперь можно сослаться на этот файл в команде `include()`, чтобы получить доступ к переменной `sources`:

ch04/02-include/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(Rental CXX)
include(cars/cars.cmake)
add_executable(Rental
               main.cpp
               ${sources} # для cars/
)
```

CMake присваивает `sources` значение в той же области видимости, где находится `add_executable`, заполняя переменную именами всех файлов. Это решение работает, но имеет несколько изъянов.

- **Переменные из вложенного каталога загрязняют область видимости верхнего уровня (и наоборот).**
  В простом примере это неважно, но в более сложном многоуровневом дереве с несколькими переменными очень быстро станет трудно отла-

живаемой проблемой. Что, если у нас имеется несколько включаемых списковых файлов, каждый из которых определяет свою переменную `sources`?

- **Все каталоги разделяют одну и ту же конфигурацию.**
  Истинный масштаб этой проблемы становится ясен с годами по мере разрастания проекта. Не имея никакого способа отделить зерна от плевел, мы вынуждены трактовать все исходные файлы одинаково и не можем ни задать различные флаги компиляции, ни выбрать новую версию языка для некоторых частей кода, ни подавить предупреждения в отдельных частях программы. Все глобально, т. е. изменения приходится вносить сразу во все единицы трансляции.
- **Условия перекомпилирования тоже общие.**
  Любое изменение конфигурации означает, что нужно перекомпилировать все файлы, даже если некоторые этим изменением не затронуты.
- **Все пути вычисляются относительно верхнего уровня.**
  Заметим, что в файле `cars.cmake` мы должны были указать полный путь к файлу `cars/car.cpp`. Это приводит к многочисленным повторениям, что ухудшает удобочитаемость и идет вразрез с принципом чистого кодирования **Не повторяйся** (**DRY**) (избыточные повторения ведут к ошибкам). Переименование каталога стало бы огромной проблемой.

Альтернатива – использовать команду `add_subdirectory()`, которая создает область видимости переменных и имеет много других преимуществ.

Управление областью видимости с помощью каталогов

Общепринято структурировать проект в соответствии с естественной структурой файловой системы, когда вложенные каталоги представляют различные элементы приложения: бизнес-логику, пользовательский интерфейс, API, отчеты. Отдельные каталоги предусмотрены для тестов, внешних зависимостей, скриптов и документации. Для поддержки этого подхода CMake предлагает следующую команду:

```
add_subdirectory(source_dir [binary_dir] [EXCLUDE_FROM_ALL])
```

Как мы уже знаем, эта команда добавляет в сборку каталог. Можно было бы также указать путь к каталогу, в который будут записываться файлы (`binary_dir` или дерево сборки). Ключевое слово `EXCLUDE_FROM_ALL` запрещает автоматическую сборку целей, определенных в подкаталоге (**цели** будут рассматриваться в следующей главе). Это может быть полезно для отделения частей проекта, не нужных для основной функциональности (например, *примеров* или *расширений*).

Команда `add_subdirectory()` вычисляет путь `source_dir` (относительно текущего каталога) и разбирает находящийся там файл `CMakeLists.txt`. Этот

файл разбирается в области видимости каталога, что устраняет проблемы, свойственные предыдущему методу:

- переменные изолированы во вложенной области видимости;
- вложенные артефакты можно конфигурировать независимо;
- модификация вложенного файла `CMakeLists.txt` не требует пересборки не связанных с ним целей;
- пути локализованы в каталоге и при желании могут быть добавлены к родительскому **пути включения**.

В нашем примере `add_subdirectory()` структура каталогов выглядит следующим образом:

```
├── CMakeLists.txt
├── cars
│ ├── CMakeLists.txt
│ ├── car.cpp
│ └── car.h
└── main.cpp
```

Здесь мы имеем два файла `CMakeLists.txt`. В файле верхнего уровня используется вложенный подкаталог `cars`:

ch04/03-add_subdirectory/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(Rental CXX)
add_executable(Rental main.cpp)
add_subdirectory(cars)
target_link_libraries(Rental PRIVATE cars)
```

Последняя строка нужна для компоновки артефактов из каталога `cars` с исполняемым файлом `Rental`. Эта команда, зависящая от цели, поэтому мы будем обсуждать ее в следующей главе.

Посмотрим, как выглядит вложенный списковый файл:

ch04/03-add_subdirectory/cars/CMakeLists.txt

```
add_library(cars OBJECT
   car.cpp
   # еще файлы в других каталогах
)
target_include_directories(cars PUBLIC .)
```

В этом примере я использовал команду `add_library()` для порождения глобально видимой **цели** `cars` и добавил каталог `cars` в ее публичные **каталоги включаемых файлов** командой `target_include_directories()`. Это информирует CMake о том, где находится файл `cars.h`, поэтому при выполнении команды `target_link_libraries()` файл `main.cpp` сможет найти заголовочный файл без указания относительного пути:

```
#include "car.h"
```

Мы видим команду `add_library()` во вложенном списковом файле; значит ли это, что в этом примере мы уже начали работать с библиотеками? На самом деле нет. Было использовано ключевое слово `OBJECT`, а это означает, что нас интересуют только **объектные файлы** (точно так же, как в предыдущем примере). Мы просто сгруппировали их в одну логическую цель (`cars`). Теперь вы уже, наверное, начинаете понимать, что такое **цель**. Не упускайте эту мысль – мы вернемся к ней в следующей главе.

Когда использовать вложенные проекты

В предыдущем разделе мы кратко упомянули, что аргумент `EXCLUDE_FROM_ALL` команды `add_subdirectory()` используется для обозначения необязательных элементов кодовой базы. В документации по CMake сказано, что если такие части находятся внутри дерева исходного кода, то в файлах `CMakeLists.txt` рекомендуется заводить для них отдельные команды `project()`, чтобы можно было генерировать собственные сборочные системы и осуществлять сборку независимо.

В каких случаях это может быть полезно? Например, когда вы работаете с несколькими проектами на C++, собираемыми в одном конвейере **CI/CD** (быть может, при сборке каркаса или набора библиотек). Или, возможно, вы переносите сборочную систему из унаследованного решения, скажем GNU Make, в котором используются простые файлы **makefile**. В таком случае желательно иметь возможность постепенно разбивать систему на относительно независимые части – быть может, чтобы поместить их в отдельный конвейер сборки или просто чтобы работать с меньшим контекстом, который можно загрузить в IDE типа **CLion**. Этого можно добиться, добавив команду `project()` в списковый файл во вложенном каталоге. Но не забывайте начать этот файл командой `cmake_minimum_required()`.

Итак, вложенные проекты поддерживаются, но можем ли мы как-то соединить между собой взаимосвязанные проекты, расположенные на одном уровне?

Внешние проекты должны оставаться внешними

Технически возможно сослаться на содержимое одного проекта из другого, но в CMake такая практика не рекомендуется. CMake предлагает для этой цели определенную поддержку, включая команду `load_cache()` для загрузки значений из кеша другого проекта. Однако это может привести к образованию циклических зависимостей и тесной связанности проектов. Лучше избегать этой команды и принять решение о том, как быть со взаимосвязанными проектами: сделать их вложенными, соединить посредством библиотек или объединить в один проект.

В нашем распоряжении имеются следующие инструменты разбиения: *включение списковых файлов*, *добавление подкаталогов* и *вложение проектов*. Но как их использовать, что проекты было легко сопровождать, осуществлять по ним навигацию и расширять? Для этого у проекта должна быть четко определенная структура.

Размышления о структуре проекта

Не секрет, что по мере разрастания проекта становится все труднее что-то найти в нем – как в списковых файлах, так и в исходном коде. Поэтому так важно с самого начала следить за гигиеной проекта.

Представьте, что нам нужно срочно внести важные изменения, но их нельзя отнести ни к одному из двух имеющихся в проекте каталогов. Тогда придется добавить *очистительную фиксацию*, которая изменит структуру иерархии файлов. Или, хуже того, воткнуть куда-нибудь и оставить комментарий TODO, чтобы решить эту проблему позже.

Проходит год, такого рода проблемы накапливаются, технический долг растет, а вместе с ним и стоимость сопровождения проекта. Особенно нетерпимым это становится, когда в работающий системе обнаруживается дефект, который требуется быстро исправить, или когда человеку, незнакомому с кодовой базой, потребовалось внести в код изменения.

Итак, нам нужна хорошая структура проекта. Но что это означает? Имеется пять правил, заимствованных из других областей разработки ПО, в частности системного проектирования. Проект должен обладать следующими характеристиками:

- простота навигации и расширения;
- четкие границы (файлы, относящиеся к проекту, должны находиться в каталоге проекта);
- цели образуют иерархическую древовидную структуру.

Не существует какого-то одного годящегося на все случаи жизни решения, но из различных имеющихся в сети шаблонов структуры проекта я предлагаю использовать представленный на рис. 4.1, поскольку он простой и расширяемый.

В этом проекте созданы каталоги для следующих компонентов:

- `cmake`: разделяемые макросы и функции, модули поиска и одноразовые скрипты;
- `src`: исходные и заголовочные файлы для исполняемых файлов и библиотек;
- `test`: исходный код автоматизированных тестов.

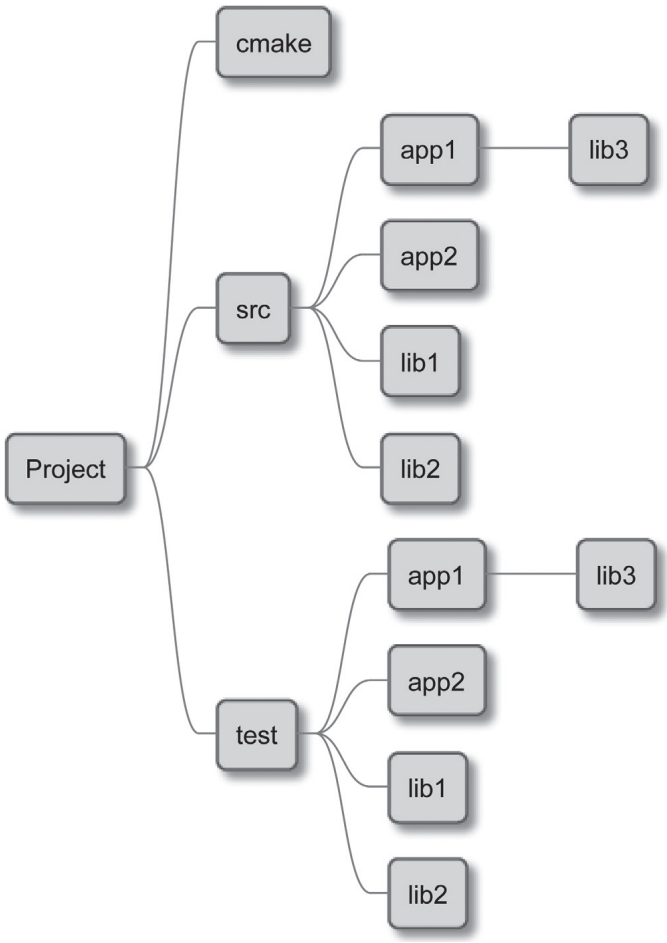


Рис. 4.1. Пример структуры проекта

В этой структуре файлы `CMakeLists.txt` должны присутствовать в следующих каталогах: верхнеуровневый каталог проекта, `test`, `src` и все их подкаталоги. Главный списковый файл не должен объявлять собственных шагов сборки, а лишь конфигурировать общие свойства проекта и делегировать ответственность за сборку вложенным списковым файлам с помощью команды `add_subdirectory()`. В свою очередь, эти списковые файлы должны при необходимости делегировать эту работу более глубоким уровням.



Некоторые разработчики предлагают отделять исполняемые файлы от библиотек и создавать не один, а два верхнеуровневых каталога: `src` и `lib`. CMake трактует оба этих артефакта одинаково, поэтому разделение на этом уровне не играет особой роли. Но если вам нравится, придерживайтесь такой модели.

Наличие нескольких подкаталогов внутри `src` оказывается удобным в крупных проектах. Но если вы создаете единственный исполняемый файл или библиотеку, то можете этого не делать и хранить все исходные файлы прямо в `src`. В любом случае не забывайте добавлять файл `CMakeLists.txt` и исполнять все вложенные списковые файлы. Вот как могло бы выглядеть дерево файлов для одной простой цели:

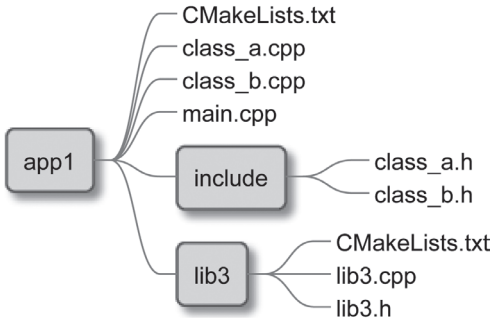


Рис. 4.2. Структура каталогов для сборки исполняемого файла

На рис. 4.1 мы видим файл `CMakeLists.txt` в корне каталога `src` – он конфигурирует основные параметры проекта и включает все списковые файлы из вложенных каталогов. Каталог `app1` (показанный на рис. 4.2) содержит еще один файл `CMakeLists.txt` наряду с `cpp`-файлами реализации: `class_a.cpp` и `class_b.cpp`. Также имеется файл `main.cpp`, содержащий точку входа в исполняемый файл. В файле `CMakeLists.txt` должна быть определена цель, в которой эти исходные файлы используются для сборки исполняемого (подробности – в следующей главе).

Наши заголовочные файлы помещаются в каталог `include`, в них можно объявить символы, используемые в других единицах трансляции C++.

Далее мы имеем каталог `lib3`, который содержит библиотеку, используемую только в этом исполняемом файле (библиотеки, которые используются в других местах проекта или экспортируются вовне, должны находиться в каталоге `src`). Такая структура предлагает высокую гибкость и легко допускает расширение проекта. По мере добавления новых классов мы можем группировать их в библиотеки для повышения скорости компиляции. Посмотрим, как выглядит библиотека.

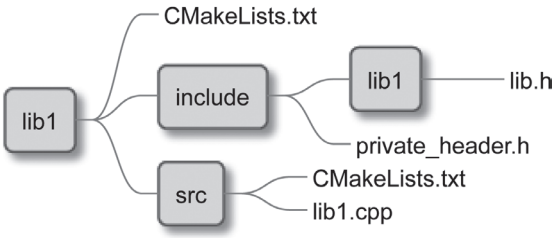


Рис. 4.3. Структура каталогов библиотеки

Библиотеки должны следовать той же структуре, что исполняемые файлы, но с одним небольшим отличием: в каталог `include` добавляется необязательный каталог `lib1`. Этот каталог включается, если библиотека предназначена для использования вне проекта. Он содержит публичные заголовочные файлы, которые другие проекты будут включать во время компиляции. Мы вернемся к этой теме, когда будем собирать библиотеки в главе 7.

Итак, мы обсудили, как размещены файлы в структуре каталогов. Пора взглянуть на то, как отдельные файлы `CMakeLists.txt` соединяются в проект и какую роль они играют в общем сценарии.

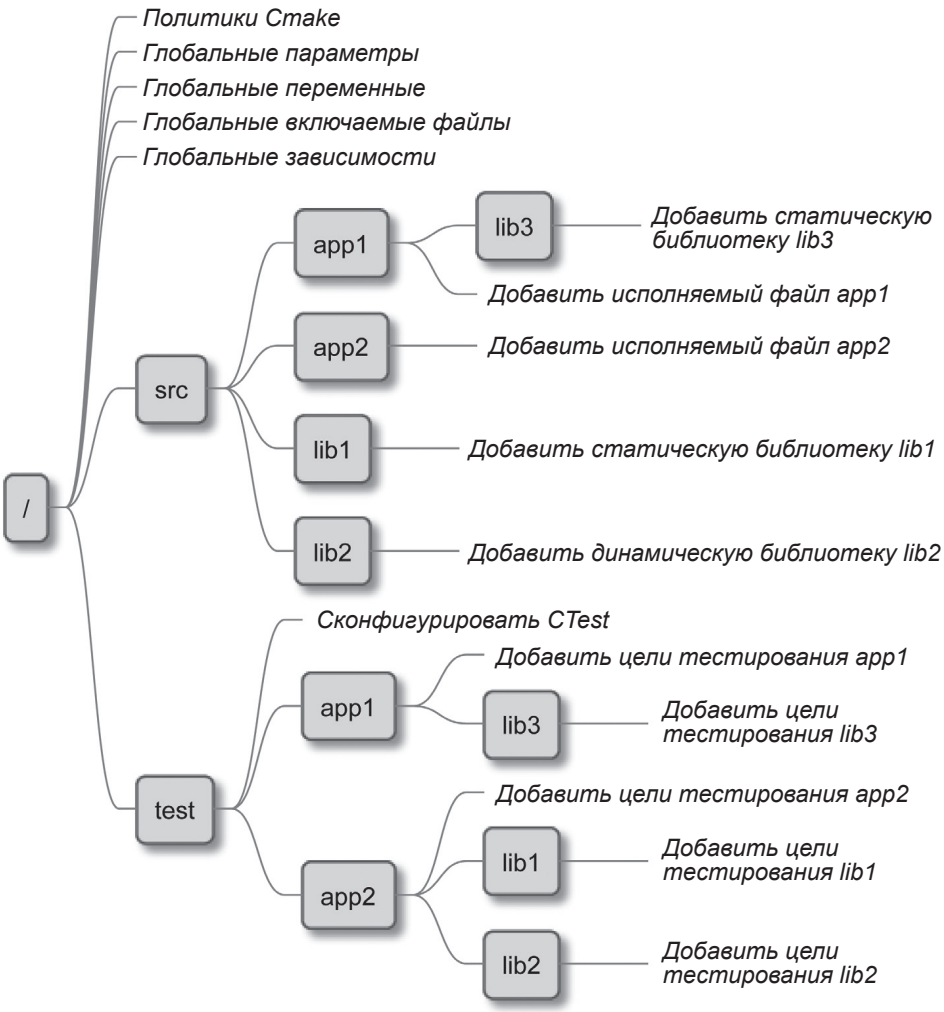


Рис. 4.4. Как Cmake объединяет списковые файлы в проект

Здесь прямоугольники представляют списковые файлы `CMakeLists.txt` в каждом каталоге, а комментарии курсивом описывают действия, выпол-

няемые файлом. Еще раз проанализируем этот проект с точки зрения CMake (все детали см. в примере в каталоге ch04/05-structure).

1. Выполнение начинается в корне проекта, т. е. со спискового файла CMakeLists.txt, который находится в корне дерева исходного кода. Этот файл определяет минимально необходимую версию CMake с подходящими политиками, задает имя проекта, поддерживаемые языки, глобальные переменные, а также включает файлы из каталога cmake, чтобы их содержимое было глобально доступно.
2. Следующий if – вход в область видимости каталога src путем вызова команды add_subdirectory(src bin) (мы хотели бы поместить откомпилированные артефакты в каталог <binary_tree>/bin, а не в <binary_tree>/src).
3. CMake читает файл src/CMakeLists.txt и обнаруживает, что его единственное назначение – добавить четыре вложенных подкаталога: app1, app2, lib1 и lib2.
4. CMake входит в область видимости app1 и узнает еще об одной вложенной библиотеке, lib3, в которой имеется собственный файл CMakeLists.txt; затем она входит в область видимости lib3. Как вы, наверное, заметили, это обход дерева каталогов в глубину.
5. Библиотека lib3 добавляет статическую библиотеку в качестве цели с тем же именем. CMake возвращается в родительскую область видимости app1.
6. Подкаталог app1 добавляет исполняемый файл, зависящий от lib3. CMake возвращается в родительскую область видимости src.
7. CMake продолжает входить в остальные вложенные области видимости и исполнять находящиеся в них списковые файлы до тех пор, пока не закончатся вызовы add_subdirectory().
8. CMake возвращается в область видимости верхнего уровня и выполняет оставшуюся команду add_subdirectory(test). При каждом выполнении этой команды CMake входит в новую область видимости и выполняет команды из соответствующего спискового файла.
9. Все цели собраны, их корректность проверена. Теперь у CMake есть вся необходимая информация для генерирования сборочной файловой системы.

Отметим, что все эти шаги выполняются точно в том порядке, в каком записаны команды в списковых файлах. В одних случаях порядок важен, в других – не очень. Мы подробнее рассмотрим этот вопрос в следующей главе и разберемся, какие следствия из него вытекают.

Так когда же следует создавать каталоги, содержащие все элементы проекта? С самого начала – создать все с прицелом на будущее, оставив пока каталоги пустыми, – или подождать, пока появятся файлы, нуждающиеся в отдельной категории? Тут у нас есть выбор: последовать принципу **YAGNI** (**тебе это не понадобится**) из **экстремального программирования** (Extreme Programming – **XP**) или заложить надежную основу для будущих разработчиков.

Попытайтесь найти баланс между этими подходами – если вы подозреваете, что в один прекрасный день в вашем проекте понадобится каталог extern, добавьте его сразу (возможно, ваша система управления версиями потребует пустого файла .keep, чтобы этот каталог попал в репозиторий).

Еще один эффективный способ рассказать другим, куда помещать внешние зависимости, – создать файл `README`, в котором описать рекомендуемую структуру. Это особенно полезно для малоопытных программистов, которые станут работать над проектом в будущем. Возможно, вы по себе замечали, что разработчики неохотно создают каталоги, особенно в корне проекта. Если вы предложите хорошую структуру проекта, то остальные с удовольствием ее примут.

Одни проекты можно собирать чуть ли не в любой среде, другие выдвигают очень строгие требования. Верхнеуровневый списковый файл – самое подходящее место для определения образа действий. Посмотрим, как это делается.

Определение окружения

CMake предлагает несколько способов опроса окружения с помощью переменных `CMAKE_`, переменных окружения и специальных команд. Например, собранную информацию можно использовать для поддержки кросс-платформенных скриптов. Эти механизмы позволяют избежать платформенно-зависимых команд оболочки, которые нелегко перенести или имена которых в разных окружениях различаются.

Для приложений, требующих высокой производительности, полезно знать все особенности платформы сборки (например, систему машинных команд, количество процессорных ядер и т. д.). Эту информацию можно затем передать в откомпилированные исполняемые файлы, чтобы точно настроить их мельчайшие детали (как это делается, мы узнаем в следующей главе). А пока изучим информацию, которую предоставляет сама CMake.

Определение операционной системы

Во многих случаях полезно знать целевую операционную систему. Даже для такой, казалось бы, обыденной вещи, как файловая система, имеются значительные различия между Windows и Unix: чувствительность к регистру, структура путей к файлам, наличие расширений, привилегии и т. д. Большинство команд одной системы отсутствуют в другой, а те, что функционально похожи, могут иметь разные имена (например, `ifconfig` в Unix и `ipconfig` в Windows). С другой стороны, одноименные команды могут порождать совершенно разный вывод.

Если вам когда-нибудь доведется поддерживать несколько операционных систем в одном скрипте CMake, опрашивайте переменную `CMAKE_SYSTEM_NAME` и действуйте соответственно, например:

```
if(CMAKE_SYSTEM_NAME STREQUAL "Linux")
  message(STATUS "Doing things the usual way")
elseif(CMAKE_SYSTEM_NAME STREQUAL "Darwin")
  message(STATUS "Thinking differently")
elseif(CMAKE_SYSTEM_NAME STREQUAL "Windows")
```

```
  message(STATUS "I'm supported here too.")
elseif(CMAKE_SYSTEM_NAME STREQUAL "AIX")
  message(STATUS "I buy mainframes.")
else()
  message(STATUS "This is ${CMAKE_SYSTEM_NAME} speaking.")
endif()
```

При необходимости можно воспользоваться переменной, содержащей версию операционной системы: `CMAKE_SYSTEM_VERSION`. Однако я рекомендую стараться делать скрипт настолько системно-независимым, насколько возможно, и пользоваться встроенной в CMake кросс-платформенной функциональностью. В частности, для операций с файловой системой пользуйтесь командой `file()`, описанной в приложении.

Кросс-компиляция: что такое хост-система и целевая система?

Кросс-компиляцией называется процесс компиляции на одной машине кода, который будет выполняться на другой платформе. Например, при наличии подходящего комплекта инструментов можно откомпилировать приложения для Android, запустив CMake на машине под управлением Windows. Хотя кросс-компиляция выходит за рамки этой книги, важно понимать, как она влияет на некоторые части CMake.

Чтобы кросс-компиляция стала возможной, необходимо, в частности, присвоить переменным `CMAKE_SYSTEM_NAME` и `CMAKE_SYSTEM_VERSION` значения, описывающие ту операционную систему, для которой производится компиляция (в документации по CMake она называется **целевой системой**). А та ОС, в которой производится компиляция, называется **хост-системой**.

Вне зависимости от конфигурации информация о хост-системе всегда доступна с помощью переменных, содержащих строку `HOST` в имени: `CMAKE_HOST_SYSTEM`, `CMAKE_HOST_SYSTEM_NAME`, `CMAKE_HOST_SYSTEM_PROCESSOR` и `CMAKE_HOST_SYSTEM_VERSION`.

Существует еще несколько переменных со строкой `HOST` в имени, просто имейте в виду, что все они относятся к хост-системе. Все остальные переменные относятся к целевой системе (которая обычно совпадает с хост-системой, если только мы не производим кросс-компиляцию).

Интересующиеся могут получить дополнительные сведения о кросс-компиляции из документации по адресу https://cmake.org/cmake/help/latest/manual/cmake-toolchains.7.html.

Сокращенные переменные

В CMake предопределено несколько переменных, содержащих информацию о хост-системе и целевой системе. Переменная, соответствующая текущей системе, будет иметь значение, отличное от `false` (т. е. `1` или `true`):

- `ANDROID`, `APPLE`, `CYGWIN`, `UNIX`, `IOS`, `WIN32`, `WINCE`, `WINDOWS_PHONE`;
- `CMAKE_HOST_APPLE`, `CMAKE_HOST_SOLARIS`, `CMAKE_HOST_UNIX`, `CMAKE_HOST_WIN32`.

Переменные `WIN32` и `CMAKE_HOST_WIN32` равны `true` для 32- и 64-разрядных версий Windows и MSYS (это значение осталось ради совместимости со старыми версиями). Переменная `UNIX` равна `true` для Linux, macOS и Cygwin.

Информация о хост-системе

CMake могла бы предоставить и больше переменных, но для экономии времени она не запрашивает у среды редко используемую информацию, например *поддерживает ли процессор расширения MMX* или *каков объем физической памяти*. Это не значит, что информация недоступна – просто для ее получения нужно явно выполнить команду

```
cmake_host_system_information(RESULT <VARIABLE> QUERY <KEY>...)
```

Мы должны задать целевую переменную и список ключей. Если задан только один ключ, то переменная будет содержать одно значение, в противном случае – список значений. Можно запросить следующие сведения об окружении и ОС:

Ключ	**Описание**
`HOSTNAME`	Имя хоста
`FQDN`	Полное доменное имя
`TOTAL_VIRTUAL_MEMORY`	Общий объем виртуальной памяти в МБ
`AVAILABLE_VIRTUAL_MEMORY`	Объем доступной виртуальной памяти в МБ
`TOTAL_PHYSICAL_MEMORY`	Общий объем физической памяти в МБ
`AVAILABLE_PHYSICAL_MEMORY`	Объем доступной физической памяти в МБ
`OS_NAME`	Вывод `uname -s`, если такая команда имеется в системе, в противном случае `Windows`, `Linux` или `Darwin`
`OS_RELEASE`	Подтип операционной системы, например `Windows Professional`
`OS_VERSION`	Номер сборки ОС
`OS_PLATFORM`	В Windows `$ENV{PROCESSOR_ARCHITECTURE}`. В Unix и macOS – вывод `uname -m`

При необходимости можно запросить даже сведения о процессоре:

Ключ	**Описание**
`NUMBER_OF_LOGICAL_CORES`	Количество логических ядер
`NUMBER_OF_PHYSICAL_CORES`	Количество физических ядер
`HAS_SERIAL_NUMBER`	1, если у процессора есть серийный номер
`PROCESSOR_SERIAL_NUMBER`	Серийный номер процессора
`PROCESSOR_NAME`	Название процессора
`PROCESSOR_DESCRIPTION`	Развернутое описание процессора

Ключ	Описание
IS_64BIT	1, если процессор 64-разрядный
HAS_FPU	1, если процессор поддерживает операции с плавающей точкой
HAS_MMX	1, если процессор поддерживает команды MMX
HAS_MMX_PLUS	1, если процессор поддерживает расширенные команды MMX
HAS_SSE	1, если процессор поддерживает расширенные команды SSE
HAS_SSE2	1, если процессор поддерживает расширенные команды SSE2
HAS_SSE_FP	1, если процессор поддерживает расширенные команды SSE FP
HAS_SSE_MMX	1, если процессор поддерживает расширенные команды SSE MMX
HAS_AMD_3DNOW	1, если процессор поддерживает расширенные команды 3DNow
HAS_AMD_3DNOW_PLUS	1, если процессор поддерживает расширенные команды 3DNow+
HAS_IA64	1, если процессор x86

Архитектура платформы 32- или 64-разрядная?

В 64-разрядной архитектуре адреса памяти, регистры процессора, команды процессора, адресные шины и шины данных имеют ширину 64 разряда. И хотя это упрощенное определение, оно дает приблизительное представление о том, чем 64-разрядные платформы отличаются от 32-разрядных.

В C++ от архитектуры зависит разрядность некоторых фундаментальных типов данных (`int` и `long`) и указателей. CMake использует размер указателя для сбора информации о целевой машине. Эта информация доступна посредством переменной `CMAKE_SIZEOF_VOID_P`, которая равна `8` для 64-разрядной архитектуры (потому что длина указателя – 8 байт) и `4` – для 32-разрядной (4 байта):

```
if(CMAKE_SIZEOF_VOID_P EQUAL 8)
  message(STATUS "Target is 64 bits")
endif()
```

Каков порядок байтов в системе?

Архитектура может иметь **прямой** (little-endian) или **обратный** (big-endian) порядок байтов в слове или естественной для процессора единице данных. В системе с обратным порядком байтов старший байт хранится в ячейке памяти с наименьшим адресом, а младший – в ячейке памяти с наибольшим адресом. В системе же с прямым порядком байтов порядок байтов противоположен, т. е. младший байт хранится в ячейке памяти с наименьшим адресом, а старший – с наибольшим.

В большинстве случаев **порядок байтов** не играет роли, но если вы пишете код поразрядной обработки и хотите сделать его переносимым, то CMake предлагает значение `BIG_ENDIAN` или `LITTLE_ENDIAN`, хранящееся в переменной `CMAKE_<LANG>_BYTE_ORDER`, где `<LANG>` – одна из строк `C`, `CXX`, `OBJC` или `CUDA`.

Конфигурирование комплекта инструментов

В проектах CMake под комплектом инструментов понимаются все инструменты, используемые для сборки и выполнения приложения, например рабочая среда, генератор, сам исполняемый файл CMake и компиляторы.

Представьте, что чувствует неопытный пользователь, когда сборка внезапно останавливается с загадочными сообщениями об ошибках компиляции и синтаксических ошибках. Он может залезть в исходный код и попытаться понять, что случилось. Потратив час на отладку, он обнаруживает, что для исправления ошибки нужно взять другой компилятор.

Нельзя ли облегчить жизнь пользователям и проверять, что компилятор содержит все необходимые функции, еще до начала сборки? Конечно, можно! Существуют способы описать такие требования. Если комплект инструментов не поддерживает их все, то CMake остановится на раннем этапе, выведет понятное сообщение о случившемся и попросит пользователя вмешаться.

Задание стандарта C++

Одним из первых шагов могло бы стать задание требуемого стандарта C++, который компилятор должен поддерживать для сборки проекта. В новых проектах рекомендуется задавать как минимум C++14, а лучше C++17 или C++20. Начиная с версии CMake 3.20 требуемый стандарт может быть C++23, если компилятор его поддерживает. Кроме того, начиная с версии CMake 3.25 имеется возможность задать стандарт C++26, хотя пока это всего лишь заявка на будущее.



Прошло больше 10 лет с момента выпуска официального стандарта **C++11**, и он уже не считается *современным стандартом C++*. Не рекомендуется в новых проектах указывать эту версию, разве что целевое окружение совсем старое.

Еще одна причина придерживаться старых стандартов – если вы собираете унаследованные цели, которые очень трудно модернизировать. Однако комитет C++ всеми силами старается сохранять обратную совместимость C++, и в большинстве случаев не будет никаких проблем с переходом на новую версию.

CMake поддерживает задание стандарта для каждой цели в отдельности (это полезно, если части кодовой базы очень старые), но вообще-то лучше свести проект к единому стандарту. Это можно сделать, присвоив переменной `CMAKE_CXX_STANDARD` одно из следующих значений: `98`, `11`, `14`, `17`, `20`, `23` или `26`:

```
set(CMAKE_CXX_STANDARD 23)
```

Это значение будет по умолчанию применяться ко всем определенным далее целям (поэтому лучше задавать ее близко к началу корневого спискового файла). При желании его можно переопределить в конкретной цели следующим образом:

```
set_property(TARGET <target> PROPERTY CXX_STANDARD <version>)
```

или

```
set_target_properties(<targets> PROPERTIES CXX_STANDARD <version>)
```

Второй вариант позволяет задать несколько целей.

Принудительная поддержка стандарта

Вышеупомянутое свойство `CXX_STANDARD` не препятствует CMake продолжать сборку, даже если компилятор не поддерживает желаемую версию – она рассматривается только как пожелание. CMake не знает, используются ли в вашем коде новые возможности, отсутствующие в предыдущих версиях компилятора, поэтому старается работать с тем, что есть.

Если мы точно знаем, что это бесполезно, то можем задать другую переменную (которая точно так же, как предыдущая, допускает переопределение), чтобы явно указать, на какой стандарт мы ориентируемся.

```
set(CMAKE_CXX_STANDARD_REQUIRED ON)
```

В этом случае, если системный компилятор не поддерживает требуемый стандарт, то пользователь увидит следующее сообщение, и сборка прекратится:

```
Target "Standard" requires the language dialect "CXX23" (with c-ompiler
extensions), but CMake does not know the compile flags to use to enable
it.
```

Запрашивать стандарт C++23, пожалуй, чересчур, даже для современного окружения. Но с C++20 на обновленных системах не должно возникать проблем, потому что он поддерживается в **GCC**/**Clang**/**MSVC** с 2021/2022 г.

Расширения производителя

В зависимости от политики, действующей в вашей организации, может потребоваться разрешить или запретить расширения производителя. Что это такое? Предположим, что процесс принятия нового стандарта C++ кажется слишком медленным какому-то производителю компилятора, поэтому он решает включить в язык собственные наработки – *расширения*. Например,

технический отчет 1 (**TR1**) по C++ представлял собой расширение библиотеки, в котором были добавлены регулярные выражения, интеллектуальные указатели, хеш-таблицы и генераторы случайных чисел еще до того, как они стали повсеместными. Для поддержки этих плагинов, добавленных в проекте GNU, CMake заменяет флаг компилятора, определяющий стандарт (`-std=c++14`), флагом `-std=gnu++14`.

С одной стороны, это может быть желательно, поскольку позволяет включить нужную функциональность. Но, с другой стороны, ваш код перестанет быть переносимым, поскольку не будет собираться, если вы (или пользователь) захотите перейти на другой компилятор. Это свойство также задается отдельно для каждой цели с помощью переменной `CMAKE_CXX_EXTENSIONS`. CMake в этом случае ведет себя либерально – разрешает расширения, если мы явно не запретим их командой

```
set(CMAKE_CXX_EXTENSIONS OFF)
```

Я рекомендую поступать именно так, поскольку в таком режиме код не будет зависеть от производителя и, стало быть, не будет налагать на пользователей неоправданных ограничений. Как и в предыдущих случаях, изменить режим для конкретной цели позволяет команда `set_property()`.

Межпроцедурная оптимизация

Обычно компиляторы оптимизируют код на уровне единицы трансляции, т. е. ваш `cpp`-файл обрабатывается препроцессором, компилируется, а затем оптимизируется. Промежуточные файлы, сгенерированные в ходе этих операций, передаются компоновщику, который создает один двоичный файл. Но современные компиляторы умеют выполнять **межпроцедурную оптимизацию** (МПО) во время компоновки, иногда ее так и называют – **оптимизация на этапе компоновки**. Это позволяет оптимизировать все единицы компиляции как единый модуль, достигая лучших результатов (иногда ценой замедления сборки и большего потребления памяти).

Если ваш компилятор поддерживает межпроцедурную оптимизацию, то имеет смысл этим воспользоваться. Переменная, отвечающая за этот режим, называется `CMAKE_INTERPROCEDURAL_OPTIMIZATION`. Но прежде чем устанавливать ее, нужно проверить, действительно ли такая оптимизация поддерживается, иначе возникнут ошибки:

```
include(CheckIPOSupported)
check_ipo_supported(RESULT ipo_supported)
set(CMAKE_INTERPROCEDURAL_OPTIMIZATION ${ipo_supported})
```

Как видите, нам пришлось включить встроенный модуль, чтобы получить доступ к команде `check_ipo_supported()`. Этот код вернется к поведению по умолчанию, если МПО не поддерживается.

Проверка возможностей, поддерживаемых компилятором

Мы уже говорили, что если нашей сборке суждено завершиться ошибкой, то лучше, чтобы это произошло как можно раньше; тогда мы сможем выдать пользователю информативное сообщение и сократить время ожидания. Иногда нас интересует, какие возможности C++ поддерживаются (а какие – нет). CMake опрашивает компилятор на этапе конфигурирования и сохраняет список доступных возможностей в переменной `CMAKE_CXX_COMPILE_FEATURES`. Мы можем запросить, поддерживается ли конкретная возможность.

ch04/07-features/CMakeLists.txt

```
list(FIND CMAKE_CXX_COMPILE_FEATURES cxx_variable_templates result)
if(result EQUAL -1)
  message(FATAL_ERROR "Variable templates are required for compilation.")
endif()
```

Как нетрудно догадаться, писать тест для каждой возможности утомительно. Даже авторы CMake рекомендуют проверять только наличие некоторых высокоуровневых **метавозможностей**: `cxx_std_98`, `cxx_std_11`, `cxx_std_14`, `cxx_std_17`, `cxx_std_20`, `cxx_std_23` и `cxx_std_26`. Каждая метавозможность показывает, поддерживает ли компилятор определенный стандарт C++. Если хотите, можете использовать их точно так, как в примере выше.

Полный список возможностей, известных CMake, имеется в документации по адресу https://cmake.org/cmake/help/latest/prop_gbl/CMAKE_CXX_KNOWN_FEATURES.html.

Компиляция тестового файла

С одним особенно интересным сценарием я встретился, когда компилировал приложение с помощью GCC 4.7.x. Прочитав справочное руководство, я лично убедился, что все интересующие меня возможности C++11 поддерживаются. Однако программа работала неправильно. Код молча игнорировал обращение к стандартному заголовку `<regex>`. Выяснилось, что в этой версии компилятора имела место ошибка, и библиотека регулярных выражений не была реализована.

Никакая одиночная проверка не сможет защитить вас от таких редких ошибок (да и необходимости проверять их нет!), но есть шанс, что захотите использовать какую-то передовую экспериментальную возможность последнего стандарта и не уверены, какие компиляторы ее поддерживают. Чтобы проверить, будет ли работать проект, можно создать небольшой тестовый пример, в котором используются требуемые возможности и который быстро компилируется и выполняется.

CMake предоставляет две команды на этапе конфигурирования, `try_compile()` и `try_run()`, позволяющие проверить, что все необходимое поддерживается на целевой платформе.

Команда `try_run()` дает больше свободы, поскольку вы можете удостовериться не только в том, что код компилируется, но и в том, что он правильно выполняется (в принципе, можно было бы проверить, работает ли `regex`). Конечно, это не подходит для сценариев кросс-компиляции (поскольку хост-система не сможет выполнить программу, собранную для другой целевой системы). Просто помните, что цель этой проверки – быстро вывести информативное сообщение, если компиляция не работает, а не прогнать автономные тесты или выполнить что-то сложное; файл должен быть максимально простым – что-то вроде следующего:

ch04/08-test_run/main.cpp

```
#include <iostream>
int main()
{
  std::cout << "Quick check if things work." << std::endl;
}
```

Воспользоваться `try_run()` совсем не трудно. Сначала мы задаем требуемый стандарт, после чего вызываем `try_run()` и печатаем собранную ей информацию:

ch04/08-test_run/CMakeLists.txt

```
set(CMAKE_CXX_STANDARD 20)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)
try_run(run_result compile_result
        ${CMAKE_BINARY_DIR}/test_output
        ${CMAKE_SOURCE_DIR}/main.cpp
        RUN_OUTPUT_VARIABLE output)
message("run_result: ${run_result}")
message("compile_result: ${compile_result}")
message("output:\n" ${output})
```

Поначалу эта команда может ошарашить, но на самом деле всего нескольких аргументов достаточно для компиляции и выполнения очень простого тестового файла. Дополнительно я использовал факультативный аргумент `RUN_OUTPUT_VARIABLE`, в который помещается текст, выведенный на `stdout`.

Следующий шаг – включить в простой тестовый файл возможности современного C++, которые мы собираемся использовать в фактическом проекте, например добавить вариадический шаблон и посмотреть, сможет ли компилятор с ним справиться.

Наконец, в условных блоках можно проверить, совпадает ли вывод с ожидаемым, и напечатать с помощью `message(SEND_ERROR <error>)` сообщение, если что-то пошло не так. Напомним, что ключевое слово `SEND_ERROR` разрешает CMake продолжить этап конфигурирования, но запрещает генерировать сборочную систему. Это позволяет показать все встретившиеся ошибки, прежде чем сборка будет прервана. Итак, мы теперь знаем, как гарантировать, что

компиляция завершится успешно. Перейдем к следующей теме – как запретить внутреннюю сборку.

Запрет внутренней сборки

В главе 1 мы говорили о внутренних сборках и о том, что рекомендуется размещать артефакты сборки вне дерева исходного кода. Это сделает дерево сборки чище, позволит упростить файл `.gitignore`, а заодно уменьшит вероятность случайной перезаписи и удаления исходных файлов.

Чтобы прервать сборку, вы можете использовать такую проверку:

ch04/09-in-source/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(NoInSource CXX)
if(PROJECT_SOURCE_DIR STREQUAL PROJECT_BINARY_DIR)
  message(FATAL_ERROR "In-source builds are not allowed")
endif()
message("Build successful!")
```

За дополнительными сведениями о префиксе STR и ссылках на переменные обратитесь к главе 2.

Заметим, однако, что вне зависимости от того, что делается в показанном выше коде, CMake, похоже, все равно создает каталог `CMakeFiles/` и файл `CMakeCache.txt`.



Вы можете встретить в сети предложения использовать недокументированные переменные, гарантирующие, что пользователь не сможет записывать в каталог исходного кода ни при каких обстоятельствах. Но опираться на недокументированные возможности не рекомендуется. Они могут работать не во всех версиях, и нет никакой гарантии, что они не будут удалены или изменены без предупреждения.

Если вас беспокоит, что пользователи оставят эти файлы в каталоге исходного кода, добавьте их в файл `.gitignore` (или эквивалентный ему) и включите в текст сообщение с напоминанием о необходимости ручной очистки.

Резюме

В этой главе мы рассмотрели полезные концепции, закладывающие основу для сборки надежных проектов, не опасающихся будущих изменений. Мы обсудили задание минимальной версии CMake и конфигурирование таких

важных аспектов проекта, как имя, языки и метаданные. Это поможет нам эффективно масштабировать проекты.

Мы поговорили о разбиении проекта на части, сравнили команды `include()` и `add_subdirectory()`, предлагающие такие преимущества, как управление переменными с учетом областей видимости, упрощение путей и улучшенная модульность. Возможность создавать вложенные проекты и собирать их независимо оказалась полезна для постепенного расслоения кода на более независимые единицы. Разобравшись в имеющихся механизмах разбиения, мы занялись созданием прозрачной, гибкой и расширяемой структуры проекта. Мы изучили, как CMake обходит списковые файлы, и объяснили важность правильного порядка шагов конфигурирования. Затем обсудили, как определить, какое окружение имеет место на целевой и хост-машине, в чем между ними разница и какую информацию о платформе и системе можно получить с помощью различных запросов. Мы также рассмотрели конфигурирование комплекта инструментов, включая задание требуемой версии C++, работу с расширениями компилятора конкретного производителя и разрешение важных оптимизаций. Мы научились проверять поддержку требуемых возможностей компилятора, включая выполнение проверочных программ.

Хотя рассмотренные до сих пор технические аспекты существенны, их еще недостаточно, чтобы проект был действительно полезным. Для этого нам предстоит разобраться в концепции **целей**. Мы мимоходом затрагивали эту тему ранее, а теперь готовы заняться ей вплотную, поскольку наконец-то понимаем фундаментальные основы. Цели – тема следующей главы – играют важнейшую роль в повышении функциональности и эффективности проектов.

Для дополнительного чтения

Дополнительные сведения о рассмотренных в этой главе вопросах можно найти по следующим ссылкам.

- Разделение обязанностей: https://nalexn.github.io/separation-of-concerns/.
- Полное руководство по переменным CMake: https://cmake.org/cmake/help/latest/manual/cmake-variables.7.html.
- Справочные материалы по командам `try_compile` и `try_run`: https://cmake.org/cmake/help/latest/command/try_compile.html, https://cmake.org/cmake/help/latest/command/try_run.html.
- Справочное руководство по CheckIPOSupported: https://cmake.org/cmake/help/latest/module/CheckIPOSupported.html.

5

Цели

В CMake приложение можно собрать из одного исходного файла (например, классического файла `helloworld.cpp`). Но точно так же можно создать проект, в котором исполняемый файл строится из множества исходных: десятков или даже тысяч. И многие начинающие идут по этому пути: собирают двоичные файлы, начав с горстки исходных, а затем позволяют проекту органически расти без строгого планирования. Они продолжают добавлять файлы по мере необходимости и не успели оглянуться, как оказывается, что все компонуется в единый двоичный файл без какой-либо структуры.

Но разработчики ПО должны осознанно проводить границы и выделять компоненты, которые будут группироваться в одну или несколько единиц трансляции (`cpp`-файлы). Это делается, чтобы код стал более удобочитаемым, чтобы управлять связностью и сцепленностью, ускорить процесс сборки и, наконец, распознать допускающие повторное использование компоненты и выделить их в автономные единицы.

Любой крупный проект нуждается в какой-то форме разбиения на части. Тут-то и приходят на помощь цели CMake. В CMake цель представляет логическую единицу, ориентированную на решение конкретной задачи. Одни цели могут зависеть от других, а при их конструировании используется декларативный подход. CMake берет на себя заботу о правильном порядке сборки целей, реализуя там, где возможно, параллельную сборку для оптимизации. В итоге генерируется артефакт, который может использоваться другими целями или является конечным результатом процесса сборки.

Обратите внимание на слово *артефакт*. Я намеренно избегаю конкретных терминов, потому что CMake умеет генерировать не только исполняемые файлы и библиотеки. На практике сгенерированные сборочные системы можно использовать для порождения самых разных типов вывода: дополнительных исходных файлов, заголовочных файлов, объектных файлов, архивов, конфигурационных файлов и т. п. Единственные требования: наличие командного инструмента (типа компилятора), факультативных входных файлов и заданного выходного пути.

Цели – невероятно мощная концепция, которая существенно упрощает процесс сборки проекта. Поэтому так важно хорошо понимать, как они ра-

ботают, и научиться конфигурировать их элегантно и организованно. Тогда разработка станет более плавной и эффективной.

В этой главе будут рассмотрены следующие вопросы:

- понятие цели;
- задание свойств целей;
- написание специальных команд.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch05.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Понятие цели

Если вы когда-нибудь работали с GNU Make, то о понятии цели уже знаете. Это рецепт, которому сборочная система следует для компиляции набора файлов с целью получения другого файла. Это может быть компиляция `cpp`-файла в **объектный файл** с расширением `.o` или упаковка группы `o`-файлов в статическую библиотеку. Существует тьма комбинаций и возможностей преобразовать цели в сборочной системе.

Однако CMake позволяет сэкономить время и избежать промежуточных шагов этих рецептов, поскольку работает на более высоком уровне абстракции. Она знает, как большинство языков собирают исполняемые файлы из исходных. Поэтому вам не нужно писать явные команды для компиляции **объектных файлов** C++ (в GNU Make это необходимо). Надо только включить команду `add_executable()`, указав имя исполняемого целевого файла и список исходных файлов:

```
add_executable(app1 a.cpp b.cpp c.cpp)
```

Мы пользовались этой командой в предыдущих главах и уже знаем, как исполняемые цели используются на практике – на этапе генерирования CMake

создает сборочную систему и помещает в нее рецепты компиляции исходных файлов и компоновки их в исполняемый файл.

В CMake цель создают следующие команды:

- `add_executable()`;
- `add_library()`;
- `add_custom_target()`.

Прежде чем собирать исполняемые файлы или библиотеки, CMake проверяет, действительно ли сгенерированный результат старше исходных файлов. Этот механизм помогает CMake избежать повторного создания и без того актуальных артефактов. Сравнивая временные метки, CMake определяет, какие цели нужно пересобрать, уменьшая тем самым общий объем компиляции.

Всем командам, определяющим цели, первым аргументом передается имя цели, чтобы впоследствии на него можно было сослаться в других командах, которые что-то делают с целями, например `target_link_libraries()`, `target_sources()` или `target_include_directories()`. Мы обсудим эти команды позже, а пока поговорим подробнее о том, какие цели можно определить.

Определение исполняемых целей

Команда для определения исполняемой цели, `add_executable()`, не нуждается в пояснениях (потому-то мы уже использовали ее в предыдущих главах). Ее формальная структура такова:

```
add_executable(<name> [WIN32] [MACOSX_BUNDLE]
               [EXCLUDE_FROM_ALL]
               [source1] [source2 ...])
```

Если мы компилируем для Windows, то добавление факультативного аргумента `WIN32` порождает исполняемый файл, который не будет показывать окно консоли по умолчанию (куда обычно направляется вывод на `std::cout`). Предполагается, что приложение создает свой собственный пользовательский интерфейс (GUI).

Назначение следующего факультативного аргумента, `MACOSX_BUNDLE`, похоже: создать приложение для macOS/iOS, которое Finder запускает как GUI-приложение.

Ключевое слово `EXCLUDE_FROM_ALL` подавляет сборку исполняемой цели обычным способом. Такая цель должна быть явно упомянута в *команде сборки*:

```
cmake --build -t <target>
```

Наконец, мы должны передать список исходных файлов, которые будут откомпилированы для получения цели. Поддерживаются следующие расширения:

- для C: `c`, `m`;
- для C++: `C`, `M`, `c++`, `cc`, `cpp`, `cxx`, `m`, `mm`, `mpp`, `CPP`, `ixx`, `cppm`, `ccm`, `cxxm`, `c++m`.

Заметим, что в список исходных файлов не входят **заголовочные файлы**. Их можно добавить либо неявно, указав путь к каталогу, в котором эти файлы находятся, с помощью команды `target_include_directories()`, либо воспользовавшись аргументом `FILE_SET` команды `target_sources()` (появившейся в версии CMake 3.23). Это важная тема, но поскольку она сложна и не связана напрямую с целями, мы отложим рассмотрение деталей до главы 7.

Определение библиотечных целей

Определение библиотек очень похоже на определение исполняемых файлов, но, конечно, не требуются ключевые слова, которые описывают, как нужно обрабатывать аспекты GUI. Команда имеет следующую сигнатуру:

```
add_library(<name> [STATIC | SHARED | MODULE]
            [EXCLUDE_FROM_ALL]
            [<source>...])
```

Правила, касающиеся имени, ключевого слова `EXCLUDE_FROM_ALL` и исходных файлов, точно такие же, как для исполняемых целей. Единственное различие – ключевые слова `STATIC`, `SHARED` и `MODULE`. Если у вас есть опыт работы с библиотеками, то вы знаете, что они определяют вид артефакта, генерируемого CMake: статически компонуемые библиотеки, разделяемые (динамически компонуемые) библиотеки или модули. Это тоже обширная тема, которую мы будем подробно рассматривать в главе 8.

Специальные цели

Специальные цели несколько отличаются от исполняемых файлов и библиотек. Они расширяют готовую функциональность CMake за счет того, что исполняют явно заданные команды. Например, их можно использовать для:

- вычисления контрольных сумм других двоичных файлов;
- запуска средства проверки кода и сбора результатов;
- отправки отчета о компиляции в конвейер метрик.

Как легко понять по этому списку, пользовательские цели полезны только в проектах повышенной сложности, поэтому мы рассмотрим лишь самые основы и потом перейдем к более важным целям.

Для определения специальной цели используется следующая команда (некоторые параметры для краткости опущены):

```
add_custom_target(Name [ALL] [COMMAND command2 [args2...] ...])
```

У специальных целей есть определенные недостатки. Поскольку в них участвуют команды оболочки, то цели получаются системно-зависимыми, что может ограничить переносимость. Кроме того, специальные цели могут не иметь простого способа сообщить CMake о том, какие артефакты или побочные продукты генерируются и генерируются ли вообще.

Специальные цели не применяют сравнение временных меток, как исполняемые и библиотечные цели (не проверяется, что исходный файл младше двоичного), потому что по умолчанию они не добавляются в **граф зависимостей** (поэтому ключевое слово `ALL` прямо противоположно `EXCLUDE_FROM_ALL`). Давайте разберемся, что такое граф зависимостей.

Граф зависимостей

Зрелые приложения часто строятся из компонентов, а именно внутренних библиотек. Разбиение проекта на части полезно с точки зрения его структурной организации. Если взаимосвязанные вещи упакованы в единую логическую сущность, то их можно компоновать с другими целями: другой библиотекой или исполняемым файлом. Особенно это удобно, когда несколько целей пользуются одной и той же библиотекой. Взгляните на рис. 5.1, на котором представлен пример графа зависимостей.

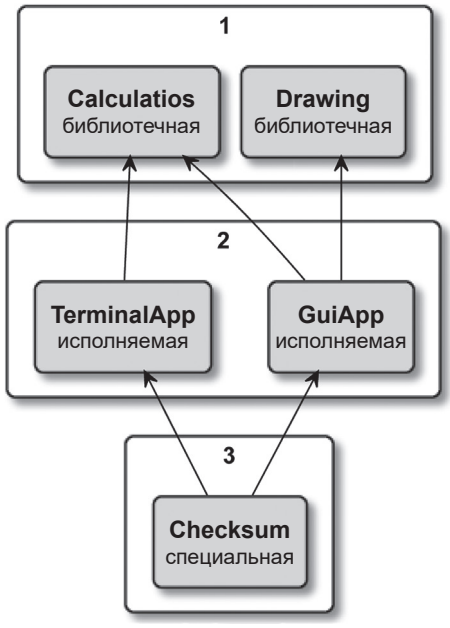


Рис. 5.1. Порядок сборки зависимостей в проекте BankApp

В этом проекте есть две библиотеки, два исполняемых файла и специальная цель. Мы создаем банковское приложение, имеющее красивый гра-

фический интерфейс для пользователей (**GuiApp**) и командную версию для использования в автоматизированных скриптах (**TerminalApp**). Оба исполняемых файла зависят от одной и той же библиотеки **Calculations**, но только одному из них нужна библиотека **Drawing**. Чтобы гарантировать, что двоичные файлы скачаны из подлинного источника, мы также вычисляем контрольную сумму и распространяем ее по отдельному безопасному каналу. CMake обладает достаточной гибкостью для написания списковых файлов в подобной ситуации:

ch05/01-targets/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(BankApp CXX)

add_executable(terminal_app terminal_app.cpp)
add_executable(gui_app gui_app.cpp)
target_link_libraries(terminal_app calculations)
target_link_libraries(gui_app calculations drawing)

add_library(calculations calculations.cpp)
add_library(drawing drawing.cpp)

add_custom_target(checksum ALL
    COMMAND sh -c "cksum terminal_app>terminal.ck"
    COMMAND sh -c "cksum gui_app>gui.ck"
    BYPRODUCTS terminal.ck gui.ck
    COMMENT "Checking the sums..."
)
```

Мы компонуем библиотеки с исполняемыми файлами с помощью команды `target_link_libraries()`. Не будь ее, сборка исполняемых файлов завершилась бы неудачно из-за неопределенных символов. Вы заметили, что эта команда вызывается до объявления библиотек? На этапе конфигурирования проекта CMake собирает информацию о целях и их свойствах: имена, зависимости, исходные файлы и другие сведения.

Разобрав все файлы, CMake пытается построить граф зависимостей. Как любой допустимый граф зависимостей, он должен быть **ориентированным** и **ациклическим** (**ОАГ**, *англ.* DAG). Это означает, что на каждом ребре проставлено направление зависимости, и при этом ребра не образуют циклов.

Когда мы выполняем `cmake` в режиме сборки, сгенерированная сборочная система проверяет, как определены цели верхнего уровня, и рекурсивно собирает их зависимости. Рассмотрим пример на рис. 5.1.

1. Начать сверху и собрать обе библиотеки в группе 1.
2. Когда библиотеки **Calculations** и **Drawing** будут собраны, собрать группу 2 – **GuiApp** и **TerminalApp**.
3. Собрать цель вычисления контрольной суммы; выполнить указанные команды для генерирования контрольных сумм (`cksum` – программа вы-

числения контрольной суммы в Unix, а это значит, что на других платформах этот пример не соберется).

Но есть одна мелкая проблема – описанное решение не гарантирует, что цель контрольной суммы будет собираться после сборки исполняемых файлов. CMake не знает, что контрольная сумма зависит от исполняемого двоичного файла, поэтому вправе с нее и начать. Чтобы решить эту проблему, мы можем поместить команду `add_dependencies()` в конец файла:

```
add_dependencies(checksum terminal_app gui_app)
```

Это гарантирует, что CMake поймет связь между контрольной суммой и исполняемыми файлами.

Замечательно, но в чем разница между `target_link_libraries()` и `add_dependencies()`? Команда `target_link_libraries()` предназначена для работы с реальными библиотеками и позволяет управлять распространением свойств. А вторая команда используется только с целями верхнего уровня для определения порядка их сборки.

По мере увеличения сложности проекта понять дерево зависимостей становится все труднее. Как можно упростить этот процесс?

Визуализация зависимостей

Даже если проект невелик, рассуждать о нем и разделять с другими разработчиками может быть сложно. Красивый рисунок может сделать очень многое для решения этой проблемы. Ведь одна картинка стоит тысячи слов. Мы могли бы нарисовать рисунок от руки, как на рис. 5.1. Но это утомительно, да и при каждом изменении проекта придется все перерисовать заново. По счастью, в состав CMake входит отличный модуль для генерирования графов зависимостей в формате `dot/graphviz`, который поддерживает как внутренние, так и внешние зависимости!

Чтобы им воспользоваться, нужно просто выполнить команду

```
cmake --graphviz=test.dot .
```

Модуль порождает текстовый файл, который можно импортировать в программу визуализации Graphviz, умеющую как отрисовывать изображение, так и создавать файл в формате PDF или SVG, сохраняемый как часть программной документации. Все любят хорошую документацию, но мало кто любит ее писать. Вот и не надо!

Специальные цели по умолчанию не показываются, и нам нужно создать специальный конфигурационный файл, `CMakeGraphVizOptions.cmake`, который позволяет настроить граф по своему вкусу. Команда `set(GRAPHVIZ_CUSTOM_TARGETS TRUE)` разрешает включать в граф специальные цели:

ch05/01-targets/CMakeGraphVizOptions.cmake

```
set(GRAPHVIZ_CUSTOM_TARGETS TRUE)
```

Другие параметры позволяют добавить имя графа, заголовок, префиксы вершин и указать, какие цели включать или исключать из результата (по имени или по типу). Смотрите полное описание этого модуля в документации по CMake.

Если вы торопитесь, то можете даже запустить Graphviz прямо в браузере, перейдя по адресу https://dreampuf.github.io/GraphvizOnline/. Нужно только скопировать и вставить содержимое файла `test.dot` в окно слева, и ваш проект будет визуализирован (рис. 5.2). Удобно, не правда ли?

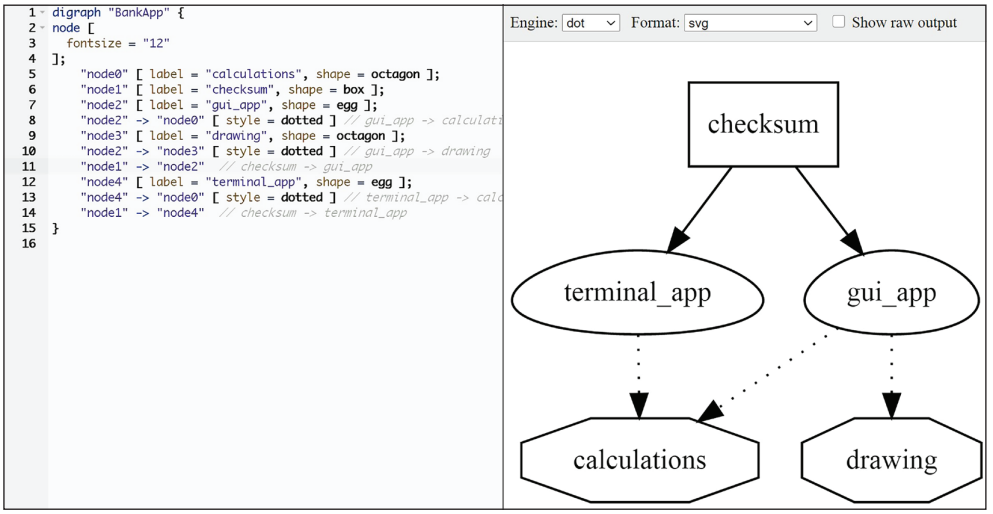


Рис. 5.2. Визуализация примера BankApp в Graphviz

Пользуясь этим методом, легко увидеть все явно определенные цели.

Теперь мы знаем, как определять различные типы целей, в т. ч. исполняемые, библиотечные и специальные, а также как создать и напечатать граф зависимостей. Воспользуемся этой информацией и посмотрим, как цели конфигурируются.

Задание свойств целей

У целей есть свойства, похожие на поля объектов в C++. Некоторые из них допускают модификацию, другие – только чтение. В CMake определен длинный список «известных свойств» (см. раздел «Для дополнительного чтения»), доступных в зависимости от типа цели (исполняемой, библиотечной или специальной). При желании можно добавить и собственные свойства. Для работы со свойствами цели служат команды:

```
get_target_property(<var> <target> <property-name>)
set_target_properties(<target1> <target2> ...
```

```
                    PROPERTIES <prop1-name> <value1>
                    <prop2-name> <value2> ...)
```

Чтобы напечатать свойство цели на экране, ее нужно сначала сохранить в переменной `<var>`, а затем включить в сообщение пользователю. Читать свойства следует по одному, а задавать можно сразу несколько для нескольких целей.



Свойства имеются не только у целей; CMake поддерживает задание свойств и в других контекстах: `GLOBAL`, `DIRECTORY`, `SOURCE`, `INSTALL`, `TEST` и `CACHE`. Для работы с любыми свойствами предназначены общие команды `get_property()` и `set_property()`. В некоторых проектах вы встретитесь с этими низкоуровневыми командами, которые делают в точности то же, что `set_target_properties()`, только чуть многословнее:

```
set_property(TARGET <target> PROPERTY <name> <value>)
```

В общем случае рекомендуется по возможности пользоваться высокоуровневыми командами. Иногда CMake предлагает сокращенные команды, снабженные дополнительными механизмами. Например, `add_dependencies(<target> <dep>)` – сокращенная форма добавления зависимостей в свойство цели `MANUALLY_ADDED_DEPENDENCIES`. И мы можем опросить его с помощью команды `get_target_property()` точно так же, как любое другое свойство. Однако изменить его командой `set_target_properties()` нельзя (оно доступно только для чтения), потому что CMake ограничивает использование команды `add_dependencies()` только дописыванием.

С дополнительными командами задания свойств мы познакомимся, когда будем обсуждать компиляцию и компоновку в последующих главах. А пока поговорим о том, как свойства одной цели можно перенести в другую.

Что такое транзитивные требования использования?

Согласимся, что придумывать имена трудно и иногда получается перл, который трудно понять. «Транзитивные требования использования» (Transitive Usage Requirements) – достойный сожаления пример загадочного заголовка в онлайновой документации по CMake. Давайте расшифруем это странное название и – чем черт не шутит – предложим термин, который понять проще.

Начнем со слова **использование**. Как уже было сказано, одна цель может зависеть от другой. В документации по CMake такая зависимость иногда называется **использованием** (usage), потому что одна цель *использует* другую.

Бывает, что такая *используемая цель* задает определенные *свойства* или *зависимости* для себя, а они, в свою очередь, оказываются **требованиями** (requirements) для *других целей*, которые ее используют: прикомпоновать какие-то библиотеки, включить каталог или потребовать некоторых возможностей от компилятора.

Последняя часть этой головоломки, **транзитивное**, корректно описывает это поведение (быть может, можно было бы и проще). CMake добавляет некоторые свойства (требования) *используемых целей* к свойствам *использующих целей*.

Можно сказать, что некоторые свойства могут переходить (распространяться) с одной цели на другую неявно, чтобы было проще выражать зависимости.

Мне эта концепция видится как **распространение свойств** между **исходной целью** (той, что используется) и **конечными целями** (теми, что используют другие цели).

Рассмотрим конкретный пример, чтобы понять, зачем все это нужно и как работает:

```
target_compile_definitions(<source> <INTERFACE|PUBLIC|PRIVATE>
[items1...])
```

Команда `target` устанавливает свойство `COMPILE_DEFINITIONS` цели `<source>`. **Определения компиляции** (compile definitions) – это просто флаги `-Dname=definition`, передаваемые компилятору и настраивающие константы препроцессора (мы будем говорить об этом в главе 7). Интерес здесь представляет второй аргумент. Нам нужно задать одно из трех значений, `INTERFACE`, `PUBLIC` или `PRIVATE`, которое управляет тем, каким целям следует передать это свойство. Но не путайте со спецификаторами доступа в C++ – это совершенно другая концепция.

Ключевые слова распространения работают следующим образом:

- `PRIVATE` устанавливает свойство исходной цели;
- `INTERFACE` устанавливает свойство конечных целей;
- `PUBLIC` устанавливает свойство начальных и конечных целей.

Если свойство не должно распространяться на конечные цели, задайте для него ключевое слово `PRIVATE`. Если должно, задайте `PUBLIC`. Если исходная цель не использует свойство в своих файлах реализации (с расширением `.cpp`), а только в заголовках, и эти заголовки передаются потребляющим целям, то используйте ключевое слово `INTERFACE`.

Как все это работает под капотом? Для управления этими свойствами CMake предоставляет несколько команд, в частности вышеупомянутую `target_compile_definitions()`. Если задано ключевое слово `PRIVATE` или `PUBLIC`, то CMake сохраняет переданные значения в свойстве цели, в данном случае `COMPILE_DEFINITIONS`. Если задано ключевое слово `INTERFACE` или `PUBLIC`, то значение сохраняется в свойстве с префиксом `INTERFACE_` – `INTERFACE_COMPILE_DEFINITIONS`. На этапе конфигурирования CMake читает интерфейсные свойства исходных целей и добавляет их содержимое в конечные цели. Вот мы и получили то, что хотели, – распространяемые свойства, или транзитивные требования использования в терминологии CMake.

Свойства, управляемые командой `set_target_properties()`, можно найти на странице https://cmake.org/cmake/help/latest/manual/cmake-properties.7.html, в разделе «Properties on Targets» (не все свойства цели транзитивны). Перечислим наиболее важные:

- COMPILE_DEFINITIONS;
- COMPILE_FEATURES;
- COMPILE_OPTIONS;
- INCLUDE_DIRECTORIES;
- LINK_DEPENDS;
- LINK_DIRECTORIES;
- LINK_LIBRARIES;
- LINK_OPTIONS;
- POSITION_INDEPENDENT_CODE;
- PRECOMPILE_HEADERS;
- SOURCES.

Мы обсудим большую часть этих свойств ниже, но все они, конечно, описаны в руководстве по CMake. Их подробные описания приведены на страницах https://cmake.org/cmake/help/latest/prop_tgt/<PROPERTY>.html (только замените <PROPERTY> именем интересующего вас свойства).

Следующий вопрос: как далеко распространяется свойство? Оно устанавливается только для первой конечной цели или распространяется до самой вершины графа зависимостей? Вам решать.

Чтобы создать зависимость между целями, используется команда `target_link_libraries()`. Ее полная сигнатура требует ключевого слова, описывающего распространение:

```
target_link_libraries(<target>
                      <PRIVATE|PUBLIC|INTERFACE> <item>...
                     [<PRIVATE|PUBLIC|INTERFACE> <item>...]...)
```

Это ключевое слово управляет тем, как свойства *исходной цели* сохраняются в *конечной цели*. На рис. 5.3 показано, что происходит с распространяемым свойством на этапе генерирования (по завершении этапа конфигурирования).

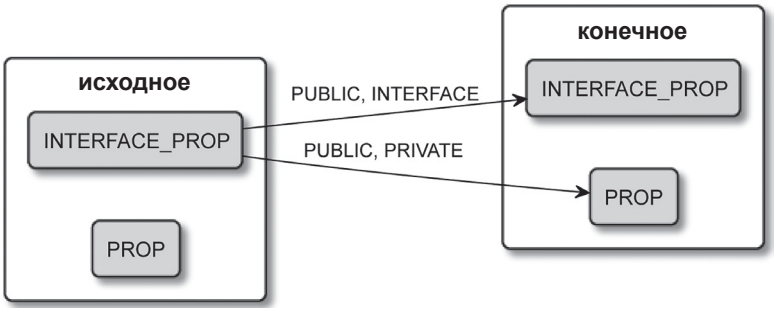


Рис. 5.3. Как свойства распространяются в конечные цели

Ключевые слова, описывающие распространение, работают следующим образом:

- PRIVATE добавляет исходное значение в **закрытое** свойство исходной цели;
- INTERFACE добавляет исходное значение в **интерфейсное** свойство исходной цели;
- PUBLIC добавляет в оба свойства исходной цели.

Как было сказано выше, интерфейсные свойства используются только для распространения свойств дальше по цепочке (до следующей *конечной цели*), а *исходная цель* не будет использовать их в процессе сборки.

В базовой команде `target_link_libraries(<target> <item>...)`, которой мы пользовались выше, неявно подразумевается ключевое слово PUBLIC.

Если вы правильно расставите ключевые слова для исходных целей, то свойства будут автоматически установлены в конечных целях – если только не возникнет конфликт...

Конфликты распространяемых свойств

Если одна цель зависит от нескольких других, может возникнуть ситуация, когда распространяемые свойства вступают в прямой конфликт друг с другом. Предположим, что одна используемая цель определяет свойство POSITION_INDEPENDENT_CODE как true, а другая – как false. CMake рассматривает это как конфликт и печатает сообщение вида

```
CMake Error: The INTERFACE_POSITION_INDEPENDENT_CODE property of "source_
target" does not agree with the value of POSITION_INDEPENDENT_CODE already
determined for "destination_target".
```

Видеть такое сообщение полезно, потому что мы понимаем, что конфликт имеет место и его нужно разрешить. В CMake есть и собственные свойства, для которых необходимо «согласие» между исходной и конечной целями.

В редких случаях это может оказаться важно – например, если вы собираете программу, в которой одна и та же библиотека используется в нескольких целях, компонуемых в один исполняемый файл. Если в этих исходных целях указаны разные версии библиотеки, то могут возникнуть проблемы.

Для гарантии того, что используется одна и та же версия, можно создать специальное интерфейсное свойство, INTERFACE_LIB_VERSION, и сохранить в нем версию. Этого недостаточно для решения проблемы, потому что CMake по умолчанию не распространяет специальные свойства (этот механизм работает только для встроенных целевых свойств). Мы должны явно добавить специальное свойство в список «совместимых» свойств.

У каждой цели есть четыре таких списка:

- COMPATIBLE_INTERFACE_BOOL;
- COMPATIBLE_INTERFACE_STRING;
- COMPATIBLE_INTERFACE_NUMBER_MAX;
- COMPATIBLE_INTERFACE_NUMBER_MIN.

Добавление свойства в любой из них инициирует проверки распространения и совместимости. Для списка BOOL проверяется, что все свойства, распро-

страняемые до конечной цели, имеют одно и то же булево значение. Аналогично для списка STRING проверяется равенство строковых значений. Списки NUMBER_MAX и NUMBER_MIN ведут себя иначе – распространяемые значения не обязаны совпадать, но конечная цель получит соответственно максимальное или минимальное значение.

Как это применяется на практике, рассмотрим на примере.

ch05/02-propagated/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(PropagatedProperties CXX)

add_library(source1 empty.cpp)
set_property(TARGET source1 PROPERTY INTERFACE_LIB_VERSION 4)
set_property(TARGET source1 APPEND PROPERTY
             COMPATIBLE_INTERFACE_STRING LIB_VERSION)

add_library(source2 empty.cpp)
set_property(TARGET source2 PROPERTY INTERFACE_LIB_VERSION 4)

add_library(destination empty.cpp)

target_link_libraries(destination source1 source2)
```

Здесь мы создаем три цели; для простоты во всех используется один и тот же пустой исходный файл. Для обеих *исходных целей* мы задаем специальное свойство с префиксом INTERFACE_ и записываем в него одну и ту же версию библиотеки. Обе *исходные цели* компонуются в конечную цель. Наконец, мы задаем требование совместимости STRING в качестве свойства для source1 (префикс INTERFACE_ не добавляется).

CMake распространит это специальное свойство до *конечной цели* и проверит, совпадают ли версии всех исходных целей (свойство совместимости может быть задано только для одной цели).

Разобравшись с нормальными целями, рассмотрим другие вещи, которые выглядят как цели, пахнут как цели и иногда даже ведут себя как цели, но целями все же не являются.

Знакомство с псевдоцелями

Понятие цели настолько полезно, что было бы неплохо перенести часть их поведения на другие вещи; те, что представляют не выходы, а входы сборочной системы: внешние зависимости, псевдонимы и т. д. Это псевдоцели, или цели, которые не попадают в сгенерированную сборочную систему:

- импортированные цели;
- псевдонимы целей;
- интерфейсные библиотеки.

Импортированные цели

Если вы хотя бы мельком проглядели оглавление этой книги, то знаете, что мы будем говорить о том, как CMake управляет внешними зависимостями – другими проектами, библиотеками и т. д. Импортированные цели – по существу, конечные продукты этого процесса. CMake может определить их как результат команды `find_package()`.

Вы можете подкорректировать свойства такой цели: **определения компиляции**, **параметры компиляции**, **каталоги включаемых файлов** и т. д., – и они даже будут поддерживать транзитивные требования использования. Однако обращаться с ними следует как с неизменяемыми: не пытайтесь модифицировать их источники или зависимости.

Область видимости импортированной цели может быть глобальной или локальной относительно каталога, в котором она определена (видима в его подкаталогах, но не в родительских каталогах).

Псевдонимы целей

Псевдонимы целей делают именно то, что вытекает из названия, – создают новую ссылку на цель под другим именем. Псевдонимы целей можно создавать для исполняемых файлов и библиотек с помощью следующих команд:

```
add_executable(<name> ALIAS <target>)
add_library(<name> ALIAS <target>)
```

Свойства псевдонимов целей допускают только чтение, их нельзя ни установить, ни экспортировать (в сгенерированной сборочной системе они не видны).

Тогда зачем они вообще нужны? Они полезны в случаях, когда часть проекта (например, подкаталог) требует цели с определенным именем, а фактическая реализация доступна под другими именами, зависящими от обстоятельств. Например, вы можете как собрать библиотеку и включить ее в свое решение, так и импортировать ее в зависимости от выбора пользователя.

Интерфейсные библиотеки

Это интересная конструкция – библиотека, которая ничего не компилирует, а играет роль служебной цели. Вся эта концепция строится вокруг распространяемых свойств (транзитивных требований использования). У интерфейсных библиотек есть два основных применения: представлять библиотеки, состоящие только из заголовков, и объединять несколько распространяемых свойств в одну логическую единицу.

Библиотеки, состоящие только из заголовков, легко создать командой `add_library(INTERFACE)`:

```
add_library(Eigen INTERFACE
  src/eigen.h src/vector.h src/matrix.h
```

```
)
target_include_directories(Eigen INTERFACE
  $<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/src>
  $<INSTALL_INTERFACE:include/Eigen>
)
```

В этом фрагменте мы создали интерфейсную библиотеку Eigen с тремя заголовочными файлами. Затем, используя **генераторные выражения** (они обозначаются знаком доллара и угловыми скобками, `$<...>`, и будут объяснены в следующей главе), мы задаем ее **каталоги включаемых файлов** как `${CMAKE_CURRENT_SOURCE_DIR}/src`, если цель экспортируется, и как `include/Eigen`, если устанавливается (объяснения будут даны в конце этой главы).

Для использования такой библиотеки нужно просто прикомпоновать ее:

```
target_link_libraries(executable Eigen)
```

Никакой компоновки здесь реально не происходит, но CMake воспринимает эту команду как запрос на распространение всех интерфейсных свойств до цели `executable`.

Во втором случае тот же механизм используется для другого – создать логическую цель, которая может служить местом для хранения распространяемых свойств. Эту цель можно использовать как зависимость для других целей и задавать свойства чистым и удобным способом. Вот пример:

```
add_library(warning_properties INTERFACE)
target_compile_options(warning_properties INTERFACE
  -Wall -Wextra -Wpedantic
)
target_link_libraries(executable warning_properties)
```

Команда `add_library(INTERFACE)` создает логическую цель `warning_properties`, которая служит для применения к цели `executable` **параметров компиляции**, заданных во второй команде. Я рекомендую использовать интерфейсные цели, потому что они делают код более удобным для чтения и повторного использования. Считайте, что это вынесение группы магических значений в переменную с осмысленным именем. Я также предлагаю явно добавлять суффикс типа `_properties`, чтобы было проще отличить интерфейсные библиотеки от обычных.

Объектные библиотеки

Объектные библиотеки применяются для группировки нескольких исходных файлов в одну логическую цель и компиляции их в *объектные файлы* (с расширением `.o`) в процессе сборки. Для создания *объектной библиотеки*

мы пользуемся тем же методом, что и для других библиотек, но с ключевым словом OBJECT:

```
add_library(<target> OBJECT <sources>)
```

Объектные файлы, порождаемые в процессе сборки, можно включить в качестве откомпилированных элементов в другие цели с помощью генераторного выражения $<TARGET_OBJECTS:objlib>:

```
add_library(... $<TARGET_OBJECTS:objlib> ...)
add_executable(... $<TARGET_OBJECTS:objlib> ...)
```

Или же можно добавить их в качестве зависимостей в команду target_link_libraries().

В контексте нашей библиотеки Calc *объектные библиотеки* будет полезны, чтобы избежать лишней компиляции исходных файлов для статической и разделяемой версии библиотеки. Важно компилировать *объектные файлы* с ключевым словом POSITION_INDEPENDENT_CODE, обязательным для разделяемых библиотек.

Вернемся к целям проекта: calc_obj поставляем откомпилированные объектные файлы, которые затем используются для библиотеки calc_static и calc_shared. Рассмотрим практические различия между этими двумя видами библиотек и разберемся, почему бывает разумно создавать обе.

Верно ли, что псевдоцелями концепция цели исчерпывается? Конечно, нет. Это было бы слишком просто. Нам еще предстоит понять, как эти цели используются для генерирования сборочных систем.

Цели сборки

Термин «цель» может иметь различные значения в зависимости от контекста внутри проекта и генерируемых сборочных систем. В контексте генерирования сборочной системы CMake «компилирует» списковые файлы, написанные на языке CMake, на язык выбранного инструмента сборки, например создает Makefile в случае GNU Make. В этих сгенерированных файлах Makefile имеется собственный набор целей. Одни из них – результат прямого преобразования целей, определенных в списковых файлах, другие создаются неявно как часть процесса генерирования сборочной системы.

Одной такой целью сборочной системы является ALL; по умолчанию CMake включает в нее все верхнеуровневые цели из спискового файла, например исполняемые файлы и библиотеки (необязательно специальные цели). ALL собирается при выполнении команды cmake --build <build tree> без указания конкретной цели. Напомним, что в первой главе мы говорили, что цель можно указать, добавив параметр --target <name> в команду сборки cmake.

Некоторые исполняемые файлы или библиотеки могут быть необходимы не для каждой сборки, но мы хотели бы сохранить их в составе проекта на тот

редкий случай, когда они могут оказаться полезны. Для оптимизации сборки по умолчанию можно было бы исключить их из цели ALL:

```
add_executable(<name> EXCLUDE_FROM_ALL [<source>...])
add_library(<name> EXCLUDE_FROM_ALL [<source>...])
```

Специальные цели ведут себя строго наоборот – по умолчанию они исключаются из цели ALL, а для включения нужно явно добавить их с помощью ключевого слова ALL, как мы и поступили в примере BankApp.

Другой неявно определенной целью сборки является clean, которая удаляет порожденные артефакты из дерева сборки. Мы будем пользоваться ей, чтобы избавиться от всех старых файлов и собрать все с чистого листа. Но важно понимать, что она не просто удаляет все из каталога сборки. Чтобы clean работала правильно, нужно вручную указать все файлы, которые ваши специальные цели могли создать в качестве побочных продуктов, BYPRODUCTS (см. пример BankApp).

На этом мы завершаем обзор целей и их различных аспектов: мы знаем, как их создавать, как конфигурировать их свойства, как использовать псевдоцели и как указать, нужно собирать их по умолчанию или нет. Существует также отличный от целей, но интересный механизм создания пользовательских артефактов, который можно использовать во всех настоящих целях, – **специальные команды** (не путайте со **специальными целями**).

Написание специальных команд

У специальных целей есть один недостаток – стоит добавить их в цель ALL или ввести зависимость от них в другие цели, как они будут собираться каждый раз. Иногда это именно то, что нужно, но бывает, что специальное поведение необходимо для порождения файлов, которые не должны пересоздаваться беспричинно:

- генерирование файла с исходным кодом, от которого зависит другая цель;
- трансляция с другого языка на C++;
- выполнение специального действия непосредственно до или после сборки другой цели.

У специальных команд имеется две возможные сигнатуры. Первая – расширенная версия add_custom_target():

```
add_custom_command(OUTPUT output1 [output2 ...]
  COMMAND command1 [ARGS] [args1...]
  [COMMAND command2 [ARGS] [args2...] ...]
  [MAIN_DEPENDENCY depend]
  [DEPENDS [depends...]]
  [BYPRODUCTS [files...]]
```

```
  [IMPLICIT_DEPENDS <lang1> depend1
                   [<lang2> depend2] ...]
  [WORKING_DIRECTORY dir]
  [COMMENT comment]
  [DEPFILE depfile]
  [JOB_POOL job_pool]
  [VERBATIM] [APPEND] [USES_TERMINAL]
  [COMMAND_EXPAND_LISTS])
```

Как нетрудно догадаться, специальная команда не создает логическую цель, а, как и специальная цель, должна быть добавлена в граф зависимостей. Сделать это можно двумя способами – используя ее выходной артефакт как исходный файл для исполняемого файла (или библиотеки) или явно добавив в список `DEPENDS` для специальной цели.

Использование специальной команды в качестве генератора

Конечно, не в каждом проекте нужно генерировать код на C++ на основе других файлов. Один такой случай – компиляция `proto`-файлов **буфера протокола Google** (**Protobuf**). Для тех, кто незнаком с этой библиотекой, скажем, что Protobuf – платформенно-независимый сериализатор структурированных данных.

Иными словами, ее можно использовать для кодирования объектов в двоичные потоки – файлы или сетевые соединения – и обратно. Чтобы обеспечить кросс-платформенность и одновременно быстроту Protobuf, инженеры из Google придумали специальный язык определения моделей, которые сохраняются в `proto`-файлах, например:

```
message Person {
  required string name = 1;
  required int32 id = 2;
  optional string email = 3;
}
```

Такой файл можно затем использовать для трансляции на разные языки: C++, Ruby, Go, Python, Java и другие. Google предоставляет компилятор, `protoc`, который читает `proto`-файлы и порождает код определения и сериализации структур на выбранном языке (который впоследствии нужно будет компилировать или интерпретировать). Разумный инженер будет хранить в репозитории не эти сгенерированные исходные файлы, а файлы в формате Protobuf, и добавит шаг генерирования в цепочку сборки.

Мы еще не знаем, как узнать, есть ли на целевом компьютере компилятор Protobuf и где он находится (но узнаем в главе 9). Пока просто предположим,

что компилятор protoc находится в известном месте системы. Мы подготовили файл person.proto и знаем, что компилятор Protobuf выведет файлы person.pb.h и person.pb.cc. Вот как определяется специальная команда для их компиляции:

```
add_custom_command(OUTPUT person.pb.h person.pb.cc
  COMMAND protoc ARGS person.proto
  DEPENDS person.proto
)
```

Затем, чтобы разрешить сериализацию в нашем исполняемом файле, нужно просто добавить результирующие файлы в состав исходных:

```
add_executable(serializer serializer.cpp person.pb.cc)
```

В предположении, что мы правильно включили заголовочные файлы и скомпоновали с библиотекой Protobuf, все будет автоматически компилироваться и обновляться при внесении изменений в proto-файл.

Упрощенный (и далеко не столь практически полезный) пример – создать необходимый заголовочный файл путем копирования его из другого места:

ch05/03-command/CMakeLists.txt

```
add_executable(main main.cpp constants.h)
target_include_directories(main PRIVATE ${CMAKE_BINARY_DIR})
add_custom_command(OUTPUT constants.h COMMAND cp
                   ARGS "${CMAKE_SOURCE_DIR}/template.xyz" constants.h)
```

В этом случае «компилятором» является команда cp. Она разрешает зависимость в цели main, создавая файл constants.h в корне дерева сборки путем копирования из дерева исходного кода.

Использование специальной команды в качестве точки вклинивания

Во второй версии команды add_custom_command() реализован механизм выполнения команд до или после сборки цели:

```
add_custom_command(TARGET <target>
  PRE_BUILD | PRE_LINK | POST_BUILD
  COMMAND command1 [ARGS] [args1...]
  [COMMAND command2 [ARGS] [args2...] ...]
  [BYPRODUCTS [files...]]
  [WORKING_DIRECTORY dir]
  [COMMENT comment]
```

```
  [VERBATIM] [USES_TERMINAL]
  [COMMAND_EXPAND_LISTS])
```

В первом аргументе мы указываем, какую цель хотели бы «дополнить» новым поведением при следующих условиях:

- `PRE_BUILD` будет выполняться раньше всех остальных правил для этой цели (только для генераторов Visual Studio; для остальных ведет себя как `PRE_LINK`);
- `PRE_LINK` ставит подлежащую выполнению команду после компиляции всех исходных файлов, но до компоновки (или архивирования) цели. Для специальных целей не работает;
- `POST_BUILD` выполняется после всех остальных правил для этой цели.

Воспользовавшись этой версией `add_custom_command()`, мы можем повторить генерирование контрольной суммы из примера BankApp:

ch05/04-command/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Command CXX)
add_executable(main main.cpp)
add_custom_command(TARGET main POST_BUILD
                   COMMAND cksum
                   ARGS "$<TARGET_FILE:main>" > "main.ck")
```

После сборки выполняемого файла `main` CMake выполнит команду `cksum` с заданными аргументами. Но что означает первый аргумент? Это не переменная, т. к. она не заключена ни в фигурные скобки (`${}`), ни в угловые (`$<>`). Это **генераторное выражение**, результатом вычисления которого является полный путь к двоичному файлу цели. Данный механизм находит применение в контексте многих свойств целей, и мы рассмотрим его в следующей главе.

Резюме

Понимать, что такое цели, критически важно для написания чистых современных проектов CMake. В этой главе мы не только обсудили смысл и назначение целей и порядок определения трех типов целей: исполняемых, библиотечных и специальных, но также объяснили, как цели зависят друг от друга и как это выражается с помощью графа зависимостей. Мы также научились визуализировать этот граф с помощью модуля Graphviz. Опираясь на понимание основ, мы рассказали о ключевой особенности целей – свойствах. Мы не только рассмотрели несколько команд для задания обычных свойств целей, но и сорвали покров тайны с транзитивных требований использования, или распространяемых свойств.

Это был крепкий орешек, поскольку нам пришлось разобраться в двух вещах: как управлять тем, какие свойства распространяются, и как распро-

странение влияет на последующие цели. Мы также узнали, как гарантировать совместимость свойств, полученных из разных источников.

Затем мы кратко обсудили псевдоцели: импортированные цели, псевдонимы целей и интерфейсные библиотеки. Все они пригодятся позже в наших проектах, особенно когда мы будем знать, как связать их с распространяемыми свойствами. Далее мы поговорили о генерируемых целях сборки и о том, как на них влияет этап конфигурирования. После этого мы потратили некоторое время на изучение механизма, похожего на цели, но все же отличающегося от них: специальных команд. Мы пояснили, как они могут генерировать файлы, потребляемые другими целями (для компиляции, трансляции и т. д.), и об их функции точки вклинивания: выполнения дополнительных шагов при сборке цели.

Заложив такой прочный фундамент, мы готовы перейти к следующей теме – компиляции исходных файлов на C++ в исполняемые файлы и библиотеки.

Для дополнительного чтения

Дополнительные сведения по рассмотренным в этой главе темам можно получить из следующих источников:

- документация по модулю Graphviz: https://gitlab.kitware.com/cmake/community/-wikis/doc/cmake/Graphviz, https://cmake.org/cmake/help/latest/module/CMakeGraphVizOptions.html;
- программа Graphviz: https://graphviz.org;
- свойства целей в CMake: https://cmake.org/cmake/help/latest/manual/cmakeproperties.7.html#properties-on-targets;
- транзитивные требования использования: https://cmake.org/cmake/help/latest/manual/cmakebuildsystem.7.html#transitive-usage-requirements.

6

Генераторные выражения

Многие пользователи CMake никогда не встречались с генераторными выражениями в своих проектах, потому что это продвинутый механизм. Однако они очень важны для проектов, которые ориентированы на широкую аудиторию, т. к. играют ключевую роль в процедурах экспорта, установки и упаковки. Если вы хотите быстро познакомиться с одними лишь основами CMake с упором на C++, то можете пропустить эту главу и вернуться к ней позднее. С другой стороны, мы обсуждаем генераторные выражения именно здесь, потому что последующие главы опираются на эти знания при объяснении более сложных аспектов CMake.

Мы начнем с введения в генераторные выражения: что это такое, зачем используются, как формируются и расширяются. Далее представим краткий обзор механизма вложенности и более подробное описание условного расширения, которое позволяет использовать булеву логику, операции сравнения и запросы. Конечно, мы не пройдем мимо широкого разнообразия имеющихся выражений.

Но сначала мы изучим преобразования строк, списков и путей, чтобы расчистить себе путь к главному предмету обсуждения. На практике генераторные выражения используются для получения информации, которая должна быть доступна на более поздних этапах сборки, и представления ее в надлежащем контексте. Определение этого контекста составляет немалую часть их ценности. Мы узнаем, как параметризовать процесс сборки в зависимости от конфигурации, выбранной пользователем, платформы и комплекта инструментов. То есть мало знать, какой компилятор какой версии используется и какими возможностями он обладает; мы узнаем, как опрашивать свойства целей сборки и ассоциированную с ними информацию.

Чтобы вы в полной мере оценили ценность генераторных выражений, я включил несколько интересных примеров их использования в последнюю часть главы. И еще краткое объяснение того, как увидеть результат генераторного выражения, поскольку тут есть свои хитрости. Но не волнуйтесь, ге-

нераторные выражения не так сложны, как может показаться, и очень скоро вы освоитесь с ними.

В этой главе будут рассмотрены следующие вопросы:

- что такое генераторные выражения;
- основные синтаксические правила составления выражения общего вида;
- условное расширение;
- запросы и преобразования;
- примеры.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch06.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Что такое генераторные выражения?

CMake собирает решение в три этапа: конфигурирование, генерирование и выполнение инструмента сборки. В общем случае все требуемые данные доступны на этапе конфигурирования. Но иногда встречается парадокс «курицы и яйца». Возьмем пример из раздела «Использование специальной команды в качестве точки вклинивания» главы 5, где одна цель должна знать путь к двоичному артефакту другой цели. К сожалению, эта информация становится доступна только после того, как все списковые файлы разобраны и этап конфигурирования завершен.

И как же решить эту проблему? Можно, например, создать место для хранения информации и отложить ее вычисление до следующего этапа – **генерирования**.

Именно это и делают генераторные выражения. Они построены на базе таких свойств, как `LINK_LIBRARIES`, `INCLUDE_DIRECTORIES`, `COMPILE_DEFINITIONS`, и рас-

пространяемых свойств, хотя и не всех. Они следуют правилам, похожим на условные предложения и вычисление переменных.

Существует немало генераторных выражений, и в каком-то смысле они образуют отдельный предметно-ориентированный язык, поддерживающий условные выражения, логические операции, сравнения, преобразования, запросы и упорядочение. Благодаря генераторным выражениям мы можем работать со строками, списками, номерами версий, путями, конфигурациями и целями сборки. В этой главе мы дадим краткий обзор этих концепций. В центре нашего внимания будет главное применение генераторных выражений – сбор информации о сгенерированной конфигурации целей и о состоянии среды сборки. Полное описание см. в официальной документации по CMake (см. URL-адрес в разделе «Для дополнительного чтения»).

Лучше всего объяснять на примере, поэтому к нему и перейдем и опишем синтаксис генераторных выражений.

Основные синтаксические правила составления выражения общего вида

Чтобы использовать генераторные выражения, их необходимо добавить в списковый файл CMake командой, которая поддерживает вычисление генераторных выражений. К таковым относится большинство целеориентированных команд и многие другие (дополнительные сведения см. в официальной документации по конкретной команде). В сочетании с генераторными выражениями часто используется команда `target_compile_definitions()`, которой нужно передать аргумент, как показано ниже:

```
target_compile_definitions(foo PUBLIC BAR=$<TARGET_FILE:baz>)
```

Эта команда добавляет к аргументам компилятора флаг `-D` (пока не обращайте внимания на `PUBLIC`), записывающий в константу препроцессора `BAR` *путь*, по которому должен быть создан **двоичный артефакт** цели `foo`. Это работает, потому что генераторное выражение хранится в переменной в своей текущей форме. Расширение откладывается до этапа генерирования, когда многие вещи уже сконфигурированы и известны.

Как формируется генераторное выражение?

Как видно на рис. 6.1, структура кажется вполне простой и удобочитаемой:

- открыть выражение знаком доллара и угловой скобкой (`$<`);
- добавить имя выражения `EXPRESSION`;
- если выражение требует аргументов, то добавить двоеточие (`:`), а после него аргументы `arg1`, `arg2` ... `argN`, разделенные запятыми;
- закрыть выражение знаком `>`.

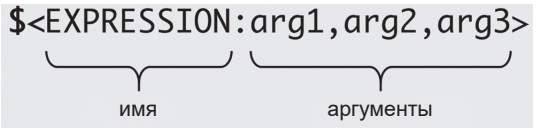


Рис. 6.1. Синтаксис генераторного выражения

Существуют выражения, не требующие аргументов, например `$<PLATFORM_ID>`.

Отметим, что если явно не оговорено противное, то выражение обычно вычисляется в контексте использующей его цели. Эта ассоциация выводится из команды, в которой используется выражение. В примере выше мы видели, что команде `target_compile_definitions()` передается `foo` – цель, к которой она применяется. Поэтому целеориентированные генераторные выражения, используемые в этой команде неявно, будут использовать `foo`. Заметим, однако, что в этом примере генераторное выражение, `$<TARGET_FILE>`, требует атрибута цели в качестве контекста, в котором производит операции. Другие генераторные выражения могут и не принимать целей в качестве аргументов (например, `$<COMPILE_LANGUAGE>`) и будут неявно использовать цель объемлющей команды. Мы обсудим этот вопрос подробнее ниже.

При использовании продвинутых возможностей генераторные выражения очень быстро могут стать весьма сложными для понимания, поэтому важно с самого начала понять основные принципы.

Вложенность

Начнем с возможности передавать генераторное выражение в качестве аргумента другому генераторному выражению. Иначе говоря, генераторные выражения могут быть вложенными:

```
$<UPPER_CASE:$<PLATFORM_ID>>
```

Это не очень сложный пример, но легко представить, что случится, если увеличить число уровней вложенности и работать с командами, принимающими несколько аргументов.

А если этого мало, то можно добавить еще и расширение обычных переменных:

```
$<UPPER_CASE:${my_variable}>
```

Сначала, на этапе конфигурирования, будет подставлена переменная `my_variable`. А на этапе генерирования будет расширено генераторное выражение. У этой возможности есть некоторые применения, но я настоятельно рекомендую избегать ее: генераторные выражения предлагают практически все необходимые функции. Включение в них еще и переменных созда-

ет лишний уровень косвенности, который трудно отлаживать. Кроме того, информация, собранная на этапе конфигурирования, часто оказывается устаревшей, поскольку пользователь может переопределить значения, используемые в генераторных выражениях, с помощью параметров командной строки на этапе сборки или установки.

Разобравшись с синтаксисом, перейдем к фундаментальным механизмам генераторных выражений.

Условное расширение

Для определения того, нужно ли расширять выражения, применяется булева логика. Возможность, конечно, замечательная, но по историческим причинам ее синтаксис временами противоречив и воспринимается с трудом. Она доступна в двух формах. Первая поддерживает «счастливую» и «печальную» ветви:

```
$<IF:condition,true_string,false_string>
```

Выражение `IF` опирается на вложенность (иначе оно было бы бесполезно): вы можете заменить любой аргумент другим выражением, реализовав тем самым довольно сложные вычисления (можно даже вкладывать одно условие `IF` в другое). В этой форме должно быть ровно три аргумента, т. е. опускать ничего нельзя. Лучшее, что мы можем сделать, если хотим пропустить значение в случае, когда условие не выполнено, – это:

```
$<IF:condition,true_string,>
```

Сокращенная версия позволяет опустить ключевое слово `IF` и запятую:

```
$<condition:true_string>
```

Как видите, здесь нарушено соглашение о включении имени `EXPRESSION` в качестве первой лексемы. Я думаю, что авторы хотели сократить выражение и избежать ввода этих нескольких символов, но получившийся результат вряд ли кому-то понравится. Вот пример из документации:

```
$<$<AND:$<COMPILE_LANGUAGE:CXX>,$<CXX_COMPILER_ID:AppleClang,Clang>>:
 COMPILING_CXX_WITH_CLANG>
```

Это выражение возвращает `COMPILING_CXX_WITH_CLANG` только для кода на C++, откомпилированного одним из компиляторов Clang (во всех остальных случаях возвращается пустая строка). Я бы хотел, чтобы синтаксис условия был такой же, как в обычной команде `IF`, но, к сожалению, это не так. Так

что вы теперь опознаете вторую форму, если когда-нибудь встретите ее, но избегайте ее в своих проектах.

Вычисление булева значения

Результатом вычисления генераторного выражения является значение одного из двух типов: булево или строковое. Булево значение представляется как `1` (истина) или `0` (ложь). Числовых типов не существует; все, что не является булевым значением, – строка.

Важно помнить, что вложенные выражения, переданные в качестве условий в условных выражениях, должны при вычислении давать булево значение.

Булево значение можно неявно преобразовать в строку, но для обратного преобразования необходим явный оператор `BOOL` (будет объяснен ниже).

Есть три категории выражений, вычисление которых дает булево значение: логические операторы, выражения сравнения и запросы.

Логические операторы

Имеется четыре логических оператора:

- `$<NOT:arg>`: отрицание булева значения;
- `$<AND:arg1,arg2,arg3...>`: возвращает `true`, если все аргументы равны `true`;
- `$<OR:arg1,arg2,arg3...>`: возвращает `true`, если хотя бы один аргумент равен `true`;
- `$<BOOL:string_arg>`: преобразовывает аргумент из строкового типа в булев.

Преобразование строки оператором `$<BOOL>` дает `true` (`1`), если не выполняется ни одно из следующих условий:

- строка пуста;
- строка равна (без учета регистра) `0`, `FALSE`, `OFF`, `N`, `NO`, `IGNORE` или `NOTFOUND`;
- строка заканчивается суффиксом `-NOTFOUND` (с учетом регистра).

Сравнения

Результат сравнения равен `1`, если сравнение истинно, и `0` в противном случае. Вот наиболее употребительные операции, которые могут показаться вам полезными:

- `$<STREQUAL:arg1,arg2>`: сравнение строк с учетом регистра;
- `$<EQUAL:arg1,arg2>`: преобразование строки в число и сравнение на равенство;
- `$<IN_LIST:arg,list>`: проверяет, присутствует ли элемент `arg` в списке `list` (с учетом регистра);
- `$<VERSION_EQUAL:v1,v2>`, `$<VERSION_LESS:v1,v2>`, `$<VERSION_GREATER:v1,v2>`, `$<VERSION_LESS_EQUAL:v1,v2>` и `$<VERSION_GREATER_EQUAL:v1,v2>` сравнивают номера версий покомпонентно;

- `$<PATH_EQUAL:path1,path2>`: лексические представления двух путей сравниваются без нормализации (начиная с версии CMake 3.24).

Запросы

Запросы возвращают булево значение переменной или результата операции. Один из простейших запросов выглядит так:

```
$<TARGET_EXISTS:arg>
```

Как легко догадаться, он возвращает `true`, если цель была определена на этапе конфигурирования.

Теперь вы знаете, как применять условное расширение, использовать логические операторы, сравнение и простые запросы. Это полезно само по себе, но генераторные выражения могут предложить гораздо больше, особенно в контексте запросов: их можно использовать в условном расширении `IF` или самостоятельно, в качестве аргументов команд. Пора познакомиться с ними в подходящем контексте.

Запросы и преобразования

Есть много генераторных выражений, но, чтобы не затеряться, сосредоточимся на самых употребительных. Начнем с простых преобразований имеющихся данных.

Работа со строками, списками и путями

Генераторные выражения предоставляют лишь минимум операций для преобразования и опроса структур данных. При работе со строками на этапе генерирования можно использовать следующие выражения:

- `$<LOWER_CASE:string>`, `$<UPPER_CASE:string>`: преобразует строку в указанный регистр.

Операции со списками до недавнего времени были довольно ограничены. Начиная с версии CMake 3.15 были доступны следующие операции:

- `$<IN_LIST:string,list>`: возвращает `true`, если список `list` содержит значение `string`;
- `$<JOIN:list,d>`: конкатенирует значения в списке `list`, используя `d` в качестве разделителя;
- `$<REMOVE_DUPLICATES:list>`: исключает дубликаты из списка (без сортировки);
- `$<FILTER:list,INCLUDE|EXCLUDE,regex>`: включает или исключает элементы из списка `list`, используя регулярное выражение `regex`.

Начиная с версии 3.27 были добавлены генераторные выражения `$<LIST:OPERATION>`, где `OPERATION` может принимать следующие значения:

- `LENGTH`;
- `GET`;
- `SUBLIST`;
- `FIND`;
- `JOIN`;
- `APPEND`;
- `PREPEND`;
- `INSERT`;
- `POP_BACK`;
- `POP_FRONT`;
- `REMOVE_ITEM`;
- `REMOVE_AT`;
- `REMOVE_DUPLICATES`;
- `FILTER`;
- `TRANSFORM`;
- `REVERSE`;
- `SORT`.

Довольно редко в генераторных выражениях приходится работать со списками, поэтому мы только сообщаем, какие есть возможности. Если вы столкнетесь с такой ситуацией, поищите в онлайновом руководстве инструкции по работе с этими операциями.

Наконец, мы можем опрашивать и преобразовывать системные пути. Это полезное дополнение благодаря независимости от операционной системы. Начиная с версии CMake 3.24 доступны следующие простые запросы:

- `$<PATH:HAS_ROOT_NAME,path>`;
- `$<PATH:HAS_ROOT_DIRECTORY,path>`;
- `$<PATH:HAS_ROOT_PATH,path>`;
- `$<PATH:HAS_FILENAME,path>`;
- `$<PATH:HAS_EXTENSION,path>`;
- `$<PATH:HAS_STEM,path>`;
- `$<PATH:HAS_RELATIVE_PART,path>`;
- `$<PATH:HAS_PARENT_PATH,path>`;
- `$<PATH:IS_ABSOLUTE,path>`;
- `$<PATH:IS_RELATIVE,path>`;
- `$<PATH:IS_PREFIX[,NORMALIZE],prefix,path>`: возвращает `true`, если `prefix` является префиксом пути.

Аналогично мы можем извлечь все компоненты пути, которые удалось найти (начиная с версии CMake 3.27 стало возможно указывать список путей, а не только один путь):

- `$<PATH:GET_ROOT_NAME,path...>`;
- `$<PATH:GET_ROOT_DIRECTORY,path...>`;
- `$<PATH:GET_ROOT_PATH,path...>`;
- `$<PATH:GET_FILENAME,path...>`;
- `$<PATH:GET_EXTENSION[,LAST_ONLY],path...>`;

- `$<PATH:GET_STEM[,LAST_ONLY],path...>`;
- `$<PATH:GET_RELATIVE_PART,path...>`;
- `$<PATH:GET_PARENT_PATH,path...>`.

Кроме того, в версии 3.24 были добавлены некоторые операции преобразования; для полноты просто перечислим их:

- `$<PATH:CMAKE_PATH[,NORMALIZE],path...>`;
- `$<PATH:APPEND,path...,input,...>`;
- `$<PATH:REMOVE_FILENAME,path...>`;
- `$<PATH:REPLACE_FILENAME,path...,input>`;
- `$<PATH:REMOVE_EXTENSION[,LAST_ONLY],path...>`;
- `$<PATH:REPLACE_EXTENSION[,LAST_ONLY],path...,input>`;
- `$<PATH:NORMAL_PATH,path...>`;
- `$<PATH:RELATIVE_PATH,path...,base_directory>`;
- `$<PATH:ABSOLUTE_PATH[,NORMALIZE],path...,base_directory>`.

Имеется еще одна операция, которая форматирует переданный путь в соответствии со стилем, поддерживаемым оболочкой хост-системы: `$<SHELL_PATH:path...>`.

Все эти выражения показаны для ссылок впоследствии, запоминать их прямо сейчас необязательно. Практически полезные сведения подробно излагаются в следующих разделах.

Параметризация конфигурирования и платформы сборки

Для сборки проекта пользователь должен предоставить CMake информацию о желаемой конфигурации. В большинстве случаев это `Debug` или `Release`. Для получения этих значений мы можем использовать следующие генераторные выражения:

- `$<CONFIG>`: возвращает текущую конфигурацию сборки в виде строки: `Debug`, `Release` или еще что-то;
- `$<CONFIG:configs>`: возвращает `true`, если `configs` содержит текущую конфигурацию сборки (сравнение без учета регистра).

Мы обсуждали платформу в главе 4. Прочитать относящуюся к ней информацию можно так же, как конфигурацию:

- `$<PLATFORM_ID>`: возвращает идентификатор текущей платформы в виде строки: `Linux`, `Windows` или `Darwin` для macOS;
- `$<PLATFORM_ID:platform>` равно `true`, если `platform` совпадает с идентификатором текущей платформы.

Такая параметризация конфигурации или платформы – полезное дополнение к вашему арсеналу. Мы можем использовать ее в сочетании с рассмотренным выше условным расширением:

```
$<IF:condition,true_string,false_string>
```

Например, можно задать один флаг компилятора при сборке тестовых двоичных файлов и другой – при сборке рабочих.

```
target_compile_definitions(my_target PRIVATE
                           $<IF:$<CONFIG:Debug>,Test,Production>
)
```

Но это только начало. Есть много других ситуаций, в которых генераторные выражения оказываются бесценным подспорьем. Разумеется, следующим важным аспектом будет присутствующий в системе инструментарий.

Настройка комплекта инструментов

Комплекты инструментов, или попросту компиляторы и компоновщики (к сожалению?), не одинаковы у разных поставщиков. Отсюда вытекают самые разные следствия. Одни из них полезны (более высокая производительность в частных случаях), другие – не очень (различные нюансы конфигурирования, различные имена флагов и т. д.).

Генераторные выражения здесь помогают тем, что с их помощью можно сформулировать серию запросов, которые смягчают проблемы и в ряде случаев делают работу пользователей удобнее.

Как и в случае конфигурации и платформы сборки, имеется несколько готовых выражений, которые возвращают сведения о комплекте инструментов – в виде строковых и булевых значений. Однако требуется указать, какой язык нас интересует (замените `#LNG` одной из строк `C`, `CXX`, `CUDA`, `OBJC`, `OBJCXX`, `Fortran`, `HIP` или `ISPC`). Поддержка `HIP` была добавлена в версии 3.21.

- `$<#LNG_COMPILER_ID>`: возвращает идентификатор CMake используемого компилятора языка `#LNG`;
- `$<#LNG_COMPILER_VERSION>`: возвращает версию используемого компилятора языка `#LNG`.

Чтобы узнать, какой компилятор будет использован для C++, нужно воспользоваться генераторным выражением `$<CXX_COMPILER_ID>`. Возвращаемый идентификатор компилятора CMake – константа, определенная для каждого поддерживаемого компилятора. Вы можете встретить такие значения, как `AppleClang`, `ARMCC`, `Clang`, `GNU`, `Intel` и `MSVC`. Полный перечень см. в официальной документации.

Как и в предыдущем разделе, информацию о комплекте можно использовать в условных выражениях. Существует несколько запросов, возвращающих `true`, если хотя бы один аргумент совпадает с заданным значением:

- `$<#LNG_COMPILER_ID:ids>`: возвращает `true`, если список `ids` содержит идентификатор компилятора CMake для языка `#LNG`;
- `$<#LNG_COMPILER_VERSION:vers>`: возвращает `true`, если содержит версию компилятора CMake для языка `#LNG`;

- `$<COMPILE_FEATURES:features>`: возвращает `true`, если все возможности, перечисленные в списке `features`, поддерживаются компилятором для этой цели.

В командах, требующих задания цели в качестве аргумента, например `target_compile_definitions()`, можно использовать одно из целеориентированных выражений для получения строкового значения:

- `$<COMPILE_LANGUAGE>`: возвращает язык исходных файлов на этапе компиляции;
- `$<LINK_LANGUAGE>`: возвращает язык исходных файлов на этапе компоновки.

Для вычисления простого булева запроса:

- `$<COMPILE_LANGUAGE:langs>`: возвращает `true`, если `langs` содержит язык, используемый для компиляции этой цели. Это можно использовать для задания флагов компилятора, зависящих от языка. Например, чтобы откомпилировать указанные в цели исходные файлы на C++ с флагом `-fno-exceptions`, следует написать:

```
target_compile_options(myapp
   PRIVATE $<$<COMPILE_LANGUAGE:CXX>:-fno-exceptions>
)
```

- `$<LINK_LANGUAGE:langs>` – следует тем же правилам, что `COMPILE_LANGUAGE`, и возвращает `true`, если `langs` содержит язык, используемый для компоновки этой цели.

Или для опроса в более сложных сценариях:

- `$<COMPILE_LANG_AND_ID:lang,compiler_ids...>`: возвращает `true`, если для этой цели используется язык `lang`, и для его компиляции будет использован один из компиляторов в списке `compiler_ids`. Это выражение полезно для задания определений константы для конкретных компиляторов:

```
target_compile_definitions(myapp PRIVATE
   $<$<COMPILE_LANG_AND_ID:CXX,AppleClang,Clang>:CXX_CLANG>
   $<$<COMPILE_LANG_AND_ID:CXX,Intel>:CXX_INTEL>
   $<$<COMPILE_LANG_AND_ID:C,Clang>:C_CLANG>
)
```

  В этом примере для исходных файлов на C++ (`CXX`), компилируемых компилятором `AppleClang` или `Clang`, будет задано определение `-DCXX_CLANG`. Для исходных файлов на C++, компилируемых компилятором `Intel`, будет задано определение `-DCXX_INTEL`. И наконец, для исходных файлов на C (`C`), компилируемых компилятором `Clang`, будет задано определение `-DC_CLANG`;
- `$<LINK_LANG_AND_ID:lang,compiler_ids...>`: похоже на `COMPILE_LANG_AND_ID`, но проверяет язык, используемый на этапе компоновки. Используй-

те это выражение для задания библиотек, параметров компоновщика, каталогов и зависимостей компоновки для конкретной комбинации языка и компоновщика в цели.

Важное замечание: одна цель может быть комбинацией исходных файлов на нескольких языках. Например, можно скомпоновать артефакты компиляции файлов на C и C++ (но необходимо объявить оба языка в команде `project()`). Поэтому генераторные выражения, ссылающиеся на некоторый язык, будут использоваться для одних исходных файлов и не будут для других.

Теперь перейдем к следующей важной категории: целеориентированные генераторные выражения.

Запрос целеориентированной информации

Существует много генераторных выражений, которые запрашивают свойства цели и проверяют относящуюся к цели информацию. Отметим, что до версии CMake 3.19 многие целеориентированные выражения, ссылающиеся на другую цель, использовались для автоматического создания зависимости между ними. В последних версиях такого не происходит.

Некоторые генераторные выражения выводят цель из вызванной команды; из них самым известным является простой запрос, возвращающий значение некоторого свойства цели:

```
$<TARGET_PROPERTY:prop>
```

- Менее известным, но полезным в команде `target_link_libraries()` является генераторное выражение `$<LINK_ONLY:deps>`. Оно позволяет сохранить зависимости компоновки `PRIVATE`, которые не будут распространяться с помощью транзитивных требований использования; они применяются в интерфейсных библиотеках, которые мы обсуждали в главе 5.

Существует также ряд выражений, связанных с установкой и экспортом, которые выводят цели из контекста, в котором используются. Мы будем обсуждать их в главе 14, а пока ограничимся кратким введением.

- `$<INSTALL_PREFIX>`: возвращает префикс установки, когда цель экспортируется командой `install(EXPORT)` или вычисляется в `INSTALL_NAME_DIR`; в противном случае префикс пустой;
- `$<INSTALL_INTERFACE:string>`: возвращает `string`, когда выражение экспортируется командой `install(EXPORT)`;
- `$<BUILD_INTERFACE:string>`: возвращает `string`, когда выражение экспортируется командой `export()` или другой целью в той же самой сборочной системе;
- `$<BUILD_LOCAL_INTERFACE:string>`: возвращает `string`, когда выражение экспортируется другой целью в той же самой сборочной системе.

Однако для большинства запросов требуется, чтобы имя цели было явно задано в первом аргументе.

- `$<TARGET_EXISTS:target>`: возвращает `true`, если цель `target` существует;
- `$<TARGET_NAME_IF_EXISTS:target>`: возвращает имя цели `target`, если она существует, и пустую строку в противном случае;
- `$<TARGET_PROPERTY:target,prop>`: возвращает значение свойства `prop` для цели `target`;
- `$<TARGET_OBJECTS:target>`: возвращает список *объектных файлов* для цели, являющейся *объектной библиотекой*.

Можно запросить путь к артефакту цели.

- `$<TARGET_FILE:target>`: возвращает полный путь;
- `$<TARGET_FILE_NAME:target>`: возвращает только имя файла;
- `$<TARGET_FILE_BASE_NAME:target>`: возвращает базовое имя;
- `$<TARGET_FILE_NAME:target>`: возвращает базовое имя без префикса и суффикса (для `libmylib.so` будет возвращено `mylib`);
- `$<TARGET_FILE_PREFIX:target>`: возвращает только префикс (например, `lib`);
- `$<TARGET_FILE_SUFFIX:target>`: возвращает только суффикс (например, `.so` или `.exe`);
- `$<TARGET_FILE_DIR:target>`: возвращает каталог.

Существуют семейства выражений, которые предлагают функциональность, схожую с обычным выражением `TARGET_FILE` (каждое выражение принимает также суффикс `_NAME`, `_BASE_NAME` или `_DIR`):

- `TARGET_LINKER_FILE`: запрашивает путь к файлу, использованному при компоновке с целью. Обычно это библиотека, порожденная целью (`.a`, `.lib`, `.so`). Однако на платформах с **динамически компонуемыми библиотеками** (**DLL**) это будет импортируемая библиотека `.lib`, ассоциированная с целевой DLL;
- `TARGET_PDB_FILE`: запрашивает путь к сгенерированному компоновщиком файлу базы данных программы (`.pdb`).

Управление библиотеками – сложная тема, и CMake предлагает в помощь много генераторных выражений. Однако знакомство с ними мы отложим до главы 8, где они нам понадобятся.

Наконец, имеются пакетные выражения для Apple:

- `$<TARGET_BUNDLE_DIR:target>`: это полный путь к каталогу пакета (`my.app`, `my.framework` или `my.bundle`) для цели;
- `$<TARGET_BUNDLE_CONTENT_DIR:target>`: это полный путь к каталогу содержимого пакета для цели. Для macOS это `my.app/Contents`, `my.framework` или `my.bundle/Contents`. В других **комплектах средств разработки ПО** (**SDK**) (например, iOS) пакет имеет плоскую структуру: `my.app`, `my.framework` или `my.bundle`.

Это основные генераторные выражения для работы с целями. Есть еще много других. Рекомендую обратиться к официальной документации для ознакомления с полным списком.

Экранирование

Изредка бывает необходимо передать в генераторное выражение символ, имеющий специальный смысл. Для экранирования используйте следующие выражения:

- `$<ANGLE-R>`: это литеральный символ `>`;
- `$<COMMA>`: это литеральный символ `,`;
- `$<SEMICOLON>`: это литеральный символ `;`.

Последнее выражение полезно, чтобы предотвратить расширение списка, когда аргумент содержит символ `;`.

Познакомившись с запросами и преобразованиями, мы можем взглянуть, как они применяются на практике. Рассмотрим несколько примеров.

Примеры

Любой материал проще усваивается, когда есть хороший практический пример, подкрепляющий теорию. Очевидно, что мы хотели бы написать код для CMake и испытать его в деле. Однако поскольку генераторные выражения не вычисляются до завершения этапа конфигурирования, мы не можем использовать для экспериментов команды этапа конфигурирования, например `message()`. Нужны какие-то специальные приемы. Для отладки генераторного выражения можно использовать следующие методы:

- записать его в файл; следующая версия команды `file()` поддерживает генераторные выражения: `file(GENERATE OUTPUT filename CONTENT "$<...>"`;
- добавить специальную цель и явно собрать ее из командной строки: `add_custom_target(gendbg COMMAND ${CMAKE_COMMAND} -E echo "$<...>")`.

Я рекомендую первый вариант, потому что он проще. Напомню, однако, что в этих командах разрешено использовать не все выражения, т. к. некоторые из них целеориентированы.

Конфигурации сборки

В главе 1 мы обсуждали тип сборки, который определяет, какую конфигурацию мы собираем: `Debug`, `Release` и т. д. Есть много случаев, когда мы хотим действовать по-разному в зависимости от типа сборки. Простой способ добиться желаемого – воспользоваться генераторным выражением `$<CONFIG>`:

```
target_compile_options(tgt $<$<CONFIG:DEBUG>:-ginline-points>)
```

В этом примере мы проверяем, равна ли конфигурация `DEBUG`; если да, то результатом вычисления вложенного выражения будет `1`. Тогда внешнее сокращенное выражение `if` оказывается равным `true`, и добавляется флаг

отладки `-ginline-points`. Знать эту форму важно, потому что это поможет вам понять подобные выражения в других проектах, но я рекомендую ради удобочитаемости использовать более многословную форму `$<IF:...>`.

Системно-зависимые однострочные выражения

Генераторные выражения также позволяют сворачивать пространные команды `if` в одну строку. Рассмотрим следующий код:

```
if (${CMAKE_SYSTEM_NAME} STREQUAL "Linux")
    target_compile_definitions(myProject PRIVATE LINUX=1)
endif()
```

Здесь мы добавляем флаг `-DLINUX=1` в состав аргументов компилятора, если целевой системой является Linux. Хотя этот код не очень длинный, его можно заменить таким сравнительно простым выражением:

```
target_compile_definitions(myProject PRIVATE
                           $<$<CMAKE_SYSTEM_NAME:LINUX>:LINUX=1>)
```

Такой код хорошо работает, но нельзя беспредельно запихивать в генераторное выражение все новый и новый код, потому что его станет трудно читать. К тому же многие пользователи CMake откладывают изучение генераторных выражений, а потом не понимают, что происходит. К счастью, у нас после прочтения этой главы таких проблем не будет.

Интерфейсные библиотеки с зависящими от компилятора флагами

Интерфейсные библиотеки, которые мы обсуждали в главе 5, можно использовать для задания флагов, соответствующих компилятору:

```
add_library(enable_rtti INTERFACE)
target_compile_options(enable_rtti INTERFACE
  $<$<OR:$<COMPILER_ID:GNU>,$<COMPILER_ID:Clang>>:-rtti>
)
```

Даже в таком простом примере мы видим, что выражение становится трудно понять, когда число вложенных генераторных выражений слишком велико. К сожалению, иногда это единственный способ достичь нужного эффекта. Объясним, что происходит в этом примере:

- мы проверяем, совпадает ли `COMPILER_ID` со строкой `GNU`; если да, то результатом вычисления выражения `OR` будет `1`;

- если нет, мы проверяем, совпадает ли `COMPILER_ID` со строкой `Clang`, и если да, то результатом вычисления выражения `OR` будет `1`. В противном случае результатом вычисления выражения `OR` будет `0`;
- если `OR` равно `1`, то мы добавляем флаг `-rtti` в состав **параметров компилятора** `enable_rtti`. В противном случае не делаем ничего.

После этого мы можем скомпоновать наши библиотеки и исполняемые файлы с интерфейсной библиотекой `enable_rtti`. CMake добавит флаг `-rtti`, если компилятор его поддерживает. Кстати: **RTTI** означает **run-time type information** (информация о типе во время выполнения); например, это используется в C++ в сочетании с ключевым словом `typeid`, чтобы определить класс объекта во время выполнения; если в вашем коде эта возможность не используется, то и не нужно добавлять этот флаг.

Вложенные генераторные выражения

Иногда непонятно, что происходит при попытке использовать в генераторном выражении вложенные элементы. Для отладки мы можем записать тестовый вывод в отладочный файл.

Рассмотрим несколько примеров и посмотрим, что происходит.

ch06/01-nesting/CMakeLists.txt

```
set(myvar "small text")
set(myvar2 "small text >")

file(GENERATE OUTPUT nesting CONTENT "
  1 $<PLATFORM_ID>
  2 $<UPPER_CASE:$<PLATFORM_ID>>
  3 $<UPPER_CASE:hello world>
  4 $<UPPER_CASE:${myvar}>
  5 $<UPPER_CASE:${myvar2}>
")
```

После сборки этого проекта мы можем прочитать созданный файл `nesting`, воспользовавшись командой Unix `cat`:

```
# cat nesting

1 Linux
2 LINUX
3 HELLO WORLD
4 SMALL TEXT
5 SMALL text>
```

Поясним, как работает каждая строка.

1. Выходным значением `PLATFORM_ID` является `LINUX`.

2. Вложенное значение правильно преобразуется в строку `LINUX` в верхнем регистре.
3. Мы можем преобразовать простые строки.
4. Мы можем преобразовать содержимое переменных этапа конфигурирования.
5. Переменные интерполируются первыми, а закрывающие угловые скобки (`>`) интерпретируются как часть генераторного выражения, так что только часть строки преобразуется в верхний регистр.

Иными словами, имейте в виду, что содержимое переменных может оказать влияние на то, как вычисляются генераторные выражения. Если частью значения переменной должна быть угловая скобка, используйте `$<ANGLE-R>`.

Различие между условным выражением и вычислением оператора BOOL

Генераторные выражения могут вызвать затруднения, когда речь заходит о преобразовании булевых значений в строки. Важно понимать, чем они отличаются от обычных условных выражений, начинающихся ключевым словом `IF`:

ch06/02-boolean/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Boolean CXX)

file(GENERATE OUTPUT boolean CONTENT "
  1 $<0:TRUE>
  2 $<0:TRUE,FALSE> (won't work)
  3 $<1:TRUE,FALSE>
  4 $<IF:0,TRUE,FALSE>
  5 $<IF:0,TRUE,>
")
```

Прочитаем созданный файл:

```
# cat boolean

1
2 (won't work)
3 TRUE,FALSE
4 FALSE
5
```

Разберем каждую строку.

1. Это булево расширение, где `BOOL` равно `0`; поэтому строка `TRUE` не печатается.

2. Это типичная ошибка – автор хотел напечатать TRUE или FALSE в зависимости от значения BOOL, но поскольку это тоже расширение булева false, два аргумента трактуются как один и не печатаются.
3. Это та же ошибка для противоположного значения – мы имеем расширение булева true, так что оба аргумента печатаются в одной строке.
4. Это правильное условное выражение, начинающееся с IF, – печатается FALSE, потому что первый аргумент равен 0.
5. Это правильное использование условного выражения, однако если мы не хотим передавать значение для булева false, то лучше использовать форму в первой строке.

Генераторные выражения печально известны своим запутанным синтаксисом. Упомянутые в этом разделе различия могут смутить даже опытных пользователей. Если сомневаетесь, скопируйте такое выражение в другой файл и проанализируйте его, добавив отступ и пробелы для лучшего понимания.

Примеры работы генераторных выражений подготовили нас к их использованию на практике. В следующих главах мы обсудим много новых тем, к которым генераторные выражения имеют прямое отношение. В должное время будут приведены дополнительные примеры их применения.

Резюме

Эта глава была посвящена различным нюансам генераторных выражений. Мы начали с основ их формирования и расширения и механизма вложенности. Затем мы подробно рассмотрели условное расширение, что повлекло за собой обсуждение булевой логики, операторов сравнения и запросов. Этот аспект генераторных выражений раскрывается во всей полноте, когда нужно адаптировать процесс сборки к таким факторам, как выбранная пользователем конфигурация сборки, платформа и имеющийся комплект инструментов.

Мы также рассмотрели простые, но важные преобразования строк, списков и путей. Ведь существенно использование генераторных выражений для запроса информации, собираемой на более поздних этапах сборки и предъявления ее, когда контекст отвечает требованиям. Мы также знаем теперь, как узнать идентификатор компилятора, его версию и возможности. Мы обсудили применение генераторных выражений для запроса свойств цели сборки и выделения нужной информации. В заключение главы мы привели практические примеры и рекомендации по просмотру вывода, когда это возможно. Зная все это, вы теперь готовы использовать генераторные выражения в своих проектах.

В следующей главе мы узнаем, как компилировать программы с помощью CMake. Точнее, мы будем говорить о том, как сконфигурировать и оптимизировать этот процесс.

Для дополнительного чтения

Дополнительные сведения по рассмотренным в этой главе темам можно получить из следующих источников:

- генераторные выражения в официальной документации: https://cmake.org/cmake/help/latest/manual/cmake-generatorexpressions.7.html;
- идентификаторы поддерживаемых компиляторов: https://cmake.org/cmake/help/latest/variable/CMAKE_LANG_COMPILER_ID.html;
- смешение языков в Cmake: https://stackoverflow.com/questions/8096887/mixing-c-and-c-with-cmake.

7

Компиляция исходного кода на C++ с помощью CMake

Для простых случаев компиляции обычно достаточно конфигурации комплекта инструментов по умолчанию или той, что предлагается **интегрированной средой разработки** (**IDE**) «из коробки». Но в профессиональной практике часто возникают более трудные ситуации. На передний план могут выдвигаться более высокая производительность, меньший размер двоичных файлов, переносимость, автоматизированное тестирование или расширенные возможности отладки – можете сами дополнить этот список. Управление всем этим таким образом, чтобы не запутаться и учесть будущее развитие, может быстро стать трудным делом (особенно если необходимо поддерживать несколько платформ).

Процесс компиляции часто объясняется в книгах по C++ недостаточно хорошо (такие темы, как виртуальные базовые классы, кажутся авторам гораздо интереснее). В этой главе мы исправим данное упущение и рассмотрим разные аспекты компиляции: мы узнаем, как работает компилятор, какие есть внутренние этапы и как они влияют на двоичный выход.

После этого мы обсудим, какие есть команды для точной настройки компиляции, как затребовать у компилятора конкретные возможности и как правильно сообщить компилятору, какие входные файлы он должен обрабатывать.

Далее мы сосредоточим внимание на первом этапе компилятора – препроцессоре. Мы зададим пути к включаемым заголовочным файлам и узнаем, как подставить переменные CMake и среды сборки в определения препроцессора. Мы рассмотрим самые интересные случаи и обсудим, как сделать переменные CMake доступными из кода на C++.

Затем поговорим об оптимизаторе и о том, как различные флаги могут повлиять на производительность. Мы также обсудим издержки оптимизации

и, в частности, как они влияют на возможность отладки созданных двоичных файлов и что делать, если эти издержки нежелательны.

Наконец, мы объясним, как уменьшить время компиляции с помощью предкомпилированных заголовков и унифицированных сборок. Мы научимся отлаживать процесс сборки и находить ошибки.

В этой главе будут рассмотрены следующие вопросы:

- основы компиляции;
- конфигурирование препроцессора;
- конфигурирование оптимизатора;
- управление процессом компиляции.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch07.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Основы компиляции

Компиляцию можно приблизительно описать как процесс трансляции команды на языке высокого уровня в низкоуровневый машинный код. Это позволяет создавать приложения с помощью таких абстрактных концепций, как классы и объекты, и не связываться с зависящими от процесса языками ассемблера со всеми их сложностями. Нам ни к чему работать непосредственно с регистрами процессора, думать о коротких и длинных командах перехода и управлять кадрами стека. Компилируемые языки более выразительны, удобочитаемы и безопасны, на них проще создавать пригодный для сопровождения код, по возможности не слишком жертвуя производительностью.

В C++ используется статическая компиляция, т. е. вся программа целиком должна быть оттранслирована в машинный код, прежде чем можно будет начать ее выполнение. Этот подход отличается от таких языков, как Java или Python, позволяющих интерпретировать и компилировать программу

динамически при каждом ее выполнении пользователем. У каждого метода имеются свои преимущества. C++ нацелен на предоставление многочисленных высокоуровневых инструментов, обеспечивая при этом высокую производительность. Компилятор C++ может создать автономное приложение почти для любой из существующих архитектур.

Создание и выполнение программы на C++ – процесс, состоящий из нескольких шагов.

1. **Проектирование приложения**: на этом шаге осуществляется планирование функциональности, структуры и поведения приложения. По завершении проектирования нужно аккуратно написать исходный код, следуя рекомендациям по удобочитаемости и удобству сопровождения.
2. **Компиляция cpp-файлов реализации, или единиц трансляции, в объектные файлы**: на этом шаге написанный вами код на языке высокого уровня преобразуется в низкоуровневый машинный код.
3. **Компоновка объектных файлов в один исполняемый файл**: на этом шаге прикомпоновываются все зависимости, в т. ч. динамические и статические библиотеки. Создается исполняемый файл, который можно запустить на заранее определенной платформе.

Для выполнения программы **операционная система** (**ОС)** пользуется **загрузчиком**, который отображает машинный код программы и всех динамических библиотек в виртуальную память. Затем загрузчик читает заголовок программы, чтобы определить, где начинаются и заканчиваются команды.

На этом этапе в игру вступает код инициализации программы. Вызывается специальная функция `_start`, предоставляемая системной библиотекой C. Она читает аргументы командной строки и переменные окружения, запускает потоки, инициализирует статические символы и регистрирует функции обратного вызова для очистки. И только после этого она вызывает функцию `main()`, написанную программистом.

Как видим, за кулисами производится немалый объем работы. В этой главе нас будет интересовать второй из перечисленных выше шагов. Но, представляя, как выглядит картина в целом, мы сможем лучше понять, откуда ждать потенциальных проблем. В разработке ПО нет никакой магии, несмотря на кажущуюся непостижимость. У всего есть объяснения и причины. Мы должны понимать, какие упущения на этапе компиляции программы могут привести к проблемам во время ее выполнения, пусть даже сама компиляция прошла успешно. Компилятор просто не в состоянии проверить все случаи. Поэтому давайте разберемся, что происходит во время работы компилятора.

Как устроена компиляция

Как уже было сказано, компиляция – процесс трансляции языка высокого уровня на язык низкого уровня. Точнее, речь идет о генерировании машинного кода, т. е. команд, которые процессор может выполнить непосредствен-

но, в двоичном формате **объектного файла**, своем для каждой платформы. В Linux чаще всего используется формат **Executable and Linkable Format** (**ELF**). В Windows используется формат PE/COFF, а в macOS – объекты Mach (формат Mach-O).

Объектные файлы – результат прямой трансляции отдельных исходных файлов. Каждый из них компилируется отдельно, а затем они объединяются компоновщиком в один исполняемый файл или библиотеку. Этот модульный процесс может заметно сэкономить время при модификации кода, поскольку перекомпилировать нужно только обновленные файлы.

Для создания объектного файла компилятор выполняет следующие действия:

- препроцессирование;
- лингвистический анализ;
- ассемблирование;
- оптимизация;
- вывод кода.

Разберем их более подробно.

Препроцессирование, хотя и вызывается большинством компиляторов автоматически, считается подготовительным шагом, предшествующим собственно компиляции. Его роль сводится к относительно простым операциям с исходным кодом: обработка директив `#include`, подстановка значений вместо символов, определенных с помощью директив `#define` и флагов `-D`, расширение простых макросов и условное включение или исключение частей кода на основе директив `#if`, `#elif` и `#endif`. Препроцессор пребывает в блаженном неведении о самом коде на C++. По существу, он исполняет функции инструмента поиска и замены, пусть и весьма развитого.

Тем не менее препроцессор крайне важен для создания сложных программ. Способность разбивать код на части и использовать одни и те же объявления в нескольких единицах трансляции – основа возможности использовать код повторно.

Далее следует **лингвистический анализ**, когда компилятор выполняет более тонкие операции. Он посимвольно просматривает препроцессированный файл (который содержит все заголовочные файлы, включенные препроцессором). В ходе процесса, называемого **лексическим анализом**, символы группируются в осмысленные лексемы; это могут быть ключевые слова, операторы, имена переменных и т. д.

Затем лексемы собираются в цепочки, которые проверяются на предмет согласованности с синтаксическими правилами C++, – этот процесс называется синтаксическим анализом, или парсингом. Именно на этой стадии генерируется большая часть сообщений об ошибках.

Наконец, компилятор выполняет **семантический анализ**. На этом этапе проверяется, являются ли предложения логически разумными. Например, компилятор проверяет правильность использования типов (нельзя присвоить целое число строковой переменной). Этот анализ удостоверяет, что программа имеет смысл в рамках языка программирования.

Этап **ассемблирования** по существу сводится к трансляции лексем в платформенные машинные команды. Некоторые компиляторы действительно генерируют выходной файл на языке ассемблера, который затем передается конкретному ассемблеру. Другие порождают машинный код прямо в памяти. Обычно такие компиляторы предоставляют возможность вывести ассемблерный код в текстовом виде, удобном для восприятия человеком. Но из того, что этот код можно прочитать, вовсе не следует, что его легко понять или что в этом есть какая-то польза.

Оптимизация не ограничивается каким-то одним шагом процесса компиляции, а производится на каждом этапе. Но вместе с тем имеется отдельный этап после первоначального ассемблирования, цель которого – минимизировать использование регистров и устранить избыточный код.

Интересная и достойная упоминания техника оптимизации – *встраивание*. Компилятор по сути дела «вырезает» тело функции и «вставляет» его в точку вызова. Стандарт C++ явно не определяет, при каких обстоятельствах это происходит, – этот аспект зависит от реализации. Встраивание может ускорить выполнение и сократить потребление памяти, но также затрудняет отладку, потому что исполняемому коду более не соответствует никакая строка в исходном коде.

На этапе **вывода кода** оптимизированный машинный код записывается в *объектный файл* в формате, согласованном со спецификациями платформы. Однако этот объектный файл еще не готов к выполнению, его нужно передать следующему инструменту – компоновщику. Задача компоновщика – переместить части *объектного файла* и разрешить ссылки на внешние символы, тем самым подготовив файл к выполнению. Этот шаг завершает преобразование исходного кода в формате **ASCII** (**American Standard Code for Information Interchange**) в *двоичный исполняемый файл*, который может быть непосредственно обработан процессором.

Каждый из описанных этапов важен и может быть сконфигурирован под конкретные требования. Посмотрим, как управлять этим процессом с помощью CMake.

Начальная конфигурация

CMake предлагает несколько команд, которые могут повлиять на каждый этап компиляции:

- `target_compile_features()`: для компиляции этой цели требуется компилятор с определенными возможностями;
- `target_sources()`: добавляет исходные файлы в уже определенную цель;
- `target_include_directories()`: задает *пути к каталогам включаемых файлов* для препроцессора;
- `target_compile_definitions()`: задает определения препроцессора;
- `target_compile_options()`: задает параметры командной строки, зависящие от компилятора;
- `target_precompile_headers()`: определяет, какие внешние заголовочные файлы следует оптимизировать путем предварительной компиляции.

Все эти команды принимают аргументы в следующем формате:

```
target_...(<имя цели> <INTERFACE|PUBLIC|PRIVATE> <аргументы>)
```

Это означает, что свойства, установленные данной командой, распространяются с помощью механизма транзитивных требований использования, которые мы обсуждали в главе 5, и могут быть использованы как для исполняемых файлов, так и для библиотек. Кроме того, стоит отметить, что все эти команды поддерживают генераторные выражения.

Требование конкретных возможностей от компилятора

Как обсуждалось в разделе «Проверка возможностей, поддерживаемых компилятором» главы 4, важно предвидеть проблемы и ясно сообщать пользователям о том, что пошло не так, – например, о том, что имеющийся компилятор, X, не поддерживает необходимую возможность, Y. Этот подход намного дружелюбнее, чем заставлять пользователя расшифровывать сообщения от несовместимого комплекта инструментов, который установлен на его машине. Мы не хотим, чтобы пользователи возлагали вину за проблемы несовместимости на наш код, а не на свою устаревшую среду.

Следующая команда описывает все возможности, необходимые для сборки цели:

```
target_compile_features(<цель> <PRIVATE|PUBLIC|INTERFACE>
                        <возможность> [...])
```

CMake знает о стандартах C++ и поддерживаемых компилятором возможностях для следующих идентификаторов компиляторов:

- `AppleClang`: Apple Clang для версий Xcode 4.4+;
- `Clang`: Clang Compiler версий 2.9+;
- `GNU`: GNU Compiler версий 4.4+;
- `MSVC`: Microsoft Visual Studio версий 2010+;
- `SunPro`: Oracle Solaris Studio версий 12.4+;
- `Intel`: Intel Compiler версий 12.1+.

Всего CMake поддерживает более 60 возможностей, их полный перечень имеется в документации на странице, посвященной переменной `CMAKE_CXX_KNOWN_FEATURES`. Однако если вас интересует что-то не слишком специфическое, я рекомендую выбирать высокоуровневую метавозможность, соответствующую общему стандарту C++:

- `cxx_std_14`;
- `cxx_std_17`;
- `cxx_std_20`;
- `cxx_std_23`;
- `cxx_std_26`.

Рассмотрим пример:

```
target_compile_features(my_target PUBLIC cxx_std_26)
```

Это по существу эквивалентно командам `set(CMAKE_CXX_STANDARD 26)` и `set(CMAKE_CXX_STANDARD_REQUIRED ON)`, с которыми мы познакомились в главе 4. Однако различие в том, что `target_compile_features()` работает на уровне целей, а не глобально для всего проекта, так что добавлять ее в каждую цель проекта может быть утомительно.

Дополнительные сведения о поддерживаемых CMake компиляторах см. в официальной документации (URL-адрес приведен в разделе «Для дополнительного чтения»).

Управление исходными файлами для целей

Мы уже знаем, как сообщить CMake о том, какие файлы составляют одну цель, будь то исполняемый файл или библиотека. Для этого в команду `add_executable()` или `add_library()` передается список файлов.

По мере развития вашей программы список файлов для каждой цели также растет. В результате команды `add_...()` становятся все длиннее. И как с этим бороться? Соблазнительным кажется использование команды `file()` в режиме `GLOB`, поскольку она может собрать все файлы из подкаталогов и сохранить их в переменной. Эту переменную можно было бы передать в виде аргумента объявлению цели и больше не думать о списке файлов:

```
file(GLOB helloworld_SRC "*.h" "*.cpp")
add_executable(helloworld ${helloworld_SRC})
```

Однако этот способ не рекомендуется – и вот почему. CMake генерирует сборочные системы на основе изменений в списковых файлах. Если никаких изменений не обнаружено, то сборка может завершиться неудачно вообще без предупреждений (кошмар разработчика). Кроме того, опустив все исходные файлы в объявлении цели, мы можем нарушить работу механизма инспекции кода в таких IDE, как CLion, которая знает, как разбирать некоторые команды CMake, чтобы лучше понять ваш проект.

Использовать переменные в объявлениях целей не рекомендуется и еще по одной причине: создается дополнительный уровень косвенности, что вынуждает разработчиков распаковывать определение цели при чтении проекта. Но чтобы последовать этому совету, мы должны ответить на другой вопрос: как условно добавить исходные файлы? Это типичная ситуация при работе с платформенно-зависимыми файлами реализации, например `gui_linux.cpp` и `gui_windows.cpp`.

Команда `target_sources()` позволяет добавить исходные файлы к ранее созданной цели:

ch07/01-sources/CMakeLists.txt

```
add_executable(main main.cpp)
if(CMAKE_SYSTEM_NAME STREQUAL "Linux")
  target_sources(main PRIVATE gui_linux.cpp)
```

```
elseif(CMAKE_SYSTEM_NAME STREQUAL "Windows")
  target_sources(main PRIVATE gui_windows.cpp)
elseif(CMAKE_SYSTEM_NAME STREQUAL "Darwin")
  target_sources(main PRIVATE gui_macos.cpp)
else()
  message(FATAL_ERROR "CMAKE_SYSTEM_NAME=${CMAKE_SYSTEM_NAME} not supported.")
endif()
```

Так каждая платформа получает собственный набор совместимых с ней файлов. Это хорошо, но как быть с длинными списками исходных файлов? Увы, надо смириться с тем, что еще не все в мире совершенно, и по-прежнему добавлять их вручную. Если вы сражаетесь с по-настоящему длинным списком, то, возможно, что-то не в порядке со структурой вашего проекта: быть может, стоило бы вынести часть исходных файлов в библиотеки.

Разобравшись с основами компиляции, перейдем к первому шагу – препроцессированию. Как всегда в информатике, дьявол кроется в деталях.

Конфигурирование препроцессора

Препроцессор играет огромную роль в процессе сборки. Это может показаться удивительным, потому что его функциональность довольно ограниченная. В следующих разделах мы рассмотрим задание путей к включаемым файлам и использование определений препроцессора. Мы также объясним, как можно использовать CMake для конфигурирования включаемых заголовочных файлов.

Задание путей к включаемым файлам

Самая главная возможность препроцессора – способность включать заголовочные файлы с расширениями `.h` и `.hpp` с помощью директивы `#include`, которая имеет две формы:

- с угловыми скобками: `#include <path-spec>`;
- с кавычками: `#include "path-spec"`.

Как мы знаем, препроцессор заменяет эти директивы содержимым файла, указанного в `path-spec`. Проблема в том, как найти эти файлы. Какие каталоги просматривать и в каком порядке? К сожалению, в стандарте C++ нет точных указаний на этот счет. Нужно смотреть в руководстве по компилятору.

Обычно в форме с угловыми скобками проверяются стандартные *каталоги включаемых файлов*, содержащие заголовочные файлы для стандартной библиотеки C++ и стандартной библиотеки C.

В случае формы с кавычками поиск начинается в каталоге текущего файла, а затем проверяются те же каталоги, что для формы с угловыми скобками.

CMake предлагает команду для работы с путями, на которых ищутся включаемые файлы:

```
target_include_directories(<target> [SYSTEM] [AFTER|BEFORE]
                           <INTERFACE|PUBLIC|PRIVATE> [item1...]
                          [<INTERFACE|PUBLIC|PRIVATE> [item2...]
...])
```

Это позволяет нам добавлять собственные пути, которые компилятор должен просматривать. CMake добавит их в вызовы компилятора из сгенерированной сборочной системы. При этом будет задан флаг, ожидаемый конкретным компилятором (чаще всего `-I`).

Команда `target_include_directories()` изменяет свойство цели `INCLUDE_DIRECTORIES`, добавляя каталоги в конец или в начало списка в зависимости от того, какое ключевое слово используется: `AFTER` или `BEFORE`. Однако только компилятору решать, будут ли указанные каталоги просматриваться до или после подразумеваемых по умолчанию (обычно до).

Ключевое слово `SYSTEM` сигнализирует компилятору, что с указанными каталогами следует обращаться как со стандартными системными каталогами (которые просматриваются формой с угловыми скобками). Для многих компиляторов эти каталоги передаются после флага `-isystem`.

Определения препроцессора

Вспомним директивы `#define` и `#if`, `#elif`, `#endif`, упомянутые при обсуждении этапов компиляции. Рассмотрим следующий пример.

ch07/02-definitions/definitions.cpp

```
#include <iostream>
int main() {
#if defined(ABC)
  std::cout << "ABC is defined!" << std::endl;
#endif
#if (DEF > 2*4-3)
  std::cout << "DEF is greater than 5!" << std::endl;
#endif
}
```

В таком виде пример не выводит ничего, потому что ни `ABC`, ни `DEF` не определено (`DEF` в этом примере по умолчанию было бы равно `0`). Это легко изменить, добавив две строчки в начале кода:

```
#define ABC
#define DEF 8
```

После компиляции и выполнения этого кода мы увидим на консоли оба сообщения:

```
ABC is defined!
DEF is greater than 5!
```

Это может показаться достаточно простым, но что, если мы хотим обусловить участки кода внешними факторами, например ОС, архитектурой или еще чем-то? Есть хорошая новость – мы можем передавать значения из CMake компилятору C++ , и это совсем не сложно.

Достаточно команды `target_compile_definitions()`.

ch07/02-definitions/CMakeLists.txt

```
set(VAR 8)
add_executable(defined definitions.cpp)
target_compile_definitions(defined PRIVATE ABC "DEF=${VAR}")
```

Этот код ведет себя точно так же, как две директивы `#define`, но теперь мы можем использовать всю гибкость переменных и генераторных выражений CMake и поместить команды в условный блок.

Традиционно эти определения передаются компилятору после флага `-D` (например, `-DFOO=1`), а некоторые программисты продолжают использовать этот флаг в следующей команде:

```
target_compile_definitions(hello PRIVATE -DFOO)
```

CMake понимает это и автоматически удаляет начальные флаги `-D`. Она также игнорирует пустые строки, так что следующая команда допустима:

```
target_compile_definitions(hello PRIVATE -D FOO)
```

В данном случае `-D` – отдельный аргумент, который становится пустой строкой после удаления и впоследствии игнорируется, что обеспечивает корректное поведение.

Избегайте доступа к закрытым полям класса в автономных тестах

Некоторые сетевые ресурсы рекомендуют использовать комбинацию определений `-D` с директивами `#ifdef` и `#ifndef` для автономного тестирования. Самое прямолинейное применение этой идеи – окружить спецификатор доступа `public` директивами условного включения, сделав все поля открытыми, если определена константа препроцессора `UNIT_TEST` (по умолчанию поля класса закрыты):

```
class X {
#ifdef UNIT_TEST
  public:
#endif
  int x_;
}
```

Хотя такая техника кажется удобной (т. к. позволяет тестам напрямую обращаться к закрытым членам класса), она не дает чистый код. В идеале автономные тесты должны проверять функциональность методов открытого интерфейса, считая их реализацию черным ящиком. Поэтому я рекомендую прибегать к этому подходу только в крайних случаях.

Использование фиксаций git для отслеживания откомпилированной версии

Подумаем о том, когда наличие информации о среде или файловой системе может принести выгоду. В профессиональной разработке важный пример – передача SHA-свертки фиксации, использованной при сборке двоичного файла. Это можно сделать так:

ch07/03-git/CMakeLists.txt

```
add_executable(print_commit print_commit.cpp)
execute_process(COMMAND git log -1 --pretty=format:%h
                OUTPUT_VARIABLE SHA)
target_compile_definitions(print_commit PRIVATE "SHA=${SHA}")
```

Затем SHA-свертку можно использовать в приложении следующим образом:

ch07/03-git/print_commit.cpp

```
#include <iostream>
// специальные макросы для преобразования определений в с-строки:
#define str(s) #s
#define xstr(s) str(s)
int main()
{
#if defined(SHA)
    std::cout << "GIT commit: " << xstr(SHA) << std::endl;
#endif
}
```

Разумеется, этот код будет работать, только если на машине пользователя Git установлена и прописана в переменной окружения `PATH`. Это особенно полезно, когда программы, работающие на производственных серверах, являются результатом конвейера непрерывной интеграции и развертывания.

Если в программе обнаруживается дефект, то мы можем быстро выяснить, какая фиксация была использована для ее сборки.

Отслеживание точной фиксации чрезвычайно полезно для отладки. Очень просто передать в код на C++ одну переменную, но как быть, если в заголовочные файлы нужно передать несколько десятков переменных?

Конфигурирование заголовочных файлов

Передавать определения с помощью команды `target_compile_definitions()` утомительно, если их много. Не проще ли оставить в заголовочном файле маркеры, ссылающиеся на эти переменные, а затем попросить CMake подставить вместо них значения? Безусловно, проще!

Команда `configure_file(<input> <output>)` позволяет генерировать новые файлы по шаблону, как в следующем примере.

ch07/04-configure/configure.h.in

```
#cmakedefine FOO_ENABLE
#cmakedefine FOO_STRING1 "@FOO_STRING1@"
#cmakedefine FOO_STRING2 "${FOO_STRING2}"
#cmakedefine FOO_UNDEFINED "@FOO_UNDEFINED@"
```

Ее можно использовать, как показано ниже.

ch07/04-configure/CMakeLists.txt

```
add_executable(configure configure.cpp)
set(FOO_ENABLE ON)
set(FOO_STRING1 "abc")
set(FOO_STRING2 "def")
configure_file(configure.h.in configured/configure.h)
target_include_directories(configure PRIVATE
                           ${CMAKE_CURRENT_BINARY_DIR})
```

Затем CMake генерирует выходной файл.

ch07/04-configure/<build_tree>/configured/configure.h

```
#define FOO_ENABLE
#define FOO_STRING1 "abc"
#define FOO_STRING2 "def"
/* #undef FOO_UNDEFINED */
```

Как видим, вместо маркеров переменных `@VAR@` и `${VAR}` подставлены значения из спискового файла CMake. Кроме того, `#cmakedefine` заменено на `#define` для определенных переменных и на `/* #undef VAR */` для неопределенных. Если вам нужны явные директивы `#define 1` или `#define 0` для блоков `#if`, то используйте вместо этого `#cmakedefine01`.

Этот сконфигурированный заголовочный файл можно сделать частью приложения, просто включив его в файл реализации.

ch07/04-configure/configure.cpp

```
#include <iostream>
#include "configured/configure.h"

// специальные макросы для преобразования определений в с-строки:
#define str(s) #s
#define xstr(s) str(s)

using namespace std;
int main()
{
#ifdef FOO_ENABLE
  cout << "FOO_ENABLE: ON" << endl;
#endif
  cout << "FOO_STRING1: " << xstr(FOO_STRING1) << endl;
  cout << "FOO_STRING2: " << xstr(FOO_STRING2) << endl;
  cout << "FOO_UNDEFINED: " << xstr(FOO_UNDEFINED) << endl;
}
```

Добавив наше двоичное дерево в пути к включаемым файлам командой `target_include_directories()`, мы сможем откомпилировать пример и получить на выходе файл, заполненный CMake:

```
FOO_ENABLE: ON
FOO_STRING1: "abc"
FOO_STRING2: "def"
FOO_UNDEFINED: FOO_UNDEFINED
```

У команды `configure_file()` есть также ряд параметров форматирования и доступа к файлам, которые мы не будем описывать из-за ограничений по объему книги. Интересующиеся могут обратиться к документации (см. раздел «Для дополнительного чтения»).

Разобравшись с компиляцией заголовочных и исходных файлов, обсудим, как на последующих шагах формируется выходной код. Хотя мы не можем как-то повлиять на лексический анализ или ассемблирование (поскольку тут действуют строгие стандарты), манипулировать конфигурацией оптимизатора мы в состоянии. Посмотрим, как это может отразиться на конечном результате.

Конфигурирование оптимизатора

Оптимизатор анализирует выход предыдущих этапов и применяет многочисленные приемы, которые программист не стал бы использовать само-

стоятельно, потому что они противоречат принципам чистого кодирования. Но это нормально – основная задача оптимизатора заключается в том, чтобы повысить производительность программы за счет меньшего потребления процессорного времени, минимизации использования регистров и сокращения потребления памяти. В процессе обхода исходного кода оптимизатор преобразует его в почти неузнаваемую форму, ориентированную на целевой процессор.

Оптимизатор не только решает, какие функции можно удалить и сделать компактнее; он также перемещает код из одного места в другое и даже может продублировать его! Если он может доказать, что некоторые строки избыточны, то безжалостно удалит их из тела важной функции (а вы этого даже не заметите). Он повторно использует одну и ту же память, размещая в ней разные переменные в разные моменты времени. Он даже может радикально изменить управляющие конструкции, если таким образом удастся сэкономить такт-другой.

Если бы программист должен был сам применять эти приемы к исходному коду, то получилась бы жуткая нечитаемая мешанина, о которой трудно рассуждать. Но, будучи применены компиляторами, они приносят выгоду, потому что компиляторы строго следуют предъявленным инструкциям. Оптимизатор неутомимо преследует одну цель – ускорить выполнение, – не обращая внимания, насколько при этом выходной код будет отличаться от входного. Выход может содержать кое-какую отладочную информацию, если мы работаем в тестовой среде, а может и не содержать, чтобы затруднить манипуляции не имеющим на то разрешения лицам.

У каждого компилятора припасен свой набор приемов, совместимых с поддерживаемой платформой и исповедуемой им философией. Мы рассмотрим несколько наиболее распространенных, доступных в GNU GCC и LLVM Clang, чтобы вы имели представление о том, что доступно и практически полезно.

Важно, что многие компиляторы (в т. ч. GCC) по умолчанию не включают никаких оптимизаций. Иногда это хорошо, а иногда – не очень. Зачем работать медленно, если можно быстро? Чтобы исправить положение, мы можем воспользоваться командой `target_compile_options()` и явно сформулировать, чего мы ожидаем от компилятора.

Синтаксис этой команды похож на синтаксис остальных, рассматриваемых в этой главе:

```
target_compile_options(<target> [BEFORE]
                       <INTERFACE|PUBLIC|PRIVATE> [items1...]
                      [<INTERFACE|PUBLIC|PRIVATE> [items2...]
...])
```

Мы задаем параметры командной строки, которые нужно будет использовать при сборке цели, а также ключевое слово, описывающее режим распространения. Во время выполнения CMake добавит эти параметры в конец переменной `COMPILE_OPTIONS` для цели. Факультативное ключевое слово `BEFORE` можно использовать, если мы хотим добавить их в начало. В некоторых ситуациях порядок важен, так что иметь такую возможность полезно.

Отметим, что `target_compile_options()` – общая команда. Ее можно использовать также для задания определений препроцессора `-D`, хотя CMake предлагает специально для этой цели команду `target_compile_definition()`. Всегда рекомендуется использовать самую специализированную из команд CMake, поскольку она гарантированно будет работать одинаково для всех поддерживаемых компиляторов.

Пора обсудить детали. В последующих разделах мы познакомимся с различными видами оптимизаций, поддерживаемыми большинством компиляторов.

Общий уровень

Любое поведение компилятора можно сконфигурировать с помощью флагов, которые передаются в виде *параметров компиляции*. Чтобы познакомиться со всеми, нужно потратить много времени и хорошо знать механизмы работы компиляторов, процессоров и памяти. А что делать, если нам просто нужен наилучший сценарий, который работает в большинстве случаев? Можно согласиться на общее решение – спецификатор уровня оптимизации.

Большинство компиляторов предлагают четыре уровня оптимизации, от `0` до `3`. Они задаются с помощью пространства `-O<level>`. `-O0` означает *без оптимизации*, обычно это уровень по умолчанию. С другой стороны, `-O2` считается *полной оптимизацией*, когда генерируется высокооптимизированный код ценой более медленной компиляции.

Промежуточный уровень `-O1` может (в зависимости от ваших требований) стать хорошим компромиссом – он обеспечивает разумную оптимизацию без чрезмерного замедления компиляции.

Наконец, уровень `-O3` тоже называется *полной оптимизацией*, как и `-O2`, но при этом компилятор более агрессивно оптимизирует встраивание функций и векторизацию циклов.

Существует также вариант, когда оптимизируется не скорость, а размер исполняемого файла, он задается флагом `-Os`. Еще имеется сверхагрессивная оптимизация, `-Ofast`, – уровень `-O3`, не строго соблюдающий стандарты C++. Самое очевидное различие – использование флагов `-ffast-math` и `-ffinite-math`, означающее, что можно отказаться от выполнения точных вычислений (хотя большинство программ работают именно в таком режиме).

CMake знает, что не все компиляторы одинаковы, и потому стандартизует усилия разработчиков, предоставляя некоторые флаги по умолчанию. Они хранятся в общесистемных (нецелеориентированных) переменных для используемого языка (`CXX` для C++) и конфигурации сборки (`DEBUG` или `RELEASE`):

- `CMAKE_CXX_FLAGS_DEBUG` равна `-g`;
- `CMAKE_CXX_FLAGS_RELEASE` равна `-O3 -DNDEBUG`.

Как видите, отладочная конфигурация сборки подавляет все оптимизации, а выпускная задает сразу уровень `O3`. Если хотите, можете изменить их напрямую командой `set()` или просто добавить параметр компиляции цели,

который переопределит поведение по умолчанию. Остальные два флага (`-g`, `-DNDEBUG`) относятся к отладке, мы обсудим их в разделе «Предоставление информации для отладчика» ниже.

Переменные вида `CMAKE_<LANG>_FLAGS_<CONFIG>` глобальны – они относятся ко всем целям. Рекомендуется конфигурировать цели с помощью свойств и команд, например `target_compile_options()`, не полагаясь на глобальные переменные. Таким образом, вы сможете контролировать цели более точно.

Выбирая уровень оптимизации `-O<level>`, мы косвенно задаем длинный список флагов, каждый из которых контролирует один аспект оптимизации. Можно настроить режим оптимизации точнее, добавив дополнительные флаги:

- разрешить что-то с помощью параметра `-f`: `-finline-functions`;
- запретить что-то с помощью параметра `-fno`: `-fno-inline-functions`.

Некоторые из этих флагов имеет смысл хорошо понимать, потому что они влияют на работу вашей программы и способы ее отладки. Разберемся.

Встраивание функций

Как вы, наверное, помните, компилятор можно поощрить к встраиванию некоторых функций, либо *определив* функцию в блоке определения класса, либо явно добавив ключевое слово `inline`:

```
struct X {
  void im_inlined(){ cout << "hi\n"; };
  void me_too();
};
inline void X::me_too() { cout << "bye\n"; };
```

Окончательное решение о том, встраивать функцию или нет, принимает компилятор. Если встраивание разрешено и функция используется только в одном месте (или сравнительно короткая функция используется в нескольких местах), то, скорее всего, она будет встроена.

Встраивание функций – любопытная техника оптимизации. Смысл ее в том, чтобы извлечь код функции и включить его во все места, где функция вызывается. В результате оригинальный вызов замещается, и экономятся драгоценные такты процессора.

Рассмотрим следующий пример на основе определенного выше класса:

```
int main() {
  X x;
  x.im_inlined();
  x.me_too();
  return 0;
}
```

Без встраивания выполнялась бы функция `main()`, пока не встретится вызов метода. Затем был бы создан новый кадр стека для `im_inlined()`, эта функция была бы выполнена в своей области видимости, и управление вернулось бы в кадр `main()`. То же самое произошло бы для метода `me_too()`.

Но если имеет место встраивание, то компилятор заменит вызовы:

```
int main() {
  X x;
  cout << "hi\n";
  cout << "bye\n";
  return 0;
}
```

Это не совсем точное описание, потому что встраивание производится на уровне ассемблерного или машинного (а не исходного) кода, но общее представление оно дает.

Компилятор применяет встраивание для экономии времени. Так он обходит создание и уничтожение нового кадра стека и необходимость искать адрес следующей выполняемой команды (и возврата); кроме того, повышается эффективность кеширования команд, потому что они находятся близко друг к другу.

Однако у встраивания имеется ряд важных побочных эффектов. Если функция используется несколько раз, то ее необходимо скопировать во все эти места, что увеличивает размер файла и потребление памяти. Сегодня это не так критично, как в былые времена, но все же небезразлично, особенно когда разрабатывается программное обеспечение для устройств с ограниченной памятью.

Кроме того, встраивание сильно влияет на отладку. Встраиваемому коду уже не соответствует оригинальный номер строки, что затрудняет или даже делает невозможным прослеживание. Поэтому точка останова, поставленная внутри встраиваемой функции, никогда не сработает (хотя код как-то выполняется). Чтобы решить эту проблему, необходимо запретить встраивание для отладочных сборок (понимая, что будет тестироваться код, отличный от выпускной версии).

Это можно сделать, задав нулевой уровень оптимизации `-O0` для цели или напрямую задав флаги, отвечающие за встраивание:

- `-finline-functions-called-once`: только для GCC;
- `-finline-functions`: для Clang и GCC;
- `-finline-hint-functions`: только для Clang.

Встраивание можно явно запретить с помощью флагов `-fno-inline-...`, но подробную информацию лучше посмотреть в документации по конкретной версии компилятора.

Разворачивание циклов

Разворачивание циклов – один из методов оптимизации. Смысл в том, чтобы преобразовать циклы в линейную последовательность предложений, дости-

гающую того же результата. Таким образом, малый размер программы приносится в жертву ее скорости, потому что устраняется команда управления циклом, арифметические операции с указателями и проверки окончания цикла.

Рассмотрим пример:

```
void func() {
  for(int i = 0; i < 3; i++)
    cout << "hello\n";
}
```

Этот код будет преобразован в такой:

```
void func() {
  cout << "hello\n";
  cout << "hello\n";
  cout << "hello\n";
}
```

Результат такой же, но теперь нам не нужно выделять память для переменной `i`, инкрементировать ее и трижды сравнивать с числом `3`. Если функция `func()` вызывается достаточно много раз, то даже разворачивание такого короткого цикла даст значительный эффект.

Однако есть два лимитирующих фактора. Во-первых, разворачивание цикла эффективно, только если компилятор знает наверняка или может точно оценить число итераций. Во-вторых, разворачивание циклов может иметь нежелательные последствия на современных процессорах, поскольку увеличение размера кода негативно сказывается на эффективности кеширования.

Каждый компилятор предлагает свою версию этого флага:

- `-floop-unroll`: только для GCC;
- `-funroll-loops`: только для Clang.

Если не уверены, тщательно протестируйте, влияет ли этот флаг на эффективность программы, и разрешите или запретите разворачивание явно. Отметим, что GCC включает его в состав уровня `-O3` как часть неявно включенного флага `-floop-unroll-and-jam`.

Векторизация циклов

Этот механизм, известный еще под названием **один поток команд, несколько потоков данных** (single instruction, multiple data – **SIMD**), был разработан в начале 1960-х годов для реализации распараллеливания. Как следует из названия, он предназначен для выполнения одной и той же операции над несколькими экземплярами данных одновременно. Рассмотрим, как это выглядит на практике:

```
int a[128];
int b[128];
// инициализировать b
for (i = 0; i<128; i++)
  a[i] = b[i] + 5;
```

Обычно такой код выполнил бы 128 итераций, но при наличии подходящего процессора его выполнение можно значительно ускорить за счет одновременного вычисления двух или более элементов массива. Это возможно в силу отсутствия зависимости между соседними элементами и перекрытия обоих массивов. Хороший компилятор может преобразовать этот цикл во что-то вроде (все это происходит на уровне ассемблера):

```
for (i = 0; i<32; i+=4) {
  a[ i ] = b[ i ] + 5;
  a[i+1] = b[i+1] + 5;
  a[i+2] = b[i+2] + 5;
  a[i+3] = b[i+3] + 5;
}
```

GCC производит такую автоматическую векторизацию циклов при уровне оптимизации `-O3`. Clang разрешает ее по умолчанию. Оба компилятора предлагают разные флаги для разрешения и запрета векторизации:

- `-ftree-vectorize -ftree-slp-vectorize`: разрешает векторизацию в GCC;
- `-fno-vectorize -fno-slp-vectorize`: запрещает векторизацию в Clang.

Своей эффективностью векторизация обязана специальным командам, предлагаемым производителями процессоров, а не просто замене оригинального цикла его развернутой версией. Поэтому достичь такого же уровня производительности вручную, оставаясь в рамках C++, невозможно (да и код получился бы *нечистым*).

Оптимизатор играет важную роль в повышении производительности программ. Эффективно используя его стратегии, мы можем получить больше за те же деньги. Думать об эффективности нужно не только по завершении кодирования, но и в течение всего процесса разработки. Если время компиляции велико, то его можно уменьшить, правильно организовав процесс.

Управление процессом компиляции

Программисты и инженеры по сборке должны также принимать во внимание такие аспекты компиляции, как время до завершения и простоту идентификации и исправления ошибок, возникающих в процессе сборки решения.

Уменьшение времени компиляции

В проектах, требующих частой перекомпиляции (возможно, несколько раз в час), важно, чтобы этот процесс был максимально быстрым. Это влияет не только на продолжительность цикла кодирования–компиляции–тестирования, но также на вашу способность к концентрации внимания и на весь технологический процесс.

По счастью, C++ и так уже неплохо справляется с управлением временем компиляции благодаря разделению на единицы трансляции. CMake перекомпилирует только те исходные файлы, которые оказались затронуты недавними изменениями. Но если мы не хотим этим ограничиваться, то найдется еще пара приемов: предкомпиляция заголовочных файлов и унифицированная сборка.

Предкомпиляция заголовочных файлов

Заголовочные файлы (с расширением `.h`) включаются в единицу трансляции препроцессором еще до начала собственно компиляции. Это означает, что их необходимо перекомпилировать всякий раз, как изменяются файлы реализации (с расширением `.cpp`). Более того, если в нескольких единицах трансляции используется один и тот же заголовочный файл, то его придется перекомпилировать при каждом включении. Это неэффективно, но долгое время было стандартом.

К счастью, начиная с версии 3.16 CMake предлагает команду, разрешающую предкомпиляцию заголовочного файла. Это позволяет компилятору обрабатывать заголовки отдельно от файлов реализации и тем самым повысить скорость компиляции. Сигнатура команды следующая:

```
target_precompile_headers(<target>
                          <INTERFACE|PUBLIC|PRIVATE> [header1...]
                         [<INTERFACE|PUBLIC|PRIVATE> [header2...]
...])
```

Список добавленных заголовков хранится в свойстве цели `PRECOMPILE_HEADERS`. В главе 5 мы говорили, что можем использовать распространяемые свойства, чтобы разделить заголовки между всеми зависимыми целями, указав ключевое слово `PUBLIC` или `INTERFACE`; однако этого не следует делать для целей, экспортированных командой `install()`. Другие проекты не нужно принуждать к потреблению наших предкомпилированных заголовков, потому что это нестандартная практика.



Используйте генераторное выражение `$<BUILD_INTERFACE:...>`, описанное в главе 6, чтобы воспрепятствовать появлению предкомпилированных заголовков в требованиях использования целей при установке. Однако они все равно будут добавляться в цели, экспортированные из дерева сборки командой `export()`. Не переживайте, если сейчас это кажется вам подозрительным, – мы дадим надлежащие объяснения в главе 14.

CMake помещает все имена заголовков в файл `cmake_pch.h` или `cmake_pch.hxx`, который затем предкомпилируется в зависящий от компилятора двоичный файл с расширением `.pch`, `.gch` или `.pchi`.

Мы можем использовать его в нашем списковом файле следующим образом:

ch07/06-precompile/CMakeLists.txt

```
add_executable(precompiled hello.cpp)
target_precompile_headers(precompiled PRIVATE <iostream>)
```

Можно также использовать его в соответствующем исходном файле:

ch07/06-precompile/hello.cpp

```
int main() {
  std::cout << "hello world" << std::endl;
}
```

Заметим, что нам не нужно включать в свой файл `main.cpp` ни `cmake_pch.h`, ни какой-то другой заголовок – он будет включен в CMake, если задать зависящие от компилятора параметры командной строки.

В примере выше я использовал встроенный заголовочный файл; однако вы легко можете добавить свои собственные заголовки, содержащие определения классов или функций. Для ссылки на заголовок есть две формы:

- `header.h` (прямой путь) интерпретируется относительно текущего каталога исходного кода и будет включен в абсолютный путь;
- путь вида `[["header.h"]]` (двойные квадратные скобки и кавычки) просматривается в соответствии со свойством цели `INCLUDE_DIRECTORIES`, которое можно сконфигурировать с помощью команды `target_include_directiories()`.

В некоторых онлайновых руководствах не рекомендуется предкомпилировать заголовки, не являющиеся частью стандартной библиотеки, например `<iostream>`, и даже вообще прибегать к предкомпилированию заголовков. Связано это с тем, что изменение списка или редактирование нестандартного заголовка влечет за собой перекомпиляцию всех единиц трансляции в цели. Но в случае CMake это опасение беспочвенно, особенно если проект структурирован правильно (относительно небольшие цели, сфокусированные на узкой области кода). У каждой цели есть отдельный предкомпилированный заголовок, что ограничивает распространение изменений заголовков.

Если ваши заголовки относительно стабильны, то можете повторно использовать предкомпилированные версии в своих целях. Для этого CMake предлагает удобную команду:

```
target_precompile_headers(<target> REUSE_FROM <other_target>)
```

Она задает свойство `PRECOMPILE_HEADERS_REUSE_FROM` цели, в которой повторно используются заголовки, и создает зависимость между этими целями.

Благодаря этому методу цель-потребитель может не задавать собственные предкомпилированные заголовки. Но все *параметры компиляции*, *флаги компиляции* и *определения компиляции* в двух целях должны совпадать.

Обращайте внимание на требования, особенно если имеются заголовки в формате с двойными квадратными скобками (`[["header.h"]]`). В обеих целях *пути к включаемым файлам* должны быть заданы правильно, чтобы компилятор мог найти эти заголовки.

Унифицированные сборки

В версии CMake 3.16 введено еще одно средство оптимизации времени компиляции – *унифицированные сборки*. Принцип работы основан на объединении нескольких исходных файлов реализации с помощью директивы `#include`. У этой идеи есть несколько интересных следствий – одни полезные, другие потенциально вредные.

Самое очевидное преимущество – предотвращение перекомпиляции заголовков в различных единицах трансляции, когда CMake создает унифицированный файл сборки:

```
#include "source_a.cpp"
#include "source_b.cpp"
```

Если оба исходных файла содержат строку `#include "header.h"`, то этот файл будет разбираться только один раз благодаря *защите от повторного включения* (при условии что она правильно реализована). Хотя это не так эффективно, как предкомпилированные заголовки, но все-таки какая-то альтернатива.

Второе преимущество – тот факт, что теперь оптимизатор видит более широкую картину и может оптимизировать межпроцедурные вызовы, встречающиеся во всех объединенных исходных файлах. Это похоже на оптимизацию на этапе компоновки, которую мы обсуждали в разделе «Межпроцедурная оптимизация» главы 4.

Однако эти преимущества даются не даром. Уменьшив число *объектных файлов* и шагов обработки, мы одновременно увеличили объем памяти, необходимой для обработки более крупных файлов. И кроме того, уменьшилось количество действий, допускающих распараллеливание. Компиляторы – не мастера в деле многопоточной обработки, потому что им этого и не нужно – сборочная система обычно запускает много задач компиляции в разных потоках. Группировка всех файлов в один осложняет это, т. к. теперь у CMake меньше файлов, которые можно было откомпилировать параллельно.

В случае унифицированной сборки вы также не должны упускать из виду некоторые последствия, вызванные семантикой C++, на первый взгляд не очевидные: анонимные пространства имен, скрывающие символы в пределах файла, теперь распространяются на весь унифицированный файл, а не на отдельную единицу трансляции. И то же самое относится к статическим

глобальным переменным, функциям и определениям макросов. Это может приводить к конфликтам имен или выполнению не того варианта перегруженной функции.

В унифицированных сборках перекомпиляция не оптимальна, потому что перекомпилируется гораздо больше файлов, чем необходимо. Лучше всего этот метод работает, когда требуется откомпилировать все файлы как можно быстрее. Тесты, выполненные в Qt Creator (популярная библиотека для разработки графических интерфейсов пользователя), показывают, что можно ожидать ускорения на 20–50 % (в зависимости от компилятора).

Для включения унифицированных сборок у нас есть два варианта:

- присвоить переменной `CMAKE_UNITY_BUILD` значение `true` – тогда будет инициализировано свойство `UNITY_BUILD` во всех определенных далее целях;
- вручную устанавливать свойство `UNITY_BUILD` в `true` для каждой цели, в которой должны использоваться унифицированные сборки.

Второй вариант реализуется путем вызова команды

```
set_target_properties(<target1> <target2> ...
                      PROPERTIES UNITY_BUILD true)
```

Вручную задавать эти свойства для многих целей, конечно, более трудоемко и удорожает сопровождение, но иногда это необходимо для точного контроля.

По умолчанию CMake создает сборки, содержащие восемь исходных файлов – в соответствии со свойством цели `UNITY_BUILD_BATCH_SIZE` (которое копируется при создании цели из переменной `CMAKE_UNITY_BUILD_BATCH_SIZE`). И свойство цели, и значение переменной по умолчанию можно изменить.

Начиная с версии 3.18 вы можете явно определить, как будут объединены файлы, с помощью именованных групп. Для этого измените свойство цели `UNITY_BUILD_MODE` на `GROUP` (по умолчанию оно равно `BATCH`). Затем распределите исходные файлы по группам, записав в свойство `UNITY_GROUP` каждого файла имя группы:

```
set_property(SOURCE <src1> <src2> PROPERTY UNITY_GROUP "GroupA")
```

В этом случае CMake игнорирует `UNITY_BUILD_BATCH_SIZE` и добавляет все файлы из группы в одну унифицированную сборку.

В документации по CMake не рекомендуется активировать унифицированные сборки для публичных проектов по умолчанию. Считается более правильным оставить пользователю вашего приложения возможность самому решить, хочет ли он иметь такие сборки, – для этого он может задать или не задавать аргумент командной строки `-DCMAKE_UNITY_BUILD`. Если ваш код написан так, что унифицированные сборки приводят к проблемам, то вы должны явно установить свойство цели в `false`. Однако ничто не мешает вам активировать эту возможность для кода, который будет использоваться

ограниченной аудиторией, например только внутри компании или в вашем личном проекте.

Мы рассмотрели самые важные способы уменьшить время компиляции в CMake. Есть и другие аспекты программирования, на которые мы зачастую тратим много времени, и один из самых главных – отладка. Посмотрим, что можно сделать здесь.

Поиск ошибок

Программисты тратят значительное время на вылавливание ошибок. Увы, это печальный факт нашей профессии. Процесс нахождения и исправления ошибок часто выводит из себя, особенно если затягивается на долгие часы. Проблема усугубляется, если приходится работать вслепую, не имея инструментов, которые выручили бы нас в трудной ситуации. Поэтому важно уделять внимание подготовке среды, так чтобы сделать этого процесс максимально простым и безболезненным. Для этого можно, в частности, сконфигурировать компилятор командой `target_compile_options()`. Так какие же *параметры компиляции* помогут нам в этом предприятии?

Конфигурирование ошибок и предупреждений

Разработка ПО часто сопровождается стрессом – исправлением критических дефектов посреди ночи, работой на виду у всех, дорогостоящими отказами в крупных системах и раздражающими ошибками компиляции. Одни ошибки трудно понять, другие очень нелегко исправить. Стремясь упростить свою работу и уменьшить вероятность отказа, вы встречаете много рекомендаций на тему конфигурирования предупреждений компилятора.

Одна такая рекомендация – задавать флаг `-Werror` по умолчанию для всех сборок. На первый взгляд, назначение этого флага обманчиво просто – рассматривать все предупреждения как ошибки, что не позволяет успешно откомпилировать код, пока все они не будут устранены. Но на поверку такой подход редко бывает удачным.

Дело в том, что предупреждения отличаются от ошибок не без причины: они призваны насторожить вас. А уж решать, как относиться к предупреждениям, – дело ваше. Возможность игнорировать предупреждение, особенно при экспериментировании или создании прототипа, бывает бесценной.

С другой стороны, если имеется идеальный, «отполированный» код без единого предупреждения, то как-то даже неловко разрешать будущие модификации, способные испортить это безупречное состояние. И что может быть плохого в том, чтобы включить этот флаг и оставить его? Вроде бы ничего, по крайней мере до перехода на новую версию компилятора. Новые версии строже относятся к средствам, которые были объявлены нерекомендуемыми, а также могут лучше подсказывать, что можно улучшить. Коль скоро предупреждения остаются предупреждениями, это хорошо, но в противном случае может приводить к неожиданным ошибкам в коде, который никто не

изменял, или, хуже того, когда необходимо быстро исправить проблему, не имеющую никакого отношения к новому предупреждению.

Итак, когда все-таки включать все предупреждения? Краткий ответ – когда вы разрабатываете общедоступную библиотеку. В таком случае вы предотвращаете жалобы на ошибки, когда ваш код используется в более строгом окружении, чем ваше собственное. Если вы выбрали этот путь, то не забывайте обновлять свой код при выходе новых версий компиляторов с их новыми предупреждениями. Важно также организовать явное управление этим процессом обновления, отличным от внесений изменений в код.

В противном случае оставьте предупреждения в покое и сконцентрируйтесь на ошибках. Если вы вынуждены проявлять педантизм, то задайте флаг `-Wpedantic`. Он включает все предупреждения, требуемые строгими стандартами ISO C и ISO C++. Но имейте в виду, что сам по себе этот флаг не удостоверяет соблюдение стандарта, а лишь обнаруживает практики, противоречащие ISO, которые требуют диагностического сообщения.

Многим снисходительным и прагматичным программистам достаточно флага `-Wall`, быть может, в сочетании с чуть более строгим `-Wextra`. Эти предупреждения считаются действительно полезными, и вы должны устранить их причины, когда найдется время.

Есть еще много флагов предупреждений, которые могут оказаться полезными в зависимости от типа проекта. Я рекомендую прочитать о том, какие имеются возможности, в руководстве по выбранному вами компилятору.

Отладка сборки

Бывает, что компиляция не проходит. Обычно это случается, когда мы пытаемся подвергнуть рефакторингу большие куски кода или привести в порядок сборочную систему. Иногда проблема решается легко, но встречаются и более сложные случаи, требующие тщательного исследования шагов конфигурирования. Мы уже знаем, как в CMake печатать более подробные сообщения (см. главу 1), но как проанализировать, что происходит под капотом на каждом этапе?

Отладка отдельных этапов

Параметр `-save-temps`, который понимают оба компилятора, GCC и Clang, позволяет отлаживать отдельные этапы компиляции. Он дает компилятору указание сохранять выход некоторых этапов в файлах, а не только в памяти.

ch07/07-debug/CMakeLists.txt

```
add_executable(debug hello.cpp)
target_compile_options(debug PRIVATE -save-temps=obj)
```

При включении этого режима порождаются два дополнительных файла (с расширениями `.ii` и `.s`) для каждой единицы трансляции.

В первом файле, `<build-tree>/CMakeFiles/<target>.dir/<source>.ii`, хранится результат препроцессора с комментариями, объясняющими, откуда взялась каждая часть кода:

```
# 1 "/root/examples/ch07/06-debug/hello.cpp"
# 1 "<built-in>"
# 1 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# / / / ... removed for brevity ... / / /
# 252 "/usr/include/x86_64-linux-
  gnu/c++/9/bits/c++config.h" 3
namespace std
{
  typedef long unsigned int size_t;
  typedef long int ptrdiff_t;
  typedef decltype(nullptr) nullptr_t;
}
...
```

Второй файл, `<build-tree>/CMakeFiles/<target>.dir/<source>.s`, содержит результат лингвистического анализа, готовый к ассемблированию:

```
  .file "hello.cpp"
  .text
  .section .rodata
  .type _ZStL19piecewise_construct, @object
  .size _ZStL19piecewise_construct, 1
_ZStL19piecewise_construct:
  .zero 1
  .local _ZStL8__ioinit
  .comm _ZStL8__ioinit,1,1
.LC0:
  .string "hello world"
  .text
  .globl main
  .type main, @function
main:
( ... )
```

В зависимости от характера проблемы мы часто можем определить, в чем ошибка. Например, вывод препроцессора может помочь при выявлении таких ошибок, как неправильные *пути к включаемым файлам* (ведущие не на те версии библиотек, что нужно), или ошибки в определениях, из-за которых неправильно вычисляется `#ifdef`. А вывод лингвистического анализатора особенно полезен в случае ориентации на конкретный процессор и для разрешения критических проблем оптимизации.

Отладка проблем из-за включения заголовочных файлов

Отладка неправильно включенных файлов может оказаться головоломной задачей. Уж я-то знаю – на своей первой работе в корпоративном секторе я должен был перенести всю кодовую базу с одной сборочной системы на

другую. Если вы окажетесь в ситуации, где требуется точно знать путь к включаемому заголовочному файлу, присмотритесь к флагу компиляции `-H`:

ch07/07-debug/CMakeLists.txt

```
add_executable(debug hello.cpp)
target_compile_options(debug PRIVATE -H)
```

Вывод должен выглядеть примерно так:

```
[ 25%] Building CXX object
CMakeFiles/inclusion.dir/hello.cpp.o
. /usr/include/c++/9/iostream
.. /usr/include/x86_64-linux-gnu/c++/9/bits/c++config.h
... /usr/include/x86_64-linux-gnu/c++/9/bits/os_defines.h
.... /usr/include/features.h
-- для краткости опущено --
.. /usr/include/c++/9/ostream
```

После имени *объектного файла* каждая строка вывода содержит путь к заголовочному файлу. В этом примере одна точка в начале строки обозначает включение верхнего уровня (когда директива `#include` находится в файле `hello.cpp`). Две точки означают, что файл включается на более низком уровне (`<iostream>`). Каждая точка обозначает дополнительный уровень вложенности.

В конце могут быть напечатаны рекомендации по улучшению кода:

```
Multiple include guards may be useful for:
/usr/include/c++/9/clocale
/usr/include/c++/9/cstdio
/usr/include/c++/9/cstdlib
```

От вас не требуется что-то делать с проблемами в стандартной библиотеке, но могут встретиться и некоторые заголовки, написанные вами. В таких случаях хорошо бы последовать рекомендации.

Предоставление информации отладчику

Машинный код – это загадочный список команд и данных, представленный в двоичном формате. У него нет ни особого смысла, ни цели, поскольку процессору безразлично назначение программы, равно как и осмысленность составляющих ее команд. Важно только, чтобы код был корректен. Компилятор транслирует исходный код в числовые идентификаторы машинных команд, инициализирует память по мере необходимости и формирует десятки тысяч адресов памяти. Иными словами, в конечном двоичном файле может не остаться и следов исходного кода, имен переменных, сигнатур функций и других деталей, интересных программисту. Это вывод компилятора по умолчанию – коротко и неясно.

Делается это ради экономии памяти и минимизации накладных расходов. Заодно мы в какой-то степени защищаем код своего приложения от обратного конструирования. Да, вы, конечно, можете понять, что делает каждая команда, даже не имея исходного кода (например, копирует такое-то значение в такой-то регистр). Но даже простые программы содержат слишком много команд, чтобы понять их общий смысл.

Если вы достаточно мотивированы, то можете воспользоваться **дизассемблером** и, обладая необходимыми – обширными – знаниями (и толикой удачи), сумеете разобраться в происходящем. Однако этот подход не очень практичен, т. к. дизассемблированный код не содержит исходных символов, поэтому его распутывание требует очень много сил и времени.

Вместо этого мы можем попросить компилятор сохранить исходный код в созданном двоичном файле, связав его с откомпилированным. Тогда можно будет присоединить отладчик к работающей программе и посмотреть, какая строка исходного кода выполняется в данный момент. Это просто бесценно при добавлении новой функциональности или исправлении ошибок.

Эти два сценария находят отражение в двух конфигурациях сборки: `Debug` и `Release`. Как мы уже видели, для управления этим процессом CMake передает компилятору флаги по умолчанию, сохраняя их в глобальных переменных:

- `CMAKE_CXX_FLAGS_DEBUG` содержит `-g`;
- `CMAKE_CXX_FLAGS_RELEASE` содержит `-DNDEBUG`.

Флаг `-g` означает просто «добавить отладочную информацию», которая хранится в системно-зависимом формате: STABS (symbolic table strings), COFF, XCOFF или DWARF. Впоследствии ей могут воспользоваться отладчики, например `gdb` (GNU debugger). Обычно этого достаточно в IDE типа CLion (поскольку они используют под капотом `gdb`). Если нет, обратитесь к руководству по отладчику и посмотрите, какой флаг нужен выбранному вами компилятору.

Для конфигурации `Release` CMake добавляет флаг `-DNDEBUG`. Это определение препроцессора, расшифровывающееся как «not a debug build» (не отладочная сборка). Некоторые связанные с отладкой макросы при наличии этого флага подавляются. Один из них – `assert`, определенный в заголовочном файле `<assert.h>`. Если вы решите использовать в своем выпускном коде утверждения, то они попросту не будут работать:

```
int main(void)
{
  assert(false);
  std::cout << "This shouldn't run. \n";
  return 0;
}
```

Вызов `assert(false)` ничего не делает в конфигурации `Release`, но прекращает выполнение в конфигурации `Debug`. А что делать, если вы поборник защитного программирования и хотите оставить `assert()` даже в выпускных сборках? Либо измените умолчания, предоставляемые CMake (уберите `NDEBUG`

из CMAKE_CXX_FLAGS_RELEASE), либо прямо в коде отмените определение макроса до включения заголовка:

```
#undef NDEBUG
#include <assert.h>
```

Дополнительные сведения об assert см. по адресу https://en.cppreference.com/w/c/error/assert.

Можете рассмотреть возможность замены assert() на static_assert(), которая была введена в C++11, если хотите, чтобы утверждения работали и на этапе выполнения, потому что эта функция, в отличие от assert(), не защищена директивой препроцессора #ifndef(NDEBUG).

Резюме

Вот и еще одна глава позади! Без сомнения, компиляция – сложный процесс. Учитывая все закоулки и особые требования, справиться с ней без надежного инструмента было бы трудно. К счастью, CMake предлагает отличную поддержку.

Итак, чему мы научились? Мы начали обсуждение с того, что такое компиляция и каково ее место в более широком контексте сборки и исполнения приложений. Затем мы рассмотрели этапы компиляции и внутренние инструменты управления ими. Эти знания очень важны для разрешения сложных проблем, с которыми мы можем столкнуться в будущем.

Далее мы объяснили, как с помощью CMake проверить, удовлетворяет ли доступный в хост-системе компилятор всем требованиям к сборке нашего кода. Как уже было сказано, пользователям нашего решения было бы гораздо удобнее видеть приветливое сообщение, предлагающее обновить компилятор, чем непонятную ошибку, выданную старым компилятором, не поддерживающим новые возможности языка.

Мы кратко обсудили, как добавлять исходные файлы в уже определенные цели, а затем перешли к конфигурированию препроцессора. Это обширная тема, потому что именно на этой стадии все части кода объединяются вместе и определяется, какие из них игнорировать. Мы поговорили о задании путей к файлам и добавлении специальных определений по отдельности и скопом (и о некоторых сценариях использования).

Затем обсудили оптимизатор – изучили общие уровни оптимизации и какие флаги неявно добавляются на каждом уровне. Мы также подробно поговорили о некоторых флагах: finline, floop-unroll и ftree-vectorize.

Наконец, настало время представить общую картину и узнать о практических вопросах компиляции. Мы затронули два аспекта: уменьшение времени компиляции (что заодно позволяет программисту сохранять концентрацию) и поиск ошибок. Последнее очень важно, т. к. помогает понять, что не работает и почему. Правильная настройка инструментов и понимание причин

некорректного поведения повышают качество кода (и помогают сохранить здравый рассудок).

В следующей главе мы поговорим о компоновке и всем том, что необходимо для сборки библиотек и их использования в проектах.

Для дополнительного чтения

Дополнительные сведения по рассмотренным в этой главе темам можно получить из следующих источников:

- компиляторы и их средства, поддерживаемые CMake: https://cmake.org/cmake/help/latest/manual/cmake-compilefeatures.7.html#supported-compilers;
- управление исходными файлами, используемыми в целях: https://stackoverflow.com/questions/32411963/why-is-cmake-file-glob-evil, https://cmake.org/cmake/help/latest/command/target_sources.html;
- ключевое слово `include`: https://en.cppreference.com/w/cpp/preprocessor/include;
- задание путей к включаемым файлам: https://cmake.org/cmake/help/latest/command/target_include_directories.html;
- конфигурирование заголовков: https://cmake.org/cmake/help/latest/command/configure_file.html;
- предкомпиляция заголовков: https://cmake.org/cmake/help/latest/command/target_precompile_headers.html;
- унифицированные сборки: https://cmake.org/cmake/help/latest/prop_tgt/UNITY_BUILD.html;
- унифицированные сборки с предкомпилированными заголовками: https://www.qt.io/blog/2019/08/01/precompiled-headers-and-unity-jumbo-builds-in-upcoming-cmake;
- поиск ошибок – флаги компилятора: https://interrupt.memfault.com/blog/best-and-worst-gcc-clang-compilerflags;
- зачем использовать библиотеки, а не объектные файлы: https://stackoverflow.com/questions/23615282/object-files-vs-libraryfiles-and-why;
- разделение обязанностей: https://nalexn.github.io/separation-of-concerns/.

8

Компоновка исполняемых файлов и библиотек

Возможно, вы думаете, что после успешной компиляции исходного кода в двоичный файл работа инженера по сборке заканчивается. Но это не совсем так. Хотя двоичные файлы содержат весь код, необходимый для выполнения на CPU, он может быть сложным образом распределен по нескольким файлам. Мы не хотим, чтобы процессор обшаривал разные файлы в поисках нужных фрагментов. Наша задача – объединить все единицы в один файл. Для этого служит процесс компоновки.

Беглый взгляд показывает, что в CMake имеется всего несколько команд компоновки, из которых главная – `target_link_libraries()`. Так зачем же посвящать целую главу единственной команде? К сожалению, в информатике практически ничто не бывает простым, и компоновка – не исключение: для получения правильного результата нужно знать всю историю – мы должны понимать, как работает компоновщик. Мы поговорим о внутренней структуре объектных файлов, о механизмах перемещения и разрешения ссылок и о том, для чего они нужны. Мы обсудим, чем конечные исполняемые файлы отличаются от своих компонентов и как система строит образ процесса, когда загружает программу в память.

Затем мы познакомимся с различными видами библиотек: статическими, разделяемыми и разделяемыми модулями. Хотя все они называются библиотеками, различия между ними очень велики. Для создания хорошо скомпонованного исполняемого файла нужна правильная конфигурация и внимание к таким деталям, как **позиционно-независимый код** (**PIC**).

Мы узнаем еще об одной проблеме компоновки – **правиле одного определения** (One Definition Rule – **ODR**). Управление дубликатами символов может оказаться трудной задачей, особенно в случае разделяемых библио-

тек. Кроме того, мы выясним, почему комоновщики иногда не могут найти внешние символы, хотя исполняемый файл правильно скомпонован с соответствующей библиотекой.

Наконец, мы научимся использовать компоновщик эффективно, подготовив свое решение к тестированию с помощью различных каркасов.

В этой главе будут рассмотрены следующие вопросы:

- основы компоновки;
- сборка библиотек разных типов;
- решение проблем с помощью ODR;
- порядок компоновки и неразрешенные символы;
- вынесение логики main() для тестирования.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch08.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Основы компоновки

Жизненный цикл программы на C++ мы обсуждали в главе 7. Он состоит из пяти этапов: написание кода, компиляция, компоновка, загрузка и выполнение. После того как все исходные файлы откомпилированы без ошибок, мы должны объединить их в один исполняемый файл. Мы говорили, что процессор не может непосредственно выполнить объектные файлы, созданные в процессе компиляции. Но почему?

Чтобы ответить на этот вопрос, нужно понимать, что формат объектных файлов – вариация на тему широко распространенного формата **Executable and Linkable Format** (**ELF**), применяемого в Unix-подобных и многих других системах. В Windows и macOS имеются собственные форматы, но принцип мы объясним на примере ELF. На рис. 8.1 показано, как компилятор структурирует эти файлы.

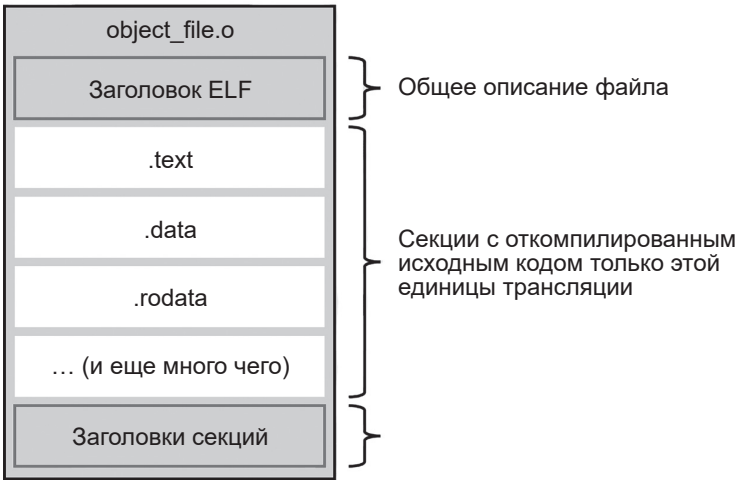


Рис. 8.1. Структура объектного файла

Компилятор создает объектный файл для каждой единицы трансляции (файла с расширением `.cpp`). Эти файлы используются для построения образа программы в памяти. Объектный файл состоит из следующих частей:

- **заголовок ELF**, идентифицирующий целевую **операционную систему** (**ОС**), тип файла, целевую систему команд, а также положение и размер двух таблиц: таблицу **заголовков программы** (которой нет в объектных файлах) и таблицу **заголовков секций**;
- двоичные секции, группирующие информацию по типу;
- таблица заголовков секций, содержащая сведения об имени, типе, флагах, конечном адресе в памяти, смещении от начала файла и другую информацию. Она позволяет понять, какие секции имеются в файле и где они находятся, т. е. играет роль оглавления.

Обрабатывая исходный код, компилятор распределяет собранную информацию по различным секциям. Они образуют основу ELF-файла и располагаются между **заголовком ELF** и **заголовками секций**. Приведем несколько примеров секций:

- секция `.text` содержит машинный код со всеми командами, предназначенными для выполнения процессором;
- секция `.data` содержит значения инициализированных глобальных и статических переменных;
- секция `.bss` резервирует место для неинициализированных глобальных и статических переменных, которые инициализируются нулем в начале выполнения программы;
- секция `.rodata` содержит значения констант, поэтому превращается в сегмент данных, доступных только для чтения;
- секция `.strtab` – таблица строк, содержащая константные строки типа «Hello World!» в демонстрационной программе `hello.cpp`;
- секция `.shstrtab` – таблица строк с именами всех остальных секций.

Эти секции практически точно соответствуют окончательной версии исполняемого файла, помещаемой в память для исполнения. И тем не менее мы не можем просто конкатенировать объектные файлы и загрузить результат в память. Такое безрассудное объединение привело бы к куче проблем. Например, к расточительному использованию памяти и времени из-за потребления чрезмерно большого количества страниц памяти. Перемещение команд и данных в процессорный кеш также осложнилось бы. Системе в целом пришлось бы иметь дело с возросшей сложностью, пустой растратой драгоценных тактов и хаотическим перескакиванием между бесчисленными секциями `.text`, `.data` и другими во время выполнения.

Мы примем более организованный подход: однотипные секции разных объектных файлов будем группировать в одну. Эта процедура называется **перемещением**, поэтому файл типа ELF называют «перемещаемым». Но перемещение не сводится просто к сборке однотипных секций. Необходимо еще изменить внутренние ссылки в файле, например адреса переменных, функций, индексы в таблицах символов и в таблицах строк. Все эти значения локальны в своих собственных объектных файлах, и их нумерация начинается с нуля. Поэтому при объединении файлов необходимо скорректировать их, так чтобы они указывали на правильные адреса в консолидированном файле.

На рис. 8.2 показано перемещение в действии – секция `.text` уже перемещена, секция `.data` находится в процессе сборки, а к секциям `.rodata` и `.strtab` будет применен такой же процесс (для простоты на рисунке опущены заголовки).

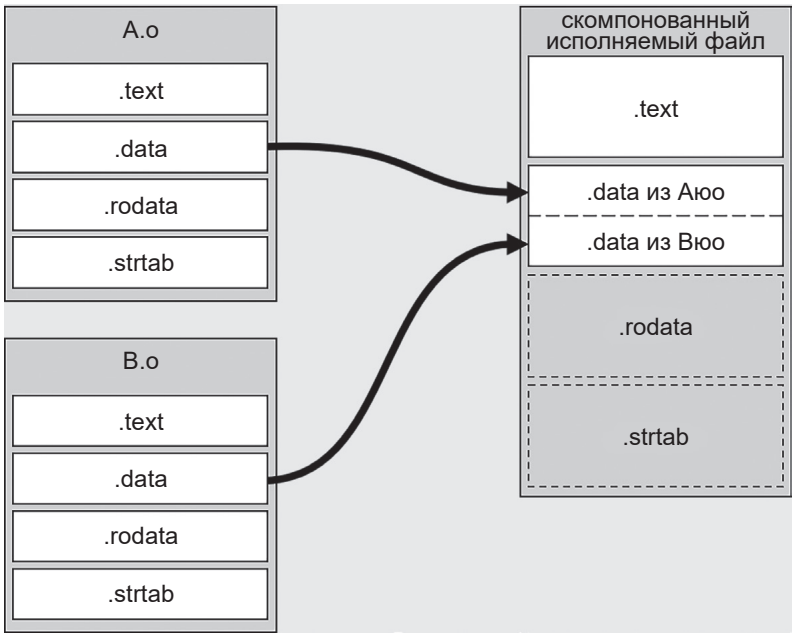


Рис. 8.2. Перемещение секции .data

Далее компоновщик должен разрешить ссылки. Если код из одной единицы трансляции ссылается на символ, определенной в другой, – путем включения ее заголовка или с использованием ключевого слова `extern`, – то компилятор считает такое объявление допустимым, предполагая, что определение будет предоставлено позже. Главная роль компоновщика заключается в сборе этих неразрешенных ссылок на внешние символы и последующей подстановке их адресов в консолидированном исполняемом файле. На рис. 8.3 показан простой пример этого процесса разрешения ссылок.

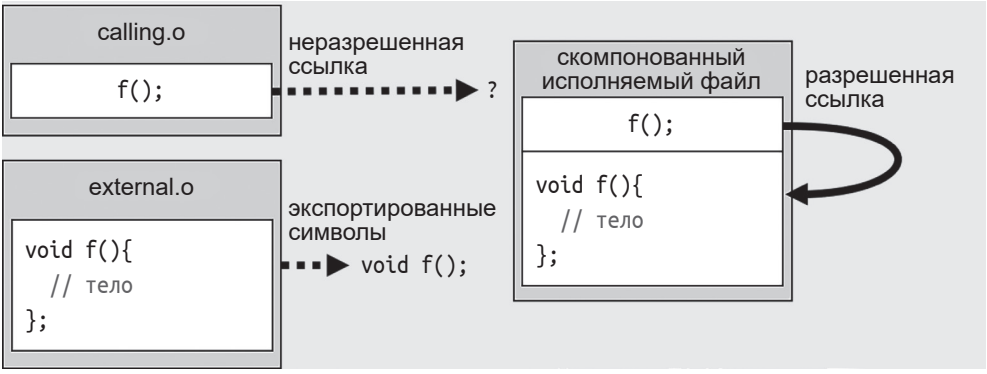


Рис. 8.3. Разрешение ссылок

Эта часть компоновки может стать источником проблем, если программист не знает, как она работает. В итоге мы можем получить неразрешенные ссылки, для которых невозможно найти соответствующие внешние символы. Или наоборот: мы предложили слишком много определений, и компоновщик не знает, какое выбрать.

Конечный исполняемый файл очень похож на объектный, поскольку содержит перемещенные секции с разрешенными ссылками, **таблицу заголовков секций** и, конечно, **заголовок ELF**, описывающий файл в целом. Основное отличие – присутствие **заголовка программы**, показанного на рис. 8.4.

Заголовок программы находится сразу после заголовка ELF. **Загрузчик** ОС читает заголовок программы, чтобы подготовить программу к выполнению, распределить память и создать образ процесса. Поля заголовка программы определяют, какие секции будут скопированы, в каком порядке и по каким адресам виртуальной памяти. Они также содержат информацию о флагах управления доступом (чтение, запись, выполнение) и другие полезные детали. Каждая именованная секция будет представлена одним **сегментом** памяти в созданном процессе.

Объектные файлы можно также собрать в библиотеку – промежуточный файл, который будет использоваться в конечном исполняемом файле или в другой библиотеке.

Разобравшись с принципом работы компоновщика, перейдем к следующему разделу, в котором обсудим три разных типа библиотек.

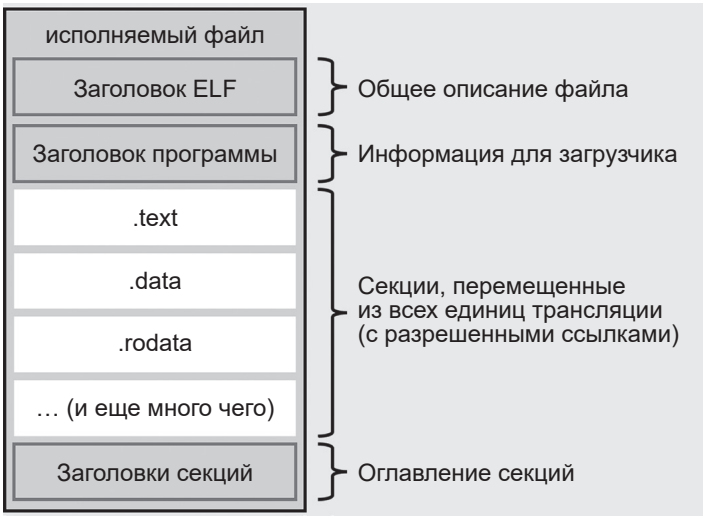


Рис. 8.4. Структура исполняемого файла формата ELF

Сборка библиотек разных типов

После компиляции исходного кода часто желательно предотвратить перекомпиляцию для той же самой платформы или даже разделить результат с внешними проектами. Можно распространять объектные файлы в том виде, в каком они сгенерированы, но это сопряжено с трудностями. Распространение нескольких файлов и включение их по одному в сборочную систему может оказаться морокой, особенно когда файлов много. Более эффективно объединить все объектные файлы в один, удобный для распространения. CMake сильно упрощает эту задачу. Для генерирования таких библиотек имеется простая команда `add_library()` (в сочетании с командой `target_link_libraries()`).

По соглашению, все библиотеки имеют общий префикс, `lib`, и системнозависимое расширение, обозначающее тип библиотеки:

- статические библиотеки имеют расширение `.a` в Unix и `.lib` в Windows;
- разделяемые библиотеки (и модули) имеют расширение `.so` в одних Unix-подобных системах (в частности, в Linux) и `.dylib` в других (macOS). В Windows они имеют расширение `.dll`;
- разделяемые модули обычно имеют такое же расширение, как разделяемые библиотеки, но не всегда. В macOS может использоваться расширение `.so`, особенно если модуль перенесен с другой Unix-платформы.

Процесс сборки библиотек (статических, разделяемых и модулей), по соглашению, называется «компоновкой» (linking), что хорошо видно на примере вывода сборки проекта `ch08/01-libraries`:

```
[ 33%] Linking CXX static library libmy_static.a
[ 66%] Linking CXX shared library libmy_shared.so
[100%] Linking CXX shared module libmy_module.so
[100%] Built target module_gui
```

Однако не для всех рассмотренных выше библиотек при создании обязательно использовать компоновщик. Для некоторых такие шаги, как перемещение и разрешение ссылок, опускаются.

Рассмотрим все типы библиотек подробнее и поймем, как они устроены.

Статические библиотеки

Статическая библиотека – это, по сути дела, собрание объектных файлов, хранящихся в архиве. Иногда она дополняется индексом, ускоряющим процесс компоновки. В Unix-подобных системах такие архивы создаются командой `ar` и индексируются командой `ranlib`.

В процессе сборки из статической библиотеки импортируются в конечный исполняемый файл только необходимые символы, чтобы уменьшить размер файла и потребление памяти. Благодаря такому выборочному включению исполняемый файл получается автономным и не нуждается во внешних файлах во время выполнения.

Для создания статической библиотеки можно воспользоваться командой, которую мы уже встречали в предыдущих главах:

```
add_library(<name> [<source>...])
```

По умолчанию она создает статическую библиотеку. Это можно изменить, присвоив переменной `BUILD_SHARED_LIBS` значение `ON`. Если мы хотим собирать статическую библиотеку вне зависимости от значения этой переменной, то можем явно добавить ключевое слово:

```
add_library(<name> STATIC [<source>...])
```

Статические библиотеки – не всегда идеальное решение, особенно если мы хотим разделить откомпилированный код между несколькими приложениями на одной машине.

Разделяемые библиотеки

Разделяемые библиотеки существенно отличаются от статических. Они строятся с использованием компоновщика, который доводит до конца оба этапа компоновки. Результирующий файл содержит заголовки секций, сами секции и таблицу заголовков секций, как показано на рис. 8.1.

Разделяемые библиотеки, которые часто называют разделяемыми объектами, можно одновременно использовать в нескольких разных приложениях. Когда первая программа обращается к разделяемой библиотеке, ОС загружает экземпляр последней в память. Последующим программам ОС сообщает тот же самый адрес благодаря изощренным механизмам виртуальной памяти. Однако в каждом процессе, использующем библиотеку, ее сегменты `.data` и `.bss` создаются отдельно. Поэтому каждый процесс может изменять свои переменные, не мешая другим процессам.

Благодаря такому подходу оптимизируется общее потребление памяти в системе. Если мы используем широко известную библиотеку, то необязательно распространять ее вместе со своей программой, потому что она, скорее всего, уже имеется на целевой машине. Если же библиотека не установлена заранее, то ожидается, что пользователь вручную установит ее перед запуском приложения. Это может приводить к проблемам, когда установленная версия библиотеки отличается от ожидаемой. Такого рода проблемы называются «адом зависимостей». Детали можно найти в разделе «Для дополнительного чтения».

Мы можем собрать разделяемую библиотеку, явно добавив ключевое слово `SHARED`:

```
add_library(<name> SHARED [<source>...])
```

Поскольку разделяемые библиотеки загружаются на этапе инициализации программы, не существует ассоциации между выполняющейся программой и фактическим файлом библиотеки на диске. Вместо этого компоновка осуществляется косвенно. В Unix-подобных системах это достигается с помощью **имени разделяемого объекта** (**shared object name** – **SONAME**), которое можно интерпретировать как «логическое имя» библиотеки.

Тем самым обеспечивается гибкость версионирования библиотек и гарантируется, что обратно совместимые изменения не приводят к немедленному отказу зависящих от них приложений.

Мы можем опросить свойства пути к порожденному SONAME-файлу с помощью генераторных выражений (не забудьте заменить `target` именем своей цели):

- `$<TARGET_SONAME_FILE:target>` возвращает полный путь (`.so.3`);
- `$<TARGET_SONAME_FILE_NAME:target>` возвращает только имя файла;
- `$<TARGET_SONAME_FILE_DIR:target>` возвращает каталог.

Это оказывается удобно в более сложных сценариях, которые мы рассмотрим ниже в этой книге, например:

- правильное использование сгенерированной библиотеки на этапах упаковки и установки;
- написание специальных правил CMake для управления зависимостями;
- использование SONAME во время тестирования;
- копирование или переименование сгенерированных библиотек в послесборочных командах.

Аналогичные потребности могут возникать для других системно-зависимых артефактов; для этой цели в CMake имеется два семейства генераторных выражений, предлагающих те же функции, что SONAME. Для Windows имеем:

- `$<TARGET_LINKER_FILE:target>` возвращает полный путь к библиотеке импорта `.lib`, ассоциированной с порожденной **динамически компонуемой библиотекой** (**DLL**). Отметим, что расширение `.lib` применяется в Windows также для статических библиотек, но по-другому;
- `$<TARGET_RUNTIME_DLLS:target>` возвращает список DLL, от которых цель зависит на этапе выполнения;
- `$<TARGET_PDB_FILE:target>` возвращает полный путь к `pdb`-файлу базы данных программы (используется для отладки).

Поскольку разделяемые библиотеки загружаются в память ОС на этапе инициализации программы, они применимы, когда заранее известно, какие библиотеки понадобятся программе. А что делать, если потребность в библиотеках определяется уже во время выполнения?

Разделяемые модули

Разделяемый модуль, или модульная библиотека, – это вариант разделяемой библиотеки, предназначенный для использования в качестве плагина. В отличие от стандартных разделяемых библиотек, которые загружаются в память автоматически в начале работы программы, разделяемый модуль загружается только после явного запроса со стороны программы. Для этого предназначены системные вызовы:

- `LoadLibrary` в Windows;
- `dlopen()` с последующим `dlsym()` в Linux и macOS.

Главная причина этого подхода – экономия памяти. Во многих программных системах имеются средства, которые не используются в жизненном цикле каждого процесса. Поэтому загружать их в память каждый раз было бы неэффективно.

Или же можно подготовить путь к расширению функциональности главной программы путем добавления специализированных средств, которые можно продавать, доставлять и загружать отдельно.

Для сборки разделяемых модулей служит ключевое слово `MODULE`:

```
add_library(<name> MODULE [<source>...])
```

Не пытайтесь компоновать свой исполняемый файл с модулями, потому что модуль предназначен для развертывания отдельно от использующей его программы.

Позиционно-независимый код (PIC)

Все современные программы в какой-то мере позиционно-независимы вследствие использования виртуальной памяти. Эта технология абстрагирует физические адреса. При вызове функции процессор обращается к **устройству управления памятью** (**MMU**) для трансляции виртуального адреса (они во всех процессах начинаются с 0) в соответствующий ему физический (определяется в момент выделения памяти). Интересно, что отображения адресов не всегда упорядочены одинаково.

При компиляции библиотеки возникает неопределенность; неясно, какие процессы могут ее использовать и в каком месте виртуальной памяти она окажется. Мы также не можем предсказать адреса символов и их местоположения в машинном коде библиотеки. Для решения этих проблем необходим еще один уровень косвенности.

Технология PIC была введена для отображения символов (например, ссылок на функции и глобальные переменные) в их адреса на этапе выполнения. Для этого понадобилась новая секция в двоичном файле: **глобальная таблица смещений** (Global Offset Table – **GOT**). В процессе компоновки вычисляется положение секции GOT относительно секции `.text` (кода программы). Вместо ссылок на символы подставляются смещения записей в GOT.

На этапе загрузки программы секция GOT преобразуется в сегмент памяти, и со временем этот сегмент заполняется адресами символов. Этот метод, названный «отложенной загрузкой», гарантирует, что загрузчик заполняет записи GOT только по мере необходимости.

Весь исходный код разделяемых библиотек и модулей следует компилировать с флагом PIC. Присваивая свойству цели `POSITION_INDEPENDENT_CODE` значение `ON`, мы просим CMake добавить необходимые зависящие от компилятора флаги, например `-fPIC` для GCC и Clang.

Это свойство автоматически включено для разделяемых библиотек. Но если разделяемая библиотека зависит от другой цели, например статической библиотеки или объекта, то его необходимо добавить также и для этой цели:

```
set_target_properties(dependency
                      PROPERTIES POSITION_INDEPENDENT_CODE ON)
```

Пренебрежение этим шагом вызовет конфликты в CMake, т. к. это свойство проверяется на непротиворечивость. Более подробное объяснение см. в разделе «Конфликты распространяемых свойств» главы 5.

Далее мы обсудим символы. Точнее, в последующих разделах рассматриваются проблемы, связанные с конфликтами имен, которые могут приводить к неоднозначности и несогласованным определениям.

Решение проблем с помощью ODR

Фил Карлтон, главный ворчун и технический родоначальник Netscape, был прав, сказав:

> *В информатике есть две трудности: инвалидация кеша и именование предметов.*

Имена вызывают трудности по нескольким причинам. Они должны быть точными, но простыми, краткими, но выразительными. Имя не только придает смысл сущности, но позволяет программистам понять идеи, лежащие в основе реализации. C++ и многие другие языки несут с собой еще одно осложнение: в большинстве случаев имена должны быть уникальны.

Это требование проявляется в форме ODR: в области видимости одной единицы трансляции (одного `cpp`-файла) символ должен быть определен только один раз, даже если одно и то же имя (будь то переменная, функция, тип класса, перечисление, концепция или шаблон) объявлено несколько раз. Проще говоря, «объявление» вводит символ в область видимости, а «определение» дает все детали, например значение переменной или тело функции.

На этапе компоновки это правило распространяется на всю программу и действует для всех невстраиваемых функций и переменных, использованных в коде. Рассмотрим пример с тремя исходными файлами:

ch08/02-odr-fail/shared.h

```
int i;
```

ch08/02-odr-fail/one.cpp

```
#include <iostream>
#include "shared.h"
int main() {
  std::cout << i << std::endl;
}
```

ch08/02-odr-fail/two.cpp

```
#include "shared.h"
```

Он также содержит списковый файл:

ch08/02-odr-fail/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(ODR CXX)
set(CMAKE_CXX_STANDARD 20)
add_executable(odr one.cpp two.cpp)
```

Как видим, пример очень простой – мы создали заголовочный файл `shared.h`, содержащий определение переменной `i` и используемый в двух единицах трансляции:

- `one.cpp` просто выводит `i` на экран;
- `two.cpp` только включает заголовок.

Но при попытке собрать пример компоновщик выдает ошибку:

```
/usr/bin/ld:
CMakeFiles/odr.dir/two.cpp.o:(.bss+0x0): multiple definition of 'i';
CMakeFiles/odr.dir/one.cpp.o:(.bss+0x0): first defined here
collect2: error: ld returned 1 exit status
```

Символы нельзя определять более одного раза. Но есть важное исключение. Типы, шаблоны и встраиваемые функции с ключевым словом `extern` могут иметь повторные определения в нескольких единицах трансляции, при условии что они идентичны (т. е. состоят из одинаковых последовательностей лексем). Для демонстрации заменим определение переменной определением типа.

ch08/03-odr-success/shared.h

```
struct shared {
  static inline int i = 1;
};
```

А затем используем его следующим образом:

ch08/03-odr-success/one.cpp

```
#include <iostream>
#include "shared.h"
int main() {
  std::cout << shared::i << std::endl;
}
```

Два других файла, `two.cpp` и `CMakeLists.txt`, остались такими же, как в примере `02-odr-fail`. После такого изменения компоновка завершается успешно:

```
[ 33%] Building CXX object CMakeFiles/odr.dir/one.cpp.o
[ 66%] Building CXX object CMakeFiles/odr.dir/two.cpp.o
[100%] Linking CXX executable odr
[100%] Built target odr
```

Альтернативно можно пометить переменную как локальную в единице трансляции (она не будет экспортироваться за пределы объектного файла). Для этого мы воспользуемся ключевым словом `static` (оно контекстно-зависимо, поэтому не путайте его со словом `static` в классах):

ch08/04-odr-success/shared.h

```
static int i;
```

Попытавшись скомпоновать этот пример, вы увидите, что все работает, откуда следует, что статические переменные хранятся отдельно для каждой единицы трансляции. Поэтому изменение одной никак не отражается на остальных.

Правило ODR работает для статических библиотек точно так же, как для объектных файлов, но при сборке кода с разделяемыми библиотеками все не так очевидно. Давайте посмотрим.

Разбираемся с динамически скомпонованными символами-дубликатами

В этом случае компоновщик допускает символы-дубликаты. В примере ниже мы создаем две разделяемые библиотеки, `A` и `B`, с одной повторяющейся функцией `duplicated()` и двумя уникальными функциями `a()` и `b()`:

ch08/05-dynamic/a.cpp

```
#include <iostream>
void a() {
  std::cout << "A" << std::endl;
}
void duplicated() {
  std::cout << "duplicated A" << std::endl;
}
```

Второй файл реализации – почти точная копия первого:

ch08/05-dynamic/b.cpp

```
#include <iostream>
void b() {
  std::cout << "B" << std::endl;
}
void duplicated() {
  std::cout << "duplicated B" << std::endl;
}
```

Теперь обратимся к каждой функции и посмотрим, что произойдет (для простоты мы объявим их локально с ключевым словом `extern`):

ch08/05-dynamic/main.cpp

```
extern void a();
extern void b();
```

```
extern void duplicated();
int main() {
  a();
  b();
  duplicated();
}
```

В этом коде сначала выполняются уникальные функции из каждой библиотеки, а затем вызывается функция, которая в обеих динамических библиотеках имеет одинаковую сигнатуру. И как вы думаете – что произойдет? Важен ли в этом случае порядок компоновки? Протестируем два случая:

- цель `main_1`: сначала производится компоновка с библиотекой `a`;
- цель `main_2`: сначала производится компоновка с библиотекой `b`.

Списковый файл выглядит следующим образом:

ch08/05-dynamic/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Dynamic CXX)
add_library(a SHARED a.cpp)
add_library(b SHARED b.cpp)
add_executable(main_1 main.cpp)
target_link_libraries(main_1 a b)
add_executable(main_2 main.cpp)
target_link_libraries(main_2 b a)
```

После сборки и выполнения обоих исполняемых файлов мы увидим следующий результат:

```
root@ce492a7cd64b:/root/examples/ch08/05-dynamic# b/main_1
A
B
duplicated A

root@ce492a7cd64b:/root/examples/ch08/05-dynamic# b/main_2
A
B
duplicated B
```

Ага! Очевидно, порядок компоновки имеет значение. Это может привести к путанице, если забыть об осмотрительности. Вопреки распространенному мнению конфликты имен встречаются на практике не так уж редко.

Локально видимые символы имеют приоритет перед определенными в DLL. Определив функцию `duplicated()` в файле `main.cpp`, мы переопределим поведение обеих целей. Всегда обращайте внимание на экспорт имен из библиотек, потому что рано или поздно вы обязательно столкнетесь с конфликтом имен.

Используйте пространства имен – не полагайтесь на компоновщик

Пространства имен в C++ были изобретены специально, чтобы избежать таких странных проблем и более эффективно обеспечить выполнение ODR. Лучше всего обернуть весь библиотечный код пространством имен, названным так же, как библиотека. Эта тактика позволит предотвратить сложности, связанные с наличием символов-дубликатов.

В своих проектах мы можем столкнуться со случаями, когда одна разделяемая библиотека компонуется с другой, так что в результате образуется длинная цепочка. Подобные ситуации вовсе не так редки, как могло бы показаться, особенно в запутанных конфигурациях. Поэтому важно понимать, что сама по себе компоновка одной библиотеки с другой не приводит ни к какому наследованию пространств имен. Символы, определенные в каждом звене цепочке, остаются в тех пространствах имен, в которых были откомпилированы.

Хотя хитросплетения компоновщиков интригуют и иногда оказываются существенными, часто возникает еще одна досадная проблема: таинственное исчезновение правильно определенных символов. Мы разберемся с ней в следующем разделе.

Порядок компоновки и неразрешенные символы

Иногда кажется, что компоновщик капризничает и жалуется без видимой причины. Это может стать мучительной проблемой для начинающих программистов, незнакомых с тонкостями инструмента. Неудивительно, что они стараются держаться подальше от конфигурирования сборки настолько долго, насколько получается. Но наступает момент, когда вносить изменение приходится – быть может, ради интеграции с разработанной ими библиотекой, – и понеслось!

Рассмотрим относительно простую цепочку зависимостей, в которой главный исполняемый файл полагается на «внешнюю» библиотеку. Эта библиотека, в свою очередь, зависит от «вложенной» библиотеки, которая содержит необходимую переменную `int b`. И тут внезапно взору программиста предстает такое загадочное сообщение об ошибке:

```
outer.cpp:(.text+0x1f): undefined reference to 'b'
```

Подобные ошибки вовсе не редкость. Обычно они означают, что мы забыли указать компоновщику библиотеку. Но в данном случае библиотека вроде бы была правильно добавлена командой `target_link_libraries()`:

ch08/06-unresolved/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Order CXX)
add_library(outer outer.cpp)
add_library(nested nested.cpp)
add_executable(main main.cpp)
target_link_libraries(main nested outer)
```

Что же тогда?! Очень немногие ошибки до такой степени трудно понять и отладить. То, что мы наблюдаем, – следствие неправильного порядка компоновки. Чтобы выяснить причину, обратимся к исходному коду.

ch08/06-unresolved/main.cpp

```
#include <iostream>
extern int a;
int main() {
  std::cout << a << std::endl;
}
```

Код кажется совсем простым – мы хотим напечатать внешнюю переменную `a`, которую можно найти в библиотеке `outer`. Мы заблаговременно объявили ее с ключевым словом `extern`. Вот исходный код библиотеки:

ch08/06-unresolved/outer.cpp

```
extern int b;
int a = b;
```

Он тоже простой – `outer` зависит от библиотеки `nested`, которая должна предоставить внешнюю переменную, `b`, присваиваемую переменной `a`. Заглянем в код `nested` и убедимся, что мы не забыли определить `b`.

ch08/06-unresolved/nested.cpp

```
int b = 123;
```

Определение `b` имеется, а поскольку оно не сделано локальным путем добавления ключевого слова `static`, оно корректно экспортируется из цели `nested`. Как мы уже видели, эта цель компонуется с исполняемым файлом `main` в `CMakeLists.txt`:

```
target_link_libraries(main nested outer)
```

Так откуда же взялось сообщение `undefined reference to 'b'`?

Разрешение неопределенных символов производится следующим образом: компоновщик обрабатывает двоичные файлы слева направо. В процессе их обхода он выполняет следующие действия:

1. Собрать все неопределенные символы, экспортируемые из этого двоичного файла, и сохранить их на будущее.
2. Попытаться разрешить неопределенные символы (собранные из всех ранее обработанных двоичных файлов), связав их с символами, определенными в этом файле.
3. Повторить этот процесс для следующего двоичного файла.

Если после завершения операции остались неопределенные символы, компоновка завершается неудачно. Именно так произошло в нашем примере (CMake размещает объектные файлы исполняемой цели перед библиотеками).

1. Компоновщик обработал `main.o`, нашел неопределенную ссылку на переменную `a` и запомнил ее для будущего разрешения.
2. Компоновщик обработал `libnested.a`, не нашел ни неопределенных ссылок, ни символов, которые можно было бы разрешить.
3. Компоновщик обработал `libouter.a`, нашел неопределенную ссылку на переменную `b` и разрешил ссылку на переменную `a`.

Мы сумели корректно разрешить ссылку на переменную `a`, но не на переменную `b`. Чтобы исправить ситуацию, мы должны изменить порядок компоновки, так чтобы `nested` стояла после `outer`:

```
target_link_libraries(main outer nested)
```

Иногда встречаются циклические ссылки, когда единицы трансляции определяют символы друг для друга и ни при каком порядке компоновки все ссылки разрешить не удается. Единственный способ разрубить этот узел – обработать некоторые цели дважды:

```
target_link_libraries(main nested outer nested)
```

Это обычная практика, хотя выглядит она не элегантно. Если вы имеете возможность использовать версию CMake 3.24 или более позднюю, то к вашим услугам генераторное выражение `$<LINK_GROUP>` с возможностью `RESCAN`, которое добавляет зависящие от компоновщика флаги, например `--start-group` или `--end-group`, гарантирующие, что все символы будут вычислены:

```
target_link_libraries(main "$<LINK_GROUP:RESCAN,nested,outer>")
```

Имейте в виду, что с этим механизмом сопряжены дополнительные шаги обработки, поэтому использовать его следует только в случае острой необходимости. Не так уж много случаев, когда без циклических ссылок не обойтись. Обычно это свидетельствует о плохом дизайне. Но эта возможность поддерживается в Linux, BSD, SunOS и Windows с комплектом инструментов GNU.

Теперь мы готовы к встрече с проблемами ODR. С чем еще можно столкнуться? С подозрительно отсутствующими символами на этапе компоновки. Посмотрим, о чем идет речь.

Символы, на которые нет ссылок

Библиотеки, особенно статические, по существу являются архивами, содержащими несколько файлов. Мы упоминали об инструментах, которые умеют создавать индексы символов для ускорения процесса компоновки. Индексы предоставляют отображение между символом и объектным файлом, в котором он определен. В процессе разрешения символа содержащий его объектный файл включается в результирующий двоичный файл (некоторые компоновщики еще сильнее оптимизируют этот процесс, включая только определенные секции файла). Если ни на один символ, определенный в объектном файле из статической библиотеки, нет ссылок, то этот файл может быть опущен. Поэтому в конечном двоичном файле присутствуют только те части статической библиотеки, которые действительно используются.

Однако есть несколько сценариев, когда нам могут понадобиться символы, на которые нет ссылок.

- **Статическая инициализация:** если в вашей библиотеке имеются глобальные объекты, нуждающиеся в инициализации (т. е. выполняются их конструкторы) до вызова `main()`, и на эти объекты больше нигде нет прямых ссылок; такие объекты компоновщик может исключить из конечного двоичного файла.
- **Архитектуры с плагинами:** если вы разрабатываете систему с плагинами (модульными библиотеками), когда код должен быть найден и загружен во время выполнения, хотя на него нет прямых ссылок.
- **Неиспользуемый код в статических библиотеках:** если вы разрабатываете статическую библиотеку, содержащую служебные функции или код, на который не всегда есть прямая ссылка, но который вы тем не менее хотите включить в конечный двоичный файл.
- **Конкретизации шаблонов:** для библиотек, в которых активно используются шаблоны; некоторые конкретизации могут быть пропущены во время компоновки, если они не упомянуты явно.
- **Проблемы компоновки:** в особо запутанных сборочных системах или кодовых базах компоновка может давать непредсказуемые результаты, если кажется, что некоторые символы или секции кода отсутствуют.

В таких ситуациях может помочь принудительное включение всех объектных файлов в процессе компоновки. Часто для этого используется режим компоновки `whole-archive`.

Для конкретных компиляторов соответствующие флаги компоновщика называются так:

- `--whole-archive` для GCC;
- `--force-load` для Clang;
- `/WHOLEARCHIVE` для MSVC.

Для этого можно воспользоваться командой `target_link_options()`:

```
target_link_options(tgt INTERFACE
  -Wl,--whole-archive $<TARGET_FILE:lib1> -Wl,--no-whole-archive
)
```

Однако эта команда зависит от компоновщика, поэтому включение генераторных выражений для определения компилятора и задания подходящих флагов обязательно. По счастью, в версии CMake 3.24 появилось новое генераторное выражение для этой цели:

```
target_link_libraries(tgt INTERFACE
  "$<LINK_LIBRARY:WHOLE_ARCHIVE,lib1>"
)
```

Использование этого метода гарантирует, что цель `tgt` включает все объектные файлы из библиотеки `lib1`. Но у него есть несколько недостатков, требующих внимания.

- **Увеличение размера двоичного файла:** этот флаг может привести к значительному увеличению конечного двоичного файла, потому что включаются все объекты из указанной библиотеки, даже неиспользуемые.
- **Возможность конфликта символов:** включение всех символов может приводить к конфликтам имен и, как следствие, к ошибкам компоновки.
- **Усложнение сопровождения:** надежда на такие флаги может скрыть внутренние проблемы дизайна или структуры кода.

Разобравшись с тем, как решать типичные проблемы компоновки, мы можем перейти к подготовке проектов к тестированию.

Вынесение логики main() для тестирования

Как мы выяснили, компоновщик требует соблюдения правила одного определения (ODR) и гарантирует, что все внешние символы получат определения в процессе компоновки. Еще одна связанная с компоновщиком проблема – как элегантно и эффективно организовать тестирование проекта.

В идеале мы должны были бы тестировать в точности тот исходный код, который будет работать в условиях эксплуатации. Полный конвейер собрал бы исходный код, прогнал тесты для результирующего двоичного кода, а затем упаковал и распространил его (возможно, исключив из пакета сами тесты).

Но как это реализовать? У исполняемых файлов обычно имеется точно определенный поток выполнения, часто подразумевающий чтение аргументов командной строки. Поскольку язык C++ компилируемый, он не поддерживает временную вставку в двоичный код плагинов, предназначенных только для тестирования. Поэтому задача требует более тонкого подхода.

По счастью, компоновщик позволяет справиться с ней весьма элегантно. Подумайте о том, чтобы вынести всю логику `main()` во внешнюю функцию, `start_program()`:

ch08/07-testing/main.cpp

```
extern int start_program(int, const char**);
int main(int argc, const char** argv) {
  return start_program(argc, argv);
}
```

Если `main()` написана в таком виде, то разумно было бы опустить ее тестирование; она лишь передает все аргументы функции, определенной в каком-то другом файле. Тогда мы можем создать библиотеку, содержащую исходный код, вынесенный из `main()` в новую функцию – `start_program()`. В примере ниже код проверяет, что количество аргументов командной строки больше 1.

ch08/07-testing/program.cpp

```
#include <iostream>
int start_program(int argc, const char** argv) {
  if (argc <= 1) {
    std::cout << "Not enough arguments" << std::endl;
    return 1;
  }
  return 0;
}
```

Теперь мы можем подготовить проект, который собирает это приложение и компонует эти две единицы трансляции:

ch08/07-testing/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Testing CXX)
add_library(program program.cpp)
add_executable(main main.cpp)
target_link_libraries(main program)
```

Цель `main` только предоставляет обязательную функцию `main()`. Логика проверки аргументов командной строки находится в цели `program`. Теперь мы можем протестировать ее, создав еще один исполняемый файл со своей собственной функцией `main()`, которая будет содержать все тестовые сценарии.

На практике каркасы типа **GoogleTest** или **Catch2** предоставляют собственный метод `main()`, которым можно заменить точку входа в вашу программу, так что он будет прогонять все определенные вами тесты. Мы подробнее рассмотрим этот вопрос в главе 11. А пока сосредоточимся на общем принципе и поместим свои тесты прямо в функцию `main()`:

ch08/07-testing/test.cpp

```
#include <iostream>
extern int start_program(int, const char**);
using namespace std;
int main()
```

```
{
  cout << "Test 1: Passing zero arguments to start_program:\n";
  auto exit_code = start_program(0, nullptr);
  if (exit_code == 0)
    cout << "Test FAILED: Unexpected zero exit code.\n";
  else
    cout << "Test PASSED: Non-zero exit code returned.\n";
  cout << endl;

  cout << "Test 2: Passing 2 arguments to start_program:\n";
  const char *arguments[2] = {"hello", "world"};
  exit_code = start_program(2, arguments);
  if (exit_code != 0)
    cout << "Test FAILED: Unexpected non-zero exit code\n";
  else
    cout << "Test PASSED\n";
}
```

Здесь `start_program` вызывается дважды, с аргументами и без, и в каждом случае проверяется, что возвращенный код правилен. В распечатке ниже показаны результаты тестов:

```
./test
Test 1: Passing zero arguments to start_program:
Not enough arguments
Test PASSED: Non-zero exit code returned

Test 2: Passing 2 arguments to start_program:
Test PASSED
```

Строка `Not enough arguments` напечатана функцией `start_program()` и является ожидаемым сообщением об ошибке (мы проверяем, что программа завершилась неудачно).

Этот автономный тест оставляет желать много лучшего с точки зрения чистоты кода и элегантности методики тестирования, но это только начало.

Мы определили `main()` дважды:

- в файле `main.cpp` для производственной версии;
- в файле `test.cpp` для тестирования.

Теперь определим тестовый исполняемый файл в конце файла `CMakeLists.txt`:

```
add_executable(test test.cpp)
target_link_libraries(test program)
```

Это добавление создает новую цель, которая компонуется с тем же двоичным кодом, что и производственный код. Но при этом мы можем вызывать те

экспортированные функции, которые нам необходимы, а потому выполнять все пути в коде автоматически и проверять, работают ли они, как ожидается. Отлично!

Резюме

Компоновка в CMake поначалу может показаться не предвещающей трудностей, но, копнув глубже, мы увидим много скрытых деталей. И действительно, скомпоновать исполняемый файл не так просто, как собрать кусочки пазла. Взгляд внутрь объектных файлов и библиотек откроет нам, что секции, где хранятся различные типы данных, команды, имена символов и прочее, нуждаются в переупорядочении. Прежде чем программу можно будет выполнить, эти секции подвергаются перемещению.

Никак не обойтись без разрешения символов. Компоновщик должен перебрать все ссылки во всех единицах трансляции, ничего не пропустив. Решив эту проблему, компоновщик создает заголовок программы и помещает его в конечный исполняемый файл. Заголовок содержит инструкции системному загрузчику, в которых описано, как преобразовать консолидированные секции в сегменты, составляющие образ процесса в памяти на этапе выполнения. Мы также обсудили все три типа библиотек: статические, разделяемые и разделяемые модули. Мы узнали, чем они отличаются и при каких условиях один следует предпочесть другому. Кроме того, мы затронули тему позиционно-независимого кода (PIC) – важной концепции, которая лежит в основе отложенного связывания символов.

Правило одного определения (ODR) – понятие C++, но, как мы видели, оно всесторонне поддержано компоновщиками. Мы поговорили о том, как решать типичные проблемы дублирования символов в статических и динамических библиотеках. Мы также подчеркнули важность использования пространств имен в тех случаях, когда это возможно, и порекомендовали не слишком полагаться на средства предотвращения конфликтов символов, встроенные в компоновщик.

Для шага, который мог бы показаться тривиальным (учитывая ограниченность набора команд CMake, посвященных компоновке), трудностей вполне достаточно. Один из самых тонких вопросов – порядок компоновки, особенно когда библиотеки имеют вложенные и циклические зависимости. Теперь мы понимаем, как компоновщик отбирает символы, попадающие в конечный двоичный файл, и как при необходимости можно переопределить это поведение.

Наконец, мы поговорили о том, как с помощью компоновщика подготовить программу к тестированию – вынеся логику функции `main()` в другую единицу трансляции. Это позволило нам ввести еще один исполняемый файл, который прогоняет тесты точно того же машинного кода, который будет выполняться в производственной программе.

С новообретенными знаниями о компоновке мы готовы включить в свои проекты CMake внешние библиотеки. В следующей главе мы рассмотрим управление зависимостями в CMake.

Для дополнительного чтения

Дополнительные сведения по рассмотренным в этой главе темам можно получить из следующих источников:

- структура ELF-файлов: https://en.wikipedia.org/wiki/Executable_and_Linkable_Format;
- руководство CMake по использованию `add_library()`: https://cmake.org/cmake/help/latest/command/add_library.html;
- ад зависимостей: https://en.wikipedia.org/wiki/Dependency_hell;
- различие между модулями и разделяемыми библиотеками: https://stackoverflow.com/questions/4845984/difference-between-modulesand-shared-libraries.

9

Управление зависимостями в CMake

Неважно, большое ваше решение или маленькое; по мере роста оно, скорее всего, начнет зависеть от других проектов. Важно не тратить силы на создание и сопровождение стереотипного кода. Это освободит вам время для действительно важных вещей: бизнес-логики. Внешние зависимости служат нескольким целям. Они предлагают каркасы и средства, решают трудные задачи и играют ключевую роль в сборке и контроле качества кода. Зависимости могут быть самыми разными: от специализированных компиляторов типа **Protocol Buffers** (**Protobuf**) до каркасов тестирования типа Google Test.

При работе с проектами с открытым исходным кодом или разрабатываемыми внутри организации эффективное управление внешними зависимостями – насущная необходимость. Если делать это вручную, то будет уходить много времени на подготовку и сопровождение. К счастью, CMake предлагает различные подходы к управлению зависимостями, ни в чем не отставая от промышленных стандартов.

Сначала мы узнаем, как идентифицировать и использовать зависимости, уже имеющиеся в хост-системе, избежав тем самым ненужного скачивания и временных затрат на компиляцию. Эта задача сравнительно проста, потому что многие пакеты или совместимы с CMake, или поддерживаются CMake «из коробки». Мы также рассмотрим, как попросить CMake найти и включить зависимости, лишенные такой поддержки. Для унаследованных пакетов в определенных ситуациях может оказаться полезным альтернативный подход: использовать когда-то популярный инструмент `pkg-config` в особенно неприятных случаях.

Кроме того, мы обсудим управление зависимостями, которые имеются в сети, но еще не установлены в систему. Мы рассмотрим, как скачать их с HTTP-серверов, из Git и репозиториев других типов. Мы также поговорим

о том, как выбрать оптимальный подход: сначала поискать в системе, а затем прибегнуть к скачиванию пакета. Наконец, мы упомянем устаревший метод скачивания внешних проектов, который может оказаться полезен в особых случаях.

В этой главе будут рассмотрены следующие вопросы:

- использование уже установленных зависимостей;
- использование зависимостей, отсутствующих в системе.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch09.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Использование уже установленных зависимостей

Если наш проект зависит от популярной библиотеки, то велики шансы на то, что нужный пакет уже установлен в системе. Нужно только подключить его к процессу сборки нашего проекта. Как это сделать? Мы должны найти пакет в системе, чтобы CMake могла использовать принадлежащие ему файлы. Это можно сделать и вручную, но все окружения чем-то да отличаются друг от друга. Способ, работающий в одной системе, может оказаться непригоден в другой. Поэтому следует автоматически искать эти пути на этапе сборки. Есть разные методы решения этой задачи, но обычно лучше всего воспользоваться встроенной в CMake командой **find_package()**, которая знает, как искать многие популярные пакеты.

Если нужный нам пакет не поддерживается, то есть два варианта:

- написать небольшой плагин `find-module`, используемый в сочетании с `find_package()`;
- воспользоваться старым методом `pkg-config`.

Начнем с рекомендованного варианта.

Нахождение пакетов с помощью команды CMake find_package()

Рассмотрим следующий сценарий: вы хотите улучшить способ взаимодействия по сети или хранения данных в своем проекте. Простые текстовые файлы или открытые текстовые форматы типа JSON и XML не устраивают вас с точки зрения размера. Неплохо было бы использовать двоичный формат, и похоже, что хорошо известная библиотека Protobuf от Google может решить проблему.

Вы прочитали инструкцию и установили все необходимое в свою систему. Что дальше? Как заставить команду CMake `find_package()` найти и использовать новую библиотеку?

В этом примере нам понадобится установить нужные зависимости, потому что `find_package()` умеет искать только пакеты, уже присутствующие в системе. Она предполагает, что вы уже все установили или что пользователи знают, как установить то, что им сказали. Если вы находитесь в иной ситуации, то необходим резервный план. Подробнее об этом см. раздел «Использование зависимостей, отсутствующих в системе» ниже.

В случае Protobuf ситуация довольно проста: вы можете либо скачать, откомпилировать и установить библиотеку самостоятельно, взяв ее из официального репозитория (https://github.com/protocolbuffers/protobuf), либо воспользоваться менеджером пакетов в своей операционной системе. Если вы прорабатываете примеры, используя Docker-образ, упомянутый в главе 1, то зависимости уже установлены и делать ничего не нужно. Но если вы хотите заняться самостоятельной установкой, то в Debian Linux нужно будет выполнить такие команды:

```
$ apt update
$ apt install protobuf-compiler libprotobuf-dev
```

В наши дни многие проекты поддерживают CMake. Для этого они создают **конфигурационный файл** и помещают его в соответствующий системный каталог при установке. Конфигурационные файлы – неотъемлемая часть проектов, поддерживающих CMake.

Если вы хотите использовать библиотеку, не имеющую конфигурационного файла, тоже не беда. CMake поддерживает внешний механизм поиска таких библиотек, называемый **модулями поиска**. В отличие от конфигурационных файлов, модули поиска не являются частью проекта, который помогают найти. На самом деле в самой CMake уже имеются модули поиска для многих популярных библиотек.

Если нет ни конфигурационного файла, ни модуля поиска, то имеются и другие варианты:

- написать собственные модули поиска для конкретного пакета и включить их в свой проект;
- использовать модуль FindPkgConfig, работающий с унаследованными файлами определения пакетов в Unix;

- написать конфигурационный файл и попросить лиц, сопровождающих пакет, включить его.

Возможно, вы думаете, что не готовы создавать такие запросы на включение. Ничего страшного, потому что, скорее всего, вам этого и не придется делать. В состав CMake входит свыше 150 модулей поиска, умеющих находить такие библиотеки, как Boost, bzip2, curl, curses, GIF, GTK, iconv, ImageMagick, JPEG, Lua, OpenGL, OpenSSL, PNG, PostgreSQL, Qt, SDL, Threads, XML-RPC, X11 и zlib, а также файл Protobuf, который мы будем использовать в этом примере. Полный перечень имеется в документации CMake (см. раздел «Для дополнительного чтения»).

В команде CMake `find_package()` можно использовать как модули поиска, так и конфигурационные файлы. Сначала CMake проверяет встроенные модули поиска. Если она не находит искомого, то переходит к проверке конфигурационных файлов, предоставленных различными пакетами. CMake просматривает каталоги, в которые обычно устанавливаются пакеты (зависят от операционной системы), и ищет файлы, отвечающие следующим образцам:

- `<CamelCasePackageName>Config.cmake`;
- `<kebab-case-package-name>-config.cmake`.

Если вы хотите добавить внешние модули поиска в свой проект, то установите переменную `CMAKE_MODULE_PATH`. Первым делом CMake будет просматривать этот каталог.

В нашем примере цель проста: я хочу показать, как можно без труда собрать проект, в котором используется `Protobuf`. Не пугайтесь, для понимания происходящего знакомство с Protobuf необязательно. Не вдаваясь в подробности, скажу, что Protobuf – это библиотека, которая сохраняет данные в некотором двоичном формате. Это позволяет легко записывать и читать объекты C++ из файлов и передавать их по сети. Для настройки мы воспользуемся `proto`-файлом, чтобы сообщить Protobuf о структуре данных:

ch09/01-find-package-variables/message.proto

```
syntax = "proto3";
message Message {
  int32 id = 1;
}
```

Это простое определение схемы, содержащее только одно 32-разрядное целое число. В пакет Protobuf входит двоичный файл, который компилирует такие `proto`-файлы в исходные и заголовочные файлы C++, пригодные для использования в нашем приложении. Мы должны будем добавить этот шаг компиляции в свой процесс сборки, но займемся этим позже. А пока посмотрим, как в нашем файле `main.cpp` используется вывод, сгенерированный Protobuf:

ch09/01-find-package-variables/main.cpp

```
#include "message.pb.h"
#include <fstream>
using namespace std;
```

```
int main()
{
  Message m;
  m.set_id(123);
  m.PrintDebugString();
  fstream fo("./hello.data", ios::binary | ios::out);
  m.SerializeToOstream(&fo);
  fo.close();
  return 0;
}
```

Я включил заголовок `message.pb.h`, который должен быть сгенерирован Protobuf. В этом заголовке будет находиться определение объекта `Message`, сгенерированное по файлу `message.proto`. В функции `main()` я создаю простой объект типа `Message`. В его поле `id` я записываю произвольное значение (`123`), а потом печатаю отладочную информацию на стандартный вывод. Затем в файловый поток записывается двоичная версия этого объекта. Это самый простой способ использования библиотеки сериализации, каковой является Protobuf.

Заголовок `message.pb.h` должен быть сгенерирован до компиляции `main.cpp`. Это делает компилятор Protobuf `protoc`, который на входе принимает файл `message.proto`. Управление этим процессом кажется сложным, но на самом деле это не так!

Вот где вершится магия CMake:

ch09/01-find-package-variables/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(FindPackageProtobufVariables CXX)
find_package(Protobuf REQUIRED)
protobuf_generate_cpp(GENERATED_SRC GENERATED_HEADER
                      message.proto)
add_executable(main main.cpp ${GENERATED_SRC} ${GENERATED_HEADER})
target_link_libraries(main PRIVATE ${Protobuf_LIBRARIES})
target_include_directories(main PRIVATE
  ${Protobuf_INCLUDE_DIRS} ${CMAKE_CURRENT_BINARY_DIR}
)
```

Рассмотрим этот код по частям.

- Первые две строки понятны: настраивается проект и указывается язык C++.
- Команда `find_package(Protobuf REQUIRED)` просит CMake найти библиотеку Protobuf (путем выполнения встроенного модуля поиска `FindProtobuf.cmake`) и подготовить ее к использованию в нашем проекте. Если найти библиотеку не удастся, то сборка остановится, потому что мы добавили ключевое слово `REQUIRED`.
- `protobuf_generate_cpp` – специальная функция, определенная в модуле поиска Protobuf. Она автоматизирует процесс вызова компилятора `pro-`

toc. После успешной компиляции она сохраняет пути к сгенерированным исходным файлам в переменных, переданных в первых двух аргументах: GENERATED_SRC и GENERATED_HEADER. Все последующие аргументы будут рассматриваться как список подлежащих компиляции файлов (message.proto).

- add_executable создает исполняемый файл из main.cpp и файлов, сгенерированных Protobuf.
- target_link_libraries просит CMake прикомпоновать библиотеки Protobuf к нашему исполняемому файлу.
- target_include_directories() добавляет в пути к включаемым файлам пути INCLUDE_DIRS, предоставляемые пакетом, и CMAKE_CURRENT_BINARY_DIR. Последний говорит компилятору, где искать заголовок message.pb.h.

Модуль поиска Protobuf предлагает следующую функциональность:

- находит библиотеку и компилятор Protobuf;
- предоставляет вспомогательные функции для компиляции proto-файлов;
- записывает в переменные пути к включаемым и компонуемым файлам.

Не каждый модуль включает такие удобные вспомогательные функции, как Protobuf, но большинство устанавливает несколько ключевых переменных, полезных для управления зависимостями в вашем проекте. Неважно, что вы используете – встроенный модуль поиска или конфигурационный файл, – после успешного нахождения пакета можно рассчитывать на установку следующих переменных:

- <PKG_NAME>_FOUND: показывает, был ли пакет найден;
- <PKG_NAME>_INCLUDE_DIRS или <PKG_NAME>_INCLUDES: указывает на каталоги, в которых находятся заголовочные файлы пакета;
- <PKG_NAME>_LIBRARIES или <PKG_NAME>_LIBS: списки библиотек, с которыми нужно компоновать исполняемый файл;
- <PKG_NAME>_DEFINITIONS: необходимые пакету определения компилятора.

Сразу после выполнения find_package() можете проверить переменную <PKG_NAME>_FOUND и узнать, смогла ли CMake найти пакет.

Если модуль пакета написан для версии CMake 3.10 или более поздней, то он, скорее всего, предоставляет также определения целей. Эти цели будут импортированными, чтобы сразу было понятно, что они происходят из внешней зависимости.

Protobuf – прекрасный пример того, как можно узнать о зависимостях в CMake, поскольку в нем определяются зависящие от модуля переменные и импортированные цели. Такие цели позволяют написать еще более краткий код.

ch09/02-find-package-targets/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(FindPackageProtobufTargets CXX)
find_package(Protobuf REQUIRED)
protobuf_generate_cpp(GENERATED_SRC GENERATED_HEADER
```

```
  message.proto)
add_executable(main main.cpp ${GENERATED_SRC} ${GENERATED_HEADER})
target_link_libraries(main PRIVATE protobuf::libprotobuf)
target_include_directories(main PRIVATE
                                ${CMAKE_CURRENT_BINARY_DIR})
```

Сравните выделенный код с предыдущей версией этого примера: вместо использования переменных для списковых файлов и каталогов разумнее использовать импортированные цели. При таком подходе упрощается списковый файл. Кроме того, автоматически учитываются транзитивные требования использования, или распространяемые свойства, как видно на примере цели `protobuf::libprotobuf`.



Если вы хотите точно знать, что предоставляет конкретный модуль поиска, то лучше всего обратиться к онлайновой документации. Например, подробную информацию о Protobuf можно найти на сайте CMake по адресу https://cmake.org/cmake/help/latest/module/FindProtobuf.html.

Для простоты примеры в этом разделе завершаются неудачно, если библиотека Protobuf не найдена в системе пользователя. Но по-настоящему надежное решение проверило бы переменную `Protobuf_FOUND` и вывело бы подробное диагностическое сообщение (чтобы пользователь мог установить пакет) или выполнило бы установку автоматически. Как это сделать, мы узнаем в следующем разделе.

Команда `find_package()` принимает несколько аргументов. Мы рассмотрим только самые важные. В простейшем случае команда имеет следующую сигнатуру:

```
find_package(<Name> [version] [EXACT] [QUIET] [REQUIRED])
```

Опишем, что означает каждый аргумент.

- `[version]`: определяет минимально необходимую версию пакета в формате `major.minor.patch.tweak` (например, `1.22`). Можно также задать диапазон, например `1.22...1.40.1`, используя три точки в качестве разделителя;
- `EXACT`: используется, когда `[version]` – не диапазон, чтобы сообщить CMake, что вам нужна именно эта версия, а не более поздняя;
- `QUIET`: подавляет сообщения о том, был пакет найден или нет;
- `REQUIRED`: останавливает сборку, если пакет не найден, и выводит диагностическое сообщение, даже если присутствует параметр `QUIET`.

Ели вы уверены, что пакет должен быть в системе, но `find_package()` его не находит, то есть способ копнуть глубже. Начиная с версии CMake 3.24 можно выполнять этап конфигурирования в отладочном режиме для получения более полной информации. Используйте команду

```
cmake -B <build tree> -S <source tree> --debug-find-pkg=<pkg>
```

Но будьте осторожны. Имя пакета чувствительно к регистру, поэтому должно быть введено точно.

Дополнительные сведения о команде `find_package()` можно найти на странице документации: https://cmake.org/cmake/help/latest/command/find_package.html.

Модули поиска задуманы как очень удобный способ предоставления CMake информации об установленных зависимостях. Большинство популярных библиотек поддержаны CMake на всех основных платформах. Но что делать, если нам нужна библиотека, для которой еще нет готового модуля поиска?

Написание модулей своими силами

Редко, но бывает, что нужная вам позарез библиотека не предоставляет конфигурационного файла и для нее в CMake еще нет готового модуля поиска. Тогда вы можете написать для нее специальный модуль поиска и поставлять его вместе со своим проектом. Не очень хочется, но в интересах пользователей это нужно сделать.

Попробуем написать специальный модуль поиска для библиотеки `libpqxx`, клиента базы данных PostgreSQL. Она уже установлена в Docker-образе для этой книги, так что беспокоиться вам не о чем. Те, кто работает с Debian, могут установить ее в составе пакета `libpqxx-dev` (в других операционных системах команды могут быть иными):

```
apt-get install libpqxx-dev
```

Начнем с написания файла `FindPQXX.cmake`, который сохраним в каталоге `cmake/module` в дереве исходного кода нашего проекта. Чтобы CMake могла найти этот модуль поиска при вызове команды `find_package()`, мы добавим путь к нему в переменную `CMAKE_MODULE_PATH` в нашем файле `CMakeLists.txt` командой `list(APPEND)`. Напомним, что CMake сначала ищет модули поиска в каталогах, перечисленных в `CMAKE_MODULE_PATH`, а только потом переходит к другим местам. Полный списковый файл должен выглядеть следующим образом:

ch09/03-find-package-custom/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(FindPackageCustom CXX)
list(APPEND CMAKE_MODULE_PATH
            "${CMAKE_SOURCE_DIR}/cmake/module/")
find_package(PQXX REQUIRED)
add_executable(main main.cpp)
target_link_libraries(main PRIVATE PQXX::PQXX)
```

Теперь можно перейти к написанию собственно модуля поиска. Если файл `FindPQXX.cmake` пуст, то CMake не выдаст никаких ошибок, даже если в команде

`find_package()` имеется параметр `REQUIRED`. Задавать переменные и следовать рекомендациям (например, возбуждать ошибки) – обязанность автора модуля поиска. Согласно предписаниям CMake, вот на что следует обратить внимание:

- когда вызывается `find_package(<PKG_NAME> REQUIRED)`, CMake устанавливает переменную `<PKG_NAME>_FIND_REQUIRED` в 1. Тогда модуль поиска должен вызвать команду `message(FATAL_ERROR)`, если библиотека не найдена;
- если используется `find_package(<PKG_NAME> QUIET)`, то CMake устанавливает переменную `<PKG_NAME>_FIND_QUIETLY` в 1. В этом случае модуль поиска не должен выдавать дополнительные сообщения;
- CMake записывает в переменную `<PKG_NAME>_FIND_VERSION` версию, указанную в списковых файлах. Если модуль поиска не может найти нужную версию, то должен возбудить ошибку `FATAL_ERROR`.

Разумеется, лучше следовать этим предписаниям, чтобы не выглядеть белой вороной на фоне других модулей поиска.

Для создания элегантного модуля поиска для PQXX выполним следующие действия.

1. Если пути к библиотеке и заголовкам уже известны (заданы пользователем или получены из кеша предыдущего выполнения), то использовать их для создания импортированной цели. На этом можно остановиться.
2. Если пути неизвестны, то начать нужно с поиска библиотеки и заголовков интересующей нас зависимости, в данном случае PostgreSQL.
3. Далее искать в хорошо известных каталогах двоичную версию клиентской библиотеки PostgreSQL.
4. Аналогично искать в известных каталогах заголовочные файлы клиента PostgreSQL.
5. Наконец, убедиться в том, что библиотека и заголовочные файлы найдены. Если это так, создать импортированную цель.

Чтобы создать надежный модуль поиска для PQXX, сосредоточимся на двух важных задачах. Прежде всего создание импортированной цели может иметь место в двух случаях – либо пользователь задал пути к библиотеке, либо пути обнаружены автоматически. Чтобы избежать дублирования кода, напишем функцию, которая управляет процессом поиска.

Определение импортированных целей

Для настройки импортированной цели нам всего лишь нужно определить библиотеку с ключевым словом `IMPORTED`. Это позволит использовать команду `target_link_libraries()` при вызове спискового файла `CMakeLists.txt`. Мы должны задать тип библиотеки и для простоты зададим его как `UNKNOWN`. Это означает, что нам не важно, статическая библиотека или динамическая: мы только хотим передать аргумент компоновщику.

Далее зададим существенные свойства нашей цели, а именно `IMPORTED_LOCATION` и `INTERFACE_INCLUDE_DIRECTORIES`. Для этого используем переданные функции аргументы. Можно было бы задать дополнительные свойства, например `COMPILE_DEFINITIONS`, но для `PQXX` они не нужны.

После этого, чтобы модуль поиска работал эффективно, сохраним найденные пути в кешированных переменных. Тогда нам не придется повторять поиск при следующих запусках. Отметим, что мы явно помещаем `PQXX_FOUND` в кеш, так что она будет глобально доступна, и пользовательский файл `CMakeLists.txt` сможет ссылаться на нее.

Наконец, мы пометим эти кешированные переменные как `advanced` и не будем показывать их в графическом интерфейсе CMake, если только режим `advanced` не активирован. Это общепринятая практика, которой мы будем следовать.

Ниже приведен код этих операций.

ch09/03-find-package-custom/cmake/module/FindPQXX.cmake

```
# Определение импортированных целей
function(define_imported_target library headers)
  add_library(PQXX::PQXX UNKNOWN IMPORTED)
  set_target_properties(PQXX::PQXX PROPERTIES
    IMPORTED_LOCATION ${library}
    INTERFACE_INCLUDE_DIRECTORIES ${headers}
  )
  set(PQXX_FOUND 1 CACHE INTERNAL "PQXX found" FORCE)
  set(PQXX_LIBRARIES ${library}
      CACHE STRING "Path to pqxx library" FORCE)
  set(PQXX_INCLUDES ${headers}
     CACHE STRING "Path to pqxx headers" FORCE)
  mark_as_advanced(FORCE PQXX_LIBRARIES)
  mark_as_advanced(FORCE PQXX_INCLUDES)
endfunction()
```

Теперь обсудим, как использовать специальные или ранее сохраненные пути, чтобы ускорить инициализацию.

Согласие с указанными пользователем путями и повторное использование кешированных значений

Рассмотрим ситуацию, когда пользователь установил `PQXX` в нестандартное место и указал необходимые пути в командной строке с помощью флага `-D`. В этом случае мы сразу же вызываем определенную ранее функцию и прекращаем поиск командой `return()`. Мы предполагаем, что пользователь указал правильные пути к библиотеке и ее зависимостям:

ch09/03-find-package-custom/cmake/module/FindPQXX.cmake (продолжение)

```
...

# Согласие с указанными пользователем путями и повторное использование
# кешированных значений
if (PQXX_LIBRARIES AND PQXX_INCLUDES)
  define_imported_target(${PQXX_LIBRARIES} ${PQXX_INCLUDES})
```

```
    return()
  endif()
```

Это условие остается справедливым и тогда, когда конфигурирование было выполнено ранее, потому что переменные PQXX_LIBRARIES и PQXX_INCLUDES сохранены в кеше.

Теперь посмотрим, как обрабатываются дополнительные библиотеки, от которых зависит PQXX.

Поиск вложенных зависимостей

Для использования PQXX в хост-системе также должна быть установлена СУБД PostgreSQL. Нет ничего плохого в том, чтобы использовать еще один модуль поиска в текущем модуле, но мы должны передать флаги REQUIRED и QUIET, чтобы поведение основного и вложенного поиска было согласовано. Для этого заведем две вспомогательные переменные для хранения ключевых слов, которые необходимо передать, и запишем в них значения аргументов, полученных от CMake: PQXX_FIND_QUIETLY и PQXX_FIND_REQUIRED.

```
# Поиск вложенных зависимостей
set(QUIET_ARG)
if(PQXX_FIND_QUIETLY)
  set(QUIET_ARG QUIET)
endif()

set(REQUIRED_ARG)
if(PQXX_FIND_REQUIRED)
  set(REQUIRED_ARG REQUIRED)
endif()
find_package(PostgreSQL ${QUIET_ARG} ${REQUIRED_ARG})
```

Далее можно заняться вопросом о том, где именно находится библиотека PQXX в операционной системе.

Поиск библиотечных файлов

Для поиска библиотечных файлов CMake предлагает команду find_library(). Она принимает имена искомых файлов и список возможных путей в формате путей CMake:

```
find_library(<VAR_NAME> NAMES <NAMES> PATHS <PATHS> <...>)
```

<VAR_NAME> – имя переменной, в которой хранится результат команды. Если искомый файл найден, то в <VAR_NAME> будет помещен путь к нему. В противном случае переменная <VAR_NAME>-NOTFOUND будет установлена в 1. Мы будем использовать в качестве VAR_NAME переменную PQXX_LIBRARY_PATH, так что в итоге либо в PQXX_LIBRARY_PATH окажется путь, либо PQXX_LIBRARY_PATH-NOTFOUND будет равна 1.

Библиотека PQXX часто экспортирует свое местоположение в переменной окружения $ENV{PQXX_DIR}, т. е. система может уже знать, где она находится. Мы можем включить этот путь в список путей поиска, предварительно отформатировав его командой file(TO_CMAKE_PATH):

ch09/03-find-package-custom/cmake/module/FindPQXX.cmake (продолжение)

```
...

# Поиск библиотечных файлов
file(TO_CMAKE_PATH "$ENV{PQXX_DIR}" _PQXX_DIR)
find_library(PQXX_LIBRARY_PATH NAMES libpqxx pqxx
  PATHS
    ${_PQXX_DIR}/lib/${CMAKE_LIBRARY_ARCHITECTURE}
    # (...) еще много путей - для краткости опущены
    /usr/lib
  NO_DEFAULT_PATH
)
```

Ключевое слово NO_DEFAULT_PATH говорит CMake, что стандартный список путей поиска следует проигнорировать. Обычно мы не хотим этого делать (т. к. пути по умолчанию чаще всего правильны), но NO_DEFAULT_PATH позволяет явно задать свой список путей, если это необходимо.

Перейдем к поиску необходимых заголовочных файлов, которые могут быть включены пользователями библиотеки.

Поиск заголовочных файлов

Для поиска всех известных заголовочных файлов мы воспользуемся командой find_path(), которая работает очень похоже на find_library(). Главное различие заключается в том, что find_library() автоматически дописывает системно-зависимые расширения библиотек, тогда как для find_path() мы должны задавать точные имена.

Кроме того, пусть вас не смущает строка pqxx/pqxx. Это действительно заголовочный файл, но его расширение сознательно было опущено авторами библиотеки в подражание директивам #include в C++. Это позволяет использовать его в угловых скобках: #include <pqxx/pqxx>.

Ниже приведен фрагмент кода.

ch09/03-find-package-custom/cmake/module/FindPQXX.cmake (продолжение)

```
...

# Поиск библиотечных файлов
find_path(PQXX_HEADER_PATH NAMES pqxx/pqxx
  PATHS
    ${_PQXX_DIR}/include
```

```
    # (...) еще много путей - для краткости опущены
    /usr/include
  NO_DEFAULT_PATH
)
```

Далее посмотрим, как завершить процесс поиска, обработать отсутствие путей и вызвать функцию, определяющую импортированные цели.

Возврат результатов

Настало время проверить, были ли установлены переменные `PQXX_LIBRARY_PATH-NOTFOUND` и `PQXX_HEADER_PATHNOTFOUND`. Мы можем либо вручную напечатать диагностические сообщения и остановить сборку, либо использовать вспомогательную функцию `find_package_handle_standard_args()` из CMake. Эта функция устанавливает переменную `<PKG_NAME>_FOUND` в 1, если переменные пути правильно заполнены. Она также печатает подходящие диагностические сообщения (с учетом ключевого слова `QUIET`) и останавливает выполнение с ошибкой `FATAL_ERROR`, если при вызове `find_package()` было указано ключевое слово `REQUIRED`.

Если библиотека найдена, то мы вызываем написанную ранее функцию для определения импортированных целей и сохранения путей в кеше.

ch09/03-find-package-custom/cmake/module/FindPQXX.cmake (продолжение)

```
...

# Возврат результатов
include(FindPackageHandleStandardArgs)
find_package_handle_standard_args(
  PQXX DEFAULT_MSG PQXX_LIBRARY_PATH PQXX_HEADER_PATH
)
if (PQXX_FOUND)
  define_imported_target(
    "${PQXX_LIBRARY_PATH};${POSTGRES_LIBRARIES}"
    "${PQXX_HEADER_PATH};${POSTGRES_INCLUDE_DIRECTORIES}"
  )
elseif(PQXX_FIND_REQUIRED)
  message(FATAL_ERROR "Required PQXX library not found")
endif()
```

Вот и все! Этот модуль поиска найдет `PQXX` и создаст соответствующие цели `PQXX::PQXX`. Полный код имеется в репозитории примеров в книге.

Для библиотек, которые хорошо поддерживаются и с большой вероятностью уже установлены, этот метод очень эффективен. Но что, если вы работаете с более старыми и не столь хорошо поддерживаемыми пакетами? В Unix-подобных системах имеется программа `pkg-config`, а в CMake – полезный обертывающий модуль для ее поддержки.

Нахождение унаследованных пакетов с помощью FindPkgConfig

Управление зависимостями и определение необходимых флагов компиляции – задача такая же старая, как сами библиотеки C++. Для ее решения были разработаны различные инструменты – от простых поделок до комплектных решений, интегрированных в сборочные системы и IDE. PkgConfig (`freedesktop.org/wiki/Software/pkg-config`) – один из таких инструментов, когда-то очень популярный и часто присутствующий в Unix-подобных системах, хотя имеются также версии для macOS и Windows.

Однако PkgConfig постепенно вытесняется более современными решениями. Так следует ли продолжать его поддержку? Пожалуй, нет – и вот почему.

- Если ваша библиотека не предоставляет `pc`-файлов PkgConfig, то не имеет смысла писать определения для уходящего со сцены инструмента; лучше присмотреться к более современным альтернативам.
- Если вы можете найти более новую версию библиотеки, которая поддерживает CMake (как скачивать зависимости из интернета, мы обсудим ниже в этой главе).
- Если пакет широко используется, то в последнюю версию CMake уже может быть включен модуль поиска для него.
- Если в сети доступен модуль поиска, созданный сообществом, и лицензия позволяет вам использовать его, то это еще один неплохой вариант.
- Если вы можете написать и сопровождать модуль поиска самостоятельно.

Используйте PkgConfig, только если имеете дело с версией библиотеки, которая уже предоставляет `pc`-файл, а ни конфигурационного файла, ни модуля поиска не существует. Кроме того, должны быть веские причины, по которым вы не можете написать модуль поиска сами. Если вы уверены, что PkgConfig не понадобится, то можете смело пропустить этот раздел.

Как ни печально, не в любом окружении можно быстро перейти на последнюю версию библиотеки. Многие компании все еще работают с унаследованными системами, которые перестали обновляться. Если у вас есть `pc`-файл для какой-то конкретной библиотеки (назовем ее `foobar`), то выглядеть он будет примерно так:

```
prefix=/usr/local
exec_prefix=${prefix}
includedir=${prefix}/include
libdir=${exec_prefix}/lib
Name: foobar
Description: A foobar library
Version: 1.0.0
Cflags: -I${includedir}/foobar
Libs: -L${libdir} -lfoobar
```

Формат файла для PkgConfig прост, и многие разработчики, знакомые с этим инструментом, по привычке предпочитают использовать его, а не изучать более передовые системы типа CMake. Но, несмотря на свою простоту, PkgConfig умеет проверять доступность библиотеки и ее конкретной версии, а также может получить флаги компоновщика и информацию о каталогах для этой библиотеки.

Чтобы использовать PkgConfig в сочетании с CMake, вы должны найти программу `pkg-config` в своей системе, выполнить определенные команды, а затем вернуть результаты для последующего использования компилятором. Выполнять все эти шаги при каждом использовании PkgConfig утомительно. По счастью, CMake предлагает модуль поиска FindPkgConfig. Если PkgConfig найден, то будет установлена переменная `PKG_CONFIG_FOUND`. После этого можно воспользоваться функцией `pkg_check_modules()` для поиска нужного пакета.

В предыдущем разделе мы уже познакомились с библиотекой `libpqxx`, а т. к. она предоставляет `pc`-файл, попробуем найти ее с помощью PkgConfig. Для этого напишем простой файл `main.cpp`, в котором используется фиктивный класс подключения.

ch09/04-find-pkg-config/main.cpp

```
#include <pqxx/pqxx>
int main()
{
  // Мы ничего не подключаем, а просто удостоверяем,
  // что pqxx доступна.
  pqxx::nullconnection connection;
}
```

В типичном списковом файле мы обычно начинаем с функции `find_package()` и переключаемся на PkgConfig, если библиотека не найдена. Этот подход полезен, когда окружение обновляется, поскольку мы можем продолжать пользоваться методом `main` без изменения кода. Для краткости мы опустим эту часть примера.

ch09/04-find-pkg-config/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(FindPkgConfig CXX)
find_package(PkgConfig REQUIRED)
pkg_check_modules(PQXX REQUIRED IMPORTED_TARGET libpqxx)
message("PQXX_FOUND: ${PQXX_FOUND}")
add_executable(main main.cpp)
target_link_libraries(main PRIVATE PkgConfig::PQXX)
```

Разберем этот код по шагам.

1. Команда `find_package()` используется для поиска PkgConfig. Если `pkg-config` отсутствует, то процесс останавливается из-за ключевого слова `REQUIRED`.

2. Макрос pkg_check_modules() из модуля поиска FindPkgConfig создает новую импортированную цель PQXX. Модуль поиска ищет зависимость libpqxx и завершается с ошибкой, если она не найдена, – тоже из-за наличия ключевого слова REQUIRED. Ключевое слово IMPORTED_TARGET важно, в противном случае пришлось бы определять цель вручную.
3. Мы сообщаем о результате поиска с помощью функции message(), которая выводит переменную PQXX_FOUND. Если бы мы не пользовались ключевым словом REQUIRED, то здесь могли бы проверить, установлена ли эта переменная, и, быть может, попробовать другие способы.
4. Объявляем исполняемый файл main с помощью команды add_executable().
5. Наконец, используем target_link_libraries(), чтобы скомпоновать цель PkgConfig::PQXX, импортированную командой pkg_check_modules(). Заметим, что PkgConfig:: – фиксированный префикс, а PQXX выведена из первого аргумента, переданного макросу.

Такой путь быстрее, чем создание модуля поиска для зависимостей, не поддерживаемых CMake. Однако у него есть недостатки. Один из них связан с тем, что мы полагаемся на устаревший инструмент pkg-config, которого может и не оказаться в операционной системе, где собирается проект. Кроме того, при таком подходе появляется специальный случай, который нужно поддерживать иначе, чем другие методы.

Мы обсудили, как работать с зависимостями, уже установленными на компьютере. Но это только часть истории. Часто бывает, что ваш проект попадает к пользователям, у которых не установлены все необходимые зависимости. Посмотрим, как быть в такой ситуации.

Использование зависимостей, отсутствующих в системе

CMake прекрасно справляется с управлением зависимостями, особенно если они еще не установлены в системе. Тут есть несколько подходов. Если вы работаете с версией CMake 3.14 или более поздней, то лучше всего воспользоваться модулем FetchContent. По существу, FetchContent является дружелюбной оберткой вокруг другого модуля, ExternalProject. Он не только упрощает процесс, но и добавляет ряд возможностей. Более глубоко мы будем изучать ExternalProject ниже в этой главе. А пока достаточно знать, что основное различие между этими двумя модулями заключается в порядке выполнения:

- FetchContent подтягивает зависимости на *этапе конфигурирования*;
- ExternalProject подтягивает зависимости на *этапе сборки*.

Этот порядок существен, потому что цели, определенные FetchContent на этапе конфигурирования, окажутся в том же самом пространстве имен, и по-

тому их легко использовать в нашем проекте. Мы можем компоновать их с другими целями, как если бы определили их самостоятельно. Редко, но бывает, что это нежелательно, и вот тогда приходится прибегать к `ExternalProject`.

Но сначала посмотрим, что делать в большинстве случаев.

FetchContent

Модуль `FetchContent` исключительно полезен; он предлагает следующие возможности:

- управление структурой каталога для внешнего проекта;
- скачивание исходного кода из интернета (и при необходимости распаковка архива);
- поддержка репозиториев Git, Subversion, Mercurial и CVS (Concurrent Versions System);
- скачивание обновлений при необходимости;
- конфигурирование и сборка проекта с помощью CMake, Make или иного инструмента, указанного пользователем;
- обслуживание вложенных зависимостей.

Работа с модулем `FetchContent` состоит из трех основных шагов.

1. Добавить в проект модуль командой `include(FetchContent)`.
2. Сконфигурировать зависимости командой `FetchContent_Declare()`. Она сообщает модулю `FetchContent`, где находятся зависимости и какую версию следует использовать.
3. Завершить настройку зависимостей командой `FetchContent_MakeAvailable()`. Она скачивает, собирает, устанавливает зависимости и добавляет списковые файлы в главный проект.

Возможно, вы удивляетесь, почему шаги 2 и 3 разделены. Чтобы разрешить **переопределение конфигурации** в многоуровневых проектах. Например, рассмотрим проект, который зависит от внешних библиотек, A и B. Библиотека A также зависит от B, но ее авторы использовали устаревшую версию, которая отличается от версии в родительском проекте (рис. 9.1).

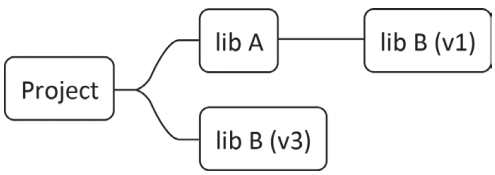


Рис. 9.1. Иерархически организованный проект

Если бы конфигурирование и скачивание производились одной командой, то родительский проект не мог бы использовать более позднюю версию, даже обратно совместимую, потому что зависимость уже сконфигурировала бы

импортированные цели для более ранней версии, что чревато конфликтами имен целей и файлов библиотеки.

Чтобы указать, какая версия необходима, проект верхнего уровня должен вызвать команду `FetchContent_Declare()` и предоставить переопределенную конфигурацию для B еще до того, как библиотека A полностью настроена. Последующее обращение к `FetchContent_Declare()` в A будет проигнорировано, потому что зависимость B уже сконфигурирована.

Рассмотрим сигнатуру команды `FetchContent_Declare()`:

```
FetchContent_Declare(<depName> <contentOptions>...)
```

Здесь `depName` – уникальный идентификатор зависимости, который впоследствии будет использован командой `FetchContent_MakeAvailable()`.

Параметр `contentOptions` описывает детальную конфигурацию зависимости, которая может быть весьма сложной. Важно понимать, что под капотом `FetchContent_Declare()` пользуется более старой командой `ExternalProject_Add()`. На самом деле многие аргументы, переданные `FetchContent_Declare`, без изменения переправляются внутреннему вызову. Прежде чем пускаться в подробные объяснения аргументов, рассмотрим практический пример скачивания зависимости из GitHub.

Простой пример программы чтения YAML

Я написал элементарную программу, которая читает имя пользователя из YAML-файла и печатает его в приветственном сообщении. YAML – простой формат для хранения конфигурации в понятном человеку виде, но он довольно сложен для машинного разбора. Я нашел небольшой проект, решающий эту проблему: `yaml-cpp` Джесси Бедера (https://github.com/jbeder/yaml-cpp).

Пример непритязательный; это программа, которая печатает сообщение `Welcome <имя>`. По умолчанию `name` равно `Guest`, но в конфигурационном YAML-файле можно задать другое имя. Ниже приведен код на C++.

ch09/05-fetch-content/main.cpp

```
#include <string>
#include <iostream>
#include "yaml-cpp/yaml.h"

using namespace std;
int main() {
  string name = "Guest";

  YAML::Node config = YAML::LoadFile("config.yaml");
  if (config["name"])
    name = config["name"].as<string>();

  cout << "Welcome " << name << endl;
  return 0;
}
```

Конфигурационный файл состоит всего из одной строки.

ch09/05-fetch-content/config.yaml

```
name: Rafal
```

Мы еще воспользуемся этим примером в других разделах, поэтому потратьте секундочку, чтобы понять, как он работает. Имея код, посмотрим, как его собрать и получить зависимость.

ch09/05-fetch-content/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(ExternalProjectGit CXX)
add_executable(welcome main.cpp)
configure_file(config.yaml config.yaml COPYONLY)
include(FetchContent)
FetchContent_Declare(external-yaml-cpp
  GIT_REPOSITORY     https://github.com/jbeder/yaml-cpp.git
  GIT_TAG            0.8.0
)
FetchContent_MakeAvailable(external-yaml-cpp)
target_link_libraries(welcome PRIVATE yaml-cpp::yaml-cpp)
```

Мы можем явно получить доступ к целям, созданным библиотекой `yaml-cpp`. Чтобы доказать это, воспользуемся вспомогательным модулем `CMakePrintHelpers`:

```
include(CMakePrintHelpers)
cmake_print_properties(TARGETS yaml-cpp::yaml-cpp
                       PROPERTIES TYPE SOURCE_DIR)
```

При сборке такого проекта на этапе конфигурирования будет напечатано:

```
Properties for TARGET yaml-cpp::yaml-cpp:
  yaml-cpp.TYPE = "STATIC_LIBRARY"
  yaml-cpp.SOURCE_DIR = "/tmp/b/_deps/external-yaml-cpp-src"
```

Это сообщение говорит, что цель, определенная зависимостью `external-yaml-cpp`, существует; это статическая библиотека, и каталог ее исходного кода расположен внутри дерева сборки. В реальных проектах такая распечатка необязательна, но она помогает при отладке, если вы не уверены, как правильно включить импортированную цель.

Поскольку мы уже скопировали `yaml`-файл в вывод командой `configure_file()`, то можем запустить программу:

```
~/examples/ch09/05-fetch-content$ /tmp/b/welcome
Welcome Rafal
```

Все работает, как по волшебству! Не приложив почти никаких усилий, мы включили внешнюю зависимость и использовали ее в своем проекте.

Если нам нужно более одной зависимости, то следует вызвать команду `FetchContent_Declare()` несколько раз, всякий раз задавая уникальный идентификатор. Но вызывать `FetchContent_MakeAvailable()` более одного раза нет нужды, потому что она поддерживает несколько идентификаторов (нечувствительных к регистру):

```
FetchContent_MakeAvailable(lib-A lib-B lib-C)
```

Теперь научимся писать объявления зависимостей.

Скачивание зависимостей

Команда `FetchContent_Declare()` предлагает широкий диапазон возможностей, обеспечиваемых модулем `ExternalProject`. Основных действий три:

- скачивание зависимостей;
- обновление зависимостей;
- исправление зависимостей.

Начнем с самого распространенного сценария: скачивания файлов из интернета. CMake поддерживает много источников скачивания:

- HTTP-сервер (URL-адрес);
- Git;
- Subversion;
- Mercurial;
- CVS.

Сначала посмотрим, как скачивать зависимости по URL-адресу и настраивать этот процесс под свои потребности.

Скачивание по URL-адресу

Мы можем указать список URL-адресов, которые будут просматриваться последовательно, пока скачивание не увенчается успехом. CMake понимает, что скачанный файл является архивом, и по умолчанию распаковывает его.

Основное объявление:

```
FetchContent_Declare(dependency-id
                     URL <url1> [<url2>...]
)
```

У этого метода есть несколько параметров:

- `URL_HASH <algo>=<hashValue>`: проверяет, что контрольная сумма скачанного файла, сгенерированная алгоритмом `<algo>`, совпадает с `<hashValue>`. Это рекомендуется делать для гарантии целостности скачанных файлов. Поддерживаются следующие алгоритмы: `MD5`, `SHA1`, `SHA224`, `SHA256`, `SHA384`, `SHA512`, `SHA3_224`, `SHA3_256`, `SHA3_384` и `SHA3_512`;

- DOWNLOAD_NO_EXTRACT <bool>: явно запрещает распаковку после скачивания. Узнать имя скачанного файла можно на последующих шагах, обратившись к переменной <DOWNLOADED_FILE>;
- DOWNLOAD_NO_PROGRESS <bool>: явно запрещает протоколирование хода скачивания;
- TIMEOUT <seconds> и INACTIVITY_TIMEOUT <seconds>: задаются тайм-ауты, по истечении которых скачивание прекращается. Первый параметр задает полное время работы, второй – период неактивности;
- HTTP_USERNAME <username> и HTTP_PASSWORD <password>: конфигурируют HTTP-аутентификацию. Не зашивайте учетные данные в код;
- HTTP_HEADER <header1> [<header2>...]: включает дополнительные заголовки в HTTP-запрос, что полезно, если нужно задать токены AWS или еще какие-то;
- TLS_VERIFY <bool>: проверяет сертификат SSL. Если параметр не задан, то CMake читает информацию из переменной CMAKE_TLS_VERIFY, которая по умолчанию равна false. Пропуск верификации TLS считается небезопасной и дурной практикой, этого следует избегать, особенно в производственном окружении;
- TLS_CAINFO <file>: задает путь к файлу авторизации; если параметр не задан, CMake читает информацию из переменной CMAKE_TLS_CAINFO. Это полезно, если ваша компания выпускает самоподписанные сертификаты SSL.

Большинство программистов обращаются за последними версиями библиотек к онлайновым репозиториям типа GitHub. Вот как это делается.

Скачивание из Git

Для скачивания зависимостей из Git убедитесь, что в хост-системе установлена версия Git 1.6.5 или более поздняя. Для клонирования проекта из Git обязательны следующие параметры:

```
FetchContent_Declare(dependency-id
                     GIT_REPOSITORY <url>
                     GIT_TAG <tag>
)
```

Параметры <url> и <tag> должны быть совместимы с командой git. В производственной среде рекомендуется использовать хеши git (а не теги), чтобы обеспечить прослеживаемость сгенерированных двоичных файлов и избежать лишних операций git fetch. Если вы предпочитаете работать с ветвями, то пользуйтесь удаленными именами, например origin/main. Так вы гарантируете надлежащую синхронизацию локального клона.

Ниже перечислены дополнительные параметры.

- GIT_REMOTE_NAME <name>: задает удаленное имя (по умолчанию origin);
- GIT_SUBMODULES <module>...: указывает, какие подмодули обновлять; начиная с версии 3.16 это значение по умолчанию равно none (раньше обновлялись все подмодули);

- `GIT_SUBMODULES_RECURSE 1`: разрешает рекурсивное обновление подмодулей;
- `GIT_SHALLOW 1`: выполняется поверхностное клонирование, более быстрое, потому что исторические фиксации не скачиваются;
- `TLS_VERIFY <bool>`: проверяется сертификат SSL. Если параметр не задан, то CMake читает информацию из переменной `CMAKE_TLS_VERIFY`, которая по умолчанию равна `false`. Пропуск верификации TLS считается небезопасной и дурной практикой, этого следует избегать, особенно в производственном окружении.

Зависимости, хранящиеся в Subversion, тоже можно скачивать с помощью CMake.

Скачивание из Subversion

Для скачивания из Subversion необходимо задать следующие параметры:

```
FetchContent_Declare(dependency-id
                     SVN_REPOSITORY <url>
                     SVN_REVISION -r<rev>
)
```

Дополнительно можно задать:

- `SVN_USERNAME <user>` и `SVN_PASSWORD <password>`: это учетные данные для извлечения файлов и обновления. Не зашивайте их в код проектов;
- `SVN_TRUST_CERT <bool>`: пропустить проверку сертификата сервера Subversion. Пользуйтесь этой возможностью, только если сетевой путь к северу и его целостность достойны доверия.

Использовать Subversion в CMake очень просто. Как и Mercurial.

Скачивание из Mercurial

Тут нет ничего сложного. Надо лишь задать два аргумента:

```
FetchContent_Declare(dependency-id
                     HG_REPOSITORY <url>
                     HG_TAG <tag>
)
```

Скачивание из CVS

Для извлечения файлов из CVS нужно задать три аргумента:

```
FetchContent_Declare(dependency-id
                     CVS_REPOSITORY <cvsroot>
                     CVS_MODULE <module>
                     CVS_TAG <tag>
)
```

Мы рассмотрим все варианты скачивания командой `FetchContent_Declare()`. CMake поддерживает дополнительные шаги, которые можно выполнить после успешного скачивания.

Обновление и исправление

По умолчанию шаг `update` повторно скачивает файлы внешнего проекта, если метод скачивания поддерживает обновления, например если мы сконфигурировали зависимость из Git, так что она указывает на ветвь `main` или `master`. Это поведение можно переопределить двумя способами:

- написать специальную команду, которая будет выполняться во время обновления командой `UPDATE_COMMAND <cmd>`;
- полностью запретить шаг обновления (чтобы разрешить сборку без подключения к сети) – `UPDATE_DISCONNECTED <bool>`. Отметим, что зависимость все равно будет скачана во время первой сборки.

С другой стороны, `patch` – факультативный шаг, который будет выполнен после скачивания обновления. Чтобы его разрешить, необходимо точно указать, какую команду мы хотим выполнить, с помощью команды `PATCH_COMMAND <cmd>`.

Документация по CMake предупреждает, что некоторые исправления могут быть «липкими». Например, в Git измененные файлы не восстанавливаются в исходное состояние в процессе обновления, и нужно внимательно следить за тем, чтобы не наложить исправления дважды. В идеале команда `patch` должна была бы быть надежной и идемпотентной.

Команды `update` и `patch` можно сцеплять:

```
FetchContent_Declare(dependency-id
                     GIT_REPOSITORY <url>
                     GIT_TAG <tag>
                     UPDATE_COMMAND <cmd>
                     PATCH_COMMAND <cmd>
)
```

Скачивать зависимости полезно, когда их еще нет в системе. А если есть? Как тогда использовать локальную версию?

Использование установленной зависимости, если это возможно

Начиная с версии CMake 3.24 в `FetchContent` появилась возможность пропускать скачивание, если зависимость уже доступна локально. Нужно лишь добавить ключевое слово `FIND_PACKAGE_ARGS` в объявление:

```
FetchContent_Declare(dependency-id
                     GIT_REPOSITORY <url>
```

```
                    GIT_TAG <tag>
                    FIND_PACKAGE_ARGS <args>
)
```

Как нетрудно догадаться, это ключевое слово говорит модулю `FetchContent` о том, что нужно вызвать функцию `find_package()` перед началом скачивания. Если пакет найден локально, то он будет использован, и ни скачивания, ни сборки не произойдет. Отметим, что это ключевое слово должно быть последним в команде, потому что оно потребляет все последующие аргументы.

Вот какие изменения нужно внести в предыдущий пример.

ch09/06-fetch-content-find-package/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(ExternalProjectGit CXX)

add_executable(welcome main.cpp)
configure_file(config.yaml config.yaml COPYONLY)

include(FetchContent)
FetchContent_Declare(external-yaml-cpp
  GIT_REPOSITORY https://github.com/jbeder/yaml-cpp.git
  GIT_TAG 0.8.0
  FIND_PACKAGE_ARGS NAMES yaml-cpp
)
FetchContent_MakeAvailable(external-yaml-cpp)
target_link_libraries(welcome PRIVATE yaml-cpp::yaml-cpp)
include(CMakePrintHelpers)
cmake_print_properties(TARGETS yaml-cpp::yaml-cpp
                       PROPERTIES TYPE SOURCE_DIR
                       INTERFACE_INCLUDE_DIRECTORIES
)
```

Ключевых изменений два:

1. Мы добавили аргумент `FIND_PACKAGE_ARGS` с ключевым словом `NAMES`, чтобы указать, что ищем пакет `yaml-cpp`. Без `NAMES` CMake по умолчанию использовала бы `dependency-id`, т. е. в данном случае `external-yaml-cpp`.
2. Мы добавили `INTERFACE_INCLUDE_DIRECTORIES` в состав печатаемых свойств. Это одноразовая проверка, так чтобы мы могли увидеть, используется ли уже установленный пакет или скачивается новый.

Перед тестированием убедитесь, что пакет действительно установлен в системе. Если нет, установите его следующими командами:

```
git clone https://github.com/jbeder/yaml-cpp.git
cmake -S yaml-cpp -B build-dir
cmake --build build-dir
cmake --install build-dir
```

Теперь мы можем собрать проект. Если все пройдет хорошо, то мы увидим отладочный вывод команды `cmake_print_properties()`. Это покажет, что мы используем локальную версию, о чем свидетельствует свойство `INTERFACE_INCLUDE_DIRECTORIES`. Имейте в виду, что вывод зависит от окружения, т. е. вы можете увидеть другой текст:

```
--
Properties for TARGET yaml-cpp::yaml-cpp:
  yaml-cpp::yaml-cpp.TYPE = "STATIC_LIBRARY"
  yaml-cpp::yaml-cpp.INTERFACE_INCLUDE_DIRECTORIES = "/usr/local/include"
```

Если вы работаете не с CMake 3.24 или хотите поддержать пользователей, работающих с более старыми версиями, то можете рассмотреть возможность запуска команды `find_package()` вручную. Тогда вы будете скачивать только пакеты, которые еще не установлены:

```
find_package(yaml-cpp QUIET)
if (NOT TARGET yaml-cpp::yaml-cpp)
  # скачать отсутствующую зависимость
endif()
```

Какой бы метод вы ни выбрали, разумно будет сначала попытаться использовать локальную версию и, только если зависимость не найдена, переходить к скачиванию.

До появления `FetchContent` в CMake присутствовал более простой модуль, `ExternalProject`. Хотя в большинстве случаев рекомендуется использовать `FetchContent`, `ExternalProject` по-прежнему имеет свои преимущества и иногда может быть полезен.

ExternalProject

Модуль `ExternalProject` был добавлен в версии 3.0.0 и использовался до появления `FetchContent`. Вы, конечно, догадались, что он использовался для скачивания внешних проектов из репозиториев. С годами функциональность модуля постепенно расширялась, и в результате получилась весьма развитая команда `ExternalProject_Add()`.

Модуль `ExternalProject` находит зависимости на этапе сборки. Этим он кардинально отличается от `FetchContent`, который работает на этапе конфигурирования. Из-за этого различия `ExternalProject` не может импортировать в проект цели, как делает `FetchContent`. С другой стороны, `ExternalProject` умеет устанавливать зависимости непосредственно в систему, прогонять их тесты и делать другие интересные вещи, например переопределять команды, используемые для конфигурирования и сборки.

Существует не так уж много случаев, когда это может оказаться необходимо. Поскольку эффективное использование этого унаследованного модуля сопряжено со значительными издержками, относитесь к нему как к курьезу.

Мы упоминаем о нем здесь, только чтобы показать, как его эволюция привела к современному методу.

`ExternalProject` предлагает команду `ExternalProject_Add`, которая конфигурирует зависимость. Приведем пример:

```
include(ExternalProject)
ExternalProject_Add(external-yaml-cpp
  GIT_REPOSITORY    https://github.com/jbeder/yaml-cpp.git
  GIT_TAG           0.8.0
  INSTALL_COMMAND   ""
  TEST_COMMAND      ""
)
```

Как уже отмечалось, это сильно напоминает команду `FetchContent_Declare` из `FetchContent`. Обратите внимание на два дополнительных ключевых слова: `INSTALL_COMMAND` и `TEST_COMMAND`. В этом случае они используются для подавления установки и прогона тестов зависимости, поскольку эти действия обычно выполняются во время сборки. Модуль `ExternalProject` производит много шагов, которые конфигурируются в широких пределах и выполняются в следующем порядке:

1) `mkdir`: создать каталог для внешнего проекта;
2) `download`: скачать файлы проекта из репозитория или по URL;
3) `update`: скачать обновления, если это поддерживается методом `fetch`;
4) `patch`: выполнить команду `patch`, которая изменяет скачанные файлы;
5) `configure`: выполнить этап конфигурирования;
6) `build`: выполнить этап сборки для проектов CMake;
7) `install`: установить проекты CMake;
8) `test`: прогнать тесты.

Для каждого шага, кроме `mkdir`, поведение по умолчанию можно переопределить, добавив команду `<STEP>_COMMAND`. Существует много других возможностей, полный перечень см. в документации. Если по какой-то причине вы хотите использовать этот метод вместо рекомендуемого `FetchContent`, то существует грязный трюк, который можно применить для принудительного импорта целей, – вызов CMake из CMake. Детали см. в примере `ch09/05-external-project` в репозитории этой книги.

Обычно мы полагаемся на присутствие библиотеки в системе. Если это не так, то мы прибегаем к модулю `FetchContent`. Этот подход особенно удобен для небольших зависимостей, которые можно быстро откомпилировать.

Но для таких громоздких библиотек, как Qt, этот метод занял бы слишком много времени. В таких случаях рекомендуется использовать менеджеры пакетов, предлагающие заранее откомпилированные библиотеки, отвечающие окружению пользователя. Такие инструменты, как Apt или Conan, предлагают решение, но они либо слишком системно-зависимы, либо сложны для рассмотрения в этой книге. Впрочем, большинство пользователей способны установить необходимые вашему проекту зависимости, если снабдить их ясными инструкциями.

Резюме

В этой главе вы узнали о том, как находить установленные в системе пакеты с помощью модулей поиска CMake и как использовать конфигурационные файлы, поставляемые вместе с библиотеками. Для более старых библиотек, которые не поддерживают CMake, но включают `pc`-файлы, можно использовать инструмент PkgConfig и входящий в состав CMake модуль поиска `FindPkgConfig`.

Мы также изучили возможности модуля `FetchContent`. Он позволяет скачивать зависимости из различных источников, но при этом конфигурировать CMake так, чтобы она сначала производила локальный поиск в системе, и тем самым избегать излишнего скачивания. Мы коснулись исторического контекста этих модулей и обсудили возможность использования модуля `ExternalProject` в особых случаях.

CMake проектировалась с расчетом на автоматическое генерирование целей сборки, когда библиотека ищется большинством из рассмотренных выше методов. Это делает процесс удобным и элегантным. Обладая этими знаниями, вы можете включать в свои проекты стандартные библиотеки.

В следующей главе мы научимся предоставлять не столь масштабный повторно используемый код с помощью модулей C++20.

Для дополнительного чтения

Дополнительные сведения по рассмотренным в этой главе темам можно получить из следующих источников:

- документация по CMake – готовые модули поиска: https://cmake.org/cmake/help/latest/manual/cmake-modules.7.html#findmodules;
- документация по CMake – руководство по использованию зависимостей: https://cmake.org/cmake/help/latest/guide/using-dependencies/index.html;
- *CMake and using git-submodule for dependence projects*: https://stackoverflow.com/questions/43761594/;
- использование возможностей PkgConfig: https://gitlab.kitware.com/cmake/community/-wikis/doc/tutorials/How-To-Find-Libraries#piggybacking-on-pkg-config;
- как использовать модуль ExternalProject: https://www.jwlawson.co.uk/interest/2020/02/23/cmake-external-project.html;
- *CMake FetchContent vs. ExternalProject*: https://www.scivision.dev/cmake-fetchcontent-vs-external-project/;
- *Using CMake with External Projects*: http://www.saoe.net/blog/using-cmake-with-external-projects/.

10

Использование модулей C++20

В C++20 в язык было добавлено новое средство: модули. Они заменяют текстовые объявления символов в заголовочных файлах файлом модуля, который предкомпилируется в промежуточный двоичный формат, что резко сокращает время сборки.

Мы обсудим наиболее важные вопросы, относящиеся к использованию модулей C++20 в CMake, и начнем с общего введения в концепцию модулей: в чем их преимущество перед стандартными заголовочными файлами и как они упрощают управление единицами трансляции в исходном коде. Обещание упростить и ускорить сборку звучит заманчиво, но в этой главе мы покажем, каким долгим и тернистым был путь к принятию этой идеи.

Разобравшись в теории, мы перейдем к практическим аспектам реализации модулей в проектах: мы обсудим их экспериментальную поддержку в ранних версиях CMake и окончательную полную версию в CMake 3.28.

Наше знакомство с модулями C++20 не ограничивается констатацией новой возможности – нам предстоит переосмыслить взаимодействие компонентов в крупных проектах на C++. К концу главы вы не только будете знать о теоретических аспектах модулей, но и получите практические навыки путем изучения примеров, что позволит с пользой применять это средство в своих проектах.

В этой главе будут рассмотрены следующие вопросы:

- что такое модули C++20;
- написание проектов с поддержкой модулей C++20;
- конфигурирование комплекта инструментов.



Технические требования в этой главе отличаются от предыдущих. Внимательно прочитайте следующий раздел.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch10.

Для проработки примеров в этой главе необходимые следующие утилиты:

- версия CMake 3.26 или более поздняя (рекомендуется 3.28);
- любой из поддерживаемых генераторов:
- Ninja 1.11 или более поздняя (Ninja и Ninja Multi-Config);
- Visual Studio 17 2022 или более поздняя;
- любой из поддерживаемых компиляторов:
- инструментарий MSVC версии 14.34 или более поздней;
- Clang 16 или более поздняя;
- GCC 14 (для ветви, находящейся в процессе разработки, после 9.20.2023) или более поздняя.

Если вы знакомы с Docker, то можете использовать образ с полным набором инструментов, описанный в разделе «Установка CMake на различные платформы» главы 1.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree> -G "Ninja" -D CMAKE_CXX_
COMPILER=clang++-18 && cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути.

Что такое модули C++20?

Я хотел написать об использовании модулей C++ больше трех лет назад. Несмотря на то что модули были приняты как часть спецификации C++20, экосистема C++ была совершенно не готова к их поддержке. К счастью, с момента выхода первого издания этой книги много изменилось, и в версии CMake 3.28 модули C++20 поддерживаются официально (а экспериментальная поддержка появилась еще в версии 3.26).

Может показаться, что три года – долгий срок для реализации одного средства, но не будем забывать, что дело не только в CMake. Многие кусочки пазла должны соединиться, чтобы все заработало как надо. Сначала должны появиться компиляторы, понимающие, как работать с модулями, и лишь после этого CMake сможет использовать новые механизмы для предоставления своей поддержки модулей.

Это означает, что еще не все располагают последним совместимым инструментарием, а даже если располагают, поддержка пока что находится на ранней стадии развития. Из-за этих ограничений модули еще непригодны

для широкой аудитории. Поэтому пока не стоит создавать проекты промышленного уровня, опирающиеся на этот механизм.

Тем не менее если вы – энтузиаст самых передовых решений, то вперед и с песней! Если вы способны строго контролировать среду сборки своего проекта, например на выделенных компьютерах или в специальных контейнерах (Docker и других), то можете эффективно использовать модули во внутренних проектах. Просто двигайтесь осторожно и не забывайте, что обстоятельства могут быть разными. Возможно, в какой-то момент вам придется вообще отказаться от модулей, потому что их поддержка отсутствует или некорректно реализована в одной из утилит.

Слово «модуль» сильно перегружено в контексте сборки проектов на C++. Выше мы обсуждали модули в контексте CMake: модули поиска, служебные модули и т. д. Поясним, что модули C++ не имеют ничего общего с модулями CMake. Это языковой механизм, добавленный в версии C++20.

В основе своей модуль C++ – это один исходный файл, инкапсулирующий функциональность заголовков и файлов реализации в единице кода. Он состоит из двух основных компонентов:

- **двоичный интерфейс модуля** (Binary Module Interface – **BMI**) служит той же цели, что заголовочный файл, но имеет двоичный формат, что значительно уменьшает потребность в перекомпиляции при потреблении другими единицами трансляции;
- **блок реализации модуля** (Module Implementation Unit) предоставляет реализацию, определения и внутренние детали модуля. Его содержимое недоступно напрямую извне модуля, что позволяет эффективно инкапсулировать детали реализации.

Модули были предложены, чтобы сократить время компиляции и решить некоторые проблемы, присущие препроцессору и традиционным заголовочным файлам. Посмотрим, как несколько единиц трансляции объединяются в типичном унаследованном проекте.

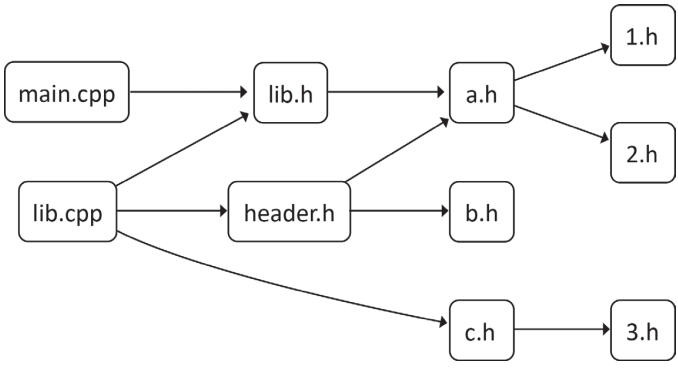


Рис. 10.1. Структура проекта с традиционными заголовками

На этом рисунке видно, что препроцессору приходится обходить дерево проекта для сборки программы. В главе 7 мы узнали, что для сборки каждой

единицы трансляции препроцессор механически сшивает файлы. В результате порождается длинный файл, содержащий все заголовки, включенные директивами препроцессора. Таким образом, `main.cpp` сначала включает свой собственный исходный код, а затем содержимое файлов `lib.h`, `a.h`, `1.h` и `2.h`. И только после этого в дело вступает компилятор и начинает разбирать каждый символ для порождения двоичного объектного файла. И все бы ничего, вот только при компиляции `lib.cpp` заголовки, уже включенные в `main.cpp`, приходится обрабатывать снова. И эта избыточность нарастает при добавлении каждой новой единицы трансляции.

С традиционными заголовками сопряжены и другие трудности:

- необходима защита от повторного включения, а если о ней забыть, то проблемы неминуемы;
- для разрешения циклических ссылок необходимы **опережающие объявления**;
- даже небольшое изменение заголовочного файла вызывает необходимость перекомпиляции всех единиц трансляции;
- макросы препроцессора трудно отлаживать и сопровождать.

Модули сходу решают многие из этих проблем, но некоторые остаются: модули, как и заголовочные файлы, могут зависеть друг от друга. Когда один модуль импортирует другой, их все равно нужно компилировать в правильном порядке, начиная с самого вложенного. Обычно это не составляет проблемы, поскольку модули гораздо больше по размеру. Во многих случаях всю библиотеку можно упаковать в один модуль.

Поговорим о том, как пишутся и используются модули. В следующем простом примере мы просто возвращаем сумму двух аргументов.

ch10/01-cxx-modules/math.cppm

```
export module math;
export int add(int a, int b) {
  return a + b;
}
```

Такой модуль не нуждается в объяснениях. Он начинается с предложения, сообщающего остальной программе, что это модуль с именем `math`. Затем идет обычное определение функции, помеченное ключевым словом `export`, которое говорит, что эта функция доступна вне модуля.



Обратите внимание, что у файлов модулей не такое расширение, как у обычных файлов с исходным кодом на C++. Это вопрос соглашения, он не влияет на способ обработки кода. Я рекомендую выбирать расширение в зависимости от используемого комплекта инструментов:

- `.ixx` – расширение, принятое в MSVC;
- `.cppm` – расширение, принятое в Clang;
- `.cxx` – расширение, принятое в GCC.

Для использования этого модуля мы должны импортировать его в программу:

ch10/01-cxx-modules/main.cpp

```
import math;

#include <iostream>

int main() {
  std::cout << "Addition 2 + 2 = " << add(2, 2) << std::endl;
  return 0;
}
```

Предложения `import math` достаточно, чтобы включить символы, экспортированные из модуля, прямо в программу `main`. Теперь мы можем использовать функцию `add()` в теле функции `main()`. На первый взгляд, модули очень похожи на заголовочные файлы. Но если бы мы попытались написать списковый файл CMake как обычно, то не смогли бы собрать проект. Настало время познакомиться с действиями, необходимыми для использования модулей C++.

Написание проектов с поддержкой модулей C++20

В этой книге обсуждается в основном версия CMake 3.26, но стоит отметить, что CMake часто обновляется, и версия 3.28 вышла прямо перед отправкой этой главы в печать. Если вы используете эту версию или более позднюю, то можете получить доступ к последним возможностям, задав в команде `cmake_minimum_required()` аргумент `VERSION 3.28.0`.

С другой стороны, если вы вынуждены работать с более старой версией или хотите охватить более широкую аудиторию, включающую пользователей, еще не перешедших на новую версию, то должны будете активировать экспериментальную поддержку, чтобы воспользоваться модулями C++20 в CMake.

Посмотрим, как это сделать.

Активация экспериментальной поддержки в CMake 3.26 и 3.27

Экспериментальная поддержка – своеобразное соглашение: разработчик понимает, что средство еще не вполне готово и должно использоваться только для тестирования. Чтобы *подписать* такое соглашение, вы должны присвоить

переменной `CMAKE_EXPERIMENTAL_CXX_MODULE_CMAKE_API` в списковом файле проекта значение, соответствующее используемой версии CMake.



Официальный репозиторий CMake, который поддерживает компания Kitware, содержит средство отслеживания проблем, в котором можно произвести поиск по метке `area:cxxmodules`. До выпуска 3.28 была зарегистрирована только одна проблема (в версии 3.25.0), что является показателем потенциально стабильного кода. Если вы решите активировать экспериментальные средства, соберите свой проект, чтобы убедиться, что он будет работать у пользователей.

Ниже перечислены значения, которые можно найти в репозитории и в документации по CMake:

- `3c375311-a3c9-4396-a187-3227ef642046` для 3.25 (не документировано);
- `2182bf5c-ef0d-489a-91da-49dbc3090d2a` для 3.26;
- `aa1f7df0-828a-4fcd-9afc-2dc80491aca7` для 3.27.

К сожалению, если у вас нет доступа хотя бы к версии CMake 3.25, то вам не повезло. До этой версии модули не поддерживались вовсе. Кроме того, в версиях, более ранних, чем 3.27, нужно установить еще одну переменную, чтобы разрешить динамические зависимости для модулей:

```
set(CMAKE_EXPERIMENTAL_CXX_MODULE_DYNDEP 1)
```

Ниже показано, как автоматически выбрать правильный ключ API для текущей версии и явно запретить сборку для версий, которые вы не поддерживаете (в этом примере мы поддерживаем только версии CMake от 3.26 и выше).

ch10/01-cxx-modules/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(CXXModules CXX)

# включить экспериментальный API
if(CMAKE_VERSION VERSION_GREATER_EQUAL 3.28.0)
  # Предполагается, что в исходных файлах C++ действительно импортируются модули
  cmake_policy(SET CMP0155 NEW)
elseif(CMAKE_VERSION VERSION_GREATER_EQUAL 3.27.0)
  set(CMAKE_EXPERIMENTAL_CXX_MODULE_CMAKE_API
      "aa1f7df0-828a-4fcd-9afc-2dc80491aca7")
elseif(CMAKE_VERSION VERSION_GREATER_EQUAL 3.26.0)
  set(CMAKE_EXPERIMENTAL_CXX_MODULE_CMAKE_API
      "2182bf5c-ef0d-489a-91da-49dbc3090d2a")
  set(CMAKE_EXPERIMENTAL_CXX_MODULE_DYNDEP 1)
else()
  message(FATAL_ERROR "Version lower than 3.26 not supported")
endif()
```

Разберем этот код по шагам.

1. Сначала проверяется, что номер версии 3.28 или больше. В этом случае мы можем включить политику `CMP0155` командой `cmake_policy()`. Это необходимо, если мы хотим поддерживать версии более ранние, чем 3.28.
2. Если это не так, то проверяем, что версия равна или старше 3.27. Если это так, то устанавливаем соответствующий ключ API.
3. Если версия не старше 3.27, то сравниваем ее с 3.26. Если она равна или старше, то устанавливаем соответствующий ключ API и разрешаем динамические зависимости для модулей C++20.
4. Если версия младше 3.26, то мы не можем поддержать ее, поэтому печатаем сообщение о фатальной ошибке.

Это позволяет поддержать диапазон версий CMake начиная с 3.26. Если мы пользуемся роскошью использовать CMake 3.28 в любой среде, где будет собираться проект, то этот блок команд `if()` не нужен. А что нужно?

Включение поддержки для CMake 3.28 и старше

Чтобы использовать модули C++20 в версиях, начиная с 3.28, необходимо явно объявить эту версию как минимальную. В заголовке проекта напишите:

```
cmake_minimum_required(VERSION 3.28.0)
project(CXXModules CXX)
```

Это разрешает политику `CMP0155` по умолчанию, если минимально необходимая версия не ниже 3.28. Чтобы узнать, какие еще аспекты необходимо сконфигурировать до определения модуля, читайте дальше. Если вы затребуете версию 3.27 или более раннюю, то сборка, скорее всего, завершится неудачно, даже если проект собирается версией CMake 3.28 или старше.

Далее рассмотрим требования к компилятору.

Задание требований к компилятору

Независимо от того, какой версией CMake вы собираете проект – 3.26, 3.27, 3.28 или более поздней, – для создания решений, в которых используются модули C++, необходимо задать две глобальные переменные. Первая запрещает неподдерживаемые расширения C++, а вторая гарантирует, что компилятор поддерживает требуемый стандарт.

ch10/01-cxx-modules/CMakeLists.txt (продолжение)

```
# В libc++ нет поддержки расширений компилятора для модулей.
set(CMAKE_CXX_EXTENSIONS OFF)
set(CMAKE_CXX_STANDARD 20)
```

Может показаться, что задание стандарта излишне, поскольку есть очень мало компиляторов, поддерживающих модули. Тем не менее это разумная практика для проектов, ориентированных на будущее развитие.

На этом общая конфигурация заканчивается, и мы можем перейти к определению модуля в CMake.

Объявление модуля C++

Для определения модуля в CMake используется команда `target_sources()` и ключевое слово `FILE_SET`:

```
target_sources(math
  PUBLIC FILE_SET CXX_MODULES TYPE CXX_MODULES FILES math.cppm
)
```

В выделенной строке мы видим новый тип набора файлов: `CXX_MODULES`. Этот тип по умолчанию поддерживается, начиная с версии CMake 3.28. Для версии 3.26 необходимо активировать экспериментальный API. Без должной поддержки будет выдано сообщение об ошибке:

```
CMake Error at CMakeLists.txt:25 (target_sources):
target_sources File set TYPE may only be "HEADERS"
```

Если вы увидите такое, проверьте правильность кода. Это сообщение выдается и тогда, когда указан ключ API, не соответствующий используемой версии.

Определение модулей в том же двоичном файле, в котором они используются, имеет свои преимущества (см. выше). Однако эти преимущества выражены еще ярче, когда мы создаем библиотеку. Такие библиотеки можно использовать в других проектах или в том же проекте из других библиотек, что дополнительно увеличивает степень модульности.

Чтобы объявить модуль и скомпоновать его с главной программой, используется следующая конфигурация CMake:

ch10/01-cxx-modules/CMakeLists.txt (continued)

```
add_library(math)
target_sources(math
  PUBLIC FILE_SET CXX_MODULES FILES math.cppm
)
target_compile_features(math PUBLIC cxx_std_20)
set_target_properties(math PROPERTIES CXX_EXTENSIONS OFF)

add_executable(main main.cpp)
target_link_libraries(main PRIVATE math)
```

Чтобы гарантировать возможность использования этой библиотеки в других проектах, мы должны выполнить команду `target_compile_features()`, явно потребовав версию стандарта `cxx_std_20`. Кроме того, мы должны повторить задание свойства `CXX_EXTENSIONS OFF` на уровне цели. Иначе CMake выдаст ошибку и прекратит сборку. Это представляется избыточным и, вероятно, будет устранено в будущих версиях CMake.

Подготовив проект, мы можем наконец собрать его.

Конфигурирование комплекта инструментов

Согласно блогу компании Kitware (см. раздел «Для дополнительного чтения»), CMake поддерживает начиная с версии 3.25. Но официальная поддержка объявлена только в версии 3.28, и это не единственный кусочек пазла, который предстоит поставить на нужное место, чтобы наслаждаться всеми преимуществами модулей.

Следующее требование относится к сборочной системе: она должна поддерживать динамические зависимости. На сегодняшний момент у нас есть только две возможности:

- Ninja 1.11 и старше (Ninja и Ninja Multi-Config);
- Visual Studio 17 2022 и старше.

Также компилятор должен генерировать файлы, которые отображают зависимости исходных файлов для CMake в определенном формате. Этот формат описан в статье разработчиков из Kitware, известной под названием `p1589r5`. Она была направлена всем компаниям-производителям основных компиляторов для реализации. В настоящее время только три компилятора реализовали это требование:

- Clang 16;
- MSVC в Visual Studio 2022 17.4 (19.34);
- GCC 14 (для ветви, находящейся в процессе разработки, после 9.20.2023) и более поздние версии.

В предположении, что у вас имеются все необходимые инструменты (можете использовать Docker-образ, прилагаемый к этой книге) и проект CMake готов к сборке, остается только сконфигурировать CMake, так чтобы использовался нужный комплект инструментов. Еще в первой главе было отмечено, что генератор сборочной системы можно выбрать с помощью аргумента командной строки `-G`:

```
cmake -B <build tree> -S <source tree> -G "Ninja"
```

Эта команда конфигурирует проект для использования сборочной системы Ninja. Следующий шаг – задать компилятор. Если ваш компилятор по умолчанию не поддерживает модули, но в системе установлен еще один для

экспериментов, то можете выбрать его, определив глобальную переменную `CMAKE_CXX_COMPILER`:

```
cmake -B <build tree> -S <source tree> -G "Ninja" -D CMAKE_CXX_
COMPILER=clang++-18
```

В этом примере мы выбрали Clang 18, потому что это последняя версия, доступная на момент написания книги (включена в Docker-образ). По успешном завершении конфигурирования (могут быть выданы предупреждения об экспериментальных средствах) необходимо собрать проект:

```
cmake --build <build tree>
```

Как всегда, не забудьте заменить поля `<build tree>` и `<source tree>` подходящими путями. Если все пройдет гладко, то вы сможете запустить программу и наблюдать, что функция, оформленная в виде модуля, работает, как положено:

```
$ ./main
Addition 2 + 2 = 4
```



В разделе «Для дополнительного чтения» имеются ссылки на статью в блоге компании Kitware и предложение о формате зависимостей между исходными файлами для компиляторов C++, в которых содержатся дополнительные сведения о реализации и использовании модулей C++20.

Резюме

В этой главе мы рассмотрели модули C++20, пояснив, что они не имеют ничего общего с модулями CMake и знаменуют значительный прогресс в области ускорения компиляции и решения проблем избыточной компиляции и макросов препроцессора. На простом примере мы продемонстрировали, как написать и импортировать модуль C++20. Затем обсудили, как настроить CMake для работы с модулями C++20. Поскольку это экспериментальное средство, необходимо задать некоторые переменные, и мы показали последовательность условных предложений, гарантирующую, что проект будет правильно сконфигурирован для используемой версии CMake.

Что касается необходимых инструментов, мы подчеркнули, что сборочная система должна поддерживать динамические зависимости; на сегодняшний момент этому условию удовлетворяет Ninja версии 1.11 или более поздней. Что до поддержки со стороны компиляторов, Clang 16 и MSVC в Visual Studio 2022 17.4 (19.34) предоставляют полную поддержку модулей C++20, а в GCC она еще только ожидается. Мы показали, как сконфигурировать CMake для

использования выбранного комплекта инструментов, в т. ч. выбрать генератор сборочной системы и указать версию компилятора. После конфигурирования и сборки проекта вы можете запустить программу и полюбоваться на модуль C++20 в действии.

В следующей главе вы узнаете о важности и применении автоматизированного тестирования и о том, какую поддержку CMake оказывает каркасам тестирования.

Для дополнительного чтения

Дополнительные сведения по рассмотренным в этой главе темам можно получить из следующих источников:

- статья в блоге с описанием новой возможности: https://www.kitware.com/import-cmake-c20-modules/;
- предложение о формате зависимостей исходных файлов для компиляторов C++: https://www.open-td.org/jtc1/sc22/wg21/docs/papers/2022/p1689r5.html.

11

Каркасы тестирования

Профессионалы знают, что тестирование должно быть автоматизировано. Кто-то объяснил им это много лет назад, или они сами пришли к этому выводу на собственном горьком опыте. Неопытным программистам эта практика не так очевидна; она кажется им необязательной, а результат якобы не стоит потраченных усилий. Оно и понятно: когда человек начинает писать код, ему еще только предстоит создать сложное решение и работать с большой кодовой базой. Очень может быть, что он единственный разработчик своего домашнего проекта. Такие небольшие проекты редко требуют больше, чем несколько месяцев работы, поэтому мало шансов увидеть, как со временем качество кода падает.

Все эти факторы подкрепляют веру в то, что написание тестов – лишняя трата времени и сил. Новичок может убедить себя в том, что он и так тестирует код всякий раз, как выполняет процедуру *сборки и запуска*. Ведь он же вручную подтвердил, что код работает и делает то, что нужно. А значит, пора переходить к следующей задаче, верно?

Автоматизированное тестирование гарантирует, что новые изменения не приведут к непреднамеренной ошибке в программе. В этой главе мы узнаем, почему тесты важны и как использовать CTest, инструмент, поставляемый вместе с CMake, чтобы координировать выполнение тестов. CTest может опросить, какие есть тесты, профильтровать их выполнение, поменять порядок, повторить и задать временные лимиты. Мы научимся использовать эти возможности, управлять выводом CTest и обрабатывать ошибки тестов.

Затем мы модифицируем структуру проекта, чтобы включить в него тестирование, и создадим собственный исполнитель тестов. Рассмотрев основные принципы, мы перейдем к популярным каркасам тестирования: Catch2 и GoogleTest, или GTest, с его библиотекой для создания подставных объектов. Наконец, мы познакомимся с инструментом оценки покрытия тестами LCOV.

В этой главе будут рассмотрены следующие вопросы:

- почему нужно задуматься об автоматизированных тестах;
- применение CTest для стандартизации тестирования в CMake;
- создание элементарного автономного теста для CTest;
- каркасы автономного тестирования;
- генерирование отчетов о покрытии тестами.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch11.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Почему нужно задуматься об автоматизированных тестах?

Представьте себе заводскую линию, на которой машина проделывает отверстия в листе стали. Отверстия должны быть определенного размера и формы, чтобы в них можно было впоследствии вставить болты. Конструктор автоматизированной линии настраивает машину, проверяет отверстия и уходит. Но рано или поздно что-то изменится: сталь может оказаться толще, рабочий может изменить размер отверстия или нужно будет проделать больше отверстий из-за смены дизайна конечного изделия. Предусмотрительный конструктор позаботится о контроле качества в ключевых точках, чтобы изделие отвечало техническим условиям. Неважно, как именно проделаны отверстия – просверлены, пробиты или прорезаны лазером, – они должны удовлетворять определенным требованиям.

Тот же принцип применим к разработке программного обеспечения. Трудно предсказать, какой код останется стабильным на протяжении многих лет, а какой будет многократно модифицироваться. Поскольку функциональность программы расширяется, необходимо гарантировать, что изменения не станут причиной неработоспособности. И все мы допускаем ошибки. Даже самые лучшие программисты не могут предвидеть последствий каждого изменения. Разработчики часто работают с кодом, который не они писали, а потому не знают всех лежащих в его основе предположений. Они читают код, строят мысленную модель, вносят изменения и надеются на лучшее. Если надежда не оправдывается, то исправление ошибки может занять несколько часов или дней, что негативно отразится на продукте и его пользователях.

Иногда оказывается, что код трудно понять. Вы даже можете начать обвинять в путанице других людей, но затем понимаете, что виной всему – вы сами. Это случается, когда код пишется впопыхах без полного понимания проблемы.

Мы, разработчики, не просто живем под прессом сроков и ограниченных бюджетов; бывает, что нас будят ночью, требуя исправить критическую ошибку. Печально, но некоторые не слишком очевидные ошибки могут проскользнуть сквозь сито инспекции кода.

Автоматизированные тесты могут предотвратить большую часть таких проблем. Это фрагменты кода, которые проверяют правильность работы другого куска кода. Как явствует из названия, эти тесты прогоняются автоматически после внесения изменений, обычно как часть процесса сборки. Часто они добавляются как отдельный шаг, гарантирующий качество кода перед его помещением в репозиторий.

Возможно, вы испытываете соблазн пропустить создание автоматизированных тестов ради экономии времени, но это ошибка, которая дорого обойдется. Стивен Райт говорил: «*Опыт – такая вещь, которую можно получить только после того, как она понадобилась*». Если код не будет выброшен сразу после использования и не является экспериментальным, не пренебрегайте тестами. Поначалу вы, возможно, будете огорчены тем, что ваш так любовно написанный код раз за разом проваливает тесты. Но помните: непройденный тест означает, что вы только что избежали внесения серьезной ошибки в рабочую программу. Время, потраченное на тесты, окупится временем на исправление ошибок впоследствии – и вы будете спокойно спать по ночам. Тесты не так трудно писать и сопровождать, как может показаться.

Применение CTest для стандартизации тестирования в CMake

В конечном счете автоматизированное тестирование – это просто запуск исполняемого файла, который переводит **тестируемую систему** (System Under Test – **SUT**) в определенное состояние, выполняет операции, которые вы хотите протестировать, и проверяет, совпадает ли полученный результат с ожидаемым. Можете считать, что это структурированный способ исполнить предложение `ДАНО_<УСЛОВИЕ>_ЕСЛИ_<СЦЕНАРИЙ>_ТО_<ОЖИДАЕМЫЙ-ИСХОД>` и проверить, будет ли результат истинным для SUT. В некоторых системах предлагается именно так и именовать тестовые функции, например `GIVEN_4_and_2_WHEN_Sum_THEN_returns_6`.

Есть много способов реализовать и выполнить тесты в зависимости от выбранного каркаса, способа подключения к SUT и настройки окружения. Для пользователя, который впервые работает с вашим проектом, даже такие мелкие детали, как имя двоичного файла с тестами, могут повлиять на впечатление о нем. Поскольку не существует стандартных соглашений об именовании, один разработчик может назвать исполняемый файл `test_my_app`, другой – `unit_tests`, а третий – выбрать не такое прямолинейное имя или вообще опустить тесты. Разбираться в том, какой файл выполнить, какой каркас используется, какие аргументы передавать и как собирать результаты, – все это детали, в которые пользователь предпочел бы не вдаваться.

CMake решает эту проблему с помощью отдельного командного инструмента `ctest`. Он конфигурируется автором проекта с помощью списковых файлов и предлагает стандартизированный способ прогона тестов. Этот единообразный интерфейс применяется ко всем проектам, собранным при помощи CMake. Следование этому стандарту дает и другие преимущества: упрощается интеграция проекта с конвейером **непрерывной интеграции и непрерывного развертывания** (**CI/CD**), а тесты становится удобнее просматривать в таких IDE, как Visual Studio или CLion. Но самое важное – вы получаете надежный исполнитель тестов, затратив минимум усилий.

И как же прогонять тесты с помощью CTest в уже сконфигурированном проекте? Нужно выбрать один из трех режимов работы:

- информационная панель;
- тестирование;
- сборка и тестирование.

В **режиме информационной панели** вы можете отправлять результаты тестов отдельному инструменту CDash, также разработанному компанией Kitware. CDash собирает и представляет результаты контроля качества ПО на информационной панели с простыми средствами навигации. Это полезно в очень больших проектах, так что выходит за рамки данной книги.

Командная строка в **режиме тестирования** выглядит так:

```
ctest [<options>]
```

В этом режиме CTest должна выполняться в дереве сборки, после того как проект собран с помощью CMake. Параметров много, но прежде чем перейти к их изучению, упомянем мелкое неудобство: двоичный файл `ctest` необходимо выполнять в дереве сборки, после того как проект собран. Это немного мешает во время цикла разработки, потому что приходится выполнять несколько команд и переходить из одного каталога в другой.

Чтобы упростить работу, CTest предлагает **режим сборки и тестирования**. Мы сначала рассмотрим именно этот режим, чтобы затем отдать все внимание режиму тестирования.

Режим сборки и тестирования

В этом режиме мы должны выполнить `ctest` с флагом `--build-and-test`:

```
ctest --build-and-test <source-tree> <build-tree>
      --build-generator <generator> [<options>...]
      [--build-options <opts>...]
      [--test-command <command> [<args>...]]
```

По существу, это простая обертка вокруг тестового режима. Она принимает параметры конфигурации сборки и команду тестирования после параметра `--test-command`. Важно отметить, что никакие тесты не будут выполнены, если не включить ключевое слово `ctest` после `--test-command`, как показано ниже:

```
ctest --build-and-test project/source-tree /tmp/build-tree --buildgenerator
"Unix Makefiles" --test-command ctest
```

В этой команде мы указываем пути к исходным файлам и к артефактам сборки, а также выбираем генератор сборки. Все три аргумента обязательны и следуют правилам команды `cmake`, подробно описанным в главе 1.

Можно задать дополнительные аргументы, относящиеся к одной из трех категорий: управление конфигурированием, процесс сборки и параметры тестирования.

Аргументы для управления этапом конфигурирования следующие:

- `--build-options` – включить дополнительные параметры для конфигурирования `cmake`. Они должны быть помещены непосредственно перед параметром `--test-command`, который должен быть последним;
- `--build-two-config` – выполнить этап конфигурирования дважды;
- `--build-nocmake` – пропустить этап конфигурирования;
- `--build-generator-platform` – указать зависящую от генератора платформу;
- `--build-generator-toolset` – указать зависящий от генератора комплект инструментов;
- `--build-makeprogram` – указать исполняемый файл `make` для генераторов, основанных на Make и Ninja.

Аргументы для этапа сборки следующие:

- `--build-target` – указать, какую цель собирать;
- `--build-noclean` – сборка без предварительной сборки цели `clean`;
- `--build-project` – имя собираемого проекта.

Для этапа тестирования есть один аргумент:

- `--test-timeout` – задать максимальное время выполнения тестов в секундах.

Теперь мы можем сконфигурировать режим тестирования, добавив аргументы после `--test-command cmake` или запустив его непосредственно.

Режим тестирования

Собрав проект, вы можете использовать команду `ctest` в каталоге сборки для прогона тестов. В режиме сборки и тестирования это будет сделано автоматически. В большинстве случаев достаточно запустить `ctest` без флагов. Если все тесты прошли успешно, то `ctest` вернет код 0 (в Unix-подобных системах), который можно проверить в конвейере CI/CD, чтобы предотвратить объединение некорректных изменений с рабочей ветвью.

Написание хороших тестов может быть столь же трудным делом, сколь и написание самого кода. Мы переводим тестируемую систему (SUT) в определенное состояние, прогоняем один тест, а затем очищаем экземпляр SUT. Это довольно сложный процесс, чреватый разного рода проблемами: межтестовое загрязнение состояния, нарушения хронометража и конкурентности,

конкуренция за ресурсы, зависания из-за взаимоблокировок и многое другое. По счастью, CTest предлагает различные меры для смягчения этих проблем. В частности, мы можем управлять тем, какие тесты прогонять, порядком их выполнения, генерируемым выводом, временными ограничениями и коэффициентами повторения. В следующих разделах описывается необходимый контекст и дается краткий обзор наиболее полезных возможностей.

Опрос тестов

Первое, что мы должны сделать, – понять, какие вообще тесты написаны для проекта. CTest принимает параметр `-N`, который подавляет выполнение и просто печатает список:

```
# ctest -N
Test project /tmp/b
  Test #1: SumAddsTwoInts
  Test #2: MultiplyMultipliesTwoInts
Total Tests: 2
```

Мы можем использовать `-N` совместно с фильтрами, описанными в следующем разделе, для проверки того, какие тесты будут выполнены в случае применения фильтра.

Если вы хотите получать вывод в формате JSON, понятном автоматизированным инструментам, то выполните `ctest` с параметром `--show-only=json-v1`.

CTest также предлагает механизм группировки тестов с помощью ключевого слова `LABELS`. Чтобы перечислить все имеющиеся метки (не выполняя самих тестов), задайте флаг `--print-labels`. Эта возможность полезна, когда тесты определены вручную командой `add_test(<name> <test-command>)` в списковом файле, поскольку тогда вы можете задать метки с помощью тестовых свойств:

```
set_tests_properties(<name> PROPERTIES LABELS "<label>")
```

Однако имейте в виду, что автоматизированные методы обнаружения тестов в разных каркасах могут не поддерживать такой уровень детализации меток.

Фильтрация тестов

Иногда нужно прогнать только конкретные тесты, а не весь комплект. Например, если вы отлаживаете один непрошедший тест, то прогонять все остальные нет нужды. Этот механизм также можно использовать для распределения тестов между несколькими машинами в крупных проектах.

Следующие флаги фильтруют тесты в соответствии с заданным **регулярным выражением** `<r>`:

- `-R <r>, --tests-regex <r>` – прогонять только тесты, имена которых сопоставляются с `<r>`;

- `-E <r>, --exclude-regex <r>` – пропускать тесты, имена которых сопоставляются с `<r>`;
- `-L <r>, --label-regex <r>` – прогонять только тесты, метки которых сопоставляются с `<r>`;
- `-LE <r>, --label-exclude <regex>` – пропускать тесты, метки которых сопоставляются с `<r>`.

Большей гибкости можно достичь с помощью параметра `--tests-information` (сокращенно `-I`). Он принимает диапазон значений через запятую: `<начало>`, `<конец>`, `<шаг>`, `<ИД-тестов>`. Любое поле можно опустить, оставив запятую. `<ИД-тестов>` – это список порядковых номеров тестов, подлежащих выполнению, через запятую. Например:

- `-I 3,,` пропускает тесты 1 и 2 (исполнение начинается с третьего теста);
- `-I ,2,` прогоняет только первый и второй тесты;
- `-I 2,,3` последовательно прогоняет каждый третий тест, начиная со второго;
- `-I ,0,,3,9,7` прогоняет только третий, девятый и седьмой тесты.

Можно также задать эти диапазоны в файле, чтобы выполнять тесты одновременно на нескольких машинах, когда комплект тестов очень велик. Если флаг `-I` используется в сочетании с `-R`, то прогоняются тесты, отвечающие обоим критериям. Если вы хотите прогнать тесты, отвечающие хотя бы одному критерию, задайте флаг `-U`. Как уже было сказано, для проверки результата фильтрации можно использовать флаг `-N`.

Тасование тестов

Писать автономные тесты бывает нелегко. Одна из самых неожиданных проблем – сцепленность тестов, когда один тест влияет на другой из-за неполной инициализации или очистки состояния SUT. Иными словами, первый тест может «оставить следы» своего состояния и тем самым испортить состояние системы перед вторым тестом. Такая сцепленность – неприятная новость, поскольку вносит неизвестные и неявные связи между тестами.

Хуже того, известно, что такого рода ошибки могут оставаться незамеченными из-за сложностей сценариев тестирования. Иногда их можно обнаружить, если какой-то тест по непонятной причине не проходит, но возможно и обратное: из-за некорректного состояния тест проходит, хотя не должен бы. Такие ложные прохождения тестов вселяют в разработчиков иллюзию уверенности, а это еще хуже, чем вообще не писать тестов. Предположение о том, что код успешно протестирован, может подвигнуть на более смелые действия, которые приведут к неожиданным результатам.

Один из способов обнаружить такие проблемы – прогонять каждый тест изолированно. Но обычно так не бывает, когда исполнители тестов запускаются прямо из каркаса тестирования без CTest. Чтобы прогнать одиночный тест, мы должны передать зависящий от каркаса аргумент исполняемому файлу. Это позволит обнаружить тесты, которые проходят в комплекте, но не проходят, если запускаются отдельно.

CTest, с другой стороны, устраняет даже теоретическую возможность взаимного влияния тестов в памяти, поскольку неявно исполняет каждый

тест в дочернем экземпляре CTest. Можно пойти еще дальше и указать флаг `--force-new-ctest-process`, который означает, что каждый тест исполняется в отдельном процессе.

К сожалению, одного этого недостаточно, если тесты совместно используют внешние ресурсы, например GPU, базы данных или файлы. Но можно предпринять дополнительную меру предосторожности – просто рандомизировать порядок выполнения тестов. Часто этого хватает, чтобы в конце концов выявить тесты, которые иногда проходят, а иногда нет. В CTest эта стратегия реализуется флагом `--schedule-random`.

Обработка ошибок

Хорошо известно высказывание Джона К. Максвелла: «Падай рано, падай часто, но всегда падай вперед». Падать вперед означает учиться на собственных ошибках. Это именно то, что нам нужно, когда мы прогоняем автономные тесты (и, пожалуй, в других жизненных ситуациях). Если только вы не исполняете тесты под отладчиком, то понять, где ошибка, нелегко, поскольку CTest не любит многословия и выводит только список непрошедших тестов, но не их вывод.

Сообщения, выведенные тестом или тестируемой системой на `stdout`, могут оказаться бесценным подспорьем для точного определения ошибки. Чтобы увидеть их, мы можем запустить `ctest` с флагом `--output-on-failure`. Того же результата можно добиться, задав переменную окружения `CTEST_OUTPUT_ON_FAILURE`.

В зависимости от размера решения бывает полезно остановить выполнение, как только какой-нибудь тест не пройдет. Для этого следует задать флаг `--stop-on-failure`.

CTest сохраняет имена непрошедших тестов. Чтобы сэкономить время при прогоне длинных комплектов тестов, мы можем сосредоточиться только на непрошедших тестах и опустить прогон прошедших до тех пор, пока проблема не будет решена. Этот режим включается заданием флага `--rerun-failed` (все остальные фильтры при этом игнорируются). Не забудьте прогнать все тесты после разрешения всех проблем, чтобы убедиться в отсутствии регрессии.

Если CTest не находит никаких тестов, то возможно одно из двух: либо тестов нет, либо возникла проблема в проекте. По умолчанию `ctest` печатает предупреждение и возвращает код 0, чтобы не вносить смуту. Как правило, у пользователя достаточно контекста, чтобы понять, с чем он столкнулся и что делать дальше. Но в некоторых ситуациях `ctest` всегда выполняется в составе автоматизированного конвейера. Тогда имеет смысл явно указать, что отсутствие тестов следует интерпретировать как ошибку (и вернуть ненулевой код). Такое поведение задается аргументом `--no-tests=error`, а противоположное (отсутствие предупреждений) – аргументом `--no-tests=ignore`.

Повтор тестов

Рано или поздно в вашей карьере настанет такой момент, когда тесты правильно работают в большинстве случаев. Я хочу подчеркнуть – «в большинстве». Но раз в сто лет тесты падают по каким-то внешним причинам: не-

правильно смоделированное время, проблемы с циклом обработки событий, неправильная обработка асинхронного выполнения, параллелизм, конфликты хеширования и другие сложные ситуации, которые не случаются при каждом прогоне. Такие тесты называются *нестабильными*.

Эта нестабильность кажется не такой уж важной проблемой. Мы могли бы сказать, что тесты – не настоящая производственная среда, и именно поэтому они время от времени не проходят. Доля истины в этом есть: тесты и не должны повторять все мельчайшие детали – это было бы попросту нереально. Тесты – это моделирование, приближение к реальности, и обычно этого достаточно. Что плохого в том, чтобы заново прогнать тесты и убедиться, что при следующем выполнении они проходят?

Оказывается, все плохо. И ниже перечислены три основные озабоченности.

- Если в кодовой базе скопилось много нестабильных тестов, то они станут серьезным препятствием к бесперебойной доставке изменений кода. Особенно это раздражает, когда вы торопитесь и, вместо того чтобы пойти домой пятничным вечером, вынуждены заниматься критическим исправлением серьезной ошибки, влияющей на заказчиков.
- Не может быть стопроцентной уверенности в том, что нестабильные тесты падают именно из-за неполной адекватности тестовой среды. Быть может, все наоборот: они падают, потому что удалось нащупать редкий случай, который может встретиться в условиях эксплуатации. Просто это недостаточно очевидно, чтобы поднимать тревогу… пока.
- Не тест нестабилен, а ваш код! Окружение время от времени глючит – но программисты обрабатывают такие случаи детерминированно. Если тестируемая система ведет себя подобным образом, это признак серьезной ошибки – например, код может читать из неинициализированной памяти.

Не существует идеального способа решить все эти проблемы – разнообразие возможных причин слишком велико. Однако мы можем повысить вероятность выявления нестабильных тестов, если будем прогонять их повторно, задав аргумент `-repeat <режим>:<#>`. Имеется три режима:

- `until-fail` – прогнать тест `<#>` раз; все прогоны должны быть успешны;
- `until-pass` – прогнать тест не более `<#>` раз; он должен пройти хотя бы единожды. Это полезно, когда мы имеем дело с тестами, которые заведомо нестабильны, но слишком трудны для отладки и слишком важны, чтобы от них отказаться;
- `after-timeout` – прогнать тест не более `<#>` раз, но повторять попытку, только если тест вылетает по тайм-ауту. Используйте в высоко нагруженных тестовых средах.

Общая рекомендация – отладить нестабильные тесты как можно скорее или избавиться от них, если доверять их результатам нельзя.

Управление выводом

Печатать все подряд на экран при каждом выполнении – только затемнять суть дела. CTest снижает шум и сохраняет вывод выполненных тестов в жур-

нальных файлах, показывая при обычном прогоне лишь наиболее полезную информацию. Если все плохо и тест не проходит, то вы можете ожидать краткую сводку и, быть может, некоторые журналы, если задали вышеупомянутый флаг `--output-on-failure`.

Я по опыту знаю, что «достаточно информации» достаточно только до тех пор, пока не оказывается, что ее не хватает. Иногда мы хотим видеть также вывод прошедших тестов, быть может, чтобы убедиться, что они действительно работают (а не просто молча останавливаются, не сообщая об ошибке). Чтобы получить доступ к более подробному выводу, задайте флаг `-V` (или `--verbose`, если хотите явно выразить свое намерение в автоматизированном конвейере). Если и этого недостаточно, задайте флаг `-VV` или `--extra-verbose`. Для максимально глубокой отладки задайте флаг `--debug` (но будьте готовы к половодью текста, содержащего все детали).

Если вам нужно нечто прямо противоположное, то CTest предлагает «режим дзен», активируемый флагом `-Q` или `--quiet`. Тогда не будет печататься ничего (вы перестанете волноваться и научитесь любить ошибки). На мой взгляд, у этой возможности единственная цель – смущать людей, но имейте в виду, что вывод все равно сохраняется в файлах (по умолчанию в каталоге `./Testing/Temporary`). Автоматизированные конвейеры могут проверить, равен ли код выхода нулю, и собрать журналы для дальнейшей обработки, не загромождая основной вывод деталями, которые могут только запутать разработчиков, незнакомых с продуктом.

Чтобы хранить журналы в конкретном каталоге, задайте аргумент `-O <file>` или `--output-log <file>`. Если вас раздражает длинный вывод, то можно ограничить его заданным числом байтов на каждый тест: `--test-output-size-passed <size>` и `--test-output-size-failed <size>`.

Разное

Есть еще несколько параметров, которые могут быть полезны при повседневном тестировании.

- `-C <cfg>`, `--build-config <cfg>` – задает тестируемую конфигурацию. В конфигурации `Debug` обычно присутствуют отладочные символы, поэтому работать с ней проще, но конфигурацию `Release` тоже необходимо тестировать, потому что агрессивная оптимизация потенциально способна повлиять на поведение SUT. Этот параметр применим только к генераторам, поддерживающим несколько конфигураций.
- `-j <jobs>`, `--parallel <jobs>` – задает, сколько тестов исполнять параллельно. Это очень полезно, когда нужно ускорить выполнение длинных тестов во время разработки. Но имейте в виду, что в высоко нагруженной среде (с разделяемым исполнителем тестов) это может оказать негативный эффект из-за планирования. Его можно до некоторой степени смягчить с помощью следующего параметра.
- `--test-load <level>` – планировать параллельные тесты, так чтобы нагрузка на CPU не превышала значения `<level>` (по мере возможностей).

- `--timeout <seconds>` – задать временной лимит по умолчанию для одного теста.

Понимая, как запускать `ctest` в различных сценариях, посмотрим, как добавить простой тест.

Создание элементарного автономного теста для CTest

Технически писать автономные тесты можно вообще без каркаса. Нужно лишь создать экземпляр тестируемого класса, выполнить один из его методов и проверить, отвечает ли ожиданиям новое состояние или возвращенное значение. Затем мы печатаем отчет о результате и удаляем объект. Проделаем это.

Мы будем использовать следующую структуру:

```
- CMakeLists.txt
- src
  |- CMakeLists.txt
  |- calc.cpp
  |- calc.h
  |- main.cpp
- test
  |- CMakeLists.txt
  |- calc_test.cpp
```

В файле `main.cpp` используется класс `Calc`:

ch11/01-no-framework/src/main.cpp

```
#include <iostream>
#include "calc.h"
using namespace std;
int main() {
  Calc c;
  cout << "2 + 2 = " << c.Sum(2, 2) << endl;
  cout << "3 * 3 = " << c.Multiply(3, 3) << endl;
}
```

Ничего интересного – `main.cpp` просто включает заголовочный файл `calc.h` и вызывает два метода объекта `Calc`. Рассмотрим интерфейс `Calc` нашей тестируемой системы:

ch11/01-no-framework/src/calc.h

```
#pragma once
class Calc {
```

```
public:
   int Sum(int a, int b);
   int Multiply(int a, int b);
};
```

Интерфейс – проще не бывает. Мы воспользовались директивой `#pragma once` – она работает точно так же, как **защита от повторного включения**, и поддерживается почти всеми современными компиляторами, хотя в официальный стандарт и не вошла.

Приведем реализацию класса.

ch11/01-no-framework/src/calc.cpp

```
#include "calc.h"
int Calc::Sum(int a, int b) {
  return a + b;
}
int Calc::Multiply(int a, int b) {
  return a * a; // ошибка!
}
```

Опаньки! Мы посадили ошибку! `Multiply` игнорирует аргумент `b` и вместо произведения аргументов возвращает квадрат `a`. Правильно написанные автономные тесты должны это обнаружить. Ну так напишем их!

ch11/01-no-framework/test/calc_test.cpp

```
#include "calc.h"
#include <cstdlib>
void SumAddsTwoIntegers() {
  Calc sut;
  if (4 != sut.Sum(2, 2))
    std::exit(1);
  }
void MultiplyMultipliesTwoIntegers() {
  Calc sut;
  if(3 != sut.Multiply(1, 3))
    std::exit(1);
}
```

В нашем файле `calc_test.cpp` имеется два тестовых метода, по одному для каждого метода SUT. Если значение, возвращенное вызванным методом, не совпадает с ожидаемым, то каждая функция вызывает `std::exit(1)`. Можно было использовать также `assert()`, `abort()` или `terminate()`, но это привело бы к сообщению `Subprocess aborted` в выводе `ctest` – менее понятному, чем простое `Failed`.

Пора создать исполнитель тестов. Он будет максимально простым, чтобы не тратить слишком много времени на работу. Взгляните на функцию `main()`, которую мы написали с единственной целью – прогнать два теста.

ch11/01-no-framework/test/unit_tests.cpp

```
#include <string>
void SumAddsTwoIntegers();
void MultiplyMultipliesTwoIntegers();
int main(int argc, char *argv[]) {
  if (argc < 2 || argv[1] == std::string("1"))
    SumAddsTwoIntegers();
  if (argc < 2 || argv[1] == std::string("2"))
    MultiplyMultipliesTwoIntegers();
}
```

Разберем ее по шагам.

1. Мы объявляем две внешние функции, которые находятся в другой единице трансляции и будут прикомпонованы.
2. Если не было задано аргументов, то выполняются оба теста (нулевой элемент массива `argv[]` всегда содержит имя программы).
3. Если первый аргумент содержит идентификатор теста, то выполнить этот тест.
4. Если какой-нибудь тест не прошел, он сам вызывает `exit()` и возвращает код `1`.
5. Если тесты не выполнялись или закончились успешно, то неявно возвращается код `0`.

Для прогона первого теста выполним команду

```
./unit_tests 1
```

Для прогона второго теста выполним команду

```
./unit_tests 2
```

Мы упростили код, насколько вообще возможно, но читать его все равно тяжело. Человеку, которому поручили сопровождать код из этого раздела, придется повозиться, если он захочет добавить еще несколько тестов. Функциональность плохо организована – отлаживать такой комплект тестов будет трудно. Тем не менее посмотрим, как можно воспользоваться им в сочетании с CTest:

ch11/01-no-framework/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(NoFrameworkTests CXX)
include(CTest)
add_subdirectory(src bin)
add_subdirectory(test)
```

Мы начинаем с обычного заголовка и команды `include(CTest)`. Это активирует CTest и всегда должно делаться в верхнеуровневом файле `CMakeLists.txt`. Далее мы включаем два вложенных списковых файла в каждом из под-

каталогов: src и test. Указанное значение bin означает, что мы хотим поместить результат компиляции исходных файлов в подкаталоге src в каталог <build_tree>/bin. В противном случае двоичные файлы оказались бы в каталоге <build_tree>/src, что только запутало бы пользователя, поскольку артефакты сборки – это не исходные файлы.

В каталоге src списковый файл простой и содержит только определение цели main:

ch11/01-no-framework/src/CMakeLists.txt

```
add_executable(main main.cpp calc.cpp)
```

Нам также нужен списковый файл в каталоге test:

ch11/01-no-framework/test/CMakeLists.txt

```
add_executable(unit_tests
               unit_tests.cpp
               calc_test.cpp
               ../src/calc.cpp)
target_include_directories(unit_tests PRIVATE ../src)
add_test(NAME SumAddsTwoInts COMMAND unit_tests 1)
add_test(NAME MultiplyMultipliesTwoInts COMMAND unit_tests 2)
```

Мы определили вторую цель unit_tests, в которой также используется файл реализации src/calc.cpp и соответствующий заголовок. Наконец, мы явно добавляем два теста:

- SumAddsTwoInts;
- MultiplyMultipliesTwoInts.

Каждый сообщает свой идентификатор в виде аргумента команды add_test(). CTest просто берет все, что стоит после ключевого слова COMMAND, и выполняет это в подоболочке, запоминая вывод и код выхода. Не обращайте слишком много внимания на команду add_test(); в разделе «Каркасы автономного тестирования» ниже мы опишем гораздо лучший способ работы с тестами.

Для прогона тестов выполните ctest в дереве сборки:

```
# ctest
Test project /tmp/b
    Start 1: SumAddsTwoInts
1/2 Test #1: SumAddsTwoInts ................... Passed 0.00 sec
    Start 2: MultiplyMultipliesTwoInts
2/2 Test #2: MultiplyMultipliesTwoInts ........***Failed 0.00 sec
50% tests passed, 1 tests failed out of 2
Total Test time (real) = 0.00 sec
The following tests FAILED:
          2 - MultiplyMultipliesTwoInts (Failed)
Errors while running CTest
```

```
Output from these tests are in: /tmp/b/Testing/Temporary/LastTest.log
Use "--rerun-failed --output-on-failure" to re-run the failed cases
verbosely.
```

CTest выполнила оба теста и сообщила, что один из них не прошел – значение, возвращенное `Calc::Multiply`, не совпало с ожидаемым. Отлично. Теперь мы знаем, что в коде есть дефект, и кто-то должен его исправить.



Вы, наверное, обратили внимание, что в большинстве рассмотренных ранее примеров не использовалась структура проекта, описанная в главе 4. Это сделано ради краткости. В этой главе обсуждаются более сложные вещи, поэтому полная структура оправдана. А в своих проектах (даже самых мелких) вам стоило бы использовать эту структуру с самого начала. Как сказал один мудрец: «Стоит ступить на дорогу, и если дашь волю ногам, неизвестно, куда тебя занесет»[1].

Я надеюсь, теперь вам ясно, что создавать каркас тестирования с нуля для собственного проекта не всегда желательно. Даже самый простой пример трудно воспринимается, сопровождается уймой накладных расходов и не добавляет никакой ценности. Но прежде чем мы перейдем к каркасу автономного тестирования, надо бы переосмыслить структуру проекта.

Структурирование проектов для тестирования

В C++ есть некоторые ограниченные возможности интроспекции, но таких развитых средств, как в Java, он предложить не может. Именно поэтому написание тестов и каркасов автономного тестирования для C++ труднее, чем в других более оснащенных средах. Одним из следствий этой ограниченности является тот факт, что программисту приходится думать о тестопригодности кода. Мы должны тщательно проектировать интерфейсы, учитывая практические аспекты. Например, как избежать двойной компиляции кода и повторно использовать в режиме эксплуатации артефакты, созданные на этапе тестирования?

Для небольших проектов время компиляции – возможно, и не такая серьезная проблема, но по мере развития хотелось бы сохранить краткие циклы компиляции. В примере выше мы в исполняемый файл автономных тестов включили весь исходный код SUT, кроме `main.cpp`. Внимательно приглядевшись, вы заметите, что кое-какой код в этом файле не был протестирован (содержимое самой функции `main()`). При двойной компиляции кода появляется небольшой шанс, что порожденные артефакты *будут не идентичны*. Эти

[1] *Дж. Р. Р. Толкиен*. Властелин колец. (Слова, сказанные Фродо Бильбо Бэггинсом.) – *Прим. перев.*

расхождения со временем накапливаются, особенно при добавлении флагов компиляции и директив препроцессора, и могут стать опасными, когда соразработчики торопятся, неопытны или незнакомы с проектом.

Существует несколько решений этой проблемы, но самое простое – собрать все решение в виде библиотеки и компоновать ее с автономными тестами. Но тогда возникает вопрос, как этот код выполнить. Ответ такой – создать стартовый исполняемый файл, который компонуется с библиотекой и исполняет ее код.

Для начала переименуем функцию `main()` во что-то типа `run()` или `start_program()`. Затем создадим еще один файл реализации (`bootstrap.cpp`), содержащий только новую функцию `main()`. Эта функция играет роль адаптера: предоставляет точку входа и вызывает `run()`, передавая ей все аргументы командной строки. После компоновки мы получим тестопригодный проект.

Благодаря переименованию `main()` мы теперь можем компоновать SUT с тестами и тестировать ее основную функциональность. В противном случае было бы нарушено **правило одного определения** (**ODR**) (см. главу 8), потому что исполнитель тестов также нуждается в собственной функции `main()`. Как и было обещано в разделе «Вынесение логики main() для тестирования» главы 8, сейчас мы обсудим этот вопрос более подробно.

Заметим также, что каркас тестирования мог бы предоставлять свою собственную функцию `main()` по умолчанию, поэтому мы могли бы ее и не писать. Обычно она автоматически находит все прикомпонованные тесты и прогоняет их согласно заданной вами конфигурации.

Артефакты, порождаемые при таком подходе, можно отнести к трем категориям:

- библиотека `sut`, содержащая производственный код;
- двоичный файл `bootstrap`, содержащий обертку `main()`, которая вызывает метод `run()` из `sut`;
- автономные тесты с оберткой `main()`, которая вызывает все тесты `sut`.

На рис. 11.1 показаны символические связи между целями.

В итоге получается шесть файлов реализации, которые порождают соответственные *объектные файлы*:

- `calc.cpp`: подлежащий тестированию класс `Calc`. Он называется **тестируемой единицей** (unit under test – **UUT**), потому что UUT – частный случай SUT;
- `run.cpp`: оригинальная точка входа, переименованная в `run()`, которую теперь можно протестировать;
- `bootstrap.cpp`: новая точка входа `main()`, из которой вызывается `run()`;
- `calc_test.cpp`: тестирует класс `Calc`;
- `run_test.cpp`: сюда можно добавлять новые тесты для `run()`;
- `unit_tests.o`: точка входа для автономных тестов, расширенная для вызова тестов `run()`.

Библиотека, которую мы собираемся написать, необязательно должна быть статической или разделяемой. Выбирая объектную библиотеку, мы можем избежать излишнего архивирования или компоновки. Технически можно сэкономить немного времени за счет использования динамической

компоновки для SUT, но тогда зачастую приходится вносить изменения в обе цели: тесты и SUT, что сводит на нет всю экономию.

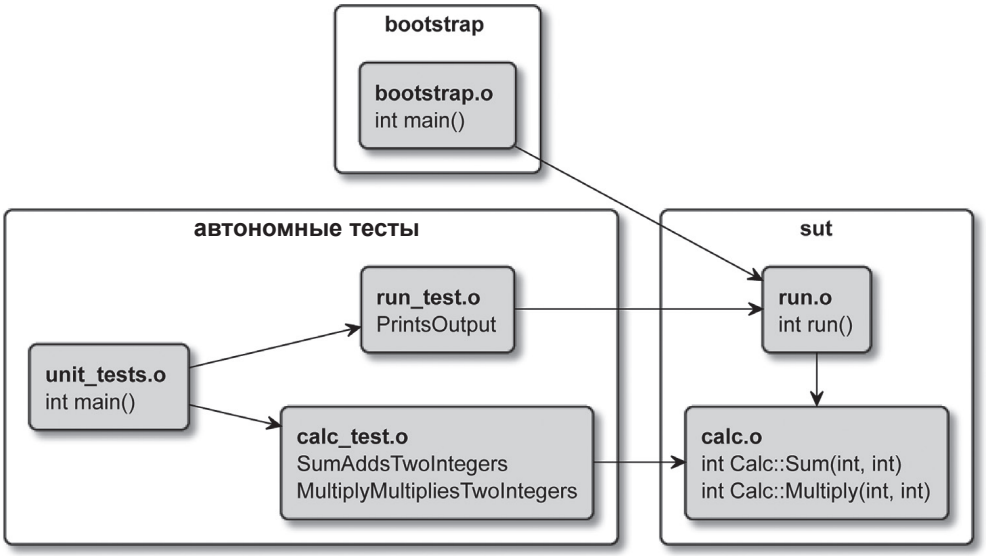


Рис. 11.1. Разделение артефактов между тестовыми и производственными исполняемыми файлами

Посмотрим, как изменились наши файлы, и начнем с того, что раньше называлось `main.cpp`:

ch11/02-structured/src/run.cpp

```
#include <iostream>
#include "calc.h"
using namespace std;
int run() {
  Calc c;
  cout << "2 + 2 = " << c.Sum(2, 2) << endl;
  cout << "3 * 3 = " << c.Multiply(3, 3) << endl;
  return 0;
}
```

Изменения незначительны: мы переименовали файл и функцию и добавили предложение `return`, потому что компилятор не станет добавлять его неявно для функций, отличных от `main()`.

Новая функция `main()` выглядит так:

ch11/02-structured/src/bootstrap.cpp

```
int run(); // объявление
int main() {
```

```
  run();
}
```

Проще говоря, мы объявляем, что компоновщик предоставит в наше распоряжение функцию `run()` из другой единицы трансляции, и вызываем ее:

Далее – списковый файл в каталоге `src`.

ch11/02-structured/src/CMakeLists.txt

```
add_library(sut STATIC calc.cpp run.cpp)
target_include_directories(sut PUBLIC .)
add_executable(bootstrap bootstrap.cpp)
target_link_libraries(bootstrap PRIVATE sut)
```

Сначала мы создаем библиотеку SUT и помечаем `.` как открытый (`PUBLIC`) *каталог включаемых файлов*, так чтобы он распространялся во все цели, компонуемые с SUT (т. е. `bootstrap` и `unit_tests`). Заметим, что пути к каталогам включаемых файлов вычисляются относительно спискового файла, что позволяет использовать точку (`.`) для ссылки на текущий каталог `<source_tree>/src`.

Пора перейти к обновлению цели `unit_tests`. Мы заменим прямую ссылку на файл `../src/calc.cpp` компонуемой ссылкой на `sut`. Мы также добавим новый тест для основной функции в файле `run_test.cpp`. Для краткости опустим это обсуждение, но интересующийся читатель может ознакомиться с примерами в репозитории.

И вот как выглядит полный списковый файл `test`:

ch11/02-structured/test/CMakeLists.txt

```
add_executable(unit_tests
               unit_tests.cpp
               calc_test.cpp
               run_test.cpp)
target_link_libraries(unit_tests PRIVATE sut)
```

ch11/02-structured/test/CMakeLists.txt (продолжение)

```
add_test(NAME SumAddsTwoInts COMMAND unit_tests 1)
add_test(NAME MultiplyMultipliesTwoInts COMMAND unit_tests 2)
add_test(NAME RunOutputsCorrectEquations COMMAND unit_tests 3)
```

Готово! Мы зарегистрировали новый тест. Следуя этому образцу, вы можете быть уверены, что исполнитель прогонит ваши тесты именно для того машинного кода, который исполняется в рабочей системе.



Имена целей `sut` и `bootstrap` выбраны так, чтобы не было ни малейших сомнений в их назначении в контексте тестирования. В реальных проектах выбирайте имена, соответствующие контексту производственного кода (а не тестов). Например, для приложения FooApp используйте имя `foo` вместо `bootstrap` и `lib_foo` вместо `sut`.

Теперь, когда мы знаем, как разбить тестопригодный проект на цели, обратимся к самим каркасам тестирования. Мы ведь не хотим добавлять каждый тест в списковые файлы вручную, правда?

Каркасы автономного тестирования

В предыдущем разделе было показано, что написать простенькую управляющую программу для автономного тестирования не так уж сложно. Возможно, она не слишком красивая, но, хотите верьте, хотите нет, встречаются профессиональные разработчики, которые *любят* изобретать колесо, считая, что их версия будет лучше во всех отношениях. Не попадайтесь в эту ловушку: вам придется написать так много стереотипного кода, что это превратится в самостоятельный проект. Лучше использовать популярные каркасы автономного тестирования – тогда ваше решение будет отвечать стандартам, признанным во многих проектах и компаниях, да вдобавок вы еще будете получать бесплатные обновления и расширения. Беспроигрышный вариант.

Как включить каркас автономного тестирования в свой проект? Просто – вам нужно будет реализовать тесты в соответствии с правилами выбранного каркаса, а затем скомпоновать их с предоставленным исполнителем тестов. Исполнитель тестов инициирует выполнение выбранных тестов и собирает их результаты. В отличие от простого файла `unit_tests.cpp`, который был рассмотрен выше, многие каркасы автоматически находят все тесты и делают их видимыми CTest. Это гораздо более комфортный процесс.

Для этой главы я выбрал два каркаса автономного тестирования, руководствуясь следующими соображениями.

- **Catch2** сравнительно прост для изучения и может похвастаться хорошей поддержкой и документацией. Он поддерживает простые сценарии тестирования, но вместе с тем включает элегантные макросы для **разработки через поведение** (behavior-driven development – **BDD**). Хотя в нем нет некоторых возможностей, его можно при необходимости дополнить внешними инструментами. Домашняя страница находится по адресу https://github.com/catchorg/Catch2.
- **GoogleTest (GTest)** удобен, но и более сложен. Он предлагает широкий круг возможностей, в т. ч. различные утверждения, тесты завершения программы, а также тесты, параметризованные значениями и типами. Он даже поддерживает генерирование отчетов о тестировании в формате XML и подставные объекты с помощью модуля GMock. Домашняя страница находится по адресу https://github.com/google/googletest.

Выбор каркаса зависит от вашего желания учиться и размера проекта. Если вы предпочитаете постепенно входить в курс дела и не нуждаетесь в полном наборе функциональности, то Catch2 станет хорошим выбором. Те же, кто любит сразу нырять с головой и хочет иметь полный комплект инструментов, сочтут GoogleTest более подходящим.

Catch2

Этот каркас сопровождает Мартин Хорженовский, он хорош для начинающих и небольших проектов. Я не хочу сказать, что для более крупных приложений он не годится, но имейте в виду, что иногда могут потребоваться дополнительные инструменты (подробное обсуждение этой темы увело бы нас слишком далеко). Для начала рассмотрим реализацию простого автономного теста для нашего класса `Calc`:

ch11/03-catch2/test/calc_test.cpp

```
#include <catch2/catch_test_macros.hpp>
#include "calc.h"

TEST_CASE("SumAddsTwoInts", "[calc]") {
  Calc sut;
  CHECK(4 == sut.Sum(2, 2));
}

TEST_CASE("MultiplyMultipliesTwoInts", "[calc]") {
  Calc sut;
  CHECK(12 == sut.Multiply(3, 4));
}
```

Вот и все. Но эти несколько строчек результативнее предыдущих примеров. Макросы `CHECK()` не только проверяют ожидаемый результат, они собирают все невыполненные утверждения и представляют их вместе, что помогает избежать многократной перекомпиляции.

Каковы преимущества? Вам не нужно вручную добавлять эти тесты в списковые файлы, чтобы сообщить о них CMake. Забудьте о команде `add_test()`, она вам больше не понадобится. Catch2 автоматически зарегистрирует ваши тесты в CTest, если вы это разрешите. Добавить каркас нетрудно после того, как проект сконфигурирован, как было описано в предыдущем разделе. Воспользуйтесь командой `FetchContent()`, чтобы включить его в свой проект.

Можно выбрать одну из двух основных версий: Catch2 v2 и Catch2 v3. Версия 2 представляет собой библиотеку для C++11, реализованную в виде одного заголовочного файла. Версия 3 компилируется как статическая библиотека и требует C++14. Рекомендуется работать с последней версией.

При работе с Catch2 выберите тег Git и закрепите его в своем списковом файле. Не гарантируется, что обновление через ветвь `main` пройдет без проблем.



В реальной обстановке вы, скорее всего, будете прогонять тесты в составе конвейера CI. В таких случаях не забывайте настроить окружение, так чтобы зависимости уже присутствовали в системе и не приходилось скачивать их при каждой сборке. Как было сказано в разделе «Использование уже установленных зависимостей» главы 9, вы должны будете включить в команду `FetchContent_Declare()` ключевое слово `FIND_PACKAGE_ARGS`, чтобы использовать пакеты, уже установленные в системе.

Включим в наш списковый файл версию 3.4.0.

ch11/03-catch2/test/CMakeLists.txt

```
include(FetchContent)
FetchContent_Declare(
  Catch2
  GIT_REPOSITORY https://github.com/catchorg/Catch2.git
  GIT_TAG v3.4.0
)
FetchContent_MakeAvailable(Catch2)
```

После этого мы должны определить цель `unit_tests` и скомпоновать ее с `sut`, с предоставленной каркасом точкой входа и с библиотекой `Catch2::Catch2WithMain`. Поскольку Catch2 предоставляет собственную функцию `main()`, нам больше нет нужды использовать файл `unit_tests.cpp` (его можно удалить). Код приведен ниже.

ch11/03-catch2/test/CMakeLists.txt (продолжение)

```
add_executable(unit_tests calc_test.cpp run_test.cpp)
target_link_libraries(unit_tests PRIVATE
                      sut Catch2::Catch2WithMain)
```

Наконец, мы используем команду `catch_discover_tests()`, которая определена в модуле, входящем в состав Catch2, чтобы автоматически находить все тесты из `unit_tests` и регистрировать их в CTest.

ch11/03-catch2/test/CMakeLists.txt (продолжение)

```
list(APPEND CMAKE_MODULE_PATH ${catch2_SOURCE_DIR}/extras)
include(Catch)
catch_discover_tests(unit_tests)
```

Готово. Мы только что добавили каркас автономного тестирования в свое решение. Посмотрим, как это выглядит на практике. Ниже приведен вывод исполнителя тестов:

```
# ./test/unit_tests
unit_tests is a Catch2 v3.4.0 host application.
Run with -? for options
-------------------------------------------------------------------
MultiplyMultipliesTwoInts
-------------------------------------------------------------------
/root/examples/ch11/03-catch2/test/calc_test.cpp:9
...................................................................
/root/examples/ch11/03-catch2/test/calc_test.cpp:11: FAILED:
  CHECK( 12 == sut.Multiply(3, 4) )
with expansion:
  12 == 9
```

```
===============================================================
test cases: 3 | 2 passed | 1 failed
assertions: 3 | 2 passed | 1 failed
```

Catch2 определил, что выражение `sut.Multiply(3, 4)` равно `9`, и дал нам дополнительный контекст, очень полезный для отладки.

Заметим, что прямое выполнение двоичного файла исполнителя (откомпилированного файла `unit_test`), наверное, было бы немного быстрее, чем с помощью `ctest`, но дополнительные преимущества, предлагаемые CTest, стоят этого небольшого замедления.

Мы рассмотрели начальную настройку Catch2. Если в будущем вам понадобятся дополнительные тесты, просто создайте новые файлы реализации и добавьте пути к ним в список исходных файлов для цели `unit_tests`.

Каркас Catch2 предлагает такие средства, как прослушиватели событий, генераторы данных и микротесты производительности, но встроенных подставных объектов в нем нет. Мы будем рассматривать подставные объекты в следующем разделе. Впрочем, эту возможность можно добавить в Catch2 с помощью одного из следующих каркасов:

- FakeIt (https://github.com/eranpeer/FakeIt);
- Hippomocks (https://github.com/dascandy/hippomocks);
- Trompeloeil (https://github.com/rollbear/trompeloeil).

Однако более прямой и функционально богатый путь предлагает другой каркас – GoogleTest.

GoogleTest

У применения каркаса GoogleTest есть несколько важных преимуществ: он существует уже долго и снискал признание сообществом C++, поэтому несколько IDE поддерживают его «из коробки». Компания, стоящая за самой большой в мире поисковой системой, сопровождает его и активно использует в собственных разработках, поэтому маловероятно, что в один прекрасный день он устареет или будет заброшен. Он умеет тестировать код для C++11 и более поздних версий, что, безусловно, порадует тех, кто работает в устаревшем окружении.

В репозитории GoogleTest размещено два проекта: GTest (основной каркас тестирования) и GMock (библиотека, добавляющая функциональность подставных объектов). Это значит, что их можно скачать одним вызовом `FetchContent()`.

Работа с GTest

Чтобы можно было использовать GTest, проект должен следовать указаниям из раздела «Структурирование проектов для тестирования». Вот как пишется автономный тест для этого каркаса.

ch11/04-gtest/test/calc_test.cpp

```
#include <gtest/gtest.h>
#include "calc.h"

class CalcTestSuite : public ::testing::Test {
protected:
  Calc sut_;
};

TEST_F(CalcTestSuite, SumAddsTwoInts) {
  EXPECT_EQ(4, sut_.Sum(2, 2));
}

TEST_F(CalcTestSuite, MultiplyMultipliesTwoInts) {
  EXPECT_EQ(12, sut_.Multiply(3, 4));
}
```

Поскольку этот пример будет использоваться также в GMock, я решил поместить тесты в один класс `CalcTestSuite`. Комплекты тестов группируют взаимосвязанные тесты, так что они могут использовать одни и те же поля, методы, а также шаги инициализации и очистки. Чтобы создать комплект тестов, объявите новый класс, наследующий `::testing::Test`, и поместите повторно используемые элементы в его защищенную секцию.

Каждый тест в комплекте объявляется макросом `TEST_F()`. Для индивидуальных тестов имеется более простой макрос `TEST()`. Поскольку мы определили в классе член `Calc sut_`, каждый тест может обратиться к нему, как если бы тесты были методами `CalcTestSuite`. На самом деле каждый тест исполняется в своем собственном экземпляре, который наследует `CalcTestSuite`, потому-то ключевое слово `protected` необходимо. Отметим, что повторно используемые поля не предназначены для разделения данных между соседними тестами; их цель – сделать код удовлетворяющим принципу DRY (не повторяйся).

GTest не предлагает естественного синтаксиса для утверждений, как Catch2. Вместо этого используются такие сравнения, как `EXPECT_EQ()`. По соглашению, первым идет ожидаемое значение, а за ним фактическое. Есть много других типов утверждений, вспомогательных функций и макросов. Подробную информацию о GTest см. в официальной справочной документации (https://google.github.io/googletest/).

Чтобы добавить эту зависимость в проект, мы должны определиться с версией. В отличие от Catch2, GoogleTest склоняется к философии «жизни на острие» (заимствованной из проекта Abseil, от которого зависит GTest). Она провозглашает: «*Если вы собираете нашу зависимость из исходных кодов и следуете нашему API, то никаких проблем у вас быть не должно*». (Дополнительные сведения см. в разделе «Для дополнительного чтения».) Если это правило вас не устраивает (но сборка из исходного кода не составляет проблемы), установите свой тег Git на ветвь `master`. В противном случае выберите версию из репозитория GoogleTest.



В реальной обстановке вы, скорее всего, будете прогонять тесты в составе конвейера CI. В таких случаях не забывайте настроить окружение, так чтобы зависимости уже присутствовали в системе и не приходилось скачивать их при каждой сборке. Как было сказано в разделе «Использование уже установленных зависимостей» главы 9, вы должны будете включить в команду `FetchContent_Declare()` ключевое слово `FIND_PACKAGE_ARGS`, чтобы использовать пакеты, уже установленные в системе.

В любом случае добавление зависимости от GTest выглядит так:

ch11/04-gtest/test/CMakeLists.txt

```
include(FetchContent)
FetchContent_Declare(
  googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
  GIT_TAG v1.14.0
)
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)
```

Мы следуем тому же методу, как в случае Catch2, – выполнить `FetchContent()` и собрать каркас из исходных файлов. Единственная разница – добавление команды `set(gtest...)`, как рекомендуют авторы GoogleTest, чтобы предотвратить переопределение настроек компилятора и компоновщика из родительского проекта в Windows.

Наконец, мы можем объявить исполняемый файл нашего исполнителя тестов, скомпоновать его с `gtest_main`, и тогда наши тесты будут обнаруживаться автоматически благодаря встроенному в CMake модулю `GoogleTest`.

ch11/04-gtest/test/CMakeLists.txt (продолжение)

```
add_executable(unit_tests
               calc_test.cpp
               run_test.cpp)
target_link_libraries(unit_tests PRIVATE sut gtest_main)
include(GoogleTest)
gtest_discover_tests(unit_tests)
```

На этом и завершается инициализация GTest. Вывод исполнителя тестов, запущенного напрямую, гораздо подробнее, чем у Catch2, но мы можем передать параметр `--gtest_brief=1`, чтобы включать в него только сведения о непрошедших тестах:

```
# ./test/unit_tests --gtest_brief=1
~/examples/ch11/04-gtest/test/calc_test.cpp:15: Failure
Expected equality of these values:
  12
```

```
  sut_.Multiply(3, 4)
    Which is: 9
[ FAILED ] CalcTestSuite.MultiplyMultipliesTwoInts (0 ms)
[==========] 3 tests from 2 test suites ran. (0 ms total)
[ PASSED ] 2 tests.
```

К счастью, даже шумный вывод подавляется при запуске из CTest (если только мы явно не разрешим его, задав в командной строке `ctest` флаг `--output-on-failure`).

Теперь, когда каркас занял свое место, давайте обсудим подставные объекты. Ведь никакой тест не может быть по-настоящему «автономным», если он тесно связан с другими элементами.

GMock

Автономные тесты – это фрагменты кода, исполняемые независимо от других частей программы. Такая тестируемая единица должна быть замкнутым элементом: классом или компонентом. Конечно, едва ли найдется программа на C++, в которой все единицы идеально изолированы.

Скорее всего, ваш код будет сильно зависеть от какой-то формы ассоциации между классами. И тут возникает проблема: объекты одного класса требуют объектов другого класса, те – объектов третьего класса и т. д. И не успели мы оглянуться, как вся программа принимает участие в «автономном тесте». Хуже того, ваш код может быть связан с внешней системой и зависеть от ее состояния. Например, он может зависеть от конкретных записей в базе данных, от входящих сетевых пакетов или от файлов, хранящихся на диске.

Чтобы разорвать связи между единицами на этапе тестирования, разработчики применяют **тестовые двойники**, или специальные версии классов, которые используются только тестируемой единицей. Примерами могут служить подделки (fake), заглушки (stub) и подставки (mock). Приведем нестрогие определения этих терминов.

- **Подделкой** (поддельным объектом) называется ограниченная реализация более сложного механизма. Пример – отображение в памяти вместо настоящего клиента базы данных.
- **Заглушка** предоставляет конкретные, зашитые в код ответы на вызовы методов, ограничиваясь теми, что используются в тестах. Она также может запоминать, какие методы вызывались и сколько раз.
- **Подставка** (подставной объект) – более развитая версия заглушки. Она может дополнительно проверять, вызывались ли методы во время тестирования так, как ожидается.

Такой тестовый двойник создается в начале теста и передается в качестве аргумента конструктору тестируемого класса для использования вместо реального объекта. Этот механизм называется **внедрением зависимости**.

Проблема простых тестовых двойников в том, что они *слишком просты*. Чтобы смоделировать поведение в разных сценариях, мы должны были бы написать много разных двойников, по одному для каждого состояния, в котором может находиться связанный объект. Это не очень практично и при-

вело бы к размазыванию кода по слишком многим файлам. Тут-то и вступает в дело GMock: он позволяет создать обобщенный тестовый двойник для определенного класса и определить его поведение для каждого теста отдельно. В GMock эти двойники называются подставками, но в действительности это смесь всех вышеупомянутых разновидностей двойников, проявляющаяся так или иначе в зависимости от обстоятельств.

Рассмотрим пример. Добавим в класс `Calc` возможность прибавлять случайное число к заданному аргументу. Она будет представлена методом `AddRandomNumber()`, который возвращает сумму в виде `int`. Как подтвердить, что возвращенное значение действительно является суммой аргумента и чего-то случайного? Как мы знаем, случайно сгенерированные числа – ключ ко многим важным процессам, и если они используются некорректно, то последствия могут быть катастрофическими. Проверять случайные числа одно за другим, пока не будут исчерпаны все возможности, практически нереально.

Для тестирования мы должны обернуть генератор случайных чисел классом, который можно было бы подставить (т. е. заменить подставным объектом). Подставки могут вернуть конкретный ответ, который используется для «подделывания» генерации случайного числа. `Calc` воспользуется этим значением в `AddRandomNumber()` и позволит нам проверить, отвечает ли значение, возвращенное методом, нашим ожиданиям. Четкое отделение генерации случайного числа от другой единицы и является добавленной ценностью (поскольку мы можем заменить один генератор другим).

Начнем с открытого интерфейса абстрактного генератора. Этот заголовочный файл позволит реализовать как фактический генератор, так и подставку и использовать одно вместо другого.

ch11/05-gmock/src/rng.h

```
#pragma once
class RandomNumberGenerator {
public:
  virtual int Get() = 0;
  virtual ~RandomNumberGenerator() = default;
};
```

Классы, реализующие этот интерфейс, возвращают случайное число методом `Get()`. Обратите внимание на ключевое слово `virtual` – оно должно присутствовать во всех подставляемых методах, если только мы не хотим связываться с более сложным механизмом подставки на основе шаблонов. Также нужно не забыть о виртуальном деструкторе.

Далее мы должны расширить класс `Calc`, чтобы он принимал и сохранял генератор; тогда мы сможем передать его настоящий генератор для выпускной сборки или подставку для тестов.

ch11/05-gmock/src/calc.h

```
#pragma once
#include "rng.h"
class Calc {
```

```
  RandomNumberGenerator* rng_;
public:
  Calc(RandomNumberGenerator* rng);
  int Sum(int a, int b);
  int Multiply(int a, int b);
  int AddRandomNumber(int a);
};
```

Мы включили заголовочный файл и добавили метод для прибавления случайного числа. Кроме того, добавлено поле для хранения указателя на созданный генератор и параметризованный конструктор. Так внедрение зависимости работает на практике. Теперь реализуем эти методы.

ch11/05-gmock/src/calc.cpp

```
#include "calc.h"
Calc::Calc(RandomNumberGenerator* rng) {
  rng_ = rng;
}
int Calc::Sum(int a, int b) {
  return a + b;
}
int Calc::Multiply(int a, int b) {
  return a * b; // now corrected
}
int Calc::AddRandomNumber(int a) {
  return a + rng_->Get();
}
```

В конструкторе мы записываем переданный указатель в поле класса. Затем это поле используется в методе `AddRandomNumber()` для получения сгенерированного значения. В производственном коде будет использоваться настоящий генератор случайных чисел, а в тестах – подставной. Напомним, что для получения плодов полиморфизма указатели нужно разыменовывать. В качестве бонуса мы могли бы создавать разные классы генераторов для различных реализаций. Мне нужен только один: вихрь Мерсенна с равномерным распределением, как показано в следующем фрагменте.

ch11/05-gmock/src/rng_mt19937.cpp

```
#include <random>
#include "rng_mt19937.h"
int RandomNumberGeneratorMt19937::Get() {
  std::random_device rd;
  std::mt19937 gen(rd());
  std::uniform_int_distribution<> distrib(1, 6);
  return distrib(gen);
}
```

Создавать новый экземпляр при каждом вызове не особенно эффективно, но для такого простого примера этого достаточно. Мы хотим, чтобы класс генерировал числа от 1 до 6 и возвращал их вызывающей стороне.

В заголовочном файле для этого класса имеется только сигнатура одного метода.

ch11/05-gmock/src/rng_mt19937.h

```
#include "rng.h"
class RandomNumberGeneratorMt19937
      : public RandomNumberGenerator {
public:
  int Get() override;
};
```

А вот как он используется в производственном коде:

ch11/05-gmock/src/run.cpp

```
#include <iostream>
#include "calc.h"
#include "rng_mt19937.h"
using namespace std;
int run() {
  auto rng = new RandomNumberGeneratorMt19937();
  Calc c(rng);
  cout << "Random dice throw + 1 = "
       << c.AddRandomNumber(1) << endl;
  delete rng;
  return 0;
}
```

Мы создали генератор и передали указатель на него конструктору `Calc`. Все готово к тому, чтобы написать подставку. Стремясь хорошо организовать код, разработчики обычно помещают подставки в отдельный каталог `test/mocks`. Во избежание неоднозначности имя заголовочного файла кончается суффиксом `_mock`. Ниже приведен код.

ch11/05-gmock/test/mocks/rng_mock.h

```
#pragma once
#include "gmock/gmock.h"
class RandomNumberGeneratorMock : public
RandomNumberGenerator {
public:
  MOCK_METHOD(int, Get, (), (override));
};
```

После добавления заголовка `gmock.h` мы можем объявить свою подставку. Как и планировалось, это класс, реализующий интерфейс `RandomNumberGenerator`. Вместо того чтобы писать методы самостоятельно, мы должны использовать макросы `MOCK_METHOD`, предоставляемые GMock. Они информируют каркас о том, какие методы интерфейса следует подставлять. Используйте следующий формат (все круглые скобки обязательны):

```
MOCK_METHOD(<return type>, <method name>,
            (<argument list>), (<keywords>))
```

Мы готовы воспользоваться подставкой в своем комплекте тестов (предыдущие тесты для краткости опущены):

ch11/05-gmock/test/calc_test.cpp

```
#include <gtest/gtest.h>
#include "calc.h"
#include "mocks/rng_mock.h"
using namespace ::testing;
class CalcTestSuite : public Test {
protected:
  RandomNumberGeneratorMock rng_mock_;
  Calc sut_{&rng_mock_};
};
TEST_F(CalcTestSuite, AddRandomNumberAddsThree) {
  EXPECT_CALL(rng_mock_, Get()).Times(1).WillOnce(Return(3));
  EXPECT_EQ(4, sut_.AddRandomNumber(1));
}
```

Рассмотрим изменения по отдельности. Мы добавили новый заголовок и создали поле `rng_mock_`. Далее адрес подставки передается конструктору `sut_`. Это можно сделать, потому что поля инициализируются в порядке объявления (`rng_mock_` предшествует `sut_`).

В тесте мы вызываем макрос `EXPECT_CALL`, предоставляемый GMock, для метода `Get()` объекта `rng_mock_`. Это инструктирует каркас провалить тест, если метод `Get()` не будет вызван во время выполнения. Сцепленный вызов `Times` явно говорит, сколько вызовов должно быть произведено, чтобы тест считался успешным. Метод `WillOnce` определяет, что делает каркас подставных объектов после вызова метода (возвращает `3`).

Благодаря использованию GMock мы смогли выразить подставное поведение наряду с ожидаемым исходом. Это повышает как удобочитаемость тестов, так и удобство их сопровождения. Но еще важнее то, что обеспечивается гибкость тестов, потому что мы смогли показать, что происходит, с помощью одного очень выразительного предложения.

Наконец, чтобы собрать проект, нужно скомпоновать библиотеку `gmock` с исполнителем тестов. Для этого добавим ее в список командой `target_link_libraries()`:

ch11/05-gmock/test/CMakeLists.txt

```
include(FetchContent)
FetchContent_Declare(
  googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
  GIT_TAG release-1.14.0
)
# Для Windows: предотвратить переопределение параметров компилятора
# и компоновщика в родительском проекте
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
FetchContent_MakeAvailable(googletest)
add_executable(unit_tests
               calc_test.cpp
               run_test.cpp)
target_link_libraries(unit_tests PRIVATE sut gtest_main gmock)
include(GoogleTest)
gtest_discover_tests(unit_tests)
```

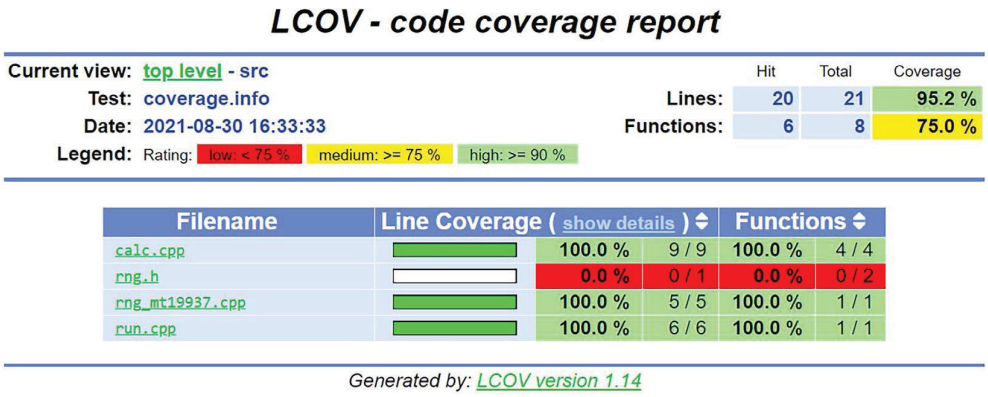
Теперь мы можем насладиться всеми преимуществами каркаса GoogleTest. И GTest, и GMock – весьма развитые инструменты со множеством концепций, утилит и вспомогательных классов для различных ситуаций. Приведенный пример (хотя и довольно длинный) дает лишь поверхностное представление о возможностях. Я призываю вас включать эти интерфейсы в свои проекты, поскольку они значительно повышают качество вашей работы. В качестве отправной точки для изучения GMock начните со страницы «Mocking for Dummies» в официальной документации (ссылка на нее имеется в разделе «Для дополнительного чтения»).

Располагая тестами, мы должны как-то измерить, что протестировано, а что нет, и попытаться улучшить ситуацию. Лучше всего использовать автоматизированные инструменты, которые собирают эту информацию и генерируют отчеты.

Генерирование отчетов о покрытии тестами

Добавить тесты к такому небольшому решению не бог весть как трудно. Настоящие трудности появляются, когда программа и подлиннее, и посложнее. С годами я понял, что по мере приближения к 1000 строк кода становится все труднее отслеживать, какие строки и ветви выполнены тестами, а какие – нет. После пересечения порога в 3000 строк это становится практически невозможным. А в большинстве профессиональных приложений кода намного больше. И при этом одной из ключевых метрик, которой многие руководители пользуются на совещаниях по оценке технического долга, является процент покрытия кода тестами, поэтому важно знать, как сгенерировать

полезные отчеты, содержащие данные для таких обсуждений. Чтобы справиться с этой проблемой, мы можем воспользоваться утилитой, показывающей, какие строки кода «покрыты» тестами. Такой инструмент вклинивается в SUT и собирает информацию об исполнении каждой строки тестами, а затем представляет в виде удобного отчета, показанного ниже.



LCOV - code coverage report

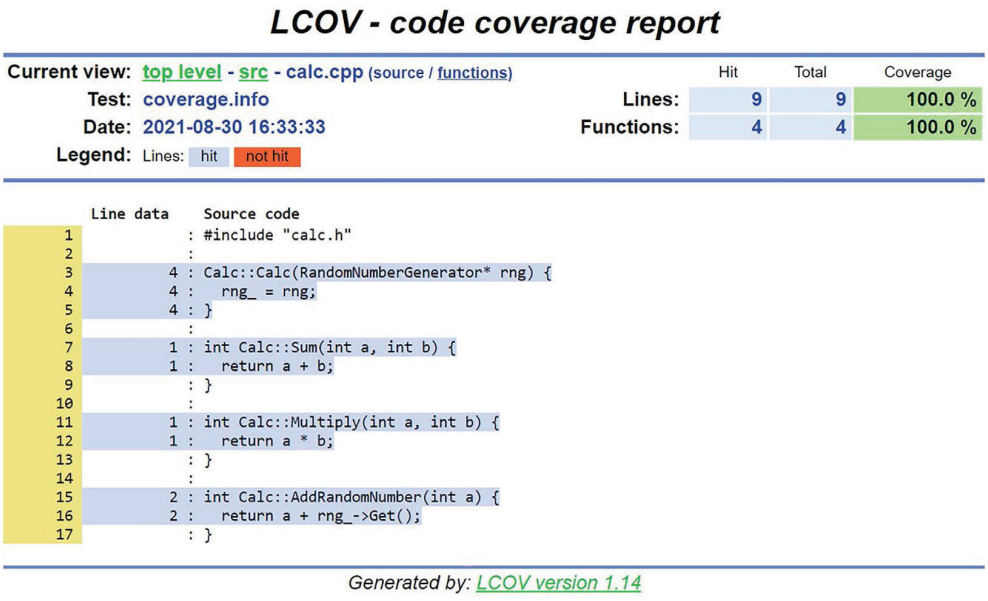
			Hit	Total	Coverage
Current view:	top level - src	**Lines:**	20	21	95.2 %
Test:	coverage.info	**Functions:**	6	8	75.0 %
Date:	2021-08-30 16:33:33				
Legend:	Rating: low: < 75 % medium: >= 75 % high: >= 90 %				

Filename	Line Coverage (show details)		Functions	
calc.cpp	100.0 %	9 / 9	100.0 %	4 / 4
rng.h	0.0 %	0 / 1	0.0 %	0 / 2
rng_mt19937.cpp	100.0 %	5 / 5	100.0 %	1 / 1
run.cpp	100.0 %	6 / 6	100.0 %	1 / 1

Generated by: LCOV version 1.14

Рис. 11.2. Отчет о покрытии тестами, сгенерированными LCOV

Эти отчеты показывают, какие файлы покрыты тестами, а какие – нет. Более того, можно заглянуть внутрь файла и точно узнать, какие строки кода выполнялись и сколько раз. На рисунке ниже столбец **Line data** сообщает, что конструктор `Calc` был выполнен 4 раза, по одному для каждого теста.



LCOV - code coverage report

			Hit	Total	Coverage
Current view:	top level - src - calc.cpp (source / functions)	**Lines:**	9	9	100.0 %
Test:	coverage.info	**Functions:**	4	4	100.0 %
Date:	2021-08-30 16:33:33				
Legend:	Lines: hit not hit				

```
      Line data    Source code
    1            : #include "calc.h"
    2            :
    3          4 : Calc::Calc(RandomNumberGenerator* rng) {
    4          4 :   rng_ = rng;
    5          4 : }
    6            :
    7          1 : int Calc::Sum(int a, int b) {
    8          1 :   return a + b;
    9            : }
   10            :
   11          1 : int Calc::Multiply(int a, int b) {
   12          1 :   return a * b;
   13            : }
   14            :
   15          2 : int Calc::AddRandomNumber(int a) {
   16          2 :   return a + rng_->Get();
   17            : }
```

Generated by: LCOV version 1.14

Рис. 11.3. Подробный отчет о покрытии

Существует много способов генерирования таких отчетов. Они зависят от платформы и компилятора, но устроены похоже: подготовить SUT к измерению, провести измерение и подготовить отчет.

Самый простой инструмент такого рода называется **LCOV**. Это не акроним, а графический интерфейс к `gcov`, утилите измерения покрытия из комплекта **GNU Compiler Collection** (**GCC**). Посмотрим, как он используется на практике.

Применение LCOV для генерирования отчетов о покрытии

LCOV генерирует отчеты о покрытии в формате HTML, используя `gcov` для измерения покрытия. Тем, кто предпочитает Clang, волноваться тоже не стоит – Clang поддерживает метрики в этом формате. Скачать LCOV можно из официального репозитория, поддерживаемого в рамках проекта **Linux Test Project** (https://github.com/linuxtest-project/lcov), или просто воспользоваться менеджером пакетов. Как следует из названия, эта утилита ориентирована на Linux. Ее можно запустить в macOS, но платформа Windows не поддерживается. Конечным пользователям покрытие тестами редко бывает интересно, поэтому обычно достаточно установить LCOV вручную в своей среде сборки и не включать ее в проект.

Для измерения покрытия нужно сделать следующее.

1. Откомпилировать в конфигурации `Debug` с флагами компилятора, разрешающими измерение покрытия тестами. При этом будут сгенерированы файлы с информацией о покрытии (с расширением `.gcno`).
2. Скомпоновать тестовый исполняемый файл с библиотекой `gcov`.
3. Собрать эталонные метрики покрытия, без прогона тестов.
4. Прогнать тесты. При этом создаются файлы с данными о покрытии (с расширением `.gcda`).
5. Собрать метрики в агрегированный информационный файл.
6. Сгенерировать отчет (с расширением `.html`).

Для начала нужно объяснить, почему код следует компилировать в конфигурации `Debug`. Самая важная причина – потому что в этой конфигурации запрещается любая оптимизация, т. к. задан флаг `-O0`. CMake делает это по умолчанию благодаря переменной `CMAKE_CXX_FLAGS_DEBUG` (хотя в документации это нигде не упомянуто). Если вы не станете переопределять эту переменную, то отладочная сборка не будет подвергнута оптимизации. Это желательно, чтобы предотвратить встраивание и другие неявные способы упрощения кода. В противном случае было бы трудно понять, с какой строкой исходного кода связана данная машинная команда.

На первом шаге мы должны проинструктировать компилятор добавить необходимое оснащение в SUT. Точный флаг зависит от компилятора, но два основных (GCC и Clang) предлагают для этой цели флаг `--coverage`, благодаря чему данные порождаются в совместимом с GCC формате `gcov`.

Вот так добавляется оснащение для измерения покрытия в нашу SUT из предыдущего раздела.

ch11/06-coverage/src/CMakeLists.txt

```
add_library(sut STATIC calc.cpp run.cpp rng_mt19937.cpp)
target_include_directories(sut PUBLIC .)
if (CMAKE_BUILD_TYPE STREQUAL Debug)
  target_compile_options(sut PRIVATE --coverage)
  target_link_options(sut PUBLIC --coverage)
  add_custom_command(TARGET sut PRE_BUILD COMMAND
                     find ${CMAKE_BINARY_DIR} -type f
                     -name '*.gcda' -exec rm {} +)
endif()
add_executable(bootstrap bootstrap.cpp)
target_link_libraries(bootstrap PRIVATE sut)
```

Разберем этот код по частям.

1. Убедиться, что мы работаем в конфигурации `Debug` с помощью команды `if(STREQUAL)`. Напомним, что никаких сведений о покрытии вы не получите, если `cmake` не была запущена с параметром `-DCMAKE_BUILD_TYPE=Debug`.
2. Добавить `--coverage` в состав частных (`PRIVATE`) *параметров компиляции* для всех *объектных файлов*, являющихся частью библиотеки `sut`.
3. Добавить `--coverage` в состав публичных (`PUBLIC`) параметров компоновщика: и GCC, и Clang интерпретируют это как требование скомпоновать библиотеку `gcov` (или совместимую с ней) со всеми целями, зависящими от `sut` (благодаря распространяемым свойствам).
4. Добавить команду `add_custom_command()`, которая удаляет старые `gcda`-файлы. Зачем это нужно, подробно обсуждается в разделе «Как избежать SEGFAULT».

Этого достаточно для измерения покрытия кода. Если вы пользуетесь такой IDE, как CLion, то сможете прогнать свои автономные тесты и получить результаты во встроенном представлении. Однако это не будет работать в автоматизированном конвейере, являющемся частью вашей инфраструктуры CI/CD. Для получения отчетов мы должны сгенерировать их самостоятельно с помощью LCOV.

Для этого лучше всего определить новую цель `coverage`. Чтобы код оставался чистым, мы напишем отдельную функцию, `AddCoverage`, которая будет использоваться в списковом файле `test`.

ch11/06-coverage/cmake/Coverage.cmake

```
function(AddCoverage target)
  find_program(LCOV_PATH lcov REQUIRED)
  find_program(GENHTML_PATH genhtml REQUIRED)
  add_custom_target(coverage
    COMMENT "Running coverage for ${target}..."
```

```
    COMMAND ${LCOV_PATH} -d . --zerocounters
    COMMAND $<TARGET_FILE:${target}>
    COMMAND ${LCOV_PATH} -d . --capture -o coverage.info
    COMMAND ${LCOV_PATH} -r coverage.info '/usr/include/*'
                         -o filtered.info
  COMMAND ${GENHTML_PATH} -o coverage filtered.info --legend
  COMMAND rm -rf coverage.info filtered.info
  WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
)
endfunction()
```

Здесь мы сначала находим пути к `lcov` и `genhtml` (две командные программы из пакета LCOV). Ключевое слово `REQUIRED` говорит CMake, что нужно вернуть ошибку, если эти файлы не найдены. Затем мы добавляем специальную цель `coverage`, состоящую из следующих шагов.

1. Стереть счетчики, оставшиеся от предыдущих запусков.
2. Запустить исполняемый файл `target` (используя генераторные выражения для получения пути к нему). `$<TARGET_FILE:target>` – исключительное генераторное выражение, в данном случае оно неявно добавит зависимость от `target`, заставив собрать ее перед выполнением всех команд. Мы передаем `target` этой функции в виде аргумента.
3. Собрать метрики для решения из текущего каталога (`-d .`) и вывести в файл (`-o coverage.info`).
4. Удалить (`-r`) ненужные данные о покрытии системных заголовков (`'/usr/include/*'`) и поместить результат в другой файл (`-o filtered.info`).
5. Сгенерировать HTML-отчет в каталоге `coverage` и добавить цвет `--legend`.
6. Удалить временные `info`-файлы.
7. Задание ключевого слова `WORKING_DIRECTORY` назначает дерево двоичных файлов рабочим каталогом для всех команд.

Все это общие шаги для GCC и Clang. Важно, чтобы версия `gcov` соответствовала версии компилятора: нельзя использовать `gcov` из GCC для кода, откомпилированного Clang. Чтобы указать `lcov` на `gcov` из Clang, можно использовать аргумент `--gcov-tool`. Единственная проблема заключается в том, что это должен быть один исполняемый файл. Для ее решения мы можем написать простой обертывающий скрипт (не забудьте сделать его исполняемым командой `chmod +x`):

```
# cmake/gcov-llvm-wrapper.sh
#!/bin/bash
exec llvm-cov gcov "$@"
```

Это будет означать, что все обращения к `${LCOV_PATH}` в предыдущей функции получат следующий флаг:

```
--gcov-tool ${CMAKE_SOURCE_DIR}/cmake/gcov-llvm-wrapper.sh
```

Убедитесь, что эта функция доступна для включения в списковый файл `test`. Это можно сделать, расширив *путь поиска включаемых файлов* в главном списковом файле следующим образом:

ch11/06-coverage/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26.0)
project(Coverage CXX)
include(CTest)
list(APPEND CMAKE_MODULE_PATH "${CMAKE_SOURCE_DIR}/cmake")
add_subdirectory(src bin)
add_subdirectory(test)
```

Выделенная строка позволяет включать все `cmake`-файлы из каталога `cmake` в наш проект. Теперь мы можем использовать файл `Coverage.cmake` в списковом файле `test`:

ch11/06-coverage/test/CMakeLists.txt (fragment)

```
# ... объявление цели unit_tests для краткости опущено
include(Coverage)
AddCoverage(unit_tests)
include(GoogleTest)
gtest_discover_tests(unit_tests)
```

Для сборки цели `coverage` используйте следующие команды (отметим, что первая команда заканчивается параметром выбора типа сборки `-DCMAKE_BUILD_TYPE=Debug`):

```
# cmake -B <binary_tree> -S <source_tree> -DCMAKE_BUILD_TYPE=Debug
# cmake --build <binary_tree> -t coverage
```

После выполнения всех вышеупомянутых шагов появится краткая сводка:

```
Writing directory view page.
Overall coverage rate:
  lines......: 95.7% (22 of 23 lines)
  functions..: 75.0% (6 of 8 functions)
[100%] Built target coverage
```

Затем откройте файл `coverage/index.html` в браузере и наслаждайтесь отчетами! Есть, правда, одна мелкая проблема...

Как избежать SEGFAULT

Мы можем попасть в беду, если станем редактировать исходные файлы в собранном решении. Дело в том, что информация о покрытии состоит из двух частей:

- gcno-файлы (**GNU Coverage Notes**), генерируемые во время компиляции SUT;
- gcda-файлы (**GNU Coverage Data**), генерируемые и **обновляемые** во время прогона тестов.

Вот это «обновление» и есть потенциальная причина ошибок сегментации. После первого прогона тестов остается кучка gcda-файлов, которые никогда не удаляются. Если внести изменения в исходный код и перекомпилировать *объектные файлы*, то будут созданы новые gcno-файлы. Но шага стирания нет – gcda-файлы, оставшиеся от предыдущего прогона, устарели. Когда мы выполняем двоичный файл unit_tests (это происходит в макросе gtest_discover_tests), файлы с данными о покрытии не соответствуют друг другу, и возникает ошибка сегментации SEGFAULT.

Чтобы избежать этой проблемы, нужно стирать устаревшие gcda-файлы. Поскольку наш экземпляр sut – статическая библиотека, можно вставить команду add_custom_command(TARGET) в состав событий сборки. Очистка будет произведена до начала пересборки.

Ссылки на дополнительные сведения см. в разделе «Для дополнительного чтения».

Резюме

На первый взгляд может показаться, что сложности, связанные с надлежащим тестированием, настолько велики, что не стоит тратить силы на их преодоление. Вы удивитесь, узнав, сколько программ работает вообще без всяких тестов под предлогом, что тестирование кода – устрашающее предприятие. Добавлю: это еще мягко сказано, если все делать вручную. Но, к сожалению, без строгого автоматизированного тестирования узнать об имеющихся в коде ошибках трудно или даже невозможно. Быть может, непротестированный код писать быстрее (и то не всегда), но на его чтение, рефакторинг и исправление точно уйдет гораздо больше времени.

В этой главе мы привели несколько причин для написания тестов с самого начала. Одна из самых убедительных – душевный покой и хороший сон по ночам. Сколько разработчиков ложатся в постель с мыслью «ох, только бы не разбудили через пару часов, чтобы гасить пожар на производстве и исправлять ошибки». А если серьезно, то вылавливание ошибок до того, как они попали в производственную программу, может стать спасением для вас (и для компании).

Что до утилит тестирования, то тут CMake демонстрирует всю свою мощь. CTest может творить чудеса при обнаружении сбойных тестов: изоляция, тасование, повтор, тайм-ауты. Все эти возможности чрезвычайно удобны и доступны с помощью флагов командной строки. Мы научились применять CTest для вывода списка тестов, их фильтрации и управления выводом тестов, но самое главное – теперь мы знаем истинную мощь адаптации стандартного решения к любым обстоятельствам. Всякий проект, собранный с помощью CMake, может быть протестирован единообразно, не вдаваясь в его внутренние детали.

Далее мы структурировали проект, чтобы упростить процесс тестирования и повторно использовать одни и те же объектные файлы как в производственном коде, так и в исполнителе тестов. Было интересно написать свой собственный исполнитель тестов, но, пожалуй, все-таки стоит сосредоточиться на той задаче, которую должна решать наша программа, а время потратить на изучение и использование какого-нибудь популярного стороннего каркаса тестирования.

Мы очень кратко рассмотрели два таких каркаса: Catch2 и GoogleTest. Затем мы углубились в детали библиотеки GMock и поняли, как работают тестовые двойники, благодаря которым возможно писать автономные тесты. Наконец, мы настроили отчеты с помощью LCOV. Ведь нет ничего лучше, чем неоспоримые данные, доказывающие, что наше решение действительно протестировано полностью.

В следующей главе мы обсудим дополнительные инструменты для улучшения качества кода и обнаружения проблем, о которых мы и не подозревали.

Для дополнительного чтения

Дополнительные сведения по рассмотренным в этой главе темам можно получить из следующих источников:

- документация по CMake – CTest: https://cmake.org/cmake/help/latest/manual/ctest.1.html;
- документация по Catch2: https://github.com/catchorg/Catch2/blob/devel/docs/;
- пособие по GMock: https://google.github.io/googletest/gmock_for_dummies.html;
- Abseil: https://abseil.io/;
- жизнь на острие в Abseil: https://abseil.io/about/philosophy#we-recommend-that-you-choose-to-live-at-head;
- почему Abseil стал зависимостью для GTest: https://github.com/google/googletest/issues/2883;
- покрытие в GCC: https://gcc.gnu.org/onlinedocs/gcc/Instrumentation-Options.html; https://gcc.gnu.org/onlinedocs/gcc/Invoking-Gcov.html; https://gcc.gnu.org/onlinedocs/gcc/Gcov-Data-Files.html;
- покрытие в Clang: https://clang.llvm.org/docs/SourceBasedCodeCoverage.html;
- документация по командным инструментам LCOV: https://helpmanual.io/man1/lcov/;
- репозиторий проекта LCOV: https://github.com/linux-test-project/lcov;
- функциональность обновления в GCOV: https://gcc.gnu.org/onlinedocs/gcc/Invoking-Gcov.html#Invoking-Gcov.

12

Инструменты анализа программ

Создание высококачественного кода – непростая задача, даже для самых опытных разработчиков. Включая в решение тесты, мы уменьшаем шансы оставить в коде простые ошибки. Но чтобы избежать более сложных проблем, этого недостаточно. Любая программа состоит из стольких деталей, что для их отслеживания нужно целый рабочий день только этим и заниматься. Группы, отвечающие за сопровождение продукта, придумали различные соглашения и практики проектирования.

Есть вопросы, связанные со стилем кодирования: какова должна быть длина строки кода – 80 или 120 столбцов? Следует ли разрешить `std::bind` или ограничиться лямбда-функциями? Разрешено ли использовать массивы в стиле C? Следует ли записывать небольшие функции в одну строку? Нужно ли писать слово `auto` всегда или только когда это делает программу понятнее? В идеале мы должны держаться подальше от конструкций, заведомо некорректных в общем случае: бесконечных циклов, использования идентификаторов, зарезервированных стандартной библиотекой, непреднамеренной потери данных, ненужных предложений `if` и вообще всего, что не является «лучшей практикой» (см. раздел «Для дополнительного чтения»).

Еще один аспект, который нужно учитывать, – модернизация кода. По мере развития в C++ появляются новые возможности. Нахождение всех мест, которые можно привести в соответствие с последним стандартом, – занятие не из простых. А если делать это вручную, то уйдет много времени и появится риск внесения ошибок, особенно в большой кодовой базе. Наконец, необходимо проверить, как все работает в движении: запустить программу и исследовать ее память. Всегда ли память правильно освобождается после использования? Правильно ли инициализированы данные, к которым мы обращаемся? Не пытается ли код обратиться по уже не существующему указателю? Разрешение всех этих проблем занимает время и чревато ошибками. По счастью, мы можем использовать автоматизированные инструменты для проверки соблюдения правил, исправления ошибок и актуализации кода.

Настало время обсудить средства анализа программ. Наш код будет подвергаться скрупулезным проверкам при каждой сборке, чтобы гарантировать следование стандартам.

В этой главе будут рассмотрены следующие вопросы:

- принудительное форматирование;
- использование статических проверщиков;
- динамический анализ с помощью Valgrind.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch12.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Принудительное форматирование

Профессиональные разработчики обычно соблюдают правила. Говорят, что старшие разработчики знают, когда их можно нарушить, потому что могут обосновать необходимость. С другой стороны, самые старшие разработчики часто воздерживаются от нарушения правил, чтобы не тратить времени на объяснение своих решений. Главное – сосредоточить внимание на вопросах, которые действительно влияют на продукт, а не увлекаться мелкими деталями.

Когда речь заходит о стиле кодирования и форматировании, разработчик сталкивается со множеством вариантов. Использовать для отступов пробелы или знаки табуляции? Если пробелы, то сколько именно? Какова максимальная длина строки или файла? Все это не изменяет поведения программы, но может приводить к бесконечным дискуссиям, которые мало что дают.

Существуют распространенные практики, но спорят часто о личных предпочтениях и случаях из жизни. Так, выбор длины строки – 80 или 120 символов – произволен. Гораздо важнее придерживаться раз выбранного стиля, потому что несогласованность мешает восприятию кода. Для обеспечения

единообразия рекомендуется использовать инструмент форматирования, например `clang-format`. Он сообщит, что код отформатирован неправильно, и даже внесет исправления. Вот пример команды форматирования кода:

```
clang-format -i --style=LLVM filename1.cpp filename2.cpp
```

Флаг `-i` инструктирует `clang-format` редактировать файлы на месте, тогда как `--style` задает стиль форматирования, например `LLVM`, `Google`, `Chromium`, `Mozilla`, `WebKit` или специальный, описанный в файле (см. раздел «Для дополнительного чтения»).

Конечно, мы не хотим выполнять эту команду вручную при внесении каждого изменения; `CMake` должна организовать это как часть процесса сборки. Мы уже знаем, как найти в системе программу `clang-format` (ее необходимо предварительно установить). А вот чего мы еще не знаем, так это как применить этот внешний инструмент ко всем нашим исходным файлам. Для этого мы напишем вспомогательную функцию, которую можно включить из каталога `cmake`.

ch12/01-formatting/cmake/Format.cmake

```
function(Format target directory)
  find_program(CLANG-FORMAT_PATH clang-format REQUIRED)
  set(EXPRESSION h hpp hh c cc cxx cpp)
  list(TRANSFORM EXPRESSION PREPEND "${directory}/*.")
  file(GLOB_RECURSE SOURCE_FILES FOLLOW_SYMLINKS
       LIST_DIRECTORIES false ${EXPRESSION}
  )
  add_custom_command(TARGET ${target} PRE_BUILD COMMAND
  ${CLANG-FORMAT_PATH} -i --style=file ${SOURCE_FILES}
)
endfunction()
```

Функция `Format` принимает два аргумента: `target` и `directory`. Она форматирует все исходные файлы из каталога непосредственно перед сборкой цели.

Технически не все файлы в каталоге должны принадлежать цели, а с другой стороны, исходные файлы, принадлежащие цели, могут быть распределены по нескольким каталогам. Однако прослеживать все исходные и заголовочные файлы, относящиеся к цели, сложно, особенно если нужно исключить заголовки из внешних библиотек. В этом случае проще сосредоточиться на каталогах, чем на логических целях. Мы можем вызывать эту функцию для каждого каталога, где есть файлы, нуждающиеся в форматировании.

Функция выполняет следующие действия.

1. Найти установленный двоичный файл `clang-format`. Ключевое слово `REQUIRED` остановит конфигурирование с ошибкой, если файл не будет найден.
2. Создать список расширений файлов, подлежащих форматированию (он будет использоваться в качестве **маски**).

3. Дописать в начало каждого выражения путь к каталогу `directory`.
4. Произвести рекурсивный поиск исходных и заголовочных файлов (используя ранее созданный список), поместить найденные пути к файлам в переменную `SOURCE_FILES` (но пропустить пути к каталогам).
5. Присоединить команду форматирования к шагу `PRE_BUILD` цели `target`.

Этот подход хорошо работает для кодовых баз малого и среднего размера. Для более крупных кодовых баз, возможно, понадобится преобразовать абсолютные пути в относительные и запустить команду форматирования, используя `directory` в качестве рабочего каталога. Это может оказаться необходимым из-за ограничения на длину команд в оболочке – обычно в районе 13 000 знаков.

Посмотрим, как эта функция используется на практике. Проект имеет следующую структуру:

```
- CMakeLists.txt
- .clang-format
- cmake
  |- Format.cmake
- src
  |- CMakeLists.txt
  |- header.h
  |- main.cpp
```

Сначала мы инициализируем проект и добавляем каталог `cmake` в путь к модулям для включения впоследствии:

ch12/01-formatting/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Formatting CXX)
enable_testing()
list(APPEND CMAKE_MODULE_PATH "${CMAKE_SOURCE_DIR}/cmake")
add_subdirectory(src bin)
```

Затем заполняем списковый файл для каталога `src`:

ch12/01-formatting/src/CMakeLists.txt

```
add_executable(main main.cpp)
include(Format)
Format(main .)
```

Здесь нет никаких хитростей. Мы создаем исполняемую цель `main`, включаем модуль `Format.cmake` и вызываем функцию `Format()` для цели `main` в текущем каталоге (`src`).

Теперь нужно завести какие-то неотформатированные исходные файлы. Заголовок содержит простую функцию `unused`:

ch12/01-formatting/src/header.h

```
int unused() { return 2 + 2; }
```

Мы также включим исходный файл с изрядным количеством расставленных невпопад пробелов:

ch12/01-formatting/src/main.cpp

```
#include <iostream>
                                using namespace std;
                        int main() {
      cout << "Hello, world!" << endl;
}
```

Почти все готово. Нужен только конфигурационный файл форматера, который указывается с помощью аргумента командной строки `--style=file`.

ch12/01-formatting/.clang-format

```
BasedOnStyle: Google
ColumnLimit: 140
UseTab: Never
AllowShortLoopsOnASingleLine: false
AllowShortFunctionsOnASingleLine: false
AllowShortIfStatementsOnASingleLine: false
```

Программа `clang-format` будет искать в родительских каталогах файл `.clang-format`, в котором определены точные правила форматирования. Это позволит нам настроить все детали. Я начал со стиля форматирования Google и внес в него несколько изменений: максимальная длина строки 140 символов, запрет табуляторов, запрет размещения коротких циклов, функций и предложений `if` в одной строке.

После сборки проекта (форматирование производится автоматически перед компиляцией) наши файлы будут выглядеть так:

ch12/01-formatting/src/header.h (отформатированный)

```
int unused() {
  return 2 + 2;
}
```

Заголовочный файл был отформатирован, хотя он и не используется целью. Короткие функции не могут располагаться в одной строке, поэтому, как и следовало ожидать, добавлены знаки новой строки. Файл `main.cpp` тоже выглядит аккуратно. Лишние пробелы убраны, а отступы приведены в соответствие со стандартом.

ch12/01-formatting/src/main.cpp (отформатированный)

```
#include <iostream>
using namespace std;
```

```
int main() {
  cout << "Hello, world!" << endl;
}
```

Автоматизация форматирования экономит время в процессе инспекции кода. Если вам когда-нибудь доводилось исправлять фиксацию только из-за неправильно расставленных пробелов, то вы понимаете, какой груз упал с ваших плеч. Единообразное форматирование позволяет поддерживать чистоту кода без всяких усилий.



Применение форматирования ко всей кодовой базе, скорее всего, закончится крупномасштабным, хотя и одноразовым, изменением большинства файлов в репозитории и может привести к *огромному* количеству конфликтов объединения, если вы (или ваши коллеги) что-то в это время делали. Лучше координировать такие действия и совершать их после того, как все ожидающие изменения зафиксированы. Если это невозможно, подумайте о постепенном переходе, скажем по одному каталогу за раз. Ваши коллеги это оценят.

Хотя форматер отлично справляется с приведением кода к единообразному виду, это не полноценный инструмент анализа программ. Для более сложных задач нужны другие утилиты, предназначенные для статического анализа.

Использование статических проверщиков

Статический анализ программ подразумевает исследование исходного кода без выполнения его откомпилированной версии. Последовательное использование статических проверщиков может значительно улучшить качество кода, сделав его более единообразным и менее подверженным ошибкам и известным уязвимостям. Сообщество C++ предлагает широкий круг таких средств, например `Astrée`, `clang-tidy`, `CLazy`, `CMetrics`, `Cppcheck`, `Cpplint`, `CQMetrics`, `ESBMC`, `FlawFinder`, `Flint`, `IKOS`, `Joern`, `PC-Lint`, `Scan-Build`, `Vera++` и другие.

Многие из этих инструментов признают `CMake` как промышленный стандарт и предлагают готовую поддержку или пособия по интеграции. Некоторые инженеры по сборке предпочитают не писать код для `CMake`, а включать статические проверщики с помощью внешних модулей, доступных в сети. Примером может служить подборка Ларс Бильке в его репозитории на GitHub: https://github.com/bilke/cmake-modules.

Многие думают, что настроить статический проверщик сложно. Такое мнение связано с тем, что статические проверщики часто эмулируют поведение настоящего компилятора, чтобы понять код. Но вовсе необязательно это трудно.

В руководстве по Cppcheck перечислены следующие простые шаги.

1. Найти исполняемый файл статического проверщика.
2. Сгенерировать *базу данных компиляции* следующей командой:
   - `cmake -DCMAKE_EXPORT_COMPILE_COMMANDS=ON`.
3. Выполнить проверщик, воспользовавшись сгенерированным JSON-файлом:
   - `<path-to-cppcheck> --project=compile_commands.json`

Эти шаги должны быть интегрированы в процесс сборки, чтобы случайно не забыть про них.

Поскольку CMake знает, как собирать наши цели, может ли она также поддержать любые статические проверщики? Безусловно, и это даже проще, чем может показаться. CMake позволяет активировать следующие проверщики для каждой цели в отдельности:

- `include-what-you-use` (https://include-what-you-use.org);
- `clang-tidy` (https://clang.llvm.org/extra/clang-tidy);
- `Link What You Use` (встроенный в CMake проверщик);
- `Cpplint` (https://github.com/cpplint/cpplint);
- `Cppcheck` (https://cppcheck.sourceforge.io).

Чтобы активировать проверщик, присвойте свойству цели список, содержащий путь к исполняемому файлу проверщика и передаваемые ему параметры командной строки (элементы списка должны быть разделены точкой с запятой):

- `<LANG>_CLANG_TIDY`;
- `<LANG>_CPPCHECK`;
- `<LANG>_CPPLINT`;
- `<LANG>_INCLUDE_WHAT_YOU_USE`;
- `LINK_WHAT_YOU_USE`.

Замените `<LANG>` на `C` для исходных файлов, написанных на C, и на `CXX` – для файлов, написанных на C++. Если вы хотите активировать проверщик для всех целей проекта, то установите глобальную переменную с префиксом `CMAKE_`, например:

```
set(CMAKE_CXX_CLANG_TIDY /usr/bin/clang-tidy-3.9;-checks=*)
```

Для любой цели, определенной после этого предложения, свойство `CXX_CLANG_TIDY` будет равно этому значению. Помните, что включение статического анализа может немного увеличить время сборки. С другой стороны, может быть полезен более точный контроль над тем, как проверщик тестирует цели. Мы можем написать для этого прямолинейную функцию:

ch12/02-clang-tidy/cmake/ClangTidy.cmake

```
function(AddClangTidy target)
  find_program(CLANG-TIDY_PATH clang-tidy REQUIRED)
  set_target_properties(${target}
```

```
    PROPERTIES CXX_CLANG_TIDY
    "${CLANG-TIDY_PATH};-checks=*;--warnings-as-errors=*"
  )
endfunction()
```

Функция `AddClangTidy` состоит из двух шагов.

1. Найти двоичный файл `clang-tidy` и сохранить путь к нему в переменной `CLANG-TIDY_PATH`. Ключевое слово `REQUIRED` гарантирует, что конфигурирование остановится с ошибкой, если двоичный файл не найден.
2. Активировать `clang-tidy` для цели, указав путь к двоичному файлу и параметры, которые включают все проверки и трактуют предупреждения как ошибки.

Для использования этой функции нужно только включить модуль и вызвать его для выбранной цели.

ch12/02-clang-tidy/src/CMakeLists.txt

```
add_library(sut STATIC calc.cpp run.cpp)
target_include_directories(sut PUBLIC .)
add_executable(bootstrap bootstrap.cpp)
target_link_libraries(bootstrap PRIVATE sut)
include(ClangTidy)
AddClangTidy(sut)
```

Этот подход краткий и очень эффективный. При сборке решения вывод `clang-tidy` будет выглядеть так:

```
[ 6%] Building CXX object bin/CMakeFiles/sut.dir/calc.cpp.o
/root/examples/ch12/04-clang-tidy/src/calc.cpp:3:11: warning: method 'Sum'
can be made static [readability-convert-member-functions-to-static]
int Calc::Sum(int a, int b) {
          ^
[ 12%] Building CXX object bin/CMakeFiles/sut.dir/run.cpp.o
/root/examples/ch12/04-clang-tidy/src/run.cpp:1:1: warning: #includes are
not sorted properly [llvm-include-order]
#include <iostream>
^        ~~~~~~~~~~
/root/examples/ch12/04-clang-tidy/src/run.cpp:3:1: warning: do not use
namespace using-directives; use using-declarations instead [google-buildusing-
namespace]
using namespace std;
^
/root/examples/ch12/04-clang-tidy/src/run.cpp:6:3: warning: initializing
non-owner 'Calc *' with a newly created 'gsl::owner<>' [cppcoreguidelinesowning-
memory]
auto c = new Calc();
^
```

Заметим, что если не добавлять аргумент командной строки `--warnings-as-errors=*`, то сборка завершится успехом. Организация должна выработать набор правил, которые необходимо строго соблюдать, чтобы в репозиторий не попал код, не следующий соглашениям.

`clang-tidy` предлагает также параметр `--fix`, который автоматически исправляет код там, где это возможно. Это экономит время и особенно полезно, когда список проверок расширяется. Как и в случае форматирования, остерегайтесь конфликтов объединения при включении изменений, произведенных статическими проверщиками в существующих кодовых базах.

В зависимости от ситуации, размера репозитория и предпочтений команды выберите небольшое число проверщиков, отвечающих вашим потребностям. Слишком широкое разнообразие может нанести вред. Ниже приведен краткий обзор проверщиков, поддерживаемых CMake «из коробки».

clang-tidy

Вот что говорит о clang-tidy официальный сайт:

> *clang-tidy – основанный на clang «линтер» для C++. Его назначение – предоставить расширяемый каркас для диагностики и исправления типичных ошибок программирования, таких как нарушение стилистических правил или неправильное использование интерфейсов, и ошибок, которые могут быть обнаружены средствами статического анализа. clang-tidy обладает модульной структурой и предлагает удобный интерфейс для написания новых проверок.*

Инструмент весьма разносторонний, включено более 400 проверок. Он хорошо работает в связке с `ClangFormat`, позволяя автоматически вносить исправления (доступно более 150) с сохранением того же формата файла. Спектр проверок покрывает производительность, удобочитаемость, модернизацию, наставления по кодированию на C++ и области, особенно уязвимые для ошибок.

Cpplint

Ниже приведено описание `Cpplint` с официального сайта:

> *Cpplint – инструмент командной строки для проверки отсутствия в файлах на C/C++ расхождений с руководством по стилю для C++ от Google. Cpplint разработан и поддерживается компанией Google Inc. в рамках проекта google/styleguide.*

Цель этого линтера – привести ваш код в соответствие с руководством Google по стилю. Он написан на Python, поэтому может добавить в проект

нежелательную зависимость. Исправления предлагаются в форматах, понятных программам `Emacs`, `Eclipse`, `VS7`, `Junit` и `sed`.

Cppcheck

Вот что говорит о `Cppcheck` официальный сайт:

> *Cppcheck – инструмент статического анализа кода на C/C++. Предлагает уникальные средства анализа для обнаружения ошибок с упором на неопределенное поведение и опасные конструкции. Цель – минимизация ложноположительных срабатываний. Cppcheck спроектирован так, что может анализировать ваш код на C/C++, даже если в нем используется нестандартный синтаксис (что часто встречается в проектах для встраиваемых систем).*

Благодаря минимизации ложноположительных срабатываний этот инструмент является надежным помощником при анализе кода. Он существует уже больше 14 лет и все еще активно сопровождается. Особенно полезен, если ваш код несовместим с Clang.

include-what-you-use

Ниже приведено описание include-what-you-use с официального сайта.

> *Основная цель программы include-what-you-use – убрать избыточные директивы #include. Для этого она определяет, какие #include не нужны в данном файле (с расширением .cc или .h), и заменяет #include опережающим объявлением там, где это возможно.*

Слишком большое число включаемых заголовков может показаться не такой уж важной проблемой в небольшом проекте, но по мере его разрастания время, сэкономленное в результате отказа от ненужной компиляции, быстро становится заметным.

Link What You Use

Ниже приведено описание «Link what you use» из блога CMake:

> *Это встроенная в CMake возможность, в которой параметры ld и ldd используются для информирования о том, что с исполняемым файлом компонуется больше библиотек, чем реально необходимо.*

Статический анализ играет важную роль в таких отраслях промышленности, как медицина, атомная энергетика, авиация, автомобилестроение и машиностроение, где программные ошибки могут угрожать жизни. Разумные разработчики применяют эту методику и в менее ответственных областях, особенно если затраты невелики. Статический анализ не только делает процесс сборки более экономичным, чем при ручном поиске дефектов, но и легко сочетается с CMake. Рискну даже утверждать, что нет почти никаких причин отказываться от этих проверок в любом программном продукте, пекущемся о качестве, а это любой продукт, которым пользуется кто-то, кроме самого разработчика.

Эта техника также позволяет сократить время сборки за счет устранения лишних двоичных артефактов. Увы, не все ошибки можно найти, не запуская программу. Но, к счастью, можно предпринять дополнительные шаги, чтобы лучше понять, как работает программа, например воспользоваться `Valgrind`.

Динамический анализ с помощью Valgrind

`Valgrind` (https://www.valgrind.org) – каркас оснащения кода для *nix инструментальными средствами с целью динамического анализа, выполняемого во время работы программы. В его состав входит широкий спектр разнообразных инструментов исследования и проверки, например:

- `Memcheck`: обнаруживает проблемы с управлением памятью;
- `Cachegrind`: профилирует процессорные кеши и идентифицирует непопадание в кеш и другие проблемы;
- `Callgrind`: расширение `Cachegrind`, которое предоставляет дополнительную информацию о графах вызовов;
- `Massif`: профилировщик кучи, который показывает, как различные части программы используют кучу во время работы;
- `Helgrind`: отладчик потоков, ищущий гонки за данные;
- `DRD`: облегченная и более ограниченная версия `Helgrind`.

Любой инструмент из этого списка полезен в соответствующей ситуации. Большинство менеджеров пакетов знают о `Valgrind` и могут установить его в вашу систему. В Linux он, скорее всего, уже установлен. Кроме того, на официальном сайте имеется исходный код для тех, кто предпочитает собирать программу самостоятельно.

Мы в своем обсуждении ограничимся в основном `Memcheck`, самым часто применяемым инструментом из комплекта `Valgrind` (когда разработчики упоминают `Valgrind`, они обычно имеют в виду именно `Memcheck`). Мы рассмотрим, как его использовать в сочетании с CMake, и это упростит освоение других входящих в комплект инструментов, если впоследствии вы сочтете это необходимым.

Memcheck

Программа Memcheck бесценна для отладки проблем при работе с памятью, особенно сложных в C++. У программистов имеется полный контроль над управлением памятью, из-за чего становятся возможны разнообразные ошибки: от чтения невыделенной или уже освобожденной памяти до многократного освобождения памяти и даже записи по неправильным адресам. Такие ошибки легко могут остаться незамеченными даже в самых простых программах. Иногда, чтобы попасть в беду, достаточно забыть об инициализации всего лишь одной переменной.

Memcheck вызывается следующим образом:

```
valgrind [valgrind-options] tested-binary [binary-options]
```

Memcheck – инструмент Valgrind, подразумеваемый по умолчанию, но можно указать его и явно:

```
valgrind --tool=memcheck tested-binary
```

Выполнение Memcheck может значительно замедлить работу программы; в руководстве (см. раздел «Для дополнительного чтения») сказано, что оснащенная программа может работать в 10–15 раз медленнее. Чтобы не ждать Valgrind при каждом прогоне тестов, мы можем создать отдельную цель, которая будет вызываться из командной строки, когда мы захотим протестировать свой код. В идеале это нужно делать перед всяким включением нового кода в основную кодовую базу. Этот шаг можно вставить в одну из ранних точек вклинивания Git или в состав конвейера **непрерывной интеграции** (**CI**).

Чтобы создать специальную цель для Valgrind, можно использовать следующую команду после этапа генерирования CMake:

```
cmake --build <build-tree> -t valgrind
```

Вот как добавляется соответствующая цель в CMake:

ch12/03-valgrind/cmake/Valgrind.cmake

```
function(AddValgrind target)
  find_program(VALGRIND_PATH valgrind REQUIRED)
  add_custom_target(valgrind
    COMMAND ${VALGRIND_PATH} --leak-check=yes
            $<TARGET_FILE:${target}>
    WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
  )
endfunction()
```

В этом примере мы определяем функцию AddValgrind, которая принимает подлежащую тестированию цель (мы сможем использовать ее в разных проектах). Эта функция делает две вещи.

1. CMake проверяет подразумеваемые по умолчанию системные пути в поисках исполняемого файла `valgrind` и сохраняет путь к нему в переменной `VALGRIND_PATH`. Благодаря ключевому слову `REQUIRED` конфигурирование будет прервано с ошибкой, если двоичный файл не найден.
2. Создается специальная цель `valgrind`. Она запускает `Memcheck` для указанного двоичного файла с параметром, задающим поиск утечек памяти.

Параметры `Valgrind` можно задать несколькими способами:

- в файле `~/.valgrindrc` (в домашнем каталоге);
- в переменной окружения `$VALGRIND_OPTS`;
- в файле `./.valgrindrc` (в рабочем каталоге).

Эти файлы проверяются в указанном порядке. Кроме того, заметим, что последний файл рассматривается, только если он принадлежит текущему пользователю, является регулярным и не имеет прав записи для всех. Это механизм защиты, потому что параметры `Valgrind` таковы, что может быть причинен вред.

Для использования функции `AddValgrind` мы передаем ей цель `unit_tests`, потому что хотим выполнять в строго контролируемом окружении, таком как автономные тесты.

ch12/03-valgrind/test/CMakeLists.txt (фрагмент)

```
# ...
add_executable(unit_tests calc_test.cpp run_test.cpp)
# ...
include(Valgrind)
AddValgrind(unit_tests)
```

Напомним, что генерирование деревьев сборки в конфигурации `Debug` позволяет `Valgrind` получить доступ к отладочной информации, благодаря чему ее вывод становится гораздо чище.

Взгляните, как это работает на практике.

```
# cmake -B <build tree> -S <source tree> -DCMAKE_BUILD_TYPE=Debug
# cmake --build <build-tree> -t valgrind
```

Здесь проект конфигурируется, собираются цели `sut` и `unit_tests` и запускается выполнение `Memcheck`, что дает нам общую информацию:

```
[100%] Built target unit_tests
==954== Memcheck, a memory error detector
==954== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==954== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==954== Command: ./unit_tests
```

Префикс `==954==` содержит идентификатор процесса, чтобы отличить комментарии `Valgrind` от вывода тестируемого процесса.

Далее прогоняем тесты как обычно с помощью `gtest`:

```
[==========] Running 3 tests from 2 test suites.
[----------] Global test environment set-up.
...
[==========] 3 tests from 2 test suites ran. (42 ms total)
[ PASSED ] 3 tests.
```

В конце печатается сводная информация:

```
==954==
==954== HEAP SUMMARY:
==954== in use at exit: 1 bytes in 1 blocks
==954== total heap usage: 209 allocs, 208 frees, 115,555 bytes allocated
```

Вот те раз! Остался по меньшей мере один неосвобожденный байт. Не каждому выделению памяти с помощью `malloc()` и `new` соответствует освобождение посредством `free()` и `delete`. Похоже, в программе имеется утечка памяти. `Valgrind` предоставляет более подробные сведения для ее нахождения:

```
==954== 1 bytes in 1 blocks are definitely lost in loss record 1 of 1
==954== at 0x483BE63: operator new(unsigned long) (in /usr/lib/x86_64-
linux-gnu/valgrind/vgpreload_memcheck-amd64-linux.so)
==954== by 0x114FC5: run() (run.cpp:6)
==954== by 0x1142B9: RunTest_RunOutputsCorrectEquations_
Test::TestBody() (run_test.cpp:14)
```

Строки, начинающиеся с `0x<адрес>`, соответствуют отдельным функциям в стеке вызовов. Я сократил вывод (в нем есть не имеющие отношения к делу строки, напечатанные `GTest`), чтобы сосредоточиться на самом интересном – самой верхней функции `run()` и ссылке на ее исходный код (`run.cpp:6`):

Внизу мы видим сводную информацию:

```
==954== LEAK SUMMARY:
==954==    definitely lost: 1 bytes in 1 blocks
==954==    indirectly lost: 0 bytes in 0 blocks
==954==      possibly lost: 0 bytes in 0 blocks
==954==    still reachable: 0 bytes in 0 blocks
==954==         suppressed: 0 bytes in 0 blocks
==954==
==954== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

`Valgrind` блестяще справляется с поиском сложных проблем. Иногда она заглядывает даже глубже и находит проблемы, которые трудно отнести к какой-то определенной категории. Они показаны в строке «`possibly lost`».

Посмотрим, какая проблема была найдена `Memcheck` в этом случае.

ch12/03-valgrind/src/run.cpp

```
#include <iostream>
#include "calc.h"
using namespace std;
int run() {
  auto c = new Calc();
  cout << "2 + 2 = " << c->Sum(2, 2) << endl;
  cout << "3 * 3 = " << c->Multiply(3, 3) << endl;
  return 0;
}
```

Все правильно: выделенный код действительно содержит ошибку. Мы создали объект, который не был удален по окончании теста. Именно по этой причине более полное покрытие тестами так важно.

`Valgrind` – ценный инструмент, но для сложных программ в его выводе нелегко разобраться. Существует более эффективный способ управления этой информацией – проект `Memcheck-Cover`.

Memcheck-Cover

Коммерческие IDE типа CLion умеют непосредственно разбирать вывод `Valgrind`, благодаря чему навигация в графическом интерфейсе оказывается проще, чем прокрутка на консоли. Если ваш редактор не располагает такой возможностью, то может помочь сторонний генератор отчетов. `Memcheck-Cover`, созданный Дэвидом Гарсином, упрощает работу путем создания HTML-файла, показанного на рис. 12.1.

Этот небольшой проект доступен на GitHub (https://github.com/Farigh/memcheck-cover); для его работы нужны `Valgrind` и `gawk` (программа GNU AWK). Чтобы воспользоваться им, напишем функцию подготовки в отдельном модуле CMake. Она состоит из двух частей:

1) скачивание и конфигурирование инструмента;
2) добавление специальной цели для запуска `Valgrind` и генерирование отчета.

Вот как производится конфигурирование.

ch12/04-memcheck/cmake/Memcheck.cmake

```
function(AddMemcheck target)
  include(FetchContent)
  FetchContent_Declare(
    memcheck-cover
    GIT_REPOSITORY https://github.com/Farigh/memcheck-cover.git
    GIT_TAG        release-1.2
  )
  FetchContent_MakeAvailable(memcheck-cover)
  set(MEMCHECK_PATH ${memcheck-cover_SOURCE_DIR}/bin)
```

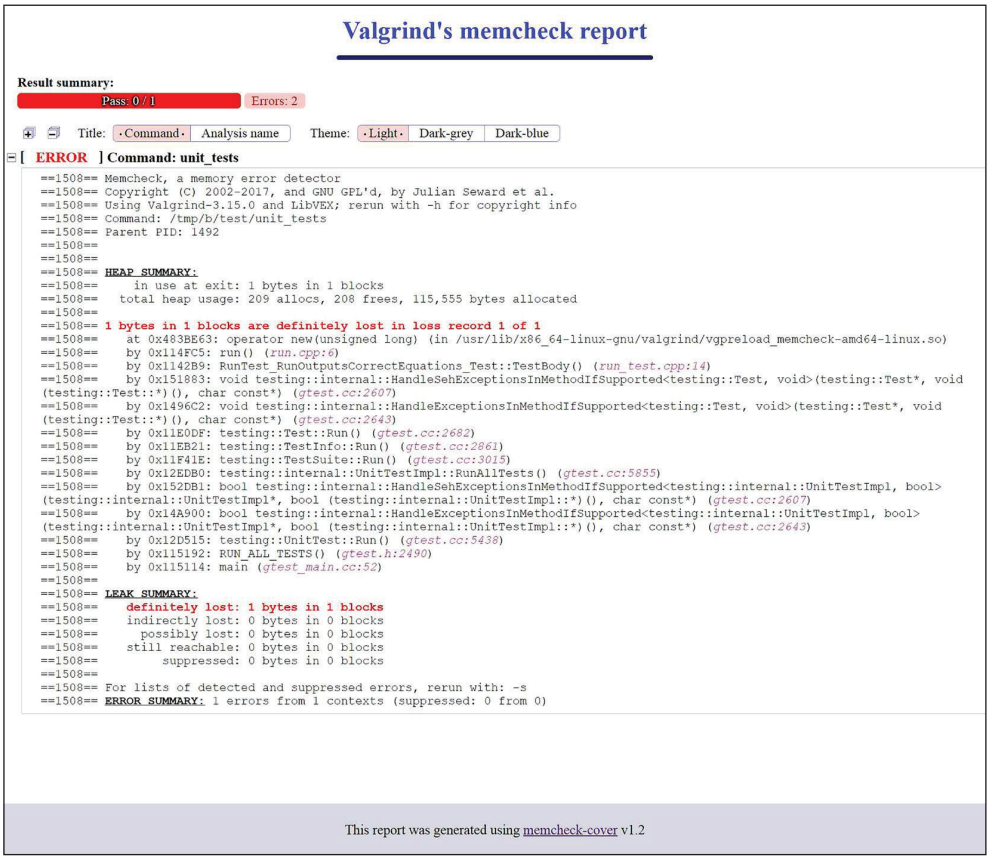


Рис. 12.1. Отчет, сгенерированный Memcheck-Cover

В первой части мы делаем то же, что для обычной зависимости: включаем модуль `FetchContent` и указываем репозиторий проекта и тег Git с помощью функции `FetchContent_Declare`. Затем мы инициируем процесс скачивания и конфигурируем путь к двоичному файлу, используя переменную `memcheck-cover_SOURCE_DIR`, установленную функцией `FetchContent_Populate` (которая вызывается из `FetchContent_MakeAvailable`).

Во второй части функции создается цель для генерирования отчетов. Мы назвали ее `memcheck` (чтобы избежать конфликта с определенной ранее целью `valgrind` на случай, если мы почему-то захотим оставить обе).

ch12/04-memcheck/cmake/Memcheck.cmake (продолжение)

```
add_custom_target(memcheck
  COMMAND ${MEMCHECK_PATH}/memcheck_runner.sh -o
    "${CMAKE_BINARY_DIR}/valgrind/report"
    -- $<TARGET_FILE:${target}>
  COMMAND ${MEMCHECK_PATH}/generate_html_report.sh
    -i "${CMAKE_BINARY_DIR}/valgrind"
```

```
    -o "${CMAKE_BINARY_DIR}/valgrind"
  WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
)
endfunction()
```

Для этого используются две команды.

1. Сначала запускается обертывающий скрипт `memcheck_runner.sh`, который выполняет `Memcheck` из комплекта `Valgrind` и перенаправляет вывод в файл, указанный в аргументе `-o`.
2. Затем мы разбираем вывод и создаем отчет скриптом `generate_html_report.sh`, для которого в аргументах `-i` и `-o` задаются входной и выходной каталоги соответственно.

Оба шага следует выполнять в рабочем каталоге `CMAKE_BINARY_DIR`, чтобы исполняемый автономный тест мог обратиться к файлам по относительным путям.

И последний шаг – добавление вызова этой функции в списковый файл.

ch12/04-memcheck/test/CMakeLists.txt (фрагмент)

```
include(Memcheck)
AddMemcheck(unit_tests)
```

После генерирования сборочной системы в конфигурации `Debug` мы можем собрать цель следующим образом:

```
# cmake -B <build tree> -S <source tree> -DCMAKE_BUILD_TYPE=Debug
# cmake --build <build-tree> -t memcheck
```

И полюбоваться отчетом, отформатированным в виде HTML-страницы.

Резюме

«Вы тратите больше времени на чтение кода, чем на его написание, поэтому отдавайте предпочтение удобству чтения, а не написания». Этот принцип часто цитируется в книгах по чистому коду. Он подкреплен опытом многих разработчиков ПО, потому-то даже такие мелкие детали, как число пробелов, разбиение на строки и порядок предложений `#import`, стандартизируются. И это не мелочная педантичность, а забота об экономии времени. Следуя практикам, описанным в этой главе, вы сможете забыть о ручном форматировании кода. Программа `ClangFormat` гарантирует, что код будет отформатирован в соответствии с выбранным вами стандартом.

Но помимо организации пробелов код должен отвечать многим другим рекомендациям. Тут-то и приходит на помощь clang-tidy. Она помогает сделать законом соглашения, принятые вашей командой или организацией. Мы подробно обсудили этот статический проверщик, а также коснулись других

программ, в т. ч. `Cpplint`, `Cppcheck`, `include-what-you-use` и `Link What You Use`. Поскольку статические проверщики работают сравнительно быстро, мы можем включить их в сборку без особых затрат, и обычно это окупается.

Мы также рассмотрели утилиты, входящие в комплект `Valgrind`, уделив особое внимание `Memcheck`, которая ищет проблемы, связанные с управлением памятью, например некорректное чтение и запись. Этот инструмент бесценен, потому что позволяет избежать многих часов ручной отладки и не допустить проникновения ошибок в производственную среду. Мы показали, как сделать вывод `Valgrind` более дружелюбным с помощью генератора HTML-отчетов `Memcheck-Cover`. Особенно это полезно, если окружение не позволяет работать с IDE, например в конвейерах CI.

Эта глава – только начало. Есть много других инструментов, бесплатных и коммерческих, которые помогут повысить качество кода. Изучите их и выберите наилучший для себя. В следующей главе мы поговорим о генерировании документации.

Для дополнительного чтения

Дополнительные сведения по рассмотренным в этой главе темам можно получить из следующих источников:

- основные принципы C++, курируемые Бьярне Страуструпом, автором C++: https://github.com/isocpp/CppCoreGuidelines;
- справочное руководство по `ClangFormat`: https://clang.llvm.org/docs/ClangFormat.html;
- статические анализаторы для C++ – тщательно подобранный список: https://github.com/analysis-tools-dev/static-analysis#cpp;
- встроенная поддержка статического анализа в CMake: https://blog.kitware.com/static-checkswith-cmake-cdash-iwyu-clang-tidy-lwyu-cpplint-and-cppcheck/;
- свойство цели, активирующее `clang-tidy`: https://cmake.org/cmake/help/latest/prop_tgt/LANG_CLANG_TIDY.html;
- руководство по `Valgrind`: https://www.valgrind.org/docs/manual/manual-core.html.

13

Генерирование документации

Высококачественный код не просто хорошо написан, протестирован и работает – он еще и тщательно документирован. Документация позволяет поделиться информацией, которая в противном случае могла бы быть утрачена, представить общую картину, описать контекст, раскрыть намерения и, наконец, передать знания внешним пользователям и сопровождающим.

Помните, как вы в последний раз вошли в новый проект и потратили много часов, чтобы разобраться в нагромождении каталогов и файлов? Этого можно избежать. По-настоящему качественная документация способна привести новичка к нужной ему строке кода за секунды. Печально, но на проблему отсутствия документации часто не обращают внимания. Неудивительно – ведь ее составление требует недюжинных навыков, которыми не все мы обладаем. К тому же документация имеет свойство быстро устаревать. Если не реализована строгая процедура обновления и инспекции, то легко забыть о том, что и документация тоже нуждается во внимании.

Некоторые коллективы (ради экономии времени или по настоянию руководства) практикуют написание *самодокументированного кода*. Выбирая осмысленные, удобные для восприятия имена файлов, функций, переменных и т. д., они надеются избежать трудностей, сопутствующих написанию документации. Но даже самая лучшая сигнатура функции не передает всей информации – например, сигнатура `int removeDuplicates();` описывает назначение, но не говорит, что именно возвращается. Это может быть число найденных дубликатов, число оставшихся элементов или что-то совсем другое – непонятно. Хотя привычка выбирать хорошие имена, безусловно, полезна, она не может заменить добросовестно написанную документацию. Помните: бесплатных завтраков не бывает.

Чтобы упростить себе жизнь, профессионалы пользуются автоматическими генераторами документации, которые анализируют код и комментарии в исходных файлах для порождения полноценной документации в различных форматах. Добавить такие генераторы в проект CMake очень просто – и сейчас вы в этом убедитесь!

В этой главе будут рассмотрены следующие вопросы:

- добавление Doxygen в проект;
- генерирование документации в современном формате;
- обогащение вывода специальным HTML-кодом.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch13.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Добавление Doxygen в проект

Один из самых устоявшихся и популярных инструментов генерирования документации на основе исходных файлов C++ – Doxygen. Я употребил слово «устоявшийся» не случайно: первую версию Димитрий ван Хееш выпустил в октябре 1997 года. С тех пор программа неимоверно разрослась и активно поддерживается почти 250 соразработчиками через репозиторий по адресу https://github.com/doxygen/doxygen.

Возможно, вы опасаетесь, что включение Doxygen в крупный проект, в котором генерирование документации не было заложено изначально, вызовет трудности. Действительно, задача аннотирования каждой функции может показаться трудноразрешимой. Но я призываю вас начать с малого. Сосредоточьтесь на документировании тех частей, над которыми недавно работали в последних фиксациях. Помните, что даже частичная документация лучше никакой, и постепенно вы будете понимать проект все лучше и лучше.

Doxygen умеет порождать документацию в следующих форматах:

- **HyperText Markup Language** (**HTML**);
- **Rich Text Format** (**RTF**);
- **Portable Document Format** (**PDF**);
- **Lamport TeX** (**LaTeX**);
- **PostScript** (**PS**)
- страница руководства Unix;
- **Microsoft Compiled HTML** (**CHM**).

Если вы аннотируете код комментариями, предоставляющими дополнительную информацию в формате, который понимает программа Doxygen, то она разберет ее и добавит в выходной файл. Более того, программа проанализирует структуру кода для создания полезных схем и диаграмм. Эта возможность факультативна, потому что требует внешнего инструмента Graphviz (https://graphviz.org/).

Разработчик должен сначала ответить на следующий вопрос: *будут ли пользователи проекта только получать документацию или смогут генерировать ее самостоятельно (быть может, при сборке из исходных файлов)*? Первый вариант подразумевает, что документация распространяется вместе с двоичными файлами, доступна онлайн или (что менее элегантно) может быть получена вместе с исходным кодом из репозитория.

Это различие имеет значение, потому что если пользователи захотят генерировать документацию в процессе сборки, то им придется установить в свою систему зависимости. Проблема не особенно серьезна, потому что Doxygen и Graphviz поддерживаются почти всеми менеджерами пакетов и нужно лишь выполнить простую команду, например следующую для Debian:

```
apt-get install doxygen graphviz
```

В двоичном виде они доступны также для Windows (зайдите на сайт проекта, упомянутый в разделе «Для дополнительного чтения»).

Но не всем пользователям устанавливать инструментарий удобно. Мы должны решить, что будем делать: генерировать документацию для пользователей или заставим их установить зависимости. Также проект может установить зависимости автоматически, как описано в главе 9. Отметим, что Doxygen собирается с применением CMake, поэтому вы уже знаете, как откомпилировать его из исходных файлов.

Если Doxygen и Graphviz установлены в системе, то мы можем добавить в свой проект генерирование документации. В отличие от того, что пишут некоторые онлайновые источники, это не так трудно, как может показаться. Нам не нужно создавать внешние конфигурационные файлы, задавать путь к исполняемому файлу Doxygen или добавлять специальные цели. Начиная с версии CMake 3.9 можно воспользоваться функцией `doxygen_add_docs()` из модуля поиска `FindDoxygen`, которая настраивает цель создания документации. Вот ее сигнатура:

```
doxygen_add_docs(targetName [sourceFilesOrDirs...]
  [ALL] [WORKING_DIRECTORY dir] [COMMENT comment])
```

В первом аргументе задается имя цели, которую мы будем явно собирать, вызвав `cmake` с флагом `-t` (после генерирования дерева сборки):

```
# cmake --build <build-tree> -t targetName
```

Или мы можем сделать так, чтобы документация генерировалась всегда, добавив аргумент `ALL`, хотя обычно это необязательно. Аргумент `WORK-`

ING_DIRECTORY определяет каталог, в котором должна выполняться команда. Значение в аргументе COMMENT отображается перед началом генерирования документации и содержит полезную информацию или инструкции.

Мы поступим так же, как в предыдущих главах, и создадим служебный модуль, содержащий вспомогательную функцию (чтобы ее можно было использовать в других проектах).

ch13/01-doxygen/cmake/Doxygen.cmake

```
function(Doxygen input output)
  find_package(Doxygen)
  if (NOT DOXYGEN_FOUND)
    add_custom_target(doxygen COMMAND false
      COMMENT "Doxygen not found")
    return()
  endif()
  set(DOXYGEN_GENERATE_HTML YES)
  set(DOXYGEN_HTML_OUTPUT
    ${PROJECT_BINARY_DIR}/${output})
  doxygen_add_docs(doxygen
    ${PROJECT_SOURCE_DIR}/${input}
    COMMENT "Generate HTML documentation"
  )
endfunction()
```

Функция принимает два аргумента – каталог input и output – и создает специальную цель doxygen. Вот что здесь происходит.

1. Сначала мы пользуемся встроенным в CMake модулем поиска, чтобы проверить, установлен ли пакет Doxygen в системе.
2. Если нет, мы создаем фиктивную цель doxygen, которая информирует пользователя и выполняет команду false, которая в Unix-подобных системах возвращает 1, что приводит к неудачному завершению сборки. В этой точке мы выходим из функции с помощью return().
3. Если Doxygen найдена, то мы настраиваем ее так, чтобы документация генерировалась в формате HTML в указанном каталоге output. Doxygen конфигурируется в очень широких пределах (подробности см. в официальной документации). Чтобы задать значение любого параметра, делайте так же, как в примере: вызовите set() и предпошлите имени префикс DOXYGEN_.
4. Настраиваем настоящую цель doxygen. Все переменные вида DOXYGEN_ будут переадресованы в конфигурационный файл Doxygen, а документация будет генерироваться на основе файлов в указанном каталоге input.

Если вы хотите, чтобы документацию генерировали пользователи, то шаг 2, вероятно, следует заменить установкой Doxygen.

Для использования эту функцию следует включить в главный списковый файл проекта, как показано ниже.

ch13/01-doxygen/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Doxygen CXX)
enable_testing()
list(APPEND CMAKE_MODULE_PATH "${CMAKE_SOURCE_DIR}/cmake")
add_subdirectory(src bin)
include(Doxygen)
Doxygen(src docs)
```

Совсем не трудно! Во время сборки цели `doxygen` генерируется HTML-документация, которая выглядит следующим образом.

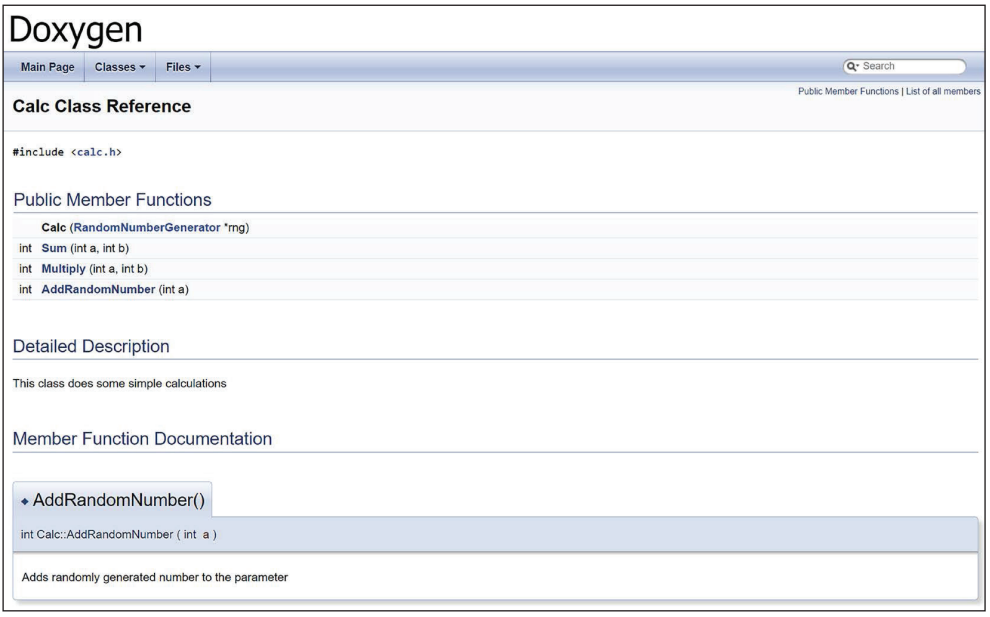


Рис. 13.1. Справочная документация по классу, сгенерированная Doxygen

Для добавления важных деталей в раздел **Member Function Documentation** мы можем предпослать объявлению метода C++ подходящий комментарий в заголовочном файле, например:

ch13/01-doxygen/src/calc.h (fragment)

```
/**
 Multiply... Who would have thought?
 @param a the first factor
```

```
 @param b the second factor
 @result The product
*/
int Multiply(int a, int b);
```

Этот формат называется Javadoc. Важно, чтобы блок комментария начинался двумя звездочками: /**. Дополнительные сведения можно найти в описании блоков документации Doxygen (см. ссылку в разделе «Для дополнительного чтения»). Документация по функции `Multiply` с такими аннотациями будет выглядеть, как показано на рисунке ниже.

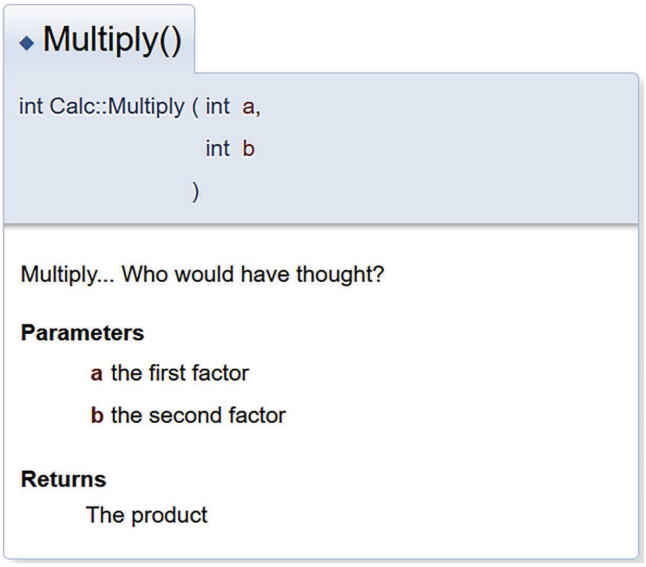


Рис. 13.2. Аннотации параметров и результата

Мы уже говорили, что если Graphviz установлен, то Doxygen обнаружит это и сгенерирует диаграммы зависимостей, как показано на рис. 13.3.

Благодаря генерированию документации непосредственно из исходного кода она обновляется синхронно с изменением кода в процессе разработки. Кроме того, обновления, не отраженные в комментариях, скорее всего, будут обнаружены во время инспекции кода.

Многие разработчики сетуют на то, что дизайн документов, предлагаемый Doxygen, выглядит старомодным, поэтому они не хотели бы показывать сгенерированную документацию своим клиентам. Однако у этой проблемы есть простое решение.

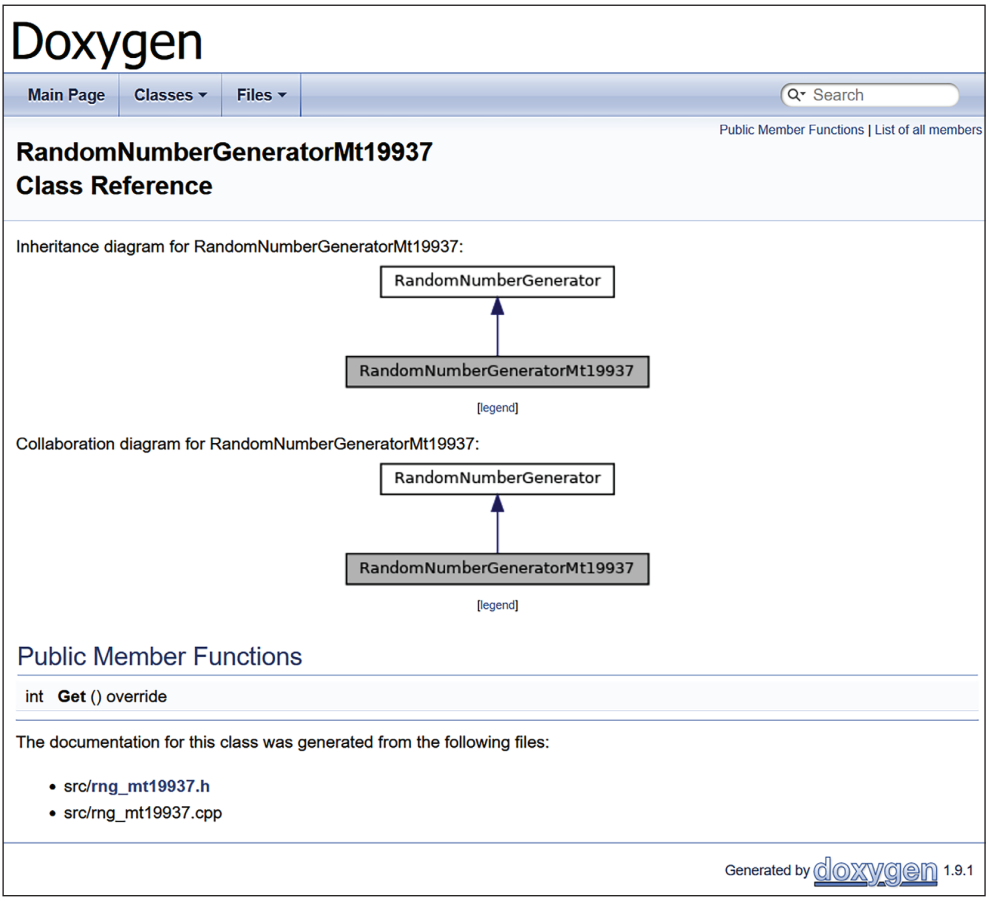


Рис. 13.3. Диаграммы наследования и кооперации, сгенерированные Doxygen

Генерирование документации, выглядящей современно

Наличие у проекта красиво и современно оформленной документации важно. Раз уж мы проделали тяжелую работу по написанию высококачественной документации для нашего ультрагиперсовременного проекта, то хорошо бы, чтобы пользователи оценили это. Но хотя функционально Doxygen очень насыщен, приверженность последним тенденциям в области визуального представления не является его сильной стороной. Однако изменить внешний вид не так уж сложно.

На наше счастье, разработчик `jothepro` создал тему `doxygen-awesome-css`, предлагающую современный допускающий настройку дизайн. Эта тема представлена на рис. 13.4.

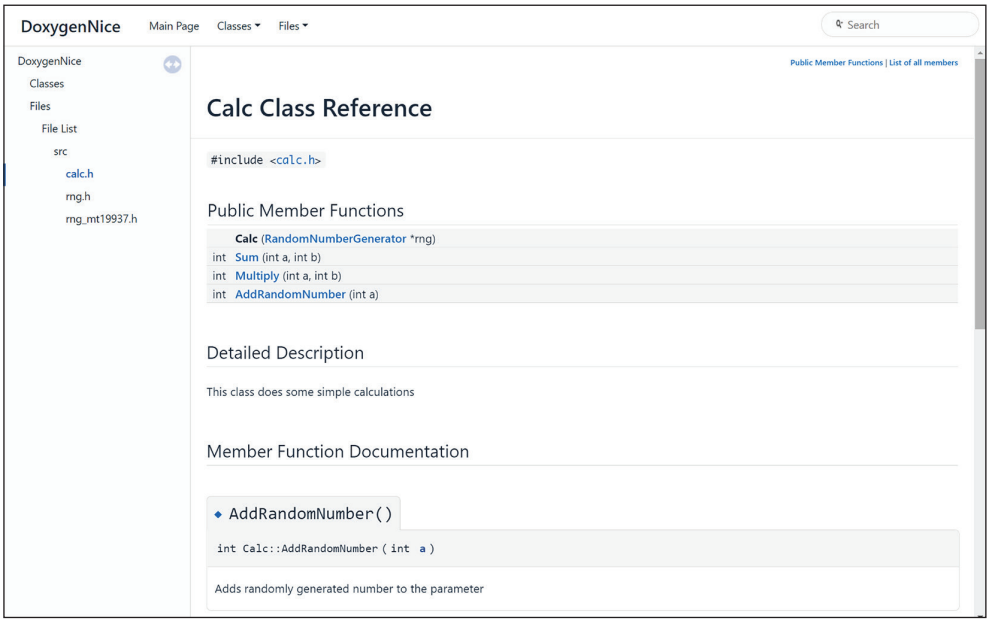


Рис. 13.4. HTML-документация, оформленная согласно теме doxygen-awesome-css

Тема не нуждается в дополнительных зависимостях, ее можно скачать со страницы GitHub https://github.com/jothepro/doxygen-awesome-css.



Некоторые онлайновые ресурсы рекомендуют использовать комбинацию приложений, например преобразовывать вывод Doxygen с помощью Sphinx с расширениями Breathe и Exhale, но этот метод сложен и требует много дополнительных зависимостей (например, Python). Более простой подход обычно и более практичен, особенно для коллективов, не все члены которых хорошо знакомы с CMake.

Эффективно внедрить эту тему позволит автоматизированный процесс. Посмотрим, как можно расширить файл `Doxygen.cmake`, добавив новый макрос.

ch13/02-doxygen-nice/cmake/Doxygen.cmake (фрагмент)

```
macro(UseDoxygenAwesomeCss)
  include(FetchContent)
  FetchContent_Declare(doxygen-awesome-css
    GIT_REPOSITORY
      https://github.com/jothepro/doxygen-awesome-css.git
    GIT_TAG
      V2.3.1
  )
```

```
    FetchContent_MakeAvailable(doxygen-awesome-css)
    set(DOXYGEN_GENERATE_TREEVIEW YES)
    set(DOXYGEN_HAVE_DOT YES)
    set(DOXYGEN_DOT_IMAGE_FORMAT svg)
    set(DOXYGEN_DOT_TRANSPARENT YES)
    set(DOXYGEN_HTML_EXTRA_STYLESHEET
      ${doxygen-awesome-css_SOURCE_DIR}/doxygen-awesome.css)
  endmacro()
```

Все эти команды уже были описаны в предыдущих главах, но давайте на всякий случай повторим, что здесь происходит.

1. Скачивание `doxygen-awesome-css` из Git с помощью модуля `FetchContent`.
2. Конфигурирование дополнительных параметров Doxygen (они рекомендуются автором темы в файле `README`).
3. Копирование `css`-файла темы в выходной каталог Doxygen.

Понятно, что вызывать этот макрос лучше всего в функции `Doxygen` прямо перед вызовом `doxygen_add_docs()`:

ch13/02-doxygen-nice/cmake/Doxygen.cmake (фрагмент)

```
function(Doxygen input output)
# ...
  UseDoxygenAwesomeCss()
  doxygen_add_docs (...)
endfunction()
macro(UseDoxygenAwesomeCss)
# ...
endmacro()
```

Напомним, что все переменные в макросах устанавливаются в области видимости вызывающей функции.

Теперь мы можем любоваться современным стилем сгенерированной HTML-документации и гордо поделиться ею с миром. Однако наша тема предлагает несколько JavaScript-модулей для более удобной работы. Как их включить?

Обогащение вывода специальным HTML-кодом

Проект Doxygen Awesome содержит несколько дополнительных возможностей, которые можно активировать, включив JavaScript-код в заголовок документации внутри HTML-тега `<head>`. Они могут быть весьма полезны, потому что позволяют переключаться между светлым и темным режимами, добав-

лять кнопку **Copy** для копирования фрагментов кода, вставлять постоянные ссылки на заголовки абзацев и интерактивное оглавление.

Однако для реализации всего этого нужно скопировать дополнительный код в выходной каталог и включить его в сгенерированные HTML-файлы.

Ниже показан JavaScript-код, который нужно включить перед тегом `</head>`:

ch13/cmake/extra_headers

```
<script type="text/javascript" src="$relpath^doxygen-awesome-darkmodetoggle.
js"></script>
<script type="text/javascript" src="$relpath^doxygen-awesome-fragmentcopy-
button.js"></script>
<script type="text/javascript" src="$relpath^doxygen-awesome-paragraphlink.
js"></script>
<script type="text/javascript" src="$relpath^doxygen-awesome-interactivetoc.
js"></script>

<script type="text/javascript">
  DoxygenAwesomeDarkModeToggle.init()
  DoxygenAwesomeFragmentCopyButton.init()
  DoxygenAwesomeParagraphLink.init()
  DoxygenAwesomeInteractiveToc.init()
</script>
```

Как видите, здесь сначала включается несколько JavaScript-файлов, а затем инициализируются различные расширения. К сожалению, этот код нельзя просто записать в какую-нибудь переменную, а приходится переопределять заголовок HTML-файла, подразумеваемый по умолчанию. Такое переопределение можно проделать, предоставив путь к этому файлу в конфигурационной переменной Doxygen `HTML_HEADER`.

Чтобы создать специальный заголовок, не зашивая в код все содержимое, можно воспользоваться командным инструментом Doxygen для генерирования заголовочного файла по умолчанию и отредактировать его перед генерированием документации:

```
doxygen -w html header.html footer.html style.css
```

Хотя мы не собираемся ни использовать, ни изменять файлы `footer.html` и `style.css`, эти аргументы обязательны, поэтому создать их придется в любом случае.

Наконец, мы должны автоматически дописать в начало тег `</head>` и содержимое файла `ch13/cmake/extra_headers`, чтобы включить требуемый JavaScript-код. Это можно сделать с помощью команды Unix `sed`, которая редактирует файл `header.html` на месте:

```
sed -i '/<\/head>/r ch13/cmake/extra_headers' header.html
```

Осталось закодировать эти шаги на языке CMake. Это делает следующий макрос.

ch13/02-doxygen-nice/cmake/Doxygen.cmake (фрагмент)

```
macro(UseDoxygenAwesomeExtensions)
  set(DOXYGEN_HTML_EXTRA_FILES
    ${doxygen-awesome-css_SOURCE_DIR}/doxygen-awesome-darkmode-toggle.js
    ${doxygen-awesome-css_SOURCE_DIR}/doxygen-awesome-fragment-copybutton.js
    ${doxygen-awesome-css_SOURCE_DIR}/doxygen-awesome-paragraph-link.js
    ${doxygen-awesome-css_SOURCE_DIR}/doxygen-awesome-interactive-toc.js
  )

  execute_process(
    COMMAND doxygen -w html header.html footer.html style.css
    WORKING_DIRECTORY ${PROJECT_BINARY_DIR}
  )
  execute_process(
    COMMAND sed -i
    "/<\\/head>/r ${PROJECT_SOURCE_DIR}/cmake/extra_headers"
    header.html
    WORKING_DIRECTORY ${PROJECT_BINARY_DIR}
  )
  set(DOXYGEN_HTML_HEADER ${PROJECT_BINARY_DIR}/header.html)
endmacro()
```

Этот код выглядит сложно, но при ближайшем рассмотрении оказывается, что он совсем простой.

Вот что он делает.

1. Копирует все четыре JavaScript-файла в выходной каталог.
2. Выполняет команду `doxygen`, которая генерирует HTML-файлы по умолчанию.
3. Выполняет команду `sed`, которая вставляет нужный JavaScript-код в заголовок.
4. Переопределяет заголовок по умолчанию нашей версией.

Для завершения интеграции вызовите этот макрос сразу после включения таблицы стилей.

ch13/02-doxygen-nice/cmake/Doxygen.cmake (фрагмент)

```
function(Doxygen input output)
# …
  UseDoxygenAwesomeCss()
  UseDoxygenAwesomeExtensions()
# …
endfunction()
```

Полный код с практическими примерами имеется в репозитории этой книги. Как всегда, я рекомендую изучить примеры и опробовать их в реальном окружении.



Другие утилиты для генерирования документации

Существуют десятки инструментов, не рассмотренных в этой книге, поскольку нас интересуют прежде всего проекты, поддерживаемые CMake. Тем не менее некоторые могут подойти к вашему конкретному случаю. Если вы ощущаете в себе авантюрную жилку, зайдите на сайты двух проектов, которые мне кажутся интересными:

- Adobe Hyde (https://github.com/adobe/hyde): ориентирован на компилятор Clang и порождает файлы в формате Markdown, которые понимают такие инструменты, как Jekyll (https://jekyllrb.com/), генератор статических страниц, поддерживаемый GitHub;
- Standardese (https://github.com/standardese/standardese): использует библиотеку `libclang` для компиляции вашего кода и умеет порождать выходные файлы в форматах HTML, Markdown, LaTex и в виде страниц руководства. Ставит себе целью (весьма смело) прийти на смену Doxygen.

Резюме

В этой главе мы изучили практические аспекты добавления Doxygen, мощного средства генерирования документации, в проект CMake и улучшения внешнего вида созданной документации. Эта задача, на первый взгляд пугающая, на самом деле вполне решаема, а в результате мы получаем более понятную информацию о решении. То, что время, потраченное на добавление и сопровождение документации, с лихвой окупается, станет понятно, когда вам или вашим коллегам придется разбираться в хитросплетениях связей в приложении.

Рассмотрев, как использовать встроенную в CMake поддержку Doxygen на практике, мы повернули в другом направлении, обратив взор не столько на удобочитаемость, сколько на приятное глазу представление документации.

Устаревший дизайн режет глаз, поэтому мы обсудили способы создания альтернативных HTML-представлений. Для этого мы воспользовались расширением Doxygen Awesome. Чтобы реализовать предлагаемые им улучшения, мы изменили стандартный заголовок, добавив необходимый JavaScript-код.

Благодаря генерированию документации мы гарантируем близость к реальному коду и синхронизацию написанных объяснений с его логикой, особенно если то и другое находится в одном файле. Кроме того, любой программист постоянно должен держать в голове многочисленные задачи и детали. Документация играет роль записной книжки, помогая сохранять и впоследствии вспоминать тонкости проекта. Помните, что даже «самый короткий карандаш лучше самой долгой памяти». Сделайте себе одолжение – записывайте то, что может понадобиться, и преуспевайте.

Подводя итоги этой главе, подчеркнем ценность Doxygen как одного из инструментов управления проектом, помогающего понять его и наладить взаимодействие внутри команды.

В следующей главе мы будем говорить об автоматизации упаковки и установки проектов с помощью CMake.

Для дополнительного чтения

- Официальный сайт Doxygen: https://www.doxygen.nl/.
- Документация по модулю поиска `FindDoxygen`: https://cmake.org/cmake/help/latest/module/FindDoxygen.html.
- Блоки документации в Doxygen: https://www.doxygen.nl/manual/docblocks.html#specialblock.

14

Установка и упаковка

Наш проект собран, протестирован и документирован. Пришло время выпустить его для пользователей. Эта глава посвящена двум последним шагам: установке и упаковке. Это продвинутые техники, опирающиеся на все, что мы уже изучили: управление целями и их зависимостями, транзитивные требования использования, генераторные выражения и многое другое.

После установки наш проект становится виден операционной системе и доступен для использования. Мы обсудим, как экспортировать цели для других проектов без установки и как установить проект, чтобы им можно было воспользоваться. Мы узнаем, как сконфигурировать проект, чтобы различные артефакты автоматически помещались в нужные каталоги. Для более сложных сценариев мы введем в рассмотрение команды низкого уровня для установки файлов и каталогов, а также для выполнения специальных скриптов и команд CMake.

Далее мы изучим подготовку пакетов CMake, которые другие проекты смогут обнаружить с помощью команды `find_package()`. Мы объясним, как сделать так, чтобы цели и их определения не были ограничены конкретным местом в файловой системе. Мы обсудим, как писать базовые и продвинутые **конфигурационные файлы**, а также **файлы версий**, ассоциированные с пакетами. Затем, чтобы повысить степень модульности, мы познакомимся с концепцией компонентов – в терминах пакетов CMake и команды `install()`. Все эти подготовительные действия проложат дорогу к последней теме, которую мы рассмотрим в этой главе: использованию CPack для генерирования архивов, установщиков и пакетов, распознаваемых всевозможными менеджерами пакетов в разных операционных системах. В виде таких пакетов можно распространять заранее собранные артефакты, исполняемые файлы и библиотеки. Для пользователей это самый простой способ приступить к работе с нашей программой.

В этой главе будут рассмотрены следующие вопросы:

- экспорт без установки;
- установка проекта в систему;
- создание повторно используемых пакетов;
- определение компонентов;
- упаковка с помощью CPack.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch14.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Экспорт без установки

Как сделать цели проекта `A` доступными проекту `B`? Обычно мы используем команду `find_package()`, но для этого необходимо создать пакет и установить его в систему. Это, конечно, полезно, но требует определенной работы. А иногда нам нужен просто быстрый способ собрать проект и сделать его цели доступными другим проектам.

Один такой экономящий время метод заключается в том, чтобы включить в проект `B` главный списковый файл `A`, который уже содержит все определения целей. Однако этот файл может также содержать глобальную конфигурацию, команды CMake с побочными эффектами, дополнительные зависимости и, быть может, нежелательные для `B` цели (например, автономные тесты). Так что это не лучший подход. Вместо этого мы можем подготовить **файл экспорта целей**, который проект-потребитель `B` включит командой `include()`:

```
cmake_minimum_required(VERSION 3.26.0)
project(B)
include(/path/to/A/TargetsOfA.cmake)
```

При этом мы определим все цели `A` с правильно установленными свойствами, используя такие команды, как `add_library()` и `add_executable()`.

Вы должны указать все экспортируемые цели после ключевого слова `TARGETS` и предоставить имя конечного файла после слова `FILE`. Все остальные аргументы факультативны:

```
export(TARGETS [target1 [target2 [...]]]
       [NAMESPACE <namespace>] [APPEND] FILE <path>
       [EXPORT_LINK_INTERFACE_LIBRARIES]
)
```

Объясним, что означают аргументы:

- `NAMESPACE` рекомендуется для того, чтобы показать, что цель была импортирована из другого проекта;
- `APPEND` говорит CMake, что не нужно стирать содержимое файла перед записью;
- `EXPORT_LINK_INTERFACE_LIBRARIES` экспортирует зависимости цели по компоновке (включая импортированные и зависящие от конфигурации варианты).

Применим этот метод экспорта к примеру библиотеки `Calc`, предоставляющей два простых метода.

ch14/01-export/src/include/calc/basic.h

```
#pragma once
int Sum(int a, int b);
int Multiply(int a, int b);
```

Мы должны объявить цель `Calc`, чтобы было что экспортировать.

ch14/01-export/src/CMakeLists.txt

```
add_library(calc STATIC basic.cpp)
target_include_directories(calc INTERFACE include)
```

Затем для генерирования файла экспорта воспользуемся командой `export(TARGETS)`.

ch14/01-export/CMakeLists.txt (фрагмент)

```
cmake_minimum_required(VERSION 3.26)
project(ExportCalcCXX)
add_subdirectory(src bin)
set(EXPORT_DIR "${CMAKE_CURRENT_BINARY_DIR}/cmake")
export(TARGETS calc
  FILE "${EXPORT_DIR}/CalcTargets.cmake"
  NAMESPACE Calc::
)
```

Наш файл объявления экспортированных целей будет находиться в подкаталоге `cmake` дерева сборки (по соглашению для `cmake`-файлов). Чтобы не повторять этот путь позже, сохраним его в переменной `EXPORT_DIR`. Затем мы вызываем команду `export()` для генерирования файла объявления целей, `CalcTargets.cmake`, который будет содержать только цель `calc`. В проектах, включающих этот файл, цель будет видна как `Calc::calc`.

Заметим, что этот файл экспорта – еще не *пакет*. Но важнее то, что все пути в этом файле абсолютные и зашиты в дерево сборки, что делает их неперемещаемыми (обсуждается в разделе «Проблемы перемещаемости целей» ниже).

У команды `export()` также имеется более короткая версия с ключевым словом `EXPORT`:

```
export(EXPORT <export> [NAMESPACE <namespace>] [FILE <path>])
```

Однако она требует имени предопределенного экспорта, а не списка экспортированных целей. Такими экземплярами `<export>` являются именованные списки целей, создаваемые командой `install(TARGETS)` (мы рассмотрим эту команду в разделе «Установка логических целей»).

В примере ниже демонстрируется, как эта сокращенная команда используется на практике.

ch14/01-export/CMakeLists.txt (продолжение)

```
install(TARGETS calc EXPORT CalcTargets)
export(EXPORT CalcTargets
  FILE "${EXPORT_DIR}/CalcTargets2.cmake"
  NAMESPACE Calc::
)
```

Этот код работает так же, как предыдущий, но теперь один и тот же список целей разделяется между командами `export()` и `install()`.

Оба метода генерирования файлов экспорта дают похожие результаты. Они включают стереотипный код и несколько строк, определяющих цель. Если `<build-tree>` содержит путь к дереву сборки, то они создают **файл экспорта целей** следующего вида:

<build-tree>/cmake/CalcTargets.cmake (фрагмент)

```
# Создать импортируемую цель Calc::calc
add_library(Calc::calc STATIC IMPORTED)
set_target_properties(Calc::calc PROPERTIES
  INTERFACE_INCLUDE_DIRECTORIES
  "/<source-tree>/include"
)
# Импортировать цель "Calc::calc" для конфигурации ""
set_property(TARGET Calc::calc APPEND PROPERTY
  IMPORTED_CONFIGURATIONS NOCONFIG
)
set_target_properties(Calc::calc PROPERTIES
  IMPORTED_LINK_INTERFACE_LANGUAGES_NOCONFIG "CXX"
  IMPORTED_LOCATION_NOCONFIG "/<build-tree>/libcalc.a"
)
```

В обычных обстоятельствах мы не стали бы ни редактировать, ни даже открывать этот файл, но важно отметить, что пути будут зашиты в него (см. выделенные строки). В текущей форме собранный проект не является перемещаемым. Чтобы исправить ситуацию, нужны дополнительные шаги.

В следующем разделе мы объясним, что такое перемещаемость и почему она так важна.

Установка проектов в систему

В главе 1 мы говорили, что CMake предлагает командный режим для установки собранных проектов в систему:

```
cmake --install <dir> [<options>]
```

Здесь `<dir>` – путь к сгенерированному дереву сборки (обязателен). Могут быть заданы следующие необязательные параметры (`<options>`):

- `--config <cfg>`: выбирает конфигурацию сборки для генераторов, поддерживающих несколько конфигураций;
- `--component <comp>`: устанавливает только заданный компонент;
- `--default-directory-permissions <permissions>`: задает разрешения по умолчанию для установленных каталогов (в формате `<u=rwx,g=rx,o=rx>`);
- `--install-prefix <prefix>`: задает отличный от умалчиваемого путь установки (хранится в переменной `CMAKE_INSTALL_PREFIX`). По умолчанию это `/usr/local` в Unix-подобных системах и `c:/Program Files/${PROJECT_NAME}` в Windows. До версии CMake 3.21 нужно было использовать менее понятное название параметра: `--prefix <prefix>`;
- `-v`, `--verbose`: выводит более подробные сведения (можно вместо этого установить переменную окружения `VERBOSE`).

В процессе установки сгенерированные артефакты и необходимые зависимости обычно копируются в системный каталог. CMake предлагает удобный стандарт установки для всех проектов CMake, а также ряд дополнительных преимуществ.

- Предоставляет платформенно-зависимые пути установки артефактов, зависящие от их типа (следуя *стандартам кодирования GNU*).
- Улучшает процесс установки благодаря генерированию файлов экспорта целей, чтобы позволяет повторно использовать цели одного проекта в других проектах.
- Создает допускающие обнаружение пакеты с помощью **конфигурационных файлов**, которые обертывают файлы экспорта целей и зависящие от пакета макросы и функции CMake, определенные автором.

Эти возможности экономят много времени и упрощают работу с подготовленными таким образом проектами. Первый шаг простой установки – копирование собранных артефактов в конечный каталог. Это подводит нас к команде `install()` и ее режимам:

- `install(TARGETS)`: выводит такие артефакты, как библиотеки и исполняемые файлы;

- install(FILES|PROGRAMS): устанавливает отдельные файлы и задает права доступа к ним. Эти файлы не обязаны быть частями логических целей;
- install(DIRECTORY): устанавливает целые каталоги;
- install(SCRIPT|CODE): выполняет скрипт или фрагмент кода CMake во время установки;
- install(EXPORT): генерирует и устанавливает файл экспорта целей;
- install(RUNTIME_DEPENDENCY_SET <set-name> [...]): устанавливает зависимость времени выполнения, определенную в проекте;
- install(IMPORTED_RUNTIME_ARTIFACTS <target>... [...]): запрашивает у импортированных целей артефакты и устанавливает их.

При добавлении этих команд в списковый файл генерируется файл cmake_install.cmake в дереве сборки. Этот скрипт можно выполнить вручную командой cmake -P, но это не рекомендуется. Файл предназначен для внутреннего использования CMake при выполнении команды cmake --install.

У каждого режима install() есть свой набор параметров, некоторые используются в разных режимах:

- DESTINATION: задает путь установки. Относительные пути начинаются префиксом CMAKE_INSTALL_PREFIX, а абсолютные используются буквально (и не поддерживаются командой cpack);
- PERMISSIONS: задает права доступа к файлам на поддерживаемых платформах. Допустимые значения: OWNER_READ, OWNER_WRITE, OWNER_EXECUTE, GROUP_READ, GROUP_WRITE, GROUP_EXECUTE, WORLD_READ, WORLD_WRITE, WORLD_EXECUTE, SETUID и SETGID. Подразумеваемые по умолчанию права доступа к каталогам, создаваемым в процессе установки, можно записать в переменную CMAKE_INSTALL_DEFAULT_DIRECTORY_PERMISSIONS;
- CONFIGURATIONS: задает конфигурации (Debug, Release). Параметры, следующие за этим ключевым словом, применимы, только если текущая конфигурация сборки находится в списке;
- OPTIONAL: предотвращает ошибку, если устанавливаемых файлов не существует.

Два разделяемых параметра, COMPONENT и EXCLUDE_FROM_ALL, используются при установке отдельных компонентов. Их мы обсудим в разделе «Определение компонентов» ниже. А пока займемся первым режимом установки: install(TARGETS).

Установка логических целей

Цели, определяемые командами add_library() и add_executable(), легко установить командой install(TARGETS). Это подразумевает копирование артефактов, созданных сборочной системой, в соответствующие конечные каталоги и задание для них прав доступа. Сигнатура этого режима такова:

```
install(TARGETS <target>... [EXPORT <export-name>]
        [<output-artifact-configuration> ...]
```

```
        [INCLUDES DESTINATION [<dir> ...]]
)
```

После начального спецификатора режима, `TARGETS`, мы должны задать список целей, которые хотим установить. Факультативно мы можем присвоить их **именованному экспорту** с помощью параметра `EXPORT`, который можно использовать в командах `export(EXPORT)` и `install(EXPORT)` для порождения файла экспорта целей. Затем мы должны сконфигурировать установку выходных артефактов (сгруппированных по типу). Факультативно можно предоставить список каталогов, которые будут добавлены в файл экспорта целей для каждой цели в виде ее свойства `INTERFACE_INCLUDE_DIRECTORIES`.

`[<output-artifact-configuration>...]` задает список конфигурационных блоков. Вот как выглядит полный синтаксис одного блока:

```
<TYPE> [DESTINATION <dir>]
       [PERMISSIONS permissions...]
       [CONFIGURATIONS [Debug|Release|...]]
       [COMPONENT <component>]
       [NAMELINK_COMPONENT <component>]
       [OPTIONAL] [EXCLUDE_FROM_ALL]
       [NAMELINK_ONLY|NAMELINK_SKIP]
```

Команда требует, чтобы блок каждого выходного артефакта начинался словом `<TYPE>` (это единственный обязательный элемент). CMake распознает несколько типов:

- `ARCHIVE`: статические библиотеки (`.a`) и библиотеки импорта DLL для Windows (`.lib`);
- `LIBRARY`: разделяемые библиотеки (`.so`), но не DLL;
- `RUNTIME`: исполняемые файлы и DLL;
- `OBJECTS`: *объектные файлы* из объектных библиотек;
- `FRAMEWORK`: статические и разделяемые библиотеки, для которых установлено свойство `FRAMEWORK` (тем самым они исключаются из типов `ARCHIVE` и `LIBRARY`). Применимо только к macOS;
- `BUNDLE`: исполняемые файлы, помеченные как `MACOSX_BUNDLE` (не являются частью `RUNTIME`);
- `FILE_SET <set>`: файлы из набора `<set>`, заданного для цели. Заголовочные файлы или заголовки файлов в C++ (начиная с версии CMake 3.23);
- `PUBLIC_HEADER`, `PRIVATE_HEADER`, `RESOURCE`: файлы, заданные в одноименных свойствах целей (на платформах Apple они должны быть установлены для целей `FRAMEWORK` или `BUNDLE`).

В документации по CMake сказано, что если вы конфигурируете только один тип артефактов (например, `LIBRARY`), то только этот тип и будет установлен. Для версии CMake 3.26.0 это неправда: все артефакты устанавливаются, как если бы были сконфигурированы с параметрами по умолчанию. Эту проблему можно решить, задав `<TYPE> EXCLUDE_FROM_ALL` для всех нежелательных типов артефактов.



В одной команде install(TARGETS) может быть несколько блоков конфигурирования артефактов. Но помните, что в одном вызове не может быть более одного экземпляра каждого типа. То есть если вы хотите задать различные конечные каталоги артефактов ARCHIVE для конфигураций Debug и Release, то должны использовать два разных вызова install(TARGETS ... ARCHIVE).

Разрешается опускать имя типа и таким образом задавать параметры для всех артефактов. Тогда будет выполнена установка всех файлов, порожденных этими целями, независимо от их типа:

```
install(TARGETS executable, static_lib1
  DESTINATION /tmp
)
```

Во многих случаях нет необходимости задавать DESTINATION явно, поскольку имеются встроенные умолчания, но следует помнить о нескольких подвохах при работе с разными платформами.

Использование конечного каталога по умолчанию для разных платформ

Устанавливая файлы проекта, CMake копирует их в определенный системный каталог. Разные файлы попадают в разные каталоги. Каталог определяется следующим выражением:

```
${CMAKE_INSTALL_PREFIX} + ${DESTINATION}
```

Как отмечалось в предыдущем разделе, мы можем либо задать DESTINATION явно, либо позволить CMake использовать встроенное умолчание, зависящее от типа артефакта.

Таблица 14.1. Конечные каталоги по умолчанию для разных типов артефактов

Тип артефакта	Встроенное умолчание	Переменная каталога установки
RUNTIME	bin	CMAKE_INSTALL_BINDIR
LIBRARY ARCHIVE	lib	CMAKE_INSTALL_LIBDIR
PUBLIC_HEADER PRIVATE_HEADER FILE_SET (тип HEADERS)	include	CMAKE_INSTALL_INCLUDEDIR

Хотя пути по умолчанию полезны, не всегда они годятся. Например, по умолчанию значение DESTINATION для библиотек равно lib. Тогда полный путь к библиотекам вычисляется как /usr/local/lib для Unix-подобных систем и как C:\Program Files (x86)\<имя-проекта>\lib в Windows. Однако это не годит-

ся для Debian с поддержкой нескольких архитектур, где требуется, чтобы путь зависел от архитектуры (например, `i386-linux-gnu`), а `INSTALL_PREFIX` определен как `/usr`. Определение правильного пути для каждой платформы – широко распространенная проблема в Unix. Чтобы ее решить, придерживайтесь документа «Стандарты кодирования GNU», ссылка на который приведена в разделе «Для дополнительного чтения».

Мы можем переопределить конечные каталоги по умолчанию для каждого типа, задав переменную `CMAKE_INSTALL_<DIRTYPE>DIR`. Вместо того чтобы придумывать алгоритм обнаружения платформы и назначения подходящих путей переменным каталогов установки, воспользуйтесь служебным модулем CMake `GNUInstallDirs`. Этот модуль обрабатывает большинство платформ и присваивает правильные значения переменным каталогов установки. Просто включите его командой `include()`, предшествующей всем командам `install()`, – и все будет хорошо.

Если пользователю понадобится специальная конфигурация, то он сможет переопределить переменные каталогов установки, задав аргумент командной строки:

```
-DCMAKE_INSTALL_BINDIR=/path/in/the/system
```

Однако установка публичных заголовков библиотек все еще представляет проблемы. Разберемся, в чем они состоят.

Публичные заголовки

Публичные заголовки в CMake рекомендуется сохранять в каталоге, который ясно показывает их происхождение и вводит пространство имен, например `/usr/local/include/calc`. Это позволяет включать их в программы на C++ директивой препроцессора

```
#include <calc/basic.h>
```

Большинство препроцессоров интерпретируют угловые скобки как запрос на поиск в стандартных системных каталогах. Мы можем воспользоваться модулем `GNUInstallDirs`, чтобы автоматически заполнить часть `DESTINATION` пути установки и тем самым гарантировать, что заголовочные файлы попадут в каталог `include`.

Начиная с версии CMake 3.23.0 мы можем явно добавлять в цель подлежащие установке заголовочные файлы командой `target_sources()` с ключевым словом `FILE_SET`. Это предпочтительный метод, потому что он учитывает *перемещение* заголовков. Ниже приведена сигнатура команды:

```
target_sources(<target>
  [<PUBLIC|PRIVATE|INTERFACE>
    [FILE_SET <name> TYPE <type> [BASE_DIR <dir>] FILES]
    <файлы>...
```

```
  ]...
)
```

В предположении, что наши заголовки находятся в каталоге `src/include/calc`, приведем практический пример:

ch14/02-install-targets/src/CMakeLists.txt (фрагмент)

```
add_library(calc STATIC basic.cpp)
target_include_directories(calc INTERFACE include)
target_sources(calc PUBLIC FILE_SET HEADERS
                           BASE_DIRS include
                           FILES include/calc/basic.h
)
```

В этом фрагменте определен новый набор файлов `HEADERS` для цели. Здесь используется особый случай: если имя набора файлов совпадает с одним из имеющихся типов, то CMake предполагает, что мы хотим, чтобы набор файлов имел такой тип, – это позволяет не определять тип явно. Если вы хотите использовать другое имя, то не забудьте определить тип набора файлов с помощью ключевого слова `TYPE <TYPE>`.

Определив набор файлов, мы можем использовать его в команде установки:

ch14/02-install-targets/src/CMakeLists.txt (продолжение)

```
...

include(GNUInstallDirs)
install(TARGETS calc ARCHIVE FILE_SET HEADERS)
```

Мы включили модуль `GNUInstallDirs` и сконфигурировали установку статической библиотеки `calc` и ее заголовков. Выполнение `cmake` в режиме установки работает как положено:

```
# cmake -S <source-tree> -B <build-tree>
# cmake --build <build-tree>
# cmake --install <build-tree>
-- Install configuration: ""
-- Installing: /usr/local/lib/libcalc.a
-- Installing: /usr/local/include/calc/basic.h
```

Поддержка ключевого слова `FILE_SET HEADERS` – относительно недавнее добавление, а современная версия CMake работает, к сожалению, не во всех окружениях.

Если вы застряли на версии, меньшей 3.23, то должны будете задать публичные заголовки (перечислив их через точку с запятой) в свойстве `PUBLIC_HEADER` библиотечной цели, а перемещение обрабатывать вручную (подробнее об этом см. раздел «Проблемы перемещаемости целей» ниже).

ch14/03-install-targets-legacy/src/CMakeLists.txt (фрагмент)

```
add_library(calc STATIC basic.cpp)
target_include_directories(calc INTERFACE include)
set_target_properties(calc PROPERTIES
  PUBLIC_HEADER src/include/calc/basic.h
)
```

Понадобится также изменить конечный каталог, включив имя библиотеки в путь `include`:

ch14/02-install-targets-legacy/src/CMakeLists.txt (продолжение)

```
...

include(GNUInstallDirs)
install(TARGETS calc
  ARCHIVE
  PUBLIC_HEADER
  DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}/calc
)
```

Явно вставлять `/calc` в путь необходимо, потому что файлы, заданные в свойстве `PUBLIC_HEADER`, не сохраняют свою принадлежность к каталогам. Они будут установлены в один и тот же конечный каталог, даже если первоначально находились в разных. Этот важный недостаток и послужил причиной разработки `FILE_SET`.

Итак, теперь вы знаете, как производится установка в простых случаях, но что делать в более сложных сценариях?

Низкоуровневая установка

Современная CMake уходит от прямых операций с файлами. В идеале хотелось бы добавить файлы в логическую цель, используя ее как более высокий уровень абстракции для представления активов: исходных файлов, заголовочных файлов, ресурсов, конфигурации и т. д. Главное преимущество – то, что код отвечает принципу DRY; обычно для добавления файла в состав цели требуется изменить не больше одной строки кода.

К сожалению, добавлять все устанавливаемые файлы в состав цели не всегда возможно или удобно. На такой случай есть три варианта: `install(FILES)`, `install(PROGRAMS)` и `install(DIRECTORY)`.

Установка командами install(FILES) и install(PROGRAMS)

Режимы `FILES` и `PROGRAMS` очень похожи. Их можно использовать для установки различных активов, включая публичные заголовочные файлы, документацию, скрипты оболочки, конфигурацию, а также файлы, используемые во время выполнения: изображения, звуковые файлы и базы данных.

Команда имеет следующую сигнатуру:

```
install(<FILES|PROGRAMS> files...
        TYPE <type> | DESTINATION <dir>
        [PERMISSIONS permissions...]
        [CONFIGURATIONS [Debug|Release|...]]
        [COMPONENT <component>]
        [RENAME <name>] [OPTIONAL] [EXCLUDE_FROM_ALL]
)
```

Главное различие между FILES и PROGRAMS – подразумеваемый по умолчанию набор разрешений для скопированных файлов: install(PROGRAMS) устанавливает разрешение EXECUTE для всех пользователей, а install(FILES) этого не делает (хотя в обоих случаях устанавливаются разрешения OWNER_WRITE, OWNER_READ, GROUP_READ и WORLD_READ).

Это поведение можно модифицировать, добавив факультативное ключевое слово PERMISSIONS и выбрав в качестве начального ключевого слова FILES или PROGRAMS, чтобы показать, что именно устанавливается. Мы уже разбирали, как работают ключевые слова PERMISSIONS, CONFIGURATIONS и OPTIONAL. Слова COMPONENT и EXCLUDE_FROM_ALL мы обсудим ниже в разделе «Определение компонентов».

После начального ключевого слова необходимо перечислить файлы, подлежащие установке. CMake поддерживает относительные и абсолютные пути, а также генераторные выражения. Напомним, что если путь к файлу начинается генераторным выражением, то он должен быть абсолютным.

Следующее обязательное ключевое слово – TYPE или DESTINATION. Вы можете либо явно задать путь DESTINATION, либо попросить CMake найти его для конкретного файла типа TYPE. В отличие от install(TARGETS), в этом контексте TYPE не выбирает никакое подмножество предоставленных файлов для установки. Тем не менее вычисление пути установки производится аналогично (символом + обозначается платформенно-зависимый разделитель компонентов пути):

```
${CMAKE_INSTALL_PREFIX} + ${DESTINATION}
```

Аналогично для каждого TYPE определены встроенные умолчания.

Таблица 14.2. Встроенные умолчания для разных типов файлов

Тип файла	Встроенное умолчание	Переменная каталога установки
BIN	bin	CMAKE_INSTALL_BINDIR
SBIN	sbin	CMAKE_INSTALL_SBINDIR
LIB	lib	CMAKE_INSTALL_LIBDIR
INCLUDE	include	CMAKE_INSTALL_INCLUDEDIR
SYSCONF	etc	CMAKE_INSTALL_SYSCONFDIR
SHAREDSTATE	com	CMAKE_INSTALL_SHARESTATEDIR

Таблица 14.2 *(окончание)*

Тип файла	Встроенное умолчание	Переменная каталога установки
LOCALSTATE	etc	CMAKE_INSTALL_LOCALSTATEDIR
RUNSTATE	$LOCALSTATE/run	CMAKE_INSTALL_RUNSTATEDIR
DATA	$DATAROOT	CMAKE_INSTALL_DATADIR
INFO	$DATAROOT/info	CMAKE_INSTALL_INFODIR
LOCALE	$DATAROOT/locale	CMAKE_INSTALL_LOCALEDIR
MAN	$DATAROOT/man	CMAKE_INSTALL_MANDIR
DOC	$DATAROOT/doc	CMAKE_INSTALL_DOCDIR

Здесь поведение определяется тем же принципом, что был описан в разделе «Использование конечного каталога по умолчанию для разных платформ»: если никакая переменная каталога установки для данного типа файлов TYPE не задана, то CMake возьмет встроенный путь по умолчанию. И снова ради переносимости можно использовать модуль GNUInstallDirs.

Некоторым встроенным догадкам в этой таблице предшествуют переменные каталога установки:

- $LOCALSTATE равно CMAKE_INSTALL_LOCALSTATEDIR или по умолчанию var;
- $DATAROOT равно CMAKE_INSTALL_DATAROOTDIR или по умолчанию share.

Как и в случае install(TARGETS), модуль GNUInstallDirs предоставляет платформенно-зависимые переменные каталогов установки. Рассмотрим пример.

ch14/04-install-files/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(InstallFiles CXX)
include(GNUInstallDirs)
install(FILES
  src/include/calc/basic.h
  src/include/calc/nested/calc_extended.h
  DESTINATION ${CMAKE_INSTALL_INCLUDEDIR}/calc
)
```

В этом случае CMake устанавливает две библиотеки, состоящие только из заголовочных файлов – basic.h и nested/calc_extended.h, – в зависящий от проекта подкаталог системного каталога include.



Из исходного кода GNUInstallDirs мы знаем, что переменная CMAKE_INSTALL_INCLUDEDIR одинакова для всех поддерживаемых платформ. Однако ее все же рекомендуется использовать из соображений удобочитаемости и согласованности с другими динамическими переменными. Например, CMAKE_INSTALL_LIBDIR зависит от архитектуры и дистрибутива – lib, lib64 или lib/<multiarch-tuple>.

Начиная с версии CMake 3.20 стало возможно использовать ключевое слово RENAME в командах install(FILES) и install(PROGRAMS). Оно работает, только

если команда устанавливает единственный файл, и за ним должно следовать новое имя файла.

Пример в этом разделе демонстрирует простоту установки файлов в нужный каталог. Однако есть одна проблема – взгляните на вывод команды установки:

```
# cmake -S <source-tree> -B <build-tree>
# cmake --build <build-tree>
# cmake --install <build-tree>
-- Install configuration: ""
-- Installing: /usr/local/include/calc/basic.h
-- Installing: /usr/local/include/calc/calc_extended.h
```

Оба файла устанавливаются в один и тот же каталог, независимо от того, на каком уровне вложенности они находились первоначально. Иногда это нежелательно. В следующем разделе мы поговорим о том, как уладить эту ситуацию.

Работа с целыми каталогами

Если вас не устраивает добавление файлов в команду установки по одному, то можно работать с целыми каталогами. Именно для этого предназначен режим `install(DIRECTORY)`, который копирует указанные каталоги в выбранное место. Команда имеет следующую сигнатуру:

```
install(DIRECTORY dirs...
        TYPE <type> | DESTINATION <dir>
        [FILE_PERMISSIONS permissions...]
        [DIRECTORY_PERMISSIONS permissions...]
        [USE_SOURCE_PERMISSIONS] [OPTIONAL] [MESSAGE_NEVER]
        [CONFIGURATIONS [Debug|Release|...]]
        [COMPONENT <component>] [EXCLUDE_FROM_ALL]
        [FILES_MATCHING]
        [[PATTERN <pattern> | REGEX <regex>] [EXCLUDE]
        [PERMISSIONS permissions...]] [...]
)
```

Многие параметры похожи на встречавшиеся нам в командах `install(FILES)` и `install(PROGRAMS)` и служат той же цели. Отметим важную деталь: если пути после ключевого слова `DIRECTORY` не оканчиваются знаком `/`, то последний каталог пути дописывается в конец конечного каталога. Например:

```
install(DIRECTORY aaa DESTINATION /xxx)
```

Эта команда создает каталог `/xxx/aaa` и копирует в него содержимое `aaa`. С другой стороны, следующая команда копирует содержимое `aaa` непосредственно в `/xxx`:

```
install(DIRECTORY aaa/ DESTINATION /xxx)
```

Команда `install(DIRECTORY)` вводит и другие механизмы, отсутствующие для файлов:

- подавление вывода;
- расширенное управление разрешениями;
- фильтрация файлов и каталогов.

Начнем с подавления вывода, `MESSAGE_NEVER`. В этом режиме никакая диагностика во время установки не выводится. Это очень полезно, когда в устанавливаемых каталогах много файлов, печать всех только засоряла бы экран.

Что касается разрешений, `install(DIRECTORY)` поддерживает три режима:

- `USE_SOURCE_PERMISSIONS` задает для установленных файлов такие же разрешения, какие были у исходных. Это работает, только если параметр `FILE_PERMISSIONS` не задан;
- `FILE_PERMISSIONS` позволяет задать разрешения для установленных файлов и каталогов. По умолчанию подразумеваются разрешения `OWNER_WRITE`, `OWNER_READ`, `GROUP_READ` и `WORLD_READ`;
- `DIRECTORY_PERMISSIONS` работает аналогично `FILE_PERMISSIONS`, но дополнительно задается разрешение `EXECUTE` для всех пользователей (потому что `EXECUTE` для каталогов в Unix-подобных системах означает возможность просмотра содержимого каталога).

Заметим, что CMake игнорирует параметры, задающие разрешения, на платформах, которые их не поддерживают. Более точное управление разрешениями возможно путем добавления ключевого слова `PERMISSIONS` после каждого выражения фильтра. Файлы и каталоги, отвечающие этому выражению, получат указанные разрешения.

Теперь поговорим о фильтрах, или «масках». Они управляют тем, какие файлы и каталоги из исходных каталогов устанавливаются. Синтаксис таков:

```
PATTERN <pat> | REGEX <reg> [EXCLUDE] [PERMISSIONS <perm>]
```

Существует два способа сопоставления с фильтром:

- более простое ключевое слово, `PATTERN`, позволяет задать образец, содержащий метасимволы `?` (сопоставляется с одним любым символом) и `*` (сопоставляется с любой строкой). Сопоставленными считаются только пути, оканчивающиеся образцом `<pat>`;
- ключевое слово `REGEX`, поддерживающее регулярные выражения. Оно позволяет производить сопоставление с любой частью пути, хотя можно использовать якорные символы `^`и `$`, обозначающие начало и конец пути.

Кроме того, перед первым фильтром может стоять ключевое слово `FILES_MATCHING`, означающее, что фильтры должны применяться к файлам, а не к каталогам.

Не забывайте о двух подводных камнях:

- `FILES_MATCHING` требует включающего фильтра. Вы можете исключить некоторые файлы, но если вы ничего не включите, то ни один файл не

будет скопирован. Однако все каталоги будут созданы вне зависимости от фильтрации;
- по умолчанию включаются все подкаталоги, вы можете только что-то отфильтровать.

Для каждого способа фильтрации можно задать исключаемые пути с помощью ключевого слова `EXCLUDE` (оно работает, только если `FILES_MATCHING` не используется).

Конкретные разрешения для всех подходящих путей можно задать, добавив после любого фильтра ключевое слово `PERMISSIONS` и список необходимых разрешений. Рассмотрим это на примере установки трех каталогов разными способами. У нас имеются некоторые статические файлы данных, используемые на этапе выполнения:

```
data
- data.csv
```

Также нужны публичные заголовки, находящиеся в каталоге `src` вместе с другими файлами:

```
src
- include
  - calc
    - basic.h
    - ignored
      - empty.file
    - nested
      - calc_extended.h
```

Наконец, нам необходимы два конфигурационных файла на разных уровнях вложенности. Чтобы было интереснее, сделаем содержимое каталога `/etc/calc/` доступным только владельцу файла:

```
etc
- calc
  - nested.conf
- sample.conf
```

Чтобы установить каталог со статическими файлами данных, начнем с самой простой формы команды `install(DIRECTORY)`:

ch14/05-install-directories/CMakeLists.txt (фрагмент)

```
cmake_minimum_required(VERSION 3.26)
project(InstallDirectories CXX)
install(DIRECTORY data/ DESTINATION share/calc)
```

Эта команда просто берет все содержимое каталога `data` и помещает его в `${CMAKE_INSTALL_PREFIX}` и `share/calc`. Заметим, что исходный путь оканчивается знаком /, – это означает, что мы не хотим копировать сам каталог `data`, а только его содержимое.

Второй случай противоположен: мы не стали добавлять завершающий знак /, потому что каталог должен быть включен. Это связано с тем, что мы опираемся на системно-зависимый путь к файлам типам `INCLUDE`, который постулирован модулем `GNUInstallDirs` (обратите внимание, что ключевые слова `INCLUDE` и `EXCLUDE` представляют не связанные между собой концепции):

ch14/05-install-directories/CMakeLists.txt (фрагмент)

```
...
include(GNUInstallDirs)
install(DIRECTORY src/include/calc TYPE INCLUDE
  PATTERN "ignored" EXCLUDE
  PATTERN "calc_extended.h" EXCLUDE
)
```

Дополнительно мы исключили из этой операции два пути: весь каталог `ignored` и все файлы, оканчивающиеся на `calc_extended.h` (вспомните, как работает `PATTERN`).

В третьем случае мы устанавливаем некоторые конфигурационные файлы по умолчанию и задаем для них разрешения:

ch14/05-install-directories/CMakeLists.txt (фрагмент)

```
install(DIRECTORY etc/ TYPE SYSCONF
  DIRECTORY_PERMISSIONS
    OWNER_READ OWNER_WRITE OWNER_EXECUTE
  PATTERN "nested.conf"
    PERMISSIONS OWNER_READ OWNER_WRITE
)
```

Мы не стали дописывать `etc` из исходного пути в конец пути `SYSCONF` (потому что `GNUInstallDirs` уже это сделал), чтобы предотвратить появление дубликатов. Мы задаем два правила установки разрешений: подкаталоги допускают редактирование и просмотр содержимого только владельцем, а файлы, оканчивающиеся на `nested.conf`, может редактировать лишь владелец.

Установка каталогов охватывает разные случаи, но в более сложных сценариях (например, конфигурирование после установки) могут потребоваться внешние инструменты. Как их интегрировать в проект?

Вызов скриптов в процессе установки

Если вы когда-нибудь устанавливали разделяемую библиотеку в Unix-подобную систему, то, наверное, помните, что динамическому компоновщи-

ку нужно сказать, чтобы он просматривал пользующиеся доверием каталоги и строил свой кеш с помощью программы `ldconfig` (см. раздел «Для дополнительного чтения»). Для реализации полностью автоматической установки CMake предлагает режимы `install(SCRIPT)` и `install(CODE)`. Ниже приведена их сигнатура:

```
install([[SCRIPT <file>] [CODE <code>]]
        [ALL_COMPONENTS | COMPONENT <component>]
        [EXCLUDE_FROM_ALL] [...]
)
```

Выберите один из режимов `SCRIPT` и `CODE` и задайте необходимые аргументы – путь к скрипту CMake, который нужно запустить, или фрагмент кода CMake, который нужно выполнить в процессе установки. Для иллюстрации модифицируем пример `02-install-targets` и построим разделяемую библиотеку.

ch14/06-install-code/src/CMakeLists.txt

```
add_library(calc SHARED basic.cpp)
target_include_directories(calc INTERFACE include)
target_sources(calc PUBLIC FILE_SET HEADERS
                           BASE_DIRS include
                           FILES include/calc/basic.h
)
```

Изменим тип артефакта с `ARCHIVE` на `LIBRARY` в скрипте установки, а затем добавим запуск `ldconfig`:

ch14/06-install-code/CMakeLists.txt (фрагмент)

```
install(TARGETS calc LIBRARY FILE_SET HEADERS))
if (UNIX)
  install(CODE "execute_process(COMMAND ldconfig)")
endif()
```

Условие `if()` гарантирует, что команда соответствует операционной системе (`ldconfig` не будет работать в Windows или macOS). Код CMake после слова `CODE` должен быть синтаксически правильным (ошибки выявятся только на этапе установки).

После выполнения команды установки подтвердим, что все прошло успешно, распечатав кешированные библиотеки:

```
# cmake -S <source-tree> -B <build-tree>
# cmake --build <build-tree>
# cmake --install <build-tree>
-- Install configuration: ""
-- Installing: /usr/local/lib/libcalc.so
-- Installing: /usr/local/include/calc/basic.h
```

```
# ldconfig -p | grep libcalc
        libcalc.so (libc6,x86-64) => /usr/local/lib/libcalc.so
```

Оба режима, `SCRIPT` и `CODE`, поддерживают генераторные выражения, что добавляет гибкости этой команде. Ее можно использовать для разных целей: печати сообщений пользователю, проверки успешности установки, расширенного конфигурирования, подписания файлов и т. д.

Далее займемся управлением зависимостями времени выполнения, одной из самых новых возможностей CMake.

Установка зависимостей времени выполнения

Мы рассмотрели почти все виды допускающих установку артефактов и соответствующие команды. Осталось обсудить только зависимости времени выполнения. Исполняемые файлы и разделяемые библиотеки часто зависят от других библиотек, которые должны присутствовать в системе и динамически загружаются на этапе инициализации программы. Начиная с версии 3.21 CMake умеет строить список таких требуемых библиотек для каждой цели и запоминать их местоположения на этапе сборки путем ссылки на соответствующие секции двоичного файла. Впоследствии этот список можно применить для установки артефактов времени выполнения в систему для будущего использования.

Для определенной в проекте цели это требует двух шагов:

```
install(TARGETS ... RUNTIME_DEPENDENCY_SET <set-name>)
install(RUNTIME_DEPENDENCY_SET <set-name> <arg>...)
```

Того же эффекта можно достичь и одной командой:

```
install(TARGETS ... RUNTIME_DEPENDENCIES <arg>...)
```

Если цель импортирована, а не определена в проекте, то ее зависимости времени выполнения можно установить следующим образом:

```
install(IMPORTED_RUNTIME_ARTIFACTS <target>...)
```

Этот фрагмент можно расширить, задав аргумент `RUNTIME_DEPENDENCY_SET <set-name>`, чтобы создать именованную ссылку, которая позже будет использована в команде `install(RUNTIME_DEPENDENCY_SET)`.

Если вам кажется, что эта возможность может найти применение в вашем проекте, рекомендую заглянуть в документацию по команде `install()` для получения дополнительных сведений.

Теперь, когда мы познакомились со всеми способами установки файлов в систему, посмотрим, как превратить их в понятный операционной системе пакет, доступный другим проектам CMake.

Создание повторно используемых пакетов

Мы много раз использовали команду `find_package()` в предыдущих главах и имели случай убедиться в ее удобстве и простоте. Чтобы эта команда могла найти наш проект, мы должны выполнить несколько шагов – тогда CMake будет рассматривать его как полноценный пакет:

1. Сделать цели перемещаемыми.
2. Установить *файл экспорта целей* в стандартное место.
3. Создать *конфигурационный файл* для пакета.
4. Сгенерировать *файл версии* для пакета.

Начнем с самого начала: почему цели должны быть перемещаемыми и как этого добиться?

Проблемы перемещаемости целей

Установка решает много проблем, но не обходится без сложностей. Переменная `CMAKE_INSTALL_PREFIX` платформенно-зависима и может быть задана пользователем на этапе установки посредством аргумента командной строки `--installprefix`. Проблема в том, что файлы экспорта целей генерируются до установки, на этапе сборки, когда конечное местоположение устанавливаемых артефактов еще неизвестно. Рассмотрим следующий код:

ch14/03-install-targets-legacy/src/CMakeLists.txt

```
add_library(calc STATIC basic.cpp)
target_include_directories(calc INTERFACE include)
set_target_properties(calc PROPERTIES
  PUBLIC_HEADER src/include/calc/basic.h
)
```

В этом примере мы добавили каталог `include` в список каталогов включаемых файлов для цели `calc`. Поскольку это относительный путь, при генерировании экспортируемой цели CMake неявно добавляет в его начало содержимое переменной `CMAKE_CURRENT_SOURCE_DIR`, которая указывает на каталог, в котором находится этот списковый файл.

И вот тут-то и возникает проблема: после установки проект не должен полагаться на файлы из дерева исходных кодов или сборки. Все, включая заголовки библиотек, копируется в разделяемый каталог, например `/usr/lib/calc/` в Linux. Цель, которая была определена в этом фрагменте, непригодна для использования в другом проекте, потому что ее каталог `include` по-прежнему указывает на дерево исходного кода.

CMake решает эту проблему *телеги впереди лошади* с помощью генераторных выражений, которые заменяются своим аргументом или пустой строкой в зависимости от контекста:

- `$<BUILD_INTERFACE:...>`: результатом вычисления является аргумент «...» для регулярной сборки, но пустая строка для установки;
- `$<INSTALL_INTERFACE:...>`: результатом вычисления является аргумент «...» для установки, но пустая строка для регулярной сборки;
- `$<BUILD_LOCAL_INTERFACE:...>`: результатом вычисления является аргумент «...» при использовании другой целью в той же сборочной системе (добавлено в версии CMake 3.26).

Эти выражения позволяют отложить решение об используемом пути до более поздних этапов процесса: сборки и установки. Вот как это работает на практике.

ch14/07-install-export-legacy/src/CMakeLists.txt (фрагмент)

```
add_library(calc STATIC basic.cpp)
target_include_directories(calc INTERFACE
  "$<BUILD_INTERFACE:${CMAKE_CURRENT_SOURCE_DIR}/include>"
  "$<INSTALL_INTERFACE:${CMAKE_INSTALL_INCLUDEDIR}>"
)
set_target_properties(calc PROPERTIES
  PUBLIC_HEADER "include/calc/basic.h"
)
```

В команде `target_include_directories()` обратите внимание на последние два аргумента. Используемые генераторные выражения взаимно исключают друг друга, т. е. на последнем шаге будет использован только один из аргументов, а другой стерт.

Для регулярных сборок свойство `INTERFACE_INCLUDE_DIRECTORIES` цели `calc` будет заменено значением первого аргумента:

```
"/root/examples/ch14/07-install-export/src/include" ""
```

С другой стороны, при установке оно будет заменено значением второго аргумента:

```
"" "/usr/lib/calc/include"
```



Кавычек в окончательном значении нет, здесь они добавлены, чтобы как-то обозначить пустой текст.

Что касается переменной `CMAKE_INSTALL_PREFIX`: ее не следует использовать как компонент путей, заданных в целях. Она вычисляется на этапе сборки, в результате чего путь оказывается абсолютным и потенциально отличным от заданного на этапе установки (если используется параметр `--install-prefix`). Вместо этого используйте генераторное выражение `$<INSTALL_PREFIX>`:

```
target_include_directories(my_target PUBLIC
  $<INSTALL_INTERFACE:$<INSTALL_PREFIX>/include/MyTarget>
)
```

Или еще лучше – используйте относительные пути, в начало которых будет добавлен нужный префикс установки:

```
target_include_directories(my_target PUBLIC
  $<INSTALL_INTERFACE:include/MyTarget>
)
```

За дополнительной информацией и примерами обращайтесь к документации (ссылка приведена в разделе «Для дополнительного чтения»).

Теперь, когда наши цели *совместимы с установкой*, мы можем безопасно сгенерировать и установить файлы экспорта целей.

Установка файлов экспорта целей

Мы уже упоминали файлы экспорта целей в разделе «Экспорт без установки». Процесс установки файлов экспорта целей очень похож, как и синтаксис команды их создания:

```
install(EXPORT <export-name> DESTINATION <dir>
        [NAMESPACE <namespace>] [[FILE <name>.cmake]|
        [PERMISSIONS permissions...]
        [CONFIGURATIONS [Debug|Release|...]]
        [EXPORT_LINK_INTERFACE_LIBRARIES]
        [COMPONENT <component>]
        [EXCLUDE_FROM_ALL])
```

Это гибрид простой команды `export(EXPORT)` с другими командами `install()` (назначение параметров аналогично). Напомним, что она создает и устанавливает файл экспорта целей для именованного экспорта, который должен быть определен командой `install(TARGETS)`. Ключевое отличие в том, что сгенерированный файл экспорта будет содержать пути целей, вычисленные генераторным выражением `INSTALL_INTERFACE`, а в случае команды `export(EXPORT)` используется выражение `BUILD_INTERFACE`. Это означает, что нужно внимательно следить за включаемыми и другими файлами с относительными ссылками.

Повторим еще раз, что начиная с версии CMake 3.23 это уже не является проблемой, если правильно используется свойство `FILE_SET HEADERS`. Посмотрим, как сгенерировать и установить файл экспорта для целей из примера `ch14/02-installexport`. Для этого мы должны вызвать `install(EXPORT)` после команды `install(TARGETS)`:

ch14/07-install-export/src/CMakeLists.txt

```
add_library(calc STATIC basic.cpp)
target_sources(calc
  PUBLIC FILE_SET HEADERS BASE_DIRS include
  FILES "include/calc/basic.h"
)

include(GNUInstallDirs)
install(TARGETS calc EXPORT CalcTargets ARCHIVE FILE_SET HEADERS)
install(EXPORT CalcTargets
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
  NAMESPACE Calc::
)
```

Обратите внимание на ссылку на экспортированное имя `CalcTargets` в `install(EXPORT)`. Выполнение `cmake --install` в дереве сборки приведет к генерированию файла экспорта в указанном месте:

```
...
-- Installing: /usr/local/lib/calc/cmake/CalcTargets.cmake
-- Installing: /usr/local/lib/calc/cmake/CalcTargets-noconfig.cmake
```

Если вы хотите переопределить подразумеваемое по умолчанию имя файла экспорта целей (`<export name>.cmake`), добавьте аргумент `FILE new-name.cmake`, чтобы задать другое имя (оно должно оканчиваться на `.cmake`).

Не путайте – *файл экспорта целей* не является *конфигурационным файлом*, поэтому пока что вы не можете использовать команду `find_package()` для поиска установленных целей. Однако при необходимости можно непосредственно включить экспортированные файлы командой `include()`. Итак, как же нам определить пакет, который может быть потреблен другими проектами? Разберемся.

Написание простых конфигурационных файлов

Полное определение пакета состоит из файлов экспорта целей, *конфигурационного файла* пакета и *файла версии* пакета. Но технически для работы команды `find_package()` достаточно конфигурационного файла. Он играет роль определения пакета и отвечает за предоставление всех функций и макросов пакета, проверку требований, нахождение зависимостей и включение файлов экспорта целей.

Как уже было сказано, пользователь может установить ваш пакет в любое место системы командой

```
# cmake --install <build tree> --install-prefix=<path>
```

Префикс определяет, куда будут скопированы установленные файлы. Для поддержки этой идеи вы должны обеспечить следующее:

- пути в свойствах целей должны быть перемещаемыми (как описано в разделе «Проблемы перемещаемости целей»);
- пути, встречающиеся в конфигурационном файле, должны вычисляться относительно него.

Чтобы использовать пакеты, установленные в нестандартное место, проект-потребитель должен задать `<installation path>` с помощью переменной `CMAKE_PREFIX_PATH` на этапе конфигурирования:

```
# cmake -B <build tree> -DCMAKE_PREFIX_PATH=<installation path>
```

Команда `find_package()` просматривает список путей, описанный в документации (см. раздел «Для дополнительного чтения») платформенно-зависимым способом. В частности, в Windows и в Unix проверяются пути вида

```
<prefix>/<name>*/(lib/<arch>|lib*|share)/<name>*/(cmake|CMake)
```

Это означает, что установка конфигурационного файла в каталог `lib/calc/cmake` должна работать. Кроме того, CMake требует, чтобы конфигурационные файлы назывались `<PackageName>-config.cmake` или `<PackageName>Config.cmake`, иначе они не будут найдены.

Добавим установку конфигурационного файла в пример `06-install-export`:

ch14/09-config-file/CMakeLists.txt (фрагмент)

```
...
install(EXPORT CalcTargets
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
  NAMESPACE Calc::
)
install(FILES "CalcConfig.cmake"
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
)
```

Эта команда устанавливает `CalcConfig.cmake` из того же самого исходного каталога (результатом вычисления `CMAKE_INSTALL_LIBDIR` будет правильный путь `lib` для данной платформы).

Простейший конфигурационный файл состоит из одной строки, включающей файл экспорта целей:

ch14/09-config-file/CalcConfig.cmake

```
include("${CMAKE_CURRENT_LIST_DIR}/CalcTargets.cmake")
```

`CMAKE_CURRENT_LIST_DIR` ссылается на каталог, в котором находится конфигурационный файл. Так как `CalcConfig.cmake` и `CalcTargets.cmake` в нашем примере устанавливаются в один и тот же каталог (определенный `install(EXPORT)`), файл экспорта целей будет включен корректно.

Чтобы убедиться в возможности использования нашего пакета, создадим простой проект с одним списковым файлом.

ch14/10-find-package/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(FindCalcPackage CXX)
find_package(Calc REQUIRED)
include(CMakePrintHelpers)
message("CMAKE_PREFIX_PATH: ${CMAKE_PREFIX_PATH}")
message("CALC_FOUND: ${Calc_FOUND}")
cmake_print_properties(TARGETS "Calc::calc" PROPERTIES
  IMPORTED_CONFIGURATIONS
  INTERFACE_INCLUDE_DIRECTORIES
)
```

Для тестирования соберем и установим пример `09-config-file` в один каталог, а затем соберем `10-find-package`, сославшись на него с помощью аргумента `DCMAKE_PREFIX_PATH`:

```
# cmake -S <source-tree-of-08> -B <build-tree-of-08>
# cmake --build <build-tree-of-08>
# cmake --install <build-tree-of-08>
# cmake -S <source-tree-of-09> -B <build-tree-of-09>
        -DCMAKE_PREFIX_PATH=<build-tree-of-08>
```

В результате будут напечатаны следующие сообщения (все маркеры `<*_tree-of_>` заменены настоящими путями):

```
CMAKE_PREFIX_PATH: <build-tree-of-08>
CALC_FOUND: 1
--
Properties for TARGET Calc::calc:
   Calc::calc.IMPORTED_CONFIGURATIONS = "NOCONFIG"
   Calc::calc.INTERFACE_INCLUDE_DIRECTORIES = "<build-tree-of-08>/include"
-- Configuring done
-- Generating done
-- Build files have been written to: <build-tree-of-09>
```

Этот вывод показывает, что файл `CalcTargets.cmake` был найден и корректно включен, а путь к каталогу `include` начинается указанным префиксом. Это решение годится для простых случаев упаковки. Теперь посмотрим, как действовать в более сложных ситуациях.

Создание продвинутых конфигурационных файлов

Если необходимо работать с несколькими файлами экспорта целей, то бывает полезно включать в конфигурационный файл макросы. Служебный модуль `CMakePackageConfigHelpers` открывает доступ к команде `configure_package_config_file()`. Чтобы воспользоваться ей, предоставим файл шаблона, в который будут интерполированы переменные CMake для генерирования конфигурационного файла с определениями двух макросов:

- `set_and_check(<переменная> <путь>)`: работает как `set()`, но проверяет, что `<путь>` существует, и завершается с ошибкой `FATAL_ERROR`, если это не так. Рекомендуется использовать в конфигурационных файлах для раннего обнаружения неправильных путей;
- `check_required_components(<PackageName>)`: добавляется в конец конфигурационного файла. Проверяет, все ли компоненты, затребованные пользователем в команде `find_package(<пакет> REQUIRED <компонент>)`, найдены.

Пути для сложных деревьев каталогов можно подготовить к установке на этапе генерирования конфигурационного файла. Команда имеет такую сигнатуру:

```
configure_package_config_file(<template> <output>
  INSTALL_DESTINATION <path>
  [PATH_VARS <var1> <var2> ... <varN>]
  [NO_SET_AND_CHECK_MACRO]
  [NO_CHECK_REQUIRED_COMPONENTS_MACRO]
  [INSTALL_PREFIX <path>]
)
```

В файл `<template>` будут интерполированы переменные, после чего он будет сохранен в каталоге `<output>`. Путь `INSTALL_DESTINATION` служит для преобразования путей, хранящихся в списке `PATH_VARS`, в пути относительно конечного места установки. Параметр `INSTALL_PREFIX` может быть задан в качестве базового пути, относительно которого вычисляется `INSTALL_DESTINATION`.

Параметры `NO_SET_AND_CHECK_MACRO` и `NO_CHECK_REQUIRED_COMPONENTS_MACRO` просят CMake не добавлять определения этих макросов в сгенерированный конфигурационный файл. Посмотрим, как такое генерирование работает на практике, для чего расширим пример `07-install-export`:

ch14/11-advanced-config/CMakeLists.txt (фрагмент)

```
...
install(EXPORT CalcTargets
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
  NAMESPACE Calc::
)
include(CMakePackageConfigHelpers)
set(LIB_INSTALL_DIR ${CMAKE_INSTALL_LIBDIR}/calc)
configure_package_config_file(
```

```
    ${CMAKE_CURRENT_SOURCE_DIR}/CalcConfig.cmake.in
    "${CMAKE_CURRENT_BINARY_DIR}/CalcConfig.cmake"
    INSTALL_DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
    PATH_VARS LIB_INSTALL_DIR
)
install(FILES "${CMAKE_CURRENT_BINARY_DIR}/CalcConfig.cmake"
    DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
)
```

Здесь мы:

1. Используем `include()`, чтобы включить служебный модуль со вспомогательными функциями.
2. Используем `set()` для присваивания значения переменной, которая будет применяться при создании перемещаемого пути.
3. Генерируем конфигурационный файл `CalcConfig.cmake` для дерева сборки, используя шаблон `CalcConfig.cmake.in` и указав `LIB_INSTALL_DIR` в качестве имени переменной, которое нужно вычислять относительно `INSTALL_DESTINATION` или `${CMAKE_INSTALL_LIBDIR}/calc/cmake`.
4. Передаем конфигурационный файл, сгенерированный для дерева сборки, команде `install(FILE)`.

Заметим, что путь в параметре `DESTINATION` при вызове `install(FILES)` и путь в параметре `INSTALL_DESTINATION` при вызове `configure_package_config_file()` совпадают, что гарантирует правильность вычисления относительного пути внутри конфигурационного файла.

Наконец, нам нужен шаблон `config` (имена таких файлов обычно оканчиваются суффиксом `.in`).

ch14/11-advanced-config/CalcConfig.cmake.in

```
@PACKAGE_INIT@
set_and_check(CALC_LIB_DIR "@PACKAGE_LIB_INSTALL_DIR@")
include("${CALC_LIB_DIR}/cmake/CalcTargets.cmake")
check_required_components(Calc)
```

Этот шаблон начинается маркером `@PACKAGE_INIT@`. Генератор подставит вместо него определения макросов `set_and_check` и `check_required_components`.

В следующей строке в параметр `CALC_LIB_DIR` записывается путь, переданный в маркере `@PACKAGE_LIB_INSTALL_DIR@`. CMake подставит вместо него значение `$LIB_INSTALL_DIR`, заданное в списковом файле, но вычисленное относительно пути установки. Затем этот путь используется в команде `include()`, чтобы включить *файл экспорта целей*. Наконец, макрос `check_required_components()` проверяет, все ли компоненты, требующиеся в проекте, где используется этот пакет, найдены. Эта команда рекомендуется, даже если в пакете вообще нет компонентов, дабы гарантировать, что пользователь предъявляет только поддерживаемые требования. В противном случае он может ошибочно подумать, что успешно добавил компоненты (которые, возможно, присутствуют только в более новых версиях пакета).

Сгенерированный таким образом конфигурационный файл `CalcConfig.cmake` выглядит следующим образом:

```
#### Получено в результате подстановки configure_package_config_file()
  вместо @PACKAGE_INIT@ #######
#### Любые изменения, внесенные в этот файл, будут затерты при
  следующем запуске CMake ####
#### На вход был подан шаблон CalcConfig.cmake.in #####

get_filename_component(PACKAGE_PREFIX_DIR
  "${CMAKE_CURRENT_LIST_DIR}/../../../" ABSOLUTE)

macro(set_and_check _var _file)
  # ... опущено для краткости
endmacro()
macro(check_required_components _NAME)
  # ... опущено для краткости
endmacro()

####################################################################
set_and_check(CALC_LIB_DIR "${PACKAGE_PREFIX_DIR}/lib/calc")
include("${CALC_LIB_DIR}/cmake/CalcTargets.cmake")
check_required_components(Calc)
```

На рис. 14.1 показано, как различные файлы пакетов связаны между собой.

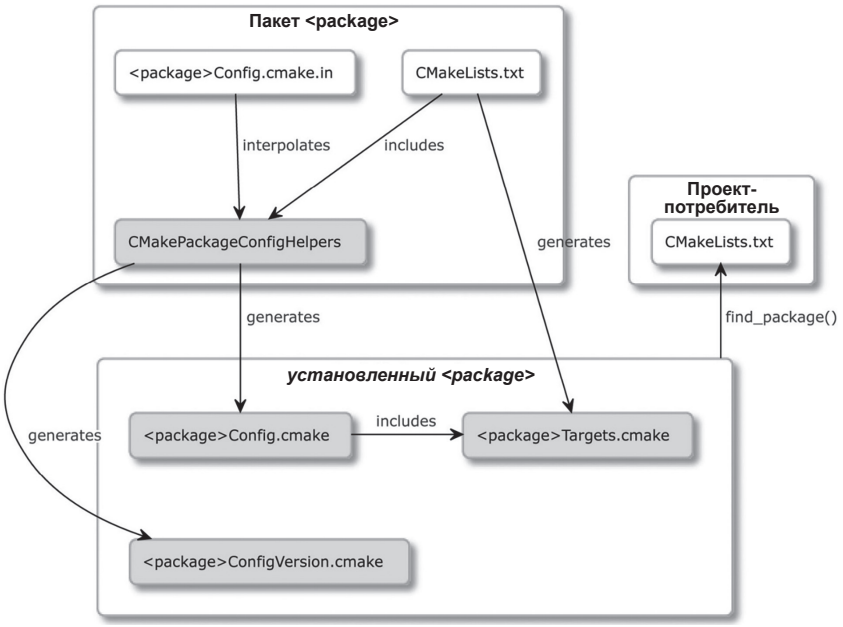


Рис. 14.1. Структура файлов для продвинутых пакетов

Поиск всех требуемых подзависимостей пакета должен быть произведен в конфигурационном файле пакета. Для этого можно вызвать макрос `find_dependency()` из служебного модуля `CMakeFindDependencyMacro`. Как им пользоваться, мы объясняли в главе 9.

Определения всех макросов или функций, раскрываемые проекту-потребителю, должны находиться в отдельном файле, включаемом в конфигурационный файл пакета. Интересно, что модуль `CMakePackageConfigHelpers` также помогает генерировать файлы версий пакетов. Эту тему мы и рассмотрим далее.

Генерирование файлов версий пакетов

По мере того как ваш пакет развивается, обретает новые функции и отказывается от старых, становится все важнее отслеживать эти изменения в журнале изменений, который был бы доступен разработчикам, пользующимся вашим пакетом. Когда требуется какая-то конкретная возможность, разработчик может указать минимальную версию, которая ее поддерживает, при вызове `find_package()`:

```
find_package(Calc 1.2.3 REQUIRED)
```

Тогда CMake произведет поиск в конфигурационном файле `Calc` и проверит, что *файл версии* с именем `<configfile>-version.cmake` или `<config-file>Version.cmake` присутствует в том же самом каталоге (например, `CalcConfigVersion.cmake`). Этот файл содержит сведения о версии и о совместимости с другими версиями. Например, даже если в вашей системе не установлена конкретная версия 1.2.3, возможно, установлена версия 1.3.5, для которой известно, что она совместима с более старыми версиями. CMake примет этот пакет, будучи уверена в его обратной совместимости.

Для генерирования файлов версий пакетов можно использовать служебный модуль `CMakePackageConfigHelpers`, вызвав из него функцию `write_basic_package_version_file()`:

```
write_basic_package_version_file(
  <filename> [VERSION <ver>]
  COMPATIBILITY <AnyNewerVersion | SameMajorVersion |
                 SameMinorVersion | ExactVersion>
  [ARCH_INDEPENDENT]
)
```

Первым делом задайте имя артефакта `<filename>`; убедитесь только, что оно не нарушает рассмотренных выше правил именования. Факультативно можно передать явную версию `VERSION` (в формате major.minor.patch). Если этого не сделать, то будет использоваться версия, указанная в команде `project()` (а если она не указана, то возникнет ошибка).

Ключевое слово COMPATIBILITY может принимать следующие значения:

- ExactVersion – все три компонента версии должны совпадать, диапазоны версий (например, find_package(<package> 1.2.8...1.3.4)) не допускаются;
- SameMinorVersion – принимается, если первые два компонента совпадают (номер исправления patch игнорируется);
- SameMajorVersion – принимается, если первые компоненты совпадают (minor и patch игнорируются);
- AnyNewerVersion – вопреки имени принимается любая более ранняя версия (например, версия 1.4.2 совместима с командой find_package(<package> 1.2.8)).

Для архитектурно-зависимых пакетов требуется точное совпадение архитектур. Но для архитектурно-независимых пакетов (например, библиотек, состоящих только из заголовочных файлов, или пакетов макросов) можно задать ключевое слово ARCH_INDEPENDENT и тем самым опустить эту проверку.

Ниже приведен практический пример задания *файла версии* для проекта, начатого в 07-install-export:

ch14/12-version-file/CMakeLists.txt (фрагмент)

```
cmake_minimum_required(VERSION 3.26)
project(VersionFile VERSION 1.2.3 LANGUAGES CXX)
...
include(CMakePackageConfigHelpers)
write_basic_package_version_file(
  "${CMAKE_CURRENT_BINARY_DIR}/CalcConfigVersion.cmake"
  COMPATIBILITY AnyNewerVersion
)
install(FILES "CalcConfig.cmake"
  "${CMAKE_CURRENT_BINARY_DIR}/CalcConfigVersion.cmake"
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
)
```

Для удобства мы указываем версию пакета в начале файла, в команде project(), заменив короткий синтаксис project(<имя> <языки>) более длинным, с ключевым словом LANGUAGE.

После включения служебного модуля мы генерируем файл версии и устанавливаем его вместе с CalcConfig.cmake. Опустив ключевое слово VERSION, мы используем переменную PROJECT_VERSION. С помощью ключевого слова COMPATIBILITY AnyNewerVersion указано, что пакет обратно совместим. Файл версии пакета устанавливается туда же, куда CalcConfig.cmake. Вот и все – наш пакет полностью сконфигурирован.

На этом мы завершаем тему создания пакетов. Теперь мы знаем, как обеспечить перемещаемость и почему это важно, как устанавливать *файлы экспорта целей* и как писать *конфигурационные файлы* и *файлы версий*. В следующем разделе мы рассмотрим компоненты и их использование вместе с пакетами.

Определение компонентов

Начнем с потенциальной путаницы вокруг термина **компонент**. Рассмотрим полную сигнатуру команды `find_package()`:

```
find_package(<PackageName>
             [version] [EXACT] [QUIET] [MODULE] [REQUIRED]
             [[COMPONENTS] [components...]]
             [OPTIONAL_COMPONENTS components...]
             [NO_POLICY_SCOPE]
)
```

Важно не путать компоненты, упомянутые здесь, с ключевым словом `COMPONENT`, используемым в команде `install()`. Хотя то и другое называется компонентами, это разные понятия, и относиться к ним нужно по-разному. Мы изучим этот вопрос подробнее в следующих подразделах.

Как использовать компоненты в find_package()

При вызове `find_package()` с указанием списка `COMPONENTS` или `OPTIONAL_COMPONENTS` мы сообщаем CMake, что нас интересуют только пакеты, которые предоставляют эти компоненты. Однако важно понимать, что проверять это требование должен сам пакет. Если поставщик пакета не реализовал необходимые проверки в конфигурационном файле, как описано в разделе «Создание продвинутых конфигурационных файлов», то процесс не будет работать, как ожидается.

Запрошенные компоненты (как обязательные, так и факультативные) передаются конфигурационному файлу посредством переменной `<package>_FIND_COMPONENTS`. Для каждого обязательного компонента устанавливается переменная `<package>_FIND_REQUIRED_<component>`. Авторы пакета могли бы написать макрос, который просматривает этот список и проверяет, что все обязательные компоненты присутствуют, но особой необходимости в этом нет. Эту задачу решает функция `check_required_components()`. Конфигурационный файл должен установить переменную `<package>_<component>_FOUND`, если обязательный компонент найден. Затем в конце файла макрос проверяет, все ли переменные, соответствующие обязательным компонентам, установлены.

Как используются компоненты в команде install()

Не все порожденные артефакты должны быть установлены в каждом случае. Например, проект мог бы установить статические библиотеки и публичные заголовки для разработки, но по умолчанию устанавливает только разделяемую библиотеку времени выполнения. Чтобы реализовать такое двой-

ственное поведение, артефакты можно сгруппировать под общим именем, воспользовавшись ключевым словом COMPONENT, доступным во всех командах install(). Пользователи, желающие ограничиться установкой конкретных компонентов, могут это сделать, выполнив следующую чувствительную к регистру команду:

```
cmake --install <build tree>
      --component=<component1 name> --component=<component2 name>
```

Сопоставление ключевого слова COMPONENT артефакту автоматически не исключает его из установки по умолчанию. Чтобы такое исключение произошло, нужно добавить ключевое слово EXCLUDE_FROM_ALL.

Проиллюстрируем это на примере кода.

ch14/13-components/CMakeLists.txt (фрагмент)

```
install(TARGETS calc EXPORT CalcTargets
  ARCHIVE
    COMPONENT lib
  FILE_SET HEADERS
    COMPONENT headers
)
install(EXPORT CalcTargets
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
  NAMESPACE Calc::
  COMPONENT lib
)
install(CODE "MESSAGE(\"Installing 'extra' component\")"
  COMPONENT extra
  EXCLUDE_FROM_ALL
)
```

В этих командах install определены следующие компоненты.

- lib: содержит статическую библиотеку и файлы экспорта целей. Устанавливается по умолчанию.
- headers: содержит заголовочные файлы C++. Также устанавливается по умолчанию.
- extra: исполняет код печати сообщения. Не устанавливается по умолчанию.

Повторим еще раз.

- cmake --install без аргумента --component устанавливает компоненты lib и headers.
- cmake --install --component headers устанавливает только публичные заголовки.
- cmake --install --component extra печатает сообщение, которое в противном случае отсутствует (его печати препятствует ключевое слово EXCLUDE_FROM_ALL).

Если ключевое слово COMPONENT не задано для устанавливаемого артефакта, то по умолчанию подразумевается компонент Unspecified, определенный переменной CMAKE_INSTALL_DEFAULT_COMPONENT_NAME.



Поскольку нет никакого способа перечислить все доступные компоненты из командной строки cmake, тщательное документирование компонентов вашего пакета может оказаться весьма ценным для пользователей. Эту информацию разумно поместить в файл INSTALL "READM".

Если cmake вызывается с аргументом --component, после которого указано имя несуществующего компонента, то команда завершится успешно, без ошибок и предупреждений, но ничего не установит.

Разбиение установки на компоненты позволяет пользователям выбирать, какие части пакета устанавливать. Теперь обратимся к управлению символическими ссылками для версионированных разделяемых библиотек, это полезно для оптимизации процесса установки.

Управление символическими ссылками для версионированных разделяемых библиотек

На целевой платформе установки могут использоваться символические ссылки, которые помогают компоновщику обнаруживать установленную на данный момент версию разделяемой библиотеки. После создания символической ссылки lib<name>.so на файл lib<name>.so.1 можно произвести компоновку с этой библиотекой, передав компоновщику аргумент -l<name>.

Блок CMake install(TARGETS <target> LIBRARY) обрабатывает создание таких символических ссылок при необходимости. Однако мы можем решить перенести этот шаг в другую команду, добавив в этот блок ключевое слово NAMELINK_SKIP:

```
install(TARGETS <target> LIBRARY
        COMPONENT cmp NAMELINK_SKIP)
```

Чтобы поручить формирование символических ссылок другому компоненту (а не запретить его полностью), мы можем повторить команду install() для той же цели и указать другой компонент, сопроводив его ключевым словом NAMELINK_ONLY:

```
install(TARGETS <target> LIBRARY
        COMPONENT lnk NAMELINK_ONLY)
```

Того же эффекта можно достичь с помощью ключевого слова NAMELINK_COMPONENT:

```
install(TARGETS <target> LIBRARY
        COMPONENT cmp NAMELINK_COMPONENT lnk)
```

Теперь, рассмотрев конфигурирование автоматической установки, мы можем предоставить пользователям заранее собранные артефакты, воспользовавшись инструментом CPack, входящим в состав CMake.

Упаковка с помощью CPack

Сборка проектов из исходного кода имеет свои преимущества, но может оказаться сложной и занимать много времени, что вряд ли понравится конечным пользователям, особенно если они не являются разработчиками. Более удобный способ распространения – двоичные пакеты, которые содержат уже откомпилированные артефакты и другие необходимые статические файлы. CMake поддерживает генерирование таких пакетов с помощью командного инструмента `cpack`.

Для генерирования пакета выберите подходящий для своей платформы генератор и тип упаковки. Не путайте генераторы пакетов с генераторами сборочных систем типа Makefile в Unix или Visual Studio в Windows.

В следующей таблице перечислены имеющиеся генераторы пакетов.

Таблица 14.3. Генераторы пакетов

Название генератора	Типы порождаемых файлов	Платформа
Archive	7Z, 7zip – (`.7z`) TBZ2 (`.tar.bz2`) TGZ (`.tar.gz`) TXZ (`.tar.xz`) TZ (`.tar.Z`) TZST (`.tar.zst`) ZIP (`.zip`)	Кросс-платформенный
Bundle	Пакет macOs (`.bundle`)	macOS
Cygwin	Пакеты Cygwin	Cygwin
DEB	Пакеты Debian (`.deb`)	Linux
External	JSON-файл (`.json`) для сторонних упаковщиков	Кросс-платформенный
FreeBSD	PKG (`.pkg`)	*BSD, Linux, macOS
IFW	Двоичные файлы для установщика QT	Linux, Windows, macOS
NSIS	Двоичные (`.exe`)	Windows
NuGet	Пакет NuGet (`.nupkg`)	Windows
productbuild	PKG (`.pkg`)	macOS
RPM	RPM (`.rpm`)	Linux
WIX	Microsoft Installer (`.msi`)	Windows

Большинство этих генераторов допускают конфигурирование в широких пределах. Мы не можем вдаваться во все детали, но желающие могут найти

информацию в разделе «Для дополнительного чтения». Мы же сосредоточимся на общем случае.

Для использования CPack сконфигурируйте установку своего проекта с помощью подходящих команд `install()` и соберите проект. Созданный в дереве сборки файл `cmake_install.cmake` используется CPack для подготовки двоичных пакетов на основе файла `CPackConfig.cmake`. Этот файл можно создать вручную, но проще воспользоваться командой `include(CPack)` в списковом файле проекта. Она генерирует конфигурацию в дереве сборки и подставляет значения по умолчанию там, где необходимо.

Дополним пример `13-components`, включив в него работу с CPack.

ch14/14-cpack/CMakeLists.txt (фрагмент)

```
cmake_minimum_required(VERSION 3.26)
project(CPackPackage VERSION 1.2.3 LANGUAGES CXX)
include(GNUInstallDirs)
add_subdirectory(src bin)
install(...)
install(...)
install(...)
set(CPACK_PACKAGE_VENDOR "Rafal Swidzinski")
set(CPACK_PACKAGE_CONTACT "email@example.com")
set(CPACK_PACKAGE_DESCRIPTION "Simple Calculator")
include(CPack)
```

Модуль CPack извлекает следующие переменные из команды `project()`:

- `CPACK_PACKAGE_NAME`;
- `CPACK_PACKAGE_VERSION`;
- `CPACK_PACKAGE_FILE_NAME`.

В переменной `CPACK_PACKAGE_FILE_NAME` хранится структура имени пакета:

```
$CPACK_PACKAGE_NAME-$CPACK_PACKAGE_VERSION-$CPACK_SYSTEM_NAME
```

Здесь `CPACK_SYSTEM_NAME` – имя целевой ОС: `Linux`, `win32` и т. д. Например, если выполнить генератор ZIP в Debian, то CPack сгенерирует файл с именем `CPackPackage-1.2.3-Linux.zip`.

Для генерирования пакетов после сборки проекта перейдите в дерево сборки и выполните команду

```
cpack [<options>]
```

CPack читает параметры из файла `CPackConfig.cmake`, но вы можете переопределить их.

- `-G <generators>`: разделенный точками с запятой список генераторов пакетов. Значение по умолчанию можно задать в `CPackConfig.cmake` в переменной `CPACK_GENERATOR`.

- `-C <configs>`: разделенный точками с запятой список конфигураций сборки (debug, release) для генерирования пакетов (требуется для генераторов сборочных систем, поддерживающих несколько конфигураций).
- `-D <var>=<value>`: переопределяет переменную `var`, заданную в файле `CPackConfig.cmake`.
- `--config <config-file>`: использование указанного конфигурационного файла вместо подразумеваемого по умолчанию `CPackConfig.cmake`.
- `--verbose, -V`: подробный вывод.
- `-P <packageName>`: переопределяет имя пакета.
- `-R <packageVersion>`: переопределяет версию пакета.
- `--vendor <vendorName>`: переопределяет поставщика пакета.
- `-B <packageDirectory>`: задает выходной каталог для `cpack` (по умолчанию текущий рабочий каталог).

Попробуем сгенерировать пакеты для нашего проекта `14-cpack`. Будем использовать генераторы пакетов ZIP, 7Z и Debian.

```
cpack -G "ZIP;7Z;DEB" -B packages
```

На выходе должны получиться следующие пакеты:

- `CPackPackage-1.2.3-Linux.7z`;
- `CPackPackage-1.2.3-Linux.deb`;
- `CPackPackage-1.2.3-Linux.zip`.

Эти двоичные пакеты можно опубликовать на сайте проекта, в виде выпускной версии на GitHub или в репозитории пакетов для конечных пользователей.

Резюме

Сложности написания кросс-платформенных скриптов установки могут отпугнуть, но CMake значительно упрощает решение этой задачи. Хотя кое-какая начальная подготовка требуется, CMake спрямляет процесс, органично интегрируя все концепции и методы, которые мы изучали в этой книге.

Мы начали с описания того, как экспортировать цели CMake из проектов, разрешив тем самым использовать их в других проектах без установки. Затем мы обсудили установку проектов, уже сконфигурированных для экспорта, и сосредоточились на важнейшем вопросе: установке целей CMake. Теперь мы понимаем, как CMake назначает различные места для разных типов артефактов, а также знаем о специальных соображениях для публичных заголовков. Мы также рассмотрели другие режимы команды `install()`, касающиеся установки файлов, программ и каталогов. Не оставили мы без внимания и выполнение скриптов во время установки.

Далее путь привел нас к повторно используемым пакетам CMake. Мы изучили, как сделать цели проекта перемещаемыми, дав тем самым пользо-

вателям возможность самостоятельно определять места установки. Попутно мы создали полностью определенные пакеты, которые может находить команда `find_package()`, что подвело нас к вопросу о файлах экспорта целей, конфигурационных файлах и файлах версий. Понимая, что у пользователей могут быть разные потребности, мы изучили, как группировать артефакты и действия в компоненты установки – понятие, отличное от компонентов пакетов CMake. И венцом главы стало знакомство с CPack. Мы узнали, как подготовить простой двоичный пакет, что дает эффективный метод распространения заранее откомпилированного ПО. Хотя овладение всеми тонкостями установки и упаковки в CMake – путешествие, которое никогда не кончается, в этой главе мы заложили прочный фундамент. Теперь мы достаточно оснащены для работы с наиболее типичными сценариями и можем уверенно копнуть глубже.

В следующей главе мы применим обретенные знания к созданию крепко сбитого профессионального проекта, продемонстрировав практическое применение рассмотренных методов работы с CMake.

Для дополнительного чтения

- Определение различных каталогов в стандартах кодирования GNU: https://www.gnu.org/prep/standards/html_node/Directory-Variables.html.
- Обсуждение нового способа управления публичными заголовками с помощью ключевого слова `FILE_SET`: https://gitlab.kitware.com/cmake/cmake/-issues/22468#note_991860.
- Установка разделяемой библиотеки: https://tldp.org/HOWTO/Program-Library-HOWTO/shared-libraries.html.
- Создание перемещаемых пакетов: https://cmake.org/cmake/help/latest/guide/importing-exporting/index.html#creating-relocatable-packages.
- Список путей, просматриваемых командой `find_package()` при поиске конфигурационного файла: https://cmake.org/cmake/help/latest/command/find_package.html#configmode-search-procedure.
- Полная документация по модулю `CMakePackageConfigHelpers`: https://cmake.org/cmake/help/latest/module/CMakePackageConfigHelpers.html.
- Генераторы пакетов в CPack: https://cmake.org/cmake/help/latest/manual/cpack-generators.7.html.
- О предпочтительных генераторах для разных платформ: https://stackoverflow.com/a/46013099.
- Документация по служебному модулю CPack: https://cmake.org/cmake/help/latest/module/CPack.html.

15

Создание профессионального проекта

Мы обладаем всеми знаниями, необходимыми для сборки профессиональных проектов: структурирование сборки, управление зависимостями, тестирование, анализ, установка и упаковка. Настало время применить эти знания, создав полноценный профессиональный проект. Важно понимать, что даже самые простые программы выигрывают от автоматизации контроля качества и органичного процесса преобразования исходного кода в завершенное решение. Да, для реализации всех проверок и процессов нужно приложить немалые усилия, поскольку для правильной настройки всего и вся необходимо много шагов. Особенно это замечание справедливо, когда необходимые механизмы включаются в уже существующие кодовые базы, зачастую большие и сложные. Именно поэтому лучше использовать CMake с самого начала и организовывать все процессы как можно раньше.

В этой главе мы разработаем новое решение, по возможности небольшое, но иллюстрирующее все рассмотренные методы работы с CMake. Чтобы сделать его простым, мы реализуем всего одну функцию – сложение двух чисел. Это позволит нам сосредоточиться на относящихся к сборке аспектах проекта, которые мы изучали в предыдущих главах. Чтобы уделить внимание более трудной проблеме сборки, проект будет включать библиотеку и исполняемый файл. Библиотека отвечает за внутреннюю бизнес-логику и будет доступна другим проектам в виде пакета CMake. Исполняемый файл, предназначенный конечным пользователям, предоставит интерфейс, демонстрирующий функциональность библиотеки.

В этой главе будут рассмотрены следующие вопросы:

- планирование работы;
- структура проекта;
- сборка и управление зависимостями;

- тестирование и анализ программы;
- установка и упаковка;
- документирование.

Технические требования

Файлы с кодом, представленным в данной главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch15.

Для сборки примеров в этой книге всегда следует пользоваться рекомендуемыми командами:

```
cmake -B <build tree> -S <source tree>
cmake --build <build tree>
```

Вместо `<build tree>` и `<source tree>` не забывайте подставлять соответствующие пути. Напоминаем, что **build tree** – путь к выходному каталогу, а **source tree** – путь к каталогу с исходным кодом.

Код в этой главе откомпилирован с помощью GCC, чтобы обеспечить совместимость оснащения средствами покрытия с инструментом `lcov` для сбора результатов. Если вы хотите откомпилировать его с помощью `llvm` или другого комплекта инструментов, не забудьте внести необходимые коррективы в обработку покрытия тестами.

Для прогона тестов выполните команду

```
ctest --test-dir <build tree>
```

Или, находясь в каталоге дерева сборки, просто выполните команду

```
ctest
```

Заметим, что в этой главе тесты будут выводить результаты в подкаталог `test`.

Планирование работы

Программа, которую мы соберем в этой главе, не претендует на сложность – мы напишем простейший калькулятор, который складывает два числа (рис. 15.1). Это будет консольное приложение с текстовым пользовательским интерфейсом, в котором используются сторонняя библиотека и отдельная библиотека вычислений, доступная другим проектам. У этого проекта нет никаких практических приложений, но благодаря своей простоте он идеален для демонстрации различных методов, рассмотренных в этой книге.

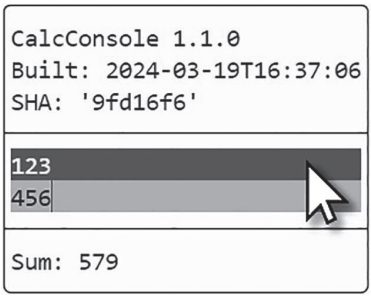


Рис. 15.1. Текстовый пользовательский интерфейс проекта, запущенного в терминале с поддержкой мыши

Обычно проекты либо генерируют исполняемый файл, непосредственно взаимодействующий с пользователем, либо библиотеку для разработчиков. То и другое порождается реже, хотя всякое бывает. Например, некоторые приложения поставляются с автономными SDK или библиотеками, поддерживающими разработку плагинов. Другой пример – библиотека, сопровождаемая примерами использования. Наш проект попадает во вторую категорию, т. к. иллюстрирует функциональность библиотеки.

Планировать мы начнем с того, что вспомним содержимое всех глав и выберем методы и инструменты, которые будем использовать для сборки приложения.

- *Глава 1 «Первые шаги»*
  В этой главе приводятся первые сведения о CMake, включая установку и использование командной строки для сборки проектов. Также излагается существенная информация о файлах проекта: их роли, типичные соглашения об именовании и разного рода тонкости.
- *Глава 2 «Язык CMake»*
  Мы познакомились с основными средствами написания корректных списковых файлов и скриптов CMake, включая комментарии, вызов команд и их аргументы. Мы объяснили, как выглядят переменные, списки и управляющие конструкции, и привели в пример несколько полезных команд. Этот фундамент будет очень важен для нашего проекта.
- *Глава 3 «Использование CMake в популярных IDE»*
  Мы обсудили три IDE: CLion, VS Code и Visual Studio – и отметили их сильные стороны. Бремя выбора IDE (или работа без нее) для нашего финального проекта лежит на вас. Приняв решение, вы можете начать проект в контейнере разработки и насладиться всеми преимуществами уже готового окружения, выполнив всего несколько шагов для построения Docker-образа (или просто скачав его с Docker Hub). Выполнение образа в контейнере гарантирует, что среда разработки будет такой же, как производственная.
- *Глава 4 «Подготовка первого проекта на основе CMake»*
  Конфигурирование проекта чрезвычайно важно, потому что определяет, какие будут действовать политики CMake, а также именование,

версионирование и языки программирования. Материал этой главы окажет влияние на фундаментальное поведение процесса сборки.
Мы также будем придерживаться установившейся практики разбиения и структурирования проекта при определении каталогов и файлов и воспользуемся системными переменными обнаружения для адаптации к различным средам сборки. Конфигурация комплекта инструментов – еще один важный аспект, который позволит нам выбрать конкретную версию C++ и стандартов, поддерживаемую компилятором. Следуя приведенной в этой главе рекомендации, мы запретим внутреннюю сборку, чтобы рабочее пространство оставалось чистым.

- *Глава 5 «Цели»*
  Из этой главы мы узнали, что любой современный проект CMake активно пользуется целями. И конечно, мы тоже применим цели для определения библиотек и исполняемых файлов (для тестовой и производственной сред); это позволит организовать проект и оставаться верными принципу **DRY** (не повторяйся). Полученные нами знания о свойствах целей и транзитивных требованиях использования (распространяемых свойствах) позволят приблизить конфигурирование к определениям целей.
- *Глава 6 «Генераторные выражения»*
  Генераторные выражения используются в разных местах нашего проекта. Мы постараемся сделать их максимально простыми. Проект будет включать специальные команды генерирования файлов для Valgrind и отчетов о покрытии тестами. Кроме того, мы воспользуемся точками вклинивания в цели, особенно `PRE_BUILD`, чтобы стереть `gcda`-файлы, созданные процессом оснащения для вычисления покрытия.
- *Глава 7 «Компиляция исходного кода на C++ с помощью CMake»*
  Не бывает проектов на C++ без компиляции. Основные идеи просты, но CMake позволяет настроить этот процесс самыми разными способами: расширить список исходных файлов цели, сконфигурировать оптимизатор и включить отладочную информацию. Для этого проекта флагов компилятора по умолчанию вполне достаточно, но мы не ограничимся ими и немного побалуемся с препроцессором:
  – сохраним метаданные сборки (версию проекта, время сборки и SHA-свертку фиксации Git) в откомпилированном исполняемом файле и будем показывать их пользователям;
  – активируем предкомпиляцию заголовочных файлов. В таком маленьком проекте это излишество, но поможет нам попрактиковаться.

  Унифицированные сборки не понадобятся – проект слишком мал, чтобы оправдать их добавление.
- *Глава 8 «Компоновка исполняемых файлов и библиотек»*
  Мы получим общую информацию о компоновке, полезную в любом проекте по умолчанию. Кроме того, поскольку этот проект включает библиотеку, мы будем явно ссылаться на некоторые команды сборки, а именно:
  – статические библиотеки для тестирования и разработки;
  – разделяемые библиотеки для выпускной версии.

В этой главе также описывается, как вынести функцию `main()` для тестирования, и этим мы тоже воспользуемся.

- *Глава 9 «Управление зависимостями в CMake»*
  Чтобы улучшить внешний вид проекта, мы добавим внешнюю зависимость: библиотеку для организации текстового пользовательского интерфейса. В главе 6 рассматриваются различные методы управления зависимостями. Выбор прост: как правило, рекомендуется служебный модуль `FetchContent`, поскольку он самый удобный.
- *Глава 10 «Использование модулей C++20»*
  Хотя мы говорили об использовании модулей C++20 и требований к среде, поддерживающей эту возможность (CMake 3.28, последние версии компиляторов), ей все еще недостает широкой поддержки. Чтобы сделать проект максимально доступным, мы пока не будем включать модули.
- *Глава 11 «Каркасы тестирования»*
  Реализация по-настоящему автоматизированных тестов необходима, чтобы качество решения не ухудшалось со временем. Мы интегрируем в проект CTest и организуем его так, чтобы облегчить тестирование, для чего вынесем функцию `main()`, как было упомянуто выше.
  В этой главе обсуждается два каркаса тестирования: Catch2 и GTest с GMock; мы будем пользоваться вторым. Для получения подробных сведений о покрытии тестами мы сгенерируем HTML-отчеты с помощью LCOV.
- *Глава 12 «Инструменты анализа программ»*
  Для статического анализа мы можем выбирать из нескольких инструментов: Clang-Tidy, Cpplint, Cppcheck, include-what-you-use и link-what-you-use. Мы остановимся на Cppcheck, потому что Clang-Tidy в меньшей степени совместим с предкомпилированными заголовками, создаваемыми GCC.
  Для динамического анализа мы применим инструмент Memcheck из комплекта Valgrind, дополненный оберткой Memcheck-cover для генерирования HTML-отчетов. Кроме того, мы будем автоматически форматировать исходный код во время сборки с помощью ClangFormat.
- *Глава 13 «Генерирование документации»*
  Наличие документации обязательно, если частью проекта является библиотека. CMake упрощает автоматизацию генерирования документации с помощью Doxygen. Мы применим этот подход, дополнив его современным дизайном на основе темы `doxygen-awesome-css`.
- *Глава 14 «Установка и упаковка»*
  Наконец, мы сконфигурируем установку и упаковку решения и подготовим соответствующие файлы для формирования пакета, а также определения целей. Мы установим их, а равно артефакты сборки целей в нужные каталоги, включив модуль `GNUInstallDirs`. Дополнительно мы сконфигурируем несколько компонентов, чтобы сделать решение модульным и подготовить его к использованию CPack.

В состав профессиональных проектов входит еще несколько файлов: `README`, `LICENSE`, `INSTALL` и т. д. Мы кратко рассмотрим их в конце главы.

Чтобы упростить задачу, мы не будем реализовывать специальную логику для проверки наличия всех необходимых утилит и зависимостей. Мы будем полагаться на то, что CMake сама выведет диагностику и сообщит пользователям, чего не хватает. Если ваши проекты окажутся популярны, то можете рассмотреть возможность добавления таких механизмов, чтобы улучшить взаимодействие с пользователями.

Составив четкий план, обсудим, как реально организовать проект с точки зрения логических целей и структуры каталогов.

Структура проекта

Сборку любого проекта следует начинать с ясного понимания того, какие логические цели мы собираемся в нем создать. В данном случае структура показана на рис. 15.2.

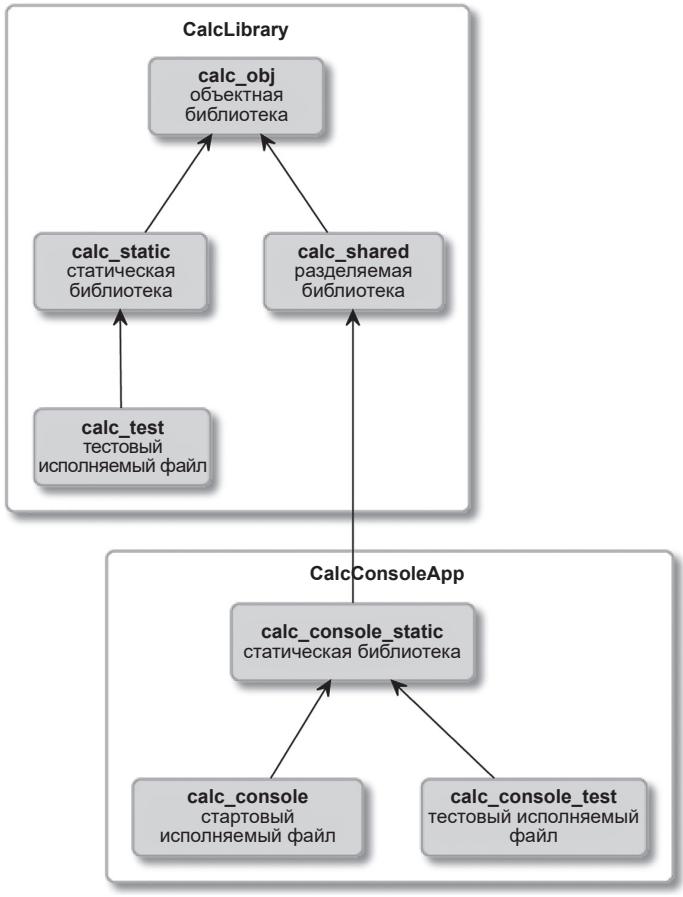


Рис. 15.2. Структура логических целей

Рассмотрим эту структуру в порядке сборки. Сначала компилируется **объектная библиотека** `calc_obj`. Если вы забыли, что это такое, обратитесь к главе 5. После этого мы должны обратить внимание на **статические библиотеки** и **разделяемые библиотеки**.

Разделяемые и статические библиотеки

В главе 8 мы познакомились с разделяемыми и статическими библиотеками. Мы отметили, что разделяемые библиотеки могут уменьшить суммарное потребление памяти в случае, когда несколько программ пользуются одной и той же библиотекой. Кроме того, в системах пользователей популярные библиотеки обычно уже установлены или они умеют их быстро устанавливать.

Важнее, впрочем, что **разделяемые библиотеки** – это отдельные файлы, которые необходимо поместить в определенные каталоги, где их сможет найти динамический компоновщик. Напротив, статические библиотеки встраиваются непосредственно в исполняемый файл, поэтому работать с ними быстрее, потому что не нужны дополнительные шаги для поиска кода в памяти.

Будучи авторами библиотеки, мы должны решить, какую версию поставлять: статическую или динамическую. Или же можно поставить обе версии, и пусть программист решает, какую из них использовать в своем проекте. Мы хотим проверить свои знания, поэтому будем поставлять обе версии.

Цель `calc_test`, включающая автономные тесты для проверки функциональности библиотеки, будет пользоваться статической библиотекой. Мы собираем библиотеки обоих типов из одних и тех же *объектных файлов*, поэтому допустимо тестировать библиотеку какого-то одного типа, т. к. функциональность обеих должна быть одинакова. Консольное приложение, ассоциированное с целью `calc_console_static`, будет компоноваться с разделяемой библиотекой. Эта цель компонуется также с внешней зависимостью – библиотекой Functional Terminal (X) User Interface (FTXUI), написанной Артуром Сонзоньи (в разделе «Для дополнительного чтения» есть ссылка на этот проект на GitHub).

Последние две цели, `calc_console` и `calc_console_test`, предназначены для решения типичной проблемы при тестировании исполняемых файлов: конфликт нескольких точек входа, предоставляемых каркасами тестирования и самими файлами. Чтобы справиться с этим, мы намеренно изолировали функцию `main()` в стартовой цели, `calc_console`, которая просто вызывает главную функцию из `calc_console_static`.

Разобравшись с необходимыми целями и их взаимосвязями, перейдем к следующему шагу – организации структуры проекта, т. е. к перечню файлов и каталогов.

Файлы проекта

Проект состоит из двух основных элементов: библиотек `calc` и исполняемого файла `calc_console`. Для эффективной организации проекта мы создадим следующую структуру каталогов:

- `src` содержит исходные коды всех выпускных целей и заголовочные файлы библиотек;
- `test` содержит тесты библиотек и исполняемых файлов;
- `cmake` содержит служебные модули и вспомогательные файлы, используемые CMake для сборки и установки проекта;
- *корневой каталог* содержит конфигурацию верхнего уровня и файлы документации.

Эта структура (см. рис. 15.3) обеспечивает четкое разделение обязанностей, что упрощает навигацию по проекту и его сопровождение.

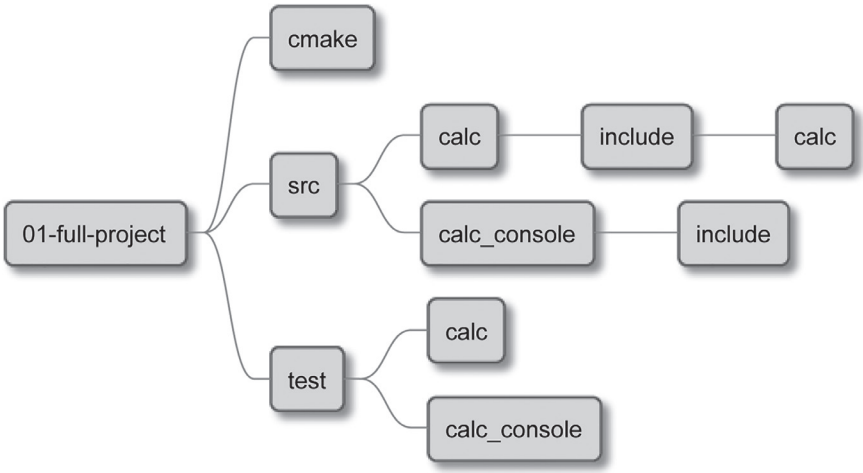


Рис. 15.3. Структура каталогов проекта

Ниже приведен полный список файлов в каждом из четырех основных каталогов.

Таблица 15.1. Полная структура проекта

Корневой каталог	**./test**
`CHANGELOG`	`CMakeLists.txt`
CMakeLists.txt	**calc/CMakeLists.txt**

`INSTALL`
`LICENSE`
`README.md`
	`calc/calc_test.cpp`
	`calc_console/CMakeLists.txt`
	`calc_console/tui_test.cpp`

Таблица 15.1 *(окончание)*

./src	./cmake
CMakeLists.txt	BuildInfo.cmake
calc/CMakeLists.txt	Coverage.cmake
calc/CalcConfig.cmake	CppCheck.cmake
calc/basic.cpp	Doxygen.cmake
calc/include/calc/basic.h	Format.cmake
calc_console/CMakeLists.txt	GetFTXUI.cmake
calc_console/bootstrap.cpp	Packaging.cmake
calc_console/include/tui.h	Memcheck.cmake
calc_console/tui.cpp	NoInSourceBuilds.cmake
	Testing.cmake
	buildinfo.h.in
	doxygen_extra_headers

Может показаться, что CMake создает значительные накладные расходы, потому что каталог `cmake` первоначально содержит гораздо больше, чем сам код, из-за которого все затевалось, но с расширением проекта это соотношение изменяется. На начальном этапе пришлось приложить немало усилий для организации чистой структуры проекта, но смею вас заверить, что в будущем эти вложения многократно окупятся.

По ходу главы мы рассмотрим все файлы, упомянутые в табл. 15.1, и узнаем, что они делают и какую роль играют. Обсуждение будет разбито на четыре шага: сборка, тестирование, установка и документирование.

Сборка и управление зависимостями

Все процессы сборки устроены одинаково. Мы начинаем с верхнеуровневого спискового файла и спускаемся вниз по дереву исходного кода проекта. На рис. 15.4 показаны файлы проекта, вовлеченные в процесс сборки, а числа в скобках показывают порядок их обработки скриптом CMake.

Верхнеуровневый списковый файл `CMakeLists.txt` (1) конфигурирует проект:

ch15/01-full-project/CMakeLists.txt

```
cmake_minimum_required(VERSION 3.26)
project(Calc VERSION 1.1.0 LANGUAGES CXX)
list(APPEND CMAKE_MODULE_PATH "${CMAKE_SOURCE_DIR}/cmake")

include(NoInSourceBuilds)

include(CTest)

add_subdirectory(src bin)
add_subdirectory(test)

include(Packaging)
```

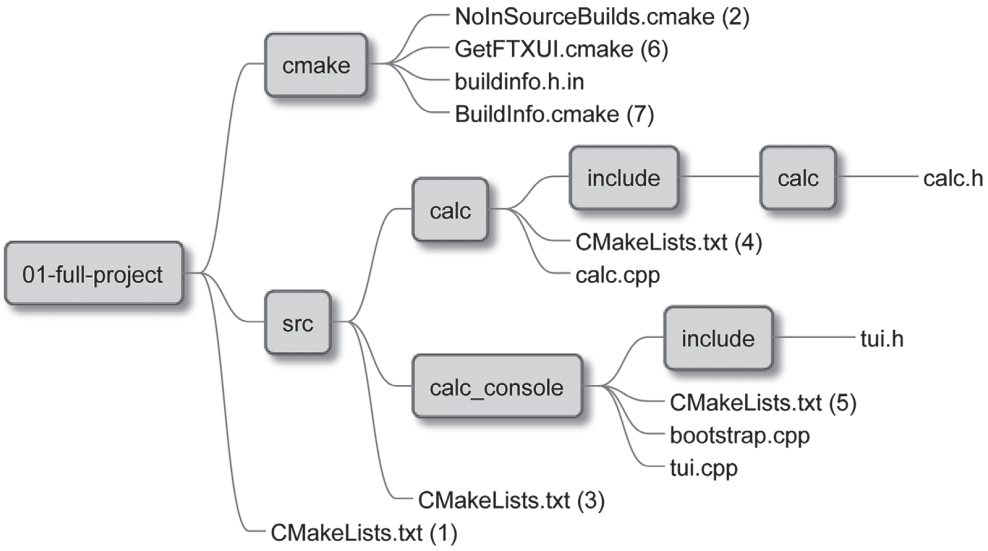


Рис. 15.4. Файлы, используемые на этапе сборки

Мы начинаем с задания основных деталей проекта и установки пути к служебным модулям CMake (в нашем проекте это каталог `cmake`). Затем запрещаем внутренние сборки с помощью специального модуля. После этого мы разрешаем тестирование с применением модуля `CTest` (встроен в CMake). Это необходимо сделать на уровне корня проекта, потому что эта команда создает файл `CTestTestfile.cmake` в том же месте двоичного дерева, какое он занимает относительно дерева исходного кода. Если поместить его куда-то еще, то `ctest` не сможет его найти.

Далее мы включаем два ключевых каталога:

- `src`, содержащий исходный код проекта (который будет назван `bin` в дереве сборки);
- `test`, содержащий все утилиты тестирования.

Наконец, мы включаем модуль `Packaging`, назначение которого обсудим в разделе «Установка и упаковка».

Рассмотрим служебный модуль `NoInSourceBuilds`, чтобы понять, как он работает.

ch15/01-full-project/cmake/NoInSourceBuilds.cmake

```
if(PROJECT_SOURCE_DIR STREQUAL PROJECT_BINARY_DIR)
  message(FATAL_ERROR
    "\n"
    "In-source builds are not allowed.\n"
    "Instead, provide a path to build tree like so:\n"
    "cmake -B <destination>\n"
    "\n"
    "To remove files you accidentally created execute:\n"
    "rm -rf CMakeFiles CMakeCache.txt\n"
```

```
)
endif()
```

Ничего неожиданного – мы проверяем, что пользователь указал отдельный конечный каталог для сгенерированных файлов с помощью команды `cmake`. Он должен отличаться от дерева исходного кода проекта. Если это не так, мы информируем пользователя, как правильно создать каталог и очистить репозиторий, если была допущена ошибка.

Далее наш верхнеуровневый списковый файл включает подкаталог `src`, что является для CMake указанием на необходимость обработать содержащийся в нем списковый файл.

ch15/01-full-project/src/CMakeLists.txt

```
include(Coverage)
include(Format)
include(CppCheck)
include(Doxygen)
add_subdirectory(calc)
add_subdirectory(calc_console)
```

С этим файлом все понятно – он включает все модули из каталога `./cmake`, который мы будем использовать, и направляет CMake во вложенные подкаталоги, где тоже находятся списковые файлы, подлежащие обработке.

Следующий шаг – знакомство со списковым файлом для библиотеки `calc`. Он довольно сложный, поэтому обсудим его по частям.

Сборка библиотеки Calc

Списковый файл в каталоге `calc` конфигурирует различные аспекты этой библиотеки, но пока сосредоточимся только на сборке.

ch15/01-full-project/src/calc/CMakeLists.txt (фрагмент)

```
add_library(calc_obj OBJECT basic.cpp)
target_sources(calc_obj
               PUBLIC FILE_SET HEADERS
               BASE_DIRS include
               FILES include/calc/basic.h
)
set_target_properties(calc_obj PROPERTIES
               POSITION_INDEPENDENT_CODE 1
)

# ... оснащение calc_obj для вычисления покрытия тестами

add_library(calc_shared SHARED)
target_link_libraries(calc_shared calc_obj)
```

```
add_library(calc_static STATIC)
target_link_libraries(calc_static calc_obj)

# ... модули тестирования и анализа программы
# ... генерирование документации
# ... установка
```

Мы определяем три цели:

- `calc_obj`, компиляция *объектной библиотеки*, содержащей основной файл реализации `basic.cpp`. Ее заголовочный файл `basic.h` включается с помощью ключевого слова `FILE_SET` в команде `target_sources()`. Тем самым мы неявно конфигурируем экспорт правильных каталогов включаемых файлов для режимов сборки и установки. Создание объектной библиотеки предотвращает излишнюю компиляцию для обеих версий библиотек, но очень важно активировать ее свойство `POSITION_INDEPENDENT_CODE`, чтобы разделяемая библиотека могла зависеть от этой цели;
- `calc_shared`, разделяемая библиотека, зависящая от `calc_obj`;
- `calc_static`, статическая библиотека, также зависящая от `calc_obj`.

Ниже для полноты контекста показан заголовочный файл базовой библиотеки. В нем просто объявлены две функции в пространстве имен `Calc`, что позволяет избежать конфликтов имен.

ch15/01-full-project/src/calc/include/calc/basic.h

```
#pragma once
namespace Calc {
  int Add(int a, int b);
  int Subtract(int a, int b);
} // namespace Calc
```

Файл реализации тоже тривиален.

ch15/01-full-project/src/calc/basic.cpp

```
namespace Calc {
  int Add(int a, int b) {
    return a + b;
  }

  int Subtract(int a, int b) {
    return a - b;
  }
} // namespace Calc
```

Это все о файлах в каталоге `src/calc`. Далее идет каталог `src/calc_console` и сборка исполняемого файла консольного калькулятора, который пользуется этой библиотекой.

Сборка исполняемого файла консольного калькулятора Calc

Каталог `calc_console` содержит несколько файлов: списковый файл, два файла реализации (бизнес-логика и стартовый файл) и заголовочный файл. Списковый файл выглядит следующим образом.

ch15/01-full-project/src/calc_console/CMakeLists.txt (фрагмент)

```
add_library(calc_console_static STATIC tui.cpp)
target_include_directories(calc_console_static PUBLIC include)
target_precompile_headers(calc_console_static PUBLIC <string>)

include(GetFTXUI)
target_link_libraries(calc_console_static PUBLIC calc_shared
                      ftxui::screen ftxui::dom ftxui::component)

include(BuildInfo)
BuildInfo(calc_console_static)

# ... оснащение calc_console_static для вычисления покрытия тестами
# ... модули тестирования и анализа программы
# ... генерирование документации

add_executable(calc_console bootstrap.cpp)
target_link_libraries(calc_console calc_console_static)

# ... установка
```

Хотя списковый файл кажется сложным, нам, теперь уже опытным пользователям CMake, не составит труда расшифровать его содержимое.

1. Определить цель `calc_console_static`, которая содержит бизнес-логику без функции `main()`, чтобы ее можно было скомпоновать с каркасом GTest, имеющим собственную точку входа.
2. Сконфигурировать *каталоги включаемых файлов*. Мы могли бы добавить заголовки по отдельности с помощью ключевого слова `FILE_SET`, но, поскольку они внутренние, упростим себе задачу.
3. Реализовать предкомпиляцию, которая здесь продемонстрирована на примере одного лишь заголовочного файла `<string>`, хотя в более крупном проекте заголовков может быть гораздо больше.
4. Включить специальный модуль CMake для скачивания зависимости FTXUI.
5. Скомпоновать код бизнес-логики с разделяемой библиотекой `calc_shared` и компонентами FTXUI.
6. Добавить специальный модуль для генерирования информации о сборке и внедрения ее в артефакт.

7. Наметить дополнительные шаги для этой цели: оснащение для измерения покрытия, тестирование, анализ программы и документирование.
8. Создать и скомпоновать стартовый исполняемый файл `calc_console`, содержащий точку входа.
9. Наметить шаг установки.

Процессы тестирования, документирования и установки мы рассмотрим в соответствующих разделах этой главы.

Мы включаем служебный модуль `GetFTXUI`, вместо того чтобы искать `config-module` в системе, потому что у большинства пользователей он, скорее всего, не установлен. Мы просто скачаем и соберем его.

ch15/01-full-project/cmake/GetFTXUI.cmake

```
include(FetchContent)
FetchContent_Declare(
FTXTUI
GIT_REPOSITORY https://github.com/ArthurSonzogni/FTXUI.git
GIT_TAG        v0.11
)
option(FTXUI_ENABLE_INSTALL "" OFF)
option(FTXUI_BUILD_EXAMPLES "" OFF)
option(FTXUI_BUILD_DOCS "" OFF)
FetchContent_MakeAvailable(FTXTUI)
```

Мы используем рекомендуемый метод `FetchContent`, подробно описанный в главе 9. Единственное необычное добавление – вызовы команды `option()`, которая позволяет опустить долгие шаги сборки FTXUI и не дает шагам его установки повлиять на процесс установки этого проекта. За дополнительными сведениями отсылаем к разделу «Для дополнительного чтения».

Списковый файл в каталоге `calc_console` включает еще один служебный модуль, связанный со сборкой: `BuildInfo`. Этот модуль собирает следующие сведения, отображаемые исполняемым файлом:

- SHA-свертка текущей фиксации Git;
- временная метка сборки;
- версия проекта, указанная в верхнеуровневом списковом файле.

В главе 7 мы узнали, что CMake может запомнить значения, имевшие место на этапе сборки, и передать их в код на C++ с помощью шаблонных файлов, например в виде структуры.

ch15/01-full-project/cmake/buildinfo.h.in

```
struct BuildInfo {
  static inline const std::string CommitSHA = "@COMMIT_SHA@";
  static inline const std::string Timestamp = "@TIMESTAMP@";
  static inline const std::string Version = "@PROJECT_VERSION@";
};
```

Чтобы заполнить эту структуру на этапе конфигурирования, используется следующий код.

ch15/01-full-project/cmake/BuildInfo.cmake

```
set(BUILDINFO_TEMPLATE_DIR ${CMAKE_CURRENT_LIST_DIR})
set(DESTINATION "${CMAKE_CURRENT_BINARY_DIR}/buildinfo")
string(TIMESTAMP TIMESTAMP)

find_program(GIT_PATH git REQUIRED)
execute_process(COMMAND ${GIT_PATH} log --pretty=format:'%h' -n 1
                OUTPUT_VARIABLE COMMIT_SHA)

configure_file(
  "${BUILDINFO_TEMPLATE_DIR}/buildinfo.h.in"
  "${DESTINATION}/buildinfo.h" @ONLY
)

function(BuildInfo target)
  target_include_directories(${target} PRIVATE ${DESTINATION})
endfunction()
```

После включения модуля мы устанавливаем переменные, в которых запоминается нужная информация, и используем `configure_file()` для генерирования `buildinfo.h`. Последний шаг – вызов функции `BuildInfo`, которая включает каталог со сгенерированными файлами в состав каталогов включаемых файлов цели.

Порожденный заголовочный файл можно затем предоставить в пользование различным потребителям, если это необходимо. В таком случае вы, вероятно, захотите добавить `include_guard(GLOBAL)` в начало спискового файла, чтобы избежать выполнения команды `git` для каждой цели.

Прежде чем переходить к реализации консольного калькулятора, я хотел бы подчеркнуть, что нет никакой нужды глубоко разбираться в тонкостях файла `tui.cpp` или библиотеки FXTUI, поскольку для нас это второстепенно. Лучше сосредоточимся на выделенных частях кода.

ch15/01-full-project/src/calc_console/tui.cpp

```
#include "tui.h"
#include <ftxui/dom/elements.hpp>
#include "buildinfo.h"
#include "calc/basic.h"

using namespace ftxui;
using namespace std;

string a{"12"}, b{"90"};
auto input_a = Input(&a, "");
```

```
auto input_b = Input(&b, "");
auto component = Container::Vertical({input_a, input_b});

Component getTui() {
  return Renderer(component, [&] {
    auto sum = Calc::Add(stoi(a), stoi(b));
    return vbox({
      text("CalcConsole " + BuildInfo::Version),
      text("Built: " + BuildInfo::Timestamp),
      text("SHA: " + BuildInfo::CommitSHA),
      separator(),
      input_a->Render(),
      input_b->Render(),
      separator(),
      text("Sum: " + to_string(sum)),
    }) |
    border;
  });
}
```

В этом коде мы видим функцию `getTui()`, которая возвращает объект `ftxui::Component`, инкапсулирующий элементы интерактивного UI: метки, текстовые поля, разделители, границы и т. д. Любопытствующие могут узнать, как работают эти элементы, по ссылке в разделе «Для дополнительного чтения».

Важнее то, что директивы `#include` ведут на заголовки из цели `calc_obj` и модуля `BuildInfo`. Взаимодействие реализовано лямбда-функцией, которая вызывает `Calc::Sum` и отображает результат с помощью функции `text()`.

Значения в файле `buildinfo.h`, полученные на этапе сборки, используются аналогично и будут показаны пользователю во время выполнения.

В том же каталоге, что `tui.cpp`, находится заголовочный файл.

ch15/01-full-project/src/calc_console/include/tui.h

```
#include <ftxui/component/component.hpp>
ftxui::Component getTui();
```

Он используется в стартовом файле в цели `calc_console`.

ch15/01-full-project/src/calc_console/bootstrap.cpp

```
#include <ftxui/component/screen_interactive.hpp>

#include "tui.h"

int main(int argc, char** argv) {
  ftxui::ScreenInteractive::FitComponent().Loop(getTui());
}
```

Этот код инициализирует экран интерактивной консоли с помощью библиотеки FTXUI, отображает объект `Component`, полученный от `getTui()`, и в цикле обрабатывает нажатия клавиш. Итак, мы описали все файлы в каталоге `src` и можем перейти к тестированию и анализу программы.

Тестирование и анализ программы

Анализ и тестирование программы – важные компоненты, которые совместно обеспечивают качество решения. Например, использовать Valgrind эффективнее при выполнении теста. Поэтому мы сконфигурируем тестирование и анализ программы в одном месте. На рис. 15.5 показаны поток выполнения и необходимые файлы.

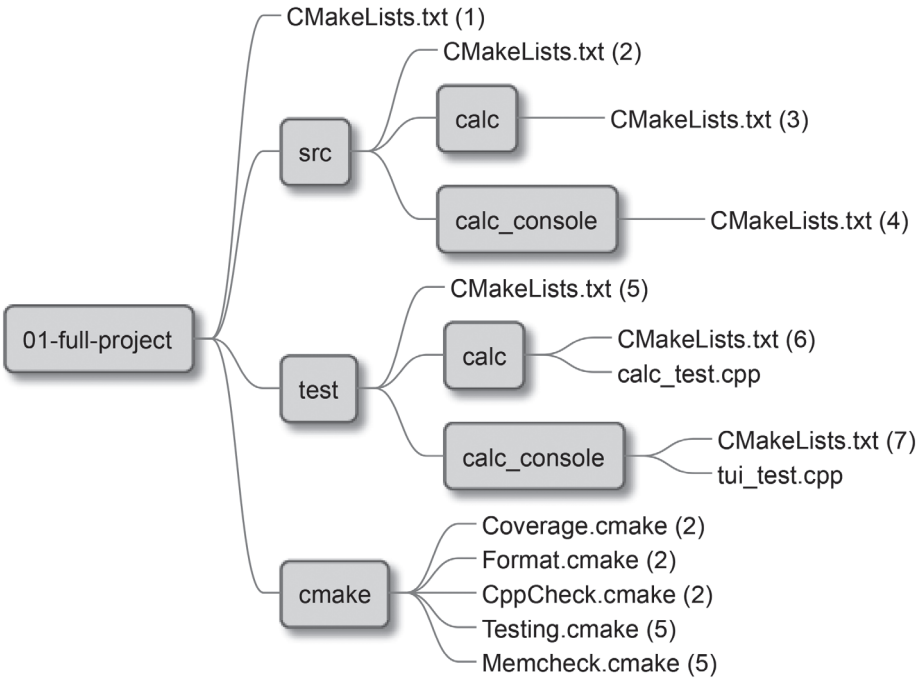


Рис. 15.5. Файлы, используемые при тестировании и анализе программы

Числа в скобках показывают последовательность обработки списковых файлов. Мы начинаем с верхнеуровневого спискового файла и добавляем каталоги `src` и `test`.

- В каталог `src` включаются модули `Coverage`, `Format` и `CppCheck` и добавляются каталоги `src/calc` и `src/calc_console`.
- В каталоге `src/calc` определяются цели, при конфигурировании которых включаются модули.

- В каталоге `src/calc_console` определяются цели, при конфигурировании которых включаются модули.
- В каталог `test` включается модуль `Testing` (который включает `Memcheck`) и добавляются каталоги `test/calc` и `test/calc_console`.
- В каталоге `test/calc` определяются тестовые цели, при конфигурировании которых включаются модули.
- В каталоге `test/calc_console` определяются тестовые цели, при конфигурировании которых включаются модули.

Рассмотрим списковый файл в каталоге `test`.

ch15/01-full-project/test/CMakeLists.txt

```
include(Testing)
add_subdirectory(calc)
add_subdirectory(calc_console)
```

На этом уровне включается служебный модуль `Testing`, который предоставляет свою функциональность обеим группам целей (из каталогов `calc` и `calc_console`):

ch15/01-full-project/cmake/Testing.cmake (фрагмент)

```
include(FetchContent)
FetchContent_Declare(
  googletest
  GIT_REPOSITORY https://github.com/google/googletest.git
  GIT_TAG v1.14.0
)
# Для Windows: предотвратить переопределение настроек компилятора
# и компоновщика, заданных в родительском проекте
set(gtest_force_shared_crt ON CACHE BOOL "" FORCE)
option(INSTALL_GMOCK "Install GMock" OFF)
option(INSTALL_GTEST "Install GTest" OFF)
FetchContent_MakeAvailable(googletest)

# ...
```

Мы разрешили тестирование и включили модуль `FetchContent` для получения GTest и GMock. Хотя каркас GMock в этом проекте не используется, он находится в том же репозитории, что и GTest, поэтому мы заодно сконфигурировали и его. Важный аспект конфигурирования – сделать так, чтобы установка этих каркасов не повлияла на установку нашего проекта, поэтому мы используем команду `option()`.

В том же файле мы определяем функцию `AddTests()`, которая организует полное тестирование целей бизнес-логики.

ch15/01-full-project/cmake/Testing.cmake (продолжение)

```
# ...
include(GoogleTest)
include(Coverage)
```

```
include(Memcheck)
macro(AddTests target)
  message("Adding tests to ${target}")
  target_link_libraries(${target} PRIVATE gtest_main gmock)
  gtest_discover_tests(${target})
  AddCoverage(${target})
  AddMemcheck(${target})
endmacro()
```

Первым делом мы включаем необходимые модули: `GoogleTest` входит в состав CMake, а `Coverage` и `Memcheck` – специальные модули, включенные в проект. Следующий далее макрос `AddTests` подготавливает цель к тестированию, добавляя оснащение для вычисления покрытия и проверку памяти. Функции `AddCoverage()` и `AddMemcheck()` определены в соответствующих служебных модулях. Теперь мы можем перейти к реализации.

Подготовка модуля Coverage

Для добавления измерения покрытия в разные цели нужно несколько шагов. Модуль `Coverage` предоставляет функцию, которая определяет цель вычисления покрытия для указанной цели.

ch15/01-full-project/cmake/Coverage.cmake (фрагмент)

```
function(AddCoverage target)
  find_program(LCOV_PATH lcov REQUIRED)
  find_program(GENHTML_PATH genhtml REQUIRED)
  add_custom_target(coverage-${target}
    COMMAND ${LCOV_PATH} -d . --zerocounters
    COMMAND $<TARGET_FILE:${target}>
    COMMAND ${LCOV_PATH} -d . --capture -o coverage.info
    COMMAND ${LCOV_PATH} -r coverage.info '/usr/include/*'
      -o filtered.info
    COMMAND ${GENHTML_PATH} -o coverage-${target}
      filtered.info --legend
    COMMAND rm -rf coverage.info filtered.info
    WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
  )
endfunction()
# ...
```

Эта реализация немного отличается от приведенной в главе 11 тем, что включает имя цели в выходной путь, чтобы предотвратить конфликт имен. Еще нам нужна функция для очистки предыдущего отчета о покрытии.

ch15/01-full-project/cmake/Coverage.cmake (продолжение)

```
# ...
function(CleanCoverage target)
```

```
  add_custom_command(TARGET ${target} PRE_BUILD COMMAND
    find ${CMAKE_BINARY_DIR} -type f
    -name '*.gcda' -exec rm {} +)
endfunction()

# ...
```

Кроме того, у нас есть функция для подготовки цели к анализу покрытия.

ch15/01-full-project/cmake/Coverage.cmake (фрагмент)

```
# ...

function(InstrumentForCoverage target)
  if (CMAKE_BUILD_TYPE STREQUAL Debug)
    target_compile_options(${target}
                           PRIVATE --coverage -fno-inline)
    target_link_options(${target} PUBLIC --coverage)
  endif()
endfunction()
```

Функция `InstrumentForCoverage()` применяется к целям `src/calc` и `src/calc_console`, что позволяет генерировать файлы данных о покрытии при выполнении целей `calc_obj` и `calc_console_static`.

Чтобы сгенерировать отчеты для обеих целей, выполните следующие команды `cmake` после конфигурирования проекта с типом сборки `Debug`:

```
cmake --build <build-tree> -t coverage-calc_test
cmake --build <build-tree> -t coverage-calc_console_test
```

Далее мы хотим выполнить динамический анализ программы в нескольких определенных нами целях, поэтому для применения модуля `Memcheck`, описанного в главе 12, нужно немного подправить его, научив просматривать более одной цели.

Подготовка модуля Memcheck

Генерирование отчетов Valgrind об управлении памятью инициируется функцией `AddTests()`. Модуль `Memcheck` мы начнем с инициализации.

ch15/01-full-project/cmake/Memcheck.cmake (фрагмент)

```
include(FetchContent)
FetchContent_Declare(
  memcheck-cover
  GIT_REPOSITORY https://github.com/Farigh/memcheck-cover.git
  GIT_TAG        release-1.2
)
FetchContent_MakeAvailable(memcheck-cover)
```

Этот код нам уже знаком. Теперь рассмотрим функцию, которая создает цели, необходимые для генерирования отчетов.

ch15/01-full-project/cmake/Memcheck.cmake (продолжение)

```
function(AddMemcheck target)
  set(MEMCHECK_PATH ${memcheck-cover_SOURCE_DIR}/bin)
  set(REPORT_PATH "${CMAKE_BINARY_DIR}/valgrind-${target}")

  add_custom_target(memcheck-${target}
    COMMAND ${MEMCHECK_PATH}/memcheck_runner.sh -o
      "${REPORT_PATH}/report"
      -- $<TARGET_FILE:${target}>
    COMMAND ${MEMCHECK_PATH}/generate_html_report.sh
      -i ${REPORT_PATH}
      -o ${REPORT_PATH}
      WORKING_DIRECTORY ${CMAKE_BINARY_DIR}
  )
endfunction()
```

Мы немного усовершенствовали функцию `AddMemcheck()` из главы 12, добавив обработку нескольких целей. А переменную `REPORT_PATH` сделали зависящей от цели.

Чтобы сгенерировать отчеты Memcheck, выполните следующие команды (заметим, что генерирование отчетов более эффективно, если на этапе конфигурирования был указан тип сборки `Debug`):

```
cmake --build <build-tree> -t memcheck-calc_test
cmake --build <build-tree> -t memcheck-calc_console_test
```

Итак, мы определили модули `Coverage` и `Memcheck` (они используются в модуле `Testing`), поэтому посмотрим, как конфигурируются реальные тестовые цели.

Тестовые сценарии

Для реализации тестирования мы будем следовать такому сценарию:

1. Написать автономные тесты.
2. Определить и сконфигурировать исполняемые цели для тестов с помощью `AddTests()`.
3. Оснастить **тестируемую программу** (Software Under Test – **SUT**), сделав возможным сбор данных о покрытии тестами.
4. Позаботиться о стирании данных о покрытии перед следующей сборкой, чтобы предотвратить ошибки сегментации.

Начнем с написания автономных тестов. Краткости ради сделаем тесты максимально простыми (и, возможно, неполными). Сначала протестируем библиотеку.

ch15/01-full-project/test/calc/basic_test.cpp

```
#include "calc/basic.h"
#include <gtest/gtest.h>

TEST(CalcTest, SumAddsTwoInts) {
  EXPECT_EQ(4, Calc::Add(2, 2));
}

TEST(CalcTest, SubtractsTwoInts) {
  EXPECT_EQ(6, Calc::Subtract(8, 2));
}
```

Далее напишем тесты для консольного калькулятора, воспользовавшись библиотекой FXTUI. Как и раньше, понимать этот код необязательно, тесты приведены только для иллюстрации.

ch15/01-full-project/test/calc_console/tui_test.cpp

```
#include "tui.h"

#include <gmock/gmock.h>
#include <gtest/gtest.h>

#include <ftxui/screen/screen.hpp>

using namespace ::ftxui;

TEST(ConsoleCalcTest, RunWorksWithDefaultValues) {
  auto component = getTui();
  auto document = component->Render();
  auto screen = Screen::Create(Dimension::Fit(document));
  Render(screen, document);
  auto output = screen.ToString();
  ASSERT_THAT(output, testing::HasSubstr("Sum: 102"));
}
```

Этот тест отрисовывает UI на статическом объекте `Screen` и проверяет, что строковый результат содержит ожидаемую сумму. Гениальным тест не назовешь, но он хотя бы короткий.

Теперь сконфигурируем тесты с помощью двух вложенных списковых файлов. Сначала для библиотеки:

ch15/01-full-project/test/calc/CMakeLists.txt

```
add_executable(calc_test basic_test.cpp)
target_link_libraries(calc_test PRIVATE calc_static)
AddTests(calc_test)
```

Затем для исполняемого файла:

ch15/01-full-project/test/calc_console/CMakeLists.txt

```
add_executable(calc_console_test tui_test.cpp)
target_link_libraries(calc_console_test
                      PRIVATE calc_console_static)
AddTests(calc_console_test)
```

Эти конфигурации позволяют CTest выполнять тесты. Нам также нужно подготовить цели бизнес-логики к анализу покрытия и обеспечить стирание данных о покрытии перед следующей сборкой.

Добавим необходимые команды в библиотечную цель `calc`:

ch15/01-full-project/src/calc/CMakeLists.txt (продолжение)

```
# ... определение цели calc_obj
InstrumentForCoverage(calc_obj)
# ... определение цели calc_shared
# ... определение цели calc_static
CleanCoverage(calc_static)
```

В `calc_obj` оснащение добавляется с помощью дополнительного флага `--coverage`, но для цели `calc_static` вызывается функция `CleanCoverage()`. При обычных обстоятельствах мы применили бы ее к цели `calc_obj` ради единообразия, но мы используем в `CleanCoverage()` ключевое слово `PRE_BUILD`, а CMake запрещает точки вклинивания `PRE_BUILD`, `PRE_LINK` и `POST_BUILD` в объектных библиотеках.

Наконец, оснастим и очистим также и консольную цель:

ch15/01-full-project/src/calc_console/CMakeLists.txt (продолжение)

```
# ... определение цели calc_console_test
# ... BuildInfo
InstrumentForCoverage(calc_console_static)
CleanCoverage(calc_console_static)
```

После этих шагов CTest готова к выполнению наших тестов и сбору данных о покрытии. Далее мы добавим команды для активации статического анализа, поскольку хотим обеспечить высокое качество проекта при первой и всех последующих сборках.

Добавление инструментов статического анализа

Мы почти завершили конфигурирование контроля качества для наших целей. Последний шаг – подключить автоматическое форматирование и интегрировать `CppCheck`:

ch15/01-full-project/src/calc/CMakeLists.txt (продолжение)

```
# ... определение цели calc_static
# ... оснащение для вычисления покрытия и очистка

Format(calc_static .)

AddCppCheck(calc_obj)
```

Тут мы сталкиваемся с мелкой проблемой: `calc_obj` не может иметь точки вклинивания `PRE_BUILD`, поэтому мы вместо этого применяем форматирование к `calc_static`. Мы также гарантируем, что цель `calc_console_static` отформатирована и проверена.

ch15/01-full-project/src/calc_console/CMakeLists.cmake (продолжение)

```
# ... определение цели calc_console_test
# ... BuildInfo
# ... оснащение для вычисления покрытия и очистка
Format(calc_console_static .)

AddCppCheck(calc_console_static)
```

Нам еще нужно определить функции `Format` и `CppCheck`. Для реализации `Format()` мы позаимствуем код, описанный в главе 12.

ch15/01-full-project/cmake/Format.cmake

```
function(Format target directory)
  find_program(CLANG-FORMAT_PATH clang-format REQUIRED)
  set(EXPRESSION h hpp hh c cc cxx cpp)
  list(TRANSFORM EXPRESSION PREPEND "${directory}/*.")
  file(GLOB_RECURSE SOURCE_FILES FOLLOW_SYMLINKS
    LIST_DIRECTORIES false ${EXPRESSION}
  )
  add_custom_command(TARGET ${target} PRE_BUILD COMMAND
    ${CLANG-FORMAT_PATH} -i --style=file ${SOURCE_FILES}
  )
endfunction()
```

Чтобы интегрировать `CppCheck` с нашим исходным кодом, воспользуемся показанной ниже функцией.

ch15/01-full-project/cmake/CppCheck.cmake

```
function(AddCppCheck target)
  find_program(CPPCHECK_PATH cppcheck REQUIRED)
  set_target_properties(${target}
    PROPERTIES CXX_CPPCHECK
    "${CPPCHECK_PATH};--enable=warning;--error-exitcode=10"
```

```
  )
endfunction()
```

Это просто и удобно. Заметно некоторое сходство с модулем Clang-Tidy (из главы 12), что иллюстрирует согласованность функциональности CMake.

Аргументы `cppcheck` следующие:

- `--enable=warning`: активирует предупреждения. О том, как включить дополнительные проверки, см. руководство по Cppcheck (ссылка имеется в разделе «Для дополнительного чтения»);
- `--error-exitcode=1`: устанавливает код ошибки, который возвращается, когда `cppcheck` обнаруживает проблему. Это может быть любое число от 1 до 255 (поскольку 0 обозначает успешное завершение), хотя некоторые значения зарезервированы системой.

После того как все файлы в каталогах `src` и `test` созданы, наше решение может быть собрано и полностью протестировано. А мы можем перейти к установке и упаковке.

Установка и упаковка

На рис. 15.6 показано, где конфигурируется установка и упаковка нашего проекта.

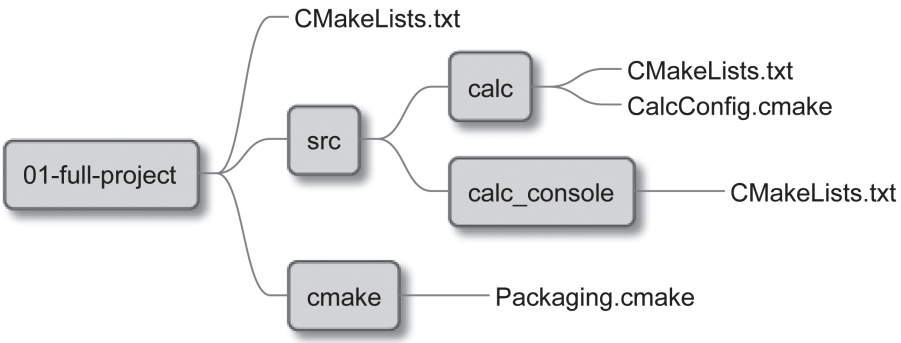


Рис. 15.6. Конфигурирование установки и упаковки

Верхнеуровневый списковый файл включает служебный модуль `Packaging`:

ch15/01-full-project/CMakeLists.txt (фрагмент)

```
# ... сконфигурировать проект
# ... разрешить тестирование
# ... включить подкаталоги src и test

include(Packaging)
```

Модуль `Packaging` детализирует конфигурацию пакета для проекта, которую мы исследуем в разделе «Упаковка с помощью CPack». Сейчас же предметом нашего внимания является установка трех основных компонентов:

- артефакты библиотеки Calc: статические и разделяемые библиотеки, заголовочные файлы и файлы экспорта целей;
- конфигурационный файл определения пакета для библиотеки Calc;
- консольный исполняемый файл Calc.

Все спланировано, пора сконфигурировать установку библиотеки.

Установка библиотеки

Установку библиотеку мы начнем с определения логических целей и конечных каталогов их артефактов, для чего используем значения по умолчанию модуля `GNUInstallDirs`, чтобы избежать ручного задания путей. Артефакты будут сгруппированы в компоненты. По умолчанию устанавливаются все файлы, но можно заказать установку только компонента `runtime`, пропустив артефакты из компонента `development`:

ch15/01-full-project/src/calc/CMakeLists.txt (продолжение)

```
# ... определение целей для библиотеки calc
# ... конфигурирование, тестирование, анализ программы

# Установка
include(GNUInstallDirs)
install(TARGETS calc_obj calc_shared calc_static
  EXPORT CalcLibrary
  ARCHIVE COMPONENT development
  LIBRARY COMPONENT runtime
  FILE_SET HEADERS COMPONENT runtime
)
```

Для UNIX-систем мы также хотим сконфигурировать послеустановочную регистрацию разделяемой библиотеки с помощью `ldconfig`:

ch15/01-full-project/src/calc/CMakeLists.txt (продолжение)

```
if (UNIX)
  install(CODE "execute_process(COMMAND ldconfig)"
    COMPONENT runtime
  )
endif()
```

Чтобы обеспечить возможность повторного использования в других проектах CMake, мы упакуем библиотеку, сгенерировав и установив файл экспорта целей и ссылающийся на него конфигурационный файл.

ch15/01-full-project/src/calc/CMakeLists.txt (продолжение)

```
install(EXPORT CalcLibrary
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
  NAMESPACE Calc::
  COMPONENT runtime
)

install(FILES "CalcConfig.cmake"
  DESTINATION ${CMAKE_INSTALL_LIBDIR}/calc/cmake
)
```

Для простоты сделаем файл `CalcConfig.cmake` минимальным.

ch15/01-full-project/src/calc/CalcConfig.cmake

```
include("${CMAKE_CURRENT_LIST_DIR}/CalcLibrary.cmake")
```

Этот файл находится в каталоге `src/calc`, потому что содержит только библиотечные цели. Если бы в нем были определения целей из других каталогов, то мы, наверное, поместили бы его на верхний уровень или в каталог `src`.

Теперь библиотека готова к установке командой `cmake --install` после сборки проекта. Однако нам еще предстоит сконфигурировать установку исполняемого файла.

Установка исполняемого файла

Конечно, мы хотим, чтобы пользователи получали удовольствие от наличия нашего исполняемого файла в своей системе, поэтому установим его с помощью CMake. Подготовка к установке двоичного исполняемого файла не вызывает трудностей, нам нужно только включить модуль `GNUInstallDirs` и вызвать единственную команду `install()`:

ch15/01-full-project/src/calc_console/CMakeLists.txt (продолжение)

```
# ... определение calc_console_static
# ... конфигурирование, тестирование, анализ программы
# ... определение стартового исполняемого файла calc_console

# Установка
include(GNUInstallDirs)
install(TARGETS calc_console
  RUNTIME COMPONENT runtime
)
```

Вот теперь исполняемый файл готов к установке. А мы перейдем к упаковке.

Упаковка с помощью CPack

Мы могли бы разгуляться и сконфигурировать множество поддерживаемых типов пакетов, но для этого проекта вполне достаточно простой конфигурации.

ch15/01-full-project/cmake/Packaging.cmake

```
# Конфигурирование CPack
set(CPACK_PACKAGE_VENDOR "Rafal Swidzinski")
set(CPACK_PACKAGE_CONTACT "email@example.com")
set(CPACK_PACKAGE_DESCRIPTION "Simple Calculator")
include(CPack)
```

Такой минималистский подход хорошо работает для стандартных архивов, например ZIP-файлов. Чтобы протестировать процессы установки и упаковки после сборки проекта, выполните следующую команду, находясь внутри дерева сборки:

```
# cpack -G TGZ -B packages
CPack: Create package using TGZ
CPack: Install projects
CPack: - Run preinstall target for: Calc
CPack: - Install project: Calc []
CPack: Create package
CPack: - package: .../packages/Calc-1.0.0-Linux.tar.gz generated.
```

На этом мы расстаемся с установкой и упаковкой и переходим к документированию.

Документирование

Последним штрихом профессионального проекта является документация. В недокументированном проекте очень трудно разобраться, когда работаешь в команде, да и представить его внешней аудитории вряд ли получится. Я бы даже рискнул сказать, что программисты часто читают собственную документацию, после того как долго не работали с каким-то файлом и хотят вспомнить, что там происходит.

Документация важна еще и по причинам соответствия законодательству и требованиям регуляторов, а также чтобы проинформировать пользователей, как работать с программой. Если время позволяет, мы должны вложиться в создание документации для своего проекта.

Обычно документацию можно отнести к одной из двух категорий:

- техническая документация (описывает интерфейсы, дизайн, классы и файлы);
- общая документация (включает все нетехнические документы).

Как мы видели в главе 13, большую часть технической документации можно автоматически сгенерировать с помощью CMake и Doxygen.

Генерирование технической документации

В некоторых проектах документация генерируется на этапе сборки и включается в пакет, но в данном случае мы решили так не делать. Однако могут быть основательные причины поступить иначе, например если документацию требуется разместить в сети.

На рис. 15.7 приведен обзор процесса генерирования документации.

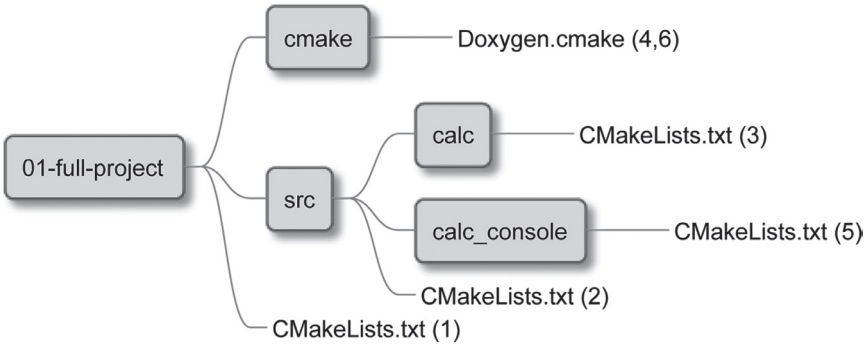


Рис. 15.7. Файлы, используемые при генерировании документации

Для генерирования документации мы напишем еще один служебный модуль CMake, `Doxygen`. Начнем с использования модуля поиска `Doxygen` и скачаем проект `doxygen-awesome-css` с темами:

ch15/01-full-project/cmake/Doxygen.cmake (фрагмент)

```
find_package(Doxygen REQUIRED)

include(FetchContent)
FetchContent_Declare(doxygen-awesome-css
  GIT_REPOSITORY
    https://github.com/jothepro/doxygen-awesome-css.git
  GIT_TAG
    v2.3.1
)
FetchContent_MakeAvailable(doxygen-awesome-css)
```

Затем нам понадобится функция для создания целей, которые генерируют документацию. Мы адаптируем код, написанный в главе 13, к обработке нескольких целей:

ch15/01-full-project/cmake/Doxygen.cmake (продолжение)

```
function(Doxygen target input)
  set(NAME "doxygen-${target}")
  set(DOXYGEN_GENERATE_HTML YES)
  set(DOXYGEN_HTML_OUTPUT ${PROJECT_BINARY_DIR}/${output})

  UseDoxygenAwesomeCss()
  UseDoxygenAwesomeExtensions()

  doxygen_add_docs("doxygen-${target}"
    ${PROJECT_SOURCE_DIR}/${input}
    COMMENT "Generate HTML documentation"
  )
endfunction()

# ... скопировано из главы 13:
#     UseDoxygenAwesomeCss
#     UseDoxygenAwesomeExtensions
```

Вызовем эту функцию для библиотечной цели.

ch15/01-full-project/src/calc/CMakeLists.txt (фрагмент)

```
# ... определение цели calc_static
# ... модули тестирования и анализа программы

Doxygen(calc src/calc)

# ... продолжение файла
```

И для исполняемого консольного калькулятора.

ch15/01-full-project/src/calc_console/CMakeLists.txt (фрагмент)

```
# ... определение цели calc_static
# ... модули тестирования и анализа программы

Doxygen(calc_console src/calc_console)

# ... продолжение файла
```

Тем самым мы добавили в проект две цели: `doxygen-calc` и `doxygen-calc_console`, которые позволяют генерировать техническую документацию по запросу. Далее посмотрим, какие еще документы следует включить.

Написание нетехнических документов для профессионального проекта

Профессиональный проект должен включать ряд нетехнических документов, хранящихся в каталоге верхнего уровня. Это существенно для уверенного понимания и соответствия юридическим нормам.

- `README`: содержит общее описание проекта.
- `LICENSE`: сведения о юридических условиях использования и распространения проекта.

Можете включить и дополнительные документы:

- `INSTALL`: пошаговые инструкции по установке;
- `CHANGELOG`: существенные изменения, внесенные в каждую версию;
- `AUTHORS`: список соразработчиков с контактной информацией, если проект разрабатывался несколькими лицами;
- `BUGS`: сведения об известных ошибках и о порядке извещения о новых.

CMake не работает с этими файлами непосредственно, потому что они не подразумевают какой-то автоматической обработки и не содержат скриптов. Тем не менее их наличие – неотъемлемая характеристика хорошо документированного проекта на C++. Ниже приведены минимальные примеры каждого документа.

ch15/01-full-project/README.md

```
# Calc Console
Calc Console – калькулятор, который складывает два числа на экране терминала.

Все математические действия выполняются с помощью библиотеки **Calc**. Эта
библиотека также входит в данный пакет.

Приложение написано на C++ и собрано с применением CMake.

## Дополнительная информация

- Инструкции по установке см. в файле INSTALL.
- Лицензия находится в файле LICENSE.
```

Файл короткий и не особенно интересный. Обратите внимание на расширение `.md` – оно означает *Markdown*, язык форматирования текста для облегчения восприятия. Веб-сайты, в частности GitHub, и многие текстовые редакторы при отображении таких файлов добавляют необходимое форматирование.

Наш файл `INSTALL` выглядит так.

ch15/01-full-project/INSTALL

```
Для установки этого программного обеспечения необходимо следующее:

- компилятор C++ с поддержкой C++17
```

```
- CMake >= 3.26
- GIT
- Doxygen + Graphviz
- CPPCheck
- Valgrind

Этот проект также зависит от GTest, GMock и FXTUI. Эти пакеты автоматически
скачиваются из внешних репозиториев в процессе установки.

Для конфигурирования проекта выполните команду:

cmake -B <temporary-directory>

Затем можете собрать проект:

cmake --build <temporary-directory>

И наконец, установить его:

cmake --install <temporary-directory>

Для генерирования документации выполните:

cmake --build <temporary-directory> -t doxygen-calc
cmake --build <temporary-directory> -t doxygen-calc_console
```

Файл `LICENSE` посложнее, потому что требует определенных знаний закона об авторских правах (и не только). Вместо того чтобы писать текст самостоятельно, мы можем поступить так же, как авторы многих других проектов, – воспользоваться готовой лицензией на программное обеспечение. Для этого проекта возьмем лицензию МТИ, которая разрешает почти все. Другие полезные ссылки см. в разделе «Для дополнительного чтения».

ch15/01-full-project/LICENSE

```
Copyright 2022 Rafal Swidzinski
Данная лицензия разрешает лицам, получившим копию данного программного
обеспечения и сопутствующей документации (далее - Программное обеспечение),
безвозмездно использовать Программное обеспечение без ограничений, включая
неограниченное право на использование, копирование, изменение, слияние,
публикацию, распространение, сублицензирование и/или продажу копий
Программного обеспечения, а также лицам, которым предоставляется данное
Программное обеспечение, при соблюдении следующих условий:

указанное выше уведомление об авторском праве и данные условия должны быть
включены во все копии или значимые части данного Программного обеспечения.

ДАННОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПРЕДОСТАВЛЯЕТСЯ "КАК ЕСТЬ", БЕЗ КАКИХ-ЛИБО
ГАРАНТИЙ, ЯВНО ВЫРАЖЕННЫХ ИЛИ ПОДРАЗУМЕВАЕМЫХ, ВКЛЮЧАЯ ГАРАНТИИ ТОВАРНОЙ
```

```
ПРИГОДНОСТИ, СООТВЕТСТВИЯ ПО ЕГО КОНКРЕТНОМУ НАЗНАЧЕНИЮ И ОТСУТСТВИЯ НАРУШЕНИЙ,
НО НЕ ОГРАНИЧИВАЯСЬ ИМИ. НИ В КАКОМ СЛУЧАЕ АВТОРЫ ИЛИ ПРАВООБЛАДАТЕЛИ НЕ НЕСУТ
ОТВЕТСТВЕННОСТИ ПО КАКИМ-ЛИБО ИСКАМ, ЗА УЩЕРБ ИЛИ ПО ИНЫМ ТРЕБОВАНИЯМ, В ТОМ
ЧИСЛЕ ПРИ ДЕЙСТВИИ КОНТРАКТА, ДЕЛИКТЕ ИЛИ ИНОЙ СИТУАЦИИ, ВОЗНИКШИМ ИЗ-ЗА
ИСПОЛЬЗОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ИЛИ ИНЫХ ДЕЙСТВИЙ С ПРОГРАММНЫМ
ОБЕСПЕЧЕНИЕМ.
```

Наконец, мы изменили файл CHANGELOG. Как уже отмечалось, полезно вести учет изменений в файле, чтобы разработчики, пользующиеся проектом, могли понять, в какой версии поддерживаются необходимые им функции. Например, было бы полезно написать, что функция перемножения была добавлена в библиотеку в версии 0.8.2. Достаточно уже и таких кратких записей, как показано ниже.

ch15/01-full-project/CHANGELOG

```
1.1.0 Обновлено для CMake 3.26 во втором издании книги.
1.0.0 Публичная версия с установщиком.
0.8.2 В библиотеку Calc добавлено умножение.
0.5.1 Появилось консольное приложение Calc.
0.2.0 Базовая библиотека Calc с функцией Sum
```

При наличии таких документов проект не только приобретает рабочую структуру, но и сообщает об использовании, изменениях и юридических условиях, снабжая пользователей и соразработчиков всей необходимой информацией.

Резюме

В этой главе мы подготовили профессиональный проект, включающий все, чему мы научились. Давайте кратко вспомним, что именно мы сделали.

Мы начали со структуры проекта и обсудили, какие файлы в каких каталогах должны находиться. Опираясь на предшествующий опыт и стремясь попрактиковаться в работе с более сложными сценариями, мы выделили в проекте две части: главное приложение для пользователей и библиотеку для разработчиков. Это определило структуру каталогов и взаимосвязи между целями CMake, которые мы намеревались собрать. Затем мы занялись конфигурированием отдельных целей сборки: мы написали исходный код для библиотеки, определили ее цели и подготовились к потреблению с параметрами позиционно-независимого кода. Для приложения, предназначенного конечным пользователям, мы тоже определили цель – исполняемый файл, написали исходный код и сконфигурировали зависимость от библиотеки FTXUI.

Зная, какие артефакты предстоит собрать, мы дополнили свой проект тестами со средствами контроля качества. Мы добавили модуль измерения покрытия для генерирования отчетов, Memcheck для проверки решения

с помощью Valgrind на этапе выполнения и CppCheck для выполнения статического анализа.

В таком виде проект уже готов к установке, поэтому мы добавили код установки библиотеки и исполняемого файла и подготовили конфигурацию пакета для CPack. Последней нашей задачей было надлежащее документирование проекта, для этого мы настроили автоматическое генерирование документации с помощью Doxygen и написали несколько простых документов, относящихся к нетехническим аспектам распространения программного обеспечения.

На этом конфигурирование проекта завершилось, и мы легко можем собрать и установить его, выполнив несколько команд CMake. Но нельзя ли воспользоваться всего одной простой командой для выполнения всего процесса? Об этом мы поговорим в последней главе.

Для дополнительного чтения

Дополнительную информацию вы можете получить по следующим ссылкам.

- Сборка одновременно статической и разделяемой библиотеки: https://stackoverflow.com/q/2152077.
- Проект библиотеки FXTUI: https://github.com/ArthurSonzogni/FTXUI.
- Документация по команде `option()`: https://cmake.org/cmake/help/latest/command/option.html.
- Руководство по подготовке к выпуску (ПО с открытым исходным кодом) от Google: https://opensource.google/docs/releasing/preparing/.
- Почему нельзя использовать Clang-Tidy для заголовков, предкомпилированных GCC: https://gitlab.kitware.com/cmake/cmake/-issues/22081#note_943104.
- Руководство по Cppcheck: https://cppcheck.sourceforge.io/manual.pdf.
- Как писать файл README: https://www.freecodecamp.org/news/how-to-write-a-good-readme-file/.
- Лицензии Creative Commons для проектов на GitHub: https://github.com/santisoler/cc-licenses.
- Распространенные лицензии проектов, распознаваемые GitHub: https://docs.github.com/en/repositories/managing-your-repositoryssettings-and-features/customizing-your-repository/licensing-a-repository.

16

Создание готовых конфигураций CMake

Готовые конфигурации впервые появились в версии CMake 3.19, чтобы упростить управление настройками проекта. До этого пользователям приходилось запоминать длинные командные строки или переопределять параметры прямо в файлах проекта – занятие непростое и чреватое ошибками. Готовые конфигурации дали пользователям возможность более естественно управлять такими аспектами, как генератор, применяемый для конфигурирования проекта, количество одновременных задач сборки и компоненты, подлежащие сборке или тестированию. Благодаря готовым конфигурациям использовать CMake стало проще. Пользователь может создать готовую конфигурацию один раз и использовать ее, когда потребуется, в результате чего командные строки для запуска CMake оказываются более единообразными и понятными. Готовые конфигурации также помогают стандартизировать настройки независимо от пользователей и компьютеров, что упрощает работу над совместными проектами.

Готовые конфигурации совместимы с четырьмя основными режимами CMake: конфигурирование сборочной системы, сборка, прогон тестов и упаковка. Они позволяют связать эти части в единый технологический процесс и тем самым сделать его более организованным и автоматизированным. Кроме того, готовые конфигурации предлагают условия и макровыражения (или просто макросы), что повышает степень контроля со стороны пользователя.

В этой главе будут рассмотрены следующие вопросы:

- использование готовых конфигураций, определенных в проекте;
- написание файла готовой конфигурации;
- определение готовых конфигураций, зависящих от этапа;
- определение готовых конфигураций технологического процесса;
- добавление условий и макросов.

Технические требования

Файлы с кодом, представленным в этой главе, имеются на GitHub по адресу https://github.com/PacktPublishing/Modern-CMake-for-Cpp-2E/tree/main/examples/ch16.

Команды, необходимые для выполнения примеров, будут приведены в каждом разделе.

Использование готовых конфигураций, определенных в проекте

Конфигурирование проектов может оказаться сложной задачей, если нужно точно определить такие элементы, как кешированные переменные, выбранные генераторы и т. д., – особенно если есть несколько способов собрать проект. Тут-то и приходят на помощь готовые конфигурации. Вместо того чтобы запоминать аргументы командной строки или писать скрипты оболочки для выполнения `cmake` с разными аргументами, мы можем создать файл готовой конфигурации и сохранить всю необходимую информацию в самом проекте.

В CMake для хранения готовых конфигураций проекта используется два факультативных файла:

- `CMakePresets.json`: официальные готовые конфигурации, предлагаемые авторами проекта;
- `CMakeUserPresets.json`: готовые конфигурации, добавленные в проект пользователями. Этот файл должен быть добавлен в список игнорируемых СУВ, чтобы пользовательские настройки случайно не попали в репозиторий и не стали общим достоянием.

Файлы готовых конфигураций должны находиться в каталоге верхнего уровня, иначе CMake их не воспримет. В каждом файле можно определить несколько конфигураций для каждого этапа: конфигурирования, сборки, тестирования и упаковки, а также конфигурации технологического процесса, охватывающие несколько этапов. Пользователь может указать, какую готовую конфигурацию выполнить, в IDE, GUI или в командной строке.

Вывести список готовых конфигураций можно, добавив аргумент `--list-presets` в командную строку вместе с указанием этапа. Например, следующая команда напечатает список готовых конфигураций этапа сборки:

```
cmake --build --list-presets
```

А готовые конфигурации этапа тестирования выводит команда

```
ctest --list-presets
```

Для использования готовой конфигурации мы действуем по такой же схеме и указываем имя конфигурации после аргумента `--preset`.

Отметим, что список готовых конфигураций этапа упаковки нельзя вывести с помощью команды `cmake`, нужно использовать `cpack`. Следующая команда служит для использования готовой конфигурации этапа упаковки:

```
cpack --preset <preset-name>
```

Выбрав готовую конфигурацию, вы, конечно, можете добавить аргументы командной строки для конкретного этапа, например задать дерево сборки или путь установки. Добавленные аргументы переопределяют те, что указаны в готовой конфигурации.

Готовые конфигурации технологических процессов – особый случай, их можно перечислить и применить, если в команде `cmake` указан дополнительный аргумент `--workflow`:

```
$ cmake --workflow --list-presets
Available workflow presets:
  "myWorkflow"
$ cmake --workflow --preset myWorkflow
Executing workflow step 1 of 4: configure preset "myConfigure"
...
```

Мы показали, как просмотреть и применить имеющиеся в проекте готовые конфигурации. Теперь обсудим структуру такого файла.

Написание файла готовой конфигурации

CMake ищет файлы `CMakePresets.json` и `CMakeUserPresets.json` в каталоге верхнего уровня. Оба файла записываются в формате JSON и имеют одинаковую структуру, так что различий между ними не так уж много. В JSON-объекте определены следующие ключи:

- `version`: целое число, определяющее версию JSON-схемы готовой конфигурации, ключ обязателен;
- `cmakeMinimumRequired`: объект, определяющий необходимую версию CMake;
- `include`: массив строк, включающий пути к внешним готовым конфигурациям (начиная с версии схемы 4);
- `configurePresets`: массив объектов, определяющий готовые конфигурации для этапа конфигурирования;
- `buildPresets`: массив объектов, определяющий готовые конфигурации для этапа сборки;

- `testPresets`: массив объектов, определяющий готовые конфигурации для этапа тестирования;
- `packagePresets`: массив объектов, определяющий готовые конфигурации для этапа упаковки;
- `workflowPresets`: массив объектов, определяющий готовые конфигурации для технологического процесса;
- `vendor`: объект, определяющий настройки, определенные IDE и другими поставщиками. CMake это поле не обрабатывает.

В файле готовой конфигурации поле `version` должно присутствовать обязательно, остальные поля факультативны. Ниже приведен пример файла готовой конфигурации (фактические конфигурации будут добавлены в последующих разделах).

ch16/01-presets/CMakePresets.json

```
{
  "version": 6,
  "cmakeMinimumRequired": {
    "major": 3,
    "minor": 26,
    "patch": 0
  },
  "include": [],
  "configurePresets": [],
  "buildPresets": [],
  "testPresets": [],
  "packagePresets": [],
  "workflowPresets": [],
  "vendor": {
  "data": "IDE-specific information"
}
}
```

Включать в объект пустые массивы, как в этом примере, необязательно; все элементы, кроме `version`, факультативны. В версии CMake 3.26 поле `version` равно `6`. Теперь, когда мы понимаем структуру файла готовой конфигурации, посмотрим, как определяются сами такие конфигурации.

Определение готовых конфигураций, зависящих от этапа

Готовые конфигурации, зависящие от этапа, – не что иное, как средство сконфигурировать отдельные этапы CMake: конфигурирования, сборки, тестирования, упаковки и установки. Они позволяют очень точно определить конфигурацию. Ниже приведен обзор общих характеристик, разделяемых

готовыми конфигурациями для всех этапов, а затем – введение в определение готовых конфигураций для конкретных этапов.

Общие характеристики всех готовых конфигураций

Существует три характеристики, относящиеся к определению готовых конфигураций независимо от этапа CMake, а именно уникальные имена полей, факультативные поля и ассоциации с готовыми конфигурациями этапа конфигурирования. В следующих разделах мы рассмотрим их по очереди.

Уникальные имена полей

Каждая готовая конфигурация должна иметь поле имени, уникальное в пределах ее этапа. Учитывая, что `CMakeUserPresets.json` (если существует) неявно включает файл `CMakePresets.json` (если тот существует), оба файла разделяют общее пространство имен, препятствующее появлению имен-дубликатов. Например, нельзя иметь в обоих файлах две готовые конфигурации этапа упаковки с именем `myPreset`.

Минимальный файл готовой конфигурации мог бы выглядеть так:

```
{
  "version": 6,
  "configurePresets": [
    {
      "name": "myPreset"
    },
    {
      "name": "myPreset2"
    }
  ]
}
```

Факультативные поля

Во всех зависящих от этапа готовых конфигурациях можно использовать одни и те же факультативные поля:

- `displayName`: строка, содержащая понятное человеку имя конфигурации;
- `description`: строка, содержащая объяснение того, что делает конфигурация;
- `inherits`: строка или массив строк, содержащие имена готовых конфигураций, которые копируются в данную в качестве базы; в дальнейшем базовая конфигурация может быть дополнена или изменена;
- `hidden`: булево поле, которое скрывает данную конфигурацию из листингов; такие скрытые готовые конфигурации можно использовать только путем наследования;

- `environment`: объект, который переопределяет переменные окружения для этого этапа; каждый ключ определяет одну переменную, а значениями могут быть строки или `null`; поддерживаются макросы;
- `condition`: объект, который разрешает или запрещает данную конфигурацию (подробнее об этом ниже);
- `vendor`: объект, который содержит зависящие от поставщика данные и следует тому же соглашению, что поле `vendor` на корневом уровне.

Готовые конфигурации могут образовывать графовую структуру наследования при условии отсутствия циклических зависимостей. Конфигурация `CMakeUserPresets.json` может наследовать готовым конфигурациям уровня проекта, но не наоборот.

Ассоциация с готовыми конфигурациями этапа конфигурирования

Все готовые конфигурации, зависящие от этапа, должны быть ассоциированы с какой-то готовой конфигурацией этапа конфигурирования, т. к. им необходимо знать о местоположении дерева сборки. Готовая конфигурация этапа конфигурирования ассоциирована сама с собой, но в готовых конфигурациях этапов сборки, тестирования и упаковки эту ассоциацию следует определить явно в поле `configurePreset`.

Вопреки интуитивным ожиданиям такая ассоциация не означает, что CMake автоматически будет выполнять эту готовую конфигурацию, когда вы решите выполнить какую-то ассоциированную с ней. Вы по-прежнему должны выполнить каждую готовую конфигурацию вручную либо использовать готовую конфигурацию технологического процесса (скоро мы дойдем до этого).

Познакомившись с этими фундаментальными концепциями, мы можем продолжить изучение специфики готовых конфигураций отдельных этапов и начнем с этапа конфигурирования. По ходу дела мы узнаем, как готовые конфигурации взаимодействуют и как их можно использовать, чтобы упростить процесс конфигурирования и сборки проекта в CMake.

Определение готовых конфигураций этапа конфигурирования

Как уже было сказано, готовые конфигурации этапа конфигурирования находятся в массиве `configurePresets`. Их список можно напечатать, добавив аргумент `--list-presets` в командную строку для этапа конфигурирования:

```
cmake --list-presets
```

Чтобы сконфигурировать проект, применив выбранную готовую конфигурацию, укажите ее имя после аргумента `--preset`:

```
cmake --preset myConfigurationPreset
```

В готовой конфигурации этапа конфигурирования имеется ряд общих полей, например `name` и `description`, а также свой собственный уникальный набор факультативных полей. Ниже приведены упрощенные описания наиболее важных:

- `generator`: строка, определяющая, какой генератор использовать в этой готовой конфигурации; обязательно, если версия схемы < 3;
- `architecture` и `toolset`: строка, которая конфигурирует генераторы, поддерживающие такие параметры;
- `binaryDir`: строка, которая содержит относительный или абсолютный путь к дереву сборки; обязательно, если версия схемы < 3; поддерживаются макросы;
- `installDir`: строка, которая содержит относительный или абсолютный путь к каталогу установки; обязательно, если версия схемы < 3; поддерживаются макросы;
- `cacheVariables`: отображение, содержащее кешированные переменные; в значениях поддерживаются макросы.

При определении отображения `cacheVariables` не забывайте, в каком порядке разрешаются переменные в проекте. Как показано на рис. 16.1, любая кешированная переменная, заданная в командной строке, переопределяет переменные в готовой конфигурации. Любая кешированная переменная или переменная окружения, определенная в готовой конфигурации, переопределяет те, что взяты из кеша или окружения хоста.

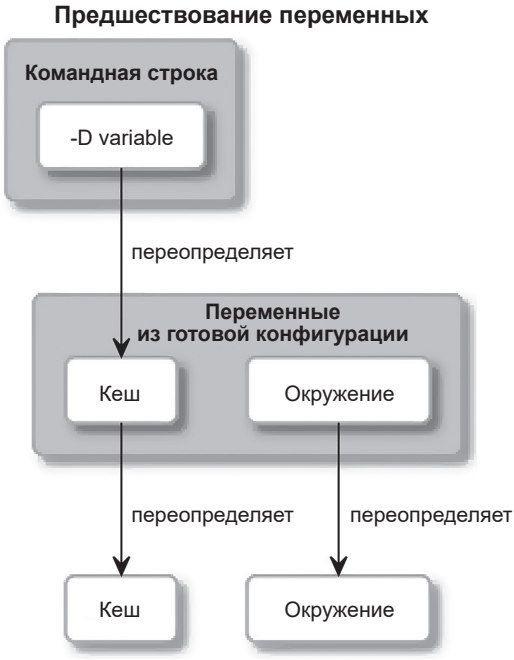


Рис. 16.1. Как готовые конфигурации переопределяют CMakeCache.txt и переменные из системного окружения

Объявим простую готовую конфигурацию этапа конфигурирования, в которой определим генератор, дерево сборки и путь к каталогу установки.

ch16/01-presets/CMakePresets.json (продолжение)

```
...

  "configurePresets": [
    {
      "name": "myConfigure",
      "displayName": "Configure Preset",
      "description": "Ninja generator",
      "generator": "Ninja",
      "binaryDir": "${sourceDir}/build",
      "installDir": "${sourceDir}/build/install"
    }
  ],
...
```

На этом мы завершаем введение в готовые конфигурации этапа конфигурирования и переходим к конфигурациям этапа сборки.

Определение готовых конфигураций этапа сборки

Вряд ли вы удивитесь, узнав, что готовые конфигурации этапа сборки хранятся в массиве `buildPresets`. Их список можно напечатать, добавив аргумент `--list-presets` в командную строку для этапа сборки:

```
cmake --build --list-presets
```

Чтобы собрать проект, применив выбранную готовую конфигурацию, укажите ее имя после аргумента `--preset`:

```
cmake --build --preset myBuildingPreset
```

В готовой конфигурации этапа сборки имеется ряд общих полей, например `name` и `description`, а также свой собственный уникальный набор факультативных полей. Ниже приведены упрощенные описания наиболее важных:

- `jobs`: целое число, задающее количество параллельных задач, запускаемых для сборки проекта;
- `targets`: строка или массив строк, задающих собираемые цели, поддерживаются макросы;
- `configuration`: строка, которая определяет тип сборки для генераторов, поддерживающих несколько конфигураций (`Debug`, `Release` и т. д.);
- `cleanFirst`: булево значение, позволяющее указать, что проект должен быть очищен перед сборкой.

Вот и все. Теперь мы можем написать готовую конфигурацию этапа сборки.

ch16/01-presets/CMakePresets.json (продолжение)

```
...

  "buildPresets": [
    {
      "name": "myBuild",
      "displayName": "Build Preset",
      "description": "Four jobs",
      "configurePreset": "myConfigure",
      "jobs": 4
    }
  ],
...
```

Обратите внимание, что обязательное поле `configurePreset` указывает на готовую конфигурацию `myConfigure`, которую мы определили в предыдущем разделе.

Определение готовых конфигураций этапа тестирования

Готовые конфигурации этапа тестирования хранятся в массиве `testPresets`. Их список можно напечатать, добавив аргумент `--list-presets` в командную строку для этапа тестирования:

```
ctest --build --list-presets
```

Чтобы собрать проект, применив выбранную готовую конфигурацию, укажите ее имя после аргумента `--preset`:

```
ctest --build --preset myTestPreset
```

В готовой конфигурации этапа тестирования также имеется свой собственный уникальный набор факультативных полей. Ниже приведены упрощенные описания наиболее важных:

- `configuration`: строка, которая определяет тип сборки для генераторов, поддерживающих несколько конфигураций (`Debug`, `Release` и т. д.);
- `output`: объект, конфигурирующий вывод;
- `filter`: объект, который определяет, какие тесты прогонять;
- `execution`: объект, конфигурирующий выполнение тестов.

Каждый объект отображает соответствующие аргументы командной строки на конфигурационные значения. Мы опишем несколько важных парамет-

ров, но этот список неполный. На полную справочную информацию имеется ссылка в разделе «Для дополнительного чтения».

К числу факультативных полей объекта `output` относятся:

- `shortProgress`: булево, ход выполнения будет отображаться в одной строке;
- `verbosity`: строка, задающая уровень детализации диагностики: по умолчанию, детальная или очень детальная;
- `outputOnFailure`: булево, печатается вывод программы, если тест не прошел;
- `quiet`: булево, подавляется весь вывод.

Вот некоторые допустимые поля объекта `exclude`:

- `name`: тесты, имена которых сопоставляются с указанным в этой строке регулярным выражением, исключаются; поддерживаются макросы;
- `label`: тесты, метки которых сопоставляются с указанным в этой строке регулярным выражением, исключаются; поддерживаются макросы;
- `fixtures`: объект, определяющий, какие фикстуры исключить из теста (детали см. в официальной документации).

Наконец, для объекта `execution` определено следующее факультативное поле:

- `outputLogFile`: строка, которая определяет путь к файлу журнала, поддерживаются макросы.

Объект `filter` может содержать ключи `include` и `exclude` для задания фильтрации тестов; следующая частично заполненная структура иллюстрирует это:

```
"testPresets": [
  {
    "name": "myTest",
    "configurePreset": "myConfigure",
    "filter": {
      "include": {
        ... name, label, index, useUnion ...
                },
      "exclude": {
        ... name, label, fixtures ...
                }
    }
  }
],

...
```

Каждый ключ определяет свой собственный параметрический объект.

Для объекта `include` определены следующие поля:

- `name`: тесты, имена которых сопоставляются с указанным в этой строке регулярным выражением, включаются; поддерживаются макросы;

- `label`: тесты, метки которых сопоставляются с указанным в этой строке регулярным выражением, включаются; поддерживаются макросы;
- `index`: объект, который выбирает, какие тесты прогонять; содержит ключи `start`, `end` и `stride` с целыми значениями и массив целых `specificTests`; поддерживаются макросы;
- `useUnion`: булево значение; если равно `true`, то множества тестов, определяемых полями `index` и `name`, объединяются, а не пересекаются.

Для объекта `exclude` определены следующие поля:

- `name`: тесты, имена которых сопоставляются с указанным в этой строке регулярным выражением, исключаются; поддерживаются макросы;
- `label`: тесты, метки которых сопоставляются с указанным в этой строке регулярным выражением, исключаются; поддерживаются макросы;
- `fixtures`: объект, определяющий, какие фикстуры исключить из теста (детали см. в официальной документации).

Наконец, мы можем добавить объект `execution`:

```
"testPresets": [
  {
    "name": "myTest",
    "configurePreset": "myConfigure",
    "execution": {
                ... stopOnFailure, enableFailover, ...
                ... jobs, repeat, scheduleRandom, ...
                ... timeout, noTestsAction ...
                }
  }
],

...
```

Он может содержать следующие поля:

- `stopOnFailure`: булево, прогон тестов останавливается, если хотя бы один не прошел;
- `enableFailover`: булево, выполнение ранее прерванных тестов возобновляется;
- `jobs`: целое, количество тестов, выполняемых параллельно;
- `repeat`: объект, определяющий, как повторять тесты; объект может иметь следующие поля:
  - `mode` – строка, которая может принимать одно из следующих значений: `until-fail`, `until-pass`, `after-timeout`;
  - `count` – целое число, определяющее количество повторений;
- `scheduleRandom`: булево, включает случайный порядок выполнения;
- `timeout`: целое число, задающее общее предельное время (в секундах) выполнения всех тестов;
- `noTestsAction`: строка, которая определяет, что делать, если ни один тест не найден, допустимы значения `default`, `error`, `ignore`.

Хотя конфигурационных параметров много, простые готовые конфигурации тоже имеют право на существование:

ch16/01-presets/CMakePresets.json (продолжение)

```
...

"testPresets": [
  {
    "name": "myTest",
    "displayName": "Test Preset",
    "description": "Output short progress",
    "configurePreset": "myConfigure",
    "output": {
      "shortProgress": true
    }
  }
],
...
```

Как и в случае готовой конфигурации сборки, мы задали обязательное поле `configurePreset`, которое связывает все вместе.

Определение готовых конфигураций этапа упаковки

Готовые конфигурации этапа упаковки впервые появились в схеме версии 6, т. е. для работы с ними нужна как минимум версия CMake 3.25. Эти конфигурации следует поместить в массив `packagePresets`. Их список можно напечатать, добавив аргумент `--list-presets` в командную строку для этапа упаковки:

```
cpack --list-presets
```

Чтобы упаковать проект с использованием готовой конфигурации, задайте ее имя после аргумента `--preset`:

```
cpack --preset myTestPreset
```

Готовая конфигурация этапа упаковки имеет те же самые общие поля, что и другие готовые конфигурации, а также ряд собственных факультативных полей:

- `generators`: массив строк, определяющий, какие генераторы пакетов использовать (`ZIP`, `7Z`, `DEB` и т. д.);
- `configuration`: массив строк, определяющий список типов сборки, упаковываемых CPack (`Debug`, `Release` и т. д.);

- `packageName`, `packageVersion`, `packageDirectory` и `vendorName`: строки, содержащие метаданные созданного пакета.

Дополним наш файл короткой готовой конфигурацией этапа упаковки:

ch16/01-presets/CMakePresets.json (продолжение)

```
...
"packagePresets": [
  {
    "name": "myPackage",
    "displayName": "Package Preset",
    "description": "ZIP generator",
    "configurePreset": "myConfigure",
    "generators": [
      "ZIP"
    ]
  }
],

...
```

Такая конфигурация ускорит создание пакета проекта, но не хватает еще одной части: установки проекта. Посмотрим, что с этим можно сделать.

Добавление готовой конфигурации установки

Вы, наверное, заметили, что объект `CMakePresets.json` не поддерживает определения ключа `"installPresets"`. Не существует явного способа установить проект с помощью готовой конфигурации, и это кажется странным, потому что в готовой конфигурации этапа конфигурирования есть поле `installDir`! Так что же, возвращаться к ручным командам установки?

К счастью, это лишнее. Есть обходной путь, позволяющий использовать готовую конфигурацию этапа сборки для достижения цели. Взгляните:

ch16/01-presets/CMakePresets.json (продолжение)

```
...

"buildPresets": [
  {
    "name": "myBuild",

    ...

  },
  {
    "name": "myInstall",
    "displayName": "Installation",
```

```
    "targets" : "install",
    "configurePreset": "myConfigure"
  }
],

...
```

В этой конфигурации поле `targets` содержит значение `install`. Цель `install` неявно определяется проектом, когда установка правильно сконфигурирована. Сборка с помощью такой конфигурации выполнит шаги, необходимые для установки проекта в каталог `installDir`, заданный в ассоциированной готовой конфигурации этапа конфигурирования (если поле `installDir` пусто, то будет использовано местоположение по умолчанию):

```
$ cmake --build --preset myInstall
[0/1] Install the project...
-- Install configuration: ""
-- Installing: .../install/include/calc/basic.h
-- Installing: .../install/lib/libcalc_shared.so
-- Installing: .../install/lib/libcalc_static.a
-- Installing: .../install/lib/calc/cmake/CalcLibrary.cmake
-- Installing: .../install/lib/calc/cmake/CalcLibrary-noconfig.cmake
-- Installing: .../install/lib/calc/cmake/CalcConfig.cmake
-- Installing: .../install/bin/calc_console
-- Set non-toolchain portion of runtime path of ".../install/bin/calc_console" to ""
```

Этот изящный прием позволяет сэкономить несколько тактов. Еще лучше было бы, если бы мы могли предоставить конечным пользователям единственную команду, которая позаботилась бы обо всем: от конфигурирования до установки. Что ж, это возможно – с помощью готовых конфигураций технологического процесса.

Определение готовых конфигураций технологического процесса

Готовые конфигурации технологического процесса – это полное решение для автоматизации проекта. Они позволяют автоматически выполнить несколько готовых конфигураций для конкретных этапов в предопределенном порядке. Таким образом, мы можем произвести сборку от начала до конца за один шаг.

Для получения списка готовых конфигураций технологического процесса, определенных в проекте, выполните команду

```
cmake --workflow --list-presets
```

Чтобы выбрать и применить готовую конфигурацию, выполните команду

```
cmake -workflow --preset <preset-name>
```

Кроме того, задав флаг `--fresh`, мы можем очистить дерево сборки и кеш.

Определить готовую конфигурацию технологического процесса просто; достаточно задать имя и – при желании – поля `displayName` и `description`, как в готовых конфигурациях для конкретных этапов. После этого мы должны перечислить все готовые конфигурации для этапов, которые предстоит выполнить в технологическом процессе. Для этого служит массив `steps`, содержащий объекты со свойствами `type` и `name`, как показано ниже.

ch16/01-presets/CMakePresets.json (продолжение)

```
...

"workflowPresets": [
  {
    "name": "myWorkflow",
    "steps": [
      {
        "type": "configure",
        "name": "myConfigure"
      },
      {
        "type": "build",
        "name": "myBuild"
      },
      {
        "type": "test",
        "name": "myTest"
      },
      {
        "type": "package",
        "name": "myPackage"
      },
      {
        "type": "build",
        "name": "myInstall"
      }
    ]
...
```

Каждый объект в массиве `steps` ссылается на готовую конфигурацию, определенную ранее в этой главе, указывая ее тип (`configure`, `build`, `test` или `package`) и имя. В совокупности эти конфигурации выполняют все шаги, необходимые для полной сборки и установки проекта, нуждаясь всего в одной команде:

```
cmake --workflow --preset myWorkflow
```

Готовые конфигурации технологического процесса – окончательное решение задачи автоматизации сборки, тестирования, упаковки и установки проекта на C++. Далее мы рассмотрим, как обработать некоторые неординарные ситуации с помощью условий и макросов.

Добавление условий и макросов

Обсуждая общие поля всех готовых конфигураций для конкретных этапов, мы упомянули поле `condition`. Настало время вернуться к этому вопросу. Поле **условия** (condition) разрешает или запрещает готовую конфигурацию, а его потенциал раскрывается в полной мере при описании технологических процессов. По сути дела, условие позволяет обойти готовые конфигурации, не отвечающие определенным условиям, и создать другие, которые им отвечают.

Для условий нужна схема готовых конфигураций версии 3 (появившаяся в CMake 3.22) и JSON-объекты, которые кодируют несколько простых логических операций, способных определить, соответствуют ли обстоятельства, например ОС, переменные окружения или даже выбранные генераторы сценарию применения готовой конфигурации. CMake предоставляет эти данные с помощью макросов – ограниченного множества доступных только для чтения переменных, которые разрешено использовать в файлах готовых конфигураций.

Структура объекта условия зависит от типа проверки. Каждое условие должно включать поле `type` и дополнительные поля, определяемые типом. Распознаются следующие простые типы:

- `const`: проверяет, что в поле `value` находится булево значение `true`;
- `equals`, `notEquals`: сравнивает значения в полях `lhs` и `rhs`;
- `inList` и `notInList`: проверяет, присутствует ли значение в поле `string` в массиве, находящемся в поле `list`;
- `matches` и `notMatches`: сопоставляет значение в поле `string` с образцом (регулярным выражением) в поле `regex`.

Ниже приведен пример условия:

```
"condition": {
               "type": "equals",
               "lhs": "${hostSystemName}",
               "rhs": "Windows"
             }
```

Смысл условия `const` прежде всего в том, чтобы деактивировать готовую конфигурацию, не удаляя ее из JSON-файла. Все прочие простые условия допускают использование макросов в соответствующих полях: `lhs`, `rhs`, `string`, `list` и `regex`.

Составные типы условий, функционирующие как операции «not», «and» и «or», используют другие условия в качестве аргументов:

- `not`: булево отрицание условия в поле `condition`;
- `anyOf` и `allOf`: проверяется, что хотя бы одно или все условия в массиве `conditions` равны `true`.

Например:

```
"condition": {
              "type": "anyOf",
              "conditions": [
                              {
                                "type": "equals",
                                "lhs": "${hostSystemName}",
                                "rhs": "Windows"
                              },{
                                "type": "equals",
                                "lhs": "${hostSystemName}",
                                "rhs": "Linux"
                              }
                            ]
            }
```

Значением этого условия будет `true`, если система – Linux или Windows.

В этих примерах мы использовали наш первый макрос: `${hostSystemName}`. Макросы имеют простой синтаксис, а их состав ограничен:

- `${sourceDir}`: путь к дереву исходного кода;
- `${sourceParentDir}`: путь к родительскому каталогу дерева исходного кода;
- `${sourceDirName}`: имя каталога проекта;
- `${presetName}`: имя готовой конфигурации;
- `${generator}`: генератор, использованный для создания сборочной системы;
- `${hostSystemName}`: имя системы: `Linux`, `Windows` или `Darwin` в случае macOS;
- `${fileDir}`: это имя файла, содержащего текущую готовую конфигурацию (применимо, когда для импорта внешних готовых конфигураций используется массив `include`);
- `${dollar}`: экранированный знак доллара ($);
- `${pathListSep}`: системно-зависимый разделитель элементов пути;
- `$env{<variable-name>}`: возвращает переменную окружения, если она определена в готовой конфигурации (с учетом регистра), или значение из родительского окружения;
- `$penv{<variable-name>}`: возвращает переменную из родительского окружения;
- `$vendor{<macro-name>}`: позволяет производителям IDE определять собственные макросы.

Эти макросы предлагают гибкость, достаточную для использования в готовых конфигурациях и условиях, что позволяет пропускать шаги технологического процесса при необходимости.

Резюме

Мы дали достаточно полный обзор готовых конфигураций, впервые появившихся в версии CMake 3.19 с целью упростить управление проектом. Готовые конфигурации позволяют авторам продуктов предлагать заранее подготовленные средства конфигурирования всех этапов сборки и поставки проекта. Они не только облегчают работу с CMake, но и способствуют единообразию и могут учитывать окружение.

Мы объяснили структуру и порядок использования файлов `CMakePresets.json` и `CMakeUserPresets.json` и показали, как определяются различные типы готовых конфигураций: этапов конфигурирования, сборки, тестирования, упаковки, а также конфигураций технологического процесса. Все типы описаны подробно: мы узнали об общих полях, об установлении отношений наследования между готовыми конфигурациями и об уникальных параметрах каждой конфигурации.

Для готовых конфигураций этапа конфигурирования мы рассмотрели такие важные вопросы, как выбор генератора, каталогов сборки и установки, а также связывания готовых конфигураций с помощью поля `configurePreset`. Мы также знаем, как работать с готовыми конфигурациями этапа сборки и как задавать количество параллельно выполняемых задач, цели и параметры очистки. Затем узнали, как готовые конфигурации этапа тестирования помогают выбирать тесты с помощью развитых средств фильтрации и упорядочения, о форматировании вывода и о таких параметрах выполнения, как величина тайм-аута и отказоустойчивость. Мы теперь понимаем, как управлять готовыми конфигурациями этапа упаковки путем задания генераторов пакетов и метаданных пакетов. Мы даже описали прием, позволяющий выполнить этап установки с помощью специализированного применения готовой конфигурации этапа сборки.

Затем мы показали, как готовые конфигурации технологического процесса позволяют группировать несколько готовых конфигураций для конкретных этапов. Наконец, мы обсудили условия и макровыражения, которые дают авторам проектов больший контроль над поведением отдельных конфигураций и их интеграцией в технологический процесс.

Наше путешествие по миру CMake подошло к концу! Мои поздравления – теперь вы располагаете всеми инструментами, необходимыми для разработки, тестирования и упаковки высококачественных программ на C++. Лучший способ идти дальше – применять все, чему вы научились, и создавать отличное программное обеспечение для пользователей. Удачи!

Для дополнительного чтения

Дополнительную информацию вы можете почерпнуть из следующего ресурса:

- официальная документация по готовым конфигурациям: https://cmake.org/cmake/help/latest/manual/cmake-presets.7.html.

Приложение

В любом языке имеются служебные команды, предназначенные для решения различных задач, и язык CMake – не исключение. Он предлагает средства для арифметических и поразрядных операций, для операций со строками, списками и файлами. Хотя потребность в этих командах снизилась вследствие усовершенствований самого языка и разработки многочисленных модулей, в проектах с высокой степенью автоматизации они по-прежнему могут быть интересны. В настоящее время наибольшую пользу они способны принести в скриптах CMake, вызываемых командой `cmake -P <filename>`.

В это приложение мы включили сводку различных команд CMake и их режимов, оно может служить удобным справочником или упрощенной версией официальной документации. За более подробной информацией отсылаем вас к ссылкам в конце приложения.

Приведенный материал актуален для версии CMake 3.26.6.

В приложении будут рассмотрены следующие вопросы:

- команда `string()`;
- команда `list()`;
- команда `file()`;
- команда `math()`.

Команда string()

Команда `string()` предназначена для операций со строками. У нее есть несколько режимов для выполнения различных действий: поиска и замены, модификации, сравнения, хеширования, генерирования и операций со строками в формате JSON (последние доступны начиная с версии CMake 3.19).

Полную информацию смотрите в онлайновой документации: https://cmake.org/cmake/help/latest/command/string.html.



Отметим, что те режимы `string()`, которые вообще принимают аргумент `<input>`, принимают произвольное число таких аргументов и конкатенируют их перед выполнением команды, т. е. команда

```
string(PREPEND myVariable "a" "b" "c")
```

эквивалентна следующей:

```
string(PREPEND myVariable "abc")
```

Поиск и замена

Поддерживаются следующие режимы.

- `string(FIND <haystack> <pattern> <out> [REVERSE])` ищет подстроку `<pattern>` в строке `<haystack>` и записывает позицию найденной подстроки (целое число) в переменную `<out>`. Если задан флаг `REVERSE`, то поиск производится от конца строки к началу. Работает только для строк в кодировке ASCII (многобайтовые кодировки не поддерживаются).
- `string(REPLACE <pattern> <replace> <out> <input>)` заменяет все вхождения `<pattern>` в `<input>` на `<replace>` и сохраняет результат в переменной `<out>`.
- `string(REGEX MATCH <pattern> <out> <input>)` сопоставляет строку `<input>` с регулярным выражением `<pattern>` и сохраняет первое соответствие в переменной `<out>`.
- `string(REGEX MATCHALL <pattern> <out> <input>)` сопоставляет строку `<input>` с регулярным выражением `<pattern>` и сохраняет все найденные соответствия в переменной `<out>` в виде списка через запятую.
- `string(REGEX REPLACE <pattern> <replace> <out> <input>)` заменяет все подстроки строки `<input>`, сопоставленные с регулярным выражением `<pattern>`, выражением `<replace>` и сохраняет результат в переменной `<out>`.

Регулярные выражения записываются в синтаксисе C++, как определено в заголовке `<regex>` стандартной библиотеки. В выражении `<replace>` можно использовать обратные ссылки на сопоставленные подстроки: `\\1`, `\\2` ... (двойной символ \ необходим для правильного разбора аргументов).

Модификация

Поддерживаются следующие режимы.

- `string(APPEND <out> <input>)` изменяет строку, хранящуюся в `<out>`, дописывая в ее конец строку `<input>`.
- `string(PREPEND <out> <input>)` изменяет строку, хранящуюся в `<out>`, дописывая в ее начало строку `<input>`.
- `string(CONCAT <out> <input>)` конкатенирует все строки в `<input>` и сохраняет результат в переменной `<out>`.
- `string(JOIN <glue> <out> <input>)` конкатенирует строки, переданные в `<input>`, вставляя между ними значение `<glue>`, и сохраняет результат в переменной `<out>` (не используйте этот режим для списковых переменных).
- `string(TOLOWER <string> <out>)` преобразует `<string>` в нижний регистр и сохраняет результат в переменной `<out>`.
- `string(TOUPPER <string> <out>)` преобразует `<string>` в верхний регистр и сохраняет результат в переменной `<out>`.
- `string(LENGTH <string> <out>)` подсчитывает количество байтов в `<string>` и сохраняет результат в переменной `<out>`.
- `string(SUBSTRING <string> <begin> <length> <out>)` выделяет подстроку `<string>` длиной `<length>` байт, начинающуюся в позиции `<begin>`, и со-

храняет результат в переменной <out>. Значение длины -1 означает «до конца строки».
- string(STRIP <string> <out>) удаляет начальные и хвостовые пробелы из <string> и возвращает результат в переменной <out>.
- string(GENEX_STRIP <string> <out>) удаляет все генераторные выражения, встречающиеся в <string>, и возвращает результат в переменной <out>.
- string(REPEAT <string> <count> <out>) генерирует строку, содержащую <count> повторений <string>, и сохраняет результат в переменной <out>.

Сравнение

Сравнение строк имеет следующий вид:

```
string(COMPARE <operation> <stringA> <stringB> <out>)
```

Аргумент <operation> может принимать следующие значения:

- LESS;
- GREATER;
- EQUAL;
- NOTEQUAL;
- LESS_EQUAL;
- GREATER_EQUAL.

Эта команда сравнивает строки <stringA> и <stringB> и сохраняет результат (true или false) в переменной <out>.

Хеширование

Сигнатура режима хеширования следующая:

```
string(<hashing-algorithm> <out> <string>)
```

Команда хеширует строку <string> алгоритмом <hashing-algorithm> и сохраняет результат в переменной <out>. Поддерживаются следующие алгоритмы:

- MD5: Message-Digest Algorithm 5, RFC 1321;
- SHA1: US Secure Hash Algorithm 1, RFC 3174;
- SHA224: US Secure Hash Algorithms, RFC 4634;
- SHA256: US Secure Hash Algorithms, RFC 4634;
- SHA384: US Secure Hash Algorithms, RFC 4634;
- SHA512: US Secure Hash Algorithms, RFC 4634;
- SHA3_224: Keccak SHA-3;
- SHA3_256: Keccak SHA-3;
- SHA3_384: Keccak SHA-3;
- SHA3_512: Keccak SHA-3.

Генерирование

Поддерживаются следующие режимы:

- `string(ASCII <number>... <out>)` сохраняет ASCII-символы заданного числа `<number>` в переменной `<out>`;
- `string(HEX <string> <out>)` преобразует строку `<string>` в 16-ричное представление и сохраняет результат в переменной `<out>` (начиная с версии CMake 3.18);
- `string(CONFIGURE <string> <out> [@ONLY] [ESCAPE_QUOTES])` работает в точности как `configure_file()`, но для строк. Результат сохраняется в переменной `<out>`. Напомним, что ключевое слово `@ONLY` ограничивает замены ссылками на переменные вида `@VARIABLE@`;
- `string(MAKE_C_IDENTIFIER <string> <out>)` преобразует символы `<string>`, отличные от букв и цифр, в знаки подчеркивания и сохраняет результат в переменной `<out>`;
- `string(RANDOM [LENGTH <len>] [ALPHABET <alphabet>] [RANDOM_SEED <seed>] <out>)` генерирует случайную строку длины `<len>` символов (по умолчанию `5`) из алфавита `<alphabet>` и сохраняет результат в переменной `<out>`. По умолчанию `len` равно 5, а в качестве алфавита подразумевается множество всех букв и цифр. Если аргумент `<seed>` задан, то он служит начальным значением генератора случайных чисел;
- `string(TIMESTAMP <out> [<format>] [UTC])` генерирует строку, представляющую текущие дату и время, сохраняет результат в переменной `<out>`;
- `string(UUID <out> NAMESPACE <ns> NAME <name> TYPE <type>)` генерирует универсально-уникальный идентификатор. Использовать этот режим непросто: необходимо задать пространство имен (которое само должно быть допустимым UUID), имя (например, доменное имя) и алгоритм (`MD5` или `SHA1`).

JSON

Операции со строками в формате JSON имеют следующую сигнатуру:

```
string(JSON <out> [ERROR_VARIABLE <error>] <operation + args>)
```

Поддерживается несколько операций. Все они сохраняют результат в переменной `<out>`, а ошибки – в переменной `<error>`.

- `GET <json> <member|index>...` возвращает значение одного или нескольких элементов в позициях строки `<json>`, заданных путем `<member>` или индексом `<index>`.
- `TYPE <json> <member|index>...` возвращает тип одного или нескольких элементов в позициях строки `<json>`, заданных путем `<member>` или индексом `<index>`.

- MEMBER <json> <member|index>... <array-index> возвращает имя array-index-го члена в позициях строки <json>, заданных путем <member> или индексом <index>.
- LENGTH <json> <member|index>... возвращает количество элементов в позициях строки <json>, заданных путем <member> или индексом <index>.
- REMOVE <json> <member|index>... возвращает результат удаления одного или нескольких элементов в позициях строки <json>, заданных путем <member> или индексом <index>.
- SET <json> <member|index>... <value> возвращает результат вставки или замены значения <value> одним или более элементами в позициях строки <json>, заданных путем <member> или индексом <index>.
- EQUAL <jsonA> <jsonB> возвращает true или false в зависимости от того, равны строки <jsonA> и <jsonB> или нет.

Команда list()

Эта команда реализует простые операции со списками: чтение, поиск, модификацию и упорядочение. В некоторых режимах список изменяется. Не забывайте скопировать исходное значение, если оно может понадобиться.

Полную информацию см. в онлайновой документации по адресу https://cmake.org/cmake/help/latest/command/list.html.

Чтение

Поддерживаются следующие режимы.

- list(LENGTH <list> <out>) подсчитывает количество элементов в переменной <list> и сохраняет результат в переменной <out>.
- list(GET <list> <index>... <out>) копирует элементы <list>, заданные списком индексов <index>, в переменную <out>.
- list(JOIN <list> <glue> <out>) конкатенирует элементы списка <list>, разделяя их значением <glue>, и сохраняет результат в переменной <out>.
- list(SUBLIST <list> <begin> <length> <out>) работает как режим GET, но индексы задаются не явно, а диапазоном. Если <length> равно -1, то возвращаются элементы <list>, начиная с индекса <begin> и до конца списка.

Поиск

В этом режиме просто ищется элемент <needle> в списке <list>, и его индекс (или -1, если элемент не был найден) сохраняется в переменной <out>.

```
list(FIND <list> <needle> <out>)
```

Модификация

Поддерживаются следующие режимы.

- `list(APPEND <list> <element>...)` добавляет значения одного или нескольких элементов `<element>` в конец списка `<list>`.
- `list(PREPEND <list> [<element>...])` работает как `APPEND`, но элементы добавляются в начало списка `<list>`.
- `list(FILTER <list> {INCLUDE | EXCLUDE} REGEX <pattern>)` фильтрует список `<list>`, оставляя (`INCLUDE`) или исключая (`EXCLUDE`) элементы, соответствующие регулярному выражению `<pattern>`.
- `list(INSERT <list> <index> [<element>...])` добавляет один или более элемент в список `<list>`, начиная с позиции `<index>`.
- `list(POP_BACK <list> [<out>...])` удаляет один элемент из конца списка `<list>` и сохраняет его в факультативной переменной `<out>`. Если задано несколько переменных `<out>`, то удаляется столько элементов, чтобы заполнить их все.
- `list(POP_FRONT <list> [<out>...])` работает как `POP_BACK`, но элементы удаляются из начала списка `<list>`.
- `list(REMOVE_ITEM <list> <value>...)` – сокращенная запись режима `FILTER EXCLUDE`, но без поддержки регулярных выражений.
- `list(REMOVE_AT <list> <index>...)` удаляет элементы из списка `<list>` с индексами `<index>`.
- `list(REMOVE_DUPLICATES <list>)` удаляет дубликаты из списка `<list>`.
- `list(TRANSFORM <list> <action> [<selector>] [OUTPUT_VARIABLE <out>])` применяет заданное преобразование к элементам списка `<list>`. По умолчанию действие применяется ко всем элементам, но их множество можно ограничить селектором `<selector>`. Переданный список будет изменен, если не задано ключевое слово `OUTPUT_VARIABLE`, а в этом случае результат сохраняется в переменной `<out>`.

Поддерживаются следующие селекторы: `AT <index>`, `FOR <start> <stop> [<step>]` и `REGEX <pattern>`.

Поддерживаются действия `APPEND <string>`, `PREPEND <string>`, `TOLOWER`, `TOUPPER`, `STRIP`, `GENEX_STRIP` и `REPLACE <pattern> <expression>`. Они работают как одноименные режимы команды `string()`.

Упорядочение

Поддерживаются следующие режимы.

- `list(REVERSE <list>)` изменяет порядок элементов `<list>` на противоположный.
- `list(SORT <list>)` сортирует список по алфавиту. Подробнее о параметрах см. в онлайновой документации.

Команда file()

Эта команда реализует разнообразные операции с файлами: чтение, передачу, блокировку и архивацию. У нее также имеются режимы для инспектирования файловой системы и операций над строками, представляющими пути.

Полную информацию см. в онлайновой документации по адресу https://cmake.org/cmake/help/latest/command/file.html.

Чтение

Поддерживаются следующие режимы.

- `file(READ <filename> <out> [OFFSET <o>] [LIMIT <max>] [HEX])` читает файл с именем `<filename>` в переменную `<out>`. Чтение может начинаться со смещения `<o>`, а количество прочитанных байтов может быть ограничено значением `<max>` (оба параметра факультативны). Флаг `HEX` означает, что данные следует преобразовать в 16-ричную форму.
- `file(STRINGS <filename> <out>)` читает строки из файла `<filename>` в переменную `<out>`.
- `file(<hashing-algorithm> <filename> <out>)` вычисляет хеш файла `<filename>`, применяя алгоритм `<hashing-algorithm>`, и сохраняет результат в переменной `<out>`. Поддерживаются те же алгоритмы хеширования, что для команды `string()`.
- `file(TIMESTAMP <filename> <out> [<format>])` генерирует строковое представление временной метки файла `<filename>` и сохраняет результат в переменной `<out>`. Факультативно может быть задана форматная строка `<format>`.
- `file(GET_RUNTIME_DEPENDENCIES [...])` получает зависимости времени выполнения для указанных файлов. Эту команду можно использовать только в сценариях `install(CODE)` или `install(SCRIPT)`. Доступна начиная с версии CMake 3.21.

Запись

Поддерживаются следующие режимы.

- `file({WRITE | APPEND} <filename> <content>...)` записывает или дописывает все аргументы `<content>` в файл `<filename>`. Если такого пути нет в файловой системе, то он будет рекурсивно создан.
- `file({TOUCH | TOUCH_NOCREATE} [<filename>...])` модифицирует временную метку файла `<filename>`. Если такого файла не существует, то он будет создан только в режиме `TOUCH`.
- `file(GENERATE OUTPUT <output-file> [...])` – специализированный режим, который генерирует выходной файл для каждой конфигурации сборки текущего генератора CMake.

- file(CONFIGURE OUTPUT <output-file> CONTENT <content> [...]) работает как GENERATE_OUTPUT, но также конфигурирует сгенерированные файлы, подставляя значения вместо маркеров.

Файловая система

Поддерживаются следующие режимы.

- file({GLOB | GLOB_RECURSE} <out> [...] [<globbing-expression>...]) генерирует список файлов, соответствующих маске <globbing-expression>, и сохраняет его в переменной <out>. В режиме GLOB_RECURSE просматриваются также вложенные подкаталоги.
- file(RENAME <oldname> <newname>) переименовывает файл из <oldname> в <newname>.
- file({REMOVE | REMOVE_RECURSE } [<files>...]) удаляет файлы <files>. В режиме REMOVE_RECURSE удаляются также подкаталоги.
- file(MAKE_DIRECTORY [<dir>...]) создает каталог.
- file(COPY <file>... DESTINATION <dir> [...]) копирует файлы в каталоге <dir>. Поддерживаются параметры для фильтрации, задания разрешений, следования по символическим ссылкам и т. д.
- file(COPY_FILE <file> <destination> [...]) копирует один файл в каталог <destination>. Доступен начиная с версии CMake 3.21.
- file(SIZE <filename> <out>) читает размер файла <filename> в байтах и сохраняет его в переменной <out>.
- file(READ_SYMLINK <linkname> <out>) читает путь, соответствующий символической ссылке <linkname>, и сохраняет его в переменной <out>.
- file(CREATE_LINK <original> <linkname> [...]) создает символическую ссылку <linkname> на файл <original>.
- file({CHMOD|CHMOD_RECURSE} <files>... <directories>... PERMISSIONS <permissions>... [...]) задает разрешения файлов и каталогов.
- file(GET_RUNTIME_DEPENDENCIES [...]) собирает зависимости времени выполнения для разных типов файлов: исполняемых, библиотек и модулей. Разрешается использовать только в сценарии install(RUNTIME_DEPENDENCY_SET).

Преобразование путей

Поддерживаются следующие режимы.

- file(REAL_PATH <path> <out> [BASE_DIRECTORY <dir>]) вычисляет абсолютный путь по относительному и сохраняет результат в переменной <out>. Факультативно принимает базовый каталог <dir>. Доступен начиная с версии CMake 3.19.
- file(RELATIVE_PATH <out> <directory> <file>) вычисляет путь к файлу <file> относительно каталога <directory> и сохраняет результат в переменной <out>.

- file({TO_CMAKE_PATH | TO_NATIVE_PATH} <path> <out>) преобразует путь <path> в формате CMake (каталоги разделены знаками прямой/косой черты) в платформенно-зависимый путь и обратно. Результат сохраняется в переменной <out>.

Передача

Поддерживаются следующие режимы.

- file(DOWNLOAD <url> [<path>] [...]) скачивает файл по адресу <url> и сохраняет его в файле с путем <path>.
- file(UPLOAD <file> <url> [...]) закачивает файл <file> по указанному URL.

Блокировка

В этом режиме ставится рекомендательная блокировка на ресурс <path>:

```
file(LOCK <path> [DIRECTORY] [RELEASE]
     [GUARD <FUNCTION|FILE|PROCESS>]
     [RESULT_VARIABLE <out>] [TIMEOUT <seconds>]
)
```

Факультативно может быть задана область действия блокировки: FUNCTION, FILE или PROCESS, а время ее существования ограничено <seconds> секундами. Для снятия блокировки служит ключевое слово RELEASE. Результат сохраняется в переменной <out>.

Архивация

Команда создания архивов имеет следующую сигнатуру:

```
file(ARCHIVE_CREATE OUTPUT <destination> PATHS <source>...
  [FORMAT <format>]
  [COMPRESSION <type> [COMPRESSION_LEVEL <level>]]
  [MTIME <mtime>] [VERBOSE]
)
```

В каталоге <destination> создается архив, содержащий файлы <source>, в одном из поддерживаемых форматов: 7zip, gnutar, pax, paxr, raw, zip (по умолчанию paxr). Если выбранный формат поддерживает уровень сжатия, то его можно задать в виде одной цифры 0-9, по умолчанию подразумевается 0.

Команда распаковки имеет следующую сигнатуру:

```
file(ARCHIVE_EXTRACT INPUT <archive> [DESTINATION <dir>]
  [PATTERNS <patterns>...] [LIST_ONLY] [VERBOSE]
)
```

Она извлекает файлы, соответствующие факультативным образцам `<patterns>`, из архива `<archive>` в каталог `<dir>`. Если задано ключевое слово `LIST_ONLY`, то файлы не извлекаются, а только перечисляются.

Команда math()

CMake также поддерживает простые арифметические операции. Полную информацию см. в онлайновой документации по адресу https://cmake.org/cmake/help/latest/command/math.html.

Для вычисления математического выражения и сохранения результата в переменной `<out>` в виде строки в факультативном формате `<format>` (`HEXADECIMAL` или `DECIMAL`) используется следующая команда:

```
math(EXPR <out> "<expression>" [OUTPUT_FORMAT <format>])
```

Значение `<expression>` – строка, поддерживающая операторы языка C (с той же семантикой):

- арифметические: `+`, `-`, `*`, `/` и `%` (остаток от деления);
- поразрядные: `|` (или), `&` (и), `^` (исключающее или), `~` (не), `<<` (сдвиг влево) и `>>` (сдвиг вправо);
- скобки `(...)`.

Значения констант можно задавать в десятичной или шестнадцатеричной форме.

Предметный указатель

A

ASCII (American Standard Code for Information Interchange), 180

C

Calc, исполняемый файл консольного калькулятора, 384
Calculations (библиотека), сборка, 141
Catch2, 286
CDash, 35
Clang/LLVM, 98
clang-tidy, 310, 312
CLion IDE, 99
  возможности, 100
  накачанный отладчик, 103
  скачивание, 101
CMake, 25
  графический интерфейс, 48
  командная строка, 35
    выполнение готовой конфигурации технологического процесса, 46
    выполнение командной программы, 45
    выполнение скрипта, 45
    генерирование сборочной системы проекта, 35
    сборка проекта, 41
    установка проекта, 44
  область видимости переменной, 72
  окружение проекта
    32- или 64-разрядная архитектура, 128
    кросс-компиляция, 126
    определение операционной системы, 125
    порядок байтов, 128
    сокращенные переменные, 126
  применение CTest для стандартизации тестирования, 269
  проект, 111
    вложенные проекты, 119
    внешние проекты, 119
    директивы и команды, 112
    задание минимальной версии, 113
    определение окружения, 125
    определение языков и метаданных, 113
    разбиение на части, 115
    структура, 120
    управление областью видимости с помощью каталогов, 117
  процедурная парадигма, 85
  сборка из исходного кода, 34
  синтаксис языка
    аргументы без кавычек, 66
    аргументы в кавычках, 65
    аргументы в квадратных скобках, 64
    аргументы команд, 63
    вызов команд, 62
    комментарии, 61
    основы, 60
  управляющие конструкции, 76
  установка в Docker, 31
  установка в Linux, 33
  установка в macOS, 34
  установка в Windows, 32
  установка на различных платформах, 30
  характеристики, 26
  этап генерирования, 27
  этап конфигурирования, 27
  этап сборки, 28
Coverage (модуль), подготовка, 390

CPack, упаковка с помощью, 368
Cppcheck, 310, 313
Cpplint, 310, 312
CTest
  командная строка, 47
  применение для стандартизации тестирования в CMake, 269
  режим сборки и тестирования, 270
  режим тестирования, 271
  создание автономного теста, 277
Cygwin, 98

D

Dev Containers, 106
Docker, 31
Doxygen
  добавление в проект, 323
  форматы документации, 323
Drawing (библиотека), 141

E

Executable and Linkable Format (ELF), 179, 207
execute_process(), команда, 91
ExternalProject, модуль, 253

F

FetchContent, 245
  использование установленной зависимости, 251
  обновление и исправление, 251
  пример программы чтения YAML, 246
  скачивание зависимостей, 248
file(), команда, 90
  архивация, 432
  блокировка, 432
  запись, 430
  передача, 432
  преобразование путей, 431
  режимы файловой системы, 431
  чтение, 430
find_package(), команда
  использование компонентов в, 365
  нахождение пакетов, 231
FindPkgConfig, нахождение унаследованных пакетов, 242
foreach(), цикл, 80

G

Git, игнорирование файлов, 55
GMock, 291
GNU Compiler Collection (GCC), 298
  URL-адрес, 98
GNU Coverage Data, 302
GNU Coverage Notes, 302
GNU Project Debugger (GDB), 103
GoogleTest (GTest), 285, 288
  GMock, 291
GuiApp, 141

I

include(), команда, 89
include_guard(), команда, 90
include-what-you-use, 310, 313
install(), команда, использование компонентов, 365

J

JSON (JavaScript Object Notation), 53

L

LCOV, применение для генерирования отчетов о покрытии, 298
Link What You Use, 310, 313
Linux Test Project, 298
list(), команда, 428
  модификация, 429
  поиск, 428
  упорядочение, 429
  чтение, 428

M

main(), вынесение для тестирования, 224
makefile, 119
math(), команда, 433
Member Function Documentation, раздел документации, 326
Memcheck, 315
  подготовка модуля, 391
Memcheck-Cover, 318
message(), команда, 87

Microsoft Visual Studio/MSVC, 98
MinGW-w64, 98

P

PkgConfig, 242

S

SEGFAULT, как избежать, 301
string(), команда, 424
  генерирование, 427
  модификация, 425
  поиск и замена, 425
  строки в формате JSON, 427
  хеширование, 426

T

TerminalApp, 141

U

Universal Windows Platform (UWP), 107

V

Valgrind, 314
Visual Studio Code (VS Code), 103
  возможности, 104
  контейнеры разработчика, 105
  URL-адрес, 105
Visual Studio (VS), 106
  возможности, 107
  отладка в режиме горячей перезагрузки, 108
  ссылка, 107

W

while(), цикл, 80

Y

YAGNI, принцип (тебе это не понадобится), 124
YAML, программа чтения, 53, 246

А

Автоматизированные тесты, 268
Автономного тестирования каркасы, 285
  Catch2, 286
  GoogleTest, 288
Автономный тест, создание для CTest, 277
Ад зависимостей, 213
Анализ программы, 388
Аргументы
  без кавычек, 66
  в кавычках, 65
  в квадратных скобках, 64
  команды, 62

Б

Блок реализации модуля, 258
Булевы
  запросы, 163
  логические операторы, 162
  сравнения, 162
Буфер протокола Google (Protobuf), 153

В

Вложенные генераторные выражения, 172
Внедрение зависимости, 291
Внутренняя сборка, 50
  запрет, 134
Время компиляции
  предкомпиляция заголовочных файлов, 195
  уменьшение, 195
  унифицированные сборки, 197
Вынесенная сборка, 50

Г

Генераторные выражения, 27, 150, 158
  вложенность, 160
  отладка, 170
  синтаксические правила , 159
  экранирование, 170
Генерирование сборочной системы проекта, 35
  выбор генератора, 37
  отладка и трассировка, 40
  очистка дерева сборки, 41
  примеры, 36
  создание готовых конфигураций, 41
  управление кешем проекта, 38
Глобальная область видимости, 72

Глобальная таблица смещений (GOT), 215
Готовые конфигурации
  зависящие от этапа
    добавление готовой конфигурации установки, 418
    определение, 409
    характеристики, 410
  использование в проекте, 407
  технологического процесса, определение, 419
  файлы, 54
    написание, 408
  этапа конфигурирования, определение, 411
  этапа сборки, определение, 413
  этапа тестирования, определение, 414
  этапа упаковки, определение, 417
Граф зависимостей, 140

Д

Двоичный артефакт, 159
Двоичный интерфейс модуля, 258
Действия, 35
Дерево
  исходного кода, 27
  сборки, 27
Дизассемблер, 203
Динамически компонуемая библиотека (DLL), 214
Документация современная, генерирование, 328

Ж

Журнал конфигурирования, 54

З

Зависимости
  времени выполнения, установка, 353
  отсутствующие в системе, 244
    ExternalProject, 253
    FetchContent, 244
  сборка, 380
  сборка библиотеки Calc, 382
  сборка исполняемого файла консольного калькулятора Calc, 384
  скачивание из CVS, 250
  скачивание из Git, 249
  скачивание из Mercurial, 250
  скачивание из Subversion, 250
  скачивание по URL-адресу, 248
  управление, 380
Заглушка, 291
Заголовок
  программы, 210
  ELF, 208
Заголовочные файлы, 139
Загрузчик, 178
Запросы, 163
Защита от повторного включения, 259, 278

И

Импортированные цели, 149
Инструмент сборки, 28
Инструменты статического анализа, добавление, 394
Интегрированные среды разработки (IDE), 23, 94
  выбор, 95
  использование примеров, 98
  установка комплекта инструментов, 97
Интерполяция переменных, 64
Интерфейсные библиотеки, 149
  с зависящими от компилятора флагами, 171
Информация о типе во время выполнения (RTTI), 172
Исполняемые цели, 138
Исходный код на C++, основные факты, 24

К

Каталоги включаемых файлов, 150
Кешированные переменные, 71
Команда, 87
  function(), 82
  macro(), 83
Командная строка
  CCMake, 49

CPack, 47
CTest, 47
Комментарии, 61
многострочные, 61
однострочные, 61
Компиляция, 177
конфигурирование ошибок и предупреждений, 199
начальная конфигурация, 180
отладка сборки, 200
поиск ошибок, 199
предоставление информации отладчику, 202
требование конкретных возможностей от компилятора, 181
управление исходными файлами для целей, 182
устройство, 178
Комплект
инструментов
конфигурирование, 264
настройка, 166
средств разработки ПО (SDK), 169
Компоненты
использование в команде install(), 365
использование в find_package(), 365
определение, 365
управление символическими ссылками для версионированных разделяемых библиотек, 367
Компоновка, 206
основы, 207
порядок, 220
Контейнеры, 31
Конфигурационные файлы, 339
Конфигурирование комплекта инструментов, 129
задание стандарта C++, 129
компиляция тестового файла, 132
межпроцедурная оптимизация, 131
принудительная поддержка стандарта, 130
проверка возможностей, поддерживаемых компилятором, 132
расширения производителя, 130
Корень проекта, 50
Кросс-компиляция, 96, 126
хост-система, 126
целевая система, 126

Л

Лексический анализ, 179
Лингвистический анализ, 179
Логические операторы, 162
Логические цели
использование конечного каталога по умолчанию для разных платформ, 342
публичные заголовки, 343
установка, 340
Локальная область видимости, 72

М

Макросы, добавление, 421
Маркеры порядка байтов, 60
Маска, 306
Межпроцедурная оптимизация, 131
Метавозможности, 132
Модули
поиска, 57, 231
возврат результатов, 241
написание, 236
определение импортированных целей, 237
повторное использование кешированных значений, 238
поиск библиотечных файлов, 239
поиск вложенных зависимостей, 239
поиск заголовочных файлов, 240
согласие с указанными пользователем путями, 238
C++
блок реализации модуля, 258
блок трудности с традиционными заголовками, 259
включение поддержки в CMake 3.28, 262
задание требований к компилятору, 262
использование в проектах, 260
объявление, 263

экспериментальная поддержка в CMake 3.26 и 3.27, 260
C++20, 257
двоичный интерфейс модуля (BMI), 258

Н

Не повторяйся (DRY), 117
Непрерывная интеграция и непрерывное развертывание (CI/CD), 270
Непрерывная интеграция (CI), конвейер, 315
Неразрешенные символы, 220
Нестабильные тесты, 275
Низкоуровневая установка, 345
вызов скриптов, 351
командами install(FILES) и install(PROGRAMS), 345
работа с целыми каталогами, 348
Нормальная переменная, 67

О

Область видимости переменной, 72
Обратный порядок байтов, 128
Объектная библиотека, 150, 378
Объектные файлы, 137, 179
Один поток команд, несколько потоков данных (SIMD), 193
Операционная система (ОС), 208
Опережающие объявления, 259
Определения компиляции, 145, 149
Оптимизатор, конфигурирование, 188
векторизация циклов, 193
встраивание функций, 191
общий уровень, 190
разворачивание циклов, 192
Ориентированный ациклический граф (ОАГ), 141
Отладка сборки, 200
отдельных этапов, 200
проблем из-за включения заголовочных файлов, 201
Отчеты о покрытии тестами
генерирование, 296
применение LCOV, 298

П

Пакеты, нахождение с помощью find_package(), 231
Параметризация конфигурирования и платформы сборки, 165
Переменные, 67
нормальные, 68
окружения, 69
ссылки на, 68
Перемещение, 209
Повторно используемые пакеты
генерирование файлов версий пакетов, 363
написание простых конфигурационных файлов, 357
проблемы перемещаемости целей, 354
продвинутые конфигурационные файлы, создание, 360
создание, 354
установка файлов экспорта целей, 356
Подделка, 291
Подставка, 291
Позиционно-независимый код (PIC), 206, 215
Порядок байтов, 128
Правило одного определения (ODR), 206, 282
решение проблем, 216
динамически скомпонованные символы-дубликаты, 218
пространства имен, 220
Препроцессирование, 179
Препроцессор
задание путей к включаемым файлам, 183
избегание доступа к закрытым полям класса в автономных тестах, 185
использование фиксаций git для отслеживания откомпилированной версии, 186
конфигурирование, 183
конфигурирование заголовочных файлов, 187
определение, 184

Программа на C++, шаги выполнения, 178
Пространства имен в C++, 220
Прямой порядок байтов, 128
Псевдонимы целей, 149
Псевдоцели, 148
  импортированные цели, 149
  интерфейсные библиотеки, 149
  псевдонимы целей, 149
Пути, 163

Р

Разворачивание циклов, 192
Разделение обязанностей, 115
Разделяемые библиотеки, 212
  сравнение со статическими, 378
Разделяемые модули, 214
Разделяемый объект, имя (SONAME), 213
Разработка через поведение, 285
Распространяемые свойства, 155
Расширенные регулярные выражения, 79
Регулярное выражение, 272
Режим
  информационной панели, 270
  тестирования, 270
    обработка ошибок, 274
    опрос тестов, 272
    параметры, 276
    повтор тестов, 274
    тасование тестов, 273
    управление выводом, 275
    фильтрация тестов, 272

С

Сборка проекта, 41
  выбор целей для сборки и очистки, 42
  запуск параллельной сборки, 42
  конфигурирование типа сборки для генераторов, поддерживающих несколько конфигураций, 43
  отладка, 43
  очистка дерева сборки, 43
Сборки и тестирования режим, 270
Сборочная система, 27
Сегмент, 210
Символы, на которые нет ссылок, 223
Системы управления версиями (СУВ), 36
Скриптовые команды, 63
Скрипты, 56
Служебные модули, 56
Соглашения об именовании, 87
Специальные команды
  использование в качестве генератора, 153
  использование в качестве точки вклинивания, 154
Специальный HTML-код, применение для обогащения вывода, 330
Списки
  использование, 74
  операции, 163
Списковые файлы, 51
  сгенерированные файлы, 53
  файл кеша, 52
  файл определения пакета, 53
  файл проекта, 51
Сравнения, 162
Ссылки на переменные, 64
Статические библиотеки, 212
  сравнение с разделяемыми, 378
Статические проверщики
  использование, 309
  clang-tidy, 312
  Cppcheck, 313
  Cpplint, 312
  include-what-you-use, 313
  Link What You Use, 313
Строки, 163
Строковая интерполяция, 64
Структура проекта, 377
  объектная библиотека, 378
  разделяемые и статические библиотеки, 378
  файлы, 379

Т

Тестируемая единица (UUT), 282
Тестируемая система (SUT), 269
Тестовые двойники, 291

Технический отчет 1 (TR1), 131

У

Удаленная разработка, 96
Унаследованные пакеты, нахождение с помощью FindPkgConfig, 242
Унифицированные сборки, 197
Управляющие конструкции CMake
  определения команд, 82
  условные блоки, 76
  циклы, 80
Управляющие последовательности, 64
Условия, добавление, 421
Условное выражение, отличие от вычисления оператора BOOL, 173
Условное расширение, 161
  вычисление булева значения, 162
Условные блоки, 76
Условные команды, синтаксис
  вычисление строки и переменной, 77
  исследование файловой системы, 79
  логические операторы, 76
  простые проверки, 79
  сравнение значений, 78
Установка проекта, 44
  выбор каталога, 44
  выбор устанавливаемых компонентов, 44
  задание прав доступа к файлам, 45
  отладка, 45
Установленные зависимости, использование, 230
Устройство управления памятью (MMU), 215

Ф

Файловый API, 54
Файлы и каталоги проекта, 49
  дерево исходного кода, 50
  дерево сборки, 50
  списковые файлы, 51
  файлы в формате JSON и YAML, 53
Файл экспорта целей, 336
  установка, 356
Форматирование принудительное, 305

Ц

Целеориентированная информация, запрос, 168
Цели сборки, 151
Цель, 42, 137
  библиотечная, 139
  визуализация зависимостей, 142
  граф зависимостей, 140
  граф исполняемый, 138
  задание свойств, 143
  конфликты распространяемых свойств, 147
  специальная, 139
  транзитивные требования использования, 144
Циклы
  foreach(), 80
  while(), 80

Ч

Чистый код, 61, 87

Э

Экстремальное программирование (XP), 124
Этапы компиляции
  лингвистический анализ, 179
  препроцессирование, 179
  этап ассемблирования, 180
  этап вывода кода, 180
  этап оптимизации, 180

Рафал Свидзиньски

CMake для C++

Главный редактор *Мовчан Д. А.*
Зам. главного редактора *Яценков В. С.*
editor@dmkpress.com
Перевод *Слинкин А. А.*
Корректор *Синяева Г. И.*
Верстка *Чаннова А. А.*
Дизайн обложки *Мовчан А. Г.*

Гарнитура PT Serif. Печать цифровая.
Усл. печ. л. 35,91. Тираж 200 экз.

Веб-сайт издательства: **www.dmkpress.com**